



CuentaVoz
Tu voz cuenta

Guía técnica

Cómo está construido CuentaVoz
por dentro: arquitectura, agente de
voz, modelo de datos, pruebas y
despliegue.

VERSIÓN **5.0**

AGOSTO DE 2026

DIRIGIDO AL **EQUIPO TÉCNICO**

QUE MANTIENE LA PLATAFORMA



PREPARADO POR STOCKXPRTS

Contenido

Las secciones 1 a 3 son el panorama y el arranque. De la 4 a la 11 va el sistema por dentro, pieza por pieza. La 12 recorre las catorce pantallas una por una. Las secciones 13 a 16 son la operación: pruebas, despliegue, decisiones y qué hacer cuando algo falla. La 17 es el código completo del proyecto, generado leyendo el repositorio.

1	Qué es CuentaVoz	PANORAMA	2
2	La arquitectura en una página	PANORAMA	3
3	Poner el proyecto a andar	ARRANQUE	4
4	Identidad y permisos	POR DENTRO	6
5	El agente de voz	POR DENTRO	11
6	El modelo de datos	POR DENTRO	13
7	La API	POR DENTRO	16
8	El frontend	POR DENTRO	17
9	Trabajar sin conexión	POR DENTRO	18
10	Reportes y archivos	POR DENTRO	19
11	Trazabilidad y auditoría	POR DENTRO	20
12	Las catorce vistas, una por una	POR DENTRO	21
13	Pruebas	OPERACIÓN	98
14	Despliegue	OPERACIÓN	99
15	Decisiones de diseño	OPERACIÓN	100
16	Cuando algo falla	OPERACIÓN	101
17	El código completo	APÉNDICE	102

1 Qué es CuentaVoz

El problema que resuelve, y por qué se resuelve hablando.

CuentaVoz reemplaza la planilla de papel del conteo de inventario en las bodegas de alimentos y bebidas. En vez de anotar en una hoja y después transcribir a un archivo, la persona **dicta lo que ve** — «arroz, veinte kilos» — y la plataforma lo entiende, lo confirma en voz alta y lo guarda con el código oficial del catálogo.

Eso quita los dos puntos donde se pierde la información: la letra en la planilla y la transcripción posterior. Y como quien cuenta tiene las manos ocupadas, hablar es la única interfaz que no lo obliga a soltar el producto.

LA REGLA QUE GOBIERNA TODO EL SISTEMA

CuentaVoz **siempre confirma antes de guardar, y nunca sobrescribe**. Cada dato que entra pasa por una confirmación explícita de la persona, y cuando se corrige una cantidad el valor anterior no se borra: queda en el registro de trazabilidad. Si alguna vez una función nueva rompe esa regla, es la función la que está mal.

Los dos perfiles

PERFIL	QUÉ HACE	QUÉ VE
Auxiliar de inventarios auxiliar	Cuenta las bodegas que le asignaron, pide insumos al almacén y legaliza el servicio del turno.	9 opciones de menú.
Administrador de bodega auditor	Todo lo anterior, más auditar, aprobar lo que se creó durante el conteo, sacar los reportes y administrar la configuración.	13 opciones de menú.

El perfil vive en `usuario.perfil` y vale `auxiliar` o `auditor`. En el código el administrador se llama siempre `auditor`; «Administrador de bodega» es solo la etiqueta que se le muestra a la gente.

2 La arquitectura en una página

Cuatro piezas y tres servicios externos. Nada más.

PIEZA	TECNOLOGÍA	DE QUÉ RESPONDE
Frontend frontend/	React 18 + Vite, PWA	Las 14 vistas, el reconocimiento de voz del navegador, la cola sin conexión y el diálogo con Cognito.
Backend backend/	FastAPI + Uvicorn	90 endpoints, las reglas del negocio, la conversación con el agente y la generación de archivos.
Base de datos	SQLAlchemy 2 sobre SQLite (local) o PostgreSQL (producción)	19 tablas. El mismo código sirve para las dos: solo cambia DB_URL .
Servicios backend/servicios/	Python puro	Intérprete local, recetas, conciliación, analítica, validación y archivos. Sin FastAPI adentro, por eso se pueden probar solos.

Los tres servicios externos

SERVICIO	PARA QUÉ	SI NO ESTÁ
AWS Cognito	Identidad: registro, ingreso, cambio de clave, verificación en dos pasos.	No se puede entrar. Es la única dependencia dura.
Google Gemini	Entender lo que la persona dice, en lenguaje libre.	Entra el intérprete local: entiende menos variantes, pero el flujo completo sigue funcionando.
Brevo (correo)	Avisar por correo de un mensaje al administrador.	El mensaje igual queda en la bandeja; se ofrece abrir el correo del propio dispositivo.

EL PATRÓN QUE SE REPITE: DEGRADARSE SIN ROMPERSE

Ninguna función se cae por completo cuando falla algo de afuera. Sin Gemini hay intérprete local; sin internet hay cola local que se sincroniza sola; sin servicio de correo el mensaje queda igual registrado. Es la diferencia entre una demo y algo que se puede usar en una bodega con señal irregular.

3 Poner el proyecto a andar

De un clon a la aplicación abierta en el navegador.

Qué necesita instalado

HERRAMIENTA	VERSIÓN	PARA QUÉ
Python	3.12 o superior	El backend.
Node.js	18 o superior	El frontend y las pruebas.
Git	cualquiera reciente	Traer el proyecto.

Backend

- 1 Cree el entorno virtual**
`cd backend` y luego `python -m venv .venv`.
- 2 Actívelo**
en Windows, `.venv\Scripts\activate`; en Linux o macOS, `source .venv/bin/activate`.
- 3 Instale las dependencias**
`pip install -r requirements.txt`.
- 4 Copie el archivo de configuración**
`.env.ejemplo` a `.env` y llene las llaves (ver el cuadro siguiente).
- 5 Arranque**
`uvicorn main:app --reload`. Queda en `http://127.0.0.1:8000`.

Frontend

- 1 Instale**
`cd frontend` y `npm install`.
- 2 Arranque**
`npm run dev`. Queda en `http://localhost:5173`.

El frontend apunta por defecto a `http://127.0.0.1:8000`. Para apuntarlo a otro backend, defina `VITE_API_URL` antes de arrancar.

Las variables de entorno

VARIABLE	OBLIGATORIA	QUÉ CONTROLA
COGNITO_REGION COGNITO_USER_POOL_ID COGNITO_APP_CLIENT_ID	Sí	Identifican el User Pool. No son secretos: viajan dentro del JavaScript que descarga cualquiera.
AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY	Sí	Credenciales de una cuenta IAM propia del backend, con permiso solo sobre ese User Pool. Nunca la cuenta que creó el pool.
DB_URL	No	Por defecto <code>sqlite:///cuentavoz.db</code> . En producción, la cadena de PostgreSQL.
GOOGLE_API_KEY	No	Activa Gemini. Sin ella entra el intérprete local.
UMBRAL_ANOMALIA	No	Qué tan grande debe ser una diferencia para disparar una alerta. Por defecto <code>0.5</code> (50 %).
ORIGEN_PERMITIDO	No	Orígenes extra para CORS, separados por coma. Los puertos 5173 y 4173 ya vienen permitidos.
SENTRY_DSN	No	Monitoreo de errores. Vacía, no se inicializa nada.

CUENTAS DE DEMOSTRACIÓN

Con la variable de siembra activa y la tabla de usuarios vacía, el backend crea `luis` , `diana` , `stephanie` y `valentina` con la clave `StockXperts1` . No la active en un despliegue con datos reales: `diana` tiene perfil de administrador.

4 Identidad y permisos

Quién es quién, y qué puede tocar cada quien.

La clave nunca pasa por el backend

El registro, el ingreso, el cambio de clave y la verificación en dos pasos los maneja **AWS Cognito directamente**: el frontend habla con el SDK de Cognito y la clave nunca llega a este servidor. El backend no puede filtrar una clave que nunca ve.

Lo único que hace **backend/seguridad.py** es comprobar que el *access token* que llega en la cabecera **Authorization** de verdad lo firmó nuestro User Pool: firma RS256 contra las llaves públicas que Cognito publica, emisor, audiencia, y que sea un access token y no un id token.

Los dos niveles de permiso

NIVEL	CÓMO SE APLICA	QUÉ PROTEGE
Por perfil	<code>Depends(requiere_perfil("auditor"))</code> en el endpoint.	Auditoría, Panel, Reportes y Ajustes.
Por bodega	La tabla <code>asignacion_bodega</code> .	Cada persona alcanza solo las zonas que le asignaron — auxiliar o administrador.

EL PERMISO POR BODEGA SE APLICA EN CADA ENDPOINT, NO SOLO EN EL MENÚ

Listar bodegas, abrirla, su detalle, sus artículos, sus firmas, las cuatro rutas de auditoría (iniciar, comparar, firmar, cerrar), reabrir, exportar el detalle y **la búsqueda por voz**. Ocultar la opción en el menú no es seguridad: si la restricción no está también en el endpoint, basta con pedir la URL a mano. Por eso el agente de voz también la respeta — es la puerta que más fácil se olvida.

Una sola función decide, `_requiere_acceso_bodega()`, para que agregar una ruta nueva sea agregar una línea y no volver a razonar la regla.

La puerta que no lleva un `bodega_id`

Revisar las rutas `/api/bodegas/{id}/...` no basta, y es el error fácil de cometer: a una bodega también se llega por el **id de sesión** y por el **respaldo de resiliencia** que el agente de voz acepta del cliente. Los dos los manda el navegador, y de los dos sale la bodega sobre la que se escribe.

Sin comprobarlos, la asignación no protegía nada por ese lado: bastaba con mandar el id de una bodega ajena en **bodega_id_respaldo** para quedar «dentro» de ella y contarle encima, sin haber pasado nunca por **/api/bodegas/abrir**, que era donde estaba el único chequeo. Lo mismo con el id de la sesión que otra persona tiene abierta.

_validar_sesion_de_voz() cierra las cuatro entradas: el turno del agente, el avance de una sesión, firmar por id de sesión y crear un producto a mitad del conteo. Un **sesion_id** negativo se deja pasar a propósito — es la conversación de Pedidos, que no tiene bodega física que proteger.

LO QUE ENSEÑÓ ESTE CASO

El respaldo de resiliencia se agregó para que un reinicio del backend no dejara al agente diciendo «no hay ninguna bodega activa» con la bodega visible en pantalla. Resolvió eso, y de paso abrió una puerta lateral: **todo dato que el cliente manda para «recordarle» algo al servidor es un dato que el cliente puede inventarse**. La comodidad y el permiso viajaban por el mismo campo.

Administrador general y administrador de sede

Manda la **asignación**, no el perfil. Un administrador **sin** bodegas asignadas es el administrador general: alcanza todo el parque, como siempre. Uno **con** bodegas asignadas es el administrador de esa sede y solo de esa — no puede iniciar, firmar ni cerrar la auditoría de otra, aunque su perfil sea el mismo.

Esto es lo que deja crecer a varias sedes sin inventar perfiles nuevos: el eje fino ya no es «auxiliar sí, administrador no», sino la lista de bodegas de cada quien. Y quien reparte bodegas reparte solo las suyas, porque si no el límite sería decorativo: bastaría con asignarse las 54 para volver a ser general. Lo que una persona tenga asignado en otra sede no se toca al guardar, para que dos administradores de sedes distintas puedan repartirle bodegas al mismo auxiliar sin pisarse.

La tabla **sede**: agrupa, no autoriza

Una sede es un sitio físico —Piscilago, la Calle 26— y agrupa bodegas: **bodega.sede_id**, nulo por defecto. Existe por una razón práctica, no de seguridad: quien lleva doce bodegas de una sede no debería marcarlas de a una al repartir, y ver la operación por sitio es lo que se pide en cuanto hay más de uno.

La sede no decide permisos. Meter dos bodegas en la misma sede no le da a nadie acceso a la que no tiene asignada: eso lo sigue resolviendo **asignacion_bodega**, bodega por bodega, con las pruebas de regresión que ya tenía. Son dos ejes a propósito, porque en la operación real una persona cubre tres bodegas de una sede y una de otra cuando le toca un turno, y un modelo que atara el permiso a la sede no sabría representar eso.

SIN SEDE ES UN ESTADO VÁLIDO, NO UN PENDIENTE

Las 54 bodegas que ya existían quedaron con **sede_id** nulo y se comportan exactamente igual que antes. Nada obliga a clasificarlas, y borrar una sede tampoco borra sus bodegas: vuelven a quedar sin sede. Una migración que hubiera repartido las 54 en sedes inventadas habría sido peor que no tener sedes.

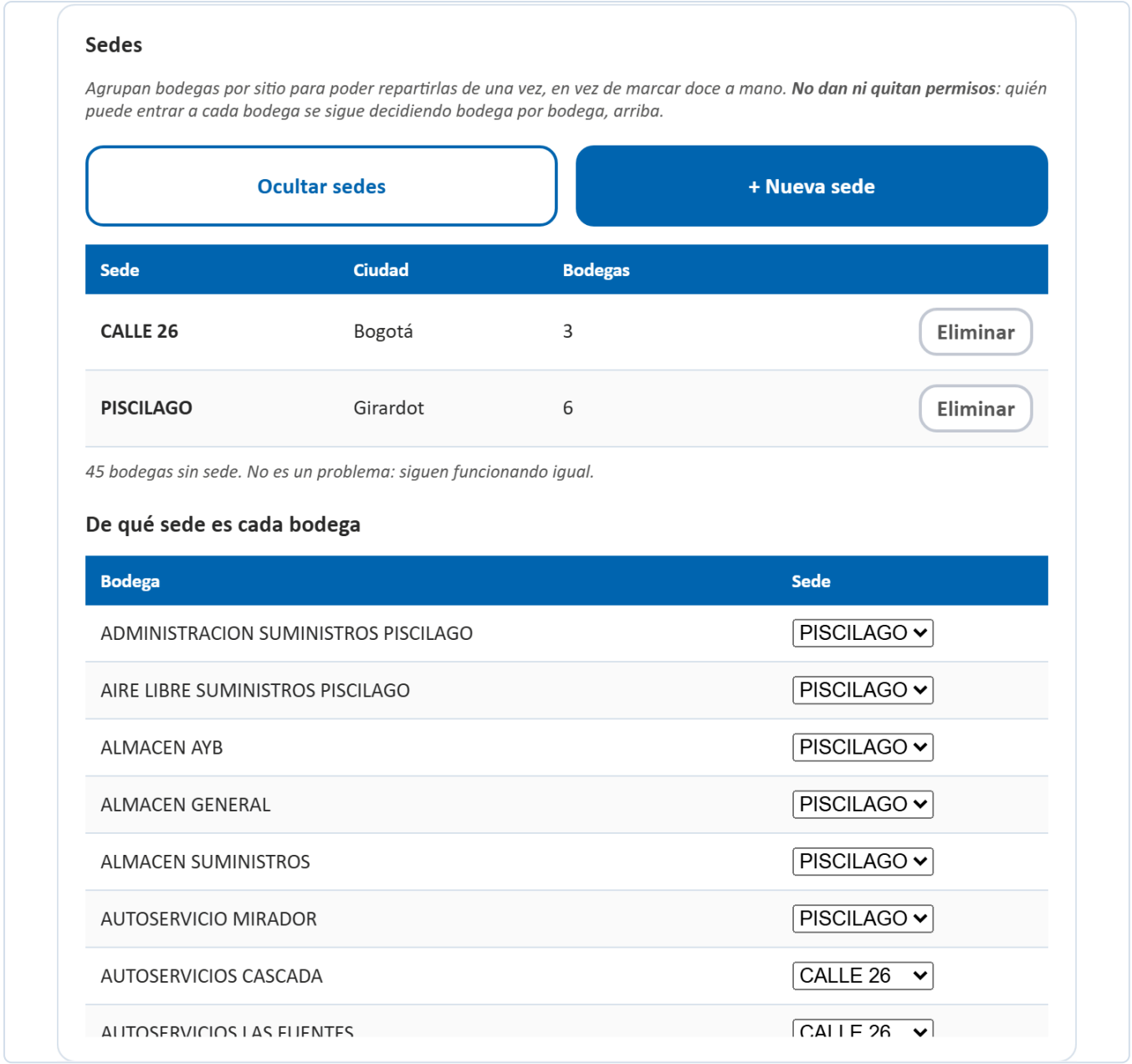


FIGURA La tarjeta de sedes en **Ajustes → Gestión de usuarios**, con la tabla que clasifica bodega por bodega.

Bodegas Para Luis

Marque las bodegas que va a poder contar esta persona.

Marcar una sede entera:

+ CALLE 26 (3)

+ PISCILAGO (6)

☐

ADMINISTRACION SUMINISTROS PISCILAGO

PISCILAGO

☐

AIRE LIBRE SUMINISTROS PISCILAGO

PISCILAGO

☒

ALMACEN AYB

PISCILAGO

☐

ALMACEN GENERAL

PISCILAGO

☒

ALMACEN SUMINISTROS

PISCILAGO

☐

AUTOSERVICIO MIRADOR

PISCILAGO

☐

AUTOSERVICIOS CASCADA

CALLE 26

☐

AUTOSERVICIOS LAS FUENTES

CALLE 26

3 bodegas marcadas

Cancelar

Guardar

FIGURA El motivo por el que existe: marcar una sede entera al repartir. Los ids los da `/api/sedes/{id}/bodegas`, que ya filtra lo que quien reparte no puede repartir.

Cerrar sesión de verdad

«Cerrar sesión» no se limita a borrar el token de este navegador: llama a `/api/usuarios/yo/cerrar-todas`, que revoca la sesión en Cognito. Sin eso, el access token seguía siendo válido hasta vencer solo — hasta una hora. En una tableta que se pasan entre turnos, eso es una cuenta abierta para el siguiente.

5 El agente de voz

De «arroz, veinte kilos» a una fila en la base de datos.

Dónde vive

ARCHIVO	DE QUÉ RESPONDE
backend/agente/orquestador.py 724 líneas	Dirige el turno de punta a punta: recibe la frase, decide qué hacer con ella, resuelve el artículo contra el catálogo, valida y contesta. Después de main.py , es el archivo más importante del proyecto.
backend/agente/cerebro.py 315 líneas	La conversación con Gemini: el <i>prompt</i> , las reglas del sistema y la lectura de la respuesta. Cuando no hay llave, aquí es donde entra el intérprete local en su lugar.
backend/servicios/interprete.py 147 líneas	El intérprete local. El mismo código, en JavaScript, corre además dentro del navegador para el modo sin conexión.

El recorrido de una frase

- 1

El navegador escucha

con la API **SpeechRecognition**, en español de Colombia. El audio no sale del dispositivo: lo que viaja es el texto.
- 2

El texto llega al backend

por **POST /api/agente/turno**, junto con la sesión de conteo y el contexto de lo que quedó pendiente de confirmar.
- 3

El agente interpreta

Gemini recibe la frase, el contexto y las reglas, y devuelve una intención con sus datos. Sin llave de Gemini, ese trabajo lo hace el intérprete local.
- 4

Se resuelve el artículo

el nombre dictado se empareja contra el catálogo con **rapidfuzz** y la tabla de alias. Si hay varias candidatas, se devuelven como opciones para que la persona elija.
- 5

Se valida

si la cantidad se sale del umbral, o la unidad no coincide, o el número es imposible, se genera una alerta

antes

de guardar.

6

Se confirma y se guarda

la plataforma repite lo que entendió y espera un «confirmo». Solo entonces se escribe la fila y se registra la traza.

Por qué hay dos intérpretes

`backend/servicios/interprete.py` es un intérprete escrito a mano: reconoce números en palabras («veinte» → 20), unidades («kilos» → `Kilogram`) y las intenciones más frecuentes. No reemplaza al modelo — entiende menos variantes de frase — pero garantiza que **la demo nunca se cae por falta de internet o de cuota**.

El mismo intérprete corre además **dentro del navegador** (`frontend/src/interpreteLocal.js`) para el modo sin conexión, donde no hay backend con quien hablar.

POR QUÉ EL TURNO DEL AGENTE CORRE EN UN HILO APARTE

Llamar a Gemini puede tardar varios segundos. Hacerlo directo dentro de la ruta `async` bloqueaba **todo** el bucle de eventos: cualquier otra petición, de cualquier otra persona, a cualquier endpoint — hasta `/api/salud` — se quedaba esperando. Por eso `procesar_turno` se saca con `run_in_threadpool`.

El asistente contextual

Aparte del conteo, `POST /api/agente/asistente` responde preguntas sobre la pantalla en la que está la persona y sabe navegar: «lléveme al panel» cambia de vista. Las claves de destino tienen que calzar exactamente con las que usa `ir(destino)` en `App.jsx`; si no calzan, la navegación por voz apunta a una vista que no existe.

6 El modelo de datos

Dieciocho tablas, en `backend/modelos.py`.

TABLA	GUARDA
usuario	Las personas, su perfil y su estado. La clave no está aquí: la guarda Cognito.
asignacion_bodega	Qué bodegas alcanza cada persona. Es la que decide los permisos.
sede	Agrupar bodegas por sitio, para repartirlas de una vez. No decide permisos.
bodega	Las zonas y su estado: pendiente, en conteo, en auditoría o cerrada.
articulo	El catálogo oficial, con su código.
stock_sistema	Lo que el sistema cree que hay, contra lo que se compara el conteo.
alias_articulo	Cómo llama la gente a las cosas, contra el nombre oficial. Es lo que hace que «papa criolla» encuentre PAPA CRIOLLA PRECOCIDA .
sesion_conteo	Una toma de una bodega por una persona. Es el candado que evita conteos duplicados.
conteo	Cada renglón contado. Las correcciones no borran: apuntan al anterior con corrige_a .
alerta	Las diferencias que quedaron marcadas durante el conteo.
traza	Quién hizo qué y cuándo. Nunca se borra.
receta · receta_ingrediente	Los platos con sus ingredientes y su rendimiento. Es la base del cálculo de pedidos.
linea_servicio	Lo pedido contra lo usado en un servicio. Es lo que compara la legalización.
aprobacion	Productos y bodegas creados en medio del conteo, esperando el visto bueno.
historial_cierre	El cierre de cada bodega, con su doble firma.
config_clave	Configuración del sistema por clave y valor: umbral de anomalía, modo sin conexión.
archivo_generado	Los reportes que se han sacado, para poder previsualizarlos y volver a descargarlos.
mensaje_soporte	La bandeja interna entre auxiliares y administrador.

Cómo se conectan

Tres cadenas explican casi todo el sistema:

CADENA	RECORRIDO
El conteo	bodega → sesion_conteo → conteo → alerta → historial_cierre . Una bodega se abre en una sesión, la sesión acumula renglones, los renglones que se salen del umbral generan alertas, y el cierre con doble firma congela el resultado.
El pedido	receta → receta_ingrediente → stock_sistema → linea_servicio . La receta dice cuánto se necesita por porción, el stock dice cuánto hay, y la diferencia es lo que hay que pedir. Al cerrar el turno, esa misma línea guarda lo que de verdad se usó.
El nombre	lo dictado → alias_articulo → articulo . Es la cadena que convierte «papa criolla» en un código oficial. Cuando no encuentra nada, nace una fila en aprobacion en vez de detener el conteo.

LA DOBLE FIRMA SON DOS FIRMAS, Y VENÍAN DE SITIOS DISTINTOS

POST /api/bodegas/{id}/cerrar exige que la sesión de conteo y la de auditoría tengan **firmada = 1**. La segunda la ponía Auditoría; la primera no la ponía ninguna pantalla — **/api/sesiones/{id}/firmar** existía desde el commit inicial y el frontend nunca lo llamó. Resultado: ninguna bodega contada llegaba a poder cerrarse, y el botón «Cerrar bodega definitivamente» quedaba apagado para siempre.

Pasó desapercibido porque la base de la demo traía sesiones ya firmadas de pruebas anteriores. Leer el backend no lo revela: ahí estaba todo. Lo que lo revela es **recorrer el flujo completo** — abrir, contar, auditar, cerrar — y ver el 409. Vale como recordatorio: un endpoint que existe y pasa sus pruebas no es un flujo que funciona, si nada lo llama.

NADA SE BORRA

Corregir un conteo escribe una fila nueva que apunta a la anterior.

Desactivar a una persona pone **activo = 0**, no borra la fila. Rechazar una aprobación la deja marcada como rechazada. Un inventario que pierde su historia no sirve para auditar.

7 La API

Noventa endpoints en **backend/main.py**, agrupados por familia.

FAMILIA	CUÁNTOS	PARA QUÉ
/api/usuarios	16	El perfil propio, la administración de cuentas y la asignación de bodegas.
/api/bodegas	15	Listar, abrir, cerrar, detalle y exportaciones.
/api/reportes	6	Generar, previsualizar y descargar.
/api/pedidos	6	Calcular desde la receta, enviar y aprobar.
/api/soporte	5	Reportar un problema, escribirle al administrador y la bandeja.
/api/recetas	5	Mantener los platos y sus ingredientes.
/api/articulos	4	Buscar en el catálogo y consultar existencias.
/api/legalizacion	3	Comparar, ajustar y confirmar el cierre del servicio.
/api/aprobaciones · /api/alertas · /api/trazabilidad	9	El control del administrador.
/api/agente · /api/voz	4	El turno de conversación, el asistente contextual y las voces.
/api/panel · /api/analisis · /api/ajustes	6	Indicadores, análisis de consumo y configuración.
/api/salud	1	Estado del sistema. Es el que consulta la pantalla de Ayuda.

Todos exigen un access token válido salvo **/api/salud** y **/api/registro-completado**. Los que solo puede usar el administrador llevan **Depends(requiere_perfil("auditor"))**.

LOS ERRORES LLEGAN CON SUS CABECERAS CORS

Un error que sale sin las cabeceras de CORS llega al navegador como «error de red», y la pantalla dice «Sin conexión con el servidor. Revise el Wi-Fi» — mandando a buscar el problema en el router mientras el servidor respondía perfecto. Por eso los manejadores de excepción las ponen a mano: para que el error se vea tal cual es.

8 El frontend

Catorce vistas, una barra lateral y un puñado de módulos compartidos.

Cómo está organizado

ARCHIVO	DE QUÉ RESPONDE
App.jsx	El menú (MENU), qué vista está activa, la sesión y el contexto que se pasa a las vistas.
vistas/	Las 14 pantallas, una por archivo.
api.js	Todas las llamadas al backend, y esFalloRed , que distingue un fallo de red de un error del servidor.
cognito.js	Registro, ingreso, clave y verificación en dos pasos.
voz.js	Escuchar y hablar.
interpreteLocal.js	El intérprete del modo sin conexión.
colaOffline.js	La cola de conteos pendientes de sincronizar.
accesibilidad.js	Alto contraste y tamaño de letra.
tutorial.js · VideoRecorrido.jsx	El recorrido en video que se abre al ingresar.

El menú es la única fuente de verdad de la navegación

La lista **MENU** de **App.jsx** define el orden, el título, la vista y si la opción es solo del administrador (**soloAuditor: true**). Agregar una pantalla es agregar una entrada ahí y su componente en **vistas/**.

QUIEN DESPLAZA ES .CONTENIDO, NO LA VENTANA

El contenedor raíz ocupa **100dvh** y el que tiene la barra de desplazamiento es **.contenido**. Un **window.scrollTo** no mueve nada. Vale la pena saberlo antes de perder una tarde.

9 Trabajar sin conexión

Lo que pasa cuando se cae el internet en plena bodega.

El conteo no se detiene. El navegador dispara el evento `offline`, la pantalla de Conteo cambia sola a un formulario de respaldo, y lo que se registre se guarda en `localStorage` bajo la clave de la **sesión de conteo activa**. Un aviso muestra cuántos registros quedan pendientes, tanto en la pantalla como en la barra lateral.

Al volver la señal, el evento `online` dispara la sincronización: cada renglón se reenvía y se confirma, y **solo se saca de la cola lo que de verdad se confirmó**. Si uno falla, o vuelve a caerse la red a mitad, el resto queda guardado para el próximo intento — antes, cualquier falla borraba toda la cola sin distinguir qué alcanzó a guardarse.

LA COLA VA POR SESIÓN DE CONTEO, NO POR USUARIO

La clave es `cv_offline_<sesionId>`. Sin una bodega abierta no hay sesión, y por lo tanto no hay dónde encolar: el modo sin conexión solo tiene sentido dentro de un conteo en curso.

10 Reportes y archivos

Seis salidas, repartidas en las pantallas a las que pertenecen.

ARCHIVO	SE GENERA DESDE
Consolidado para My Inventory	Reportes › Consolidado de la toma
Diferencias por bodega	Reportes › Consolidado de la toma
Análisis de consumo	Reportes › Análisis de consumo
Estado del tablero	Bodegas
Detalle de una bodega	Bodegas › detalle
Registro de trazabilidad	Ajustes › Registro de trazabilidad

Todo lo generado queda en **archivo_generado**, así que se puede **previsualizar en pantalla antes de descargar** y volver a bajar después. Los archivos salen en XLSX con **openpyxl**, con los códigos oficiales del catálogo — que es lo que My Inventory espera recibir.

11 Trazabilidad y auditoría

Cómo se responde a «¿quién cambió esto?».

Cada acción con consecuencia llama a **registrar**(usuario, accion, detalle, nivel), que escribe en **traza**. Ahí quedan los conteos, las correcciones, las aprobaciones, los cierres, los reportes de soporte y los mensajes. El administrador lo consulta y lo exporta desde **Ajustes › Registro de trazabilidad**.

Las cuatro pestañas de Auditoría

PESTAÑA	QUÉ RESUELVE
Recuento ciego y cierre	Volver a contar una bodega sin ver los números del auxiliar. La comparación solo se revela al terminar, y el cierre lleva doble firma.
Aprobaciones	Los productos y bodegas que alguien creó en medio del conteo para no detenerse. Al aprobarlos entran al catálogo con su código definitivo.
Pedidos pendientes	Autorizar lo solicitado antes de que salga del almacén.
Bandeja de alertas	Las diferencias marcadas durante el conteo, agrupadas por tipo.

POR QUÉ EL RECUESTO ES CIEGO

Si quien audita ve el número del primer conteo, lo confirma. El sesgo de anclaje convierte la auditoría en un trámite. Ocultar el dato hasta el final es lo que hace que la segunda pasada valga algo.

12 Las catorce vistas, una por una

Qué hay en cada pantalla, qué problema resuelve y con qué endpoints habla. En el orden del menú.

Cada vista es un archivo en `frontend/src/vistas/`. Reciben por props el `token`, el `usuario`, el contexto de la sesión y la función `ir()` para navegar. Ninguna guarda estado global: lo que tiene que sobrevivir a un cambio de pantalla vive en `App.jsx`.

12.1 Ingreso SIN SESIÓN

ARCHIVO	Ingreso.jsx · 657 líneas
QUÉ TIENE	Siete modos en una sola pantalla: <code>login</code> , <code>login-mfa</code> , <code>registro</code> , <code>registro-codigo</code> , <code>registro-mfa</code> , <code>recuperar</code> y <code>recuperar-codigo</code> . El perfil se detecta mientras se escribe el usuario.
QUÉ RESUELVE	Todo el ciclo de identidad sin salir de una pantalla ni depender de soporte: crear la cuenta, confirmar el correo, activar el segundo factor y recuperar la clave.
CON QUÉ HABLA	Con Cognito directamente (vía <code>cognito.js</code>), no con el backend. La clave nunca llega al servidor. Al terminar llama a <code>/api/registro-completado</code> para crear la fila local.

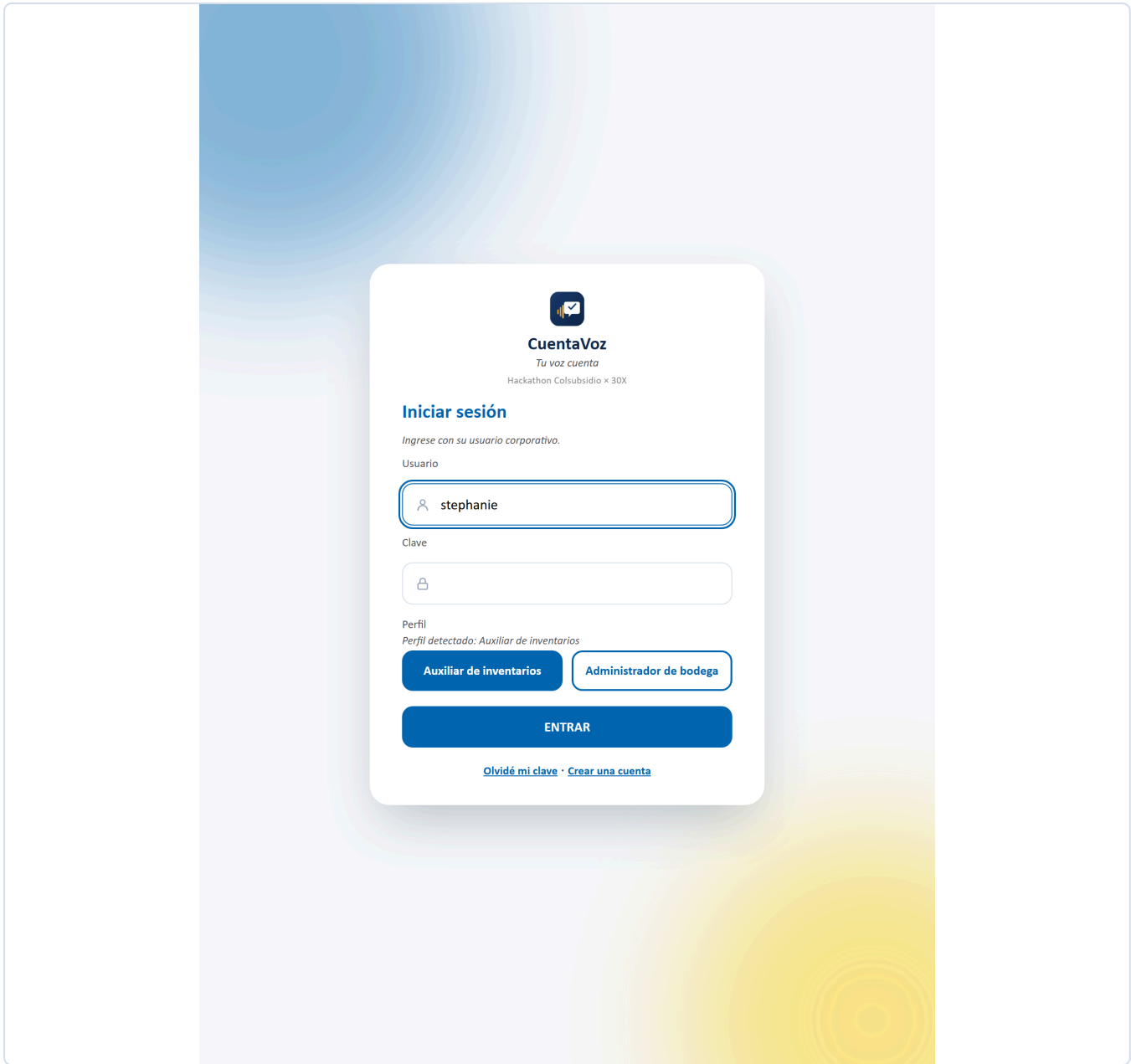


FIGURA 1 Modo **login** . El perfil se detecta mientras se escribe el usuario.



CuentaVoz
Tu voz cuenta
Hackathon Colsubsidio x 30X

Crear una cuenta

La cuenta queda como auxiliar de inventarios.

Nombre completo

María Fernanda Ríos

Código de empleado (opcional)

Correo corporativo

mfrios@colsubsidio.com

Usuario

mfrios

Clave

.....

Confirmar clave

✓ Al menos 8 caracteres

✓ Una letra mayúscula

✓ Una letra minúscula

✓ Un número

CREAR CUENTA

[Ya tengo una cuenta](#)

FIGURA 2 Modo **registro**. Los cuatro requisitos de clave se marcan en verde a medida que se escribe.

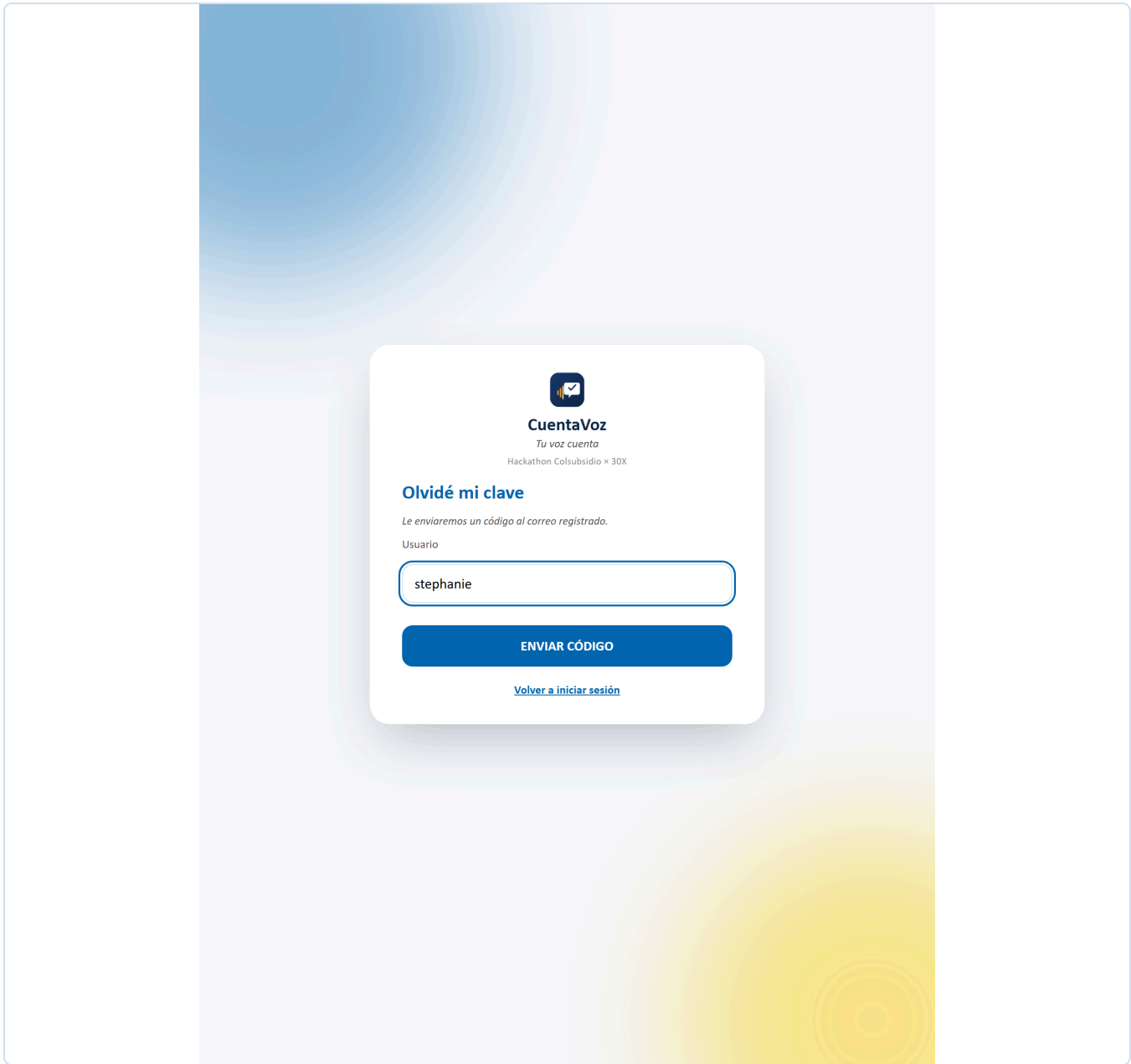


FIGURA 3 Modo **recuperar** . Se pide el usuario, no el correo: el código va a la dirección ya registrada.

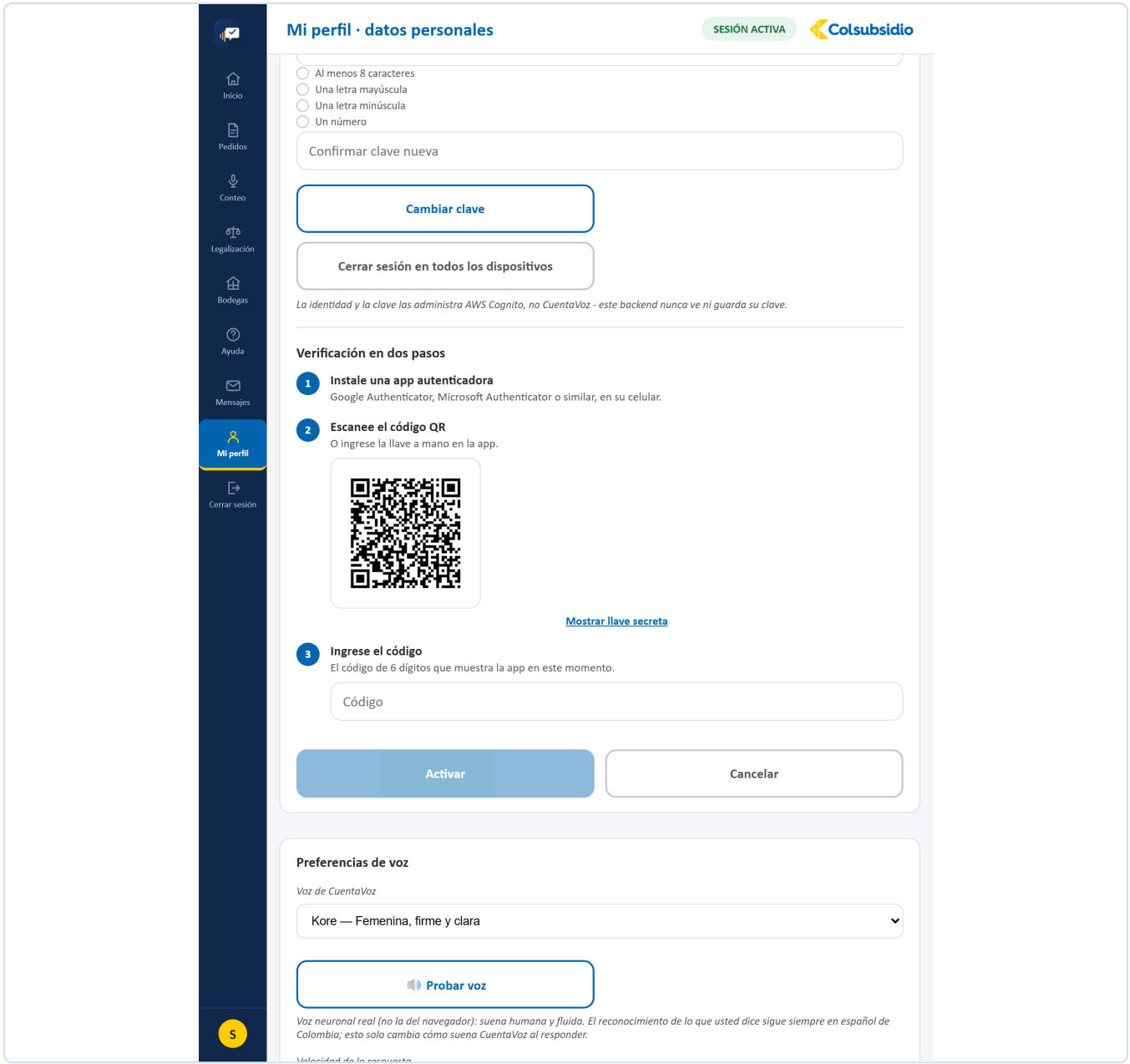


FIGURA 4 Modo `registro-mfa`. El mismo componente que abre Mi perfil: QR, llave secreta y código de seis dígitos.

12.2 Inicio MENÚ 1

ARCHIVO	<code>Inicio.jsx</code> · 171 líneas
QUÉ TIENE	El saludo con la fecha, cuatro indicadores del día (bodegas asignadas, referencias contadas, alertas por revisar, exactitud del mes), cuatro accesos directos y la actividad reciente del equipo.
QUÉ RESUELVE	Que la persona sepa en diez segundos qué le toca hoy, sin recorrer el menú.
CON QUÉ HABLA	<code>/api/usuarios/yo/resumen</code> y <code>/api/trazabilidad</code> para la actividad.

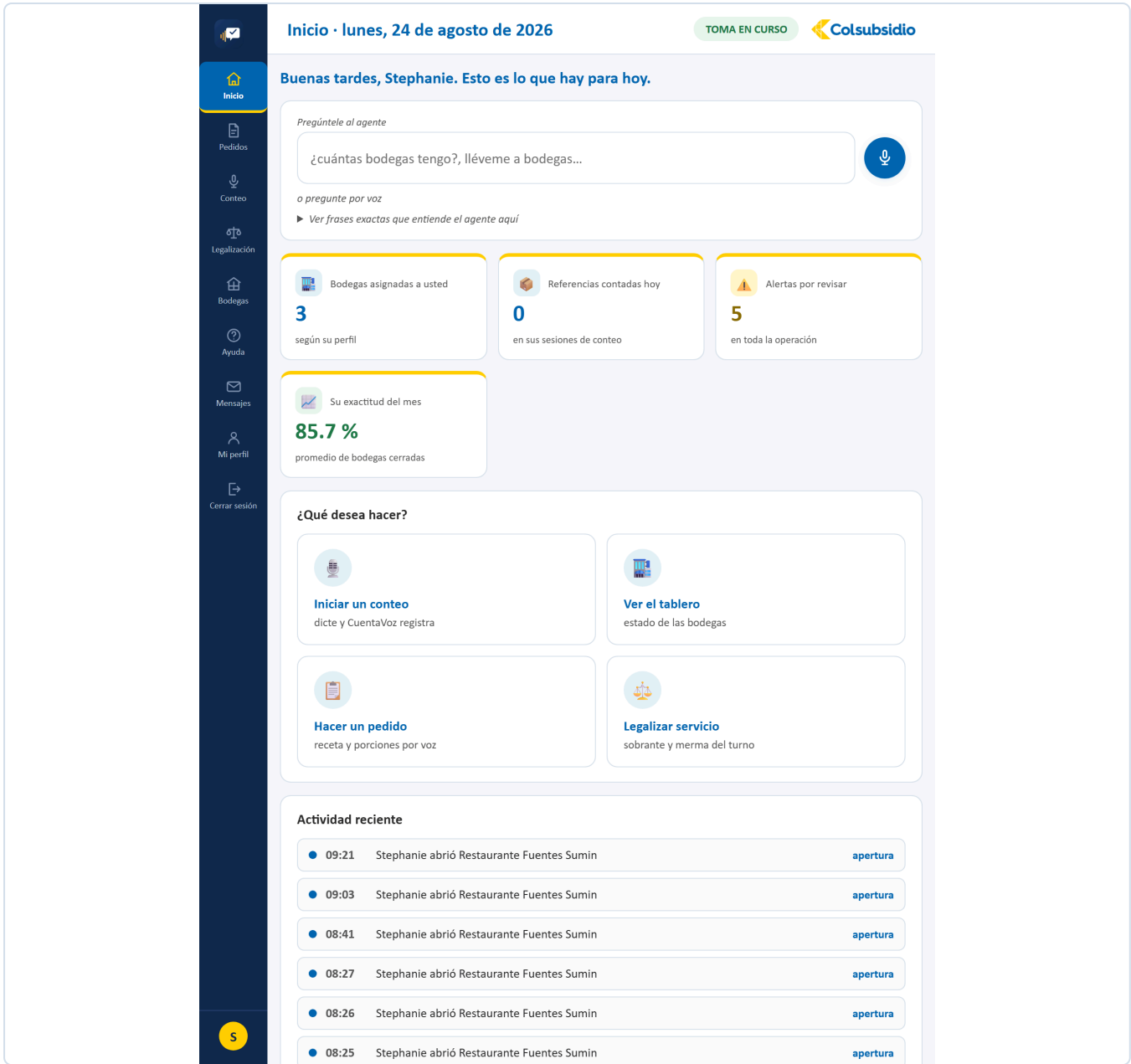


FIGURA 5 Inicio del auxiliar: sus bodegas, lo que lleva contado y su exactitud.

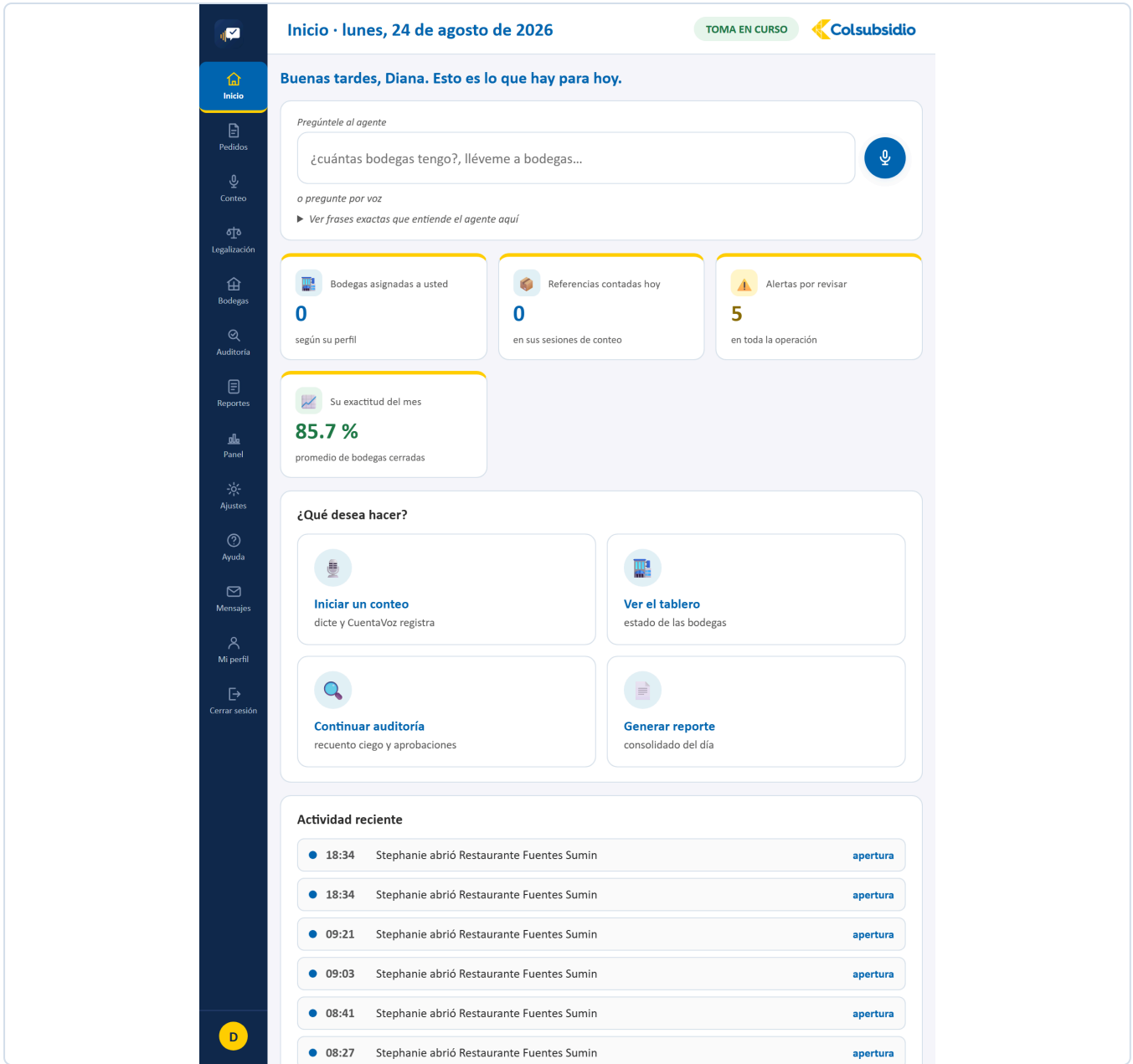


FIGURA 6 El mismo Inicio con perfil de administrador: los indicadores son de la operación entera.

12.3 Pedidos

MENÚ 2

ARCHIVO	Pedido.jsx · 849 líneas
QUÉ TIENE	El micrófono para dictar el plato y las porciones, la burbuja con lo que entendió el agente, la tabla de insumos calculados (necesario, hay en bodega, falta pedir) y el botón de enviar al almacén.
QUÉ RESUELVE	El caso del reto: «hoy preparamos cincuenta ajiacos». La persona no dicta ingredientes — dicta el plato, y la receta hace el resto.
CON QUÉ HABLA	/api/agente/turno para entender la frase, /api/pedidos/calcular para la tabla y /api/pedidos para enviarlo.

La cuenta que hace el backend: **cantidad por porción × porciones – lo que hay en esa bodega**. Si el resultado es cero o negativo, el insumo no entra al pedido: no se pide lo que ya está.

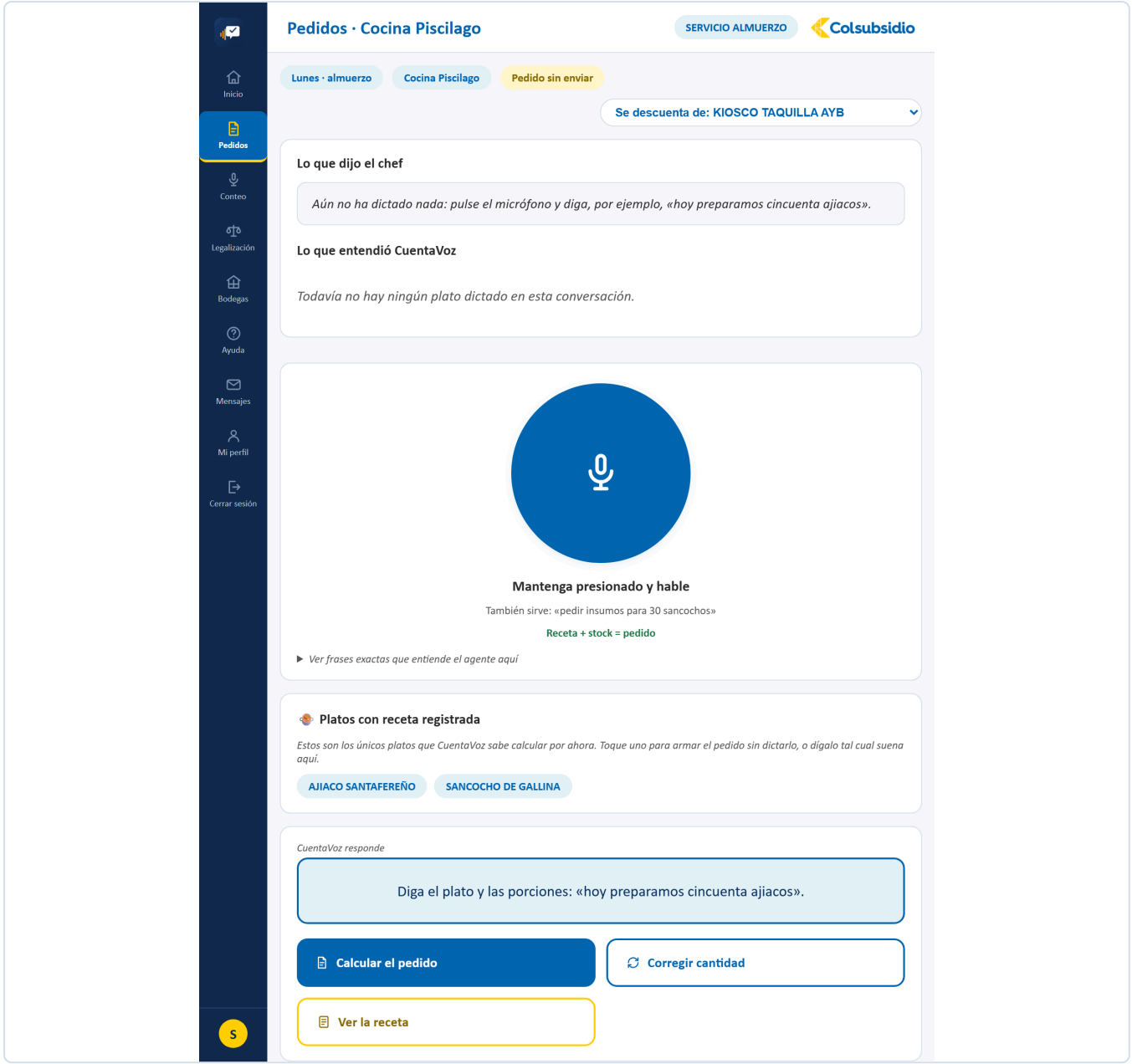


FIGURA 7 Pedidos. Se dicta el plato y las porciones.

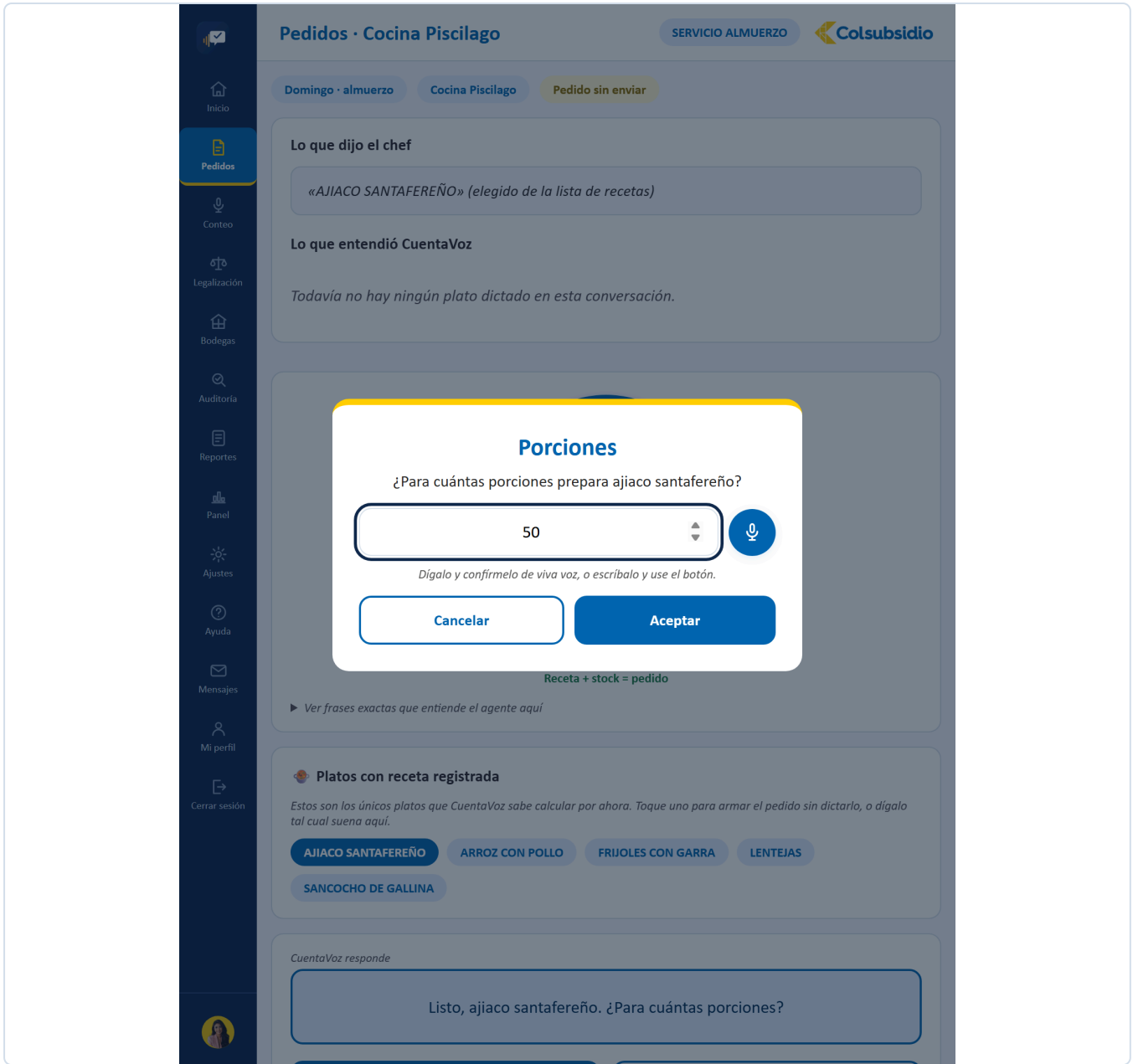


FIGURA 8 Antes de calcular, la plataforma confirma cuántas porciones entendió.

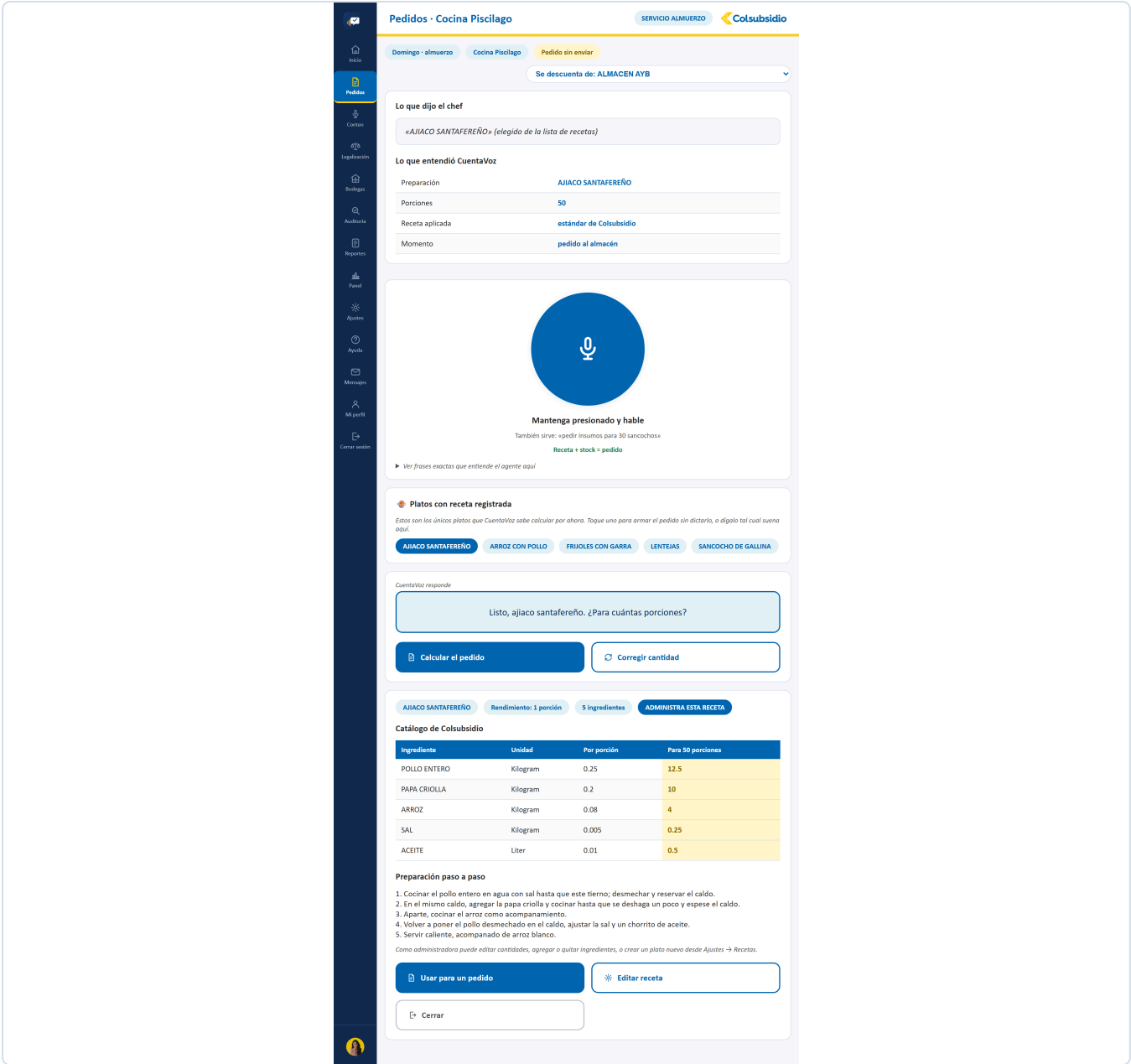


FIGURA 9 «Ver la receta»: el catálogo de Colsubsidio con la preparación paso a paso.

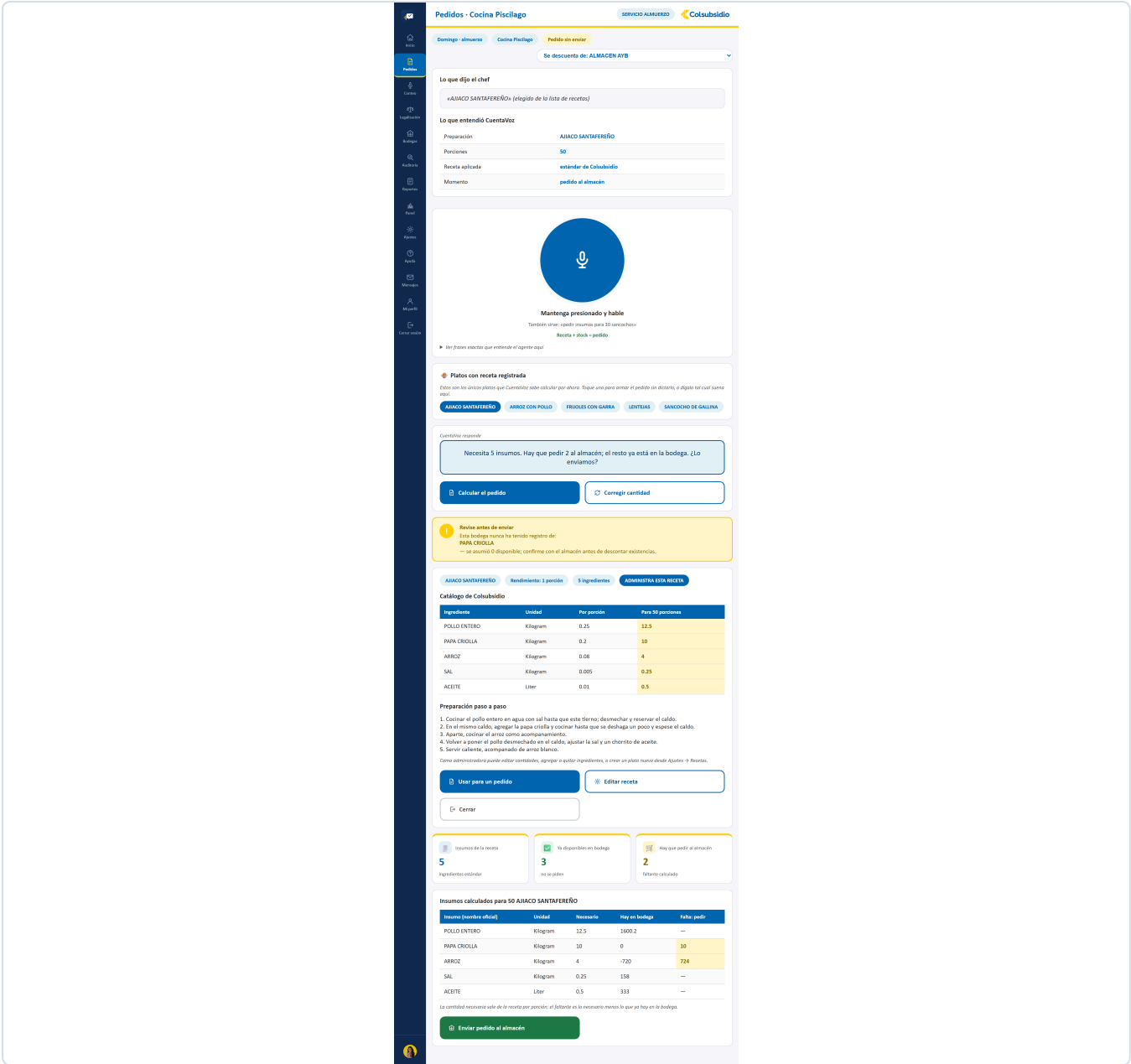


FIGURA 10 La tabla de insumos: cuánto se necesita, cuánto hay y cuánto falta pedir.

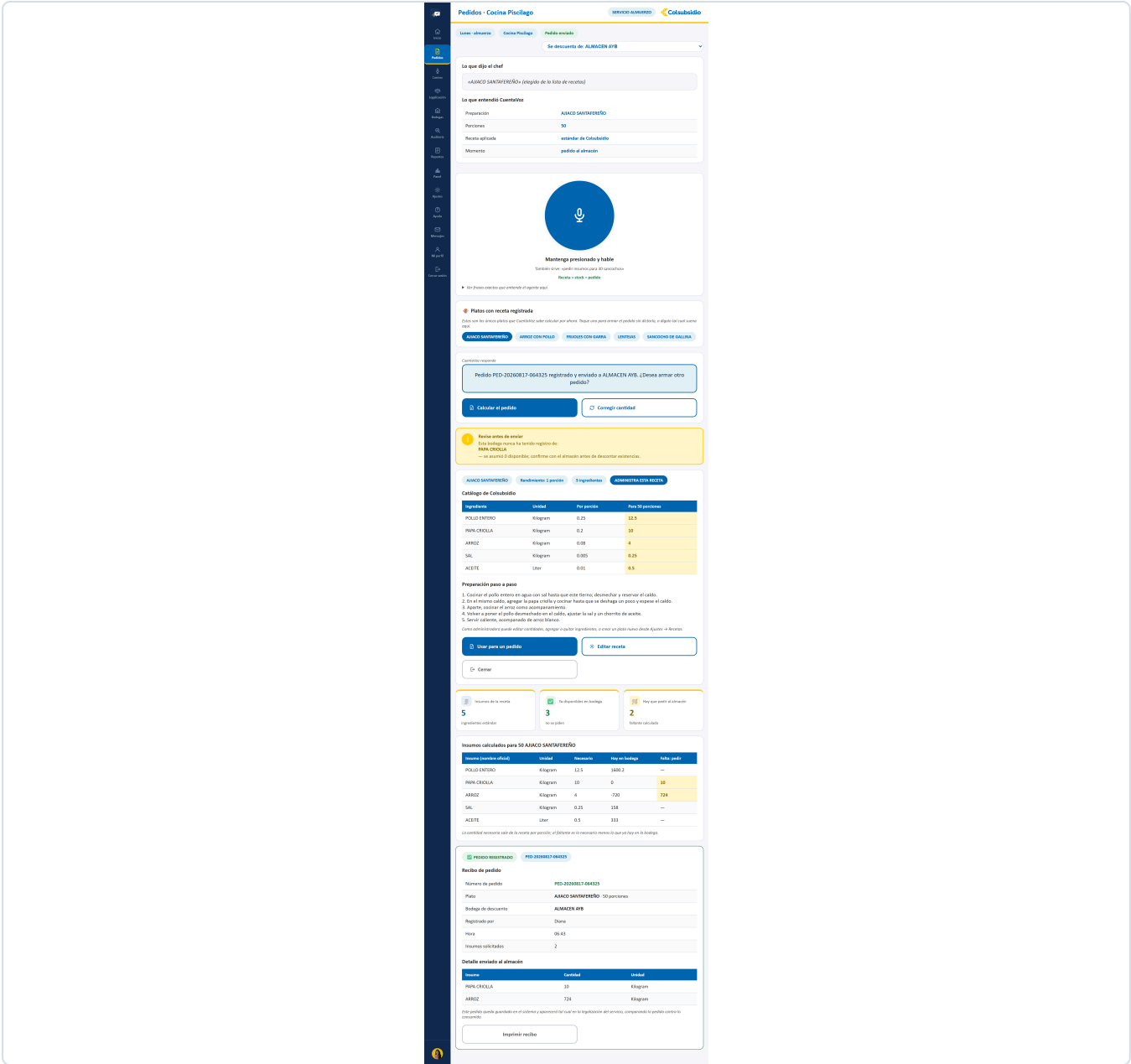


FIGURA 11 El recibo, con número real de pedido.

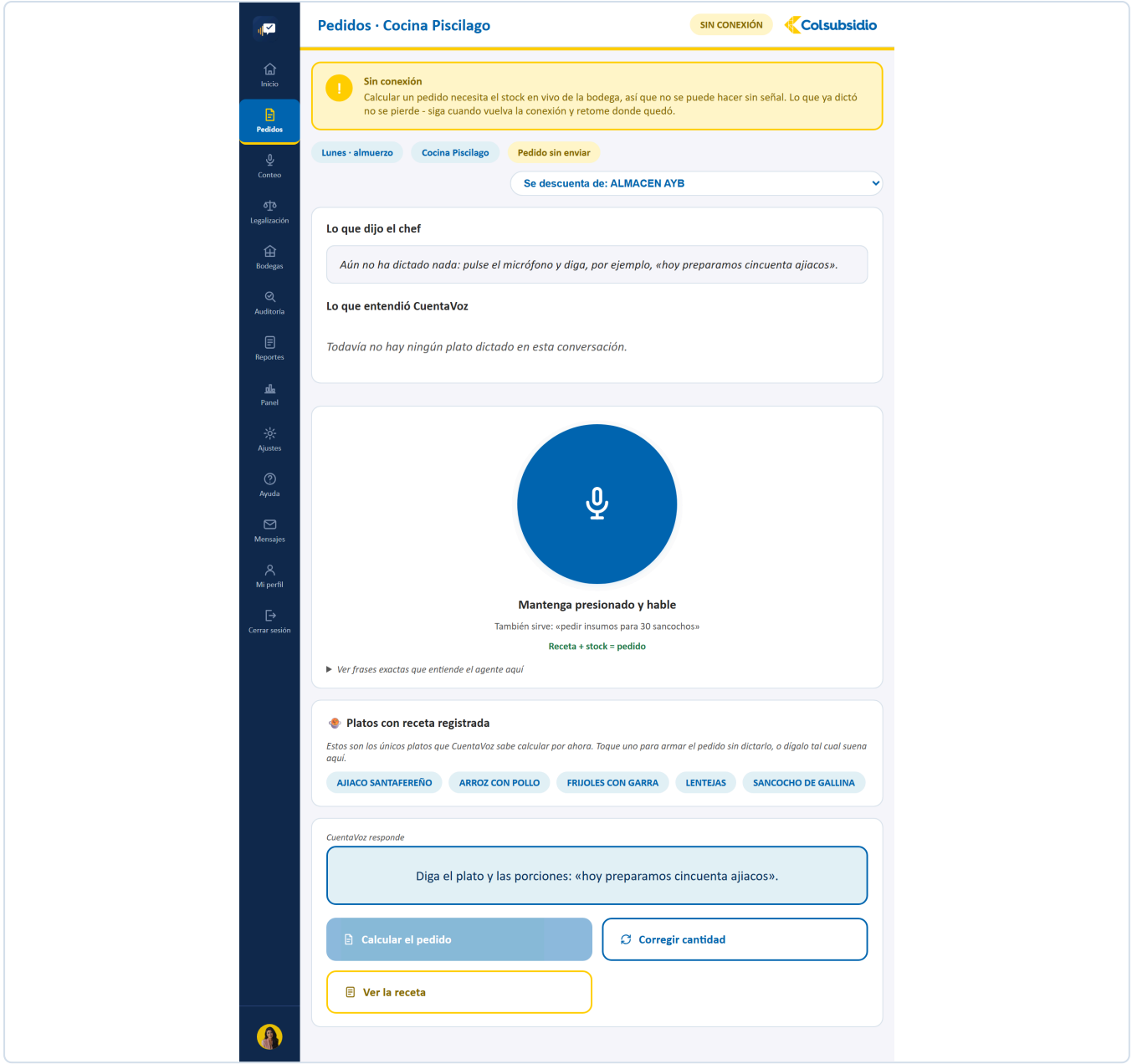


FIGURA 12 Sin red no se calcula: pedir con existencias desactualizadas es peor que no pedir.

12.4 Conteo

MENÚ 3

ARCHIVO	Conteo.jsx · 1.174 líneas — la vista más grande del proyecto
QUÉ TIENE	El selector de bodega (tarjetas o nombre exacto), el avance y las alertas arriba, la burbuja del agente, los cuatro botones (Confirmar , Corregir , Teclado , Pedir ayuda), la desambiguación por tarjetas, el aviso de anomalía, el alta de producto nuevo y el formulario de respaldo sin conexión.
QUÉ RESUELVE	Es el corazón del sistema: convertir lo que alguien dice frente a un estante en una fila con código oficial, sin que nada se guarde sin confirmación.
CON QUÉ HABLA	/api/bodegas/abrir, /api/agente/turno, /api/sesiones/{id}/avance y /api/conteo/crear-producto.

LOS CUATRO CAMINOS DE UN TURNO DE CONTEO

El agente puede responder cuatro cosas distintas a una frase: **confirmar** (entendió, pregunta si guarda), **desambiguar** (varias candidatas en el catálogo, muestra tarjetas), **alertar** (la cantidad se sale del umbral, pide revisar) o **proponer crear** (no existe en el catálogo). Los cuatro terminan en una pregunta, nunca en una escritura silenciosa.

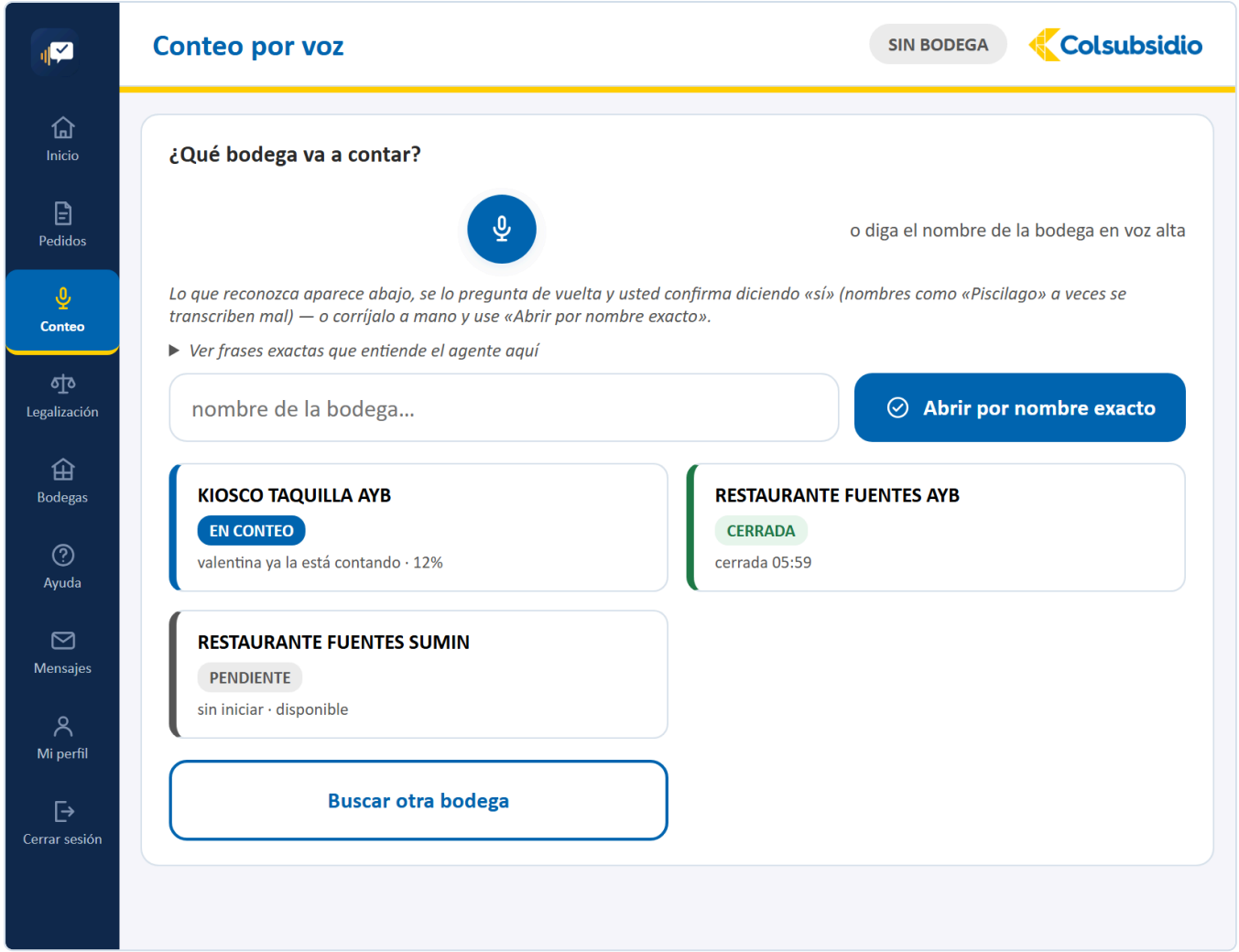


FIGURA 13 Conteo. El selector, con el estado y el avance de cada zona.

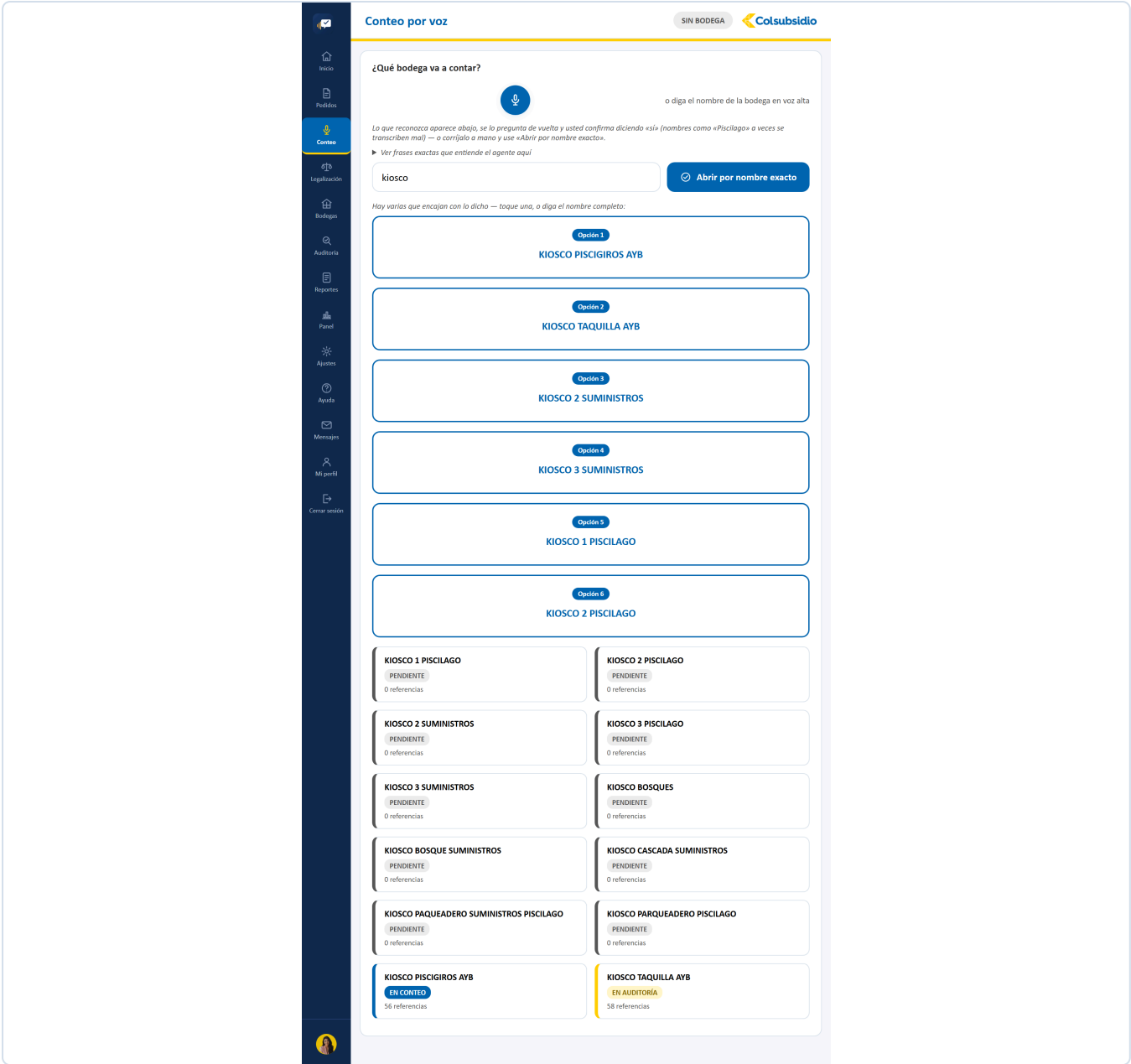


FIGURA 14 Un nombre ambiguo («kiosco») devuelve las candidatas en vez de adivinar.

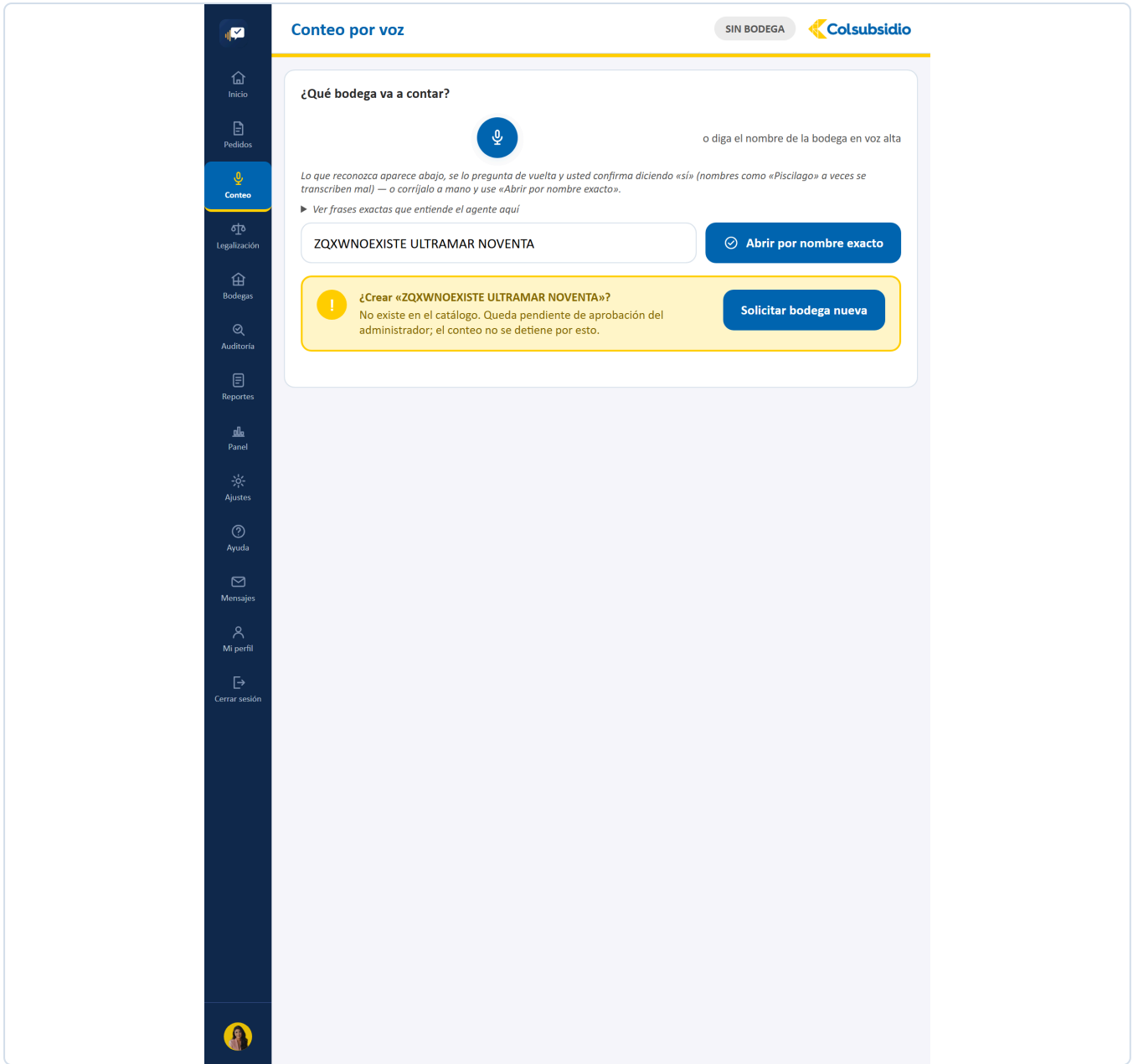


FIGURA 15 Cuando el nombre exacto no existe, se dice: no se abre otra parecida.

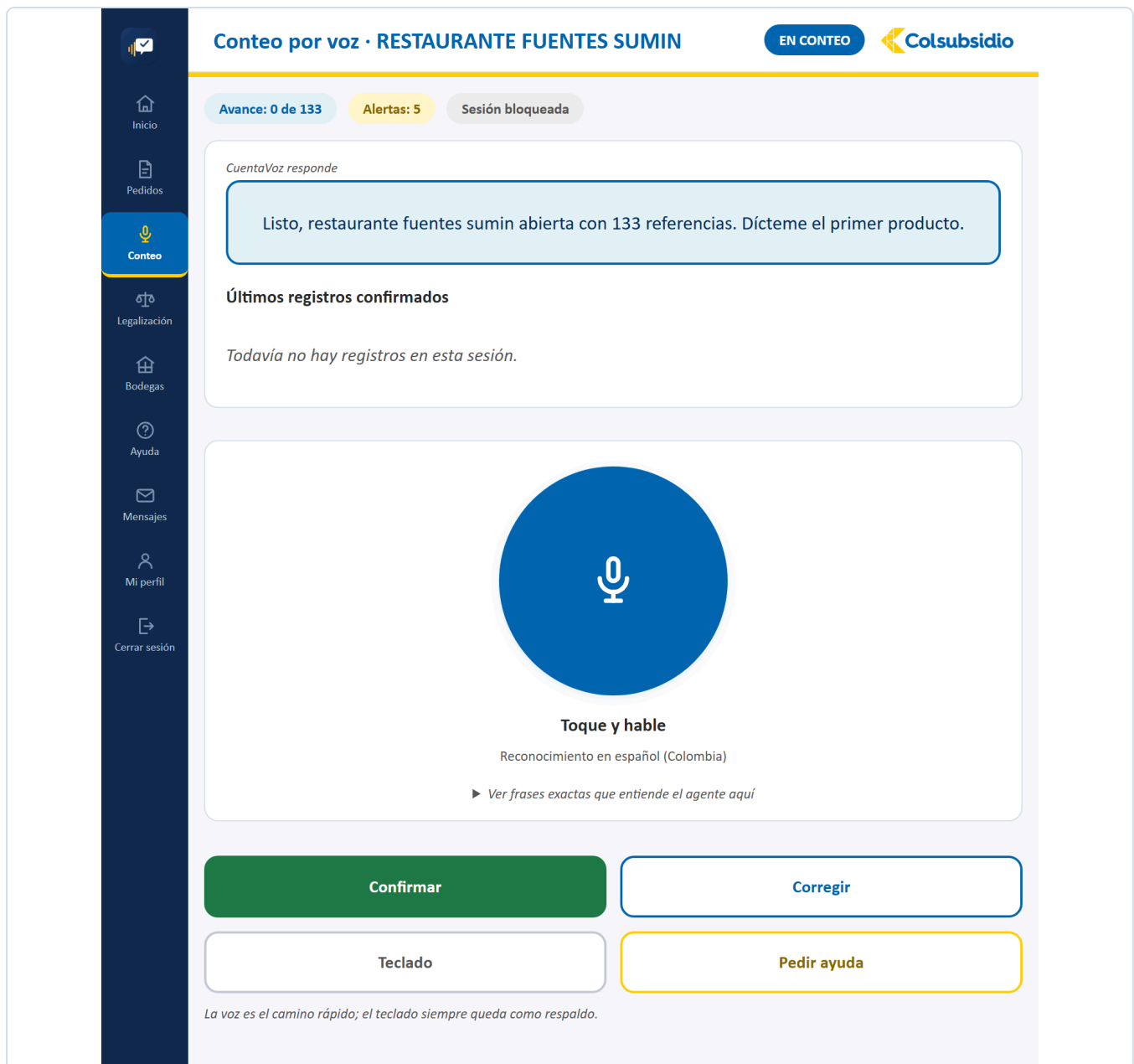


FIGURA 16 Bodega abierta: el avance y las alertas arriba, y los cuatro botones abajo.

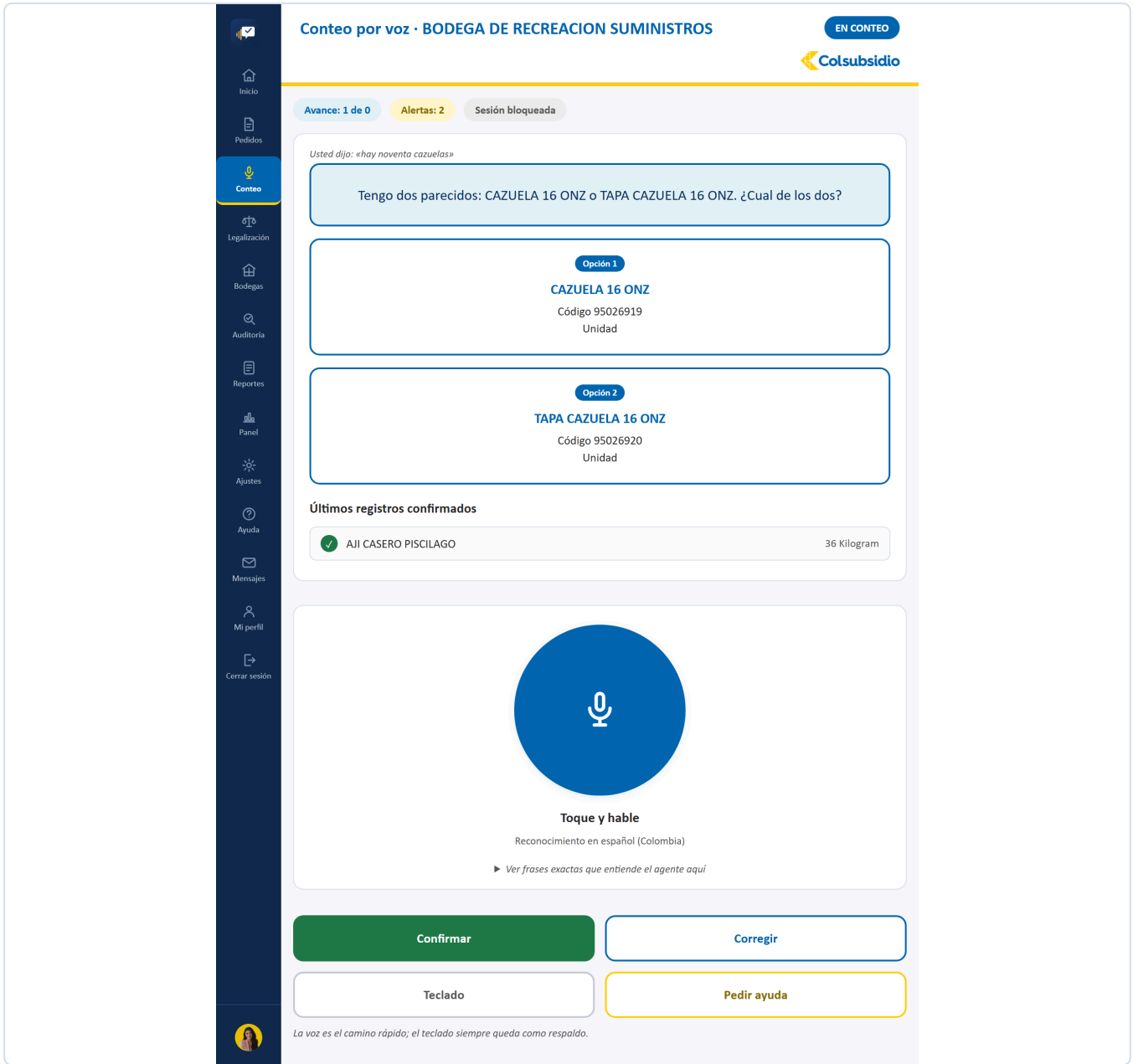


FIGURA 17 Varias candidatas del catálogo, como tarjetas con su código.

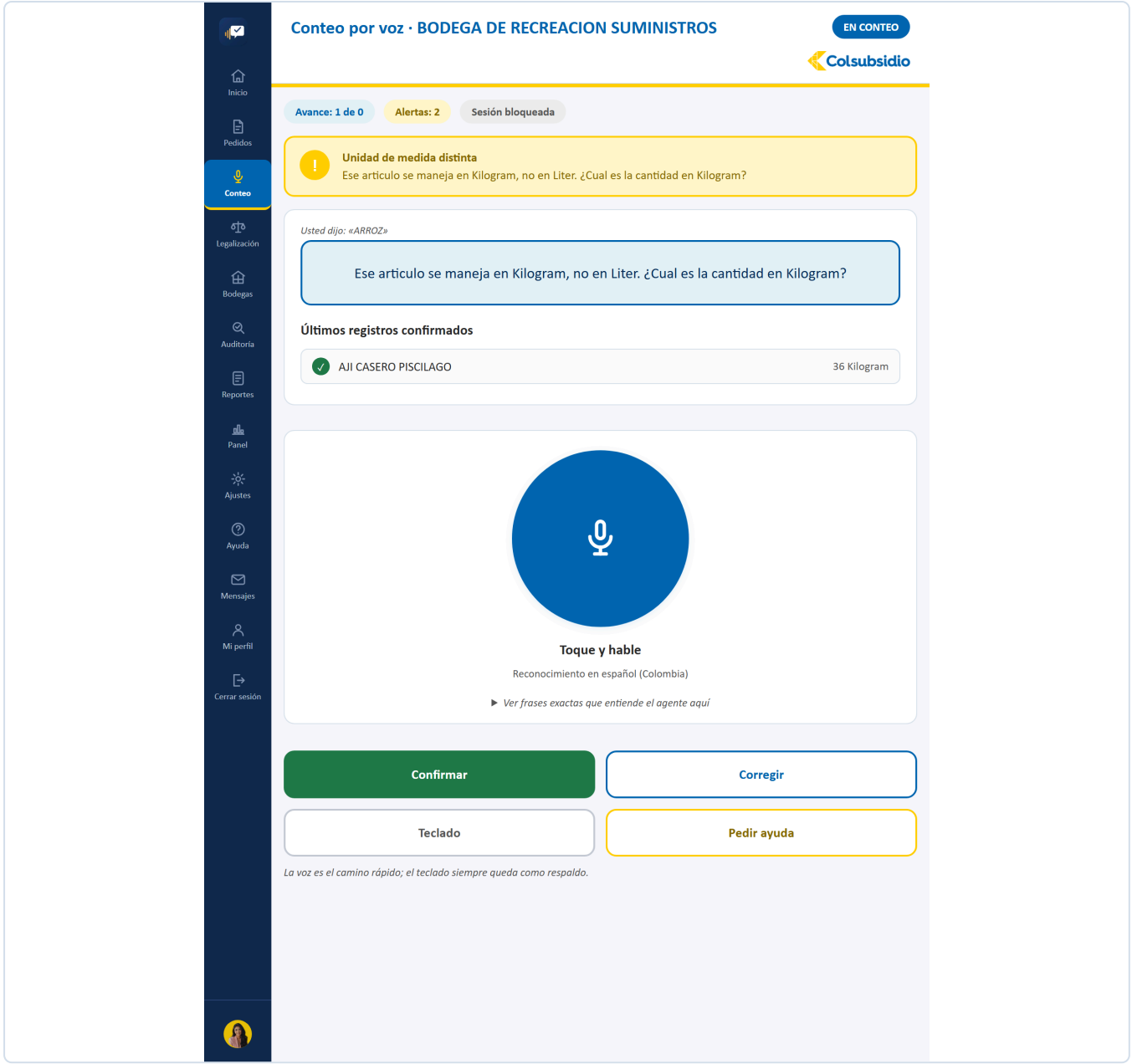


FIGURA 18 La cantidad o la unidad se salen de lo esperado: se pregunta antes de guardar.

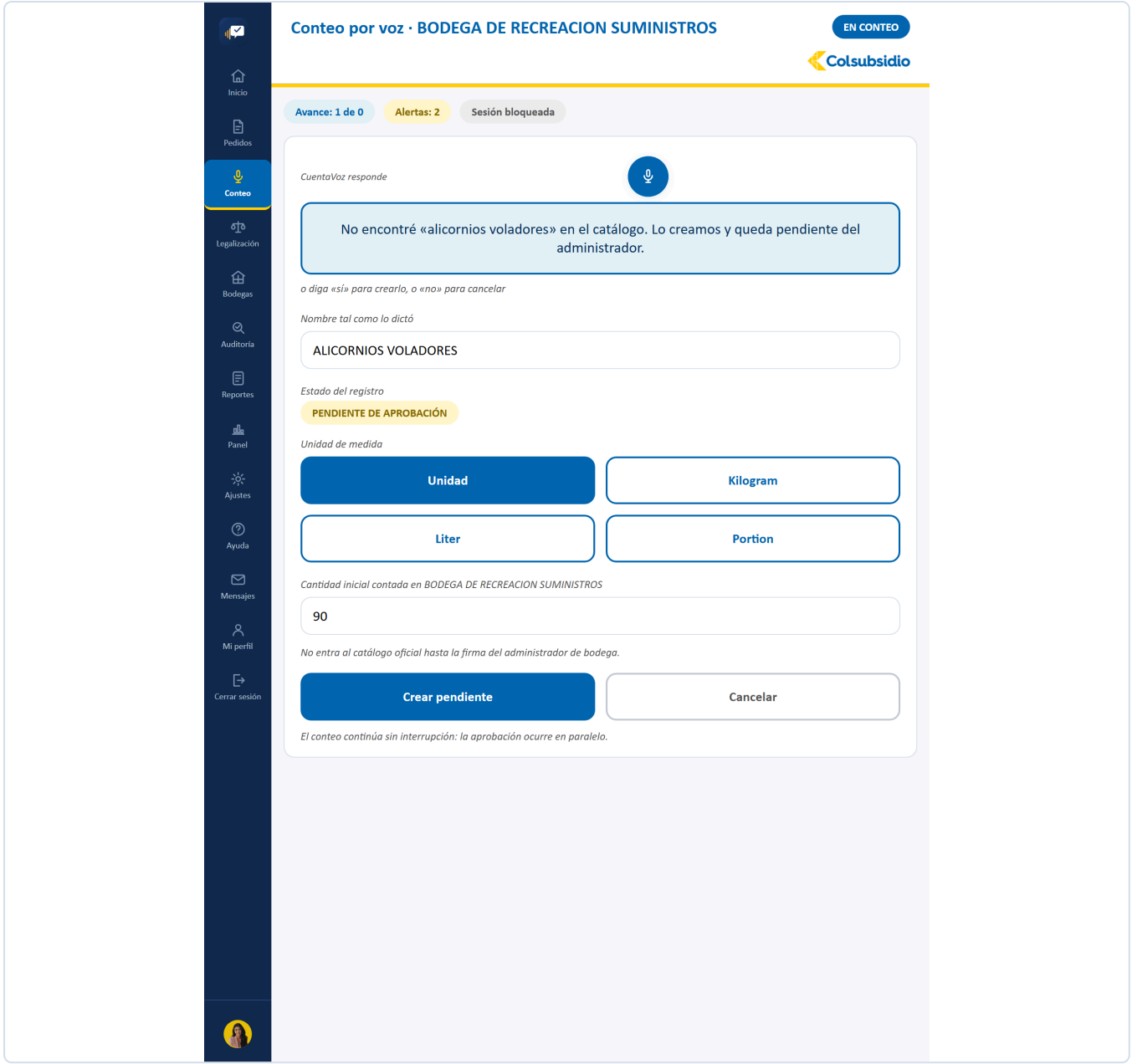


FIGURA 19 Lo que no está en el catálogo se crea sobre la marcha y queda pendiente de aprobación.

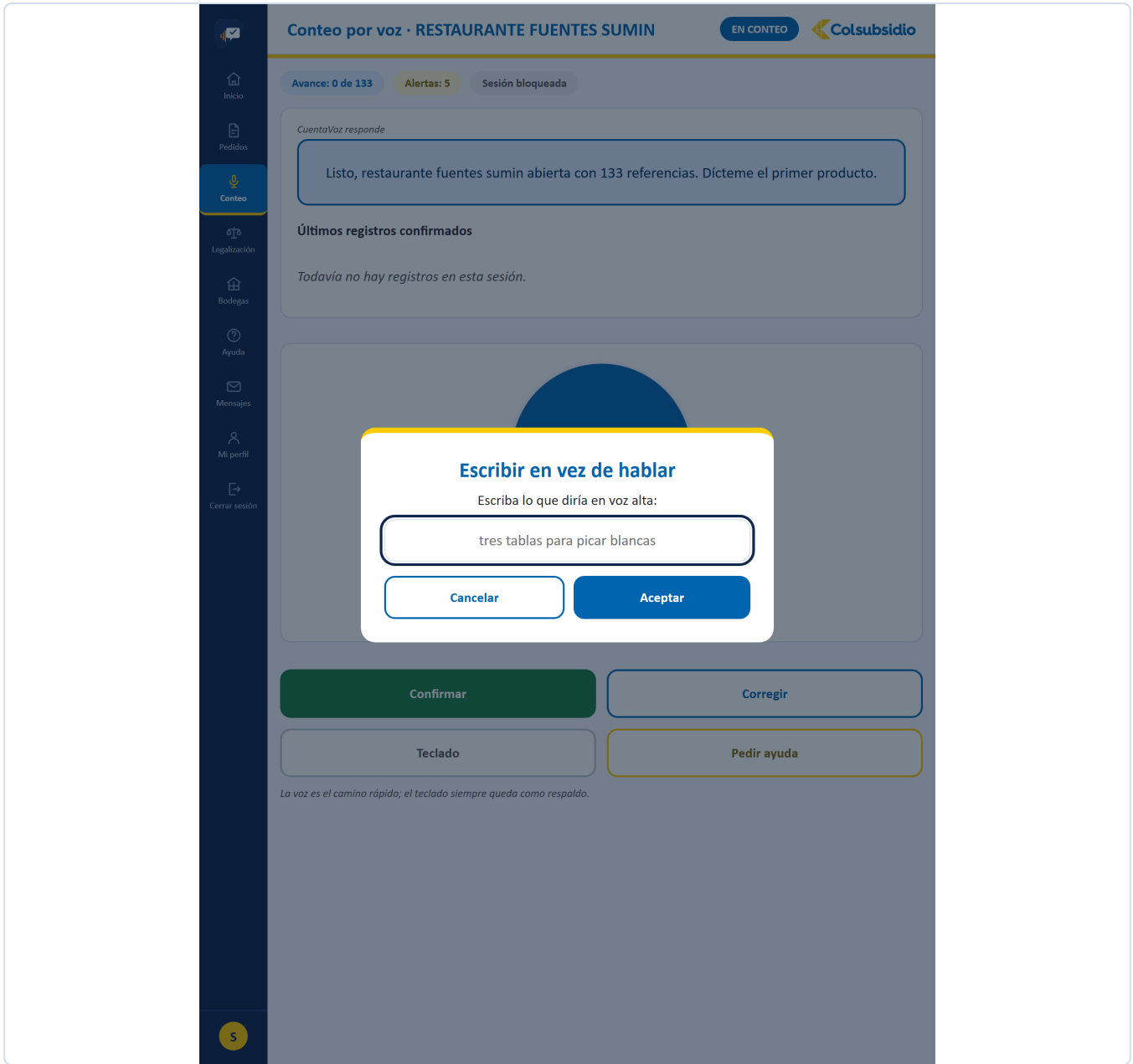


FIGURA 20 El respaldo de siempre: si la voz falla, se escribe.

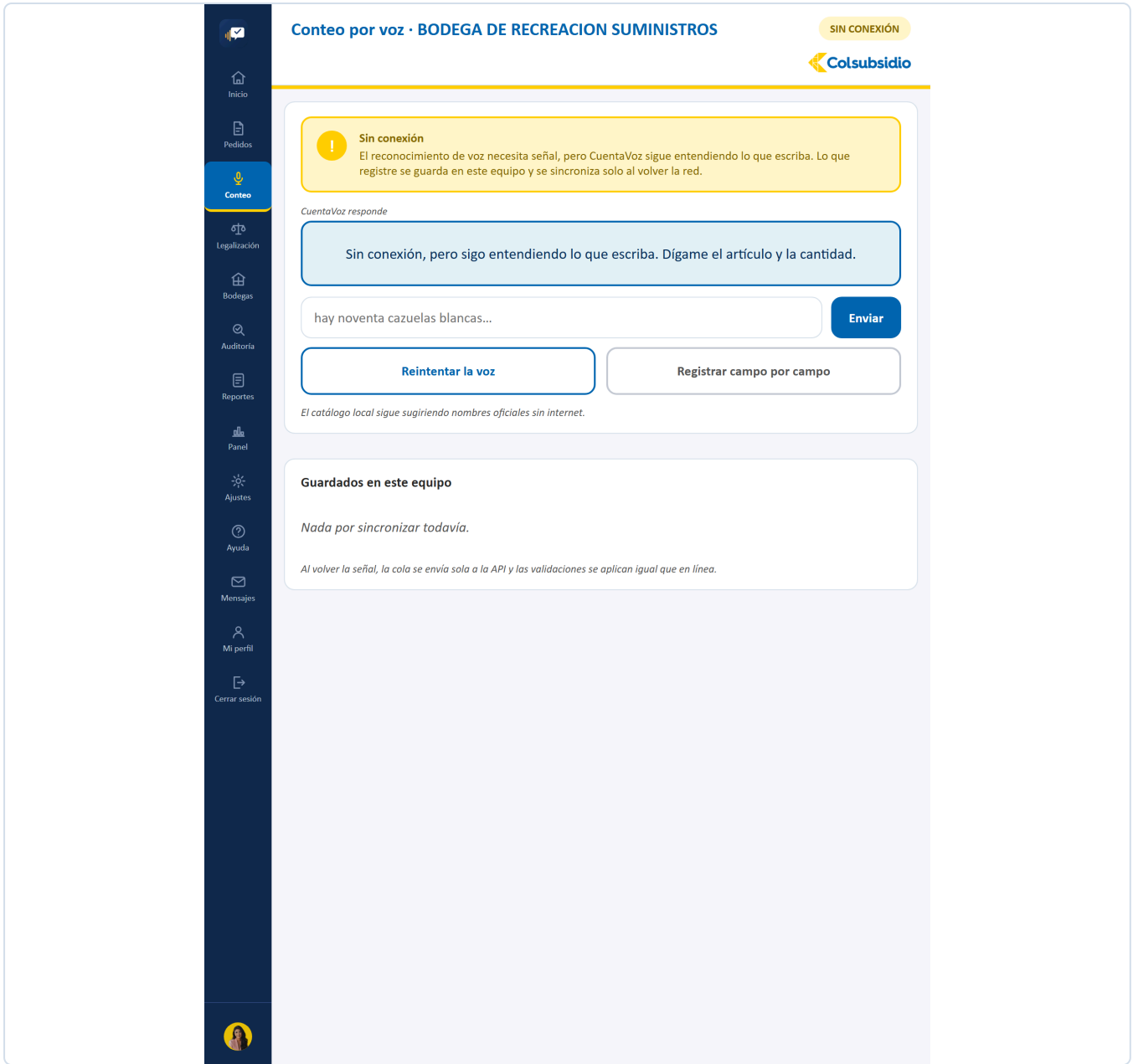


FIGURA 21 Sin red el conteo sigue por escrito, y se sincroniza solo al volver la señal.

12.5 Legalización MENÚ 4

ARCHIVO	Legalizacion.jsx · 297 líneas
QUÉ TIENE	La tabla de lo pedido contra lo usado, con la diferencia y su nombre (cuadro exacto, sobrante que vuelve a bodega, o merma). Cada renglón se puede ajustar dictando.
QUÉ RESUELVE	Cerrar el turno con las cuentas claras: lo que sobró vuelve al inventario y lo que faltó queda explicado, en vez de desaparecer.
CON QUÉ HABLA	/api/legalizacion/{servicio}, /api/legalizacion/ajustar y /api/legalizacion/confirmar.



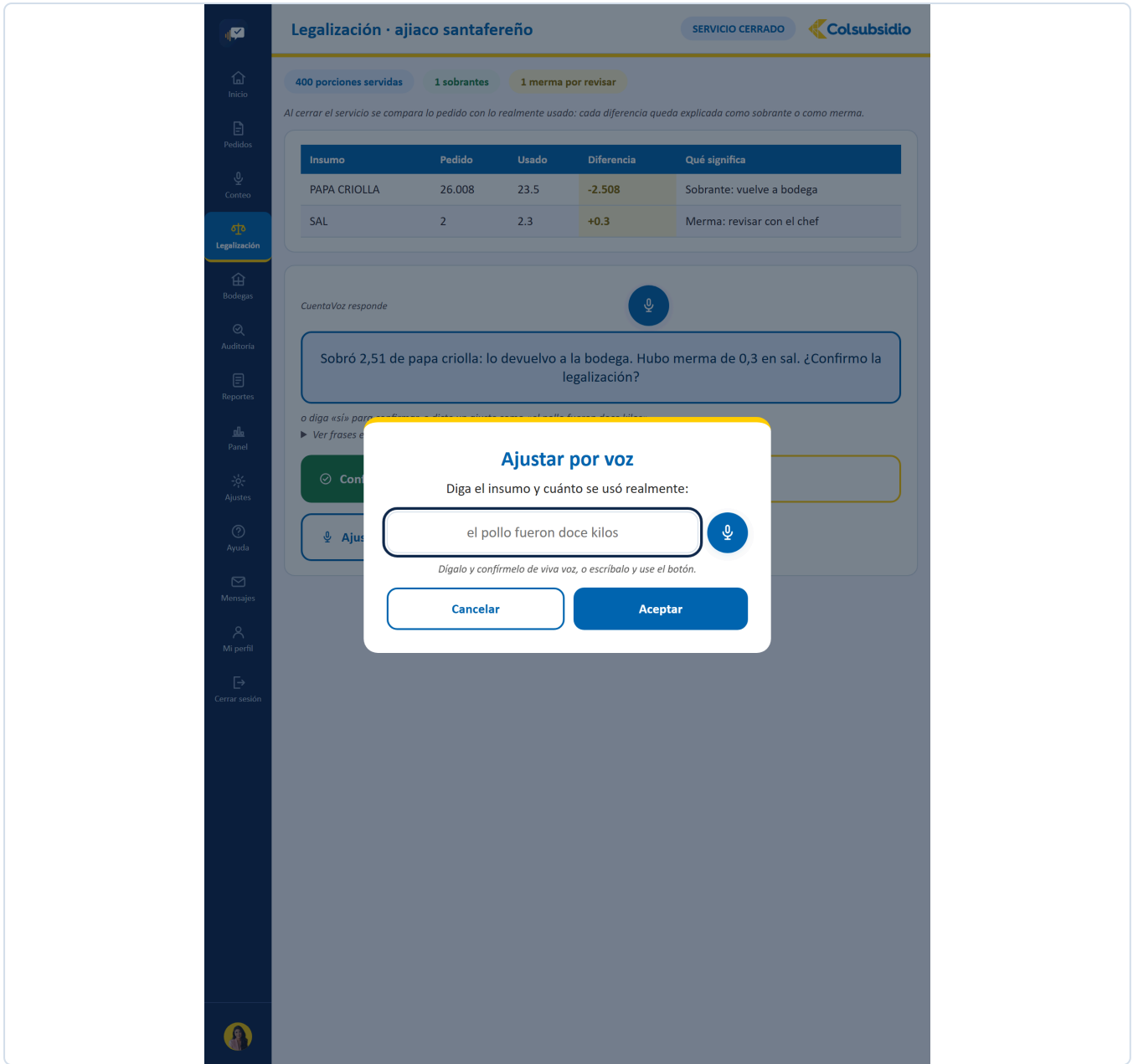


FIGURA 23 Cada cifra se corrige dictándola, sin salir de la tabla.

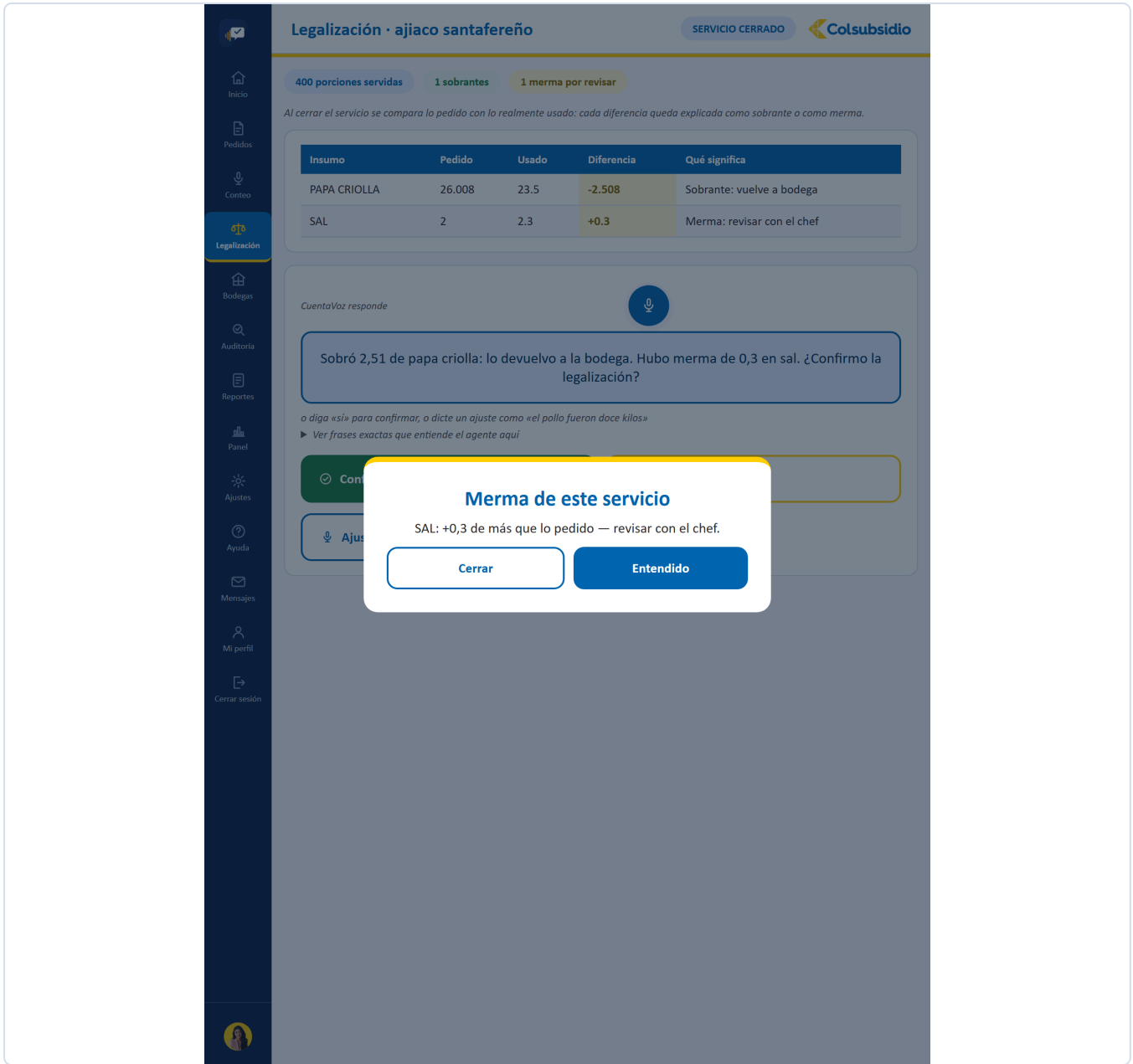


FIGURA 24 Lo que faltó no desaparece: queda explicado y firmado.

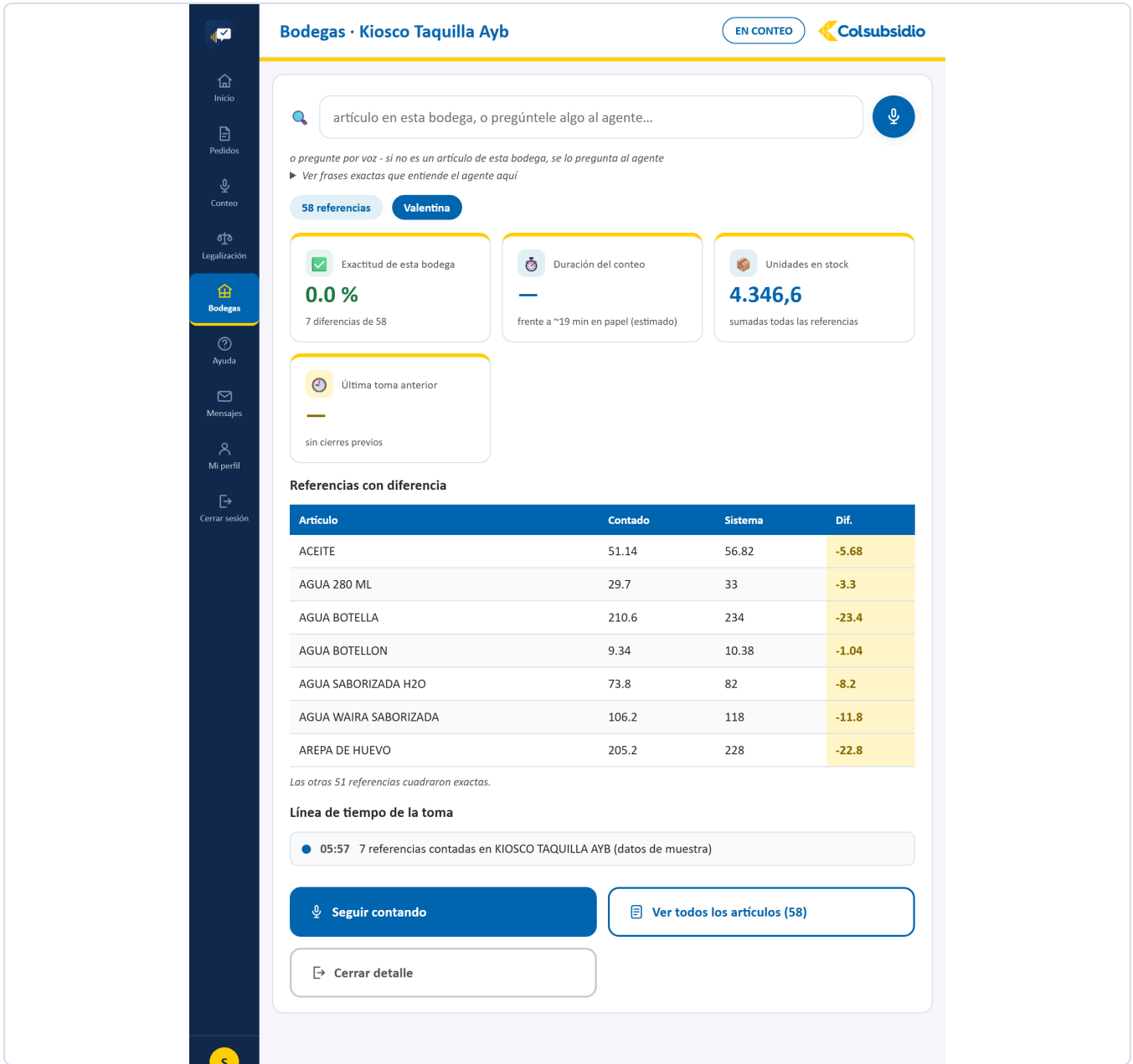
12.6 Bodegas

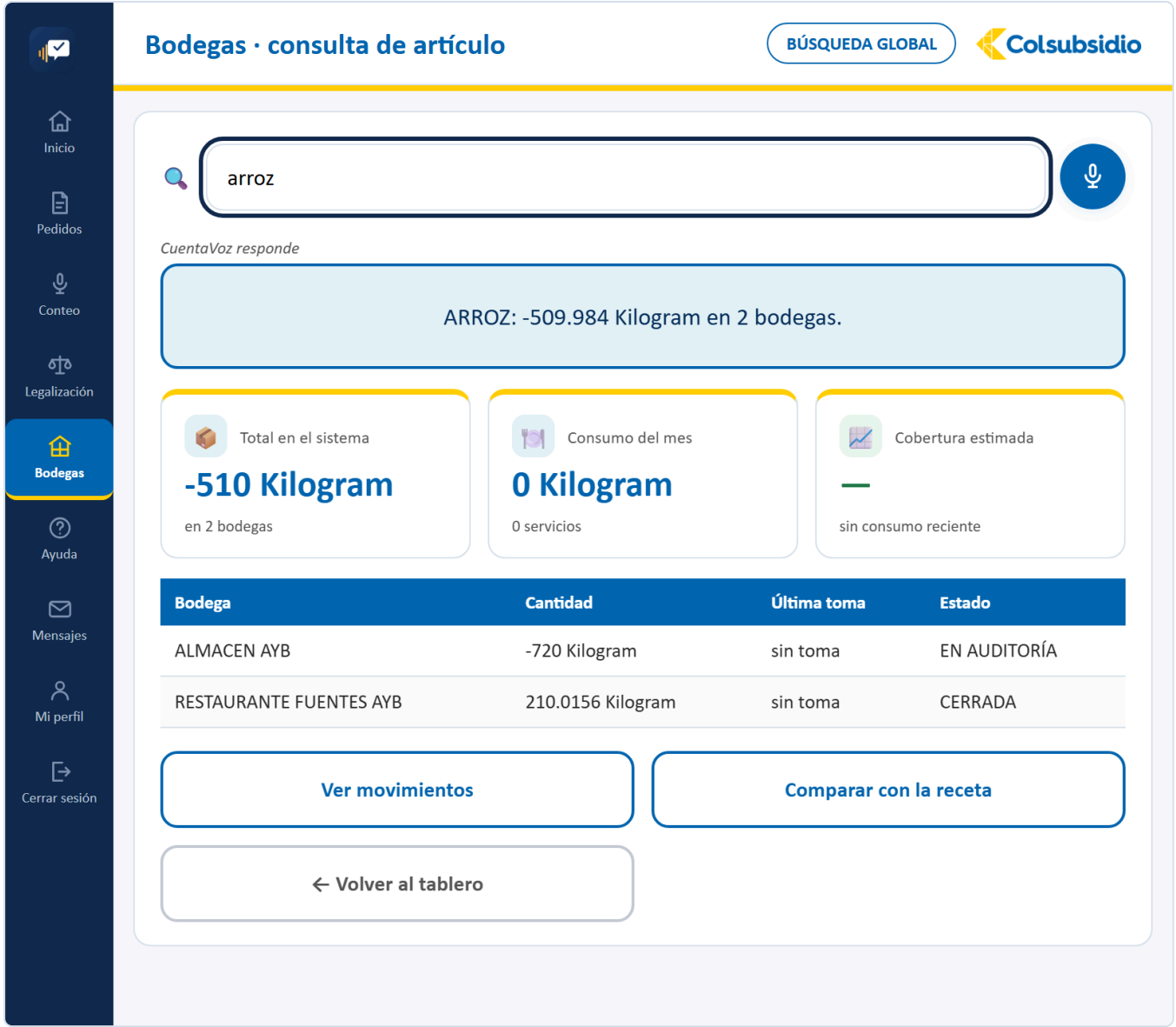
MENÚ 5

ARCHIVO	Bodegas.jsx · 848 líneas
QUÉ TIENE	Una tarjeta por zona con su estado por color (pendiente, en conteo, en auditoría, cerrada) y su avance; el detalle al tocar una; el buscador de artículos por voz («¿cuánto arroz hay y en qué bodegas?»); y el alta de una bodega nueva, que queda pendiente de aprobación.
QUÉ RESUELVE	Saber en qué va el parque completo sin llamar a nadie, y encontrar un artículo sin recorrer estantes.
CON QUÉ HABLA	/api/bodegas y sus derivados, /api/articulos/buscar. Se actualiza sola por WebSocket cuando alguien abre o cierra una bodega.



FIGURA 25 Bodegas. El parque completo, con el estado de cada zona por color.





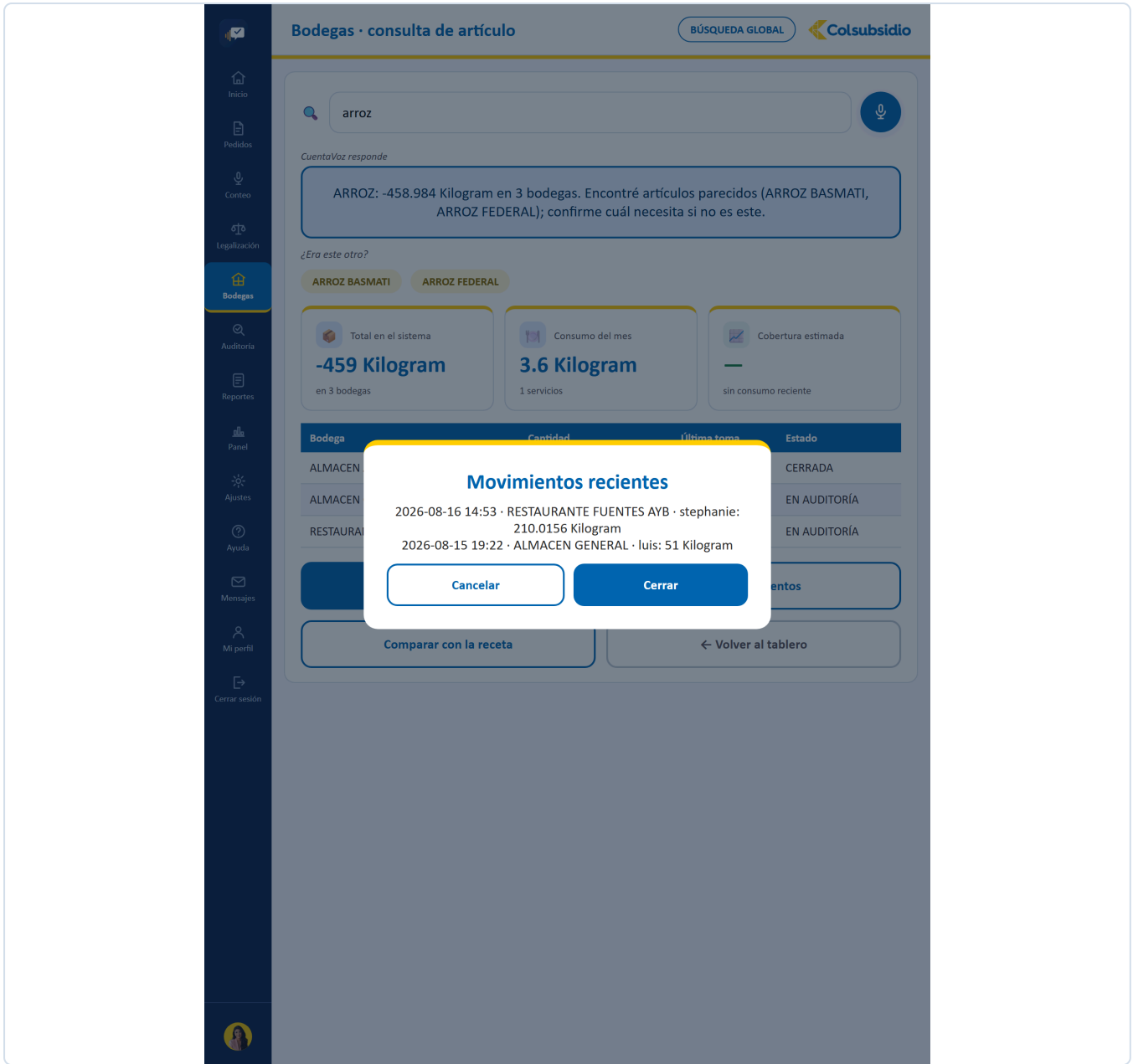


FIGURA 28 Su trazabilidad en el tiempo.

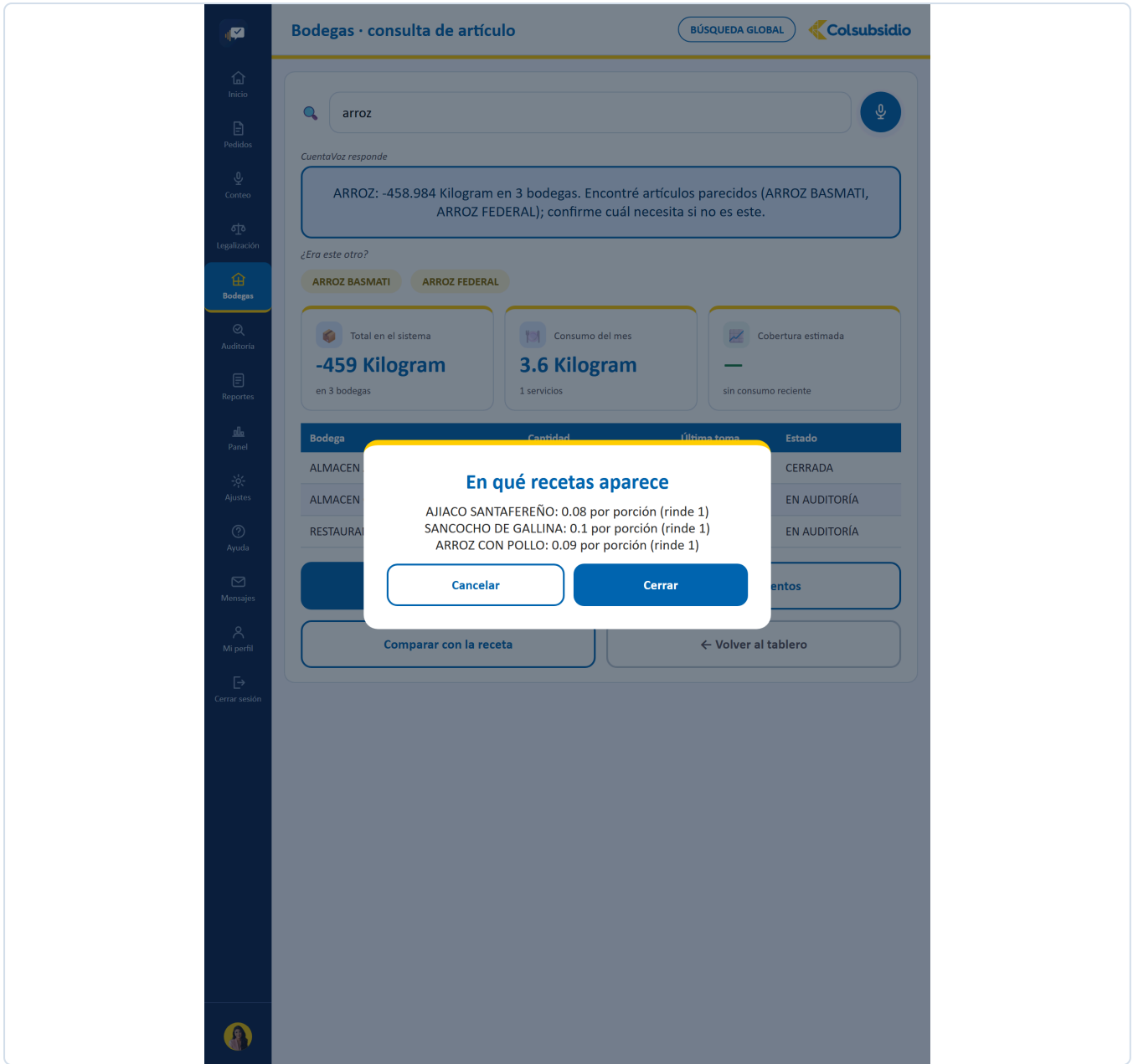


FIGURA 29 Con qué platos se relaciona ese insumo.

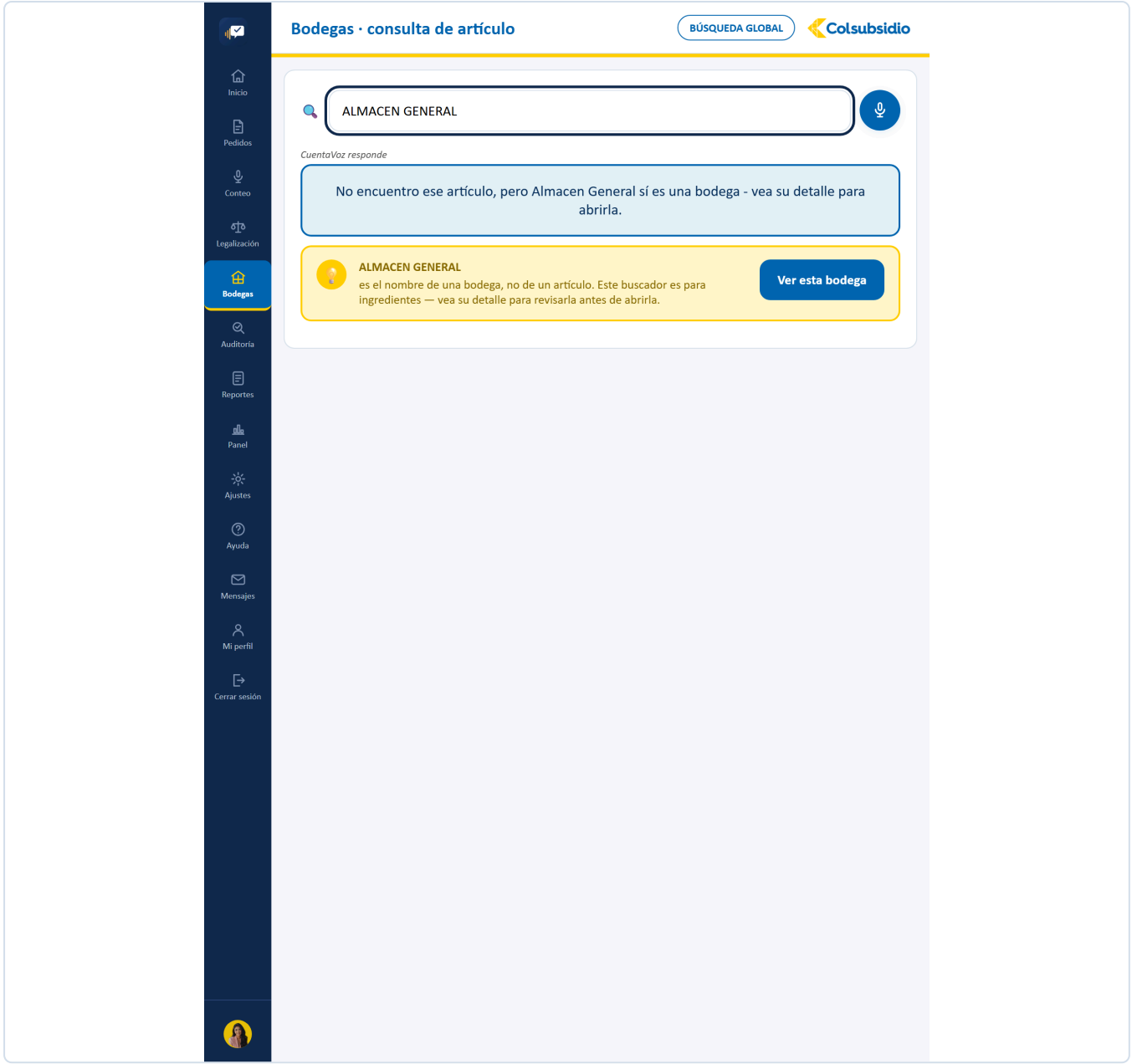


FIGURA 30 La plataforma lo nota y sugiere la bodega, en vez de no encontrar nada.

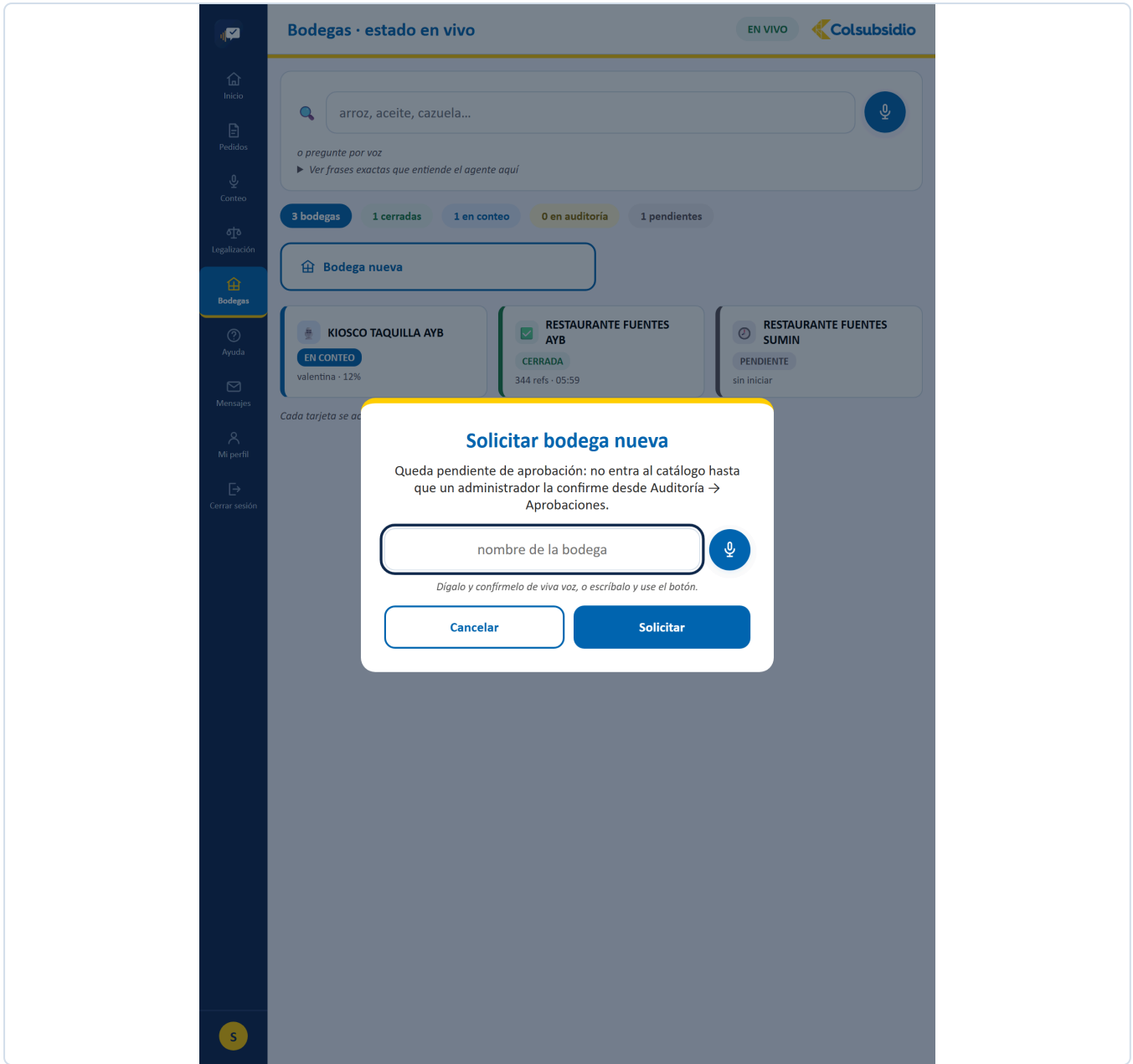


FIGURA 31 Queda pendiente de aprobación del administrador.

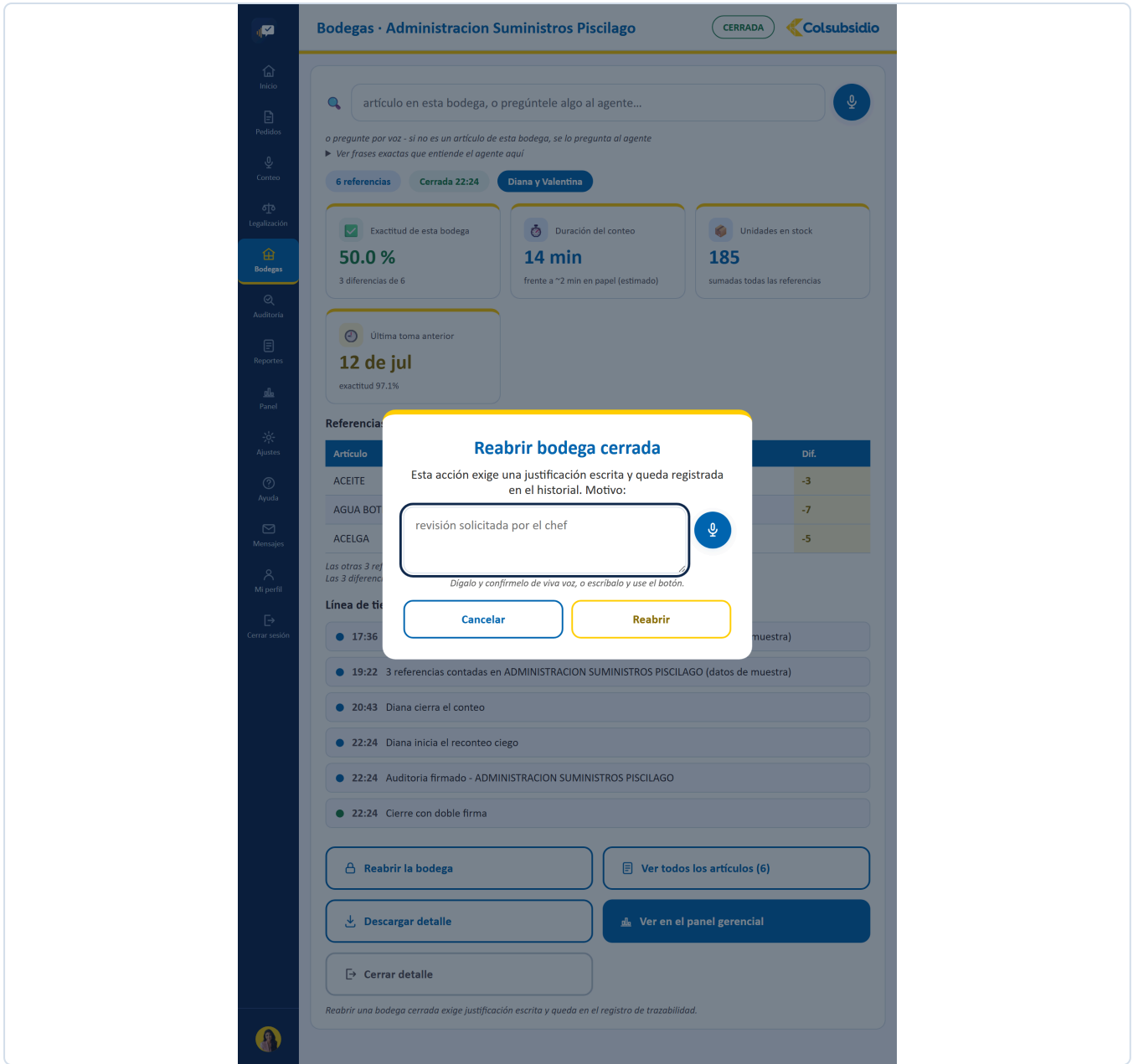


FIGURA 32 Es una acción de peligro y se confirma como tal.

12.7 Auditoría

MENÚ 6 · ADMINISTRADOR

ARCHIVO	Auditoria.jsx · 949 líneas
PESTAÑAS	Recuento ciego y cierre · Aprobaciones · Pedidos pendientes · Bandeja de alertas
QUÉ RESUELVE	El control que solo puede hacer el administrador: verificar una bodega sin ver los números del auxiliar, dar visto bueno a lo que se creó sobre la marcha, autorizar lo que sale del almacén y resolver las diferencias marcadas.
CON QUÉ HABLA	/api/aprobaciones, /api/alertas, /api/pedidos/pendientes y los endpoints de cierre de bodega.

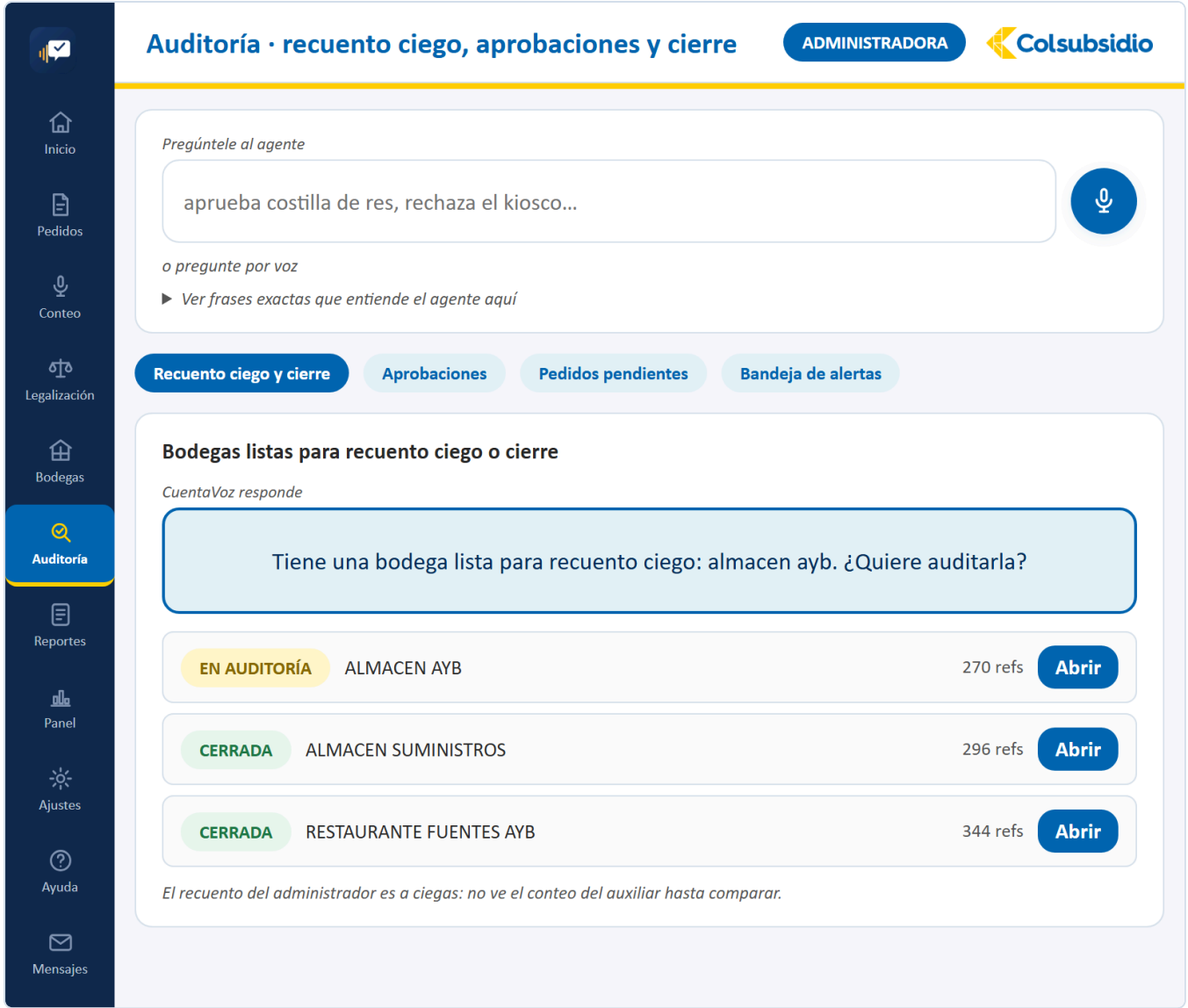


FIGURA 33 Auditoría · Recuento ciego y cierre, la pestaña por defecto.

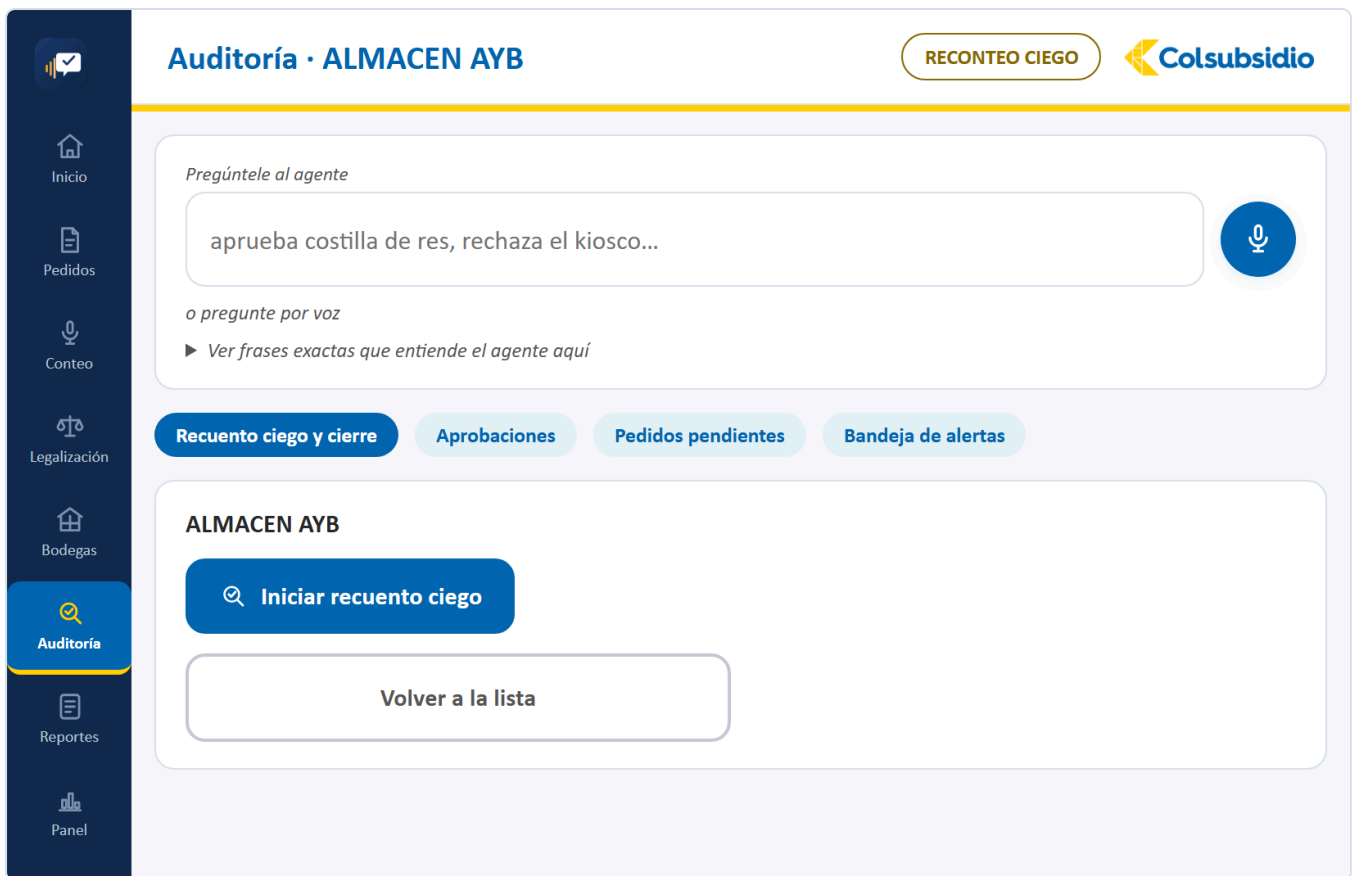


FIGURA 34 Elegida la bodega, el encabezado cambia a **RECONTEO CIEGO** y aparece el botón para empezar.

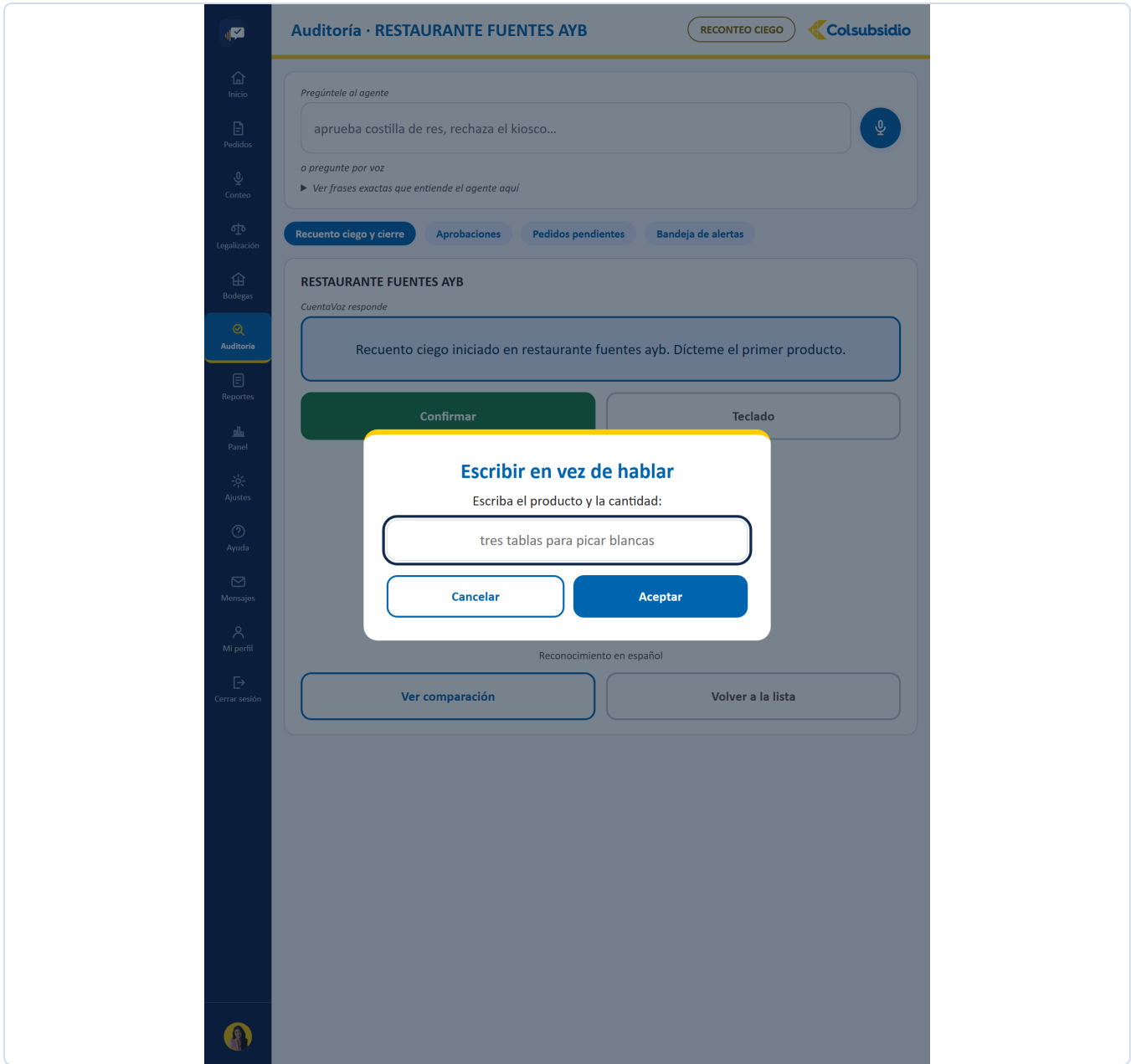


FIGURA 35 El mismo respaldo del conteo, dentro de la auditoría.

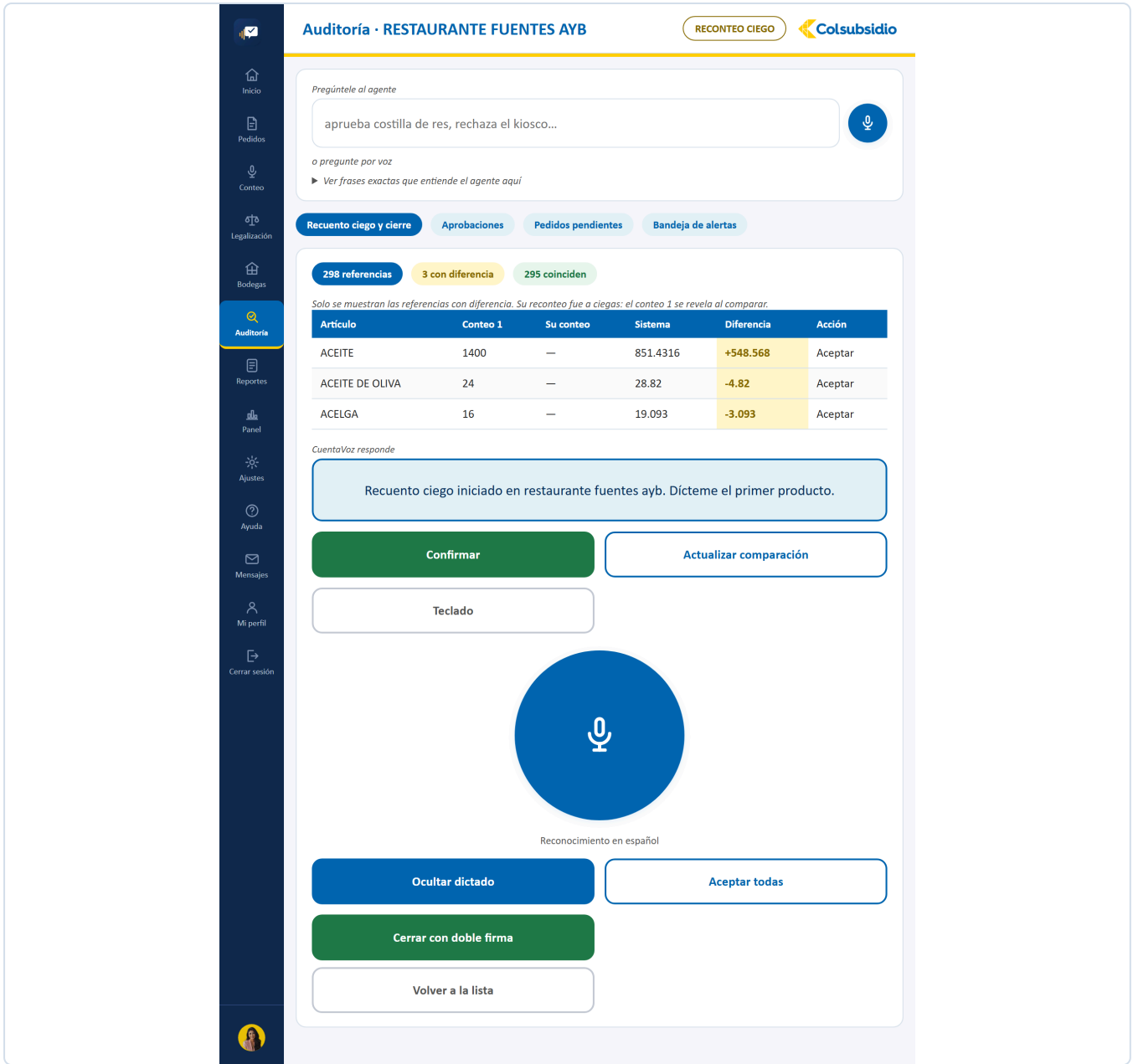


FIGURA 36 Solo al terminar se revela: sistema contra conteo original contra el suyo.

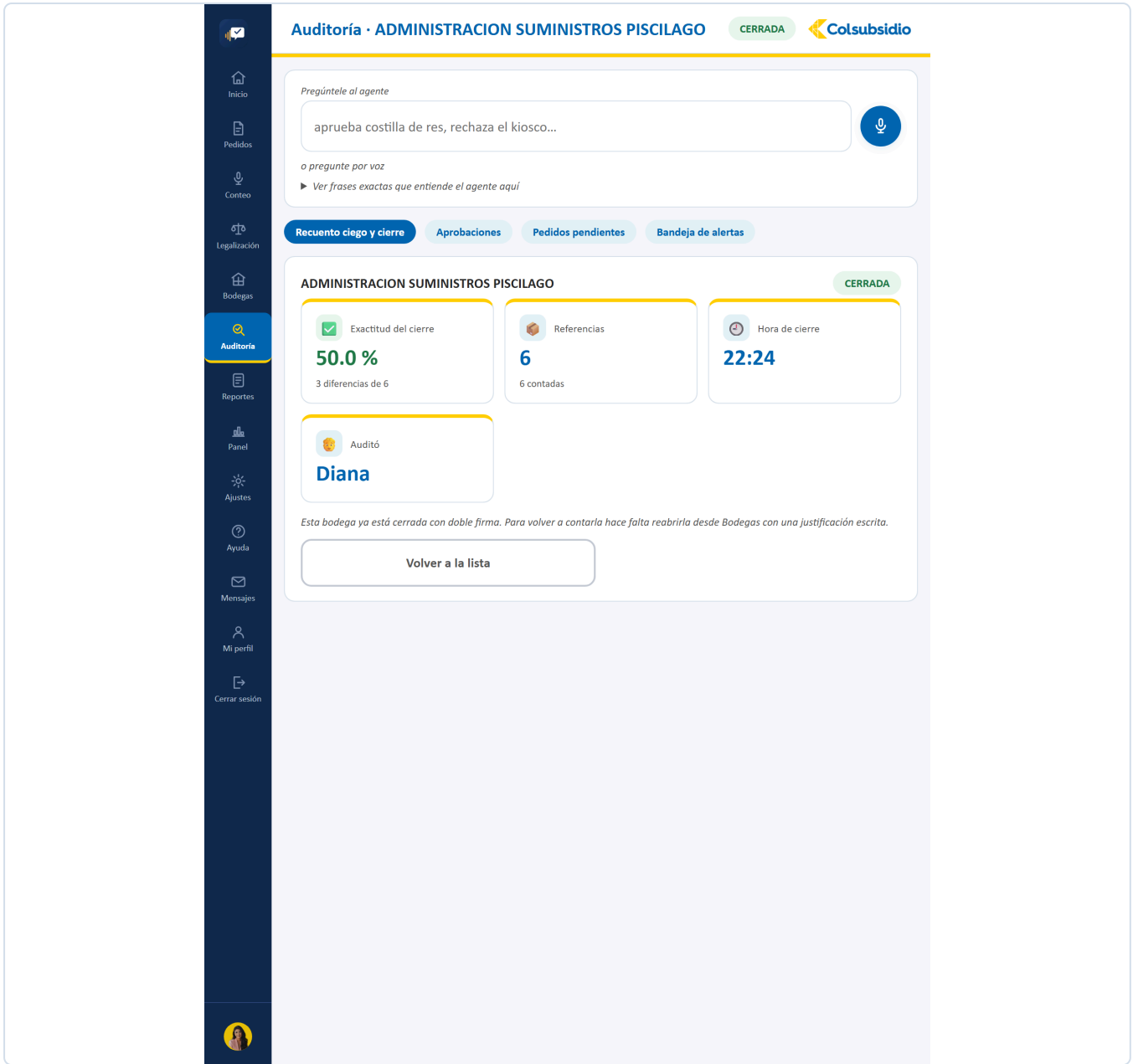


FIGURA 37 Solo lectura, con los indicadores del cierre.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Aprobaciones pendientes

2 PENDIENTES

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre

Aprobaciones

Pedidos pendientes

Bandeja de alertas

Productos y bodegas creados durante la toma que esperan su firma para entrar al catálogo oficial.
CuentaVoz responde

Tiene 2 aprobaciones pendientes: galletas surtidas paquete x24, bodega temporal evento especial. ¿Cuál quiere aprobar o rechazar?

GALLETAS SURTIDAS PAQUETE X24
Producto nuevo · Unidad
30 Unidad contados en KIOSCO PISCIGIROS AYB
Luis · 06:01

Rechazar

Aprobar

BODEGA TEMPORAL EVENTO ESPECIAL
Bodega nueva
sin referencias asignadas todavía
Luis · 06:01

Rechazar

Aprobar

Al aprobar, el producto entra al catálogo con su código definitivo y el conteo asociado deja de estar marcado. Mientras tanto el conteo nunca se detiene: la aprobación ocurre en paralelo al trabajo en bodega.

FIGURA 38 Lo que se creó en medio del conteo, esperando visto bueno.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Pedidos pendientes de aprobación

1 PENDIENTES

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre

Aprobaciones

Pedidos pendientes

Bandeja de alertas

El auxiliar pide y queda registrado de una vez, pero solo sale del almacén cuando usted lo aprueba aquí - igual que con productos y bodegas nuevas.
CuentaVoz responde

Tiene un pedido pendiente: sancocho de gallina, de luis. ¿Quiere aprobarlo o rechazarlo?

SANCOCHO DE GALLINA · 25 porciones **PED-20260819-060310-B333**

Se descontaría de: ALMACEN SUMINISTROS
Pedido Por Luis · 06:03

- POLLO ENTERO: 7.5
- PAPA CRIOLLA: 6.25
- YUCA: 3.75
- ARROZ: 2.5
- ACEITE: 0.375

Rechazar

Aprobar

Al aprobar, el pedido queda abierto para su legalización al final del servicio. Al rechazar, no cuenta como enviado y el auxiliar debe volver a intentarlo.

FIGURA 39 Se autoriza antes de que salga del almacén.

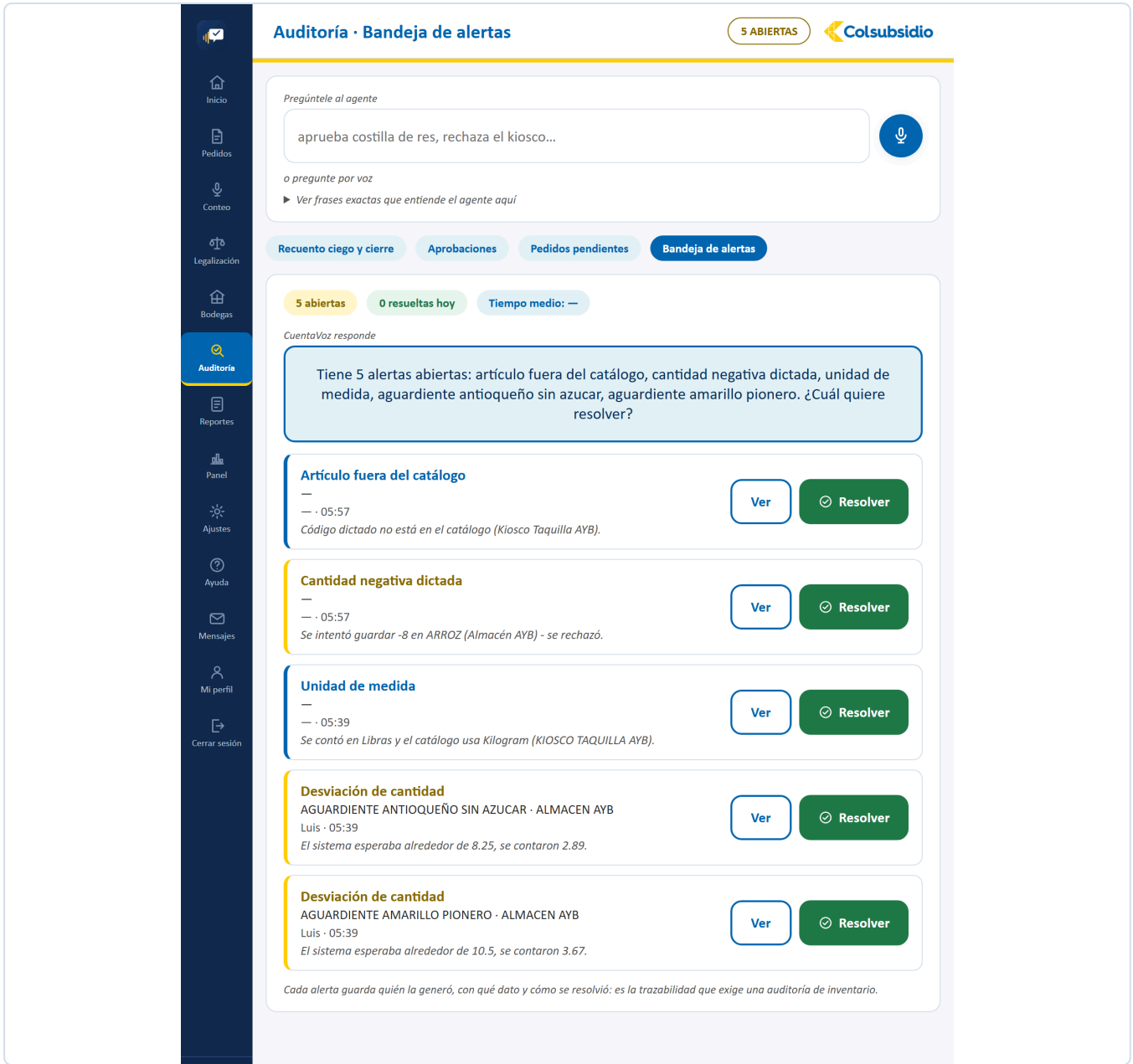


FIGURA 40 Las diferencias marcadas, agrupadas por tipo.

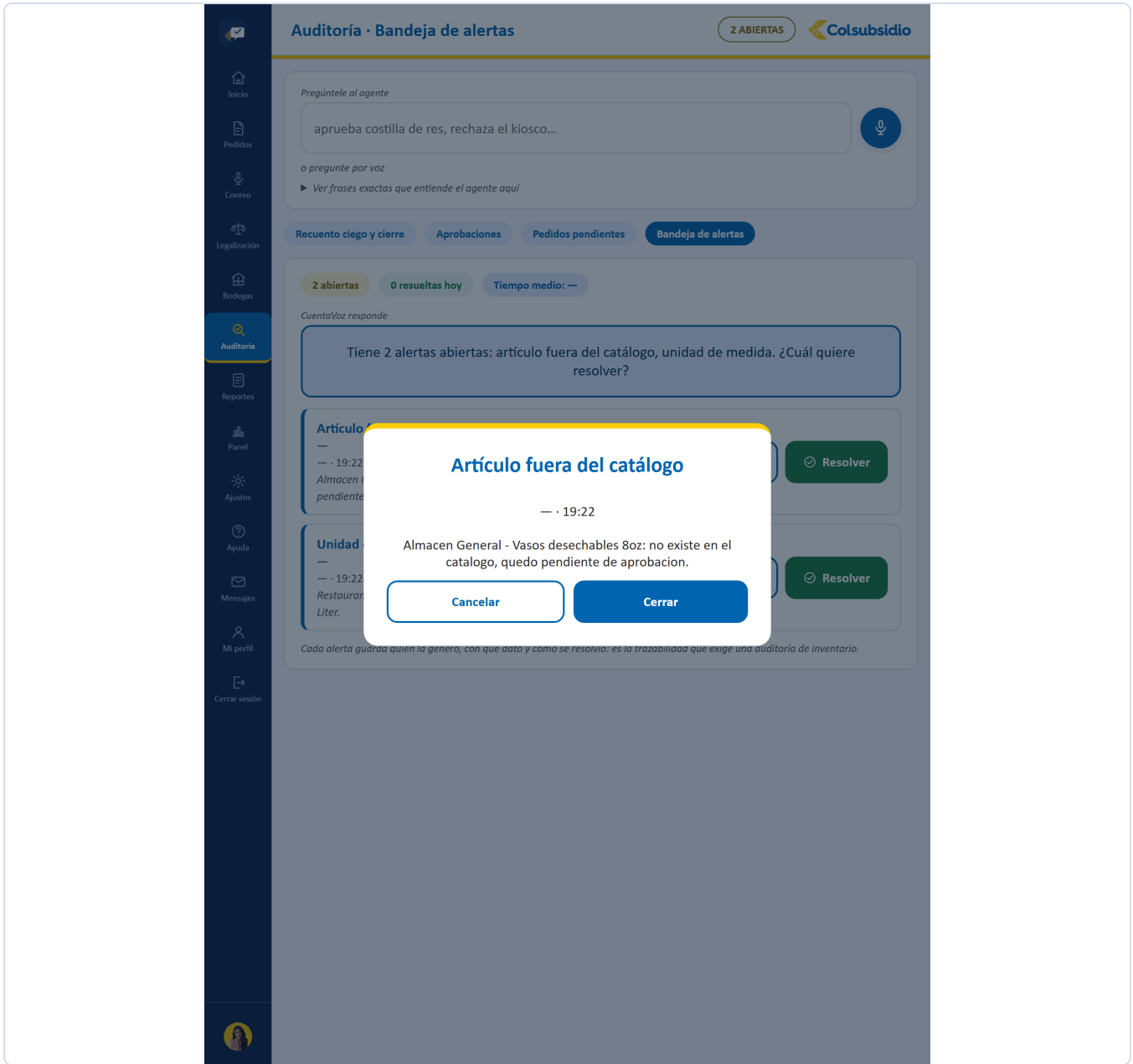
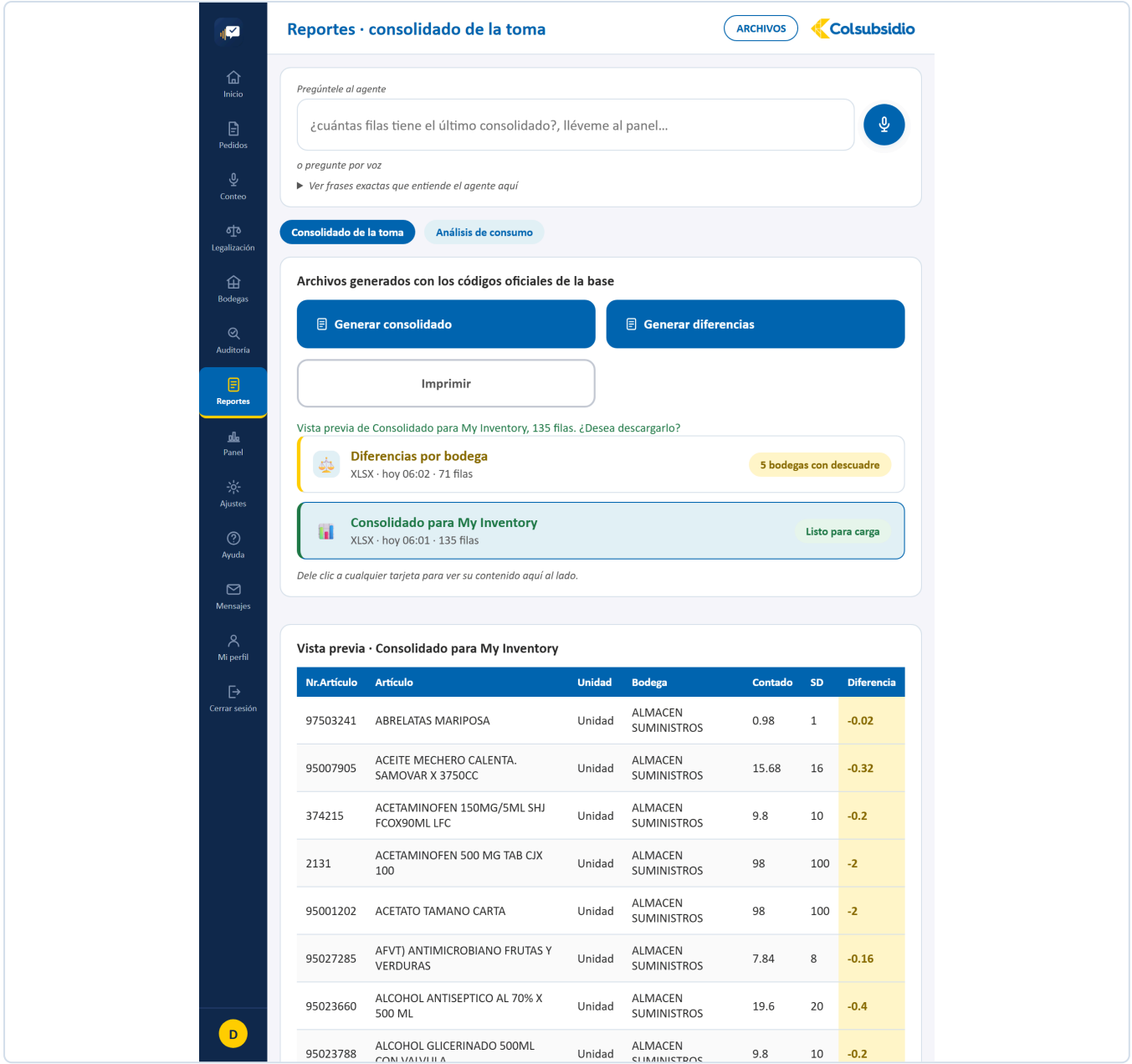


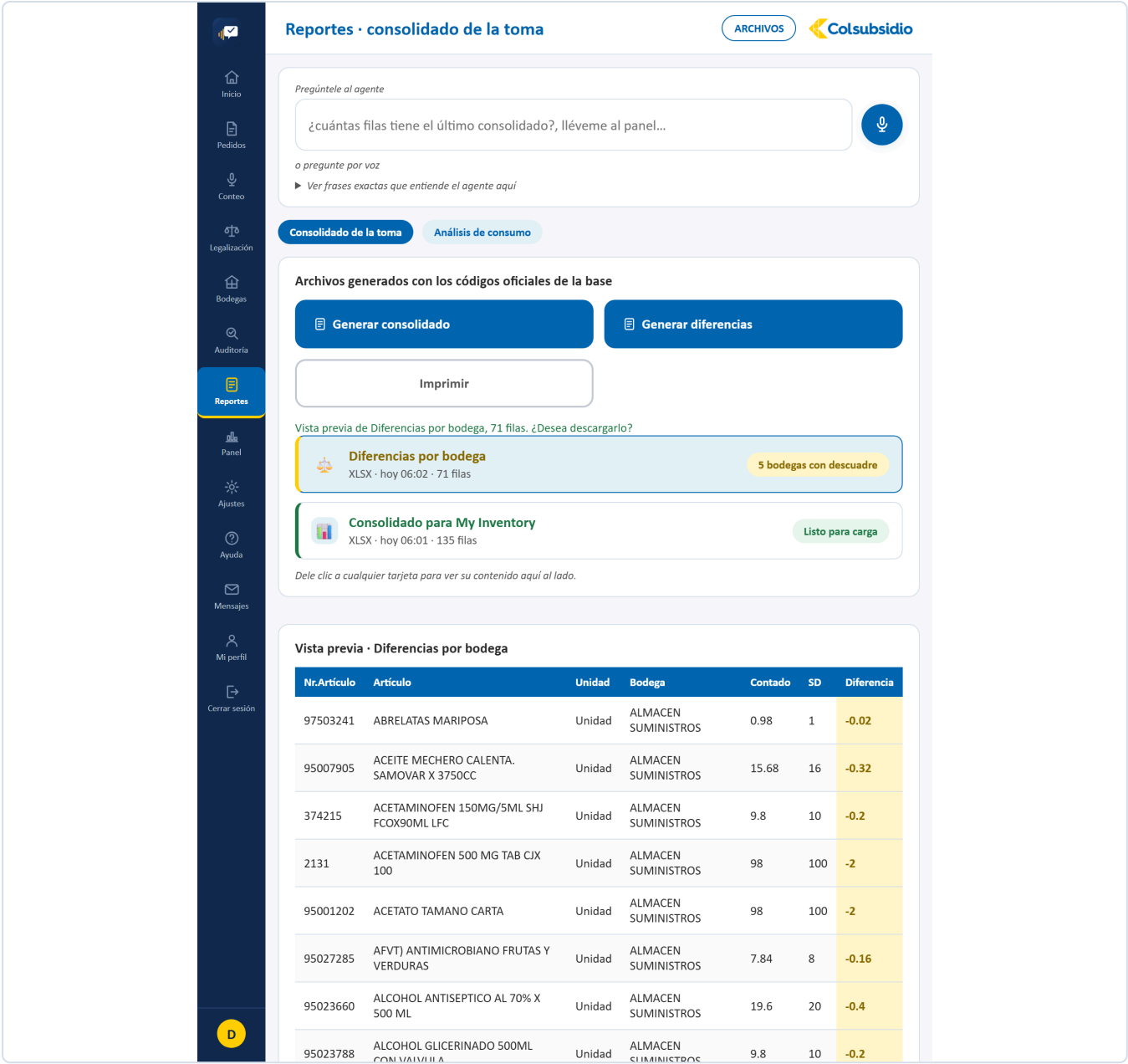
FIGURA 41 Su motivo exacto, para poder resolverla.

12.8 Reportes **MENÚ 7 · ADMINISTRADOR**

ARCHIVO	Reportes.jsx · 453 líneas
PESTAÑAS	Consolidado de la toma · Análisis de consumo
QUÉ TIENE	Los botones de generar (consolidado y diferencias), la lista de archivos ya generados con su estado, y el panel de vista previa a la derecha.
QUÉ RESUELVE	La salida hacia My Inventory, con los códigos oficiales. La previsualización evita cargar el archivo equivocado.
CON QUÉ HABLA	/api/reportes, /api/reportes/diferencias, /api/reportes/vista-previa y /api/reportes/descargar.

FIGURA 42 Reportes · Consolidado de la toma.





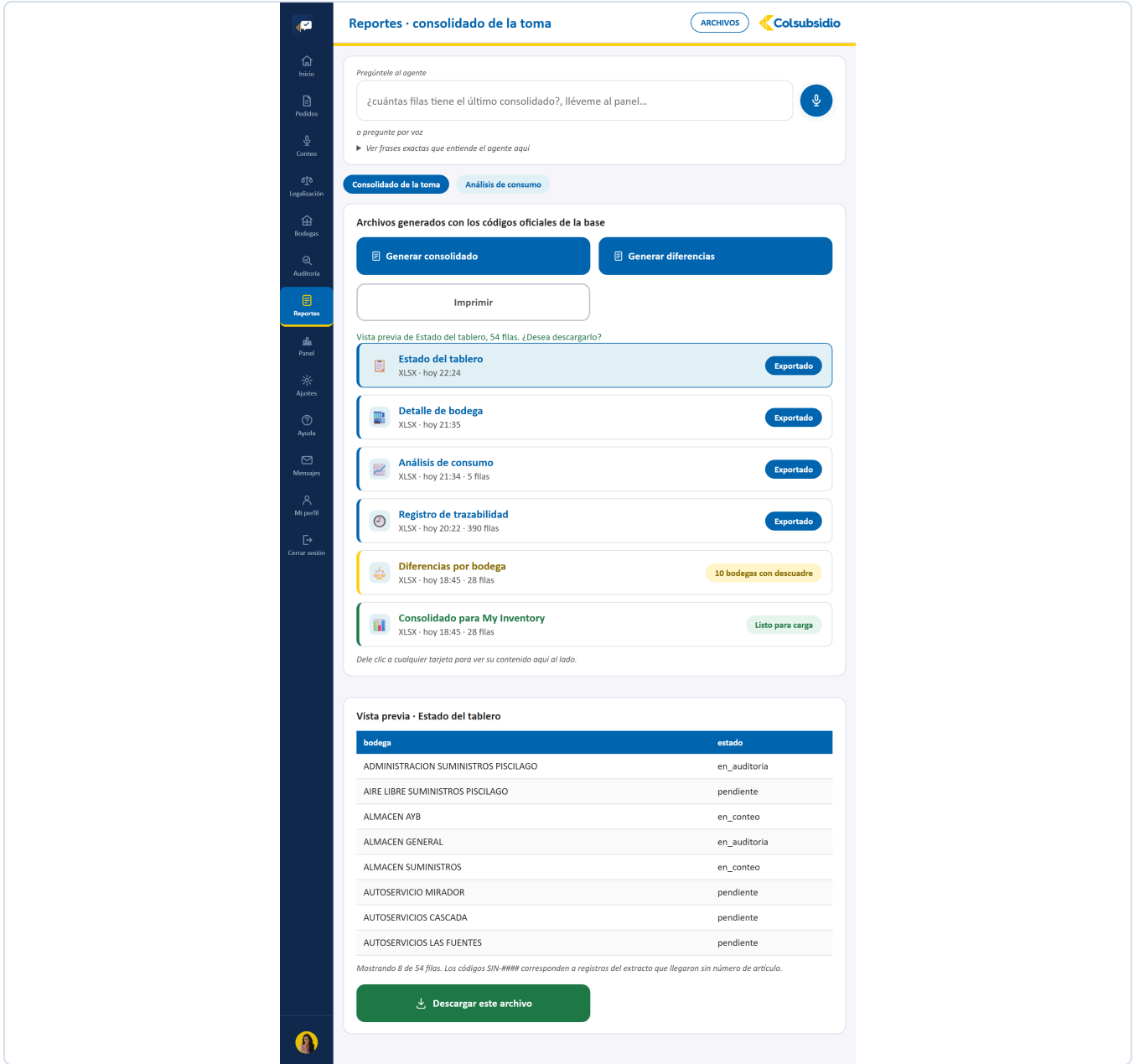
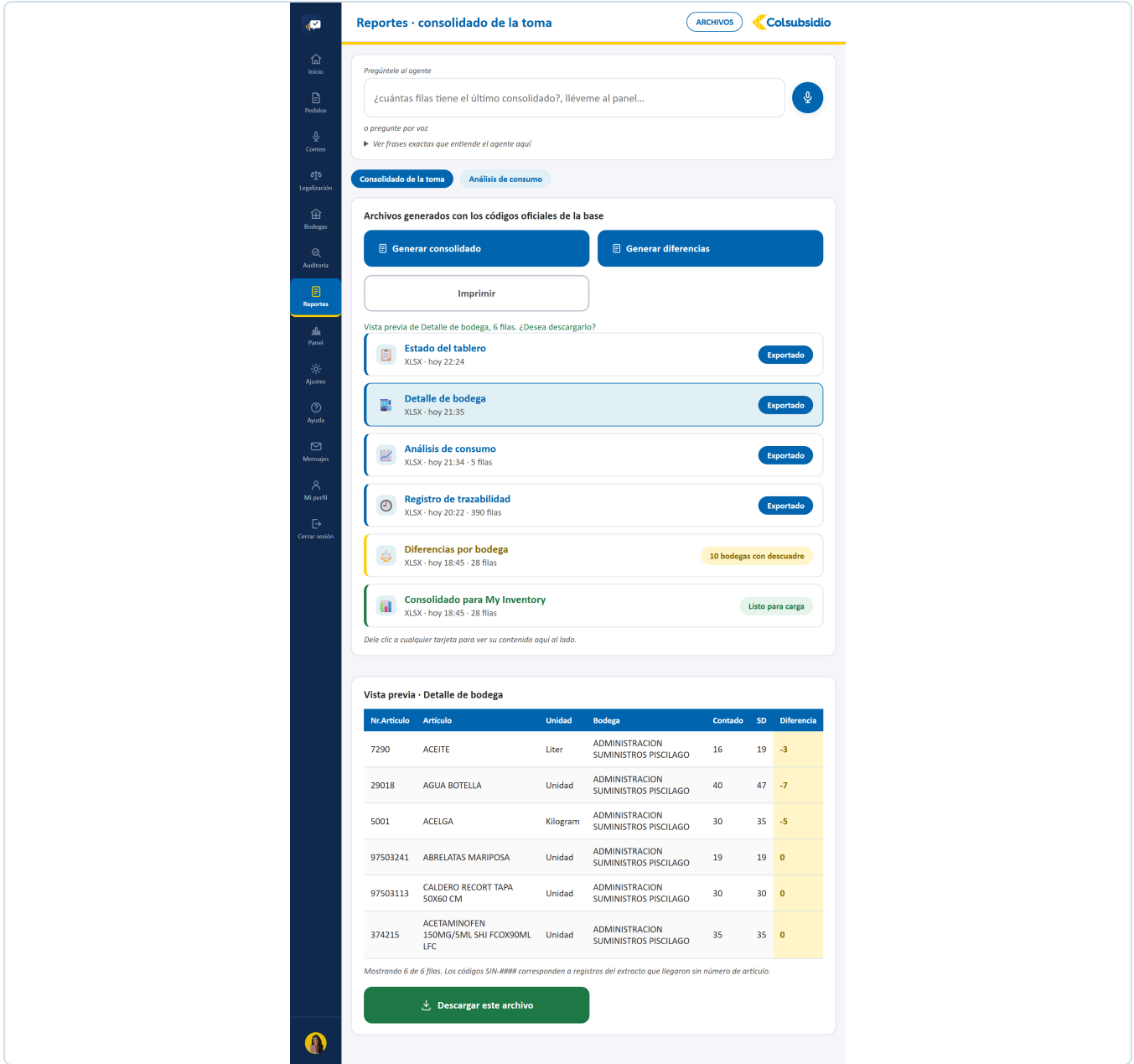
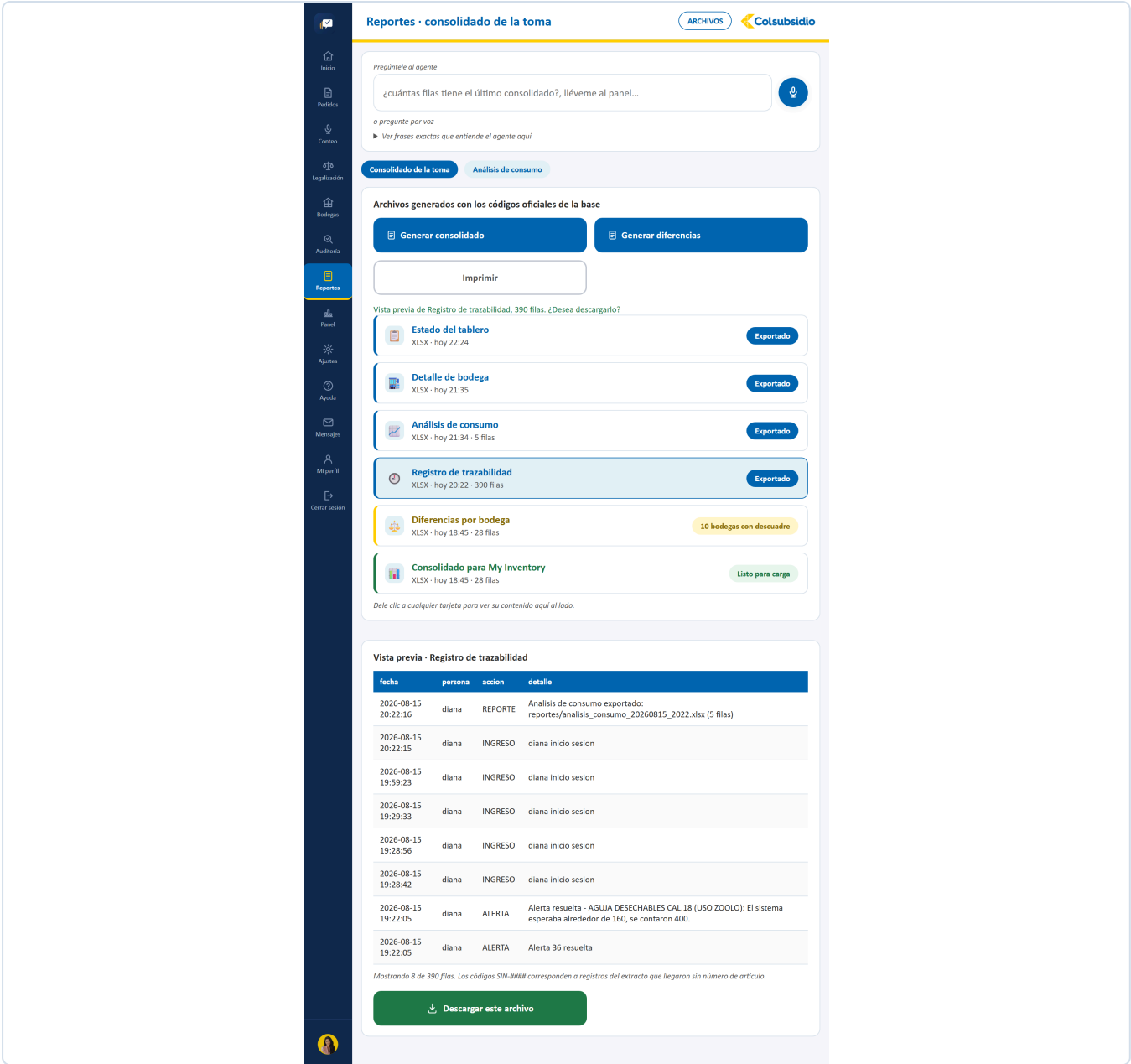


FIGURA 45 El estado de cada bodega.





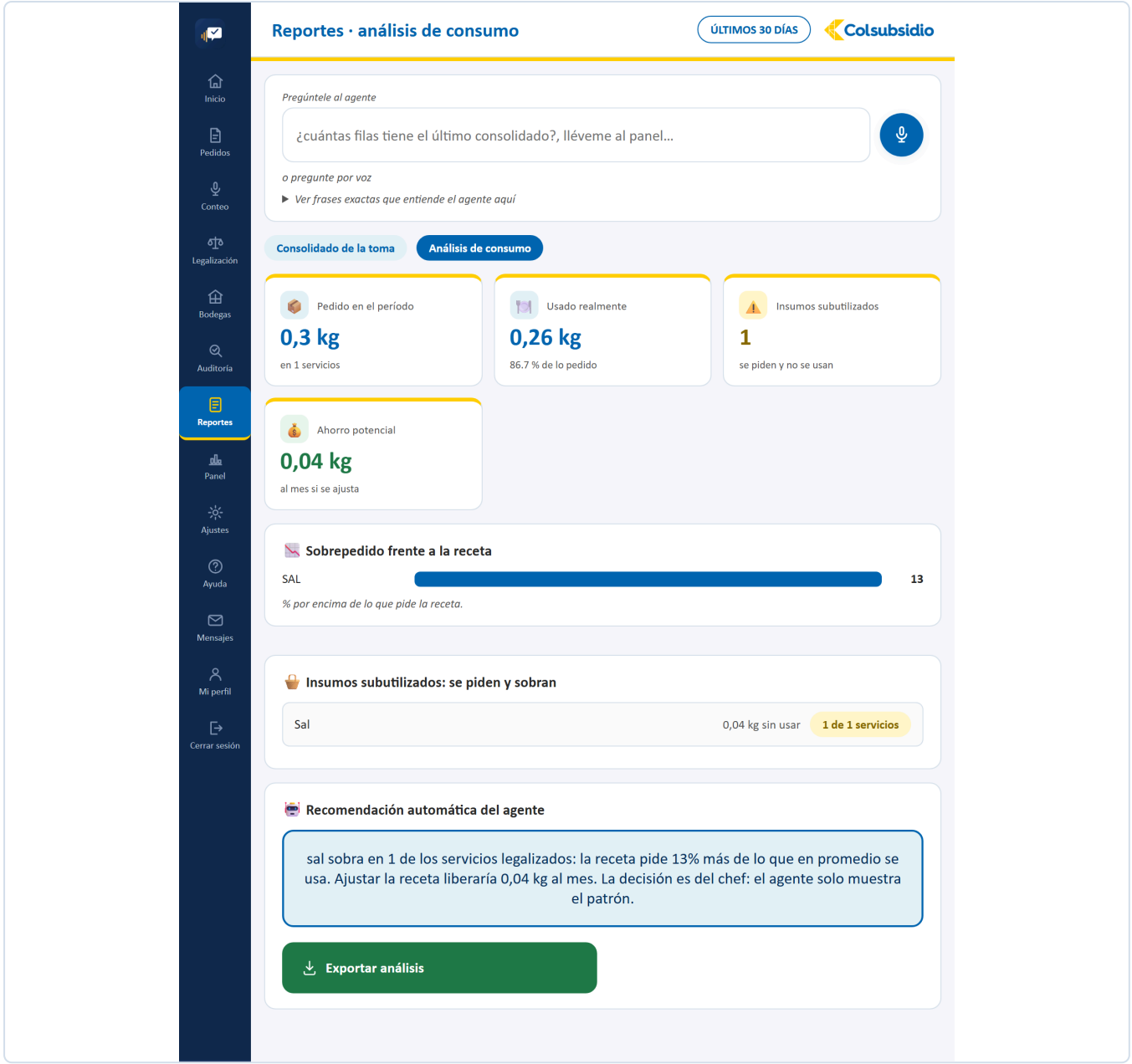


FIGURA 48 El sobrepedido frente a la receta y la recomendación del agente.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Reportes · consolidado de la toma

ARCHIVOS

Colsubsidio

Pregúntele al agente

¿cuántas filas tiene el último consolidado?, lléveme al panel...

o pregunte por voz

▶ Ver frases exactas que entiende el agente aquí

Consolidado de la toma

Análisis de consumo

Archivos generados con los códigos oficiales de la base

Generar consolidado

Generar diferencias

Imprimir

Vista previa de Análisis de consumo, 5 filas. ¿Desea descargarlo?

Estado del tablero

XLSX · hoy 22:24

Exportado

Detalle de bodega

XLSX · hoy 21:35

Exportado

Análisis de consumo

XLSX · hoy 21:34 · 5 filas

Exportado

Registro de trazabilidad

XLSX · hoy 20:22 · 390 filas

Exportado

Diferencias por bodega

XLSX · hoy 18:45 · 28 filas

10 bodegas con descuadre

Consolidado para My Inventory

XLSX · hoy 18:45 · 28 filas

Listo para carga

Dele clic a cualquier tarjeta para ver su contenido aquí al lado.

Vista previa · Análisis de consumo

nombre	sobra	veces	sobrepedido_pct
POLLO ENTERO	1.25	1	10
PAPA CRIOLLA	1	1	10
ARROZ	0.4	1	10
SAL	0.025	1	10
ACEITE	0.05	1	10

Mostrando 5 de 5 filas. Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de artículo.

Descargar este archivo

FIGURA 49 El análisis de consumo como archivo.

12.9 Panel

MENÚ 8 · ADMINISTRADOR

ARCHIVO	Panel.jsx · 396 líneas
PESTAÑAS	Resumen ejecutivo · Bodegas y alertas
QUÉ TIENE	Exactitud de primera pasada, referencias contadas, alertas gestionadas y la gráfica de diferencia absoluta por bodega. En la segunda pestaña, los saldos negativos y las alertas por tipo.
QUÉ RESUELVE	Decidir con datos qué zona revisar primero, en vez de revisarlas todas.
CON QUÉ HABLA	/api/panel y /api/reportes/diferencias-por-bodega.

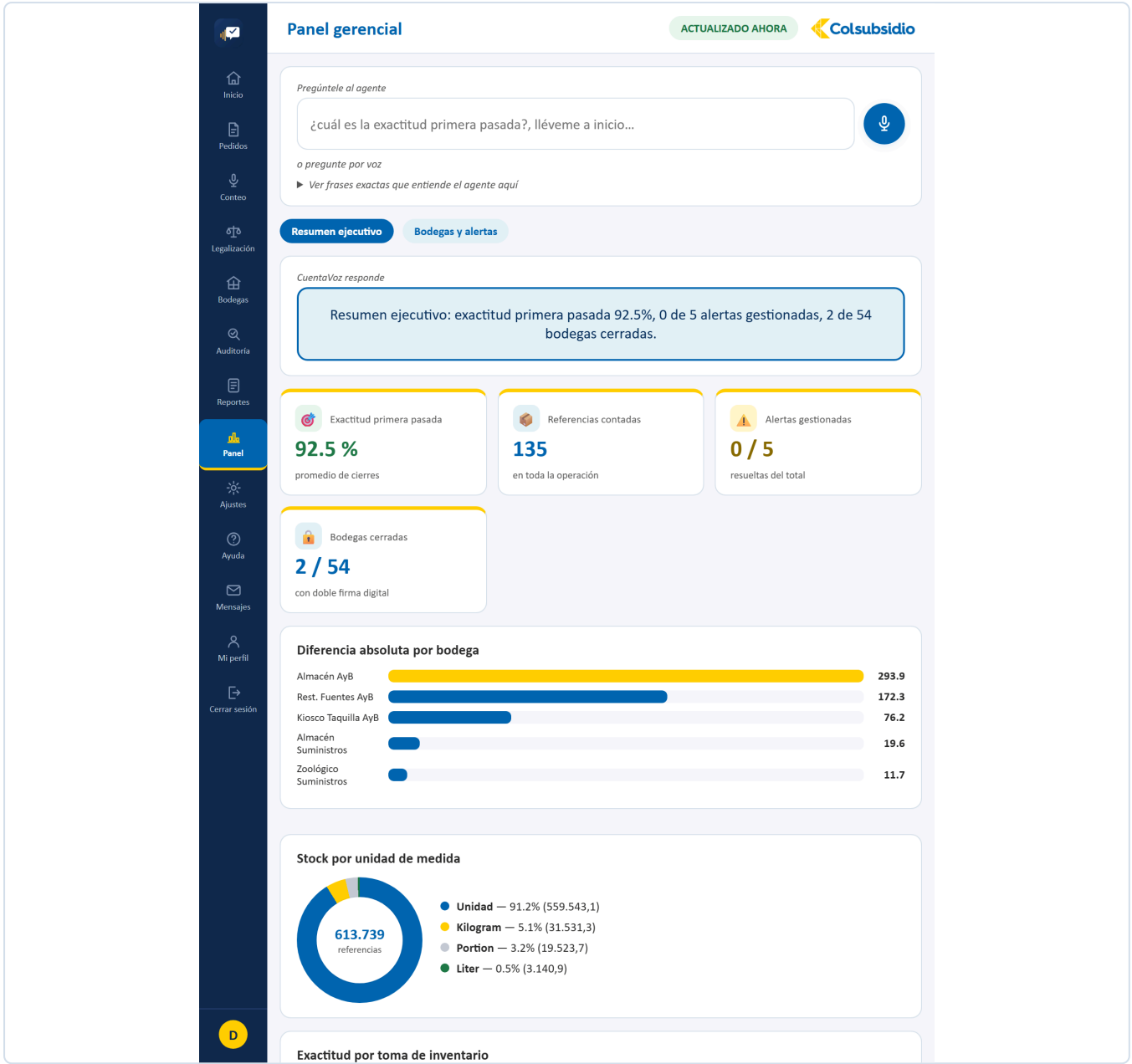
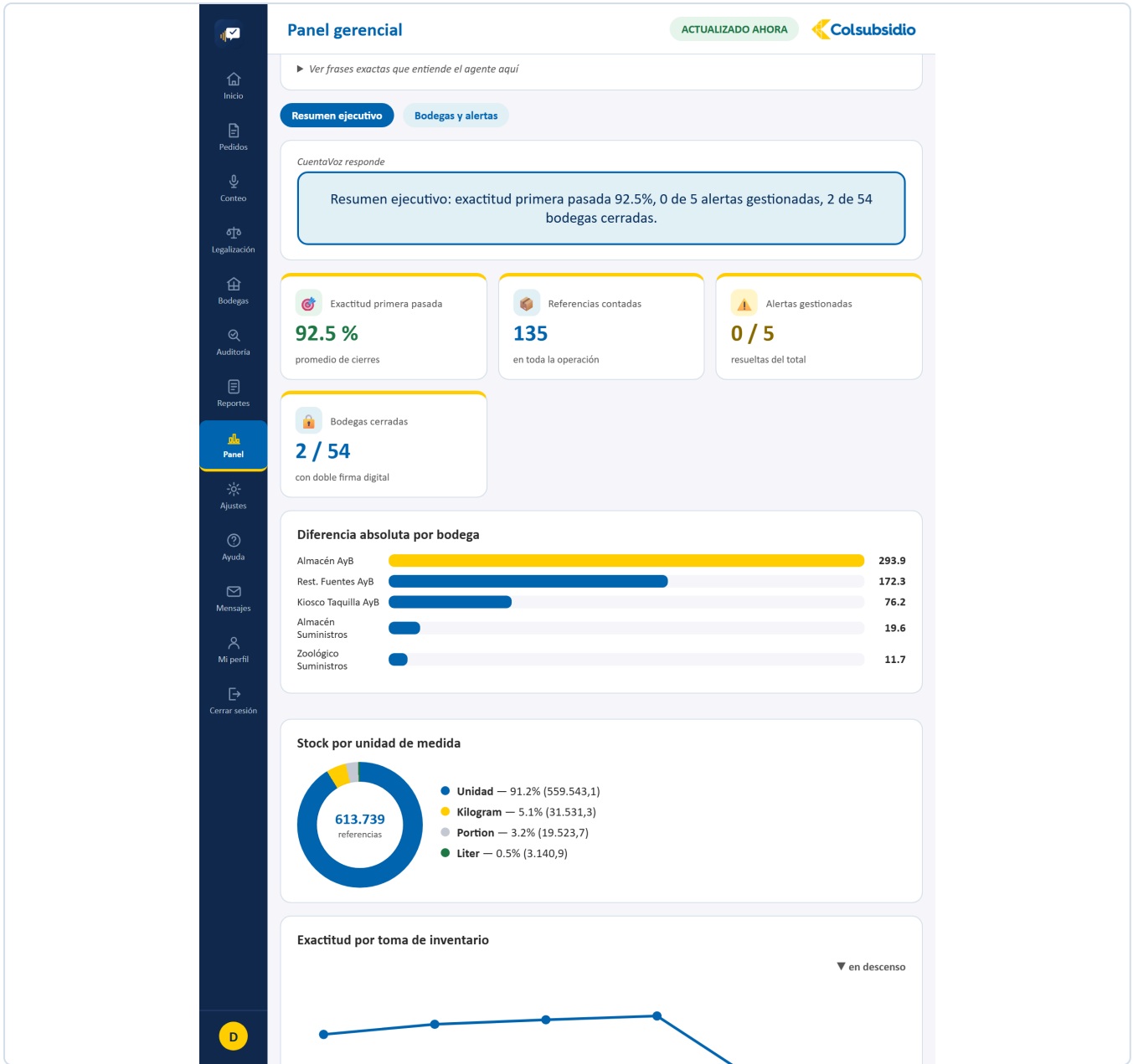


FIGURA 50 Panel · Resumen ejecutivo.



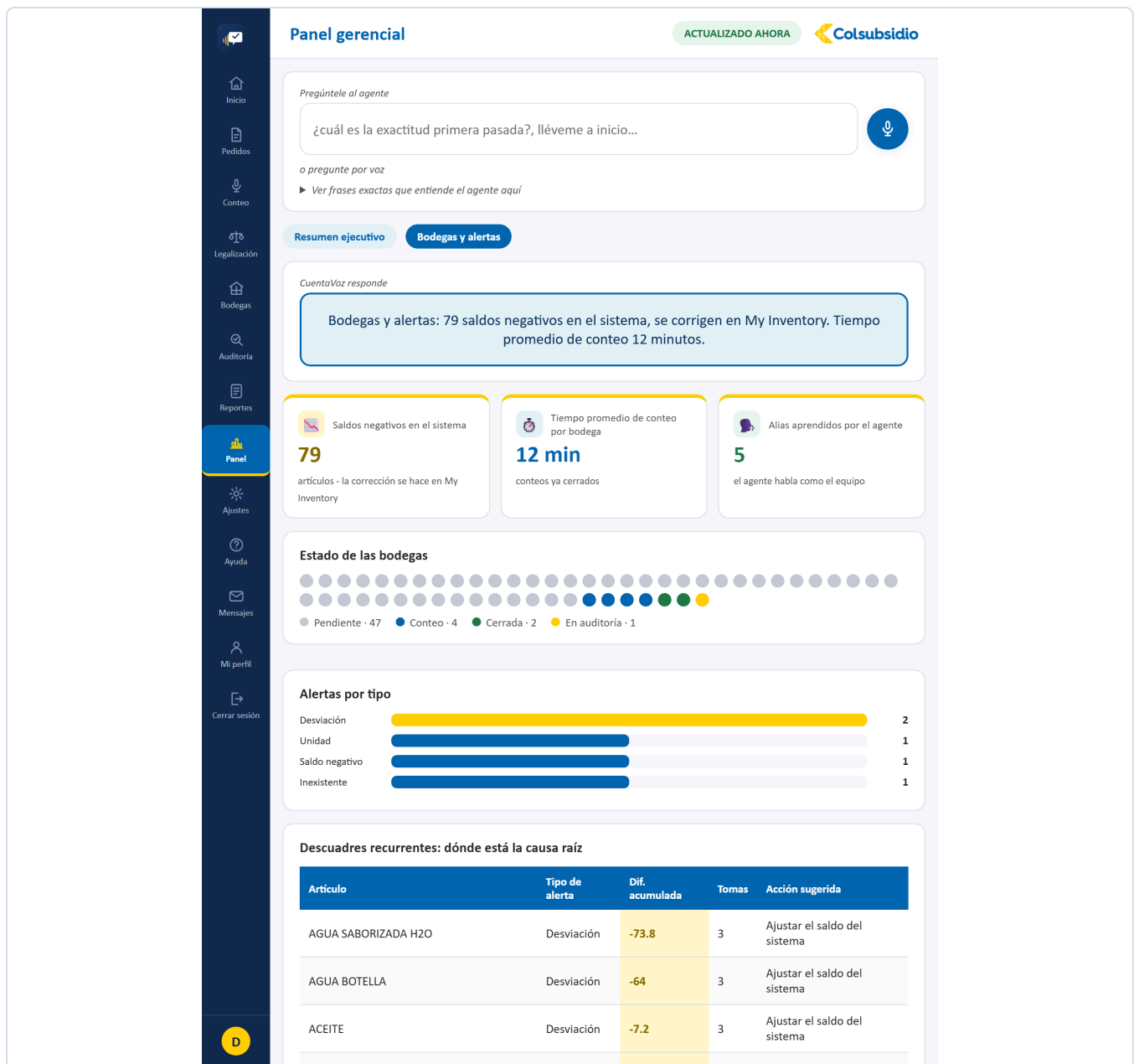


FIGURA 52 Panel · Bodegas y alertas: saldos negativos y alertas por tipo.

12.10 Ajustes MENÚ 9 · ADMINISTRADOR

ARCHIVO	Ajustes.jsx · 1.074 líneas
PESTAÑAS	Configuración · Gestión de usuarios · Recetas · Registro de trazabilidad
QUÉ RESUELVE	Adaptar la plataforma a la operación sin tocar código: el umbral que dispara las alertas, quién existe y qué bodegas le tocan, los platos con sus ingredientes, y el registro de todo lo que pasó.
CON QUÉ HABLA	/api/ajustes, /api/usuarios, /api/recetas y /api/trazabilidad.

POR QUÉ EL AUXILIAR NO VE ESTA PANTALLA

El umbral y el modo sin conexión ya estaban bloqueados en el backend con `requiere_perfil`. Mostrarle la pantalla igual, en solo lectura, le enseñaba controles administrativos que no le sirven y que no puede tocar. Se oculta del todo, como Auditoría, Reportes y Panel.

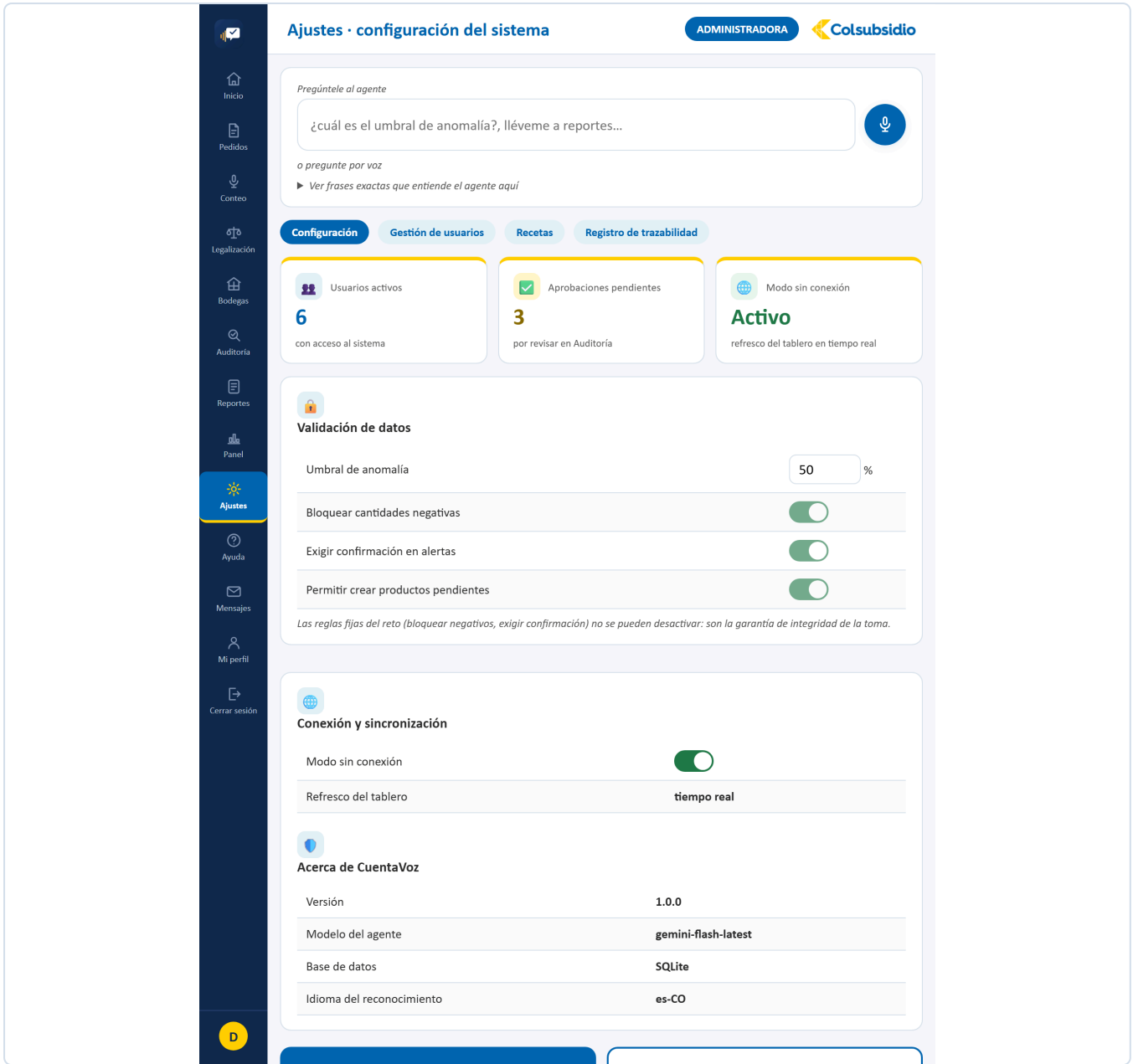


FIGURA 53 Ajustes · Configuración: el umbral de alertas y el modo sin conexión.

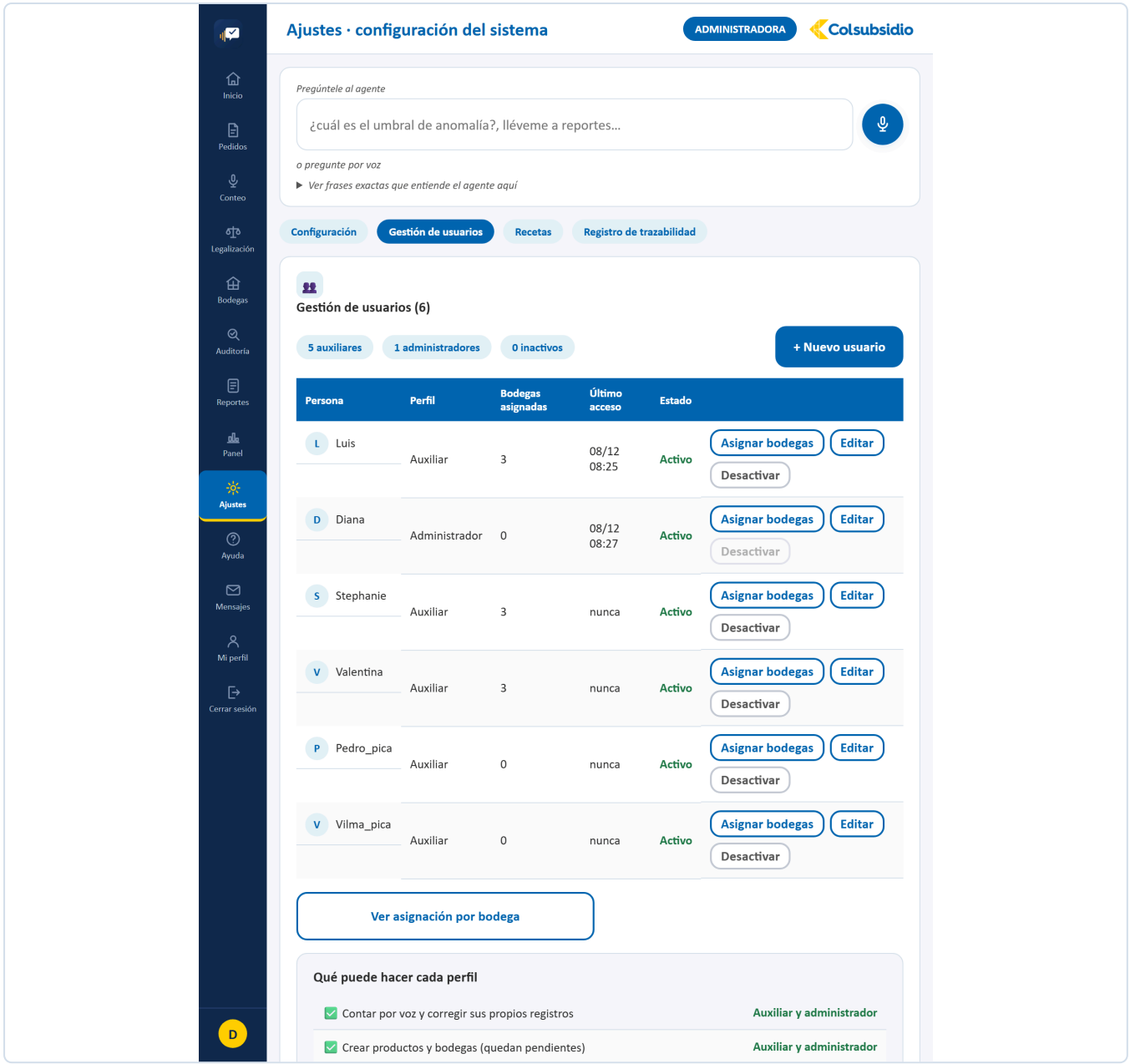


FIGURA 54 Ajustes · Gestión de usuarios.

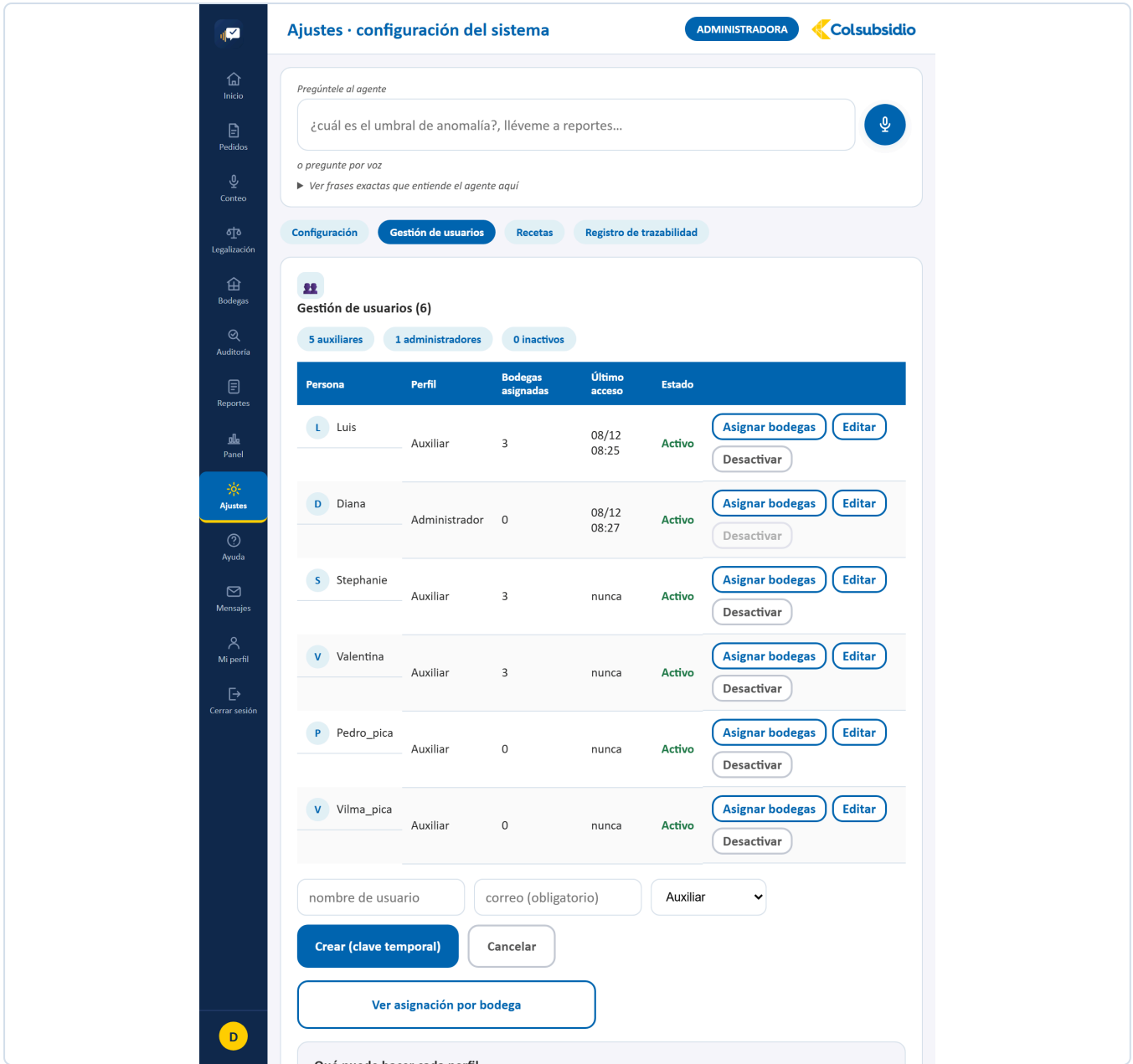


FIGURA 55 Crear una cuenta con su perfil.

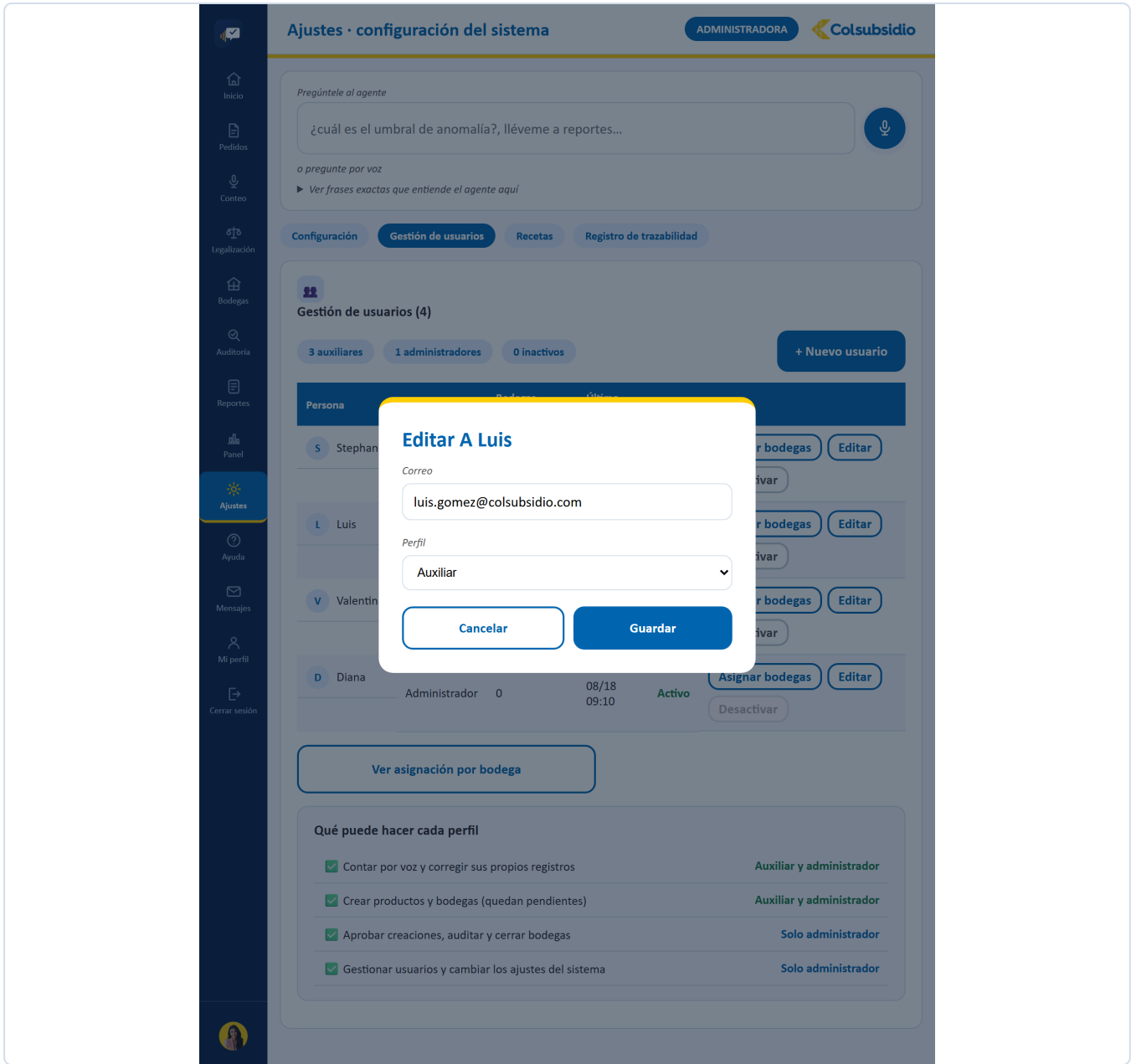


FIGURA 56 Corregir el correo, el perfil o desactivar la cuenta.

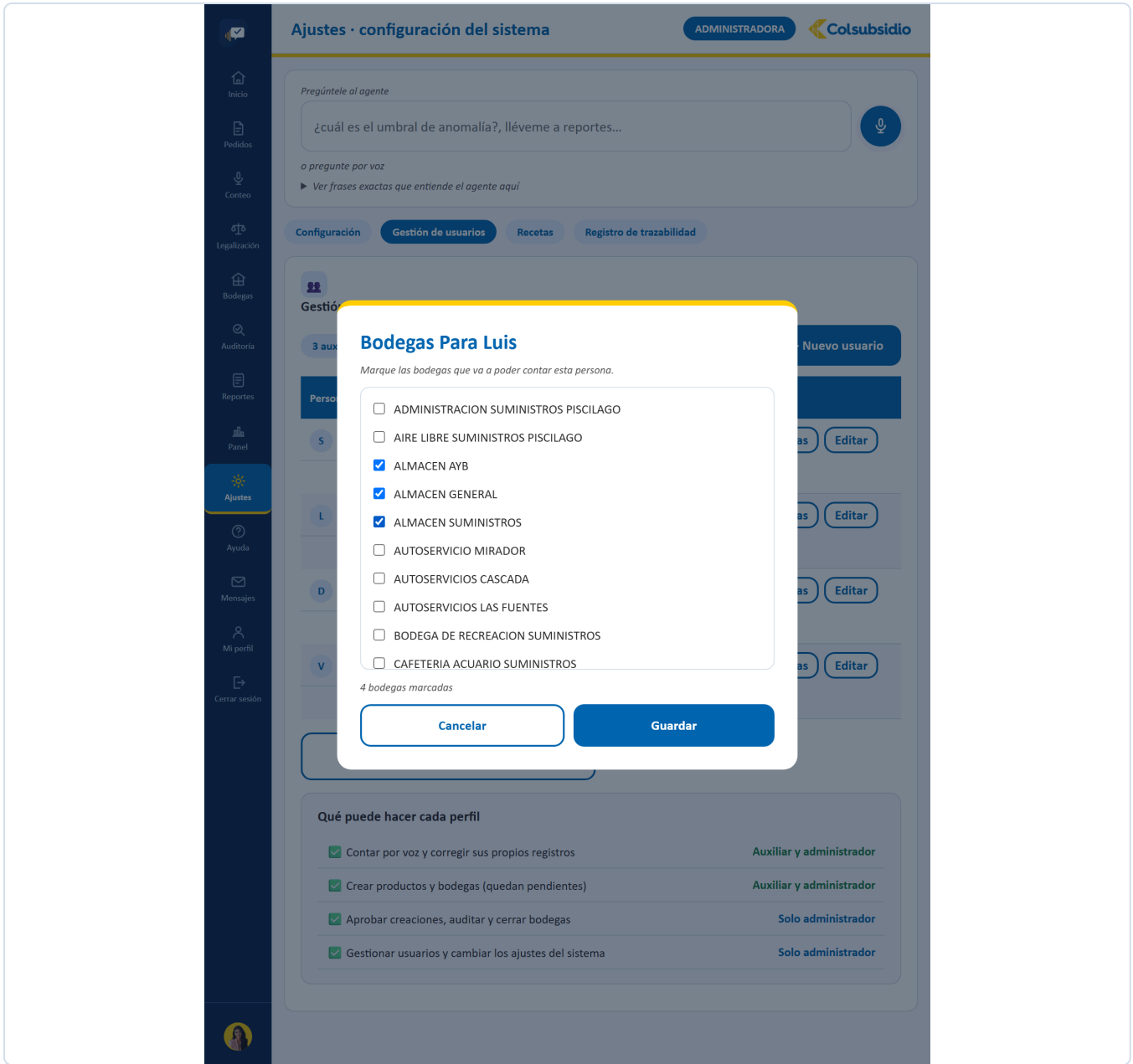


FIGURA 57 Es lo que decide qué puede contar cada auxiliar — también por voz.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Ajustes · configuración del sistema

ADMINISTRADORA

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Configuración

Gestión de usuarios

Recetas

Registro de trazabilidad

Recetas (2)

Las porciones que dicte el chef se calculan sobre estas recetas: cantidad por porción x porciones, menos lo que ya haya en la bodega. Aquí se crean, editan y borran — CuentaVoz sí gestiona el catálogo de recetas y menús.

Receta	Rendimiento	Ingredientes		
AIACO SANTA FERENO	1 porción	5	Editar	Eliminar
SANCOCHO DE GALLINA	1 porción	5	Editar	Eliminar

+ Nueva receta

FIGURA 58 Los platos con sus ingredientes, base del cálculo de pedidos.

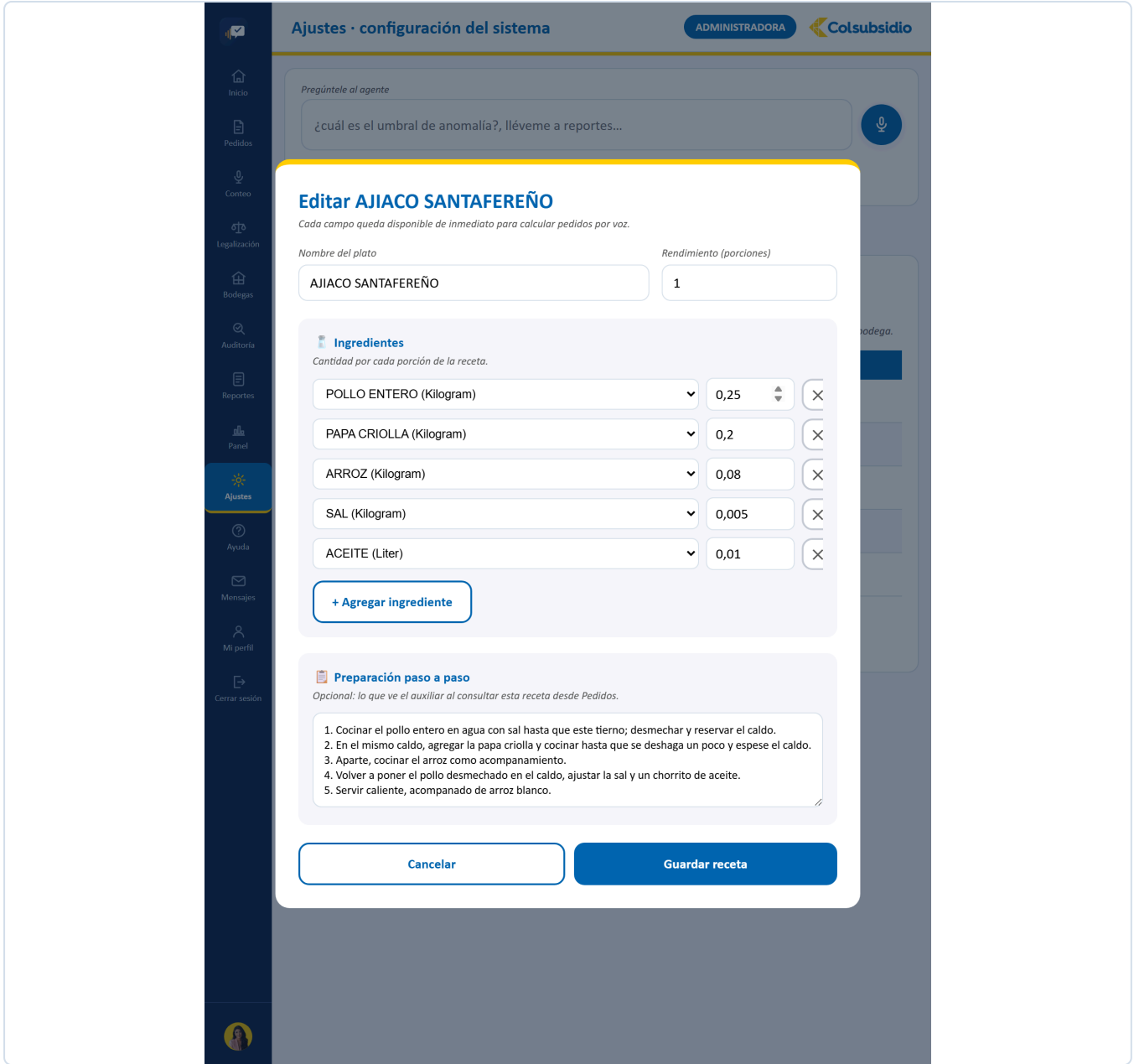
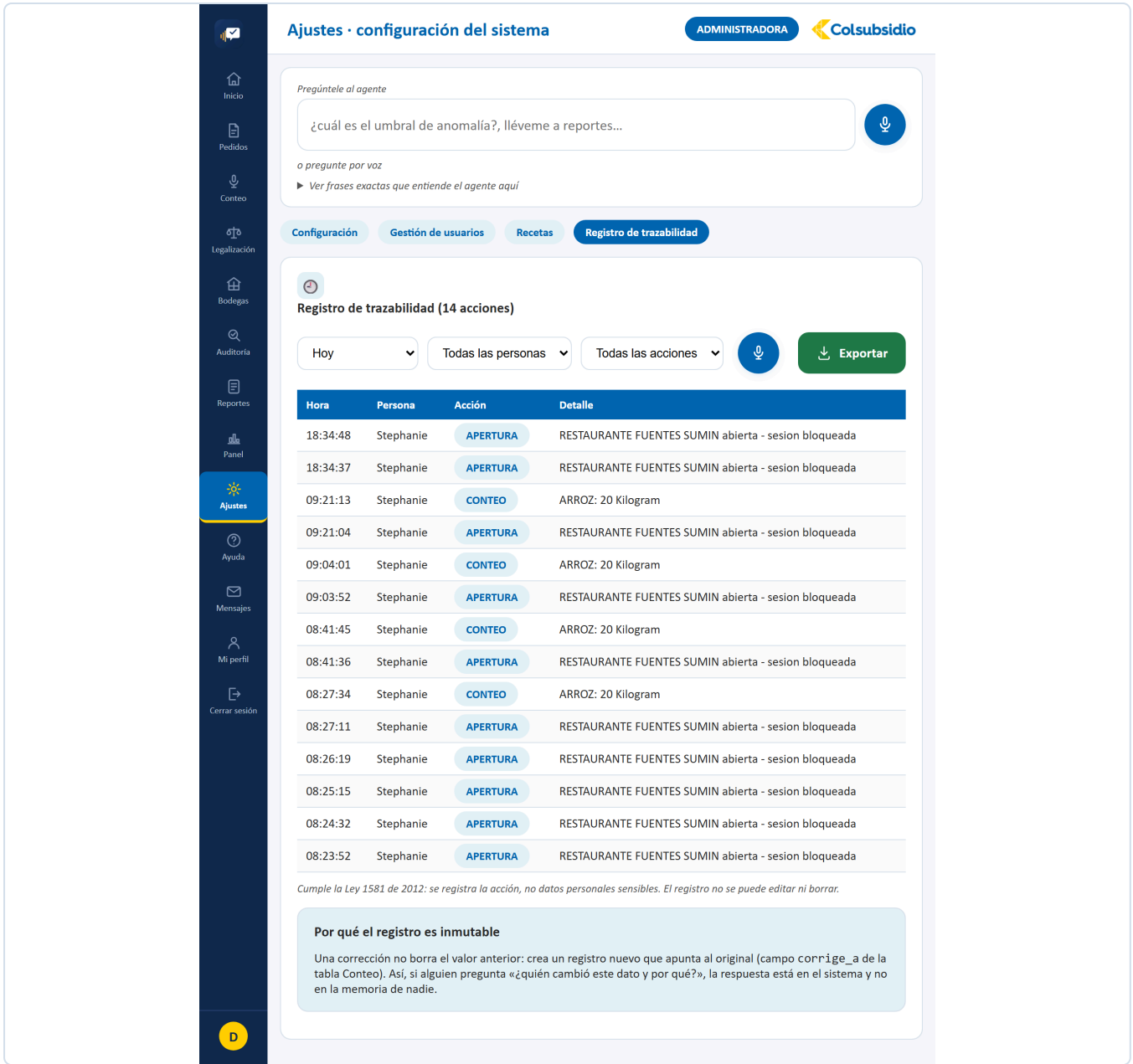


FIGURA 59 Ingredientes, rendimiento y preparación.



ARCHIVO	Ayuda.jsx · 364 líneas
QUÉ TIENE	Preguntas frecuentes filtrables, el cuadro para preguntarle al agente por voz o por escrito, la guía de frases que entiende, el estado del sistema en vivo, y tres botones: Escribirle al administrador , Reportar un problema y Ver mensajes . Arriba a la derecha, Ver el recorrido en video .
QUÉ RESUELVE	Que una duda no obligue a salir de la aplicación ni a llamar a nadie.
CON QUÉ HABLA	<code>/api/salud</code> , <code>/api/agente/asistente</code> , <code>/api/soporte/reportar</code> , <code>/api/soporte/mensaje-administrador</code> y <code>/api/soporte/administrador</code> .

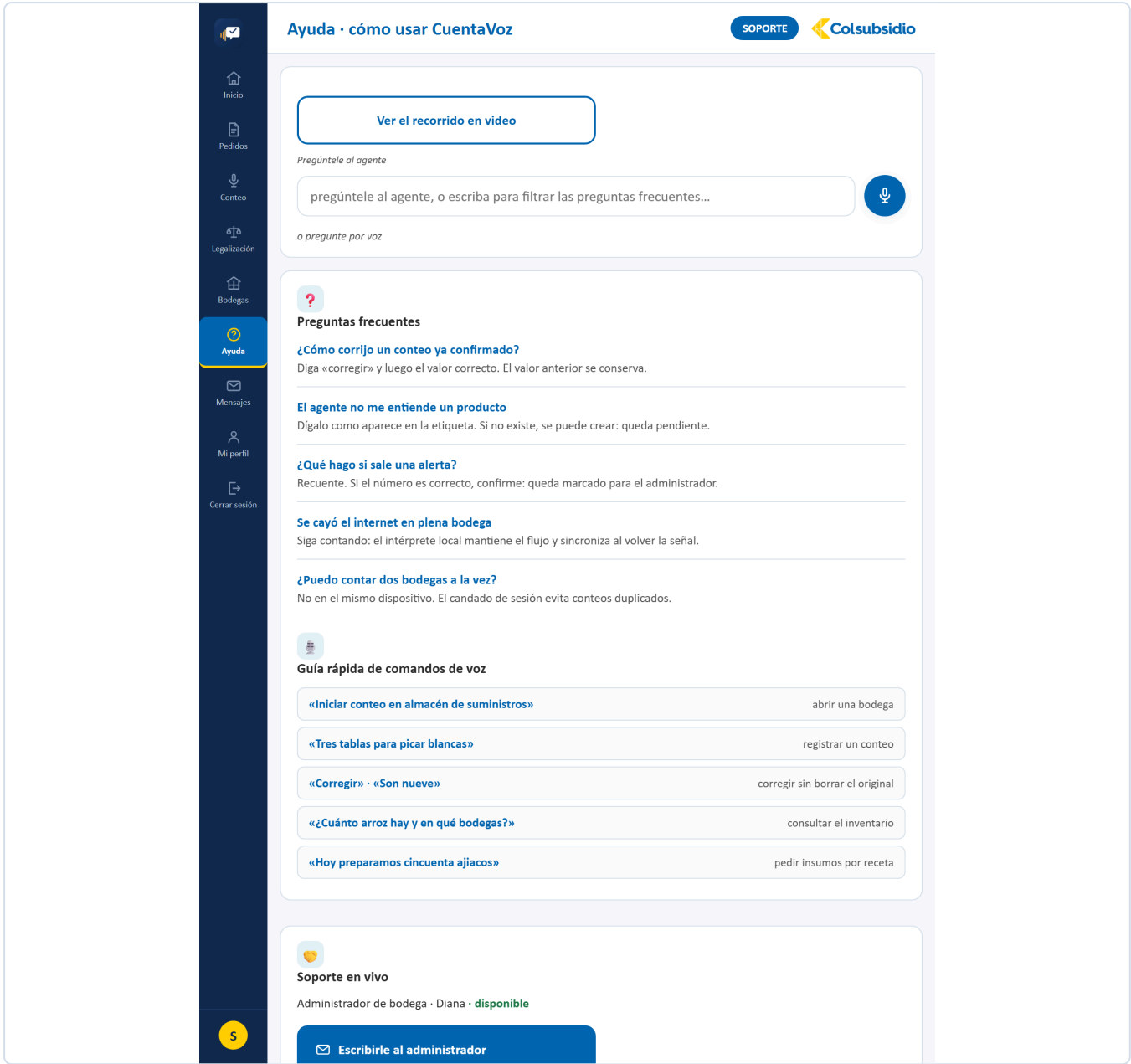


FIGURA 61 Ayuda. Preguntas frecuentes, el agente y los botones de soporte.

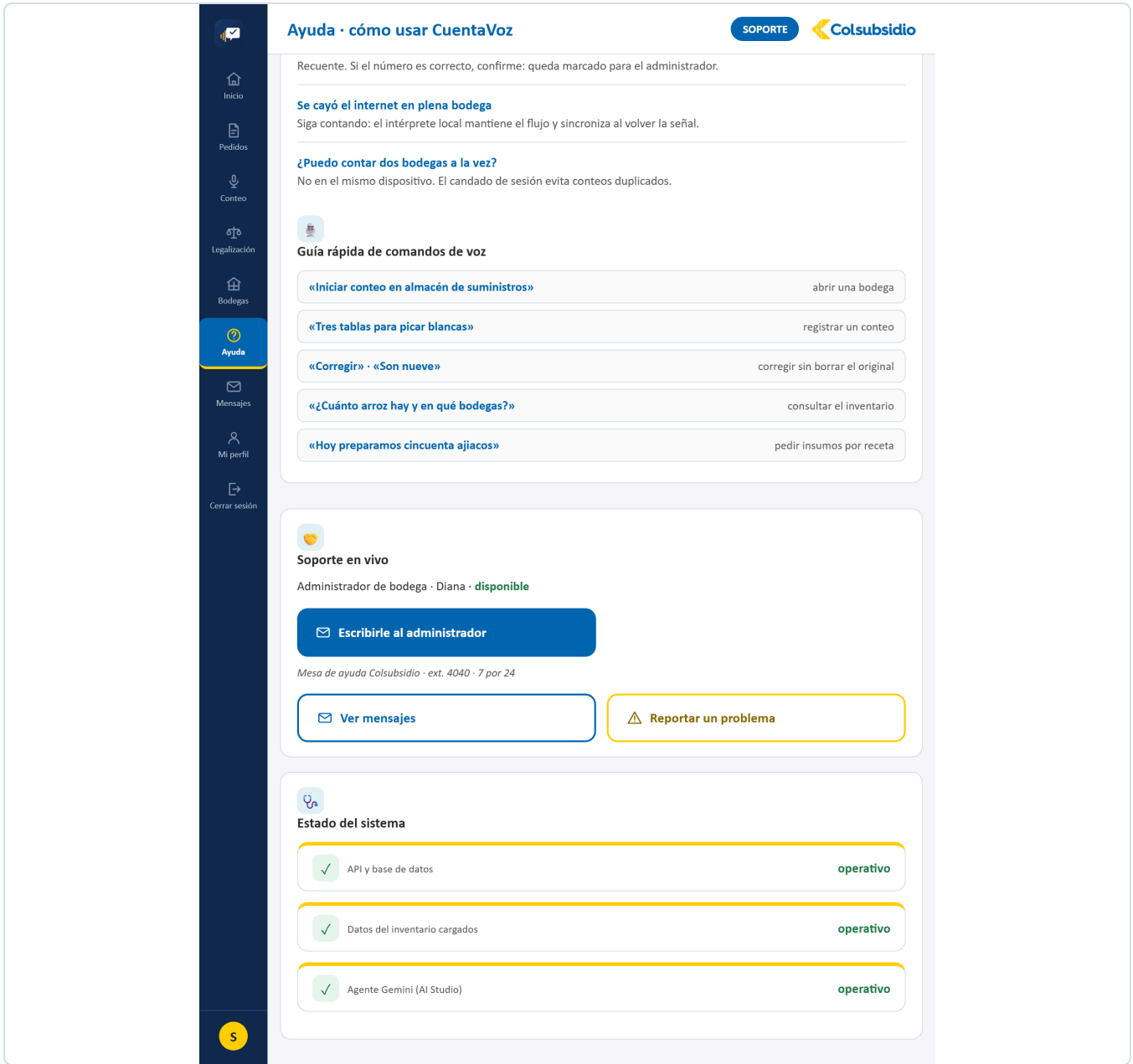


FIGURA 62 API, datos de inventario y agente, en vivo.

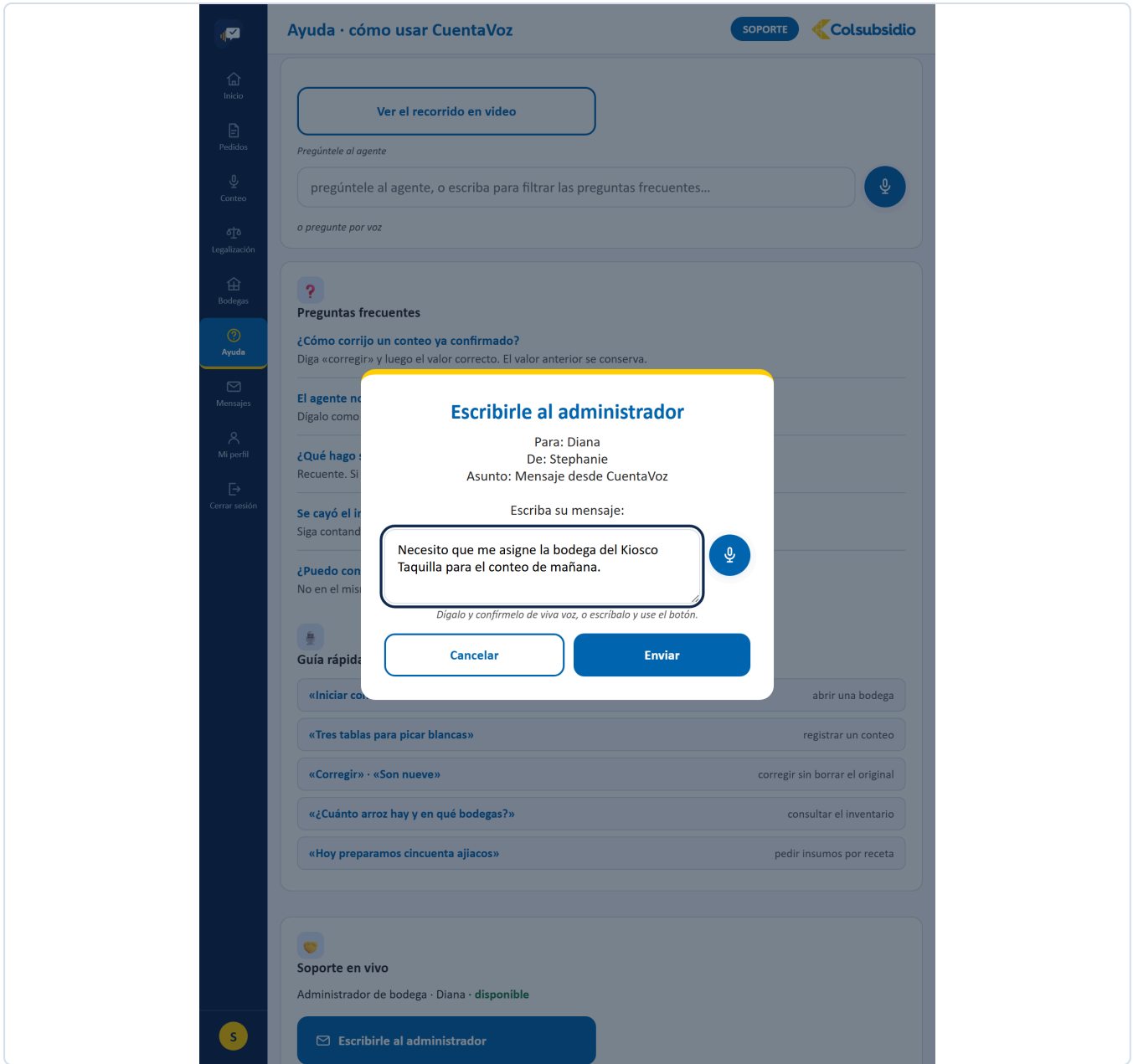


FIGURA 63 Para lo que es de la operación.

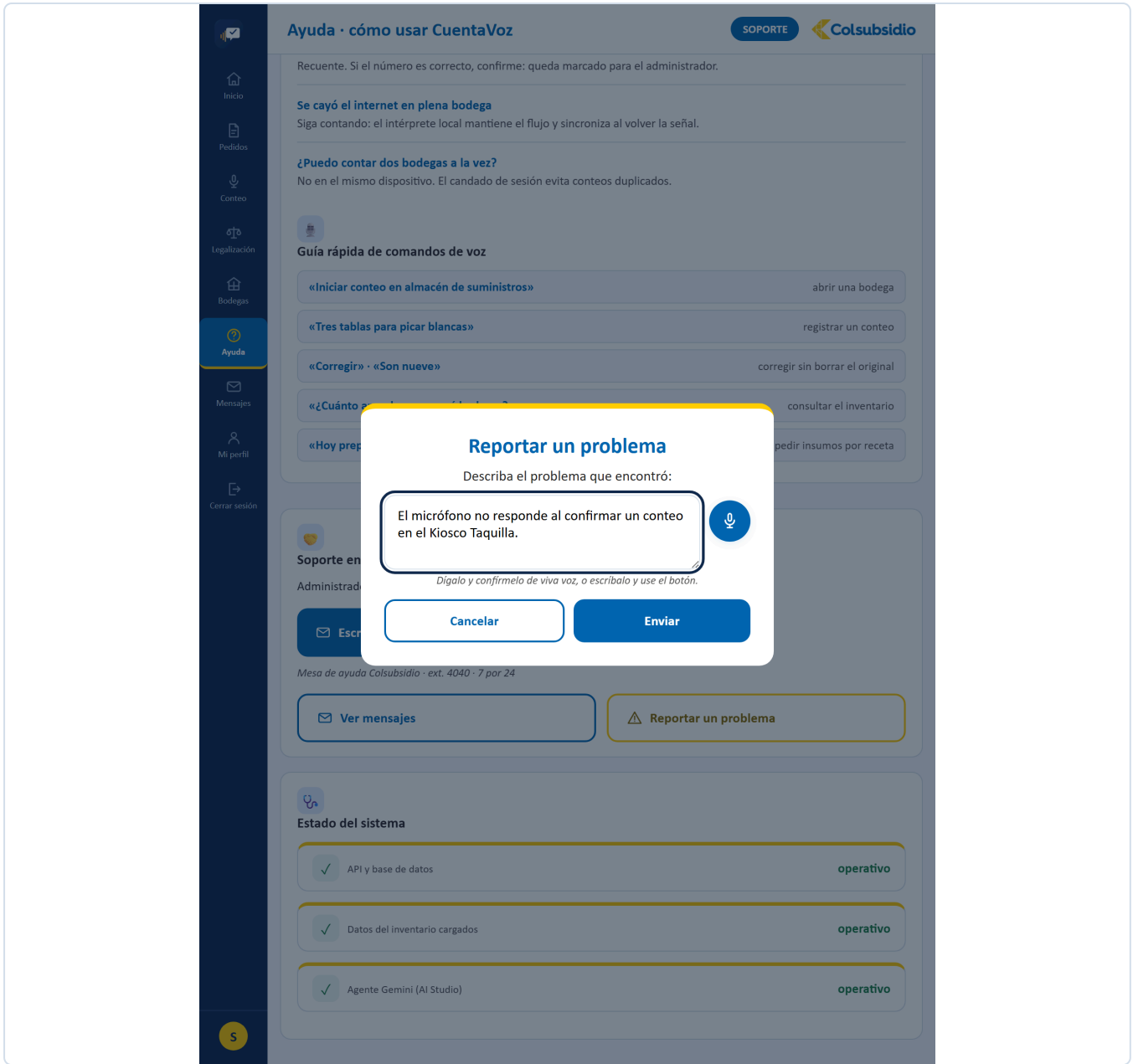


FIGURA 64 Para cuando lo que falla es la aplicación.

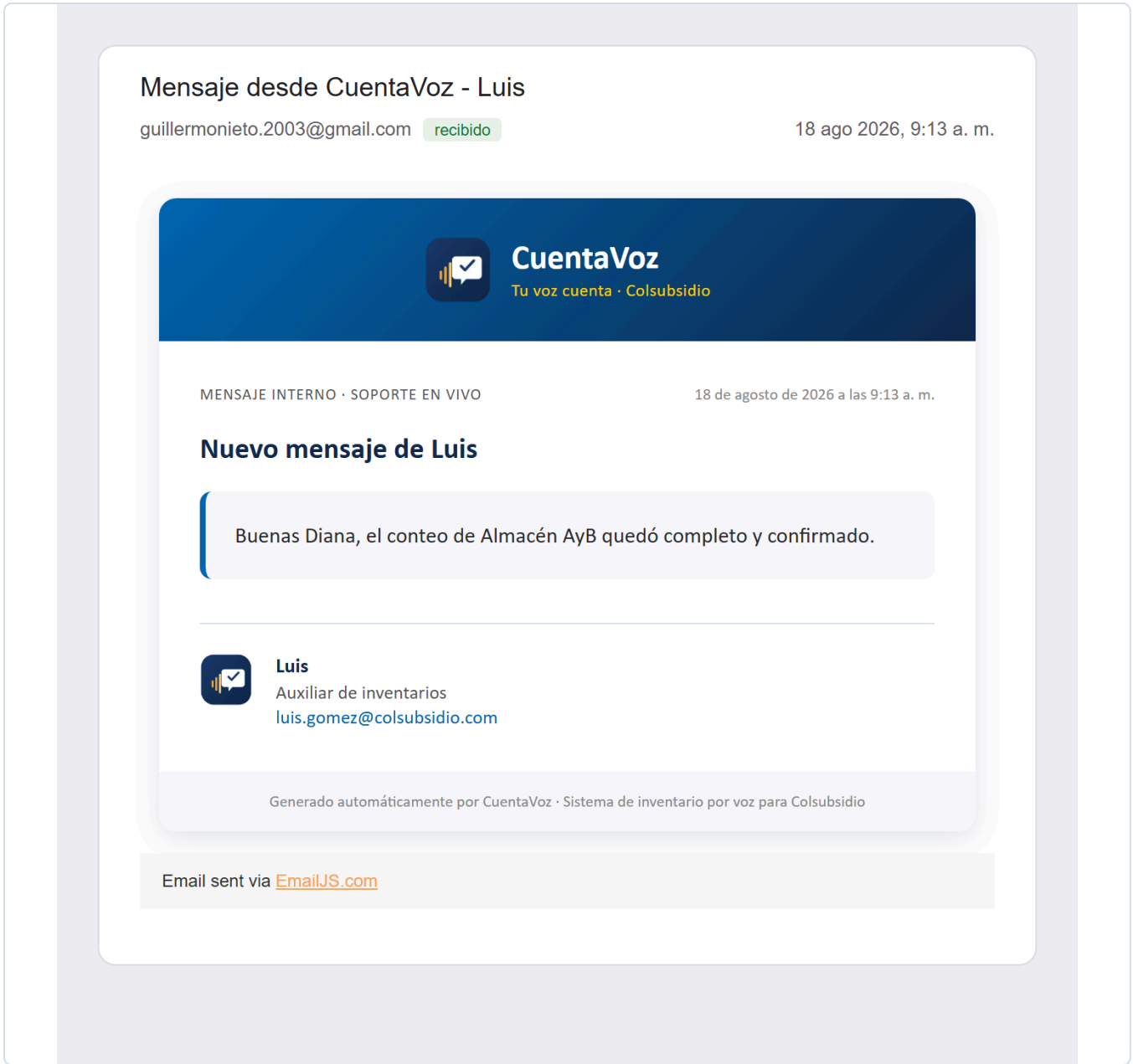


FIGURA 65 Cuando el envío por correo está configurado, esto es lo que recibe el administrador.

12.12 Mensajes

MENÚ 11

ARCHIVO	Mensajes.jsx · 248 líneas
QUÉ TIENE	Tres contadores que cambian de significado según el perfil (enviados/recibidos, esperando/por responder, respondidos/resueltos) y la lista de mensajes. Para el administrador, el botón Responder y, si hay uno solo pendiente, un cuadro de responder por voz.
QUÉ RESUELVE	Comunicación de la operación sin salir a un correo que nadie revisa en bodega.
CON QUÉ HABLA	/api/soporte/mensajes y /api/soporte/mensajes/{id}/responder.

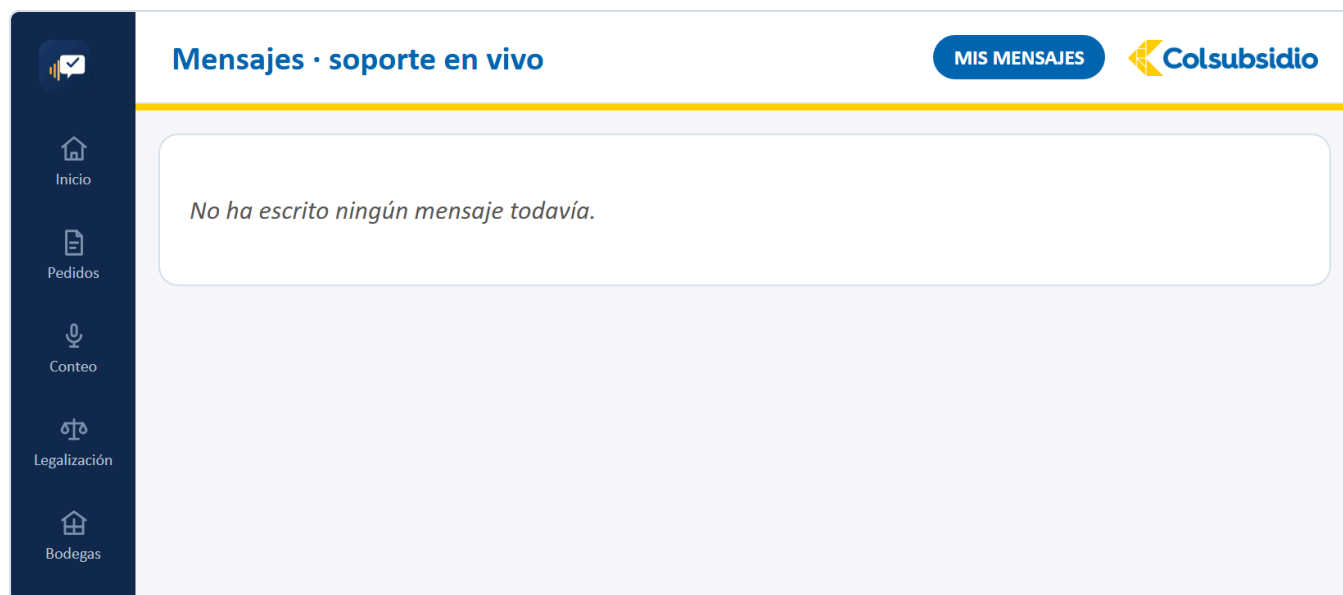


FIGURA 66 Mensajes. Lo que escribió y si ya le respondieron.

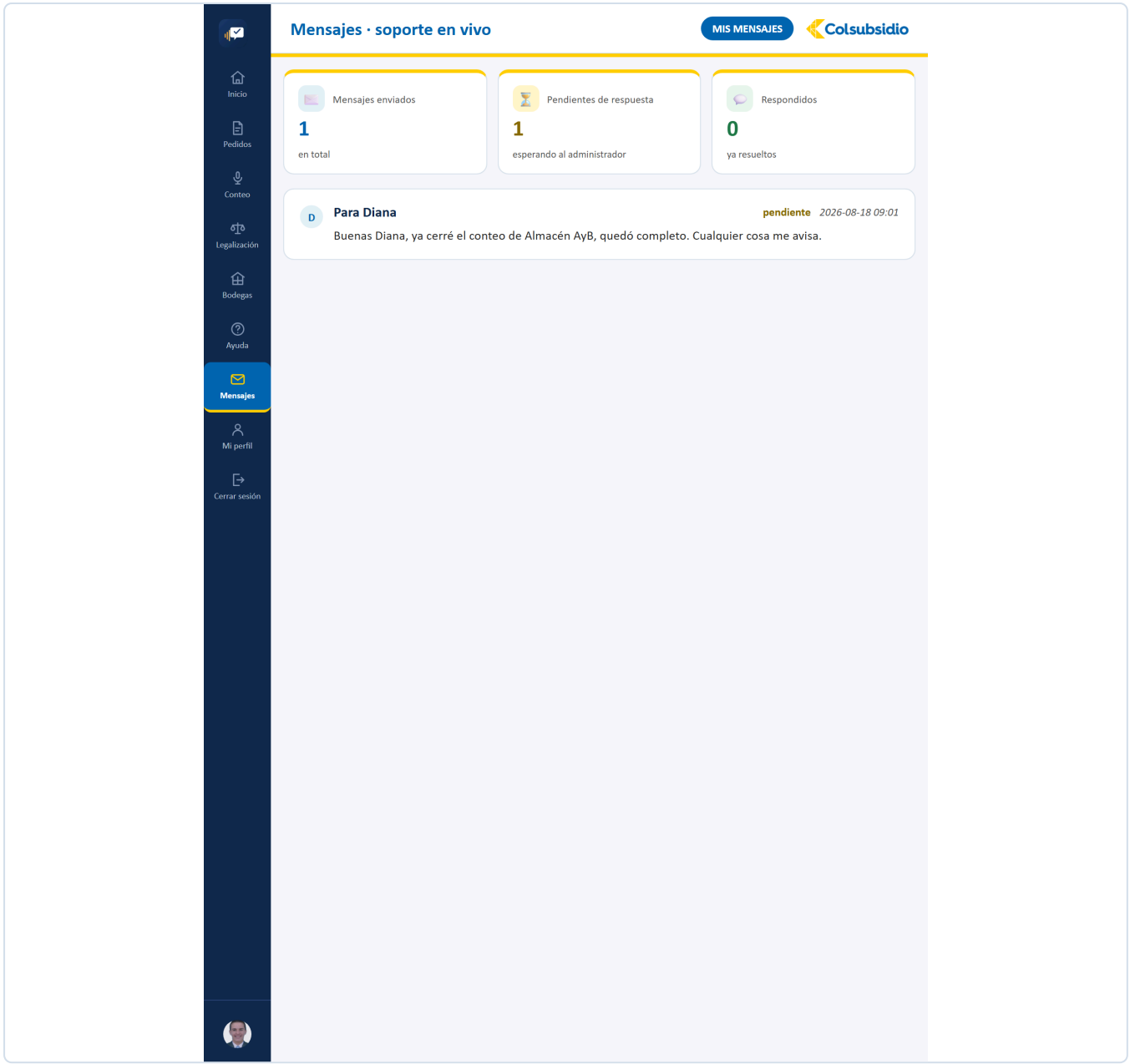
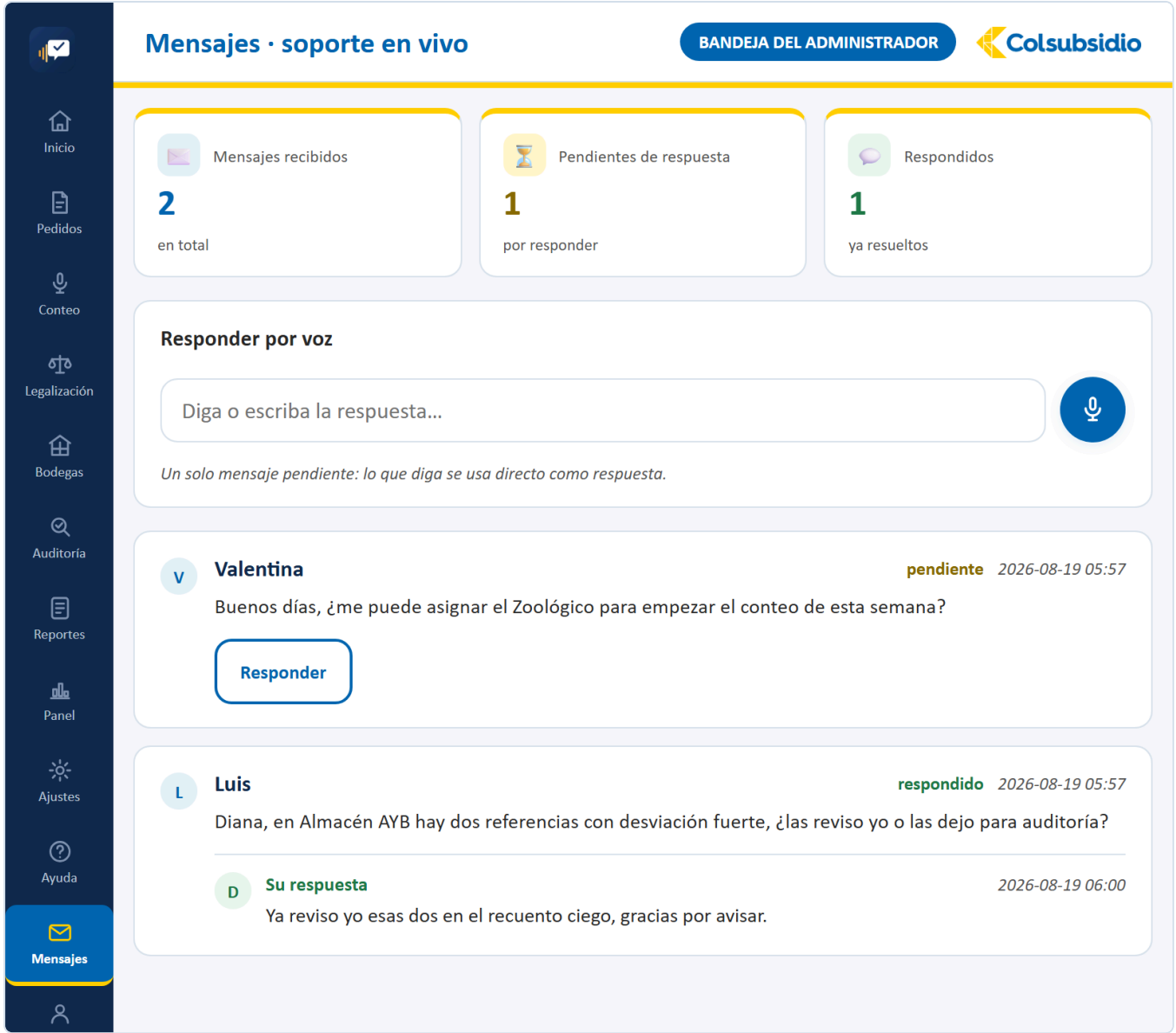


FIGURA 67 La bandeja después de escribirle al administrador.



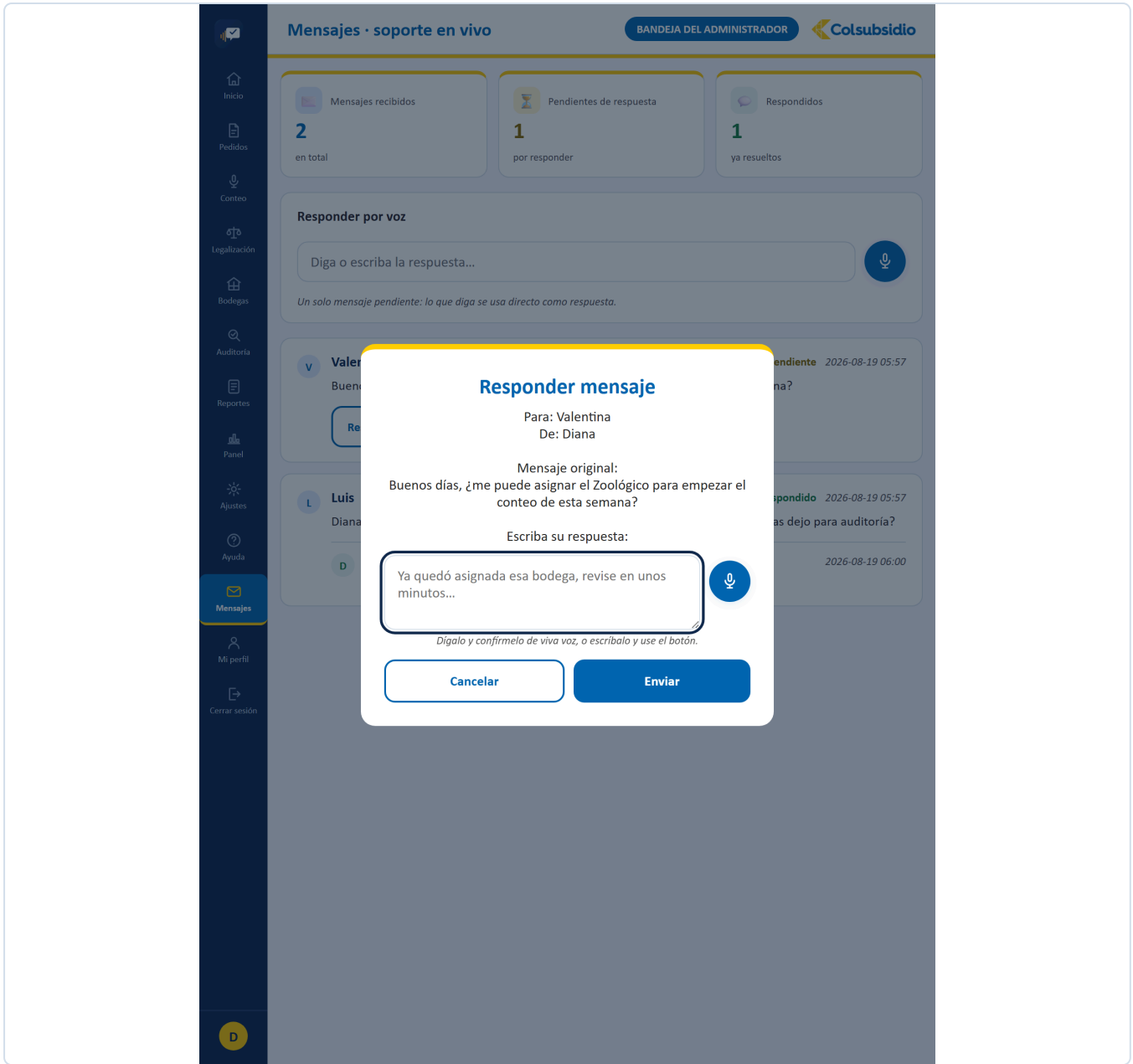


FIGURA 69 Con el mensaje original a la vista, para no responder de memoria.

12.13 Mi perfil MENÚ 12

ARCHIVO	MiPerfil.jsx · 429 líneas
QUÉ TIENE	Datos personales y foto, cambio de clave, cerrar sesión en todos los dispositivos, verificación en dos pasos, preferencias de voz y el bloque de accesibilidad (alto contraste y tamaño de letra).
QUÉ RESUELVE	Que cada quien administre su propia cuenta sin pedirle nada a un administrador.
CON QUÉ HABLA	/api/usuarios/yo y sus derivados; el cambio de clave y el segundo factor van directo a Cognito.

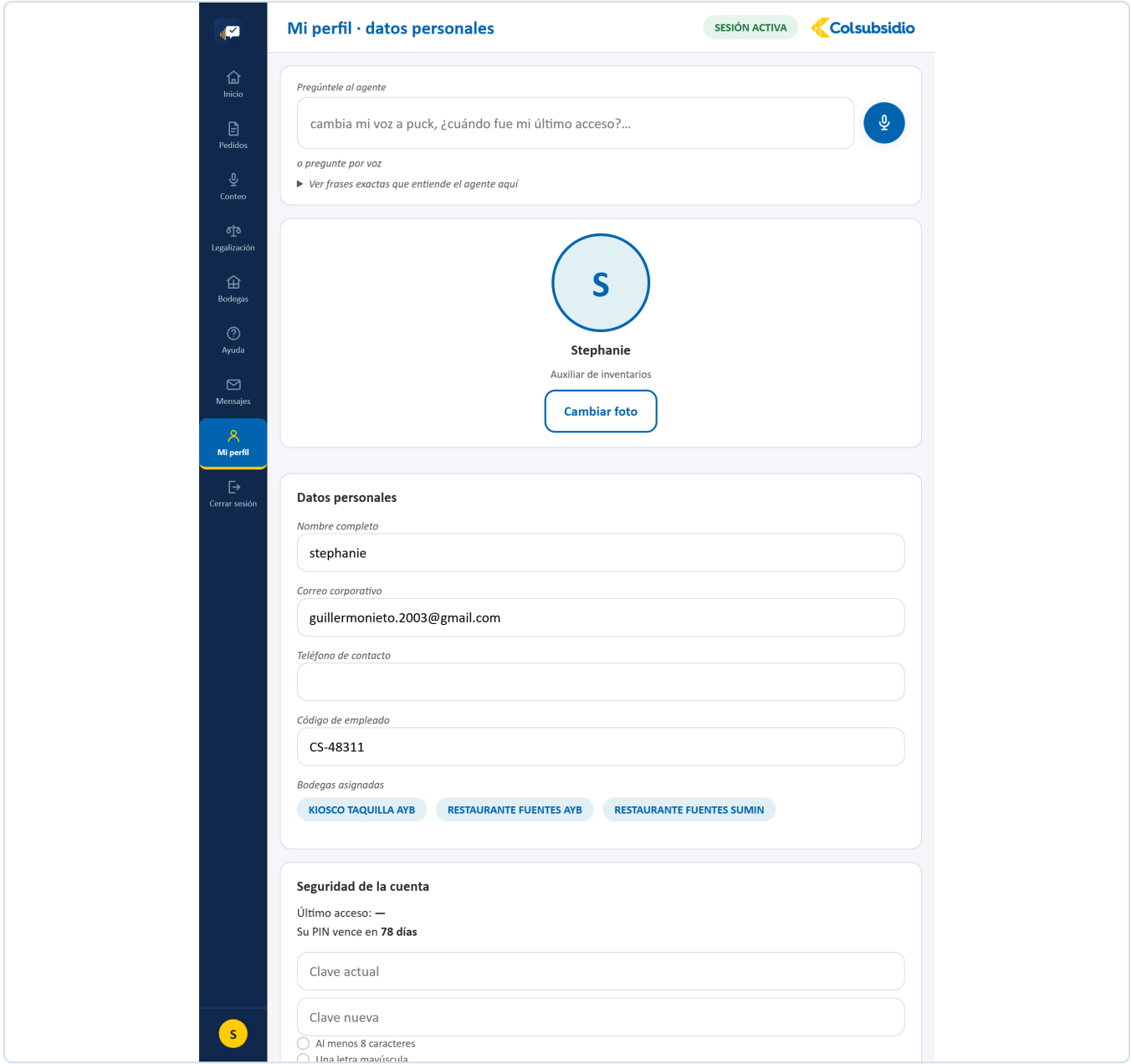


FIGURA 70 Mi perfil. Datos personales, seguridad y preferencias de voz.

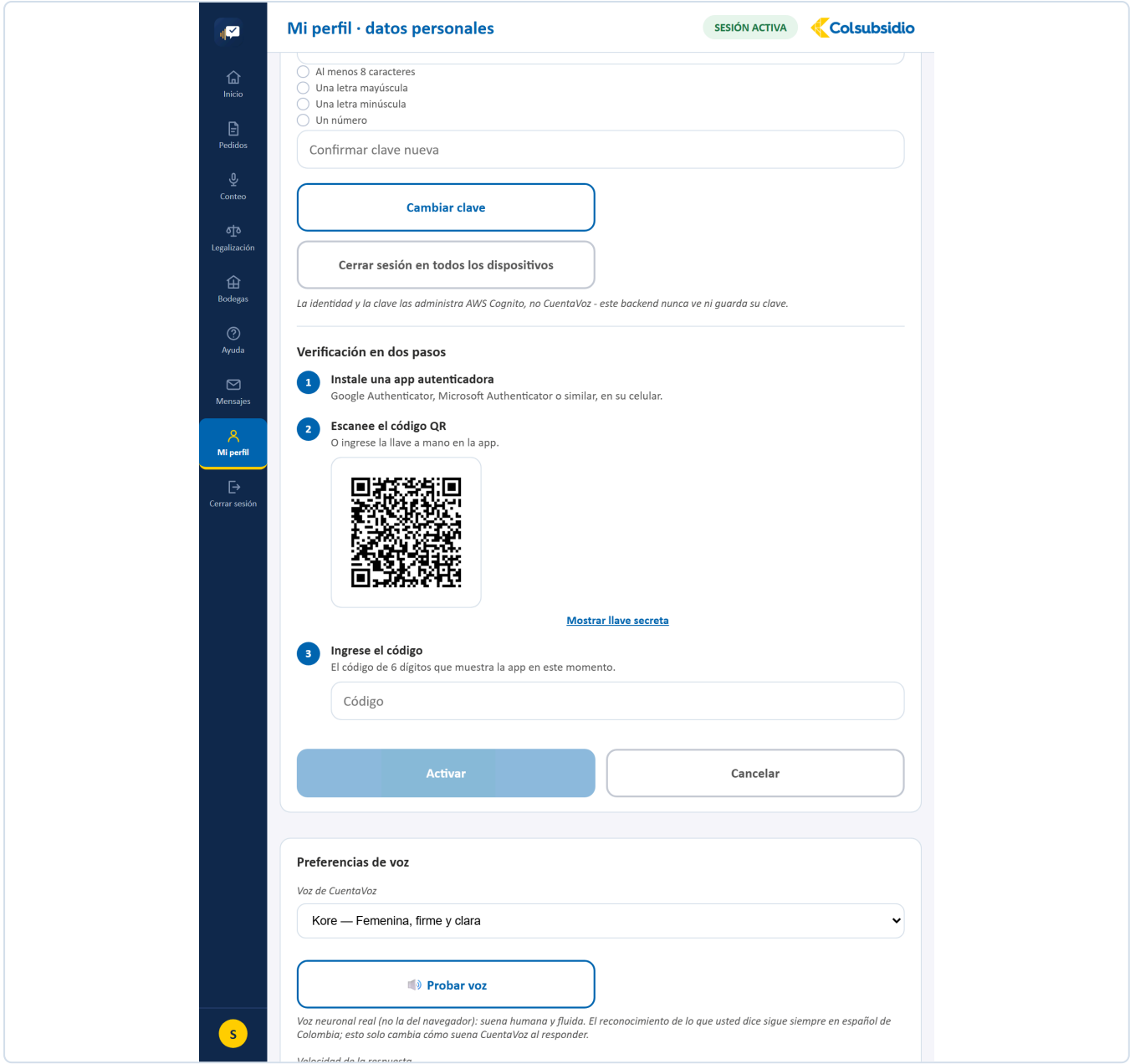


FIGURA 71 El QR del segundo factor, o la llave secreta si el celular no puede escanear.

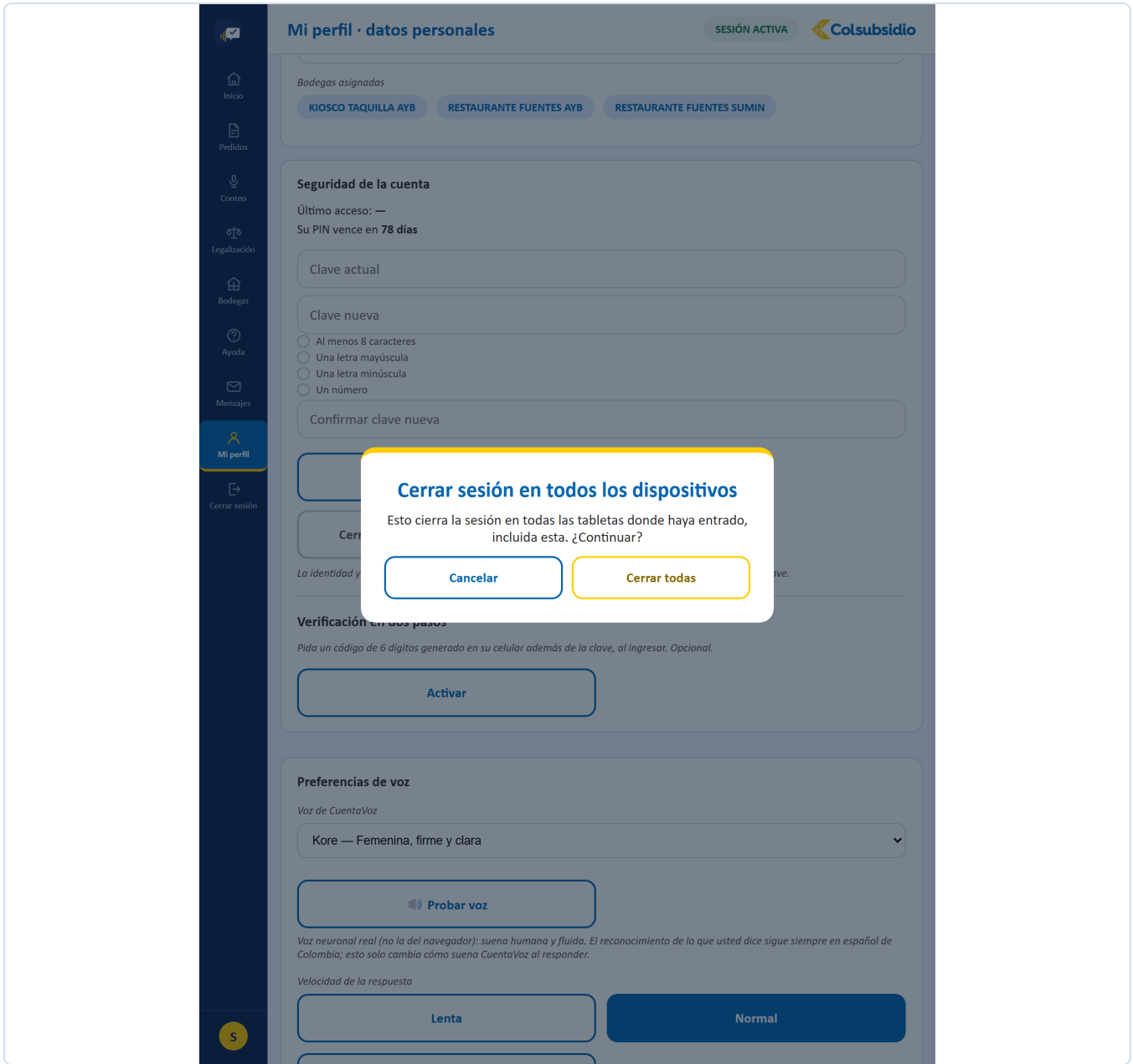


FIGURA 72 Revoca en Cognito, no solo en este navegador.

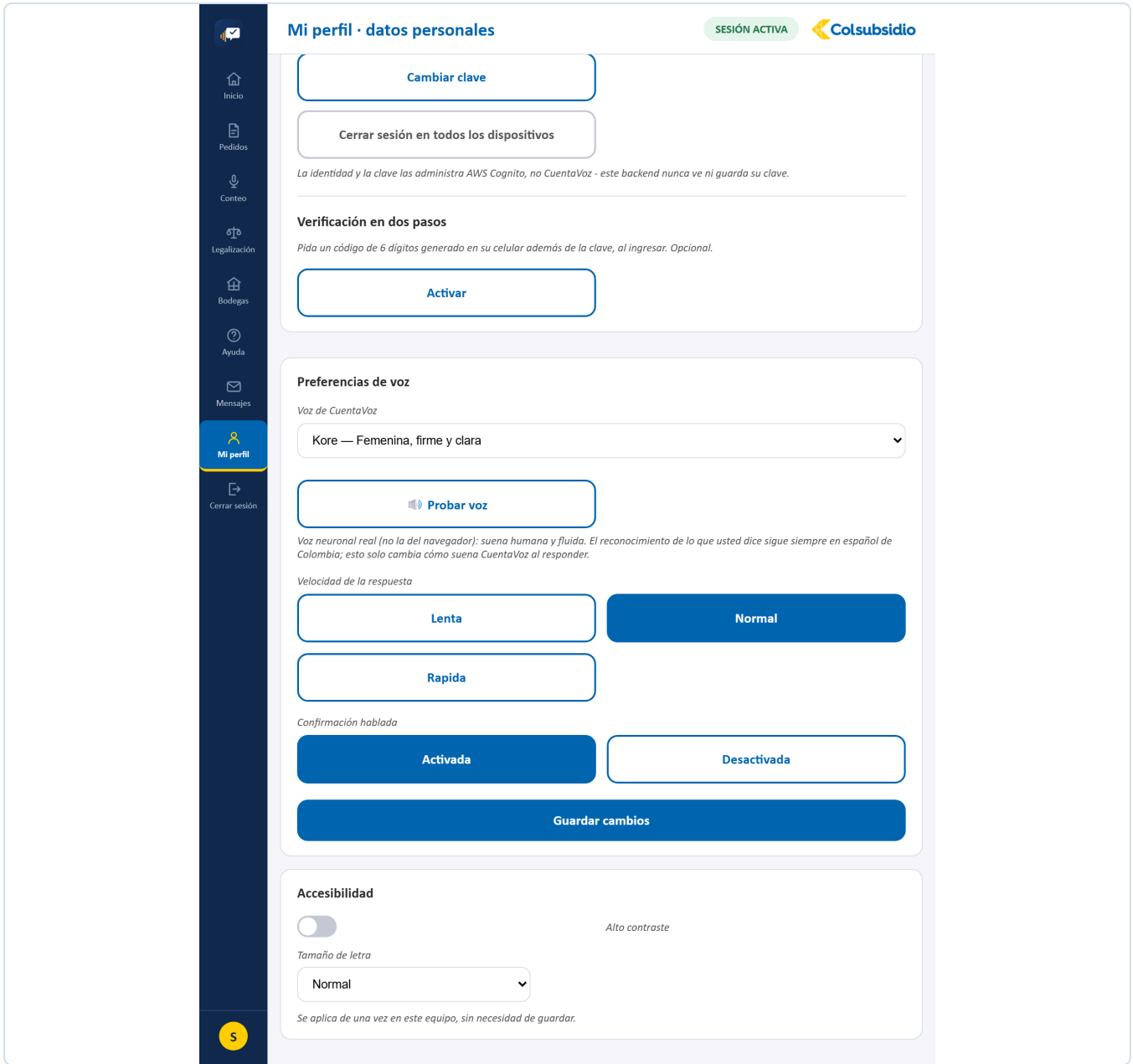


FIGURA 73 Alto contraste y tamaño de letra, para bodegas con poca luz.

12.14 Cerrar sesión MENÚ 13

ARCHIVO	CerrarSesion.jsx
QUÉ TIENE	Un diálogo que primero consulta si hay trabajo abierto y lo dice con la bodega y cuántas referencias lleva, antes de ofrecer salir.
QUÉ RESUELVE	Que nadie se vaya con la duda de si perdió lo contado. Y que la sesión se cierre también en el servidor, no solo en ese navegador.
CON QUÉ HABLA	<code>/api/sesiones/{id}/avance</code> para el aviso y <code>/api/usuarios/yo/cerrar-todas</code> para revocar en Cognito.

SI NO SE PUEDE COMPROBAR, NO SE DICE QUE TODO ESTÁ BIEN

Cuando la consulta de trabajo pendiente falla (sin conexión, sesión vencida), el diálogo **no** asume que no hay nada abierto: avisa que no se pudo comprobar. «Puede salir sin problema» es la respuesta más tranquilizadora posible justo cuando menos se sabe, y por eso es la que no se da.

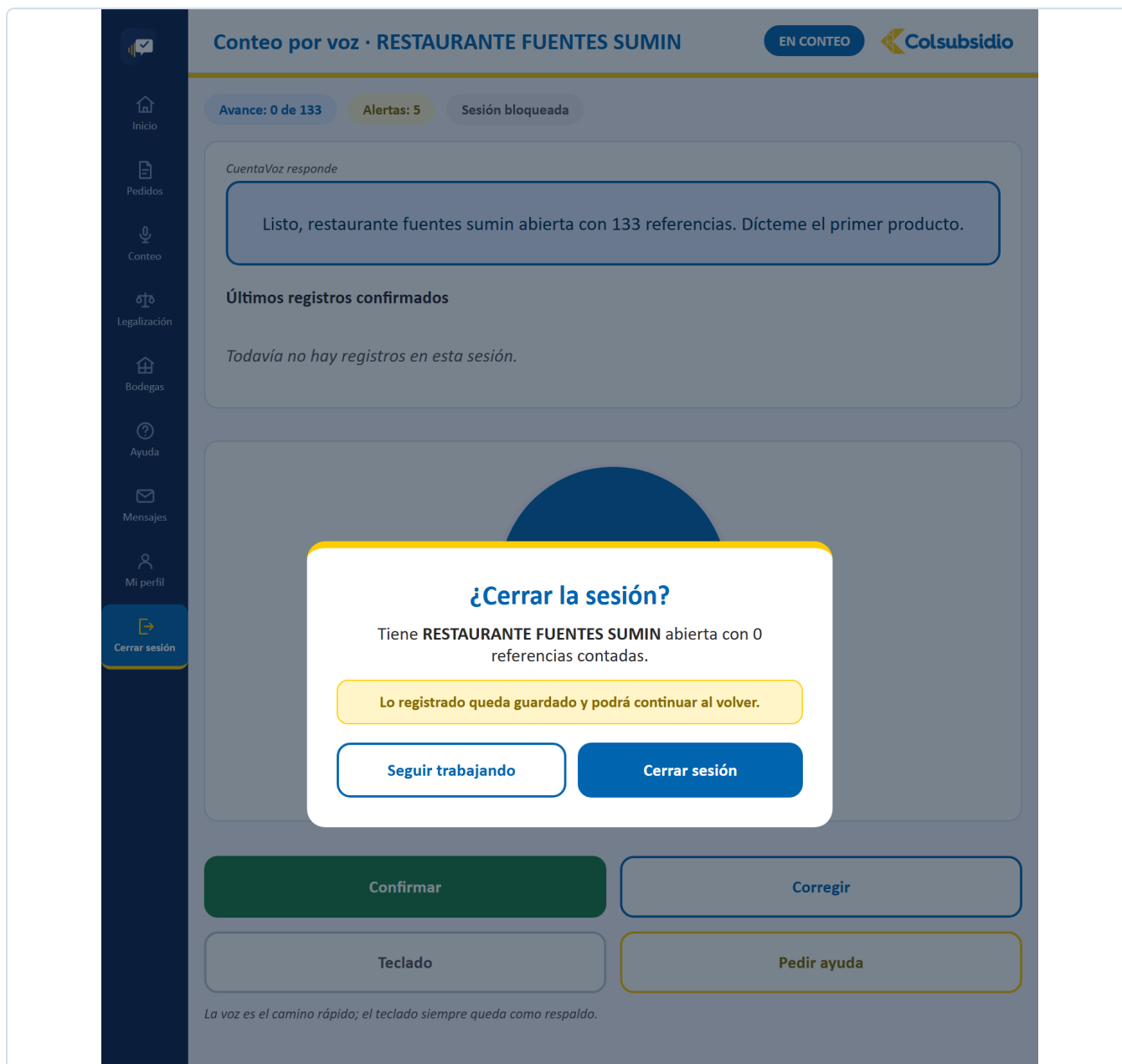


FIGURA 74 Avisa qué quedó abierto y con cuántas referencias, antes de dejar salir.

13 Pruebas

Qué hay, cómo se corre y qué cubre.

SUITE	CÓMO SE CORRE	QUÉ CUBRE
Backend 128 pruebas	cd backend python -m pytest tests/ -q	El agente y sus conversaciones, el intérprete, la conciliación, las recetas, la seguridad y las regresiones de legalización.
Frontend 5 archivos	cd frontend npm test	Lógica pura: intérprete local, cola sin conexión, accesibilidad, confirmación por voz y tutorial.

Las pruebas de backend usan **conftest.py** con una base en memoria, así que no tocan **cuentavoz.db**. Las de frontend usan **node:test**, sin infraestructura de renderizado: se prueba la lógica que no depende del DOM.

LAS PRUEBAS CON NOMBRE REGRESIONES NO SON OPCIONALES

test_regresiones_seguridad.py y **test_regresiones_legalizacion.py** existen porque esos fallos ya ocurrieron una vez. Si una de ellas se pone en rojo, no es una prueba frágil: es el mismo error volviendo.

14 Despliegue

Cómo llega esto a producción.

El despliegue está descrito en **render.yaml** y detallado en **DESPLIEGUE.md**.

Son tres servicios en Render:

SERVICIO	TIPO	QUÉ ES
cuentavoz-db	PostgreSQL	La base de producción.
cuentavoz-api	Web (Python)	El backend con Uvicorn.
cuentavoz	Estático	El build del frontend, con reescritura para que las rutas de la aplicación funcionen.

La misma base de código sirve en local y en producción: lo único que cambia es **DB_URL** y las llaves. Las credenciales se cargan como variables de entorno del servicio, nunca en el repositorio.

EL SERVIDOR SE DUERME

En el plan gratuito de Render el backend entra en reposo tras un rato sin tráfico, y la primera petición puede tardar hasta un minuto. Por eso las pantallas que dependen de la red avisan «el servidor estaba en reposo y está despertando» en vez de dar un error seco.

15 Decisiones de diseño

Por qué las cosas son como son. Útil antes de cambiarlas.

DECISIÓN	POR QUÉ
Confirmar siempre antes de guardar	El reconocimiento de voz se equivoca. Un inventario con datos que nadie aprobó es peor que no tener inventario.
Nunca sobrescribir	Sin el valor anterior no hay auditoría posible.
Intérprete local además de Gemini	Una bodega con mala señal, o una cuota agotada, no pueden dejar a nadie sin poder contar.
La clave la guarda Cognito	Lo que no se almacena no se puede filtrar.
Permisos también en el endpoint	Ocultar una opción del menú no es seguridad.
El auxiliar no ve pantallas que no puede usar	Mostrarle Ajustes en solo lectura le enseña controles que no puede tocar y hace ver la herramienta más complicada de lo que es para él.
Los reportes se previsualizan	Descargar el archivo equivocado y cargarlo a My Inventory cuesta más que un clic de más.
Un mensaje se responde una sola vez	Dos respuestas al mismo mensaje pisaban la anterior en silencio, y quien ya la había leído no se enteraba del cambio.

16 Cuando algo falla

Los tropiezos más frecuentes, y qué revisar primero.

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
«Sesión inválida o vencida» al entrar	Quedó un token viejo en <code>localStorage</code> .	Limpiar el almacenamiento del sitio y volver a entrar.
El navegador dice «error de red» pero el servidor responde	El origen no está en la lista de CORS.	Agregarlo a <code>ORIGEN_PERMITIDO</code> y reiniciar el backend.
El agente entiende menos de lo esperado	No hay <code>GOOGLE_API_KEY</code> : está corriendo el intérprete local.	Ponerla en el <code>.env</code> . Lo confirma Ayuda › Estado del sistema .
Un auxiliar no ve su bodega	No se la asignaron.	Ajustes › Gestión de usuarios › asignar bodegas.
La primera petición tarda un minuto	El servidor estaba en reposo.	Esperar. La aplicación ya lo avisa en pantalla.
El correo al administrador no sale	No hay <code>BREVO_API_KEY</code> .	Es esperado: el mensaje igual queda en la bandeja y en la trazabilidad.
Una prueba de regresiones se pone en rojo	Volvió un fallo que ya se había corregido.	No ajustar la prueba. Arreglar el código.

CuentaVoz · Tu voz cuenta

Guía técnica, versión 5.0 — agosto de 2026.
Elaborada por el equipo StockXperts para la
operación de bodegas de Colsubsidio.

El código manda

Esta guía explica el porqué y el cómo, no reemplaza al repositorio. Cuando una y otro no coincidan, el código tiene la razón — y esta guía tiene un error que vale la pena corregir.

17 El código completo

El proyecto entero, archivo por archivo. Este apéndice **se genera leyendo el repositorio** cada vez que se compila la guía: no está transcrito a mano, así que no puede quedar desfasado del código real.

17.1 Backend · el nucleo

backend/main.py

5189 LÍNEAS

La aplicacion FastAPI entera: los 90 endpoints, el orquestador del agente, los middlewares de CORS y cabeceras, y el arranque.

```

1  """La API de CuentaVoz. Aqui se conectan la tableta, el agente y la base."""
2  import json
3  import os
4  import re
5  import secrets
6  import socket
7  import threading
8  import unicodedata
9  import urllib.error
10 import urllib.request
11 from datetime import timedelta
12 from horario import ahora
13
14 from dotenv import load_dotenv
15 load_dotenv()
16
17 # Apagado por defecto (sin SENTRY_DSN no llama a init ni manda nada a
18 # ningun lado) - se deja cableado para que activarlo en produccion sea
19 # solo poner la variable de entorno, sin tocar codigo. send_default_pii
20 # en False a proposito: Usuario.correo es un correo real de empleado, no
21 # hay que dejarlo viajar a un tercero por defecto.
22 import sentry_sdk
23 _SENTRY_DSN = os.environ.get("SENTRY_DSN", "").strip()
24 if _SENTRY_DSN:
25     sentry_sdk.init(dsn=_SENTRY_DSN, traces_sample_rate=0.0, send_default_pii=False)
26
27 from fastapi import (FastAPI, WebSocket, WebSocketDisconnect, Depends,
28                     HTTPException, UploadFile, File, Request)
29 from fastapi.concurrency import run_in_threadpool
30 from fastapi.middleware.cors import CORSMiddleware
31 from fastapi.responses import JSONResponse, Response
32 from pydantic import BaseModel, Field
33 from slowapi import Limiter
34 from slowapi.util import get_remote_address
35 from slowapi.errors import RateLimitExceeded
36
37 from bd import Sesion, iniciar_bd
38 from modelos import (Usuario, Bodega, Artículo, StockSistema, SesionConteo,
39                     Conteo, Alerta, Traza, LíneaServicio, AsignacionBodega,
40                     Aprobacion, HistorialCierre, ConfigClave, Sede,
41                     Receta, RecetaIngrediente, MensajeSoporte)
42 from seguridad import (usuario_actual, requiere_perfil, registrar, esquema,
43                     COGNITO_REGION, COGNITO_USER_POOL_ID)
44 from agente.orquestador import procesar_turno, ESTADOS, avance
45 from servicios.recetas import (calcular_pedido, comparar_legalizacion,
46                             analisis_consumo, detalle_receta, _filas_a_legalizar)
47 from servicios.validacion import umbral_actual, validar_conteo
48 from servicios import analitica
49 from servicios.interprete import _numero as _numero_de_texto
50 import reportes
51
52
53 def _cliente_cognito():

```

```

54     """El backend administra Cognito (sembrar las cuentas demo, cerrar
55     todas las sesiones de una persona) con SU PROPIA credencial de AWS -
56     distinta de la que se uso para crear el User Pool: esta solo puede
57     AdminCreateUser/AdminSetUserPassword/AdminGetUser/
58     AdminUserGlobalSignOut sobre este User Pool en particular, nada mas
59     (ver el usuario de IAM "cuentavoz-backend"). Sin AWS_ACCESS_KEY_ID
60     configurada (desarrollo local sin AWS a mano), boto3 revienta al
61     hacer la primera llamada real, no aqui - se deja igual que el patron
62     ya usado para Gemini/Brevo: se degrada en el momento de usarla, no al
63     arrancar."""
64     import boto3
65     return boto3.client("cognito-idp", region_name=COGNITO_REGION)
66
67 app = FastAPI(title="CuentaVoz", version="1.0.0",
68               description="Asistente por voz para inventarios · Colsubsidio")
69
70 limiter = Limiter(key_func=get_remote_address)
71 app.state.limiter = limiter
72
73
74 @app.exception_handler(RateLimitExceeded)
75 def _manejador_limite_excedido(request: Request, exc: RateLimitExceeded):
76     # El manejador por defecto de slowapi responde {"error": "..."} en vez
77     # de {"detail": "..."}, que es la clave que pedir() (api.js) y toda la
78     # app ya saben leer para mostrar el mensaje real - sin esto, cualquier
79     # pantalla que tropieze con un limite (ingreso, huella, busqueda de
80     # perfil...) mostraba "Error 429" en vez de avisar que hay que esperar.
81     respuesta = JSONResponse(
82         {"detail": "Demasiados intentos. Espere un momento y vuelva a intentarlo."},
83         status_code=429)
84     _poner_cabeceras_cors(respuesta, request) # mismo motivo que en el de 500
85     return respuesta
86
87
88 @app.exception_handler(Exception)
89 def _manejador_error_no_capturado(request: Request, exc: Exception):
90     # FastAPI ya responde 500 solo ante una excepcion no manejada, pero sin
91     # la forma {"detail": "..."} que pedir() (api.js) espera - sin esto, un
92     # error real (no uno ya convertido a HTTPException) se veia en pantalla
93     # como un mensaje crudo en vez del aviso en español de siempre.
94     if _SENTRY_DSN:
95         sentry_sdk.capture_exception(exc)
96     print(f"[error no capturado] {request.method} {request.url.path}: "
97           f"{type(exc).__name__}: {exc}")
98     respuesta = JSONResponse({"detail": "Ocurrió un error inesperado. Intente de nuevo."},
99                             status_code=500)
100     _poner_cabeceras_cors(respuesta, request)
101     return respuesta
102
103
104 def _poner_cabeceras_cors(respuesta, request: Request) -> None:
105     """Le pega las cabeceras CORS a mano a una respuesta de error.
106
107     Los manejadores de excepcion de Starlette corren POR FUERA del
108     CORSMiddleware, asi que sus respuestas salen sin esas cabeceras. El
109     navegador entonces no bloquea el error: bloquea la RESPUESTA ENTERA, y
110     fetch() lanza un TypeError indistinguible de quedarse sin red. Costo
111     real de ese detalle: un 500 de verdad (una insercion que fallaba en
112     Postgres) llevo a la pantalla como "Sin conexion con el servidor.
113     Revise el Wi-Fi", mandando a buscar el problema en el router mientras
114     el servidor respondia perfecto. Con las cabeceras puestas, el error se
115     ve tal cual es."""
116     origen = request.headers.get("origin")
117     if origen and origen in _origenes:
118         respuesta.headers["Access-Control-Allow-Origin"] = origen
119         respuesta.headers["Access-Control-Allow-Credentials"] = "true"
120         respuesta.headers["Vary"] = "Origin"
121
122
123 _origenes = {o.strip() for o in os.getenv("ORIGEN_PERMITIDO", "").split(",") if o.strip()}
124 # 5173 es "npm run dev"; 4173 es "npm run preview" (el build de produccion,
125 # el unico que arma el service worker real - hace falta para probar el
126 # modo sin conexion tal como queda desplegado, no solo en modo desarrollo).
127 _origenes |= {"http://localhost:5173", "http://127.0.0.1:5173",

```

```

128         "http://localhost:4173", "http://127.0.0.1:4173"}
129
130 app.add_middleware(
131     CORSMiddleware,
132     allow_origins=list(_origenes),
133     allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
134 )
135
136
137 @app.middleware("http")
138 async def cabeceras(request: Request, llamar):
139     resp = await llamar(request)
140     resp.headers["X-Content-Type-Options"] = "nosniff"
141     resp.headers["X-Frame-Options"] = "SAMEORIGIN"
142     resp.headers["Referrer-Policy"] = "no-referrer"
143     # obliga HTTPS en cada visita futura del navegador, no solo en esta -
144     # sin esto, un enlace http:// (o un portal cautivo de Wi-Fi) puede
145     # interceptar la primera peticion antes de que el servidor la redirija.
146     resp.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
147     return resp
148
149
150 # Las 4 cuentas de demostracion (luis/diana/stephanie/valentina, clave
151 # "StockXperts1" - la misma que aparece en el README) solo se siembran si
152 # esta variable esta activa. Por defecto NO lo esta: sin esto, cualquier
153 # despliegue con la base de usuarios vacia -incluido un Render de
154 # produccion real, con datos reales, antes de que alguien cree las
155 # cuentas de verdad- quedaba con una cuenta de perfil auditor (diana, con
156 # permisos de administrador) accesible con una clave publicada. Para el
157 # hackathon esto se activa a proposito en render.yaml; para un uso real
158 # despues, basta con no ponerla (o quitarla) para que esas cuentas nunca
159 # aparezcan solas.
160 SEMBRAR_DEMO = os.getenv("SEMBRAR_DEMO", "").strip() == "1"
161 # La clave de las cuentas de demostracion vive en Cognito, no en esta base -
162 # tiene que cumplir la politica del User Pool (min. 8, mayuscula+minuscuala+
163 # numero). "StockXperts" sola no trae numero; se le agrego un "1" al
164 # migrar a Cognito (ver README.md/LEEME_PRIMERO.md, ya actualizados).
165 CLAVE_DEMO = "StockXperts1"
166
167
168 def _crear_usuario_cognito(nombre: str, correo: str, clave: str) -> bool:
169     """Crea la cuenta en Cognito con la clave dada, ya confirmada
170     (MessageAction=SUPPRESS: no le manda el correo de bienvenida
171     automatico de Cognito - esta app ya avisa la clave por su cuenta,
172     por voz o en pantalla - y Permanent=True: entra de una con esa clave,
173     sin el paso extra de "cambie su clave temporal" que exige Cognito
174     para claves no permanentes). Usado tanto al sembrar las cuentas de
175     demo como al crear un usuario nuevo por voz o desde Ajustes. Si ya
176     existe, no revienta: se ignora y se avisa por consola."""
177     cliente = _cliente_cognito()
178     try:
179         cliente.admin_create_user(
180             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre,
181             UserAttributes=[{"Name": "email", "Value": correo},
182                             {"Name": "email_verified", "Value": "true"},
183                             {"Name": "name", "Value": nombre}],
184             MessageAction="SUPPRESS",
185         )
186         cliente.admin_set_user_password(
187             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre,
188             Password=clave, Permanent=True,
189         )
190         return True
191     except cliente.exceptions.UsernameExistsException:
192         return False
193     except Exception as e:
194         print(f"[cognito] no se pudo crear la cuenta de {nombre}: {e}")
195         return False
196
197
198 def _actualizar_correo_cognito(nombre: str, correo: str) -> bool:
199     """Cognito es quien manda el código al recuperar clave o al confirmar
200     un registro - si el correo cambia aquí (Mi perfil o Ajustes) sin
201     avisarle a Cognito, esos códigos seguirían yendo al correo viejo para

```

```

202     siempre, sin que nada en la pantalla lo delatara. email_verified=true
203     porque este cambio lo hace un administrador o la propia persona ya
204     autenticada, no alguien sin verificar tratando de robar la cuenta.""
205     try:
206         _cliente_cognito().admin_update_user_attributes(
207             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre,
208             UserAttributes=[{"Name": "email", "Value": correo},
209                             {"Name": "email_verified", "Value": "true"}],
210         )
211         return True
212     except Exception as e:
213         print(f"[cognito] no se pudo actualizar el correo de {nombre}: {e}")
214         return False
215
216
217 @app.on_event("startup")
218 def arranque():
219     iniciar_bd()
220     with Sesion() as s:
221         if SEMBRAR_DEMO and s.query(Usuario).count() == 0:
222             s.add_all([
223                 Usuario(nombre="luis", perfil="auxiliar",
224                         correo="lnieto@colsubsidio.com", codigo="CS-48127"),
225                 Usuario(nombre="diana", perfil="auditor",
226                         correo="diana@colsubsidio.com", codigo="CS-48200"),
227                 Usuario(nombre="stephanie", perfil="auxiliar",
228                         correo="stephanie@colsubsidio.com", codigo="CS-48311"),
229                 Usuario(nombre="valentina", perfil="auxiliar",
230                         correo="valentina@colsubsidio.com", codigo="CS-48342"),
231             ])
232             s.commit()
233             for nombre, correo in (("luis", "lnieto@colsubsidio.com"),
234                                   ("diana", "diana@colsubsidio.com"),
235                                   ("stephanie", "stephanie@colsubsidio.com"),
236                                   ("valentina", "valentina@colsubsidio.com")):
237                 _crear_usuario_cognito(nombre, correo, CLAVE_DEMO)
238             print(f"[arranque] usuarios de prueba creados (clave {CLAVE_DEMO})")
239         elif s.query(Usuario).count() == 0:
240             print("[arranque] base de usuarios vacia y SEMBRAR_DEMO no esta activo: "
241                   "no se crearon cuentas de prueba. Regístrese desde Ingreso, "
242                   "o active SEMBRAR_DEMO=1 para la demo.")
243
244         # bodegas asignadas por persona: solo la primera vez, solo si ya hay
245         # bodegas cargadas (cargar_excel.py corre antes que la API), y solo
246         # si las cuentas de prueba de arriba existen de verdad - sin
247         # SEMBRAR_DEMO no hay "luis"/"stephanie"/"valentina" a quien
248         # asignarles nada.
249         if SEMBRAR_DEMO and s.query(AsignacionBodega).count() == 0 and s.query(Bodega).count() > 0:
250             # del Excel real de Colsubsidio, solo un subconjunto de bodegas
251             # trae existencia de sistema (SD) cargada - las demas son
252             # ubicaciones del catalogo sin stock_sistema asociado. Priorizar
253             # esas para el reparto demo evita que un auxiliar de prueba abra
254             # una bodega con "0 referencias" y no pueda mostrar diferencias
255             # contra el sistema en una demo en vivo.
256             con_stock = {bid for (bid,) in s.query(StockSistema.bodega_id).distinct()}
257             todas = s.query(Bodega).order_by(Bodega.nombre_oficial).all()
258             bodegas = sorted(todas, key=lambda b: (b.id not in con_stock, b.nombre_oficial))
259             usuarios = {u.nombre: u for u in s.query(Usuario).all()}
260             reparto = {"luis": bodegas[0:3], "stephanie": bodegas[3:6],
261                       "valentina": bodegas[6:9]}
262             total = 0
263             for nombre, asignadas in reparto.items():
264                 u = usuarios.get(nombre)
265                 if not u:
266                     continue
267                 for b in asignadas:
268                     s.add(AsignacionBodega(usuario_id=u.id, bodega_id=b.id))
269                     total += 1
270             s.commit()
271             if total:
272                 print(f"[arranque] {total} bodegas asignadas a los auxiliares de prueba")
273
274         # el conteo inicial de negativos, para que el panel muestre "N -> 0"
275         if s.query(ConfigClave).filter_by(clave="negativos_iniciales").first() is None:

```

```

276     neg = s.query(StockSistema).filter(StockSistema.cantidad_sd < 0).count()
277     s.add(ConfigClave(clave="negativos_iniciales", valor=str(neg)))
278     s.commit()
279
280     # un par de cierres de ejemplo, para que el histórico de exactitud
281     # del panel no arranque vacío (etiquetados como semilla, no reales)
282     if s.query(HistorialCierre).count() == 0 and s.query(Bodega).count() > 0:
283         primera = s.query(Bodega).order_by(Bodega.nombre_oficial).first()
284         from datetime import timedelta
285         base = ahora() - timedelta(days=120)
286         for i, exact in enumerate([94.2, 95.8, 96.5, 97.1]):
287             s.add(HistorialCierre(bodega_id=primera.id, exactitud=exact,
288                                 referencias=0, diferencias=0,
289                                 fecha=base + timedelta(days=30 * i)))
290     s.commit()
291     print("[arranque] histórico de exactitud sembrado (4 puntos de ejemplo)")
292
293
294 @app.get("/api/salud")
295 def salud():
296     with Sesion() as s:
297         return {"api": "ok",
298                 "bodegas": s.query(Bodega).count(),
299                 "articulos": s.query(Articulo).count(),
300                 "stock": s.query(StockSistema).count(),
301                 "gemini": bool(os.getenv("GOOGLE_API_KEY", "").strip()),
302                 # Solo si esta configurado, nunca el DSN: mismo criterio que
303                 # con gemini. Sirve para saber desde fuera si la variable de
304                 # Render quedo puesta, que si no hay que adivinarlo esperando
305                 # a que ocurra un error real para ver si llega o no.
306                 "sentry": bool(_SENTRY_DSN),
307                 "cognito": _estado_cognito()}
308
309
310 def _estado_cognito() -> dict:
311     """Con que User Pool y App Client esta trabajando ESTE backend.
312
313     No es informacion secreta: el User Pool Id y el App Client Id viajan en
314     el JavaScript que se descarga cualquiera que abra la aplicacion (ver
315     frontend/src/cognito.js) - por eso pueden ir aqui sin problema, y no se
316     expone nada mas (ni llaves de AWS, ni claves).
317
318     Existe porque estas tres variables van como sync:false en render.yaml,
319     o sea que viven solo en el panel de Render: si una queda vacia o
320     desactualizada, TODOS los tokens se rechazan con "Sesion invalida o
321     vencida.", que suena a problema de la persona cuando en realidad es de
322     configuracion. Comparar esto contra el frontend responde en un segundo
323     algo que si no toca adivinar."""
324     from seguridad import (COGNITO_REGION as reg, COGNITO_USER_POOL_ID as pool,
325                           COGNITO_APP_CLIENT_ID as cli, _jwks_client)
326     return {"region": reg,
327            "user_pool_id": pool or "(sin definir)",
328            "app_client_id": cli or "(sin definir)",
329            "puede_verificar_tokens": _jwks_client is not None}
330
331
332 # ----- identidad -----
333 def _buscar_usuario_por_entrada(s, entrada: str):
334     """Se puede ingresar con el nombre de usuario o con el codigo de
335     empleado (ej. CS-48127) - lo que la persona tenga a la mano."""
336     entrada = entrada.strip()
337     return s.query(Usuario).filter(
338         (Usuario.nombre == entrada.lower()) | (Usuario.codigo == entrada.upper())
339     ).first()
340
341
342 @app.get("/api/usuarios/perfil")
343 @limiter.limit("20/minute")
344 def perfil_por_usuario(request: Request, usuario: str = ""):
345     """Para que la pantalla de ingreso muestre «Auxiliar» o «Administrador»
346     apenas se escribe el usuario, sin esperar a iniciar sesion. Solo
347     devuelve el perfil (nada mas sensible: ni nombre, ni correo) y esta
348     limitado por minuto para no servir de lista para adivinar usuarios
349     validos. El login mismo (usuario+clave) lo resuelve Cognito

```

```

350     directamente desde el frontend - este backend nunca ve la clave.""
351     if not usuario.strip():
352         return {"perfil": None}
353     with Sesion() as s:
354         u = _buscar_usuario_por_entrada(s, usuario)
355         if not u or not u.activo:
356             return {"perfil": None}
357     return {"perfil": u.perfil}
358
359
360 class RegistroCompletadoIn(BaseModel):
361     nombre_completo: str
362     codigo: str = ""
363     correo: str
364
365
366 @app.post("/api/registro-completado")
367 @limiter.limit("10/minute")
368 def registro_completado(request: Request, datos: RegistroCompletadoIn,
369                         token: str = Depends(esquema)):
370     """Cognito ya creo Y confirmo la cuenta (el codigo que Cognito manda al
371     correo al registrarse) antes de llegar aqui - el frontend inicia
372     sesion contra Cognito justo despues de confirmar y llama esto con ese
373     token, no con datos sueltos. Aqui solo se crea la fila LOCAL que sabe
374     el perfil y las bodegas asignadas, cosas que Cognito no conoce. El
375     nombre de usuario sale del token ya verificado, nunca del cuerpo de
376     la peticion - y el perfil queda SIEMPRE "auxiliar": nadie puede
377     autoasignarse el perfil de auditor solo registrandose, ese ascenso
378     lo hace un administrador despues desde Ajustes.
379
380     La cuenta queda ACTIVA de inmediato: quien confirmo el codigo que
381     Cognito le mando al correo ya puede entrar, sin esperar a que un
382     administrador la habilite. Se probó la variante con aprobacion previa
383     y se descarto por decision de producto - agregaba una espera entre
384     registrarse y poder trabajar que no compensaba, dado que el perfil
385     siempre nace como auxiliar y sus permisos estan limitados a las
386     bodegas que un auditor le asigne (ver AsignacionBodega). Si alguna vez
387     hay que bloquear a alguien, editar_usuario ya permite desactivarlo."""
388     from seguridad import _reclamos_cognito
389     reclamos = _reclamos_cognito(token) if token else None
390     if reclamos is None:
391         raise HTTPException(401, "Sesion de Cognito invalida o vencida.")
392     nombre = (reclamos.get("username") or "").strip().lower()
393     if not nombre:
394         raise HTTPException(400, "No se pudo identificar el usuario de Cognito.")
395     with Sesion() as s:
396         if s.query(Usuario).filter_by(nombre=nombre).first():
397             # ya existe (ej. reintento de red tras un primer registro que
398             # si alcanzo a crear la fila) - no es un error, solo confirma.
399             return {"ok": True, "ya_existia": True}
400         nuevo = Usuario(nombre=nombre, perfil="auxiliar",
401                        correo=(datos.correo or "").strip(),
402                        codigo=(datos.codigo or "").strip().upper())
403         s.add(nuevo)
404         s.commit()
405         s.refresh(nuevo)
406         if not nuevo.codigo:
407             nuevo.codigo = f"CS-{48000 + nuevo.id}"
408             s.commit()
409     registrar(nuevo, "USUARIO", f"{nombre} se registro por su cuenta", "ok")
410     return {"ok": True, "ya_existia": False}
411
412
413 # ----- el agente -----
414 class TurnoIn(BaseModel):
415     texto: str
416     sesion_id: int = 1
417     # respaldo de resiliencia: si el backend se reinicio a mitad de una
418     # pregunta "¿cual de los dos?", el frontend ya sabe cuales opciones
419     # estaban pendientes y se las recuerda - ver Pedido.jsx/Conteo.jsx.
420     opciones_pendientes: list[dict] | None = None
421     opciones_para: str | None = None
422     # el mismo respaldo mas la bodega abierta: sin esto, un reinicio del
423     # backend deja el frontend mostrando "en conteo" mientras el servidor

```



```

424 # ya no sabe cual bodega es - el agente termina diciendo "no hay
425 # ninguna activa" con la bodega bien visible en la pantalla.
426 bodega_id_respaldo: int | None = None
427 bodega_nombre_respaldo: str | None = None
428 # lo mismo para Pedidos: el plato y las porciones casi siempre se dictan
429 # en frases separadas ("arroz con pollo" y despues "para cuatro"); sin
430 # este respaldo, un reinicio del backend a mitad de esas dos frases deja
431 # al agente sin memoria del plato aunque la pantalla lo siga mostrando.
432 preparacion_respaldo: str | None = None
433 porciones_respaldo: int | None = None
434
435
436 @app.post("/api/agente/turno")
437 async def turno(t: TurnoIn, u: Usuario = Depends(usuario_actual)):
438     _validar_sesion_de_voz(u, t.sesion_id, t.bodega_id_respaldo)
439     # procesar_turno() es sincrónico y llama a Gemini (pensar()), que puede
440     # tardar varios segundos - llamarlo directo aquí (una ruta async def)
441     # bloqueaba TODO el event loop mientras tanto: cualquier otra petición,
442     # de cualquier otra persona, a cualquier endpoint (hasta /api/salud) se
443     # quedaba esperando. run_in_threadpool lo saca del hilo principal para
444     # que el resto de la aplicación siga respondiendo mientras Gemini piensa.
445     r = await run_in_threadpool(procesar_turno, t.texto, t.sesion_id, u,
446                                opciones_respaldo=t.opciones_pendientes,
447                                opciones_para_respaldo=t.opciones_para,
448                                bodega_id_respaldo=t.bodega_id_respaldo,
449                                bodega_nombre_respaldo=t.bodega_nombre_respaldo,
450                                preparacion_respaldo=t.preparacion_respaldo,
451                                porciones_respaldo=t.porciones_respaldo)
452     if r.get("bodega"):
453         await difundir_estado()
454     return r
455
456
457 class PreguntarAsistenteIn(BaseModel):
458     texto: str
459     vista: str
460
461
462 # claves = lo mismo que usa ir(destino) en App.jsx; deben calzar exacto,
463 # si no la navegación por voz apunta a una vista que no existe.
464 _DESTINOS_ASISTENTE = {
465     "inicio": "Inicio", "pedido": "Pedidos", "conteo": "Conteo",
466     "legalizacion": "Legalización", "bodegas": "Bodegas",
467     "auditoria": "Auditoría", "reportes": "Reportes", "panel": "Panel",
468     "ajustes": "Ajustes", "ayuda": "Ayuda", "perfil": "Mi perfil",
469 }
470 _SOLO_AUDITOR_ASISTENTE = {"auditoria", "reportes", "panel", "ajustes"}
471
472 # pantallas con pestañas propias: para no obligar a la persona a llegar y
473 # después dar clic en la pestaña que quería, si la pide de una vez
474 # ("llévame al análisis de consumo") el agente navega directo a ella. La
475 # clave interna debe ser la misma que usa cada vista en su useState(tab).
476 _PESTANAS_ASISTENTE = {
477     "reportes": {"consolidado": "Consolidado de la toma", "analisis": "Análisis de consumo"},
478     "ajustes": {"config": "Configuración", "usuarios": "Gestión de usuarios",
479                "recetas": "Recetas", "traza": "Registro de trazabilidad"},
480     "panel": {"resumen": "Resumen ejecutivo", "alertas": "Bodegas y alertas"},
481     "auditoria": {"recuento": "Recuento ciego y cierre", "aprobaciones": "Aprobaciones",
482                  "pedidos": "Pedidos pendientes", "alertas": "Bandeja de alertas"},
483 }
484
485 _FAQ_ASISTENTE = [
486     ("¿Cómo corrijo un conteo ya confirmado?",
487      "Diga «corregir» y luego el valor correcto; el valor anterior se conserva."),
488     ("El agente no entiende un producto",
489      "Dígalo como aparece en la etiqueta; si no existe se puede crear, queda pendiente."),
490     ("¿Qué hago si sale una alerta?",
491      "Recuente; si el número es correcto, confirme: queda marcado para el administrador."),
492     ("Se cayó el internet en plena bodega",
493      "Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal."),
494     ("¿Puedo contar dos bodegas a la vez?",
495      "No en el mismo dispositivo; el candado de sesión evita conteos duplicados."),
496 ]
497

```

```

498
499 def _datos_panel_narrados() -> dict:
500     """Los mismos datos de /api/panel/resumen y /api/panel/alertas, ya
501     convertidos a frases listas para hablar - los usan tanto el contexto
502     completo que recibe Gemini como las respuestas determinísticas de
503     _responder_panel_por_voz, para no calcular el texto dos veces."""
504     r = analitica.resumen_ejecutivo()
505     a = analitica.resumen_alertas_panel()
506     dif_bod = r["diferencia_por_bodega"]
507     texto_dif = ("sin diferencias registradas todavía" if not dif_bod else
508                 "; ".join(f"{x['bodega'].title(): {x['diferencia']}" for x in dif_bod[:6]))
509     stock = r["stock_por_unidad"]
510     texto_stock = ("sin datos de stock todavía" if not stock else
511                   ", ".join(f"{x['unidad']} {x['pct']}% ({x['cantidad']})" for x in stock))
512     hist = r["historial_exactitud"]
513     if len(hist) < 2:
514         texto_hist = "todavía no hay suficientes cierres para ver una tendencia"
515     else:
516         tendencia = "mejorando" if hist[-1]["exactitud"] >= hist[0]["exactitud"] else "bajando"
517         texto_hist = (f"{len(hist)} tomas registradas, la más reciente en "
518                     f"{hist[-1]['bodega'].title()} con {hist[-1]['exactitud']}%, "
519                     f"tendencia {tendencia}")
520     etiquetas_estado = {"cerrada": "cerradas", "en_conteo": "en conteo",
521                        "en_auditoria": "en auditoría", "pendiente": "pendientes"}
522     texto_estado = (" ".join(f"{n} {etiquetas_estado.get(e, e)}"
523                             for e, n in a["estado_bodegas"].items())
524                    or "sin bodegas registradas")
525     alertas_tipo = a["alertas_por_tipo"]
526     texto_alertas_tipo = ("sin alertas registradas todavía" if not alertas_tipo else
527                          ", ".join(f"{x['tipo']}: {x['cantidad']}" for x in alertas_tipo))
528     descuadres = a["descuadres_recurrentes"]
529     texto_descuadres = ("sin descuadres repetidos todavía" if not descuadres else
530                        "; ".join(f"{x['articulo'].title(): {x['tipo']}, diferencia "
531                                f"{x['diferencia']} en {x['tomas']} tomas, acción sugerida: "
532                                f"{x['accion']}" for x in descuadres[:5]))
533     return {"r": r, "a": a, "texto_dif": texto_dif, "texto_stock": texto_stock,
534            "texto_hist": texto_hist, "texto_estado": texto_estado,
535            "texto_alertas_tipo": texto_alertas_tipo, "texto_descuadres": texto_descuadres}
536
537
538 # Preguntas frecuentes del Panel gerencial, resueltas sin pasar por Gemini.
539 # Sin esto, una pregunta tan común como "¿cuántas bodegas están cerradas?"
540 # a veces terminaba navegando a la pantalla Bodegas en vez de contestar
541 # (confirmado en pruebas: al mencionar el nombre de otra pantalla, Gemini
542 # no siempre distingue preguntar de navegar) - las mismas frases que se
543 # ofrecen en el desplegable "Ver frases exactas" de Panel quedan así
544 # garantizadas.
545 _PANEL_PREGUNTAS = [
546     (re.compile(r"\bprimera\s+pasada\b", re.IGNORECASE),
547      lambda d: f"La exactitud primera pasada es del {d['r']['exactitud_primera_pasada']}%, "
548               "promedio de cierres."),
549     (re.compile(r"\bpreferencias?\b.*\bcontadw*\b|bcu[aá]ntw*\s+referencias\b", re.IGNORECASE),
550      lambda d: f"Se han contado {d['r']['referencias_contadas']} referencias en toda la "
551               "operación."),
552     (re.compile(r"\balertas?\b.*\bgestionadw*", re.IGNORECASE),
553      lambda d: f"Van {d['r']['alertas_gestionadas']} alertas gestionadas de "
554               f"{d['r']['alertas_total']} en total."),
555     (re.compile(r"\bc[oó]mo\s+(est[aá]n|van)\s+las\s+bodegas\b|bestado\s+de\s+las\s+bodegas\b",
556                re.IGNORECASE),
557      lambda d: f"Estado de las bodegas: {d['texto_estado']}."),
558     (re.compile(r"\bcu[aá]ntw*\s+bodegas\b.*\bcerradw*\b|bbodegas\s+cerradas\b", re.IGNORECASE),
559      lambda d: f"Van {d['r']['bodegas_cerradas']} bodegas cerradas de {d['r']['bodegas_total']} "
560               "en total, con doble firma digital."),
561     (re.compile(r"\bqu[eé]\s+bodega\b.*\bdiferencia\b|bdiferencia\s+absoluta\s+por\s+bodega\b",
562                re.IGNORECASE),
563      lambda d: f"Diferencia absoluta por bodega: {d['texto_dif']}."),
564     (re.compile(r"\bstock\s+por\s+unidad\b|bc[oó]mo\s+va\s+el\s+stock\b", re.IGNORECASE),
565      lambda d: f"Stock por unidad de medida: {d['texto_stock']}."),
566     (re.compile(r"\btendencia\s+de\s+exactitud\b|bexactitud\s+por\s+toma\b", re.IGNORECASE),
567      lambda d: f"Tendencia por toma de inventario: {d['texto_hist']}."),
568     (re.compile(r"\bnegativos?\b.*\bcorregidw*\b|bcu[aá]ntw*\s+negativos\b", re.IGNORECASE),
569      lambda d: (f"Saldo negativo en el sistema: {d['a']['negativos_actuales']} artículos. "
570               "La corrección se hace en My Inventory, no en CuentaVoz."
571               if d['a']['negativos_iniciales'] == d['a']['negativos_actuales'] else

```

```

572         f"Saldos negativos: {d['a']['negativos_iniciales']} al inicio de este período, "
573         f"{d['a']['negativos_actuales']} ahora.")),
574     (re.compile(r"\btipo\s+promedio\s+de\s+conteo\b", re.IGNORECASE),
575     lambda d: f"El tiempo promedio de conteo por bodega es de "
576     f"{d['a']['tiempo_promedio_min']} minutos."),
577     (re.compile(r"\balias\s+aprendid\w*\b|bcu[aá]nt\w*\s+alias\b", re.IGNORECASE),
578     lambda d: f"El agente tiene {d['a']['alias_aprendidos']} alias aprendidos."),
579     (re.compile(r"\balertas\s+por\s+tipo\b", re.IGNORECASE),
580     lambda d: f"Alertas por tipo: {d['texto_alertas_tipo']}."),
581     (re.compile(r"\bdescuadre\w*\s+(principal|recurrent\w*)\b\bcausa\s+ra[ií]z\b",
582     re.IGNORECASE),
583     lambda d: f"Descuadres recurrentes: {d['texto_descuadres']}."),
584 ]
585
586
587 def _responder_panel_por_voz(texto: str, u: Usuario) -> dict | None:
588     if u.perfil != "auditor":
589         return None
590     for patron, respuesta in _PANEL_PREGUNTAS:
591         if patron.search(texto):
592             return {"respuesta_hablada": respuesta(_datos_panel_narrados()),
593                 "accion": None, "destino": None, "pestana": None}
594     return None
595
596
597 def _formato_acceso_hablado(fecha_dt) -> str | None:
598     """Mismo criterio que formatoAcceso() en MiPerfil.jsx (hoy HH:MM vs
599     DD/MM HH:MM) - para que lo que diga el agente coincida con lo que la
600     pantalla ya muestra escrito."""
601     if not fecha_dt:
602         return None
603     if fecha_dt.date() == ahora().date():
604         return f"hoy a las {fecha_dt.strftime('%H:%M')}"
605     return f"el {fecha_dt.strftime('%d/%m')} a las {fecha_dt.strftime('%H:%M')}"
606
607
608 def _datos_perfil_narrados(u: Usuario) -> dict:
609     from agente.cerebro import VOCES, VOZ_DEFEECTO
610     with Sesion() as s:
611         usr = s.get(Usuario, u.id)
612         asigs = s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
613         nombres_bodegas = [b.nombre_oficial.title() for b in
614             (s.get(Bodega, a.bodega_id) for a in asigs) if b]
615         dias_pin = (ahora() - (usr.pin_actualizado or ahora())).days
616         voz = usr.idioma_voz if usr.idioma_voz in VOCES else VOZ_DEFEECTO
617         return {
618             "ultimo_acceso_hablado": _formato_acceso_hablado(usr.ultimo_acceso),
619             "pin_vence_en_dias": max(90 - dias_pin, 0),
620             "n_bodegas": len(nombres_bodegas),
621             "texto_bodegas": "; ".join(nombres_bodegas),
622             "perfil": usr.perfil,
623             "velocidad_voz": usr.velocidad_voz,
624             "confirmacion_hablada": bool(usr.confirmacion_hablada),
625             "voz_nombre": VOCES[voz]["nombre"],
626             "voz_etiqueta": VOCES[voz]["etiqueta"],
627         }
628
629
630 _PERFIL_PREGUNTAS = [
631     (re.compile(r"\b[ú]ltimo\s+acceso\b|cu[aá]ndo\s+(entr[eé]|ingres[eé])", re.IGNORECASE),
632     lambda d: f"Su último acceso fue {d['ultimo_acceso_hablado']}." if d["ultimo_acceso_hablado"]
633     else "Todavía no hay un acceso anterior registrado."),
634     # "pin" se sigue reconociendo aunque ya no se llame así: quien lleva
635     # tiempo usando el sistema lo va a decir igual, y no entenderlo sería
636     # peor que conservar el nombre viejo en la respuesta.
637     (re.compile(r"\b(pin|clave|contrase[ñ]a)\b.*\bvence|vence\b.*\b(pin|clave|contrase[ñ]a)\b"
638     r"|cu[aá]nt\w*\s+d[ií]as.*\b(pin|clave|contrase[ñ]a)\b",
639     re.IGNORECASE),
640     lambda d: f"Su clave vence en {d['pin_vence_en_dias']} días."),
641     (re.compile(r"\b(bodegas?\s+(tengo|asignad\w*)|cu[aá]nt\w*\s+bodegas", re.IGNORECASE),
642     lambda d: (f" Tiene {d['n_bodegas']} bodegas asignadas: {d['texto_bodegas']}."
643     if d["n_bodegas"] else "Todavía no tiene bodegas asignadas.")),
644     (re.compile(r"\b(mi\s+perfil\b|qu[eé]\s+perfil|soy\s+(auxiliar|admin\w*)", re.IGNORECASE),
645     lambda d: f"Su perfil es {'administrador de bodega' if d['perfil'] == 'auditor' else 'auxiliar de

```

```

inventarios'}."),
646     (re.compile(r"qu[eé]\s+voz\s+(tengo|est[aá]|uso)|voz\s+actual", re.IGNORECASE),
647     lambda d: f"Está usando la voz {d['voz_nombre']}: {d['voz_etiqueta'].lower()}."),
648 ]
649
650
651 def _responder_perfil_por_voz(texto: str, u: Usuario) -> dict | None:
652     for patron, respuesta in _PERFIL_PREGUNTAS:
653         if patron.search(texto):
654             return {"respuesta_hablada": respuesta(_datos_perfil_narrados(u)),
655                 "accion": None, "destino": None, "pestana": None}
656     return None
657
658
659 _VOCES_HABLADAS = ("kore", "aoede", "puck", "charon")
660 _VERBO_CAMBIAR_VOZ = re.compile(
661     r"\b(cambia|usa|pon|quiero|eli[jg]\w*|selecciona\w*)\b", re.IGNORECASE)
662
663
664 def _cambiar_voz_por_voz(texto: str, u: Usuario) -> dict | None:
665     """Cambia mi voz a puck" / "usa la voz aoede" - exige el nombre EXACTO
666     de una de las cuatro voces (no un genero o descripcion), para no
667     competir con "¿qué voz tengo?" ni con "prueba la voz" (probarVoz solo
668     demuestra la que ya está elegida, no cambia nada)."""
669     if not _VERBO_CAMBIAR_VOZ.search(texto):
670         return None
671     clave = next((k for k in _VOCES_HABLADAS if re.search(rf"\b{k}\b", texto, re.IGNORECASE)), None)
672     if not clave:
673         return None
674     with Sesion() as s:
675         usr = s.get(Usuario, u.id)
676         usr.idioma_voz = clave
677         s.commit()
678     from agente.cerebro import VOCES
679     registrar(u, "PERFIL", f"{u.nombre} cambió su voz a {VOCES[clave]['nombre']} por voz", "ok")
680     return {"respuesta_hablada": f"Listo, ahora hablo con la voz {VOCES[clave]['nombre']}: "
681         f"{VOCES[clave]['etiqueta'].lower()}.",
682         "accion": "actualizar", "destino": None, "pestana": None, "idioma_voz": clave}
683
684
685 _VELOCIDAD_LENTA = re.compile(r"\blenta\b|m[aá]\s+despacio|m[aá]\s+lento", re.IGNORECASE)
686 _VELOCIDAD_RAPIDA = re.compile(r"\br[aá]pida\b|m[aá]\s+r[aá]pido", re.IGNORECASE)
687 _VELOCIDAD_NORMAL = re.compile(r"velocidad\s+normal|voz\s+normal", re.IGNORECASE)
688
689
690 def _cambiar_velocidad_por_voz(texto: str, u: Usuario) -> dict | None:
691     if _VELOCIDAD_LENTA.search(texto):
692         nueva = "lenta"
693     elif _VELOCIDAD_RAPIDA.search(texto):
694         nueva = "rapida"
695     elif _VELOCIDAD_NORMAL.search(texto):
696         nueva = "normal"
697     else:
698         return None
699     with Sesion() as s:
700         usr = s.get(Usuario, u.id)
701         usr.velocidad_voz = nueva
702         s.commit()
703     registrar(u, "PERFIL", f"{u.nombre} cambió la velocidad de voz a {nueva} por voz", "ok")
704     return {"respuesta_hablada": f"Listo, la velocidad queda en {nueva}.",
705         "accion": "actualizar", "destino": None, "pestana": None, "velocidad_voz": nueva}
706
707
708 _CONFIRMACION_HABLADA_FRASE = re.compile(r"confirmaci[oó]n\s+hablada", re.IGNORECASE)
709
710
711 def _cambiar_confirmacion_hablada_por_voz(texto: str, u: Usuario) -> dict | None:
712     if not _CONFIRMACION_HABLADA_FRASE.search(texto):
713         return None
714     if _ACTIVAR.search(texto):
715         nuevo = True
716     elif _DESACTIVAR.search(texto):
717         nuevo = False
718     else:

```

```

719         return None
720     with Sesion() as s:
721         usr = s.get(Usuario, u.id)
722         usr.confirmacion_hablada = 1 if nuevo else 0
723         s.commit()
724     estado = "activada" if nuevo else "desactivada"
725     registrar(u, "PERFIL", f"{u.nombre} dejó la confirmación hablada {estado} por voz", "ok")
726     return {"respuesta_hablada": f"Listo, la confirmación hablada quedó {estado}.",
727            "accion": "actualizar", "destino": None, "pestana": None,
728            "confirmacion_hablada": nuevo}
729
730
731 def _contexto_asistente(vista: str, u: Usuario) -> str:
732     """Un resumen honesto de lo que hay AHORA en esa pantalla, para que el
733     agente conteste con datos reales y no invente nada. Reusa las mismas
734     funciones que ya alimentan cada pantalla, no vuelve a calcular nada."""
735     if vista == "inicio":
736         with Sesion() as s:
737             n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=u.id).count()
738             alertas = s.query(Alerta).filter_by(resuelta=0).count()
739             historial = s.query(HistorialCierre).all()
740             exact = (round(sum(h.exactitud for h in historial) / len(historial), 1)
741                      if historial else 100.0)
742         return (f"Pantalla: Inicio, de {u.nombre} ({u.perfil}). "
743               f"Bodegas asignadas a esta persona: {n_bodegas}. "
744               f"Alertas por revisar en toda la operación: {alertas}. "
745               f"Exactitud del mes: {exact}%.")
746     if vista == "ajustes":
747         a = ver_ajustes(u)
748         # El permiso real de cambiar el modo sin conexión por voz ya se
749         # resuelve antes de llegar aquí (_cambiar_modos_sin_conexion, que
750         # exige perfil auditor) - este texto solo llega a Gemini cuando
751         # esa orden exacta no aplicó (una pregunta suelta, o quien
752         # pregunta no es administrador). Sin condicionarlo al perfil,
753         # Gemini le decía a CUALQUIERA "ya lo active" aunque el cambio de
754         # verdad solo lo hace un administrador - la misma deshonestidad
755         # que la Regla 6 ya evita en otras respuestas.
756         permiso_offline = (
757             "El modo sin conexión sí se puede cambiar por voz, pero solo por un "
758             "administrador: decir «activa/desactiva el modo sin conexión» lo aplica de "
759             "una, sin tocar el interruptor. En Gestión de usuarios, un administrador "
760             "también puede decir «activa/desactiva a <nombre>» para cambiar el estado de "
761             "otra persona, «crea un usuario llamado <nombre> perfil <auxiliar o "
762             "auditor»» para crear una cuenta nueva (con clave temporal), «cambia el "
763             "perfil de <nombre> a auxiliar/administrador» para lo mismo que el botón "
764             "«Editar» (el correo no se cambia por voz), «asigne/quítale la bodega "
765             "<bodega> a <nombre>» para lo mismo que el botón «Asignar bodegas» pero "
766             "sumando o quitando solo esa bodega, no reemplazando toda la lista, y "
767             "«¿quién tiene la bodega <bodega>?» o «¿cuántas bodegas sin asignar hay?» "
768             "para contestar una pregunta puntual sin abrir nada, y «muéstrame/oculta "
769             "la asignación por bodega» para abrir o cerrar la tabla completa - lo "
770             "mismo que el botón «Ver/Ocultar asignación por bodega». En Recetas, un "
771             "administrador también puede decir «crea una receta llamada "
772             "<nombre>, rendimiento <N> porciones, con <cantidad> de <ingrediente>» "
773             "(crea la receta con ese primer ingrediente ya resuelto contra el "
774             "catálogo), «agrega <cantidad> de <ingrediente> a la receta <nombre>» "
775             "(para sumarle más ingredientes después), «quita el ingrediente "
776             "<ingrediente> de la receta <nombre>», «cambia el rendimiento de la "
777             "receta <nombre> a <N> porciones», «agrega la preparación a la receta "
778             "<nombre>: <pasos>» (reemplaza los pasos de cocina completos, tal cual "
779             "se dicten, con los dos puntos como separador del nombre), y «elimina la "
780             "receta <nombre>». "
781             "En Registro de trazabilidad, «exporta el registro de trazabilidad» "
782             "descarga el archivo con los filtros de fecha/persona/acción que ya "
783             "estén puestos en la pantalla en ese momento (no los que diga la "
784             "orden), «acciones por persona» resume cuántas hizo cada quien "
785             "(hoy por defecto; «esta semana»/«este mes»/«en total» cambian el "
786             "periodo), y «¿cuántas acciones tiene <nombre>?» contesta solo por esa "
787             "persona. "
788             "Todo esto SOLO funciona si la orden usa exactamente esas formas («crea un "
789             "usuario llamado...», «cambia el perfil de... a...», «asigne/quítale la "
790             "bodega... a...», «crea una receta llamada..., rendimiento... porciones, "
791             "con... de...», «agrega... a la receta...», «quita el ingrediente... de "
792             "la receta...», «cambia el rendimiento de la receta... a...», «agrega la "

```

```

793     "preparación a la receta...: ...», «elimina la receta...», «exporta el "
794     "registro de trazabilidad», «acciones por persona», «cuántas acciones "
795     "tiene...») - si "
796     "el mensaje llega hasta usted es porque esa frase no se dijo así, el "
797     "nombre de la persona/bodega no coincidió con ninguna existente, o el "
798     "ingrediente no se encontró en el catálogo, así que no invente que ya "
799     "creó, cambió, asignó, agregó o eliminó nada: pida que lo repitan con ese "
800     "formato exacto y el nombre completo tal como aparece en la pantalla."
801     if u.perfil == "auditor" else
802     "El modo sin conexión, el estado de otras personas, crear/editar usuarios, "
803     "asignar bodegas y crear/editar/eliminar recetas solo los puede hacer un "
804     "administrador - esta persona no lo es, así que si lo pide, dígalos con "
805     "honestidad en vez de decir que ya quedó hecho.")
806     return (f"Pantalla: Ajustes. Sección Validación de datos: umbral de anomalía "
807            f"{a['umbral']}%; bloquear cantidades negativas activado (regla fija, no "
808            f"se puede desactivar); exigir confirmación en alertas activado (regla "
809            f"fija); permitir crear productos pendientes activado (regla fija). "
810            f"Sección Conexión y sincronización: modo sin conexión "
811            f"{'activado' if a['offline'] else 'desactivado'}; refresco del tablero "
812            f"en tiempo real. "
813            f"Sección Administración: usuarios activos {a['usuarios_activos']}; "
814            f"aprobaciones pendientes {a['aprobaciones_pendientes']}. Sección Acerca "
815            f"de CuentaVoz: versión {a['version']}, modelo del agente {a['modelo']}. "
816            f"El umbral solo lo puede CAMBIAR un administrador, y únicamente con los "
817            f"controles de la pantalla (número y botón «Guardar configuración») - eso "
818            f"todavía no cambia por voz. {permiso_offline}")
819     if vista == "panel" and u.perfil == "auditor":
820         d = _datos_panel_narrados()
821         r, a = d["r"], d["a"]
822         return (
823             "Pantalla: Panel gerencial, con dos pestañas: «Resumen ejecutivo» y «Bodegas y "
824             "alertas» - decir el nombre de cualquiera de las dos (o «llévame a...») navega "
825             "directo a ella. "
826             f"Pestaña Resumen ejecutivo - tarjetas: exactitud primera pasada "
827             f"{r['exactitud_primera_pasada']}%, referencias contadas "
828             f"{r['referencias_contadas']}, alertas gestionadas {r['alertas_gestionadas']} de "
829             f"{r['alertas_total']}, bodegas cerradas {r['bodegas_cerradas']} de "
830             f"{r['bodegas_total']}. Gráfica «Diferencia absoluta por bodega»: {d['texto_dif']}. "
831             f"Gráfica «Stock por unidad de medida»: {d['texto_stock']}. Gráfica «Exactitud por "
832             f"toma de inventario»: {d['texto_hist']}. "
833             f"Pestaña Bodegas y alertas - tarjetas: saldos negativos en el sistema "
834             f"{a['negativos_actuales']} artículos (la corrección se hace en My Inventory, no "
835             f"en CuentaVoz - dentro de una sola toma este número normalmente no se mueve), "
836             f"tiempo promedio de conteo por bodega {a['tiempo_promedio_min']} minutos, "
837             f"alias aprendidos por el agente {a['alias_aprendidos']}. Tarjeta «Estado de las "
838             f"bodegas»: {d['texto_estado']}. Tarjeta «Alertas por tipo»: {d['texto_alertas_tipo']}. "
839             f"Tabla «Descuadres recurrentes»: {d['texto_descuadres']}")
840     if vista == "reportes" and u.perfil == "auditor":
841         d = _datos_reportes_narrados(u)
842         a = d["analisis"]
843         return ("Pantalla: Reportes, con dos pestañas: «Consolidado de la toma» y «Análisis de "
844                "consumo». Aquí se genera el consolidado para My "
845                "Inventory, las diferencias por bodega, y el análisis de consumo "
846                "de los últimos 30 días. Los tres SÍ se pueden generar por voz "
847                "(«genera el consolidado», «exporta las diferencias», «exporta el "
848                "análisis de consumo») - queda igual que si le hubiera dado clic "
849                f"al botón. Archivos generados recientemente: {d['texto_recientes']}. "
850                f"Pestaña Análisis de consumo, últimos {a['dias']} días: pedido total "
851                f"{a['pedido_total']} kg en {a['servicios_periodo']} servicios, usado "
852                f"realmente {a['usado_total']} kg ({a['aprovechamiento']}% de "
853                f"aprovechamiento), ahorro potencial {a['ahorro_potencial']} kg al mes. "
854                f"Insumos subutilizados: {d['texto_subutil']}. También se puede pedir "
855                "«muéstrame el archivo de <consolidado o diferencias>» para ver su "
856                "contenido, igual que darle clic a la tarjeta.")
857     if vista == "auditoria" and u.perfil == "auditor":
858         d = _datos_auditoria_narrados(u)
859         ra = d["resumen_alertas"]
860         nombres_aprob = ("; ".join(a["nombre"].title() for a in d["aprobaciones"]))
861         if d["aprobaciones"] else "ninguna")
862         nombres_pedidos = ("; ".join(f"{p['persona'].title()}: {p['plato']}" for p in d["pedidos"]))
863         if d["pedidos"] else "ninguno")
864         return (f"Pantalla: Auditoría, con cuatro pestañas: «Recuento ciego y cierre», "
865                f"«Aprobaciones», «Pedidos pendientes» y «Bandeja de alertas» - decir el "
866                f"nombre de cualquiera (o «llévame a...») navega directo a ella. ")

```



```

867     f"Pestaña Aprobaciones: {len(d['aprobaciones'])} pendientes "
868     f"({nombres_aprob}) - «aprueba/rechaza <nombre>» resuelve una. "
869     f"Pestaña Pedidos pendientes: {len(d['pedidos'])} pendientes "
870     f"({nombres_pedidos}) - «aprueba/rechaza el pedido de <persona o plato>» "
871     f"resuelve uno. Pestaña Bandeja de alertas: {ra['abiertas']} abiertas, "
872     f"{ra['resueltas_hoy']} resueltas hoy, tiempo medio "
873     f"{ra['tiempo_medio_min']} if ra['tiempo_medio_min'] is not None else '-' } "
874     "minutos - «resuelve la alerta de <artículo o bodega>» resuelve una. "
875     "Pestaña Recuento ciego y cierre: «audita <bodega>» o «abre <bodega>» "
876     "selecciona esa bodega lista para recuento o cierre, igual que darle clic "
877     "a «Abrir» en su tarjeta - desde ahí, dictar los productos SÍ es por voz "
878     "con el micrófono normal de esa pantalla (igual que en Conteo). Pero "
879     "«Iniciar recuento ciego», «Ver comparación», «Aceptar todas», «Cerrar con "
880     "doble firma», «Firmar la auditoría», y «Cerrar bodega definitivamente» "
881     "son A PROPÓSITO solo manuales - nadie cierra una bodega con doble firma "
882     "por accidente con la voz. Si el mensaje llegó hasta usted es porque "
883     "«aprueba/rechaza <nombre>», «aprueba/rechaza el pedido de <...>», "
884     "«resuelve la alerta de <...>» o «audita <bodega>» NO encontraron ese "
885     "nombre exacto entre lo recién listado - esas acciones YA se resuelven "
886     "antes de llegar aquí cuando el nombre coincide, así que usted NUNCA debe "
887     "decir que ya aprobó, rechazó, resolvió o abrió algo (sería falso: no "
888     "tiene esa capacidad desde aquí). Dígalo con honestidad y sugiera repetirlo "
889     "con el nombre completo tal como aparece en la pantalla.")
890 if vista == "ayuda":
891     faq = "; ".join(f"{p} -> {r}" for p, r in _FAQ_ASISTENTE)
892     return f"Pantalla: Ayuda. Preguntas frecuentes conocidas: {faq}."
893 if vista == "perfil":
894     d = _datos_perfil_narrados(u)
895     return (
896         f"Pantalla: Mi perfil, de {u.nombre} ({u.perfil}). Datos personales, seguridad "
897         "de la cuenta y preferencias de voz. "
898         f"Último acceso: {d['ultimo_acceso_hablado']} or 'sin registro'. La clave vence en "
899         f"{d['pin_vence_en_dias']} días. Bodegas asignadas: {d['n_bodegas']}"
900         + (f" ({d['texto_bodegas']})" if d['texto_bodegas'] else "") + ". "
901         f"Voz actual: {d['voz_nombre']} ({d['voz_etiqueta'].lower()}), velocidad "
902         f"{d['velocidad_voz']}, confirmación hablada "
903         f"{'activada' if d['confirmacion_hablada'] else 'desactivada'}."
904         "Sí se puede cambiar por voz: la voz neuronal («cambia mi voz a puck/kore/aoede/"
905         "charon»), la velocidad («habla más lento/rápido», «velocidad normal») y la "
906         "confirmación hablada («activa/desactiva la confirmación hablada»). Cambiar la "
907         "clave, subir una foto, y editar nombre/correo/teléfono son A PROPÓSITO solo "
908         "manuales, por seguridad - nunca diga que ya cambió la clave por voz, eso "
909         "sería falso.")
910 if vista == "bodegas":
911     with Sesion() as s:
912         ids_permitidos = _ids_permitidos_para_buscar(s, u)
913         q = s.query(Bodega)
914         if ids_permitidos is not None:
915             q = q.filter(Bodega.id.in_(ids_permitidos))
916         bodegas = q.all()
917         n_pendientes = sum(1 for b in bodegas if b.estado == "pendiente")
918         n_en_conteo = sum(1 for b in bodegas if b.estado == "en_conteo")
919     return (f"Pantalla: Bodegas. Aquí se busca el stock de un artículo en todo "
920         f"el catálogo, y se ve el estado/historial de cada bodega (no se "
921         f"cuanta desde aquí, eso es en Conteo). De las bodegas de esta "
922         f"persona: {n_pendientes} pendientes, {n_en_conteo} en conteo. Si "
923         f"piden abrir o seguir contando una bodega por nombre, se resuelve "
924         f"aparte antes de este mensaje - esto solo se usa para explicar o "
925         f"navegar a otra pantalla.")
926     return f"Pantalla: {vista}."
927
928
929 _VERBOS_ABRIR_BODEGA = re.compile(
930     r"\b(abr[ae])|abrir|contar|cuenta|siga|sigue|continu[ae]|continuar\b",
931     re.IGNORECASE)
932
933 _MODOSINCONEXION = re.compile(r"sin conexi[oó]n", re.IGNORECASE)
934 _ACTIVAR = re.compile(
935     r"\b(act[ii]v[w*]|enc[ie][eé]nd[w*]|prend[w*])\b", re.IGNORECASE)
936 _DESACTIVAR = re.compile(
937     r"\b(desact[ii]v[w*]|ap[aá]g[w*]|qu[ii]t[w*])\b", re.IGNORECASE)
938
939
940 def _cambiar_modosinconexion(texto: str, u: Usuario) -> dict | None:

```

```

941     """Único ajuste que de verdad se puede cambiar por voz en esta
942     pantalla (el resto - umbral, reglas fijas - sigue exigiendo los
943     controles de la pantalla): solo un administrador, y solo si la
944     orden nombra "sin conexión" junto con un verbo de activar/desactivar
945     inequívoco. None si no aplica, para caer al agente general."""
946     if u.perfil != "auditor" or not _MODOSINCONEXION.search(texto):
947         return None
948     if _ACTIVAR.search(texto):
949         nuevo = True
950     elif _DESACTIVAR.search(texto):
951         nuevo = False
952     else:
953         return None
954     with Sesion() as s:
955         existente = s.get(ConfigClave, "offline")
956         valor = "1" if nuevo else "0"
957         if existente:
958             existente.valor = valor
959         else:
960             s.add(ConfigClave(clave="offline", valor=valor))
961         s.commit()
962     estado = "activado" if nuevo else "desactivado"
963     registrar(u, "AJUSTE", f"{u.nombre} {estado} el modo sin conexión por voz", "ok")
964     return {"respuesta_hablada": f"Listo, el modo sin conexión quedó {estado}.",
965           "accion": "actualizar", "destino": None, "pestana": None}
966
967
968 def _normalizar_nombre(t: str) -> str:
969     """Sin quitar la puntuación, una pausa natural al hablar ("Cebolla,
970     cabeza blanca.") deja una coma que rompe la comparación por
971     substring contra "CEBOLLA CABEZONA BLANCA" (sin coma) - confirmado
972     con una captura real donde por eso no lograba resolver una
973     ambigüedad pendiente aunque dijo el nombre correcto. Mismo arreglo
974     que normalizar() ya tiene en servicios/conciliacion.py, por la
975     misma razón (el reconocimiento de voz también suele cerrar la
976     frase con un punto que nadie dijo)."""
977     t = str(t).lower().strip()
978     t = re.sub(r"[.,;:;!¿?]+", " ", t)
979     t = re.sub(r"\s+", " ", t).strip()
980     return "".join(c for c in unicodedata.normalize("NFD", t)
981                   if unicodedata.category(c) != "Mn")
982
983
984 _COLA_RELLENO = re.compile(
985     r"\s*,?\s*(?:por\s+favor|porfa\w*|gracias|si\s+puedes|si\s+es\s+posible)"
986     r"\s*[.,!¿?]*\s*$", re.IGNORECASE)
987
988
989 def _quitar_relleno(texto: str) -> str:
990     """Los comandos de una sola frase («crea un usuario llamado...»,
991     «crea una receta llamada..., con... de...») quedan anclados al
992     FINAL del texto para extraer nombre/cantidad/ingrediente con
993     precisión - pero cualquier cortesía dicha al final ("...por favor",
994     "...gracias", cosa que la gente realmente dice) caía fuera de ese
995     ancla y el campo capturado se tragaba esas palabras de más (un
996     usuario terminaba llamándose "juan perfil auxiliar por favor" en
997     vez de "juan"). Se quita ANTES de intentar cualquier comando, en un
998     bucle por si hay más de una coletilla seguida ("por favor, gracias")."""
999     anterior = None
1000     t = texto
1001     while anterior != t:
1002         anterior = t
1003         t = _COLA_RELLENO.sub("", t)
1004     return t
1005
1006
1007 def _cambiar_estado_usuario(texto: str, u: Usuario) -> dict | None:
1008     """Activa/desactiva a <nombre> desde Gestión de usuarios - mismo
1009     patrón determinístico que el modo sin conexión (sin pasar por
1010     Gemini, así no depende de su cuota ni arriesga un cambio real por
1011     una mala interpretación del modelo). Reusa las mismas reglas que ya
1012     protegen el PUT /api/usuarios/{id}: nadie se desactiva a sí mismo
1013     por voz, y solo un administrador puede tocar a otra persona.
1014     "Quita" es sinónimo de desactivar aquí, pero "quítale la BODEGA X a

```



```

1015 Luis" no debe desactivar a Luis por accidente solo porque comparte
1016 el verbo - de ahí la exclusión explícita cuando se menciona una
1017 bodega, que es harina de otro costal (_asignar_bodega_por_voz)."""
1018 if (u.perfil != "auditor" or _MODOSINCONEXION.search(texto)
1019     or re.search(r"\bbodega\b", texto, re.IGNORECASE)):
1020     return None
1021 if _ACTIVAR.search(texto):
1022     nuevo = True
1023 elif _DESACTIVAR.search(texto):
1024     nuevo = False
1025 else:
1026     return None
1027 t = _normalizar_nombre(texto)
1028 with Sesion() as s:
1029     candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
1030     encontrado = next(
1031         (c for c in candidatos
1032          if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t)),
1033         None)
1034     if not encontrado:
1035         return None
1036     if bool(encontrado.activo) == nuevo:
1037         estado = "activo" if nuevo else "inactivo"
1038         return {"respuesta_hablada": f"{encontrado.nombre.capitalize()} ya estaba {estado}.",
1039               "accion": "actualizar", "destino": None, "pestaña": None}
1040     encontrado.activo = int(nuevo)
1041     nombre = encontrado.nombre
1042     s.commit()
1043 if not nuevo:
1044     try:
1045         _cliente_cognito().admin_user_global_sign_out(
1046             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre)
1047     except Exception as e:
1048         print(f"[cognito] no se pudo cerrar la sesion de {nombre}: {e}")
1049     estado = "activado" if nuevo else "desactivado"
1050     registrar(u, "USUARIO", f"{nombre} {estado} por voz", "ok")
1051     return {"respuesta_hablada": f"Listo, {nombre.capitalize()} quedó {estado}.",
1052           "accion": "actualizar", "destino": None, "pestaña": None}
1053
1054
1055 # Cuando alguien encadena dos órdenes en una sola frase ("...con 2 kg
1056 # de papa, agrega 300 g de cebolla a la receta sopa" - confirmado con
1057 # una captura real), el último grupo capturado (.+? anclado al final
1058 # con $) se tragaba TODO lo que seguía, incluida la segunda orden
1059 # completa, como si fuera parte del dato ("no encontré «papa, agrega
1060 # 300 g de cebolla a la receta sopa» en el catálogo"). Este sufijo se
1061 # detiene apenas aparece una coma seguida de otro verbo reconocido, y
1062 # descarta el resto - la primera orden se cumple bien, y la segunda
1063 # queda pendiente de decirse aparte.
1064 _CORTE_ORDEN_ENCADENADA = (
1065     r"(?:,\s*(?:cre[ae]\w*|agr[eé]ga|añad|elimin|borr|cambia|as[ii]gn|qu[ii]t|"
1066     r"muestra|enseñ|a|oculta|act[ii]v|desact[ii]v)\w*.*)?")
1067 _CREAR_USUARIO = re.compile(
1068     r"\b(?:cre[ae]\w*\s+)?(?:un\s+|una\s+)?(?:nuev[oa]\s+)?usuario\s+llamad[oa]\s+(?P<nombre>.+?)"
1069     r"(?:\s+(?:de\s+|con\s+)?perfil\s+(?P<perfil>auxiliar|auditor|administrador))?"
1070     r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[\.!]?s*$", re.IGNORECASE)
1071 _INTENCION_CREAR_USUARIO = re.compile(
1072     r"\b(cre[ae]\w*|necesito|quiero)\b.*\busuarios?\b\b\busuario\s+llamad[oa]\b",
1073     re.IGNORECASE)
1074
1075
1076 def _crear_usuario_por_voz(texto: str, u: Usuario) -> dict | None:
1077     """Crea un usuario llamado <nombre> perfil <auxiliar/auditor> -
1078     determinístico, sin pasar por Gemini. A diferencia de
1079     activar/desactivar (que busca un nombre YA existente), aquí no hay
1080     nada contra qué validar el nombre, así que se exige un formato de
1081     frase concreto en vez de adivinar de cualquier forma - más fácil de
1082     aprender a decir que arriesgar un nombre mal extraído. Sin perfil
1083     dicho, queda auxiliar (el caso más común y el de menor alcance).
1084
1085     Decir solo "crea un usuario" (sin nombre) coincidía por casualidad
1086     con la navegación a la pestaña Gestión de usuarios (por la palabra
1087     "usuario") - si ya estaba en esa pestaña, no pasaba nada visible y
1088     parecía que el agente no hacía nada. Ahora, si se nota la intención

```

```

1089 de crear pero falta el nombre, se explica el formato exacto en vez
1090 de caer en la navegación por casualidad.""
1091 if u.perfil != "auditor":
1092     return None
1093 m = _CREAR_USUARIO.search(texto.strip())
1094 if not m:
1095     if (_INTENCION_CREAR_USUARIO.search(texto)
1096         and not _ES_PREGUNTA_PESTANA.search(texto)):
1097         return {"respuesta_hablada": "Para crear un usuario diga el nombre completo así: "
1098             "«crea un usuario llamado» y el nombre, «perfil» y "
1099             "auxiliar o administrador.",
1100             "accion": None, "destino": None, "pestana": None}
1101     return None
1102 nombre = re.sub(r"[.,;:!\?]+$", "", m.group("nombre")).strip().lower()
1103 if not nombre:
1104     return None
1105 perfil_dicho = (m.group("perfil") or "").lower()
1106 perfil = "auditor" if perfil_dicho in ("auditor", "administrador") else "auxiliar"
1107 with Sesion() as s:
1108     if s.query(Usuario).filter_by(nombre=nombre).first():
1109         return {"respuesta_hablada": f"Ya existe un usuario llamado {nombre.capitalize()}.",
1110             "accion": None, "destino": None, "pestana": None}
1111     pin_generado = secrets.token_urlsafe(6) + "1Aa" # cumple la politica de Cognito
1112     # Cognito exige un correo real para crear la cuenta (no lo pide la
1113     # frase por voz) - se usa un correo institucional predecible; el
1114     # administrador lo puede corregir despues desde Ajustes si no es
1115     # el correcto. La cuenta en si SI queda usable de una: el PIN
1116     # dictado aqui es real, no un marcador de posicion.
1117     correo = f"{nombre.replace(' ', '.')}@colsubsidio.com"
1118     nuevo = Usuario(nombre=nombre, perfil=perfil, correo=correo)
1119     s.add(nuevo)
1120     s.commit()
1121     s.refresh(nuevo)
1122     nuevo.codigo = f"CS-{48000 + nuevo.id}"
1123     s.commit()
1124 if not _crear_usuario_cognito(nombre, correo, pin_generado):
1125     # la fila local ya quedo creada (igual que en POST /api/usuarios) -
1126     # se avisa por voz en vez de anunciar un PIN que en realidad no
1127     # sirve para entrar.
1128     registrar(u, "USUARIO", f"Usuario {nombre} creado ({perfil}) por voz, "
1129         f"pero fallo en Cognito", "alerta")
1130     return {"respuesta_hablada": f"Creé a {nombre.capitalize()} en el sistema, pero no "
1131         "pude generar su acceso. Créelo de nuevo desde Gestión "
1132         "de usuarios.",
1133         "accion": "actualizar", "destino": None, "pestana": None}
1134 registrar(u, "USUARIO", f"Usuario {nombre} creado ({perfil}) por voz, "ok")
1135 return {"respuesta_hablada": f"Listo, creé a {nombre.capitalize()} como {perfil}. "
1136     f"Su clave temporal es {pin_generado} - dígasela para que "
1137     "la cambie en Mi perfil.",
1138     "accion": "actualizar", "destino": None, "pestana": None}
1139
1140
1141 _VERBOS_GENERAR_REPORTE = re.compile(
1142     r"\b(gener[ae]|generar|exporta|exportar|crea|crear|descarga|descargar|saca|sacar)\b",
1143     re.IGNORECASE)
1144 _REP_DIFERENCIAS = re.compile(r"diferencia", re.IGNORECASE)
1145 _REP_ANALISIS = re.compile(r"an[aa]lisis|consumo", re.IGNORECASE)
1146 _REP_CONSOLIDADO = re.compile(r"consolidado", re.IGNORECASE)
1147 _REP_TABLERO = re.compile(r"tablero", re.IGNORECASE)
1148 _REP_DETALLE_BODEGA = re.compile(r"detalle\s+d?el?\s+(la\s+)?bodega|detalle\s+de\s+almac[eé]n", re.IGNORECASE)
1149 _REP_TRAZA_ARCHIVO = re.compile(r"trazabilidad", re.IGNORECASE)
1150
1151
1152 def _generar_reporte_por_voz(texto: str, u: Usuario) -> dict | None:
1153     """Genera el consolidado / "exporta las diferencias" / "exporta el
1154     análisis de consumo" desde Reportes - mismo patrón determinístico
1155     que Ajustes (sin pasar por Gemini): llama la MISMA función que ya
1156     usa el botón correspondiente, así el resultado es idéntico al de
1157     darle clic, solo que solo un administrador puede pedirlo por voz."""
1158     if u.perfil != "auditor" or not _VERBOS_GENERAR_REPORTE.search(texto):
1159         return None
1160     if _REP_DIFERENCIAS.search(texto):
1161         d = reporte_diferencias(formato="xlsx", u=u)
1162         # archivo/titulo_archivo (ademas de "actualizar") - para que el

```

```

1163     # panel de vista previa muestre este archivo de una, igual que ya
1164     # pasa al darle clic al boton "Generar diferencias" (que llama a
1165     # previsualizar() apenas termina). Sin esto, pedirlo por voz lo
1166     # generaba pero la vista previa se quedaba vacia hasta un clic
1167     # manual en la tarjeta.
1168     return {"respuesta_hablada": f"Listo, generé las diferencias: {d['filas']} filas en "
1169           f"{d['bodegas_con_descuadre']} bodegas con descuadre.",
1170           "accion": "actualizar", "destino": None, "pestana": "consolidado",
1171           "archivo": d["archivo"], "titulo_archivo": "Diferencias por bodega"}
1172 if _REP_ANALISIS.search(texto):
1173     # a diferencia del consolidado/diferencias (que solo generan y
1174     # dejan una vista previa - descargar es un clic aparte, igual por
1175     # voz que a mano), el botón "Exportar análisis" Sí descarga de
1176     # una vez: sin "descargar_reporte" aquí, pedirlo por voz creaba
1177     # el archivo en el servidor pero nunca llegaba al dispositivo -
1178     # la persona no notaba nada distinto de antes de pedirlo.
1179     d = exportar_analisis(formato="xlsx", dias=30, u=u)
1180     return {"respuesta_hablada": "Listo, exporté el análisis de consumo de los últimos 30 días.",
1181           "accion": "descargar_reporte", "destino": None, "pestana": "análisis",
1182           "archivo": d["archivo"]}
1183 if _REP_CONSOLIDADO.search(texto):
1184     d = reporte(formato="xlsx", u=u)
1185     return {"respuesta_hablada": f"Listo, generé el consolidado: {d['filas']} filas.",
1186           "accion": "actualizar", "destino": None, "pestana": "consolidado",
1187           "archivo": d["archivo"], "titulo_archivo": "Consolidado para My Inventory"}
1188 return None
1189
1190
1191 def _datos_reportes_narrados(u: Usuario) -> dict:
1192     """Los archivos recientes (misma fuente que la lista de Reportes) y el
1193     análisis de consumo de los últimos 30 días, ya en frases listas para
1194     hablar - los usan tanto el contexto completo que recibe Gemini como
1195     las respuestas determinísticas de _responder_reportes_por_voz."""
1196     recientes = reportes_recientes(u=u)
1197     analisis = api_analisis(dias=30, u=u)
1198
1199     def _fila(a):
1200         extra = f", {a['filas']} filas" if a["filas"] is not None else ""
1201         return f"{a['titulo']} ({a['formato']}, hoy {a['hora']}{extra})"
1202
1203     texto_recientes = ("todavía no se ha generado ningún archivo" if not recientes else
1204           "; ".join(_fila(x) for x in recientes[:5]))
1205     ultimo_consolidado = next((x for x in recientes if x["titulo"] == "Consolidado para My Inventory"), None)
1206     ultimas_diferencias = next((x for x in recientes if x["titulo"] == "Diferencias por bodega"), None)
1207     subtil = analisis["subutilizados"]
1208     texto_subutil = ("sin insumos subutilizados en el período" if not subtil else
1209           "; ".join(f"{s['nombre'].title():} sobran {s['sobra']} kg "
1210                 f"({s['sobrepedido_pct']}% de sobrepedido, en {s['veces']} servicios)"
1211                 for s in subtil[:5]))
1212     return {"recientes": recientes, "análisis": analisis, "texto_recientes": texto_recientes,
1213           "ultimo_consolidado": ultimo_consolidado, "ultimas_diferencias": ultimas_diferencias,
1214           "texto_subutil": texto_subutil}
1215
1216
1217 _REPORTES_PREGUNTAS = [
1218     (re.compile(r"\bcu[aá]nt\w*\s+(filas\b.*\b[uú]ltimo\s+consolidado\b|"
1219           r"\bfilas\s+tiene\s+el\s+consolidado\b", re.IGNORECASE),
1220     lambda d: (f"El último consolidado tiene {d['ultimo_consolidado']['filas']} filas, "
1221           f"generado hoy {d['ultimo_consolidado']['hora']}."
1222           if d["ultimo_consolidado"] else "Todavía no se ha generado ningún consolidado.")),
1223     (re.compile(r"\bcu[aá]nt\w*\s+(filas\bodegas\b.*\bdiferencias\b|"
1224           r"\bbodegas\s+tienen\s+descuadre\b", re.IGNORECASE),
1225     lambda d: (f"Las últimas diferencias exportadas tienen {d['ultimas_diferencias']['filas']} "
1226           f"filas, generadas hoy {d['ultimas_diferencias']['hora']}."
1227           if d["ultimas_diferencias"] else "Todavía no se han exportado diferencias por bodega.")),
1228     (re.compile(r"\bqu[eé]\s+archivos\b.*\bgenerad\w*\b|barchivos\s+(recientes|generados)\b",
1229           re.IGNORECASE),
1230     lambda d: f"Archivos recientes: {d['texto_recientes']}.",
1231     (re.compile(r"\bcu[aá]nto\b.*\bpidi[oó]\b|\bpedido\s+total\b|\bpedido\s+en\s+el\s+per[ií]odo\b",
1232           re.IGNORECASE),
1233     lambda d: f"En el período se pidieron {d['análisis']['pedido_total']} kilogramos, en "
1234           f"{d['análisis']['servicios_perodo']} servicios."),
1235     (re.compile(r"\bcu[aá]nto\b.*\bus[oó]\b|\baprovechamiento\b|\busado\s+realmente\b",
1236           re.IGNORECASE),

```

```

1237     lambda d: f"Se usaron realmente {d['analisis']['usado_total']} kilogramos, el "
1238         f"{d['analisis']['aprovechamiento']}% de lo pedido.",
1239 (re.compile(r"\bahorro\s+potencial\b", re.IGNORECASE),
1240     lambda d: f"El ahorro potencial es de {d['analisis']['ahorro_potencial']} kilogramos al "
1241         f"mes si se ajustan las recetas."),
1242 (re.compile(r"\bcu[aá]nt\w*\s+insumos\s+(est[aá]n\s+)?subutilizad\w*|\binsumos\s+subutilizados\b",
1243     re.IGNORECASE),
1244     lambda d: f"Hay {len(d['analisis']['subutilizados'])} insumos subutilizados: "
1245         f"{d['texto_subutil']}."),
1246 (re.compile(r"\b(m[aá]s|mayor)\s+sobrepedido\b|\brecomendaci[oó]n\s+(del\s+)?agente\b",
1247     re.IGNORECASE),
1248     lambda d: (f"El insumo con más sobrepedido es "
1249         f"{d['analisis']['subutilizados'][0]['nombre'].title()}: sobra en "
1250         f"{d['analisis']['subutilizados'][0]['veces']} servicios, "
1251         f"{d['analisis']['subutilizados'][0]['sobrepedido_pct']}% más de lo que se usa."
1252         if d['analisis']['subutilizados'] else
1253         "No hay insumos subutilizados en el período.")),
1254 ]
1255
1256
1257 def _responder_reportes_por_voz(texto: str, u: Usuario) -> dict | None:
1258     """Preguntas frecuentes de Reportes (ambas pestañas), resueltas sin
1259     pasar por Gemini - mismo motivo que _responder_panel_por_voz: estas
1260     frases ya se ofrecen como garantizadas en el desplegable de ejemplos."""
1261     if u.perfil != "auditor":
1262         return None
1263     for patron, respuesta in _REPORTES_PREGUNTAS:
1264         if patron.search(texto):
1265             return {"respuesta_hablada": respuesta(_datos_reportes_narrados(u)),
1266                 "accion": None, "destino": None, "pestana": None}
1267     return None
1268
1269
1270 _VER_ARCHIVO_VERBO = re.compile(
1271     r"\b(mu[eé]stra\w*|ense[ñn]a\w*|[aá]bre\w*)\b", re.IGNORECASE)
1272 _VER_ARCHIVO_PALABRA = re.compile(r"\barchivo\b", re.IGNORECASE)
1273
1274
1275 def _previsualizar_reporte_por_voz(texto: str, u: Usuario) -> dict | None:
1276     """Muéstrame el detalle de bodega" / "ábreme el archivo de diferencias" -
1277     lo mismo que darle clic a una tarjeta de "Archivos generados", sin
1278     tocar la pantalla.
1279
1280     Diferencias, tablero, detalle de bodega y trazabilidad NO son el
1281     nombre de ninguna pestaña de esta pantalla, así que decir su nombre
1282     con un verbo de "mostrar" no compite con nada más - se abren directo,
1283     sin exigir la palabra "archivo". Consolidado y análisis SÍ son
1284     pestañas reales («muéstrame el consolidado» ya significa cambiar de
1285     pestaña, ver _navegar_pestana_por_voz), así que para esos dos la
1286     palabra "archivo" sigue siendo obligatoria para desambiguar - sin
1287     ella, decir solo el nombre de la pestaña dejaría de poder cambiarla.
1288     Antes la palabra "archivo" era obligatoria para los seis tipos por
1289     igual, así que pedir "muéstrame el detalle de bodega" (la frase más
1290     natural, sin decir "archivo") no abría nada y caía al agente
1291     general, que solo podía describir el archivo en palabras y mandar a
1292     dar clic a mano - no a abrirlo de verdad.
1293
1294     Cubre los cinco tipos que puede mostrar la lista (antes solo distinguía
1295     diferencias y consolidado; el resto - tablero, detalle de bodega,
1296     análisis, trazabilidad - siempre caía al último archivo generado, que
1297     con frecuencia NO era el que se pedía)."""
1298     if u.perfil != "auditor" or not _VER_ARCHIVO_VERBO.search(texto):
1299         return None
1300     recientes = reportes_recientes(u=u)
1301     if not recientes:
1302         return {"respuesta_hablada": "Todavía no se ha generado ningún archivo.",
1303             "accion": None, "destino": None, "pestana": "consolidado"}
1304     tiene_archivo = bool(_VER_ARCHIVO_PALABRA.search(texto))
1305     if _REP_DIFERENCIAS.search(texto):
1306         elegido = next((x for x in recientes if x["titulo"] == "Diferencias por bodega"), None)
1307     elif _REP_TABLERO.search(texto):
1308         elegido = next((x for x in recientes if x["titulo"] == "Estado del tablero"), None)
1309     elif _REP_DETALLE_BODEGA.search(texto):
1310         elegido = next((x for x in recientes if x["titulo"] == "Detalle de bodega"), None)

```

```

1311 elif _REP_TRAZA_ARCHIVO.search(texto):
1312     elegido = next((x for x in recientes if x["titulo"] == "Registro de trazabilidad"), None)
1313 elif _REP_ANALISIS.search(texto) and tiene_archivo:
1314     elegido = next((x for x in recientes if x["titulo"] == "Análisis de consumo"), None)
1315 elif _REP_CONSOLIDADO.search(texto) and tiene_archivo:
1316     elegido = next((x for x in recientes if x["titulo"] == "Consolidado para My Inventory"), None)
1317 elif tiene_archivo:
1318     elegido = recientes[0]
1319 else:
1320     return None
1321 if not elegido:
1322     return {"respuesta_hablada": "No encontré ese archivo entre los generados recientemente.",
1323            "accion": None, "destino": None, "pestaña": "consolidado"}
1324 detalle_filas = f", {elegido['filas']} filas" if elegido["filas"] is not None else ""
1325 return {"respuesta_hablada": f"Aquí está: {elegido['titulo']}{detalle_filas}. "
1326        "¿Desea descargarlo?",
1327        "accion": "previsualizar_reporte", "destino": None, "pestaña": "consolidado",
1328        "archivo": elegido["archivo"], "titulo_archivo": elegido["titulo"]}
1329
1330
1331 # Palabras que identifican cada pestaña, para navegar entre ellas sin
1332 # pasar por Gemini - la cuota de Gemini ha fallado justo en este tipo de
1333 # orden ("llévame a gestión de usuarios"), dejando la pestaña sin
1334 # cambiar aunque la respuesta hablada dijera lo contrario.
1335 _PALABRAS_PESTANA = {
1336     "ajustes": {
1337         "config": re.compile(r"configuraci[ó]n", re.IGNORECASE),
1338         "usuarios": re.compile(r"usuarios?", re.IGNORECASE),
1339         "recetas": re.compile(r"recetas?", re.IGNORECASE),
1340         "traza": re.compile(r"trazabilidad", re.IGNORECASE),
1341     },
1342     "reportes": {
1343         "consolidado": re.compile(r"consolidado", re.IGNORECASE),
1344         "analisis": re.compile(r"an[ál]lisis|consumo", re.IGNORECASE),
1345     },
1346     "panel": {
1347         "resumen": re.compile(r"resumen", re.IGNORECASE),
1348         "alertas": re.compile(r"\balertas\b", re.IGNORECASE),
1349     },
1350     "auditoria": {
1351         "recuento": re.compile(r"recuento|recont[ae]o|\bcierre\b", re.IGNORECASE),
1352         "aprobaciones": re.compile(r"aprobaciones", re.IGNORECASE),
1353         "pedidos": re.compile(r"pedidos", re.IGNORECASE),
1354         "alertas": re.compile(r"\balertas\b", re.IGNORECASE),
1355     },
1356 }
1357 _VERBOS_IR_PESTANA = re.compile(
1358     r"\b(ir|ll[eé]va(me)?|vamos|ve|mu[eé]stra(me)?|[\aá]bre(me)?|ens[eé][\ñn]a(me)?)\b",
1359     re.IGNORECASE)
1360 _ES_PREGUNTA_PESTANA = re.compile(r"[\i?]|bcu[\aá]nt|\bqu[eé]\b|bcu[\aá]l|\bpor qu[eé]", re.IGNORECASE)
1361
1362
1363 def _navegar_pestaña_por_voz(vista: str, texto: str) -> dict | None:
1364     """Llévame a gestión de usuarios" (o cualquier pestaña de Ajustes,
1365     Reportes o Panel) - determinístico, sin pasar por Gemini, para que
1366     la navegación entre pestañas no dependa de su cuota. También navega
1367     si lo dicho es prácticamente el nombre de la pestaña solo ("Gestión
1368     de usuarios.", sin decir "llévame a" - la persona lo dice tal como
1369     lo lee en la pantalla), siempre que sea corto y no suene a pregunta
1370     ("¿cuántos usuarios hay?" no debe navegar, solo responderse)."""
1371     palabras = _PALABRAS_PESTANA.get(vista)
1372     if not palabras:
1373         return None
1374     tiene_verbo = bool(_VERBOS_IR_PESTANA.search(texto))
1375     limpio = re.sub(r"[.,;:!\i?]+", "", texto.strip())
1376     es_solo_el_nombre = (not _ES_PREGUNTA_PESTANA.search(texto)
1377                        and 0 < len(limpio.split()) <= 4)
1378     if not tiene_verbo and not es_solo_el_nombre:
1379         return None
1380     for clave, patron in palabras.items():
1381         if patron.search(texto):
1382             etiqueta = _PESTANAS_ASISTENTE[vista][clave]
1383             return {"respuesta_hablada": f"Listo, vamos a {etiqueta}.",
1384                    "accion": "navegar", "destino": vista, "pestaña": clave}

```

```

1385     return None
1386
1387
1388 # Los 4 "accesos rápidos" de Inicio ("Iniciar un conteo", "Ver el tablero",
1389 # "Continuar auditoria"/"Hacer un pedido", "Generar reporte"/"Legalizar
1390 # servicio") - las mismas tarjetas que ya se navegan con un clic.
1391 _ACCESOS_INICIO = {
1392     "conteo": re.compile(r"\bconteo\b", re.IGNORECASE),
1393     "bodegas": re.compile(r"\btablero\b|\bbodegas\b", re.IGNORECASE),
1394     "auditoria": re.compile(r"auditor[ii]a", re.IGNORECASE),
1395     "pedido": re.compile(r"\bpedidos?\b", re.IGNORECASE),
1396     "reportes": re.compile(r"reporte", re.IGNORECASE),
1397     "legalizacion": re.compile(r"legaliza", re.IGNORECASE),
1398 }
1399 _VERBOS_ACCESO_INICIO = re.compile(
1400     r"\b(inicia|iniciar|continu[ae]|continuar|genera|generar|legaliza|legalizar|hacer|haga)\b",
1401     re.IGNORECASE)
1402
1403
1404 def _acceso_rapido_inicio_por_voz(texto: str, u: Usuario) -> dict | None:
1405     """Decir el nombre de una tarjeta de "¿Qué desea hacer?" navega directo
1406     a ella - determinístico, sin pasar por Gemini: se confirmó con
1407     pruebas que a veces decidía contestar en vez de navegar aunque la
1408     frase mencionara el destino tal cual ("iniciar un conteo" se quedaba
1409     en Inicio). Mismo filtro que _navegar_pestana_por_voz para no
1410     robarle la frase a las preguntas de KPI de esta pantalla («¿cuántas
1411     bodegas tengo?» debe contestarse, no navegar a Bodegas)."""
1412     tiene_verbo = bool(_VERBOS_IR_PESTANA.search(texto)) or bool(_VERBOS_ACCESO_INICIO.search(texto))
1413     limpio = re.sub(r"[.,;:!?;]+", "", texto.strip())
1414     es_solo_el_nombre = (not _ES_PREGUNTA_PESTANA.search(texto)
1415                          and 0 < len(limpio.split()) <= 4)
1416     if not tiene_verbo and not es_solo_el_nombre:
1417         return None
1418     for destino, patron in _ACCESOS_INICIO.items():
1419         if not patron.search(texto):
1420             continue
1421         if destino in _SOLO_AUDITOR_ASISTENTE and u.perfil != "auditor":
1422             continue
1423         etiqueta = _DESTINOS_ASISTENTE[destino]
1424         return {"respuesta_hablada": f"Vamos a {etiqueta.lower()}.",
1425               "accion": "navegar", "destino": destino, "pestana": None}
1426     return None
1427
1428
1429 _APROBAR_ITEM = re.compile(r"\bapru[eé]b\w*|\baprobar\b", re.IGNORECASE)
1430 _RECHAZAR_ITEM = re.compile(r"\brech[aá]z\w*", re.IGNORECASE)
1431
1432
1433 def _resolver_aprobacion_por_voz(texto: str, u: Usuario) -> dict | None:
1434     """Aprueba/rechaza <nombre> desde Auditoría (pestaña Aprobaciones) -
1435     mismo patrón determinístico que Ajustes/Reportes: busca el nombre
1436     completo entre las aprobaciones pendientes (como frase exacta, no
1437     palabra por palabra - "aprueba costilla de res" no debe aprobar
1438     cualquier cosa que contenga "res") y llama la MISMA función que ya
1439     usa el botón, solo administrador."""
1440     if u.perfil != "auditor":
1441         return None
1442     if _APROBAR_ITEM.search(texto):
1443         verbo = "aprobar"
1444     elif _RECHAZAR_ITEM.search(texto):
1445         verbo = "rechazar"
1446     else:
1447         return None
1448     t = _normalizar_nombre(texto)
1449     with Sesion() as s:
1450         pendientes = s.query(Aprobacion).filter_by(estado="pendiente").all()
1451         encontrada = next(
1452             (a for a in pendientes
1453              if re.search(rf"\b{re.escape(_normalizar_nombre(a.nombre))}\b", t)),
1454             None)
1455         if not encontrada:
1456             return None
1457         aid, nombre = encontrada.id, encontrada.nombre
1458         if verbo == "aprobar":

```



```

1459         aprobar(aprobar_id=aid, u=u)
1460         return {"respuesta_hablada": f"Listo, aprobé {nombre.title()}. Ya entra al catálogo oficial.",
1461                "accion": "actualizar", "destino": None, "pestana": None}
1462     rechazar(aprobar_id=aid, u=u)
1463     return {"respuesta_hablada": f"Listo, rechacé {nombre.title()}.",
1464            "accion": "actualizar", "destino": None, "pestana": None}
1465
1466
1467     _APROBAR_RECHAZAR_PEDIDO = re.compile(
1468         r"\b(apru[eé]b\w*|rech[aá]z\w*)\b.*\bpedido\b|\bpedido\b.*\b(apru[eé]b\w*|rech[aá]z\w*)\b",
1469         re.IGNORECASE)
1470
1471
1472     def _resolver_pedido_pendiente_por_voz(texto: str, u: Usuario) -> dict | None:
1473         """Aprueba/rechaza el pedido de <persona o plato> desde Auditoría
1474         (pestaña Pedidos pendientes) - mismo patrón que _resolver_aprobacion_por_voz,
1475         pero exige la palabra "pedido" para no competir con aprobar/rechazar
1476         un producto o bodega nueva, que es otra lista con los mismos verbos."""
1477         if u.perfil != "auditor" or not _APROBAR_RECHAZAR_PEDIDO.search(texto):
1478             return None
1479         if _APROBAR_ITEM.search(texto):
1480             aprobar_bool = True
1481         elif _RECHAZAR_ITEM.search(texto):
1482             aprobar_bool = False
1483         else:
1484             return None
1485         t = _normalizar_nombre(texto)
1486         pendientes = pedidos_pendientes(u=u)
1487         if not pendientes:
1488             return {"respuesta_hablada": "No hay pedidos pendientes de aprobación.",
1489                    "accion": None, "destino": None, "pestana": "pedidos"}
1490         encontrado = next(
1491             (p for p in pendientes
1492              if (p["persona"] and p["persona"] != "-"
1493                  and re.search(rf"\b{re.escape(_normalizar_nombre(p['persona']))}\b", t))
1494              or re.search(rf"\b{re.escape(_normalizar_nombre(p['plato']))}\b", t)),
1495             None)
1496         if not encontrado:
1497             return None
1498         resolver_pedido(numero_pedido=encontrado["numero_pedido"],
1499                        datos=ResolverPedidoIn(aprobar=aprobar_bool), u=u)
1500         verbo_pasado = "aprobé" if aprobar_bool else "rechacé"
1501         return {"respuesta_hablada": f"Listo, {verbo_pasado} el pedido de "
1502                f"{encontrado['persona'].title(): {encontrado['plato']}.",
1503                "accion": "actualizar", "destino": None, "pestana": "pedidos"}
1504
1505
1506     _RESOLVER_ALERTA = re.compile(r"\bresuelve\w*|\bresolver\b", re.IGNORECASE)
1507
1508
1509     def _resolver_alerta_por_voz(texto: str, u: Usuario) -> dict | None:
1510         """Resuelve la alerta de <artículo o bodega> desde Auditoría
1511         (pestaña Bandeja de alertas) - mismo patrón determinístico, busca
1512         entre las alertas abiertas por el artículo o la bodega que mencionan."""
1513         if u.perfil != "auditor" or not _RESOLVER_ALERTA.search(texto):
1514             return None
1515         abiertas = ver_alertas(resueltas=0, u=u)
1516         if not abiertas:
1517             return {"respuesta_hablada": "No hay alertas abiertas.",
1518                    "accion": None, "destino": None, "pestana": "alertas"}
1519         t = _normalizar_nombre(texto)
1520         encontrada = next(
1521             (a for a in abiertas
1522              if (a["articulo"] and re.search(rf"\b{re.escape(_normalizar_nombre(a['articulo']))}\b", t))
1523              or (a["bodega"] and re.search(rf"\b{re.escape(_normalizar_nombre(a['bodega']))}\b", t))),
1524             None)
1525         if not encontrada:
1526             return None
1527         resolver_alerta(alerta_id=encontrada["id"], u=u)
1528         detalle = encontrada["articulo"] or encontrada["bodega"] or encontrada["titulo"]
1529         return {"respuesta_hablada": f"Listo, resolví la alerta de {detalle.title()}.",
1530                "accion": "actualizar", "destino": None, "pestana": "alertas"}
1531
1532

```

```

1533 _VERBOS_AUDITAR_BODEGA = re.compile(r"\b(abr[ae]|abrir|audit\w*)\b", re.IGNORECASE)
1534
1535
1536 def _abrir_bodega_para_auditoria_por_voz(texto: str, u: Usuario) -> dict | None:
1537     """Audita <bodega> / "abre <bodega>" desde Auditoría (pestaña
1538     Recuento ciego y cierre) - selecciona esa bodega, lo mismo que darle
1539     clic a "Abrir" en su tarjeta. A propósito NO llega hasta firmar ni
1540     cerrar: eso sigue siendo manual, es la garantía de un cierre con
1541     doble firma que nadie dispare sin querer con la voz."""
1542     if u.perfil != "auditor" or not _VERBOS_AUDITAR_BODEGA.search(texto):
1543         return None
1544     from servicios.conciliacion import buscar_bodega
1545     with Sesion() as s:
1546         candidatas_ids = {b.id for b in s.query(Bodega)
1547                             .filter(Bodega.estado.in_([ "en_auditoria", "cerrada" ])).all()}
1548         if not candidatas_ids:
1549             return {"respuesta_hablada": "No hay ninguna bodega lista para recuento ciego o "
1550                     "cierre en este momento.",
1551                     "accion": None, "destino": None, "pestaña": "recuento"}
1552         bodega = buscar_bodega(s, texto, candidatas_ids)
1553     if not bodega:
1554         return None
1555     return {"respuesta_hablada": f"Vamos a auditar {bodega.nombre_oficial.title()}.",
1556           "accion": "navegar", "destino": "auditoria", "pestaña": "recuento",
1557           "bodega_auditar": bodega.nombre_oficial}
1558
1559
1560 def _datos_auditoria_narrados(u: Usuario) -> dict:
1561     return {"resumen_alertas": resumen_alertas(u=u), "pedidos": pedidos_pendientes(u=u),
1562           "aprobaciones": listar_aprobaciones(estado="pendiente", u=u)}
1563
1564
1565 _AUDITORIA_PREGUNTAS = [
1566     (re.compile(r"\bcu[aá]nt\w*\s+alertas\b.*\babiertas\b|\balertas\s+abiertas\b", re.IGNORECASE),
1567      lambda d: f"Hay {d['resumen_alertas']['abiertas']} alertas abiertas."),
1568     (re.compile(r"\bcu[aá]nt\w*\b.*\bresolvieron\s+hoy\b|\bresueltas\s+hoy\b", re.IGNORECASE),
1569      lambda d: f"Se resolvieron {d['resumen_alertas']['resueltas_hoy']} alertas hoy."),
1570     (re.compile(r"\btipo\w*\s+medio\b", re.IGNORECASE),
1571      lambda d: (f"El tiempo medio de resolución es de "
1572                 f"{d['resumen_alertas']['tiempo_medio_min']} minutos."
1573                 if d["resumen_alertas"]["tiempo_medio_min"] is not None else
1574                 "Todavía no se ha resuelto ninguna alerta hoy para calcular el tiempo medio.")),
1575     (re.compile(r"\bcu[aá]nt\w*\s+pedidos\b.*\bpendientes\b", re.IGNORECASE),
1576      lambda d: (f"Hay {len(d['pedidos'])} pedidos pendientes de aprobación."
1577                 if d["pedidos"] else "No hay pedidos pendientes de aprobación.")),
1578     (re.compile(r"\bcu[aá]nt\w*\s+aprobaciones\b", re.IGNORECASE),
1579      lambda d: (f"Hay {len(d['aprobaciones'])} aprobaciones pendientes: " +
1580                 "; ".join(a["nombre"].title() for a in d["aprobaciones"][:6]) + "."
1581                 if d["aprobaciones"] else "No hay aprobaciones pendientes.")),
1582 ]
1583
1584
1585 def _responder_auditoria_por_voz(texto: str, u: Usuario) -> dict | None:
1586     if u.perfil != "auditor":
1587         return None
1588     for patron, respuesta in _AUDITORIA_PREGUNTAS:
1589         if patron.search(texto):
1590             return {"respuesta_hablada": respuesta(_datos_auditoria_narrados(u)),
1591                   "accion": None, "destino": None, "pestaña": None}
1592     return None
1593
1594
1595 _UNIDADES_RECETA = r"kilos?|kg|litros?|lt|unidades?|und?|gramos?|gr"
1596 # Cuando alguien encadena dos órdenes en una sola frase (...con 2 kg
1597 # de papa, agrega 300 g de cebolla a la receta sopa" - confirmado con
1598 # una captura real), el último grupo capturado (.+? anclado al final
1599 # con $) se tragaba TODO lo que seguía, incluida la segunda orden
1600 # completa, como si fuera parte del dato ("no encontré «papa, agrega
1601 # 300 g de cebolla a la receta sopa» en el catálogo"). Este sufijo se
1602 # detiene apenas aparece una coma seguida de otro verbo reconocido, y
1603 # descarta el resto - la primera orden se cumple bien, y la segunda
1604 # queda pendiente de decirse aparte (tal como ya se explica: "puede
1605 # agregarle más ingredientes diciendo «agrega...»").
1606 # El verbo "crea/crear" es OPCIONAL: el reconocimiento de voz a veces

```



```

1607 # recorta la primera palabra si la persona empieza a hablar apenas
1608 # hace clic en el micrófono, antes de que el navegador esté listo -
1609 # confirmado con una captura real ("Una receta llamada sopa,
1610 # rendimiento, cuatro porciones." sin "crea" al inicio). El resto de
1611 # la frase (receta llamada... rendimiento... con... de...) ya es lo
1612 # bastante específico para no confundirse con una pregunta suelta.
1613 _CREAR_RECETA = re.compile(
1614     r"\b(?:cre[ae]\w*\s+)?(?:una\s+)?(?:nuev[oa]\s+)?receta\s+llamad[ao]\s+(?P<nombre>.+)"
1615     r"\s*[,\y]*\s*(?:con\s+)?rendimiento\s+(?:de\s+)?(?P<rend>.+)\s*porciones?"
1616     r"\s*[,\y]*\s*con\s+(?P<cant>.+)\s*"
1617     rf"(?:{_UNIDADES_RECETA})?\s*de\s+(?P<ing>.+)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
1618     re.IGNORECASE)
1619 _AGREGAR_INGREDIENTE = re.compile(
1620     r"\b(?:agr[eé]ga\w*|a[ñn]ad\w*)\s+(?P<cant>.+)\s*"
1621     rf"(?:{_UNIDADES_RECETA})?\s*de\s+(?P<ing>.+)"
1622     r"\s+a\s+la\s+receta\s+(?P<receta>.+)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
1623     re.IGNORECASE)
1624 _ELIMINAR_RECETA = re.compile(
1625     r"\b(elimin\w*|borr\w*)\s+(?:la\s+)?receta\s+(?P<receta>.+)\s*"
1626     + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
1627     re.IGNORECASE)
1628
1629
1630 # Memoria viva de una sola pregunta pendiente por persona: "crea una
1631 # receta... con papa" -> ambiguo -> la persona contesta solo "papa
1632 # criolla" en el siguiente turno, y eso completa la orden original en
1633 # vez de tener que repetirla entera. Mismo patron que ESTADOS en
1634 # agente/orquestador.py (memoria en RAM, se pierde si el servidor se
1635 # reinicia - aceptable aqui: en el peor caso, toca repetir la orden
1636 # completa, que es lo que ya se pedia antes de esto). Expira sola
1637 # despues de _VENCIMIENTO_AMBIGUEDAD segundos para no quedar pegada a
1638 # una respuesta que en realidad era para otra cosa, dicha mucho despues.
1639 _PENDIENTE_AMBIGUEDAD: dict[int, dict] = {}
1640 _VENCIMIENTO_AMBIGUEDAD = 90
1641
1642
1643 def _elegir_ingredientes_sin_ambigüedad(dicho: str) -> dict | None:
1644     """buscar_articulo() puntúa por cobertura de palabras: una consulta
1645     de una sola palabra genérica ("papa") saca 100% de cobertura contra
1646     CUALQUIER artículo que la contenga, sin importar cuánto más tenga
1647     ese nombre alrededor - confirmado con una captura real donde "papa"
1648     empataba a 100 de confianza con "EMPANADA DE PAPA Y CARNE X50 GR",
1649     "PAPA A LA FRANCESA" y "PAPA CABELLO DE ANGEL" por igual, y se
1650     quedaba con el primero de la lista sin ningún criterio real. Igual
1651     que buscar_bodega ya hace con bodegas ambiguas: mejor decir "sea
1652     más específico" que adivinar mal en silencio y crear una receta con
1653     el ingrediente equivocado."""
1654     from servicios.conciliacion import buscar_articulo, normalizar
1655     # límite alto: buscar_articulo por defecto solo devuelve los 3
1656     # mejores, y con un empate a 100 (el caso ambiguo que nos interesa
1657     # aquí) esos 3 son un subconjunto ARBITRARIO del catálogo real -
1658     # confirmado con una captura real donde salían "Empanada De Papa Y
1659     # Carne" y "Papa A La Francesa" en vez de las papas de verdad
1660     # (Criolla, Sabanera, Pastusa) que sí existen en el catálogo. Con
1661     # más candidatos a la vista, se puede priorizar los nombres más
1662     # simples (probablemente el ingrediente crudo) al armar la lista.
1663     candidatos = buscar_articulo(dicho, limite=20)
1664     if not candidatos or candidatos[0]["confianza"] < 60:
1665         return {"error": f"No encontré «{dicho}» en el catálogo. Dígallo como aparece "
1666             "en la etiqueta."}
1667     tope = candidatos[0]["confianza"]
1668     empatados = [c for c in candidatos if c["confianza"] == tope]
1669     if len(empatados) > 1:
1670         # Un empate NO es ambiguo si uno de ellos es el nombre EXACTO
1671         # dicho ("papa criolla" == "PAPA CRIOLLA") - eso gana solo,
1672         # aunque "PAPA CRIOLLA PRECOCIDA" haya empatado en puntaje.
1673         clave = normalizar(dicho)
1674         exacto = next((c for c in empatados if normalizar(c["nombre"]) == clave), None)
1675         if exacto:
1676             return {"articulo": exacto}
1677         # De los empatados, primero los nombres más cortos: un
1678         # ingrediente crudo ("PAPA CRIOLLA") es más probable que sea lo
1679         # que la persona quiso decir que un producto elaborado con
1680         # media frase de más ("EMPANADA DE PAPA Y CARNE X50 GR").

```

```

1681     empatados.sort(key=lambda c: len(c["nombre"].split()))
1682     mostrados = empatados[:5]
1683     opciones = ", ".join(c["nombre"].title() for c in mostrados)
1684     return {"error": f"«{dicho}» es ambiguo, encontré varios parecidos: {opciones}. "
1685             "Dígalo más específico, como aparece completo en la etiqueta.",
1686             "opciones": mostrados}
1687     return {"articulo": candidatos[0]}
1688
1689
1690 def _buscar_receta_por_nombre(s, nombre_dicho: str):
1691     """Una coincidencia EXACTA siempre gana primero: con dos recetas
1692     "Sopa" y "Sopa de la casa", decir "Sopa de la casa" no debe caer en
1693     "Sopa" solo porque su nombre es substring - eso pasaba antes (se
1694     quedaba con la PRIMERA que calzara, sin importar el orden de
1695     especificidad) y edita/borraba la receta equivocada en silencio."""
1696     t = _normalizar_nombre(nombre_dicho)
1697     recetas = s.query(Receta).all()
1698     exacta = next((r for r in recetas if _normalizar_nombre(r.nombre) == t), None)
1699     if exacta:
1700         return exacta
1701     return next(
1702         (r for r in recetas if re.search(rf"\b{re.escape(_normalizar_nombre(r.nombre))}\b", t)
1703          or re.search(rf"\b{re.escape(t)}\b", _normalizar_nombre(r.nombre))),
1704         None)
1705
1706
1707 _INTENCION_CREAR_RECETA = re.compile(
1708     r"\b(cre[ae]\w*|necesito|quiero)\b.*\brejetas?\b|\brequeta\s+llamad[ao]\b",
1709     re.IGNORECASE)
1710
1711
1712 def _completar_crear_receta(nombre: str, rend: float, cant: float, articulo: dict,
1713                             u: Usuario) -> dict:
1714     with Sesion() as s:
1715         if s.query(Receta).filter(Receta.nombre.ilike(nombre)).first():
1716             return {"respuesta_hablada": f"Ya existe una receta llamada {nombre}.",
1717                     "accion": None, "destino": None, "pestaña": None}
1718         r = Receta(nombre=nombre, rendimiento=int(rend), preparacion="")
1719         s.add(r)
1720         s.flush()
1721         s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=articulo["codigo"],
1722                                 cantidad_por_porcion=float(cant)))
1723         s.commit()
1724     registrar(u, "RECETA", f"Receta creada: {nombre} (1 ingrediente) por voz", "ok")
1725     return {"respuesta_hablada": f"Listo, creé la receta {nombre} con {articulo['nombre'].title()}. "
1726             "Puede agregarle más ingredientes diciendo «agrega... a la "
1727             f"receta {nombre}», o editarla desde la pantalla.",
1728             "accion": "actualizar", "destino": None, "pestaña": None}
1729
1730
1731 def _crear_receta_por_voz(texto: str, u: Usuario) -> dict | None:
1732     """Crea una receta llamada <nombre>, rendimiento <N> porciones, con
1733     <cantidad> de <ingrediente> - determinístico, con el mismo criterio
1734     que crear usuario: un formato de frase concreto, en vez de adivinar
1735     de cualquier forma un nombre y una lista de ingredientes que no
1736     existen todavía contra qué validar. El ingrediente SÍ se busca
1737     contra el catálogo real (misma función que usa Conteo/Pedido) - si
1738     no lo encuentra con confianza suficiente, no crea nada a medias.
1739     Decir solo "crea una receta" (sin los demás datos) coincidía por
1740     casualidad con la navegación a la pestaña Recetas - mismo arreglo
1741     que crear usuario."""
1742     if u.perfil != "auditor":
1743         return None
1744     m = _CREAR_RECETA.search(texto.strip())
1745     if not m:
1746         if (_INTENCION_CREAR_RECETA.search(texto)
1747             and not _ES_PREGUNTA_PESTANA.search(texto)):
1748             return {"respuesta_hablada": "Para crear una receta diga: «crea una receta "
1749                     "llamada» el nombre, «rendimiento» el número de "
1750                     "porciones, «con» la cantidad «de» el primer "
1751                     "ingrediente.",
1752                     "accion": None, "destino": None, "pestaña": None}
1753         return None
1754     nombre = re.sub(r"[.,;:!?;]+$", "", m.group("nombre")).strip()

```

```

1755     if not nombre:
1756         return None
1757     # El rendimiento y la cantidad se dicen casi siempre en palabras
1758     # ("cuatro porciones", no "4 porciones" - así habla la gente de
1759     # verdad), así que se interpretan con el mismo parser de números
1760     # que ya usa Conteo/Pedido, no un \d+ que solo entiende dígitos.
1761     rend = _numero_de_texto(m.group("rend"))
1762     cant = _numero_de_texto(m.group("cant"))
1763     if rend is None or cant is None:
1764         return {"respuesta_hablada": "No entendí el rendimiento o la cantidad. Dígalos en "
1765             "número, por ejemplo «cuatro porciones».",
1766             "accion": None, "destino": None, "pestana": None}
1767     elegido = _elegir_ingrediente_sin_ambigüedad(m.group("ing"))
1768     if "error" in elegido:
1769         if "opciones" in elegido:
1770             _PENDIENTE_AMBIGÜEDAD[u.id] = {
1771                 "tipo": "crear_receta", "nombre": nombre, "rend": rend, "cant": cant,
1772                 "opciones": elegido["opciones"], "ts": ahora()}
1773             return {"respuesta_hablada": elegido["error"],
1774                 "accion": None, "destino": None, "pestana": None}
1775     return _completar_crear_receta(nombre, rend, cant, elegido["articulo"], u)
1776
1777
1778 _INTENCION_AGREGAR_INGREDIENTE = re.compile(
1779     r"\b(agr[eé]ga\w*|a[ññ]ad\w*)\b.*\brequeta\b", re.IGNORECASE)
1780
1781
1782 def _completar_agregar_ingrediente(cantidad_nueva: float, receta_dicha: str,
1783     articulo: dict, u: Usuario) -> dict | None:
1784     with Sesion() as s:
1785         r = _buscar_receta_por_nombre(s, receta_dicha)
1786         if not r:
1787             return None
1788         lineas = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
1789         ya_tiene = next((li for li in lineas if li.articulo_codigo == articulo["codigo"]), None)
1790         if ya_tiene:
1791             ya_tiene.cantidad_por_porcion = cantidad_nueva
1792         else:
1793             s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=articulo["codigo"],
1794                 cantidad_por_porcion=cantidad_nueva))
1795             s.commit()
1796             nombre_receta = r.nombre
1797             registrar(u, "RECETA", f"Receta editada: {nombre_receta} (ingrediente agregado por voz)", "ok")
1798             return {"respuesta_hablada": f"Listo, agregué {articulo['nombre'].title()} a la receta {nombre_receta}.",
1799                 "accion": "actualizar", "destino": None, "pestana": None}
1800
1801
1802 def _agregar_ingrediente_por_voz(texto: str, u: Usuario) -> dict | None:
1803     """Agrégle <cantidad> de <ingrediente> a la receta <nombre>" -
1804     determinístico. Reusa la misma búsqueda de artículo que crear
1805     receta, y conserva los ingredientes que ya tenía (PUT reemplaza
1806     toda la lista, así que se lee la receta primero)."""
1807     if u.perfil != "auditor":
1808         return None
1809     m = _AGREGAR_INGREDIENTE.search(texto.strip())
1810     if not m:
1811         # "preparación" en la frase es de _agregar_preparacion_por_voz,
1812         # no de esta - sin la exclusión, "agrega la preparación a la
1813         # receta X: ..." caía aquí primero (comparte "agrega...receta")
1814         # y nunca le daba la oportunidad a la función correcta.
1815         if (_INTENCION_AGREGAR_INGREDIENTE.search(texto)
1816             and not re.search(r"preparaci[oó]n", texto, re.IGNORECASE)
1817             and not _ES_PREGUNTA_PESTANA.search(texto)):
1818             return {"respuesta_hablada": "Para agregar un ingrediente diga: «agrega» la "
1819                 "cantidad «de» el ingrediente «a la receta» el "
1820                 "nombre de la receta.",
1821                 "accion": None, "destino": None, "pestana": None}
1822         return None
1823     cantidad_nueva = _numero_de_texto(m.group("cant"))
1824     if cantidad_nueva is None:
1825         return {"respuesta_hablada": "No entendí la cantidad. Dígala en número, por "
1826             "ejemplo «trescientos gramos».",
1827             "accion": None, "destino": None, "pestana": None}
1828     elegido = _elegir_ingrediente_sin_ambigüedad(m.group("ing"))

```

```

1829     if "error" in elegido:
1830         if "opciones" in elegido:
1831             _PENDIENTE_AMBIGUEDAD[u.id] = {
1832                 "tipo": "agregar_ingredient", "cant": cantidad_nueva,
1833                 "receta": m.group("receta"), "opciones": elegido["opciones"],
1834                 "ts": ahora()}
1835             return {"respuesta_hablada": elegido["error"],
1836                     "accion": None, "destino": None, "pestana": None}
1837     return _completar_agregar_ingredient(cantidad_nueva, m.group("receta"),
1838                                         elegido["articulo"], u)
1839
1840
1841 def _resolver_ambiguedad_pendiente(texto: str, u: Usuario) -> dict | None:
1842     """Si la última respuesta de esta persona fue "es ambiguo, encontré
1843     Papa Sabanera, Papa Criolla, Papa Pastusa..." y en este turno dice
1844     solo "papa pastusa" (sin repetir la orden completa), esto retoma la
1845     orden original con ese ingrediente ya resuelto. Se revisa AL FINAL
1846     de la cadena de comandos de Ajustes, después de que crear/agregar
1847     ya tuvieron su oportunidad de matchear la frase completa - así una
1848     orden repetida entera (en vez de solo el nombre corto) sigue
1849     tomando el camino normal, con sus datos frescos, no los guardados
1850     aquí de la vez anterior."""
1851     pendiente = _PENDIENTE_AMBIGUEDAD.get(u.id)
1852     if not pendiente:
1853         return None
1854     if (ahora() - pendiente["ts"]).total_seconds() > _VENCIMIENTO_AMBIGUEDAD:
1855         del _PENDIENTE_AMBIGUEDAD[u.id]
1856         return None
1857     t = _normalizar_nombre(texto)
1858     elegido = next(
1859         (c for c in pendiente["opciones"]
1860          if re.search(rf"\b{re.escape(_normalizar_nombre(c['nombre']))}\b", t)
1861          or re.search(rf"\b{re.escape(t)}\b", _normalizar_nombre(c["nombre"]))),
1862         None)
1863     if not elegido:
1864         return None
1865     del _PENDIENTE_AMBIGUEDAD[u.id]
1866     if pendiente["tipo"] == "crear_receta":
1867         return _completar_crear_receta(pendiente["nombre"], pendiente["rend"],
1868                                       pendiente["cant"], elegido, u)
1869     return _completar_agregar_ingredient(pendiente["cant"], pendiente["receta"], elegido, u)
1870
1871
1872 def _eliminar_receta_por_voz(texto: str, u: Usuario) -> dict | None:
1873     """Elimina la receta <nombre>" - determinístico, busca por nombre
1874     completo entre las recetas existentes, igual que aprobaciones. NUNCA
1875     borra en el mismo turno que se pide: pregunta primero y solo borra si
1876     el siguiente turno confirma (ver _resolver_confirmacion_eliminar_receta)
1877     - es una acción irreversible (se lleva también los ingredientes) y un
1878     "elimina la receta X" mal escuchado, o una intención mal adivinada por
1879     Gemini, no debe bastar para borrarla sin confirmación explícita, igual
1880     que ya exige el botón equivalente en pantalla (window.confirm)."""
1881     if u.perfil != "auditor":
1882         return None
1883     m = _ELIMINAR_RECETA.search(texto.strip())
1884     if not m:
1885         return None
1886     with Sesion() as s:
1887         r = _buscar_receta_por_nombre(s, m.group("receta"))
1888         if not r:
1889             return None
1890         rid, nombre = r.id, r.nombre
1891         _PENDIENTE_AMBIGUEDAD[u.id] = {"tipo": "eliminar_receta", "receta_id": rid,
1892                                       "nombre": nombre, "ts": ahora()}
1893     return {"respuesta_hablada": f"¿Confirma que elimina la receta {nombre}? ",
1894           "accion": None, "destino": None, "pestana": None}
1895
1896
1897
1898 def _resolver_confirmacion_eliminar_receta(texto: str, u: Usuario) -> dict | None:
1899     """El sí/no al "¿confirma que elimina la receta X?" de arriba. Se
1900     revisa ANTES que cualquier otra orden de esta pantalla: un "sí" o un
1901     "no" sueltos no deben caer en ningún otro reconocedor mientras esta
1902     confirmación sigue pendiente."""

```

```

1903 pendiente = _PENDIENTE_AMBIGUEDAD.get(u.id)
1904 if not pendiente or pendiente.get("tipo") != "eliminar_receta":
1905     return None
1906 if (ahora() - pendiente["ts"]).total_seconds() > _VENCIMIENTO_AMBIGUEDAD:
1907     del _PENDIENTE_AMBIGUEDAD[u.id]
1908     return None
1909 t = texto.lower()
1910 palabras = t.replace(", ", " ").replace(".", " ").split()
1911 dice_si = (any(p in ("si", "sí", "claro", "listo", "dale", "correcto", "confirmo") for p in palabras)
1912            or "confirmo" in t or "así es" in t or "asi es" in t)
1913 dice_no = "no" in palabras or "mejor no" in t or "cancela" in t
1914 if not (dice_si or dice_no):
1915     return None
1916 del _PENDIENTE_AMBIGUEDAD[u.id]
1917 if dice_no:
1918     return {"respuesta_hablada": f"Entendido, no elimino la receta {pendiente['nombre']}.",
1919            "accion": None, "destino": None, "pestana": None}
1920 with Sesion() as s:
1921     r = s.get(Receta, pendiente["receta_id"])
1922     if r is None:
1923         return {"respuesta_hablada": "Esa receta ya no existe.",
1924                "accion": None, "destino": None, "pestana": None}
1925     nombre = r.nombre
1926     s.query(RecetaIngrediente).filter_by(receta_id=r.id).delete()
1927     s.delete(r)
1928     s.commit()
1929 registrar(u, "RECETA", f"Receta eliminada: {nombre} por voz (confirmada)", "ok")
1930 return {"respuesta_hablada": f"Listo, eliminé la receta {nombre}.",
1931        "accion": "actualizar", "destino": None, "pestana": None}
1932
1933
1934 _QUITAR_INGREDIENTE = re.compile(
1935     r"\b(?:qu[ií]ta|w*|elimin\w*|borr\w*)\s+(?:el\s+)?ingrediente\w*\s+(?P<ing>.+)"
1936     r"\s+de\s+la\s+receta\s+(?P<receta>.+)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
1937     re.IGNORECASE)
1938
1939
1940 def _quitar_ingredientes_por_voz(texto: str, u: Usuario) -> dict | None:
1941     """Quita/elimina el ingrediente <ingrediente> de la receta <nombre> -
1942     mismo botón «Editar» de Recetas, pero solo para sacar UN ingrediente
1943     (no reemplaza toda la lista). Busca el ingrediente entre los que YA
1944     tiene esa receta, no contra todo el catálogo - así "quita la
1945     cebolla" no confunde una cebolla que ni siquiera está en esa
1946     receta. Nunca deja una receta sin ingredientes (la misma regla que
1947     ya exige el botón)."""
1948     if u.perfil != "auditor":
1949         return None
1950     m = _QUITAR_INGREDIENTE.search(texto.strip())
1951     if not m:
1952         return None
1953     with Sesion() as s:
1954         r = _buscar_receta_por_nombre(s, m.group("receta"))
1955         if not r:
1956             return None
1957         lineas = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
1958         t_ing = _normalizar_nombre(m.group("ing"))
1959         objetivo = None
1960         nombre_articulo = None
1961         for li in lineas:
1962             art = s.get(Articulo, li.articulo_codigo)
1963             if not art:
1964                 continue
1965             nombre_art = _normalizar_nombre(art.nombre_oficial)
1966             if (re.search(rf"\b{re.escape(nombre_art)}\b", t_ing)
1967                 or re.search(rf"\b{re.escape(t_ing)}\b", nombre_art)):
1968                 objetivo, nombre_articulo = li, art.nombre_oficial
1969             break
1970         if not objetivo:
1971             return {"respuesta_hablada": f"{r.nombre.title()} no tiene un ingrediente llamado "
1972                    f"«{m.group('ing')}».",
1973                    "accion": None, "destino": None, "pestana": None}
1974         if len(lineas) <= 1:
1975             return {"respuesta_hablada": f"No puedo quitar {nombre_articulo.title()}: "
1976                    f"{r.nombre.title()} se quedaría sin ingredientes.",

```

```

1977         "accion": None, "destino": None, "pestana": None}
1978     s.delete(objetivo)
1979     nombre_receta = r.nombre
1980     s.commit()
1981     registrar(u, "RECETA", f"Receta editada: {nombre_receta} ({nombre_articulo} quitado por voz)", "ok")
1982     return {"respuesta_hablada": f"Listo, quité {nombre_articulo.title()} de la receta {nombre_receta}.",
1983           "accion": "actualizar", "destino": None, "pestana": None}
1984
1985
1986 _CAMBIAR_RENDIMIENTO = re.compile(
1987     r"\bcambia\w*\s+el\s+rendimient\w*\s+de\s+la\s+receta\s+(?P<receta>.+?)"
1988     r"\s+a\s+(?P<rend>.+?)\s*porcion\w*\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
1989     re.IGNORECASE)
1990
1991
1992 def _cambiar_rendimiento_por_voz(texto: str, u: Usuario) -> dict | None:
1993     """Cambia el rendimiento de la receta <nombre> a <N> porciones" -
1994     otro campo editable del botón «Editar» de Recetas (además de los
1995     ingredientes, que se agregan/quitan aparte, y la preparación, que
1996     se dicta con _agregar_preparacion_por_voz)."""
1997     if u.perfil != "auditor":
1998         return None
1999     m = _CAMBIAR_RENDIMIENTO.search(texto.strip())
2000     if not m:
2001         return None
2002     nuevo = _numero_de_texto(m.group("rend"))
2003     if nuevo is None:
2004         return {"respuesta_hablada": "No entendí el rendimiento. Dígallo en número, por "
2005               "ejemplo «seis porciones». ",
2006               "accion": None, "destino": None, "pestana": None}
2007     nuevo = int(nuevo)
2008     with Sesion() as s:
2009         r = _buscar_receta_por_nombre(s, m.group("receta"))
2010         if not r:
2011             return None
2012         if r.rendimiento == nuevo:
2013             return {"respuesta_hablada": f"{r.nombre.title()} ya rendía {nuevo} porciones.",
2014                   "accion": "actualizar", "destino": None, "pestana": None}
2015         r.rendimiento = nuevo
2016         nombre = r.nombre
2017         s.commit()
2018     registrar(u, "RECETA", f"Receta editada: {nombre} (rendimiento -> {nuevo} por voz)", "ok")
2019     return {"respuesta_hablada": f"Listo, {nombre.title()} ahora rinde {nuevo} porciones.",
2020           "accion": "actualizar", "destino": None, "pestana": None}
2021
2022
2023 _AGREGAR_PREPARACION = re.compile(
2024     r"\b(?:agr[eé]ga\w*|cambia\w*|pon\w*|dicta\w*|escrib\w*)\s+(?:la\s+)?preparaci[oó]n"
2025     r"\s+(?:de\s+|a\s+)?(?:la\s+receta\s+)?(?P<receta>.+?)\s*[:,\s]*(?P<pasos>.+?)\s*$",
2026     re.IGNORECASE)
2027 _INTENCION_AGREGAR_PREPARACION = re.compile(
2028     r"\b(?:agr[eé]ga\w*|cambia\w*|pon\w*|dicta\w*|escrib\w*)\b.*\bpreparaci[oó]n\b",
2029     re.IGNORECASE)
2030
2031
2032 def _agregar_preparacion_por_voz(texto: str, u: Usuario) -> dict | None:
2033     """Agrega la preparación a la receta <nombre>: <pasos>" - a
2034     diferencia de nombre/ingrediente/rendimiento (frases cortas), aquí
2035     todo lo que sigue a los dos puntos ES el dato, tal cual, sin cortar
2036     en la primera coma o punto (los pasos de cocina naturalmente tienen
2037     varias oraciones: "primero pele las papas. Luego sofía...") - por
2038     eso esta es la ÚNICA función de Recetas que NO usa
2039     _CORTE_ORDEN_ENCADENADA. Reemplaza la preparación completa (como el
2040     textarea del botón «Editar»), no la suma a lo que ya había."""
2041     if u.perfil != "auditor":
2042         return None
2043     m = _AGREGAR_PREPARACION.search(texto.strip())
2044     if not m:
2045         if (_INTENCION_AGREGAR_PREPARACION.search(texto)
2046             and not _ES_PREGUNTA_PESTANA.search(texto)):
2047             return {"respuesta_hablada": "Para dictar la preparación diga: «agrega la "
2048                   "preparación a la receta» el nombre, dos puntos, y "
2049                   "los pasos - por ejemplo «agrega la preparación a "
2050                   "la receta Sopa: pele las papas, sofía la cebolla, "

```

```

2051         "y cocine todo junto veinte minutos».",
2052         "accion": None, "destino": None, "pestana": None}
2053     return None
2054     pasos = re.sub(r"[.,;:!?i]+$", "", m.group("pasos")).strip()
2055     if not pasos:
2056         return None
2057     with Sesion() as s:
2058         r = _buscar_receta_por_nombre(s, m.group("receta"))
2059         if not r:
2060             return None
2061         r.preparacion = pasos
2062         nombre = r.nombre
2063         s.commit()
2064     registrar(u, "RECETA", f"Receta editada: {nombre} (preparación dictada por voz)", "ok")
2065     return {"respuesta_hablada": f"Listo, guardé la preparación de la receta {nombre}.",
2066           "accion": "actualizar", "destino": None, "pestana": None}
2067
2068
2069 def _cambiar_perfil_por_voz(texto: str, u: Usuario) -> dict | None:
2070     """Cambia el perfil de <nombre> a auxiliar/administrador" - mismo
2071     botón «Editar» de Gestión de usuarios, solo el campo perfil (el
2072     correo no se toca por voz: un correo mal transcrito rompería el
2073     contacto sin que se note). Llama la MISMA función que usa el botón
2074     (editar_usuario), así que hereda su regla de no poder cambiarse el
2075     propio rol - excluida además desde la búsqueda del nombre."""
2076     if u.perfil != "auditor" or not re.search(r"\bperfil\b", texto, re.IGNORECASE):
2077         return None
2078     m = re.search(r"\b(auxiliar|auditor|administrador)\b", texto, re.IGNORECASE)
2079     if not m:
2080         if not _ES_PREGUNTA_PESTANA.search(texto):
2081             return {"respuesta_hablada": "Diga a qué perfil: «cambia el perfil de» el "
2082                   "nombre «a» auxiliar o administrador.",
2083                   "accion": None, "destino": None, "pestana": None}
2084         return None
2085     perfil_nuevo = "auditor" if m.group(1).lower() in ("auditor", "administrador") else "auxiliar"
2086     t = _normalizar_nombre(texto)
2087     with Sesion() as s:
2088         candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
2089         encontrado = next(
2090             (c for c in candidatos
2091              if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t)),
2092             None)
2093     if not encontrado:
2094         return {"respuesta_hablada": "No encontré a esa persona. Diga el nombre completo "
2095               "tal como aparece en la lista.",
2096               "accion": None, "destino": None, "pestana": None}
2097     etiqueta = "administrador" if perfil_nuevo == "auditor" else "auxiliar"
2098     if encontrado.perfil == perfil_nuevo:
2099         return {"respuesta_hablada": f"{encontrado.nombre.capitalize()} ya era {etiqueta}.",
2100               "accion": "actualizar", "destino": None, "pestana": None}
2101     eid, nombre = encontrado.id, encontrado.nombre
2102     editar_usuario(usuario_id=eid, datos=EditarUsuarioIn(perfil=perfil_nuevo), u=u)
2103     return {"respuesta_hablada": f"Listo, {nombre.capitalize()} ahora es {etiqueta}.",
2104           "accion": "actualizar", "destino": None, "pestana": None}
2105
2106
2107 _ASIGNAR_BODEGA = re.compile(
2108     r"\bas[ií]gn\w*\s+(?:le\s+)?(?:la\s+)?bodega\s+(?<bodega>.+?)\s+a\s+(?<nombre>.+?)"
2109     r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[!]?s*$", re.IGNORECASE)
2110 _QUITAR_BODEGA = re.compile(
2111     r"\bqu[ií]t\w*\s+(?:le\s+)?(?:la\s+)?bodega\s+(?<bodega>.+?)\s+a\s+(?<nombre>.+?)"
2112     r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[!]?s*$", re.IGNORECASE)
2113
2114
2115 def _asignar_bodega_por_voz(texto: str, u: Usuario) -> dict | None:
2116     """Asigne/quítale la bodega <bodega> a <nombre>" - mismo botón
2117     «Asignar bodegas», pero sumando o quitando SOLO la bodega dicha en
2118     vez de reemplazar toda la lista (que es lo que hace el botón, que
2119     parte de las bodegas ya marcadas): así una orden de voz nunca borra
2120     por accidente asignaciones hechas por otro medio. El nombre de la
2121     bodega se busca con el mismo buscador que «abra <bodega>» en
2122     Conteo, así que entiende apodos parciales igual ("kiosco taquilla"
2123     encuentra "KIOSCO TAQUILLA AYB")."""
2124     if u.perfil != "auditor":

```



```

2125         return None
2126     m = _ASIGNAR_BODEGA.search(texto.strip())
2127     quitar = False
2128     if not m:
2129         m = _QUITAR_BODEGA.search(texto.strip())
2130         quitar = True
2131     if not m:
2132         # OJO: el verbo debe terminar justo en "a/ale/ar" (con \b después)
2133         # para no confundirse con el SUSTANTIVO "asignación" - sin ese
2134         # límite, "muéstrame la asignación por bodega" (que es la orden
2135         # de abrir el panel, no de asignar nada) caía aquí por error,
2136         # porque "asign" es raíz común de las dos palabras.
2137         if (re.search(r"\bbodega\b", texto, re.IGNORECASE)
2138             and re.search(r"\bas[íí]gn(?:a(?:le)?|ar)\b|\bqu[íí]t(?:a(?:le)?|ar)\b",
2139                          texto, re.IGNORECASE)
2140             and not _ES_PREGUNTA_PESTANA.search(texto)):
2141             return {"respuesta_hablada": "Diga: «asígnale/quítale la bodega» el nombre de "
2142                  "la bodega «a» el nombre de la persona.",
2143                  "accion": None, "destino": None, "pestana": None}
2144     return None
2145 from servicios.conciliacion import buscar_bodega
2146 t_nombre = _normalizar_nombre(m.group("nombre"))
2147 with Sesion() as s:
2148     bodega = buscar_bodega(s, m.group("bodega"), None)
2149     if not bodega:
2150         return {"respuesta_hablada": f"No encontré una bodega llamada «{m.group('bodega')}».",
2151                 "accion": None, "destino": None, "pestana": None}
2152     candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
2153     persona = next(
2154         (c for c in candidatos
2155          if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t_nombre)),
2156         None)
2157     if not persona:
2158         return {"respuesta_hablada": "No encontré a esa persona. Diga el nombre completo "
2159              "tal como aparece en la lista.",
2160                 "accion": None, "destino": None, "pestana": None}
2161     actuales = {a.bodega_id for a in
2162                 s.query(AsignacionBodega).filter_by(usuario_id=persona.id).all()}
2163     ya_tenia = bodega.id in actuales
2164     if quitar and not ya_tenia:
2165         return {"respuesta_hablada": f"{persona.nombre.capitalize()} no tenía asignada "
2166              f"{bodega.nombre_oficial.title()}.",
2167                 "accion": "actualizar", "destino": None, "pestana": None}
2168     if not quitar and ya_tenia:
2169         return {"respuesta_hablada": f"{persona.nombre.capitalize()} ya tenía asignada "
2170              f"{bodega.nombre_oficial.title()}.",
2171                 "accion": "actualizar", "destino": None, "pestana": None}
2172     if quitar:
2173         actuales.discard(bodega.id)
2174     else:
2175         actuales.add(bodega.id)
2176     pid, pnombre, bnombre = persona.id, persona.nombre, bodega.nombre_oficial
2177     nuevos_ids = list(actuales)
2178     asignar_bodegas(usuario_id=pid, body={"bodega_ids": nuevos_ids}, u=u)
2179     verbo = "quité" if quitar else "asigné"
2180     return {"respuesta_hablada": f"Listo, le {verbo} {bnombre.title()} a {pnombre.capitalize()}.",
2181            "accion": "actualizar", "destino": None, "pestana": None}
2182
2183
2184 _QUIEN_TIENE_BODEGA = re.compile(
2185     r"\bqui[eé]n\s+(?:tiene|es\s+responsable\s+de|maneja)\s+(?:la\s+)?bodega\s+"
2186     r"^(?P<bodega>.+)\s*[\!]?*\s*$", re.IGNORECASE)
2187 _BODEGAS_SIN_ASIGNAR = re.compile(r"\bbodegas?\b.*\bsin\s+asignar\b", re.IGNORECASE)
2188
2189
2190 def _consultar_asignacion_bodega_por_voz(texto: str, u: Usuario) -> dict | None:
2191     """Lo mismo que el botón «Ver asignación por bodega», pero como
2192     pregunta hablada en vez de abrir la tabla completa de 54 filas:
2193     "¿quién tiene la bodega <bodega>?" contesta solo esa fila, y
2194     "¿cuántas bodegas sin asignar hay?" resume las que no tiene nadie -
2195     de solo lectura, así que no exige ninguna confirmación extra."""
2196     if u.perfil != "auditor":
2197         return None
2198     if _BODEGAS_SIN_ASIGNAR.search(texto):

```



```

2199         with Sesion() as s:
2200             asignadas = {a.bodega_id for a in s.query(AsignacionBodega).all()}
2201             sin = [b.nombre_oficial for b in s.query(Bodega).all() if b.id not in asignadas]
2202             if not sin:
2203                 return {"respuesta_hablada": "Todas las bodegas tienen al menos una persona asignada.",
2204                         "accion": None, "destino": None, "pestana": None}
2205             listado = ", ".join(n.title() for n in sin[:8])
2206             extra = f" y {len(sin) - 8} más" if len(sin) > 8 else ""
2207             return {"respuesta_hablada": f"{len(sin)} bodegas sin asignar: {listado}{extra}.",
2208                     "accion": None, "destino": None, "pestana": None}
2209         m = _QUIEN_TIENE_BODEGA.search(texto.strip())
2210         if not m:
2211             return None
2212         from servicios.conciliacion import buscar_bodega
2213         with Sesion() as s:
2214             bodega = buscar_bodega(s, m.group("bodega"), None)
2215             if not bodega:
2216                 return {"respuesta_hablada": f"No encontré una bodega llamada «{m.group('bodega')}».",
2217                         "accion": None, "destino": None, "pestana": None}
2218             asignados = [s.get(Usuario, a.usuario_id) for a in
2219                          s.query(AsignacionBodega).filter_by(bodega_id=bodega.id).all()]
2220             nombres = [p.nombre.capitalize() for p in asignados if p]
2221             titulo = bodega.nombre_oficial.title()
2222             if not nombres:
2223                 return {"respuesta_hablada": f"{titulo} no tiene a nadie asignado todavía.",
2224                         "accion": None, "destino": None, "pestana": None}
2225             return {"respuesta_hablada": f"{titulo} está asignada a {'', '.join(nombres)}.",
2226                     "accion": None, "destino": None, "pestana": None}
2227
2228
2229         _ABRIR_COBERTURA = re.compile(
2230             r"\b(mu[eé]stra[w*]ens[eé][ñn]a[w*][aá]bre[w*]ver)\b.*\basignaci[oó]n\b.*\bbodega",
2231             re.IGNORECASE)
2232         _CERRAR_COBERTURA = re.compile(
2233             r"\b(oc[uú]lta[w*]ci[eé]rra[w*]esc[oó]nde[w*])\b.*\basignaci[oó]n\b.*\bbodega",
2234             re.IGNORECASE)
2235
2236
2237     def _alternar_cobertura_por_voz(texto: str, u: Usuario) -> dict | None:
2238         """El botón «Ver/Ocultar asignación por bodega» abre o cierra la
2239         tabla completa de 54 filas (quién tiene cada bodega) - a diferencia
2240         de _consultar_asignacion_bodega_por_voz (que contesta una pregunta
2241         puntual sin tocar la pantalla), esto es la orden de abrir/cerrar el
2242         panel mismo, igual que el clic al botón."""
2243         if u.perfil != "auditor":
2244             return None
2245         if _ABRIR_COBERTURA.search(texto):
2246             return {"respuesta_hablada": "Listo, ahí tiene la asignación por bodega.",
2247                     "accion": "mostrar_cobertura", "destino": None, "pestana": None}
2248         if _CERRAR_COBERTURA.search(texto):
2249             return {"respuesta_hablada": "Listo, la oculto.",
2250                     "accion": "ocultar_cobertura", "destino": None, "pestana": None}
2251         return None
2252
2253
2254         _EXPORTAR_TRAZA = re.compile(
2255             r"\b(export[w*]descarg[w*]genera[w*])\b.*\b(traza|trazabilidad)\b", re.IGNORECASE)
2256
2257
2258     def _exportar_trazabilidad_por_voz(texto: str, u: Usuario) -> dict | None:
2259         """El botón «Exportar» de Registro de trazabilidad exporta con los
2260         filtros (persona/acción/rango) que YA están marcados en pantalla -
2261         esos viven como estado de React en el frontend, no llegan en el
2262         texto de esta orden, así que no se puede armar el archivo aquí en
2263         el backend. En cambio, se avisa con la misma acción genérica que ya
2264         usa mostrar/ocultar cobertura, y el frontend llama a su propia
2265         función exportar() con los filtros que tenga puestos - igual que si
2266         hubiera dado clic al botón."""
2267         if u.perfil != "auditor":
2268             return None
2269         if not _EXPORTAR_TRAZA.search(texto):
2270             return None
2271         return {"respuesta_hablada": "Listo, exportando el registro de trazabilidad con los "
2272                 "filtros que tiene puestos.",

```

```

2273         "accion": "exportar_trazabilidad", "destino": None, "pestaña": None}
2274
2275
2276 _ETIQUETA_RANGO_TRAZA = {"hoy": "hoy", "semana": "en la última semana",
2277                          "mes": "en el último mes", "": "en total"}
2278
2279
2280 def _rango_dicho_traza(texto: str) -> str:
2281     t = texto.lower()
2282     if re.search(r"\bsemana\b", t):
2283         return "semana"
2284     if re.search(r"\bmes\b", t):
2285         return "mes"
2286     if re.search(r"\btodo\b|\bsiempre\b", t):
2287         return ""
2288     return "hoy" # mismo valor por defecto con el que abre la pestaña
2289
2290
2291 _ACCIONES_DE_PERSONA = re.compile(
2292     r"\b(?:[aá]ntas\s+acciones\s+tiene\s+(?<persona>.+?)\s*[.!?]*\s*$", re.IGNORECASE)
2293 _ACCIONES_POR_PERSONA = re.compile(
2294     r"\b(?:acciones\s+por\s+persona\b|\b(?:[aá]ntas\s+acciones\s+tiene\s+cada\s+persona\b|
2295     r"\bresumen\s+de\s+acciones\b", re.IGNORECASE)
2296
2297
2298 def _acciones_por_persona_por_voz(texto: str, u: Usuario) -> dict | None:
2299     """Pedido explícito del usuario: "acciones por persona" contesta un
2300     resumen con cuántas hizo cada quien, y "¿cuántas acciones tiene
2301     <nombre>?" contesta solo por esa persona - los dos de solo lectura,
2302     sin abrir ni cambiar nada en pantalla, así que no hace falta
2303     confirmación. El periodo por defecto es "hoy" (igual que la
2304     pestaña al abrirse); decir "esta semana"/"este mes"/"en total"
2305     lo cambia."""
2306     if u.perfil != "auditor":
2307         return None
2308     m = _ACCIONES_DE_PERSONA.search(texto.strip())
2309     if m:
2310         rango = _rango_dicho_traza(texto)
2311         t_nombre = _normalizar_nombre(m.group("persona"))
2312         with Sesion() as s:
2313             candidatos = s.query(Usuario).all()
2314             persona_obj = next(
2315                 (c for c in candidatos
2316                  if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t_nombre)),
2317                 None)
2318             if not persona_obj:
2319                 return None
2320             total = _filtro_traza(s, persona_obj.nombre, "", rango).count()
2321             nombre = persona_obj.nombre
2322             return {"respuesta_hablada": f"{nombre.capitalize()} tiene {total} acciones "
2323                    f"{_ETIQUETA_RANGO_TRAZA[rango]}.",
2324                    "accion": None, "destino": None, "pestaña": None}
2325     if _ACCIONES_POR_PERSONA.search(texto):
2326         rango = _rango_dicho_traza(texto)
2327         with Sesion() as s:
2328             filas = _filtro_traza(s, "", "", rango).all()
2329             if not filas:
2330                 return {"respuesta_hablada": f"No hay acciones registradas {_ETIQUETA_RANGO_TRAZA[rango]}.",
2331                        "accion": None, "destino": None, "pestaña": None}
2332             conteos: dict[str, int] = {}
2333             for f in filas:
2334                 conteos[f.persona] = conteos.get(f.persona, 0) + 1
2335             ordenado = sorted(conteos.items(), key=lambda par: -par[1])
2336             resumen = ", ".join(f"{p.capitalize()}: {n}" for p, n in ordenado[:8])
2337             extra = f", y {len(ordenado) - 8} personas más" if len(ordenado) > 8 else ""
2338             return {"respuesta_hablada": f"Acciones por persona {_ETIQUETA_RANGO_TRAZA[rango]}: "
2339                    f"{resumen}{extra}.",
2340                    "accion": None, "destino": None, "pestaña": None}
2341     return None
2342
2343
2344 def _resolver_asistente(vista: str, texto: str, u: Usuario) -> dict:
2345     """Lo que responde el agente liviano ante una pregunta suelta o una
2346     orden de navegar - usado tanto por /api/agente/asistente (Inicio,

```

```

2347 Ajustes, Ayuda, Reportes, Panel) como, cuando ni un artículo ni una
2348 bodega coinciden, por el buscador de Bodegas (ver consulta_articulo):
2349 un solo buscador que primero prueba si es un ingrediente, luego si
2350 es una bodega, y si no es ninguna de las dos, cae aquí.""
2351 texto = _quitar_relleno(texto)
2352 if vista == "inicio":
2353     acceso = _acceso_rapido_inicio_por_voz(texto, u)
2354     if acceso:
2355         return acceso
2356 # Desde Bodegas, "abra/siga contando <nombre>" es una orden concreta y
2357 # frecuente - se resuelve aparte, ANTES de Gemini, con la misma
2358 # búsqueda restringida que usa el resto de la app (nunca ofrece una
2359 # bodega ajena a lo asignado). El verbo es obligatorio: sin él, decir
2360 # el nombre de una bodega podría ser una PREGUNTA sobre ella ("¿cómo
2361 # va kiosco taquilla ayb?"), no una orden de ir a contarla.
2362 if vista == "bodegas" and _VERBOS_ABRIR_BODEGA.search(texto):
2363     from servicios.conciliacion import buscar_bodega
2364     with Sesion() as s:
2365         bodega = buscar_bodega(s, texto, _ids_permitidos_para_buscar(s, u))
2366     if bodega:
2367         return {"respuesta_hablada": f"Vamos a Conteo a abrir {bodega.nombre_oficial.title()}.",
2368                 "accion": "navegar", "destino": "conteo", "pestanas": None,
2369                 "bodega": bodega.nombre_oficial}
2370 if vista == "ajustes":
2371     # el sí/no a "¿confirma que elimina la receta X?" tiene prioridad
2372     # sobre cualquier otra cosa: mientras esa confirmación siga
2373     # pendiente, un "sí" o un "no" sueltos son la respuesta a ESO, no
2374     # una orden nueva - revisarlo antes evita que caiga en otro
2375     # reconocedor de la cadena de abajo.
2376     confirmacion = _resolver_confirmacion_eliminar_receta(texto, u)
2377     if confirmacion:
2378         return confirmacion
2379     # identidad (no solo presencia) de lo que había pendiente ANTES
2380     # de este turno - crear_receta/agregar_ingredientes pueden dejar
2381     # una ambigüedad NUEVA en el mismo turno (cambio ya sale
2382     # "truthy" con el mensaje de "es ambiguo...") y esa no se debe
2383     # borrar; solo se limpia la que YA estaba ahí sin que nada de
2384     # este turno la haya tocado.
2385     pendiente_antes = _PENDIENTE_AMBIGUEDAD.get(u.id)
2386     cambio = (_cambiar_modos_sin_conexion(texto, u) or _cambiar_estado_usuario(texto, u)
2387              or _crear_usuario_por_voz(texto, u) or _cambiar_perfil_por_voz(texto, u)
2388              or _asignar_bodega_por_voz(texto, u) or _alternar_cobertura_por_voz(texto, u)
2389              or _consultar_asignacion_bodega_por_voz(texto, u)
2390              or _crear_receta_por_voz(texto, u)
2391              or _agregar_ingredientes_por_voz(texto, u) or _quitar_ingredientes_por_voz(texto, u)
2392              or _cambiar_rendimiento_por_voz(texto, u) or _agregar_preparacion_por_voz(texto, u)
2393              or _eliminar_receta_por_voz(texto, u) or _exportar trazabilidad_por_voz(texto, u)
2394              or _acciones_por_persona_por_voz(texto, u))
2395     if cambio:
2396         # una orden distinta que sí coincidió significa que la
2397         # persona siguió para otra cosa - una pregunta ambigua sin
2398         # contestar de hace un rato no debe "resucitar" más tarde
2399         # por una frase corta que por casualidad se parezca a una
2400         # de esas opciones viejas.
2401         if _PENDIENTE_AMBIGUEDAD.get(u.id) is pendiente_antes:
2402             _PENDIENTE_AMBIGUEDAD.pop(u.id, None)
2403         return cambio
2404     resuelta_pendiente = _resolver_ambigüedad_pendiente(texto, u)
2405     if resuelta_pendiente:
2406         return resuelta_pendiente
2407 if vista == "reportes":
2408     generado = _generar_reporte_por_voz(texto, u)
2409     if generado:
2410         return generado
2411     previsualizado = _previsualizar_reporte_por_voz(texto, u)
2412     if previsualizado:
2413         return previsualizado
2414     respondida = _responder_reportes_por_voz(texto, u)
2415     if respondida:
2416         return respondida
2417 if vista == "auditoria":
2418     resuelta = _resolver_aprobacion_por_voz(texto, u)
2419     if resuelta:
2420         return resuelta

```

```

2421     resuelto_pedido = _resolver_pedido_pendiente_por_voz(texto, u)
2422     if resuelto_pedido:
2423         return resuelto_pedido
2424     resuelta_alerta = _resolver_alerta_por_voz(texto, u)
2425     if resuelta_alerta:
2426         return resuelta_alerta
2427     auditada = _abrir_bodega_para_auditoria_por_voz(texto, u)
2428     if auditada:
2429         return auditada
2430     respondida = _responder_auditoria_por_voz(texto, u)
2431     if respondida:
2432         return respondida
2433 if vista == "panel":
2434     respondida = _responder_panel_por_voz(texto, u)
2435     if respondida:
2436         return respondida
2437 if vista == "perfil":
2438     cambio = (_cambiar_voz_por_voz(texto, u) or _cambiar_velocidad_por_voz(texto, u)
2439              or _cambiar_confirmacion_hablada_por_voz(texto, u))
2440     if cambio:
2441         return cambio
2442     respondida = _responder_perfil_por_voz(texto, u)
2443     if respondida:
2444         return respondida
2445 navegada = _navegar_pestana_por_voz(vista, texto)
2446 if navegada:
2447     return navegada
2448 destinos = dict(_DESTINOS_ASISTENTE)
2449 if u.perfil != "auditor":
2450     for k in _SOLO_AUDITOR_ASISTENTE:
2451         destinos.pop(k, None)
2452 # Sin esto, Gemini podia "sugerir" ir a la pantalla en la que la
2453 # persona YA esta (confirmado: ofrecio "llevarla a Gestion de
2454 # usuarios" estando ella ahi mismo) - una orden real de cambiar de
2455 # PESTANA dentro de la misma pantalla ya se resolvió antes, arriba
2456 # (_navegar_pestana_por_voz), así que si el mensaje llega hasta aqui
2457 # nunca es eso.
2458 destinos.pop(vista, None)
2459 contexto = _contexto_asistente(vista, u)
2460 from agente.cerebro import pensar_asistente
2461 turno = pensar_asistente(contexto, texto, destinos, _PESTANAS_ASISTENTE)
2462 destino = turno.get("destino")
2463 if (turno.get("intencion") or "").lower() == "navegar" and destino in destinos:
2464     pestana = turno.get("pestana")
2465     pestanas_validas = _PESTANAS_ASISTENTE.get(destino, {})
2466     if pestana not in pestanas_validas:
2467         pestana = None
2468     return {"respuesta_hablada": turno.get("respuesta_hablada", ""),
2469           "accion": "navegar", "destino": destino, "pestana": pestana}
2470 return {"respuesta_hablada": turno.get("respuesta_hablada", ""),
2471       "accion": None, "destino": None, "pestana": None}
2472
2473
2474 @app.post("/api/agente/asistente")
2475 def api_asistente(p: PreguntarAsistenteIn, u: Usuario = Depends(usuario_actual)):
2476     """Version liviana del agente para pantallas que no cuentan ni piden
2477     (Inicio, Ajustes, Ayuda, Reportes, Panel): responde preguntas sobre lo
2478     que hay en esa pantalla y navega a otra si se lo piden. Sin el estado
2479     multi-turno de /api/agente/turno, que es específico del conteo/pedido."""
2480     return _resolver_asistente(p.vista, p.texto, u)
2481
2482
2483 @app.get("/api/sesiones/{sesion_id}/avance")
2484 def ver_avance(sesion_id: int, bodega_id_respaldo: int | None = None,
2485               u: Usuario = Depends(usuario_actual)):
2486     _validar_sesion_de_voz(u, sesion_id, bodega_id_respaldo)
2487     est = ESTADOS.setdefault(sesion_id, {})
2488     # mismo respaldo que en /agente/turno: si el backend se reinicio, el
2489     # frontend todavia sabe en que bodega estaba y este numero deja de
2490     # verse roto ("avance: 4 de 0") en vez de exigir que se reabra.
2491     if bodega_id_respaldo and not est.get("bodega_id"):
2492         est["bodega_id"] = bodega_id_respaldo
2493     with Sesion() as s:
2494         hechas = s.query(Conteo).filter_by(sesion_id=sesion_id,

```

```

2495         estado="confirmado").count()
2496     alertas = s.query(Alerta).filter_by(resuelta=0).count()
2497     total = (s.query(StockSistema).filter_by(bodega_id=est.get("bodega_id")).count()
2498             if est.get("bodega_id") else 0)
2499     ultimos = (s.query(Conteo).filter_by(sesion_id=sesion_id, estado="confirmado")
2500               .order_by(Conteo.id.desc()).limit(5).all())
2501     detalle = []
2502     for c in ultimos:
2503         a = s.get(Articulo, c.articulo_codigo)
2504         detalle.append({"nombre": a.nombre_oficial if a else c.articulo_codigo,
2505                        "cantidad": c.cantidad, "unidad": c.unidad})
2506     return {"hechas": hechas, "total": total, "alertas": alertas,
2507            "bodega": est.get("bodega_nombre"), "ultimos": detalle}
2508
2509
2510 def _limpiar_nombre_dictado(t: str) -> str:
2511     """El reconocimiento de voz del navegador cierra la frase con
2512     puntuación que nadie dijo ("Juguetería."). Ya se le quita para
2513     BUSCAR una bodega (servicios.conciliacion.normalizar); esto hace lo
2514     mismo pero para GUARDAR un nombre nuevo - sin tocar tildes ni
2515     mayúsculas, que sí importan en lo que queda escrito en el catálogo."""
2516     import re
2517     return re.sub(r"[.,;:;!¿?¡]+\\s*$", "", t.strip()).strip()
2518
2519
2520 class CrearProductoIn(BaseModel):
2521     nombre: str
2522     unidad_medida: str
2523     cantidad_inicial: float
2524     sesion_id: int = 1
2525
2526
2527 @app.post("/api/conteo/crear-producto")
2528 def crear_producto_pendiente(p: CrearProductoIn, u: Usuario = Depends(usuario_actual)):
2529     """El conteo no se detiene: el producto entra pendiente y sigue contando.
2530     Si quien cuenta es el administrador, el producto se confirma de una -
2531     igual que en crear_bodega_pendiente, dejarlo pendiente significaría que
2532     solo el mismo administrador puede aprobarlo despues."""
2533     # mismo motivo que en /api/agente/turno: el sesion_id lo manda el
2534     # cliente y de el sale la bodega donde va a quedar el producto nuevo.
2535     _validar_sesion_de_voz(u, p.sesion_id)
2536     est = ESTADOS.get(p.sesion_id, {})
2537     bodega_id = est.get("bodega_id")
2538     if not bodega_id:
2539         raise HTTPException(409, "Abra una bodega antes de crear un producto.")
2540     if p.cantidad_inicial < 0:
2541         raise HTTPException(400, "La cantidad inicial no puede ser negativa.")
2542     nombre = _limpiar_nombre_dictado(p.nombre).upper()
2543     # el sufijo de hora sin el sufijo aleatorio solo tenia ~100 microsegundos
2544     # de resolucioin real (se truncaba a 10 caracteres): dos personas creando
2545     # un producto casi al mismo tiempo podian generar el mismo codigo y
2546     # tumbar la peticion con un IntegrityError sin capturar.
2547     codigo = f"PEND-{ahora().strftime('%Y%m%d%H%M%S')}-{secrets.token_hex(3).upper()}"
2548     es_auditor = u.perfil == "auditor"
2549     with Sesion() as s:
2550         s.add(Articulo(codigo=codigo, nombre_oficial=nombre,
2551                       unidad_medida=p.unidad_medida))
2552         s.commit()
2553         conteo = Conteo(sesion_id=p.sesion_id, articulo_codigo=codigo,
2554                        cantidad=p.cantidad_inicial, unidad=p.unidad_medida,
2555                        estado="confirmado" if es_auditor else "pendiente_aprobacion")
2556         s.add(conteo)
2557         s.commit()
2558         s.refresh(conteo)
2559         if es_auditor:
2560             s.add(StockSistema(articulo_codigo=codigo, bodega_id=bodega_id, cantidad_sd=0))
2561         else:
2562             s.add(Aprobacion(tipo="producto", nombre=nombre,
2563                             unidad_medida=p.unidad_medida, cantidad_inicial=p.cantidad_inicial,
2564                             bodega_id=bodega_id, articulo_codigo=codigo,
2565                             conteo_id=conteo.id, creado_por_id=u.id))
2566         s.commit()
2567     if es_auditor:
2568         registrar(u, "CREACION", f"{nombre} creado")

```

```

2569         return {"ok": True, "codigo": codigo,
2570                 "respuesta_hablada": "Creado y confirmado en el catálogo."}
2571     registrar(u, "CREACION", f"{nombre} creado, pendiente de aprobacion")
2572     return {"ok": True, "codigo": codigo,
2573           "respuesta_hablada": "Creado. Queda pendiente de aprobación del administrador; "
2574           "el conteo sigue sin interrupción."}
2575
2576
2577 # ----- bodegas -----
2578 class AbrirIn(BaseModel):
2579     bodega: str
2580
2581
2582 def _contexto_bodega(s, b, refs):
2583     """Segun el estado, lo que hace falta para entender de un vistazo quien
2584     esta en que y cuanto lleva - se usa tanto en el tablero completo como
2585     en «mis bodegas asignadas» de Conteo."""
2586     item = {}
2587     if b.estado == "en_conteo":
2588         ses = (s.query(SesionConteo)
2589               .filter_by(bodega_id=b.id, tipo="conteo", estado="abierta")
2590               .order_by(SesionConteo.id.desc()).first())
2591         if ses:
2592             persona = s.get(Usuario, ses.usuario_id)
2593             hechas = s.query(Conteo).filter_by(
2594                 sesion_id=ses.id, estado="confirmado").count()
2595             item["persona"] = persona.nombre if persona else None
2596             item["avance_pct"] = round(hechas / refs * 100) if refs else 0
2597     elif b.estado == "en_auditoria":
2598         ses = (s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="auditoria")
2599               .order_by(SesionConteo.id.desc()).first())
2600         if ses:
2601             persona = s.get(Usuario, ses.usuario_id)
2602             item["persona"] = persona.nombre if persona else None
2603             conteo_ses = (s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo")
2604                           .order_by(SesionConteo.id.desc()).first())
2605             difs = 0
2606             if conteo_ses:
2607                 for c in (s.query(Conteo).filter_by(
2608                     sesion_id=conteo_ses.id, estado="confirmado").all()):
2609                     st = s.query(StockSistema).filter_by(
2610                         articulo_codigo=c.articulo_codigo, bodega_id=b.id).first()
2611                     if abs((c.cantidad or 0) - (st.cantidad_sd if st else 0)) > 0.001:
2612                         difs += 1
2613             item["diferencias"] = difs
2614     elif b.estado == "cerrada":
2615         hist = (s.query(HistorialCierre).filter_by(bodega_id=b.id)
2616               .order_by(HistorialCierre.fecha.desc()).first())
2617         if hist:
2618             item["hora_cierre"] = hist.fecha.strftime("%H:%M")
2619     return item
2620
2621
2622 @app.get("/api/bodegas")
2623 def listar_bodegas(propias: int = 0, u: Usuario = Depends(usuario_actual)):
2624     """El tablero en vivo: cada tarjeta necesita un dato distinto segun su
2625     estado (quien cuenta y cuanto lleva, quien audita y cuantas diferencias
2626     encontro, o a que hora se cerro) - no solo el total de referencias.
2627
2628     ?propias=1 limita el tablero a las bodegas asignadas a quien pregunta -
2629     para un auditor/administrador. Si la persona no tiene ninguna bodega
2630     asignada, ve todas (administrador general, sin zona propia); si tiene
2631     algunas asignadas, ve solo esas (un supervisor de zona con varios
2632     administradores repartindose el parque). El resto de pantallas
2633     (Pedidos elige donde descontar, Ajustes arma la lista para asignar)
2634     siguen pidiendo la lista completa sin este parametro - pantallas que
2635     solo un auditor/administrador alcanza a abrir.
2636
2637     Un auxiliar, en cambio, nunca ve bodegas fuera de su asignacion: para
2638     ese perfil la restriccion aplica siempre, con o sin ?propias=1, para
2639     que ninguna pantalla (incluida Pedidos) filtre por accidente el parque
2640     completo a quien solo debe ver su zona."""
2641     with Sesion() as s:
2642         ids_asignadas = {a.bodega_id for a in

```

```

2643         s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
2644     restringir = propias or u.perfil == "auxiliar"
2645     if restringir and ids_asignadas:
2646         bodegas = (s.query(Bodega).filter(Bodega.id.in_(ids_asignadas))
2647                     .order_by(Bodega.nombre_oficial).all())
2648     else:
2649         bodegas = s.query(Bodega).order_by(Bodega.nombre_oficial).all()
2650     salida = []
2651     for b in bodegas:
2652         refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
2653         item = {"id": b.id, "bodega": b.nombre_oficial,
2654                "estado": b.estado, "referencias": refs,
2655                "sede_id": b.sede_id, "sede": _nombre_sede(s, b.sede_id)}
2656         item.update(_contexto_bodega(s, b, refs))
2657         salida.append(item)
2658     return salida
2659
2660
2661 @app.post("/api/bodegas/buscar")
2662 def buscar_bodega_endpoint(a: AbrirIn, u: Usuario = Depends(usuario_actual)):
2663     """Solo resuelve el nombre, no abre nada - para que el selector de
2664     bodega pueda preguntar "¿confirma que abro X?" con el nombre REAL que
2665     de verdad se va a abrir, en vez de repetir a ciegas lo que se
2666     reconoció (decir «kiosco» no debe confirmarse como si "KIOSCO" fuera
2667     una bodega real, cuando lo que existe es "KIOSCO TAQUILLA AYB").
2668
2669     Si hay varias candidatas igual de razonables ("restaurante" a secas
2670     encaja en "RESTAURANTE FUENTES AYB" Y "RESTAURANTE FUENTES SUMIN"),
2671     se devuelven como opciones en vez de decir "no la encuentro" a secas
2672     - así la persona puede elegir la que quería, no repetir a ciegas."""
2673     from servicios.conciliacion import buscar_bodegas_candidatas
2674     with Sesion() as s:
2675         ids_permitidos = _ids_permitidos_para_buscar(s, u)
2676         candidatas = buscar_bodegas_candidatas(s, a.bodega, ids_permitidos)
2677         if len(candidatas) == 1:
2678             b = candidatas[0]
2679             return {"encontrada": True, "bodega": b.nombre_oficial, "bodega_id": b.id,
2680                    "opciones": []}
2681         opciones = [{"bodega": c.nombre_oficial, "bodega_id": c.id} for c in candidatas[:6]]
2682         return {"encontrada": False, "bodega": None, "bodega_id": None, "opciones": opciones}
2683
2684
2685 @app.post("/api/bodegas/abrir")
2686 async def abrir(a: AbrirIn, u: Usuario = Depends(usuario_actual)):
2687     from servicios.conciliacion import buscar_bodega
2688     with Sesion() as s:
2689         # buscar_bodega() (misma funcion que usa el agente conversacional):
2690         # quita tildes y puntuación de sobra (el reconocimiento de voz a
2691         # veces cierra la frase con un "." que nadie dijo) y prioriza la
2692         # coincidencia exacta - sin esto, "Zoológico" podía no encontrarse
2693         # por el punto final, o abrir "ZOOLOGICO SUMINISTROS" en vez de la
2694         # bodega "ZOOLOGICO" que de verdad se pidió. Restringida a lo
2695         # asignado si es auxiliar: no debe ni encontrar, ni ofrecer,
2696         # bodegas de otra zona que igual no podría abrir.
2697         b = buscar_bodega(s, a.bodega, _ids_permitidos_para_buscar(s, u))
2698         if b is None:
2699             raise HTTPException(404, "No encuentro esa bodega.")
2700         _requiere_acceso_bodega(s, u, b.id)
2701         abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
2702                                                    estado="abierta").first()
2703         if abierta and abierta.usuario_id != u.id:
2704             raise HTTPException(409, "Esa bodega ya esta en conteo por otra persona.")
2705         ses = abierta or SesionConteo(bodega_id=b.id, usuario_id=u.id)
2706         if not abierta:
2707             s.add(ses)
2708             b.estado = "en_conteo"
2709             s.commit()
2710             s.refresh(ses)
2711         refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
2712         sid, nombre, bid = ses.id, b.nombre_oficial, b.id
2713         ESTADOS.setdefault(sid, {}).update(
2714             {"bodega_id": bid, "bodega_nombre": nombre, "sesion_bd": sid})
2715         registrar(u, "APERTURA", f"{nombre} abierta - sesion bloqueada")
2716         await difundir_estado()

```



```

2717     return {"sesion_id": sid, "bodega": nombre, "referencias": refs, "bodega_id": bid}
2718
2719
2720 class CrearBodegaIn(BaseModel):
2721     nombre: str
2722
2723
2724 @app.post("/api/bodegas/crear-pendiente")
2725 def crear_bodega_pendiente(p: CrearBodegaIn, u: Usuario = Depends(usuario_actual)):
2726     """La bodega no se crea de una para un auxiliar: queda pendiente de
2727     aprobacion del administrador (Aprobacion tipo "bodega", ya soportada
2728     en aprobar()/rechazar()) - asi el catalogo no crece con lo que
2729     cualquiera escriba sin control. Un administrador ya tiene esa
2730     autoridad, asi que para el la bodega se crea de inmediato - de lo
2731     contrario terminaria aprobando su propia solicitud, lo cual no
2732     verifica nada (visto en produccion: Diana pidio "ALIMENTOS" y quedo
2733     esperando su propia firma en su propia bandeja)."""
2734     nombre = _limpiar_nombre_dictado(p.nombre).upper()
2735     if not nombre:
2736         raise HTTPException(400, "Dígame el nombre de la bodega.")
2737     with Sesion() as s:
2738         if s.query(Bodega).filter_by(nombre_oficial=nombre).first():
2739             raise HTTPException(409, "Ya existe una bodega con ese nombre.")
2740         if u.perfil == "auditor":
2741             s.add(Bodega(nombre_oficial=nombre, estado="pendiente"))
2742             s.commit()
2743             registrar(u, "CREACION", f"Bodega {nombre} creada")
2744             return {"ok": True,
2745                     "respuesta_hablada": f"{nombre.capitalize()} creada. Ya está en el catálogo."}
2746             s.add(Aprobacion(tipo="bodega", nombre=nombre, creado_por_id=u.id))
2747             s.commit()
2748             registrar(u, "CREACION", f"Bodega {nombre} solicitada, pendiente de aprobacion")
2749         return {"ok": True,
2750                 "respuesta_hablada": f"Solicitud de {nombre.lower()} enviada. Queda "
2751                                     "pendiente de aprobación del administrador."}
2752
2753
2754 def _ultima_sesion(s, bodega_id: int, tipo: str):
2755     return (s.query(SesionConteo).filter_by(bodega_id=bodega_id, tipo=tipo)
2756             .order_by(SesionConteo.id.desc()).first())
2757
2758
2759 def _nombre_sede(s, sede_id):
2760     """None cuando la bodega no tiene sede, que es un estado normal: las 54
2761     que ya existian quedaron asi y no hay ninguna obligacion de
2762     clasificarlas."""
2763     if sede_id is None:
2764         return None
2765     sede = s.get(Sede, sede_id)
2766     return sede.nombre if sede else None
2767
2768
2769 def _requiere_acceso_bodega(s, u: Usuario, bodega_id: int):
2770     """El tablero (listar_bodegas) ya oculta las bodegas ajenas, pero eso
2771     no basta: sin este chequeo, pedir el detalle, las firmas, el inventario
2772     completo o la auditoria por su bodega_id directamente (sin pasar por el
2773     tablero) dejaba ver -y hasta cerrar- zonas ajenas con solo cambiar el
2774     numero en la URL.
2775
2776     Manda la asignacion, no el perfil. Un administrador SIN bodegas
2777     asignadas es el administrador general de siempre y sigue alcanzando
2778     todo el parque; uno CON bodegas asignadas es el administrador de esa
2779     sede y solo de esa, para que crecer a varias sedes no signifique que
2780     el administrador de una pueda firmar o cerrar la auditoria de la otra.
2781     Un auxiliar sin asignaciones no alcanza ninguna: para ese perfil la
2782     asignacion es la unica puerta."""
2783     asignadas = {a.bodega_id for a in
2784                  s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
2785     if (asignadas or u.perfil == "auxiliar") and bodega_id not in asignadas:
2786         raise HTTPException(403, "Esa bodega no esta asignada a usted.")
2787
2788
2789 def _validar_sesion_de_voz(u: Usuario, sesion_id: int,
2790                            bodega_id_respaldo: int | None = None) -> None:

```



```

2791 """El id de sesion y el bodega_id_respaldo los manda el CLIENTE, y de
2792 ellos sale la bodega sobre la que el agente escribe. Sin comprobarlos,
2793 la asignacion de bodegas no protegía nada por este lado: bastaba con
2794 mandar el id de una bodega ajena -o el de la sesion que otra persona
2795 tiene abierta- para contarle encima, sin haber pasado nunca por
2796 /api/bodegas/abrir, que es donde estaba el unico chequeo.
2797
2798 Un sesion_id negativo es la conversacion de Pedidos (ver App.jsx, que
2799 arranca en -2000 menos el id de la persona): ahi no hay bodega fisica
2800 que proteger."""
2801 if sesion_id < 0:
2802     return
2803 with Sesion() as s:
2804     ses = s.get(SesionConteo, sesion_id)
2805     if ses is not None:
2806         if ses.usuario_id != u.id and u.perfil != "auditor":
2807             raise HTTPException(403, "Esa sesion de conteo no es suya.")
2808         _requiere_acceso_bodega(s, u, ses.bodega_id)
2809     if bodega_id_respaldo:
2810         _requiere_acceso_bodega(s, u, bodega_id_respaldo)
2811
2812 def _ids_permitidos_para_buscar(s, u: Usuario) -> set[int] | None:
2813     """Para restringir buscar_bodega()/buscar_bodegas_candidatas() a lo
2814     que un auxiliar de verdad puede abrir - sin esto, decir "restaurante"
2815     ofrecia como opciones bodegas de zonas ajenas que ni siquiera podia
2816     llegar a abrir, puro ruido (y un nombre que no tenia por qué ver).
2817     None solo para el administrador general (el que no tiene ninguna
2818     bodega asignada): ese si busca en todo el catálogo. Mismo criterio que
2819     _requiere_acceso_bodega, para que la voz no ofrezca lo que la puerta
2820     va a rechazar."""
2821     ids = {a.bodega_id for a in
2822             s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
2823     if not ids and u.perfil != "auxiliar":
2824         return None
2825     return ids
2826
2827 @app.get("/api/bodegas/{bodega_id}/firmas")
2828 def ver_firmas(bodega_id: int, u: Usuario = Depends(usuario_actual)):
2829     with Sesion() as s:
2830         _requiere_acceso_bodega(s, u, bodega_id)
2831         b = s.get(Bodega, bodega_id)
2832         conteo = _ultima_sesion(s, bodega_id, "conteo")
2833         auditoria = _ultima_sesion(s, bodega_id, "auditoria")
2834
2835     def _lado(ses):
2836         if ses is None:
2837             return None
2838         au = s.get(Usuario, ses.usuario_id)
2839         return {"sesion_id": ses.id, "persona": au.nombre if au else "?",
2840                 "firmada": bool(ses.firmada),
2841                 "hora": ses.fin.strftime("%H:%M") if ses.fin else None}
2842
2843     referencias = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
2844     alertas_resueltas = (s.query(Alerta).join(Conteo, Alerta.conteo_id == Conteo.id)
2845                          .join(SesionConteo, Conteo.sesion_id == SesionConteo.id)
2846                          .filter(SesionConteo.bodega_id == bodega_id,
2847                                  Alerta.resuelta == 1).count())
2848     return {"bodega": b.nombre_oficial if b else None, "referencias": referencias,
2849            "alertas_resueltas": alertas_resueltas,
2850            "conteo": _lado(conteo), "auditoria": _lado(auditoria),
2851            "lista_para_cerrar": bool(conteo and conteo.firmada
2852                                     and auditoria and auditoria.firmada)}
2853
2854 @app.post("/api/sesiones/{sesion_id}/firmar")
2855 def firmar_sesion(sesion_id: int, u: Usuario = Depends(usuario_actual)):
2856     """El auxiliar firma su conteo; el administrador firma su recuento ciego."""
2857     with Sesion() as s:
2858         ses = s.get(SesionConteo, sesion_id)
2859         if ses is None:
2860             raise HTTPException(404, "Sesion no encontrada.")
2861         if ses.usuario_id != u.id and u.perfil != "auditor":

```

```

2865         raise HTTPException(403, "Solo quien contó puede firmar este conteo.")
2866     # el perfil no basta: por aquí se rodeaba el límite por sede que si
2867     # aplica /api/bodegas/{id}/auditoria/firmar, porque a la bodega se
2868     # llega igual pero por el id de la sesión.
2869     _requiere_acceso_bodega(s, u, ses.bodega_id)
2870     ses.firmada = 1
2871     ses.fin = ahora()
2872     b = s.get(Bodega, ses.bodega_id)
2873     if ses.tipo == "conteo" and b and b.estado == "en_conteo":
2874         b.estado = "en_auditoria"
2875     s.commit()
2876     nombre = b.nombre_oficial if b else "?"
2877     tipo = ses.tipo
2878     etiqueta = "Conteo" if tipo == "conteo" else "Auditoria"
2879     registrar(u, "FIRMA", f"{etiqueta} firmado - {nombre}", "ok")
2880     return {"ok": True}
2881
2882
2883 @app.post("/api/bodegas/{bodega_id}/auditoria/iniciar")
2884 async def iniciar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2885     with Sesion() as s:
2886         _requiere_acceso_bodega(s, u, bodega_id)
2887         b = s.get(Bodega, bodega_id)
2888         if b is None:
2889             raise HTTPException(404, "Bodega no encontrada.")
2890         existente = s.query(SesionConteo).filter_by(
2891             bodega_id=bodega_id, tipo="auditoria", estado="abierta").first()
2892         ses = existente or SesionConteo(bodega_id=bodega_id, usuario_id=u.id, tipo="auditoria")
2893         if not existente:
2894             s.add(ses)
2895             s.commit()
2896             s.refresh(ses)
2897         refs = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
2898         sid, nombre = ses.id, b.nombre_oficial
2899         ESTADOS.setdefault(sid, {}).update({"bodega_id": bodega_id, "bodega_nombre": nombre})
2900         registrar(u, "AUDITORIA", f"Recuento ciego iniciado en {nombre}")
2901         await difundir_estado()
2902         return {"sesion_id": sid, "bodega": nombre, "referencias": refs}
2903
2904
2905 @app.get("/api/bodegas/{bodega_id}/auditoria/comparar")
2906 def comparar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2907     with Sesion() as s:
2908         _requiere_acceso_bodega(s, u, bodega_id)
2909         b = s.get(Bodega, bodega_id)
2910         ses_conteo = _ultima_sesion(s, bodega_id, "conteo")
2911         ses_audit = _ultima_sesion(s, bodega_id, "auditoria")
2912
2913     def _mapa(ses):
2914         m = {}
2915         if not ses:
2916             return m
2917         for c in (s.query(Conteo).filter_by(sesion_id=ses.id, estado="confirmado")
2918                 .order_by(Conteo.id).all()):
2919             m[c.articulo_codigo] = c.cantidad
2920         return m
2921     m1, m2 = _mapa(ses_conteo), _mapa(ses_audit)
2922     codigos = set(m1) | set(m2)
2923     filas = []
2924     diagnosticos = []
2925     for cod in codigos:
2926         art = s.get(Articulo, cod)
2927         nombre = art.nombre_oficial if art else cod
2928         st = s.query(StockSistema).filter_by(articulo_codigo=cod, bodega_id=bodega_id).first()
2929         sistema = st.cantidad_sd if st else 0
2930         c1, c2 = m1.get(cod), m2.get(cod)
2931         autoridad = c2 if c2 is not None else c1
2932         dif = round((autoridad or 0) - sistema, 3)
2933         if abs(dif) < 0.01 and (c1 == c2 or c2 is None):
2934             continue
2935         filas.append({"codigo": cod, "articulo": nombre,
2936                     "conteo1": c1, "conteo2": c2, "sistema": sistema,
2937                     "diferencia": dif,
2938                     "accion": "Revisar" if (c1 is not None and c2 is not None and c1 != c2)

```

```

2939         else "Aceptar"})
2940     # el sistema ya traia un saldo negativo antes de esta toma (una
2941     # salida sin su entrada correspondiente): no es un error del
2942     # conteo físico, es un dato roto que My Inventory debe corregir.
2943     if sistema < 0:
2944         diagnosticos.append(
2945             f"El {dif:+g} de {nombre.lower()} no viene de su conteo: el sistema "
2946             f"ya traía saldo negativo ({sistema:g}, salida registrada sin entrada). "
2947             "Su conteo físico es el que vale; el reporte lo marca como negativo "
2948             "del sistema para corrección en My Inventory.")
2949     filas.sort(key=lambda f: -abs(f["diferencia"]))
2950     return {"bodega": b.nombre_oficial if b else None,
2951            "filas": filas, "coinciden": len(codigos) - len(filas) if codigos else 0,
2952            "total": len(codigos), "diagnosticos": diagnosticos}
2953
2954
2955 @app.post("/api/bodegas/{bodega_id}/auditoria/firmar")
2956 def firmar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2957     with Sesion() as s:
2958         _requiere_acceso_bodega(s, u, bodega_id)
2959         ses = _ultima_sesion(s, bodega_id, "auditoria")
2960         if ses is None:
2961             raise HTTPException(404, "No hay un recuento de auditoria iniciado.")
2962         ses.firmada = 1
2963         ses.fin = ahora()
2964         s.commit()
2965         b = s.get(Bodega, bodega_id)
2966         nombre = b.nombre_oficial if b else ">"
2967         registrar(u, "FIRMA", f"Auditoria firmada - {nombre}", "ok")
2968         return {"ok": True}
2969
2970
2971 @app.post("/api/bodegas/{bodega_id}/cerrar")
2972 async def cerrar(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2973     with Sesion() as s:
2974         _requiere_acceso_bodega(s, u, bodega_id)
2975         b = s.get(Bodega, bodega_id)
2976         if b is None:
2977             raise HTTPException(404, "Bodega no encontrada.")
2978         ses_conteo = _ultima_sesion(s, bodega_id, "conteo")
2979         ses_audit = _ultima_sesion(s, bodega_id, "auditoria")
2980         if not (ses_conteo and ses_conteo.firmada and ses_audit and ses_audit.firmada):
2981             raise HTTPException(409, "Faltan firmas: el conteo y la auditoría deben "
2982                                   "estar firmados antes de cerrar.")
2983         conteos = (s.query(Conteo).filter_by(sesion_id=ses_conteo.id, estado="confirmado").all())
2984         difs = 0
2985         for c in conteos:
2986             st = s.query(StockSistema).filter_by(
2987                 articulo_codigo=c.articulo_codigo, bodega_id=bodega_id).first()
2988             if abs((c.cantidad or 0) - (st.cantidad_sd if st else 0)) > 0.001:
2989                 difs += 1
2990         exact = round((len(conteos) - difs) / len(conteos) * 100, 1) if conteos else 100.0
2991         b.estado = "cerrada"
2992         for ses in s.query(SesionConteo).filter_by(bodega_id=bodega_id,
2993                                                    estado="abierta").all():
2994             ses.estado = "cerrada"
2995             if not ses.fin:
2996                 ses.fin = ahora()
2997         s.add(HistorialCierre(bodega_id=bodega_id, exactitud=exact,
2998                             referencias=len(conteos), diferencias=difs))
2999         s.commit()
3000         nombre = b.nombre_oficial
3001         registrar(u, "CIERRE", f"{nombre} cerrada con doble firma", "ok")
3002         await difundir_estado()
3003         return {"ok": True, "bodega": nombre}
3004
3005
3006 @app.post("/api/bodegas/{bodega_id}/reabrir")
3007 async def reabrir_bodega(bodega_id: int, body: dict,
3008                          u: Usuario = Depends(requiere_perfil("auditor"))):
3009     motivo = (body.get("motivo") or "").strip()
3010     if not motivo:
3011         raise HTTPException(400, "Reabrir una bodega cerrada exige una justificación escrita.")
3012     with Sesion() as s:

```

```

3013     _requiere_acceso_bodega(s, u, bodega_id)
3014     b = s.get(Bodega, bodega_id)
3015     if b is None:
3016         raise HTTPException(404, "Bodega no encontrada.")
3017     if b.estado != "cerrada":
3018         raise HTTPException(409, "Solo se reabre una bodega que esté cerrada.")
3019     # las sesiones firmadas quedan como historial (nada se borra), pero
3020     # se marcan como reemplazadas para que abrir() arme una ronda de
3021     # conteo realmente nueva: reabrir sin esto no devolvía nada útil al
3022     # auxiliar, porque "abrir" seguía encontrando su sesión ya firmada.
3023     for ses in s.query(SesionConteo).filter_by(bodega_id=bodega_id, estado="abierta").all():
3024         ses.estado = "reemplazada"
3025     b.estado = "en_conteo"
3026     s.commit()
3027     nombre = b.nombre_oficial
3028     registrar(u, "REAPERTURA", f"{nombre} reabierta - motivo: {motivo}", "alerta")
3029     await difundir_estado()
3030     return {"ok": True, "bodega": nombre}
3031
3032
3033 def _narrar_hito(t):
3034     p = (t.persona or "alguien").title()
3035     if t.accion == "APERTURA":
3036         return f"{p} abre la bodega"
3037     if t.accion == "AUDITORIA":
3038         return f"{p} inicia el recuento ciego"
3039     if t.accion == "CORRECCION":
3040         return "Alerta: " + t.detalle.split(" (valor anterior)") [0]
3041     if t.accion == "FIRMA" and "Conteo firmado" in t.detalle:
3042         return f"{p} cierra el conteo"
3043     if t.accion == "FIRMA" and "Auditoria firmada" in t.detalle:
3044         return f"{p} firma el recuento"
3045     if t.accion == "CIERRE":
3046         return "Cierre con doble firma"
3047     if t.accion == "REAPERTURA":
3048         return f"{p}: {t.detalle}"
3049     return t.detalle
3050
3051
3052 def _color_hito(t):
3053     if t.accion == "CIERRE":
3054         return "verde"
3055     if t.accion in ("CORRECCION", "REAPERTURA"):
3056         return "oro"
3057     return "azul"
3058
3059
3060 @app.get("/api/bodegas/{bodega_id}/detalle")
3061 def detalle_bodega(bodega_id: int, u: Usuario = Depends(usuario_actual)):
3062     with Sesion() as s:
3063         _requiere_acceso_bodega(s, u, bodega_id)
3064         b = s.get(Bodega, bodega_id)
3065         if b is None:
3066             raise HTTPException(404, "Bodega no encontrada.")
3067         total = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
3068         # solo la ronda de conteo mas reciente: tras un reabrir(), la sesión
3069         # anterior queda como historial y no debe mezclarse con el recuento
3070         # nuevo en la exactitud/diferencias que se muestran aquí.
3071         ses_conteo_actual = _ultima_sesion(s, bodega_id, "conteo")
3072         conteos = (s.query(Conteo).filter_by(
3073             sesion_id=sesion_conteo_actual.id, estado="confirmado").all()
3074                    if ses_conteo_actual else [])
3075         difs = []
3076         for c in conteos:
3077             st = s.query(StockSistema).filter_by(
3078                 articulo_codigo=c.articulo_codigo, bodega_id=bodega_id).first()
3079             esperado = st.cantidad_sd if st else 0
3080             if abs((c.cantidad or 0) - esperado) > 0.001:
3081                 a = s.get(Articulo, c.articulo_codigo)
3082                 difs.append({"articulo": a.nombre_oficial if a else c.articulo_codigo,
3083                             "contado": c.cantidad, "sistema": esperado,
3084                             "diferencia": round(c.cantidad - esperado, 3)})
3085
3086         sesiones = s.query(SesionConteo).filter_by(bodega_id=bodega_id).all()

```

```

3087 usuario_ids = {ses.usuario_id for ses in sesiones if ses.usuario_id}
3088 personas_nombres = []
3089 for uid in usuario_ids:
3090     us = s.get(Usuario, uid)
3091     if us and us.nombre.title() not in personas_nombres:
3092         personas_nombres.append(us.nombre.title())
3093 if len(personas_nombres) <= 1:
3094     personas = personas_nombres[0] if personas_nombres else None
3095 else:
3096     personas = ", ".join(personas_nombres[:-1]) + " y " + personas_nombres[-1]
3097
3098 # hitos: los que ya mencionan la bodega por nombre (apertura/cierre/etc)
3099 # mas las correcciones de la persona en el rango de su sesion (esas
3100 # no mencionan la bodega, solo el articulo).
3101 hitos_t = list(s.query(Traza).filter(
3102     Traza.detalle.contains(b.nombre_oficial)).all())
3103 # el nombre de una bodega puede ser substring del de otra (ej.
3104 # "ZOOLOGICO" de "ZOOLOGICO SUMINISTROS" y "ZOOLOGICO PISCILAGO";
3105 # tambien "MOVIL FONDA", "MOVIL MIRADOR", "PANADERIA" con su propia
3106 # "... SUMINISTROS") - confirmado en produccion: abrir "ZOOLOGICO
3107 # SUMINISTROS" hacia aparecer ese evento en la linea de tiempo de
3108 # "ZOOLOGICO" a secas, una bodega distinta que ni se habia tocado.
3109 # Cuando el texto contiene el nombre de mas de una bodega real, se
3110 # prefiere siempre el mas largo (el mas especifico) como dueño real
3111 # del evento.
3112 if hitos_t:
3113     _todos_nombres = [x.nombre_oficial for x in s.query(Bodega).all()]
3114     def _es_de_esta_bodega(detalle):
3115         candidatos = [n for n in _todos_nombres if n in detalle]
3116         return not candidatos or max(candidatos, key=len) == b.nombre_oficial
3117     hitos_t = [t for t in hitos_t if _es_de_esta_bodega(t.detalle)]
3118 vistos = {t.id for t in hitos_t}
3119 for ses in sesiones:
3120     if not ses.usuario_id:
3121         continue
3122     fin = ses.fin or ahora()
3123     correcciones = (s.query(Traza)
3124         .filter(Traza.accion == "CORRECCION",
3125             Traza.usuario_id == ses.usuario_id,
3126             Traza.creado >= ses.inicio, Traza.creado <= fin)
3127         .all())
3128     for t in correcciones:
3129         if t.id not in vistos:
3130             hitos_t.append(t)
3131             vistos.add(t.id)
3132 hitos_t.sort(key=lambda t: t.creado)
3133 hitos = [{"hora": t.creado.strftime("%H:%M"), "texto": _narrar_hito(t),
3134     "tipo": _color_hito(t)} for t in hitos_t]
3135
3136 alertas_resueltas = (s.query(Alerta).join(Conteo, Alerta.conteo_id == Conteo.id)
3137     .join(SesionConteo, Conteo.sesion_id == SesionConteo.id)
3138     .filter(SesionConteo.bodega_id == bodega_id,
3139         Alerta.resuelta == 1).count())
3140
3141 exact = round((len(conteos) - len(difs)) / len(conteos) * 100, 1) if conteos else 0
3142 duracion_min = None
3143 if ses_conteo_actual and ses_conteo_actual.fin:
3144     duracion_min = round((ses_conteo_actual.fin - ses_conteo_actual.inicio).total_seconds() / 60)
3145 # referencia fija de un conteo manual en papel (no hay dato real de
3146 # esto: es un supuesto declarado, no una medicion de Colsubsidio)
3147 tiempo_papel_min = round(total * 20 / 60) if total else None
3148
3149 unidades_stock = sum(f.cantidad_sd or 0 for f in
3150     s.query(StockSistema).filter_by(bodega_id=bodega_id).all())
3151
3152 cierre_t = next((t for t in reversed(hitos_t) if t.accion == "CIERRE"), None)
3153 hora_cierre = cierre_t.creado.strftime("%H:%M") if cierre_t else None
3154
3155 ses_auditoria = next((ses for ses in sesiones if ses.tipo == "auditoria"), None)
3156 revisor = None
3157 if ses_auditoria and ses_auditoria.usuario_id:
3158     us = s.get(Usuario, ses_auditoria.usuario_id)
3159     revisor = us.nombre.title() if us else None
3160

```

```

3161     historial = (s.query(HistorialCierre).filter_by(bodega_id=bodega_id)
3162                  .order_by(HistorialCierre.fecha.desc()).all())
3163     anterior = historial[1] if len(historial) > 1 else None
3164     return {"bodega": b.nombre_oficial, "estado": b.estado,
3165            "referencias": total, "contadas": len(conteos),
3166            "exactitud": f"{exact} %", "diferencias": difs, "hitos": hitos,
3167            "duracion_min": duracion_min, "tiempo_papel_min": tiempo_papel_min,
3168            "unidades_stock": round(unidades_stock, 1),
3169            "personas": personas, "alertas_resueltas": alertas_resueltas,
3170            "hora_cierre": hora_cierre, "revisor": revisor,
3171            "ultima_toma_anterior": ({ "fecha": anterior.fecha.strftime("%Y-%m-%d"),
3172                                     "exactitud": anterior.exactitud}
3173                                     if anterior else None)}
3174
3175
3176 @app.get("/api/bodegas/{bodega_id}/articulos")
3177 def articulos_bodega(bodega_id: int, u: Usuario = Depends(usuario_actual)):
3178     """El extracto completo de My Inventory para esta bodega, tal cual
3179     quedo cargado del Excel: codigo, articulo, unidad y saldo del sistema."""
3180     with Sesion() as s:
3181         _requiere_acceso_bodega(s, u, bodega_id)
3182         b = s.get(Bodega, bodega_id)
3183         if b is None:
3184             raise HTTPException(404, "Bodega no encontrada.")
3185         filas = (s.query(StockSistema, Articulo)
3186                 .join(Articulo, Articulo.codigo == StockSistema.articulo_codigo)
3187                 .filter(StockSistema.bodega_id == bodega_id)
3188                 .order_by(Articulo.nombre_oficial).all())
3189         return [{"codigo": a.codigo, "articulo": a.nombre_oficial,
3190                "unidad": a.unidad_medida, "sd": st.cantidad_sd}
3191                for st, a in filas]
3192
3193
3194 # ----- consultas y reportes -----
3195 @app.get("/api/articulos/catalogo-ligero")
3196 def catalogo_ligero(u: Usuario = Depends(usuario_actual)):
3197     """El catálogo completo pero liviano (solo codigo/nombre/unidad), para
3198     que el modo sin conexión pueda sugerir nombres oficiales guardandolo
3199     en el equipo mientras hay señal."""
3200     with Sesion() as s:
3201         return [{"codigo": a.codigo, "nombre": a.nombre_oficial, "unidad": a.unidad_medida}
3202                for a in s.query(Articulo).all()]
3203
3204
3205 @app.get("/api/articulos/consulta")
3206 def consulta_articulo(q: str, codigo: str = "", u: Usuario = Depends(usuario_actual)):
3207     from servicios.conciliacion import buscar_articulo, buscar_bodegas_candidatas
3208     # Una orden explícita ("abra kiosco taquilla ayb") se resuelve ANTES
3209     # que cualquier otra cosa: si se dejara caer primero por la búsqueda
3210     # de artículo/bodega, el propio "abra" sumaba como palabra de más en
3211     # la coincidencia por palabras contra esa misma bodega y la orden
3212     # explícita nunca llegaba a reconocerse como tal - terminaba en la
3213     # sugerencia de "vea su detalle", no en abrirla de una vez.
3214     if _VERBOS_ABRIR_BODEGA.search(q):
3215         r = _resolver_asistente("bodegas", q, u)
3216         return {
3217             "resumen": r.get("respuesta_hablada", ""),
3218             "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
3219             "accion": r.get("accion"), "destino": r.get("destino"),
3220             "pestana": r.get("pestana"), "bodega": r.get("bodega"),
3221         }
3222     cand_todos = buscar_articulo(q)
3223     # Este buscador acepta CUALQUIER frase (nombre de ingrediente, de
3224     # bodega, una pregunta, una orden) - no solo un nombre de artículo
3225     # dictado a propósito, como sí es el caso en Conteo/Pedido (donde
3226     # este mismo umbral global de 45 sigue igual, sin tocar). Con ese
3227     # umbral más flojo aquí, frases sueltas sin relación con ningún
3228     # producto ("seguir contando", "no entendí", "espera") coincidían
3229     # por casualidad con productos reales que comparten una raíz de 4
3230     # letras (SEGUir/SEGUndos, ENTEndí/ENTero...) - una coincidencia
3231     # LEGÍTIMA en el sentido estructural (la misma que hace que BLANCA
3232     # encuentre BLANCO), pero sin ninguna relación real de significado.
3233     # Probado contra el catálogo real: una búsqueda de artículo genuina,
3234     # hasta parcial o descuidada ("tomate", "vino", "tabla picar"),

```

```

3235 # siempre puntuó 84 o más; el ruido de frases sueltas se quedó entre
3236 # 45 y 80. 82 separa limpio los dos grupos.
3237 UMBRAL_CONFIANZA_BODEGAS = 82
3238 # ya eligió una alternativa específica (clic en un chip de "¿era este
3239 # otro?") - se respeta tal cual, sin el filtro extra: ahí la persona
3240 # ya confirmó cuál quería, así que la confianza original no importa.
3241 cand = cand_todos if codigo else [c for c in cand_todos if c["confianza"] >= UMBRAL_CONFIANZA_BODEGAS]
3242 if not cand:
3243     # lo dicho no es ningun articulo del catalogo - pero si SI es el
3244     # nombre de una bodega (p. ej. alguien dice "restaurante" en este
3245     # buscador de ingredientes por error, cuando quería ir a Conteo a
3246     # abrirla), vale la pena decirlo en vez de un "no encuentre" a
3247     # secas que deja a la persona sin saber por que. "restaurante" a
3248     # secas encaja en mas de una bodega real - ahi no hay una sola
3249     # que prellenar, pero igual vale la pena decir que existen y
3250     # mandar a Conteo a terminar de decir cual.
3251     with Sesion() as s:
3252         ids_permitidos = _ids_permitidos_para_buscar(s, u)
3253         candidatas = buscar_bodegas_candidatas(s, q, ids_permitidos)
3254         if ids_permitidos is not None:
3255             # buscar_bodegas_candidatas() no restringe una coincidencia
3256             # EXACTA (para que /abrir pueda decir "existe pero no es
3257             # suya" en vez de "no existe") - aqui, en cambio, esto es
3258             # solo una sugerencia informativa sin ese motivo: no hay
3259             # por qué nombrarle a un auxiliar una bodega ajena.
3260             candidatas = [c for c in candidatas if c.id in ids_permitidos]
3261         if len(candidatas) == 1:
3262             b = candidatas[0]
3263             return {
3264                 "resumen": (f"No encuentro ese artículo, pero {b.nombre_oficial.title()} "
3265                             "sí es una bodega - vea su detalle para abrirla."),
3266                 "bodegas": [], "sugerencia_bodega": b.nombre_oficial,
3267                 "sugerencia_bodega_id": b.id,
3268             }
3269         if candidatas:
3270             nombres = ", ".join(c.nombre_oficial.title() for c in candidatas[:4])
3271             return {
3272                 "resumen": (f"No encuentro ese artículo. Ese nombre coincide con varias "
3273                             f"bodegas ({nombres}) - búsquelas en Bodegas para ver cuál es."),
3274                 "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
3275             }
3276     # Ni artículo ni bodega: puede ser una pregunta suelta ("¿cuántas
3277     # bodegas están pendientes?") o una orden de navegar ("llévame a
3278     # reportes") - un solo buscador para todo, en vez de dos cuadros
3279     # de búsqueda separados (uno para ingredientes, otro para
3280     # cualquier otra cosa) que además terminaban dando respuestas
3281     # distintas para lo mismo.
3282     r = _resolver_asistente("bodegas", q, u)
3283     return {
3284         "resumen": r.get("respuesta_hablada", ""),
3285         "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
3286         "accion": r.get("accion"), "destino": r.get("destino"),
3287         "pestaña": r.get("pestaña"), "bodega": r.get("bodega"),
3288     }
3289 # si la persona ya eligio una de las alternativas, usa esa; si no, la mejor
3290 a = next((c for c in cand if c["codigo"] == codigo), None) or cand[0]
3291 hoy = ahora().date()
3292 with Sesion() as s:
3293     filas = s.query(StockSistema).filter_by(articulo_codigo=a["codigo"]).all()
3294     total = sum(f.cantidad_sd or 0 for f in filas)
3295     det = []
3296     for f in filas:
3297         b = s.get(Bodega, f.bodega_id)
3298         ultima = (s.query(Conteo).join(SesionConteo)
3299                 .filter(SesionConteo.bodega_id == f.bodega_id,
3300                        Conteo.articulo_codigo == a["codigo"],
3301                        Conteo.estado == "confirmado")
3302                 .order_by(Conteo.id.desc()).first())
3303         if ultima:
3304             ultima_toma = (f"hoy {ultima.creado.strftime('%H:%M')} "
3305                             if ultima.creado.date() == hoy
3306                             else ultima.creado.strftime("%d %b").lower())
3307         elif b and b.estado == "en_conteo":
3308             ultima_toma = "en conteo"

```



```

3309         else:
3310             ultima_toma = "sin toma"
3311             det.append({"bodega": b.nombre_oficial if b else "?",
3312                        "cantidad": f.cantidad_sd, "estado": b.estado if b else "?",
3313                        "ultima_toma": ultima_toma})
3314             # consumo real del ultimo mes, sobre servicios ya legalizados
3315             hace_30 = ahora() - timedelta(days=30)
3316             lineas_mes = (s.query(LineaServicio)
3317                           .filter(LineaServicio.articulo_codigo == a["codigo"],
3318                                  LineaServicio.estado == "legalizado",
3319                                  LineaServicio.creado >= hace_30).all())
3320             consumo_mes = sum(l.usado or 0 for l in lineas_mes)
3321             servicios_mes = len(lineas_mes)
3322             cobertura_dias = round(total / (consumo_mes / 30)) if consumo_mes > 0 and total > 0 else None
3323             # otros articulos parecidos y CERCANOS en puntaje: un top score alto no
3324             # basta si el segundo esta empatado o casi (ARROZ y ARROZ BASMATI
3325             # empatan al 100 cuando solo se dice "arroz"); ahi es donde se confunde
3326             # el producto sin que la persona se de cuenta.
3327             alternativas = [c for c in cand if c["codigo"] != a["codigo"]
3328                             and (a["confianza"] - c["confianza"]) < 15][:2]
3329             ambiguo = bool(alternativas)
3330             resumen = f"{a['nombre']}: {total:g} {a['unidad']} en {len(filas)} bodegas."
3331             if ambiguo:
3332                 resumen += (f" Encontré artículos parecidos ({', '.join(c['nombre'] for c in alternativas)}); "
3333                             "confirme cuál necesita si no es este.")
3334             return {"articulo": a["nombre"], "codigo": a["codigo"], "unidad": a["unidad"],
3335                    "total": total, "bodegas": det, "resumen": resumen,
3336                    "ambiguo": ambiguo, "alternativas": alternativas,
3337                    "consumo_mes": round(consumo_mes, 2), "servicios_mes": servicios_mes,
3338                    "cobertura_dias": cobertura_dias}
3339
3340
3341 @app.get("/api/articulos/{codigo}/movimientos")
3342 def movimientos_articulo(codigo: str, u: Usuario = Depends(usuario_actual)):
3343     """«Ver movimientos»: los ultimos conteos confirmados de este articulo,
3344     en cualquier bodega, con quien y cuando."""
3345     with Sesion() as s:
3346         conteos = (s.query(Conteo).filter_by(articulo_codigo=codigo, estado="confirmado")
3347                    .order_by(Conteo.id.desc()).limit(20).all())
3348         salida = []
3349         for c in conteos:
3350             ses = s.get(SesionConteo, c.sesion_id)
3351             b = s.get(Bodega, ses.bodega_id) if ses else None
3352             persona = s.get(Usuario, ses.usuario_id) if ses else None
3353             salida.append({"hora": c.creado.strftime("%Y-%m-%d %H:%M"),
3354                           "bodega": b.nombre_oficial if b else "?",
3355                           "persona": persona.nombre if persona else "?",
3356                           "cantidad": c.cantidad, "unidad": c.unidad})
3357     return salida
3358
3359
3360 @app.get("/api/articulos/{codigo}/en-recetas")
3361 def articulo_en_recetas(codigo: str, u: Usuario = Depends(usuario_actual)):
3362     """«Comparar con la receta»: en que recetas aparece este articulo y
3363     cuanto pide por porcion."""
3364     with Sesion() as s:
3365         ings = s.query(RecetaIngrediente).filter_by(articulo_codigo=codigo).all()
3366         salida = []
3367         for ing in ings:
3368             r = s.get(Receta, ing.receta_id)
3369             if r:
3370                 salida.append({"receta": r.nombre, "por_porcion": ing.cantidad_por_porcion,
3371                               "rendimiento": r.rendimiento})
3372     return salida
3373
3374
3375 @app.post("/api/reportes")
3376 def reporte(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
3377     ruta, filas, vista_previa = reportes.consolidado(formato)
3378     registrar(u, "REPORTE", f"Consolidado generado: {ruta} ({filas} filas)")
3379     return {"archivo": ruta, "filas": filas, "vista_previa": vista_previa}
3380
3381
3382 @app.post("/api/reportes/diferencias")

```



```

3383 def reporte_diferencias(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
3384     ruta, filas, bodegas_con_descuadre = reportes.diferencias_archivo(formato)
3385     registrar(u, "REPORTE",
3386             f"Diferencias por bodega exportado: {ruta} ({filas} filas, "
3387             f"{bodegas_con_descuadre} bodegas)")
3388     return {"archivo": ruta, "filas": filas, "bodegas_con_descuadre": bodegas_con_descuadre}
3389
3390
3391 @app.post("/api/bodegas/exportar-estado")
3392 def exportar_estado_bodegas(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
3393     ruta = reportes.estado_bodegas(formato)
3394     registrar(u, "REPORTE", f"Estado de bodegas exportado: {ruta}")
3395     return {"archivo": ruta}
3396
3397
3398 def _parsear_archivo_reporte(t):
3399     """Convierte una traza cruda de tipo REPORTE en la tarjeta de archivo
3400     que muestra Reportes.jsx - sin esto, cada archivo generado se perdía
3401     en cuanto se salía de la pantalla (no había historial real)."""
3402     d = t.detalle
3403     if ":" not in d or "reportes/" not in d:
3404         return None
3405     _, resto = d.split(":", 1)
3406     archivo = resto.split("(")[0].strip()
3407     filas = None
3408     if "(" in resto and "filas" in resto:
3409         try:
3410             filas = int(resto.split("(")[1].split(" filas")[0])
3411         except (ValueError, IndexError):
3412             filas = None
3413     if d.startswith("Consolidado generado"):
3414         titulo, subtítulo = "Consolidado para My Inventory", "Listo para carga"
3415     elif d.startswith("Diferencias por bodega"):
3416         titulo = "Diferencias por bodega"
3417         bodegas_txt = resto.split(", ")[-1].replace(" bodegas", "").strip() if "bodegas" in resto else None
3418         subtítulo = f"{bodegas_txt} bodegas con descuadre" if bodegas_txt else "Exportado"
3419     elif d.startswith("Estado de bodegas"):
3420         titulo, subtítulo = "Estado del tablero", "Exportado"
3421     elif d.startswith("Detalle de bodega"):
3422         # el nombre de la bodega (no solo "Detalle de bodega" a secas) va
3423         # en la clave de deduplicación mas abajo: sin esto, exportar el
3424         # detalle de una bodega distinta hacia desaparecer de "recientes"
3425         # el de la anterior, como si nunca se hubiera generado.
3426         titulo = "Detalle de bodega"
3427         nombre_bodega = d.split("Detalle de bodega ", 1)[1].split(" exportado:")[0].strip()
3428         subtítulo = nombre_bodega.title() if nombre_bodega else "Exportado"
3429     elif d.startswith("Análisis de consumo"):
3430         titulo, subtítulo = "Análisis de consumo", "Exportado"
3431     elif d.startswith("Registro de trazabilidad"):
3432         titulo, subtítulo = "Registro de trazabilidad", "Exportado"
3433     else:
3434         titulo, subtítulo = "Reporte", "Exportado"
3435     # clave de deduplicación: para casi todos los tipos es el título (solo
3436     # importa el mas reciente); "Detalle de bodega" es la excepción, porque
3437     # el mismo título genérico cubre un archivo distinto por cada bodega.
3438     clave = f"{titulo}::{subtítulo}" if titulo == "Detalle de bodega" else titulo
3439     return {"titulo": titulo, "subtítulo": subtítulo, "archivo": archivo, "filas": filas,
3440           "formato": archivo.rsplit(".", 1)[-1].upper() if "." in archivo else "?",
3441           "hora": t.creado.strftime("%H:%M"), "persona": (t.persona or "").title(),
3442           "clave": clave}
3443
3444
3445 @app.get("/api/reportes/recientes")
3446 def reportes_recientes(u: Usuario = Depends(requiere_perfil("auditor"))):
3447     """El historial real de archivos generados (vía pantalla o por voz),
3448     leído de la trazabilidad - para que la lista sobreviva a salir de la
3449     pantalla o recargar, en vez de vivir solo en el estado del componente.
3450     Un solo archivo por tipo (el mas reciente): generar el mismo reporte
3451     varias veces en el día no debe ir apilando copias viejas - lo que
3452     importa es el último, los anteriores ya quedaron obsoletos apenas se
3453     genero uno nuevo del mismo tipo."""
3454     with Sesion() as s:
3455         trazas = (s.query(Traza).filter_by(accion="REPORTE")
3456                  .order_by(Traza.id.desc()).limit(40).all())

```

```

3457     vistos = set()
3458     salida = []
3459     for t in trazas:
3460         x = _parsear_archivo_reporte(t)
3461         if not x or x["clave"] in vistos:
3462             continue
3463         vistos.add(x["clave"])
3464         salida.append(x)
3465     return salida[:10]
3466
3467
3468 @app.post("/api/bodegas/{bodega_id}/exportar-detalle")
3469 def exportar_detalle_bodega(bodega_id: int, formato: str = "xlsx",
3470                             u: Usuario = Depends(requiere_perfil("auditor"))):
3471     with Sesion() as s:
3472         _requiere_acceso_bodega(s, u, bodega_id)
3473         b = s.get(Bodega, bodega_id)
3474         nombre_bodega = b.nombre_oficial if b else str(bodega_id)
3475         # el archivo se genera despues de comprobar el acceso: no tiene
3476         # sentido escribir en disco el detalle de una bodega que quien pregunta
3477         # no puede pedir.
3478         ruta = reportes.detalle_bodega(bodega_id, formato)
3479         # el nombre (no solo el id) queda en el propio mensaje para que
3480         # _parsear_archivo_reporte pueda mostrar de cual bodega es la tarjeta,
3481         # y distinguir el archivo de esta bodega del de otra en "recientes".
3482         registrar(u, "REPORTE", f"Detalle de bodega {nombre_bodega} exportado: {ruta}")
3483     return {"archivo": ruta}
3484
3485
3486 @app.get("/api/reportes/descargar")
3487 def descargar(archivo: str, u: Usuario = Depends(requiere_perfil("auditor"))):
3488     if "." in archivo or not archivo.startswith("reportes/"):
3489         raise HTTPException(400, "Ruta no permitida.")
3490     from servicios.archivos import leer_bytes
3491     contenido = leer_bytes(archivo)
3492     if contenido is None:
3493         raise HTTPException(404, "Archivo no encontrado.")
3494     tipo = "text/csv" if archivo.endswith(".csv") else (
3495         "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
3496     return Response(content=contenido, media_type=tipo, headers={
3497         "Content-Disposition": f'attachment; filename="{os.path.basename(archivo)}"'})
3498
3499
3500 @app.get("/api/reportes/vista-previa")
3501 def vista_previa_reporte(archivo: str, u: Usuario = Depends(requiere_perfil("auditor"))):
3502     """Para poder ver el contenido de cualquier archivo ya generado (no solo
3503     el que se acaba de crear) con solo dar clic en su tarjeta, sin tener
3504     que descargarlo primero. Lee el archivo tal cual quedo guardado (de la
3505     base, no de disco - ver servicios/archivos.py), asi que la vista previa
3506     de un reporte viejo muestra lo que ese reporte realmente tenia, no el
3507     estado actual de la base, y sigue funcionando despues de un redeploy."""
3508     if "." in archivo or not archivo.startswith("reportes/"):
3509         raise HTTPException(400, "Ruta no permitida.")
3510     import io
3511     import pandas as pd
3512     from servicios.archivos import leer_bytes
3513     contenido = leer_bytes(archivo)
3514     if contenido is None:
3515         raise HTTPException(404, "Archivo no encontrado.")
3516     buf = io.BytesIO(contenido)
3517     df = pd.read_csv(buf) if archivo.endswith(".csv") else pd.read_excel(buf)
3518     df = df.fillna(0)
3519     return {"filas": df.head(8).to_dict("records"), "total": len(df)}
3520
3521
3522 # ----- los tres momentos -----
3523 class PedidoIn(BaseModel):
3524     plato: str
3525     porciones: int = Field(gt=0)
3526     bodega_id: int = 1
3527
3528
3529 @app.post("/api/pedidos/calculador")
3530 def api_calcular(p: PedidoIn, u: Usuario = Depends(usuario_actual)):

```

```

3531     return calcular_pedido(p.plato, p.porciones, p.bodega_id)
3532
3533
3534 class EnviarPedidoIn(BaseModel):
3535     plato: str
3536     porciones: int = Field(gt=0)
3537     bodega_id: int
3538     servicio_id: int = 1
3539
3540
3541 # protege el bloque de verificar-duplicado + insertar: sin esto, dos
3542 # peticiones concurrentes (un reintento de red, un doble toque en la
3543 # tableta) podían pasar juntas la verificación de "ya_existe" antes de que
3544 # cualquiera terminara de guardar, y las dos quedaban registradas como
3545 # pedidos reales en vez de que la segunda se detectara como duplicada.
3546 # Alcanza con un lock en memoria porque el servicio corre en un solo
3547 # proceso (uvicorn sin --workers, ver render.yaml); con varios procesos
3548 # haría falta una constraint a nivel de base de datos.
3549 _lock_enviar_pedido = threading.Lock()
3550
3551
3552 @app.post("/api/pedidos/enviar")
3553 def api_enviar(datos: EnviarPedidoIn, u: Usuario = Depends(usuario_actual)):
3554     # las líneas ya no se reciben del cliente: se vuelven a calcular aquí
3555     # mismo contra la receta y el stock en vivo, igual que hace "Calcular
3556     # el pedido". Confiar en las cantidades que mandara el navegador
3557     # permitía pedir cualquier cosa (cualquier código de artículo, en
3558     # cualquier cantidad) para cualquier plato, incluso uno sin receta.
3559     calculo = calcular_pedido(datos.plato, datos.porciones, datos.bodega_id)
3560     if not calculo["receta_encontrada"]:
3561         raise HTTPException(404, f"No encontré una receta para «{datos.plato}».")
3562     lineas_a_pedir = [l for l in calculo["lineas"] if l["falta"] > 0]
3563     if not lineas_a_pedir:
3564         raise HTTPException(400, "No hay nada que pedir: ya tiene todo en la bodega.")
3565
3566     with _lock_enviar_pedido:
3567         with Sesion() as s:
3568             # si el mismo pedido (servicio + plato + porciones) ya quedó
3569             # abierto o esperando aprobación, no lo duplica: cubre un doble
3570             # clic, un doble disparo por voz, o un reintento tras una
3571             # desconexión que si alcanzo a llegar la primera vez.
3572             ya_existe = (s.query(LineaServicio)
3573                          .filter_by(servicio_id=datos.servicio_id, plato=datos.plato,
3574                                    porciones=datos.porciones)
3575                          .filter(LineaServicio.estado.in_([ "abierto", "pendiente_aprobacion" ]))
3576                          .first())
3577             if ya_existe:
3578                 return {"ok": True, "duplicado": True}
3579             # un auxiliar pide, pero es el administrador quien de verdad autoriza
3580             # que salga del almacén - igual que ya pasa con productos y bodegas
3581             # creados en plena toma. El administrador autoriza su propio pedido
3582             # de una vez: no tiene sentido pedirse permiso a sí mismo.
3583             estado_inicial = "abierto" if u.perfil == "auditor" else "pendiente_aprobacion"
3584             # el sufijo evita que dos pedidos distintos creados en el mismo
3585             # segundo (dos auxiliares pidiendo casi a la vez) terminen con
3586             # el mismo número_pedido - eso los mezclaba en una sola tarjeta
3587             # de aprobación en Auditoría, con los ítems de ambos revueltos.
3588             numero = f"PED-{ahora().strftime('%Y%m%d-%H%M%S')}-{secrets.token_hex(2).upper()}"
3589             n = 0
3590             items = []
3591             for l in lineas_a_pedir:
3592                 s.add(LineaServicio(servicio_id=datos.servicio_id,
3593                                    articulo_codigo=l["codigo"],
3594                                    nombre=l["nombre"], pedido=l["falta"],
3595                                    plato=datos.plato, porciones=datos.porciones,
3596                                    estado=estado_inicial, bodega_id=datos.bodega_id,
3597                                    creado_por_id=u.id, numero_pedido=numero))
3598                 items.append({"nombre": l["nombre"], "cantidad": l["falta"],
3599                             "unidad": l.get("unidad", "")})
3600             n += 1
3601             s.commit()
3602             b = s.get(Bodega, datos.bodega_id)
3603             bodega_nombre = b.nombre_oficial if b else None
3604             if estado_inicial == "pendiente_aprobacion":

```

```

3605         registrar(u, "PEDIDO", f"Pedido {numero} enviado, pendiente de aprobacion ({n} lineas)")
3606     else:
3607         registrar(u, "PEDIDO", f"Pedido {numero} enviado y aprobado ({n} lineas)")
3608     return {"ok": True, "numero_pedido": numero, "estado": estado_inicial,
3609           "hora": ahora().strftime("%H:%M"),
3610           "bodega": bodega_nombre, "persona": u.nombre,
3611           "items": items, "total_lineas": n}
3612
3613
3614 @app.get("/api/pedidos/{numero_pedido}/estado")
3615 def estado_pedido(numero_pedido: str, u: Usuario = Depends(usuario_actual)):
3616     """Para que quien pidió pueda enterarse si un administrador ya lo
3617     aprobó o rechazó, aunque haya salido de Pedidos y vuelto - la pantalla
3618     no se refresca sola."""
3619     with Sesion() as s:
3620         fila = s.query(LineaServicio).filter_by(numero_pedido=numero_pedido).first()
3621         if fila is None:
3622             raise HTTPException(404, "Ese pedido no existe.")
3623         if fila.creado_por_id != u.id and u.perfil != "auditor":
3624             raise HTTPException(403, "Ese pedido no es suyo.")
3625         return {"numero_pedido": numero_pedido, "estado": fila.estado}
3626
3627
3628 @app.get("/api/pedidos/pendientes")
3629 def pedidos_pendientes(u: Usuario = Depends(requiere_perfil("auditor"))):
3630     """Pedidos de auxiliares esperando la firma del administrador antes de
3631     contar como enviados de verdad al almacen - el pedir no se detiene
3632     (queda registrado de una vez), pero salir del almacen si depende de
3633     esta aprobacion, igual que con productos y bodegas nuevas."""
3634     with Sesion() as s:
3635         filas = (s.query(LineaServicio)
3636                 .filter_by(estado="pendiente_aprobacion")
3637                 .order_by(LineaServicio.creado.desc()).all())
3638         grupos = {}
3639         for f in filas:
3640             g = grupos.setdefault(f.numero_pedido, {
3641                 "numero_pedido": f.numero_pedido, "plato": f.plato,
3642                 "porciones": f.porciones, "hora": f.creado.strftime("%H:%M"),
3643                 "bodega_id": f.bodega_id, "creado_por_id": f.creado_por_id,
3644                 "items": [],
3645             })
3646             g["items"].append({"nombre": f.nombre, "cantidad": f.pedido})
3647         out = []
3648         for g in grupos.values():
3649             b = s.get(Bodega, g["bodega_id"]) if g["bodega_id"] else None
3650             persona = s.get(Usuario, g["creado_por_id"]) if g["creado_por_id"] else None
3651             out.append(**g, "bodega": b.nombre_oficial if b else None,
3652                        "persona": persona.nombre if persona else "--")
3653         out.sort(key=lambda x: x["numero_pedido"], reverse=True)
3654         return out
3655
3656
3657 class ResolverPedidoIn(BaseModel):
3658     aprobar: bool
3659
3660
3661 @app.post("/api/pedidos/{numero_pedido}/resolver")
3662 def resolver_pedido(numero_pedido: str, datos: ResolverPedidoIn,
3663                    u: Usuario = Depends(requiere_perfil("auditor"))):
3664     with Sesion() as s:
3665         filas = (s.query(LineaServicio)
3666                 .filter_by(numero_pedido=numero_pedido, estado="pendiente_aprobacion").all())
3667         if not filas:
3668             raise HTTPException(404, "Pedido no encontrado o ya resuelto.")
3669         nuevo_estado = "abierto" if datos.aprobar else "rechazado"
3670         plato = filas[0].plato
3671         for f in filas:
3672             f.estado = nuevo_estado
3673         s.commit()
3674         accion = "aprobado" if datos.aprobar else "rechazado"
3675         registrar(u, "APROBACION", f"Pedido {numero_pedido} ({plato}) {accion}",
3676                "ok" if datos.aprobar else "alerta")
3677     return {"ok": True, "estado": nuevo_estado}
3678

```

```

3679
3680 @app.get("/api/legalizacion/{servicio_id}")
3681 def api_legalizacion(servicio_id: int, u: Usuario = Depends(usuario_actual)):
3682     return comparar_legalizacion(servicio_id)
3683
3684
3685 @app.post("/api/legalizacion/confirmar")
3686 def api_confirmar(body: dict, u: Usuario = Depends(usuario_actual)):
3687     sid = body.get("servicio_id", 1)
3688     # opcional: permite mandar lo usado junto con la confirmación en vez de
3689     # exigir un "/legalizacion/ajustar" por cada insumo antes de cerrar.
3690     usos = {item.get("codigo"): item.get("usado") for item in body.get("usos", [])}
3691     with Sesion() as s:
3692         # solo el pedido que de verdad se le mostro a la persona (el mas
3693         # reciente) - si hubiera otro pedido distinto todavia abierto para
3694         # este servicio, sigue esperando su propio turno, no se legaliza
3695         # de arrastre solo porque comparte servicio_id.
3696         for l in _filas_a_legalizar(s, sid):
3697             if l.articulo_codigo in usos and usos[l.articulo_codigo] is not None:
3698                 l.usado = usos[l.articulo_codigo]
3699                 dif = (l.usado or 0) - (l.pedido or 0)
3700                 if dif < 0:
3701                     # sobrante: vuelve a bodega
3702                     st = s.query(StockSistema).filter_by(
3703                         articulo_codigo=l.articulo_codigo, bodega_id=l.bodega_id).first()
3704                     if st:
3705                         st.cantidad_sd = (st.cantidad_sd or 0) + abs(dif)
3706                         l.estado = "legalizado"
3707                     s.commit()
3708             registrar(u, "LEGALIZACION", f"Servicio {sid} legalizado", "ok")
3709         return {"ok": True}
3710
3711 @app.post("/api/legalizacion/ajustar")
3712 def api_ajustar_legalizacion(body: dict, u: Usuario = Depends(usuario_actual)):
3713     """«Ajustar por voz»: corrige lo realmente usado de un insumo antes de
3714     confirmar, dictando en vez de editar celda por celda."""
3715     from agente.cerebro import pensar
3716     sid = body.get("servicio_id", 1)
3717     texto = (body.get("texto") or "").strip()
3718     if not texto:
3719         raise HTTPException(400, "Dígame el insumo y la cantidad.")
3720     turno = pensar("Esta corrigiendo cuanto se uso de un insumo en un "
3721                  "servicio de comidas ya cerrado, antes de legalizarlo.", texto)
3722     articulo_texto = (turno.get("articulo_texto") or texto).upper()
3723     cantidad = turno.get("cantidad")
3724     if cantidad is None:
3725         raise HTTPException(400, "No entendí la cantidad. ¿Me la repite?")
3726     palabras = [p for p in articulo_texto.split() if len(p) >= 3]
3727     with Sesion() as s:
3728         # mismo alcance que comparar_legalizacion/confirmar: solo el
3729         # pedido mas reciente, no cualquier otro que siga abierto para
3730         # este servicio.
3731         filas = _filas_a_legalizar(s, sid)
3732         objetivo = next((f for f in filas
3733                        if any(p in f.nombre.upper() for p in palabras)), None)
3734         if objetivo is None:
3735             raise HTTPException(404, "No encontré ese insumo en este servicio.")
3736         objetivo.usado = cantidad
3737         s.commit()
3738         nombre = objetivo.nombre
3739         registrar(u, "LEGALIZACION", f"Ajuste por voz: {nombre} -> {cantidad:g}", "alerta")
3740     return comparar_legalizacion(sid)
3741
3742
3743 @app.get("/api/analisis/consumo")
3744 def api_analisis(dias: int = 30, u: Usuario = Depends(requiere_perfil("auditor"))):
3745     return analisis_consumo(dias)
3746
3747
3748 @app.post("/api/analisis/exportar")
3749 def exportar_analisis(formato: str = "xlsx", dias: int = 30,
3750                      u: Usuario = Depends(requiere_perfil("auditor"))):
3751     import pandas as pd
3752     from servicios.archivos import guardar_df

```

```

3753     datos = analisis_consumo(dias)
3754     df = pd.DataFrame(datos["subutilizados"])
3755     marca = ahora().strftime("%Y%m%d_%H%M")
3756     ruta = f"reportes/analisis_consumo_{marca}.{formato}"
3757     guardar_df(ruta, df, formato)
3758     registrar(u, "REPORTE", f"Análisis de consumo exportado: {ruta} ({len(df)} filas)")
3759     return {"archivo": ruta}
3760
3761
3762 @app.get("/api/pedidos/receta")
3763 def api_receta(plato: str, u: Usuario = Depends(usuario_actual)):
3764     return detalle_receta(plato)
3765
3766
3767 # ----- recetas: catálogo administrado -----
3768 # CuentaVoz si gestiona las recetas: crearlas, editarlas y borrarlas queda a
3769 # cargo del administrador (perfil auditor), igual que la gestión de usuarios
3770 # o de bodegas. El auxiliar solo las consulta al armar un pedido.
3771 class IngredienteIn(BaseModel):
3772     articulo_codigo: str
3773     cantidad_por_porcion: float
3774
3775
3776 class RecetaIn(BaseModel):
3777     nombre: str
3778     rendimiento: int = 1
3779     preparacion: str = ""
3780     ingredientes: list[IngredienteIn]
3781
3782
3783 @app.get("/api/recetas")
3784 def listar_recetas(u: Usuario = Depends(usuario_actual)):
3785     with Sesion() as s:
3786         recetas = s.query(Receta).order_by(Receta.nombre).all()
3787         return [{"id": r.id, "nombre": r.nombre, "rendimiento": r.rendimiento,
3788                 "ingredientes": s.query(RecetaIngrediente)
3789                     .filter_by(receta_id=r.id).count()}
3790                 for r in recetas]
3791
3792
3793 @app.get("/api/recetas/{receta_id}")
3794 def obtener_receta(receta_id: int, u: Usuario = Depends(usuario_actual)):
3795     with Sesion() as s:
3796         r = s.get(Receta, receta_id)
3797         if r is None:
3798             raise HTTPException(404, "Receta no encontrada.")
3799         ings = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
3800         lineas = []
3801         for i in ings:
3802             art = s.get(Articulo, i.articulo_codigo)
3803             lineas.append({"articulo_codigo": i.articulo_codigo,
3804                           "nombre": art.nombre_oficial if art else i.articulo_codigo,
3805                           "unidad": art.unidad_medida if art else "",
3806                           "cantidad_por_porcion": i.cantidad_por_porcion})
3807         return {"id": r.id, "nombre": r.nombre, "rendimiento": r.rendimiento,
3808               "preparacion": r.preparacion or "", "lineas": lineas}
3809
3810
3811 def _validar_ingredientes(s, ingredientes):
3812     if not ingredientes:
3813         raise HTTPException(400, "La receta necesita al menos un ingrediente.")
3814     vistos = set()
3815     for ing in ingredientes:
3816         art = s.get(Articulo, ing.articulo_codigo)
3817         if art is None:
3818             raise HTTPException(400, f"El artículo {ing.articulo_codigo} no existe en el catálogo.")
3819         if ing.cantidad_por_porcion <= 0:
3820             raise HTTPException(400, "La cantidad por porción debe ser mayor que cero.")
3821         # el mismo artículo en dos líneas de la receta no se suma solo: el
3822         # editor no impide elegirlo dos veces por error, y calcular_pedido
3823         # terminaba mostrando dos filas separadas del mismo insumo (con el
3824         # mismo código, lo que además rompe la key de React en la tabla) en
3825         # vez de una sola cantidad combinada.
3826         if ing.articulo_codigo in vistos:

```

```

3827         raise HTTPException(400, f"«{art.nombre_oficial}» está repetido en la receta: "
3828                                "combine las cantidades en una sola línea.")
3829     vistos.add(ing.articulo_codigo)
3830
3831
3832 @app.post("/api/recetas")
3833 def crear_receta(datos: RecetaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
3834     with Sesion() as s:
3835         _validar_ingredientes(s, datos.ingredientes)
3836         if s.query(Receta).filter(Receta.nombre.ilike(datos.nombre.strip())).first():
3837             raise HTTPException(400, "Ya existe una receta con ese nombre.")
3838         r = Receta(nombre=datos.nombre.strip(), rendimiento=datos.rendimiento,
3839                  preparacion=datos.preparacion.strip())
3840         s.add(r)
3841         s.flush()
3842         for ing in datos.ingredientes:
3843             s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=ing.articulo_codigo,
3844                                    cantidad_por_porcion=ing.cantidad_por_porcion))
3845         s.commit()
3846         rid, nombre = r.id, r.nombre
3847     registrar(u, "RECETA", f"Receta creada: {nombre} ({len(datos.ingredientes)} ingredientes)", "ok")
3848     return {"ok": True, "id": rid}
3849
3850
3851 @app.put("/api/recetas/{receta_id}")
3852 def editar_receta(receta_id: int, datos: RecetaIn,
3853                  u: Usuario = Depends(requiere_perfil("auditor"))):
3854     with Sesion() as s:
3855         r = s.get(Receta, receta_id)
3856         if r is None:
3857             raise HTTPException(404, "Receta no encontrada.")
3858         _validar_ingredientes(s, datos.ingredientes)
3859         otra = s.query(Receta).filter(Receta.nombre.ilike(datos.nombre.strip()),
3860                                     Receta.id != receta_id).first()
3861         if otra:
3862             raise HTTPException(400, "Ya existe otra receta con ese nombre.")
3863         r.nombre = datos.nombre.strip()
3864         r.rendimiento = datos.rendimiento
3865         r.preparacion = datos.preparacion.strip()
3866         s.query(RecetaIngrediente).filter_by(receta_id=receta_id).delete()
3867         for ing in datos.ingredientes:
3868             s.add(RecetaIngrediente(receta_id=receta_id, articulo_codigo=ing.articulo_codigo,
3869                                    cantidad_por_porcion=ing.cantidad_por_porcion))
3870         s.commit()
3871         nombre = r.nombre
3872     registrar(u, "RECETA", f"Receta editada: {nombre} ({len(datos.ingredientes)} ingredientes)", "ok")
3873     return {"ok": True}
3874
3875
3876 @app.delete("/api/recetas/{receta_id}")
3877 def eliminar_receta(receta_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
3878     with Sesion() as s:
3879         r = s.get(Receta, receta_id)
3880         if r is None:
3881             raise HTTPException(404, "Receta no encontrada.")
3882         nombre = r.nombre
3883         s.query(RecetaIngrediente).filter_by(receta_id=receta_id).delete()
3884         s.delete(r)
3885         s.commit()
3886     registrar(u, "RECETA", f"Receta eliminada: {nombre}", "ok")
3887     return {"ok": True}
3888
3889
3890 @app.get("/api/reportes/diferencias-por-bodega")
3891 def api_diferencias_por_bodega(u: Usuario = Depends(requiere_perfil("auditor"))):
3892     return analitica.diferencias_por_bodega(limite=50)
3893
3894
3895 @app.get("/api/panel/resumen")
3896 def api_panel_resumen(u: Usuario = Depends(requiere_perfil("auditor"))):
3897     return analitica.resumen_ejecutivo()
3898
3899
3900 @app.get("/api/panel/alertas")

```



```

3901 def api_panel_alertas(u: Usuario = Depends(requiere_perfil("auditor"))):
3902     return analitica.resumen_alertas_panel()
3903
3904
3905 # ----- alertas, usuarios y trazas -----
3906 _TIPO_ALERTA_TITULO = {
3907     "desviacion": "Desviación de cantidad", "negativo": "Cantidad negativa dictada",
3908     "unidad": "Unidad de medida", "inexistente": "Artículo fuera del catálogo",
3909 }
3910
3911
3912 @app.get("/api/alertas")
3913 def ver_alertas(resueltas: int = 0, u: Usuario = Depends(usuario_actual)):
3914     with Sesion() as s:
3915         salida = []
3916         for a in s.query(Alerta).filter_by(resuelta=resueltas).order_by(
3917             Alerta.id.desc()).all():
3918             art = bodega = persona = None
3919             if a.conteo_id:
3920                 c = s.get(Conteo, a.conteo_id)
3921                 if c:
3922                     ar = s.get(Articulo, c.articulo_codigo)
3923                     art = ar.nombre_oficial if ar else None
3924                     ses = s.get(SesionConteo, c.sesion_id)
3925                     if ses:
3926                         b = s.get(Bodega, ses.bodega_id)
3927                         bodega = b.nombre_oficial if b else None
3928                         us = s.get(Usuario, ses.usuario_id)
3929                         persona = us.nombre.title() if us else None
3930             salida.append({"id": a.id, "tipo": a.tipo,
3931                           "titulo": _TIPO_ALERTA_TITULO.get(a.tipo, a.tipo),
3932                           "detalle": a.detalle, "articulo": art, "bodega": bodega,
3933                           "persona": persona, "hora": a.creado.strftime("%H:%M")})
3934     return salida
3935
3936
3937 @app.get("/api/alertas/resumen")
3938 def resumen_alertas(u: Usuario = Depends(usuario_actual)):
3939     """Estadísticas reales para la bandeja: abiertas, resueltas hoy y el
3940     tiempo medio real entre que se genera la alerta y se resuelve."""
3941     hoy = ahora().date()
3942     with Sesion() as s:
3943         abiertas = s.query(Alerta).filter_by(resuelta=0).count()
3944         resueltas_hoy = (s.query(Alerta).filter(Alerta.resuelta == 1,
3945             Alerta.resuelto.isnot(None)).all())
3946         resueltas_hoy = [a for a in resueltas_hoy if a.resuelto.date() == hoy]
3947         tiempo_medio_min = None
3948         if resueltas_hoy:
3949             segundos = [(a.resuelto - a.creado).total_seconds() for a in resueltas_hoy]
3950             tiempo_medio_min = round(sum(segundos) / len(segundos) / 60, 1)
3951     return {"abiertas": abiertas, "resueltas_hoy": len(resueltas_hoy),
3952           "tiempo_medio_min": tiempo_medio_min}
3953
3954
3955 @app.post("/api/alertas/{alerta_id}/resolver")
3956 def resolver_alerta(alerta_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
3957     with Sesion() as s:
3958         a = s.get(Alerta, alerta_id)
3959         if a is None:
3960             raise HTTPException(404, "Alerta no encontrada.")
3961         a.resuelta = 1
3962         a.resuelto = ahora()
3963         s.commit()
3964         nombre = None
3965         if a.conteo_id:
3966             c = s.get(Conteo, a.conteo_id)
3967             if c:
3968                 art = s.get(Articulo, c.articulo_codigo)
3969                 nombre = art.nombre_oficial if art else None
3970         detalle = f"Alerta resuelta - {nombre}: {a.detalle}" if nombre else f"Alerta {alerta_id} resuelta"
3971         registrar(u, "ALERTA", detalle, "ok")
3972     return {"ok": True}
3973
3974

```



```

3975 @app.get("/api/usuarios")
3976 def listar_usuarios(u: Usuario = Depends(requiere_perfil("auditor"))):
3977     with Sesion() as s:
3978         salida = []
3979         for x in s.query(Usuario).all():
3980             n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=x.id).count()
3981             salida.append({"id": x.id, "nombre": x.nombre, "perfil": x.perfil,
3982                           "correo": x.correo, "activo": bool(x.activo),
3983                           "bodegas_asignadas": n_bodegas,
3984                           "ultimo_acceso": (x.ultimo_acceso.strftime("%Y-%m-%d %H:%M")
3985                                             if x.ultimo_acceso else None)})
3986     return salida
3987
3988
3989 class CrearUsuarioIn(BaseModel):
3990     nombre: str
3991     perfil: str
3992     correo: str = ""
3993     pin: str | None = None    # None: se genera uno temporal, ver mas abajo
3994
3995
3996 @app.post("/api/usuarios")
3997 def crear_usuario(datos: CrearUsuarioIn, u: Usuario = Depends(requiere_perfil("auditor"))):
3998     nombre = datos.nombre.strip().lower()
3999     if datos.perfil not in ("auxiliar", "auditor"):
4000         raise HTTPException(400, "El perfil debe ser auxiliar o auditor.")
4001     # Cognito exige un correo real para crear la cuenta - antes era
4002     # opcional porque solo se guardaba en esta base; ahora sin correo no
4003     # hay como crear la cuenta de verdad, asi que se exige aqui.
4004     correo = (datos.correo or "").strip()
4005     if not correo or "@" not in correo:
4006         raise HTTPException(400, "El correo es obligatorio y debe ser valido.")
4007     # sin esto, todo usuario creado desde Ajustes (la pantalla nunca pide
4008     # un PIN) quedaba con la misma clave de siempre - la misma que
4009     # aparece publicada en el README para las cuentas de demostracion.
4010     # Un temporal aleatorio, distinto cada vez, obliga a que quien lo
4011     # reciba lo cambie por uno propio desde Mi perfil.
4012     pin_generado = None
4013     if datos.pin is None:
4014         pin_generado = secrets.token_urlsafe(6) + "1Aa"    # cumple la politica de Cognito
4015         pin_para_guardar = pin_generado
4016     else:
4017         pin_para_guardar = datos.pin
4018     if len(pin_para_guardar) < 8:
4019         raise HTTPException(400, "La clave debe tener al menos 8 caracteres.")
4020     with Sesion() as s:
4021         if s.query(Usuario).filter_by(nombre=nombre).first():
4022             raise HTTPException(409, "Ya existe un usuario con ese nombre.")
4023         nuevo = Usuario(nombre=nombre, perfil=datos.perfil, correo=correo)
4024         s.add(nuevo)
4025         s.commit()
4026         s.refresh(nuevo)
4027         nid = nuevo.id
4028         # codigo de empleado: para poder ingresar con el ademas del nombre
4029         nuevo.codigo = f"CS-{48000 + nid}"
4030         s.commit()
4031     if not _crear_usuario_cognito(nombre, correo, pin_para_guardar):
4032         raise HTTPException(502, "La cuenta local se creo, pero no se pudo crear en Cognito. "
4033                               "Revise que el correo no este ya registrado.")
4034     registrar(u, "USUARIO", f"Usuario {nombre} creado ({datos.perfil})", "ok")
4035     respuesta = {"ok": True, "id": nid}
4036     if pin_generado:
4037         respuesta["pin_temporal"] = pin_generado
4038     return respuesta
4039
4040
4041 @app.put("/api/usuarios/yo")
4042 def editar_perfil(datos: dict, u: Usuario = Depends(usuario_actual)):
4043     """Cada quien edita su propio perfil - va antes de /usuarios/{usuario_id}
4044     a proposito: "yo" no es un entero, y si esta ruta queda despues, esa otra
4045     la intercepta primero y revienta con un 422 (el mismo problema que ya
4046     paso con /usuarios/yo/bodegas)."""
4047     correo_nuevo = (datos.get("correo") or "").strip()
4048     # si Cognito no queda enterado del correo nuevo, el codigo de "olvide

```

```

4049 # mi clave" seguiria yendo al correo viejo para siempre - se sincroniza
4050 # ANTES de guardar localmente, para no dejar los dos lados desfasados.
4051 if correo_nuevo and correo_nuevo != u.correo:
4052     if not _actualizar_correo_cognito(u.nombre, correo_nuevo):
4053         raise HTTPException(502, "No se pudo actualizar el correo en este momento.")
4054 with Sesion() as s:
4055     usr = s.get(Usuario, u.id)
4056     for k in ("nombre", "correo", "telefono"):
4057         if k in datos:
4058             setattr(usr, k, datos[k])
4059     s.commit()
4060 registrar(u, "PERFIL", "Datos personales actualizados")
4061 return {"ok": True}
4062
4063
4064 class EditarUsuarioIn(BaseModel):
4065     correo: str | None = None
4066     perfil: str | None = None
4067     activo: bool | None = None
4068
4069
4070 @app.put("/api/usuarios/{usuario_id}")
4071 def editar_usuario(usuario_id: int, datos: EditarUsuarioIn,
4072                    u: Usuario = Depends(requiere_perfil("auditor"))):
4073     """Editar correo, rol o estado de OTRA persona - no es lo mismo que
4074     /usuarios/yo, que cada quien usa para su propio perfil. No se borra a
4075     nadie (romperia la trazabilidad de sus conteos y firmas pasadas):
4076     desactivar es el equivalente seguro a eliminar, y ya bloquea el
4077     ingreso (ver usuario_actual)."""
4078     if datos.perfil is not None and datos.perfil not in ("auxiliar", "auditor"):
4079         raise HTTPException(400, "El perfil debe ser auxiliar o auditor.")
4080     if usuario_id == u.id and datos.activo is False:
4081         raise HTTPException(400, "No puede desactivar su propia cuenta.")
4082     if usuario_id == u.id and datos.perfil is not None and datos.perfil != u.perfil:
4083         raise HTTPException(400, "No puede cambiar su propio rol.")
4084     with Sesion() as s:
4085         obj = s.get(Usuario, usuario_id)
4086         if obj is None:
4087             raise HTTPException(404, "Usuario no encontrado.")
4088         cambios = []
4089         if datos.correo is not None and datos.correo != obj.correo:
4090             # mismo motivo que en editar_perfil: sin esto Cognito le
4091             # seguiria mandando los codigos de recuperar clave al correo
4092             # viejo de esa persona, sin que nada lo avisara.
4093             if not _actualizar_correo_cognito(obj.nombre, datos.correo):
4094                 raise HTTPException(502, "No se pudo actualizar el correo en este momento.")
4095             obj.correo = datos.correo
4096             cambios.append("correo")
4097         if datos.perfil is not None and datos.perfil != obj.perfil:
4098             obj.perfil = datos.perfil
4099             cambios.append(f"perfil -> {datos.perfil}")
4100         desactivado = False
4101         if datos.activo is not None and bool(datos.activo) != bool(obj.activo):
4102             obj.activo = int(datos.activo)
4103             desactivado = not datos.activo
4104             cambios.append("activo" if datos.activo else "inactivo")
4105         s.commit()
4106         nombre = obj.nombre
4107     if desactivado:
4108         # cierra sus sesiones vivas en Cognito - el access token ya
4109         # emitido sigue valido hasta que vence solo (ver seguridad.py)
4110         try:
4111             _cliente_cognito().admin_user_global_sign_out(
4112                 UserPoolId=COGNITO_USER_POOL_ID, Username=nombre)
4113         except Exception as e:
4114             print(f"[cognito] no se pudo cerrar la sesion de {nombre}: {e}")
4115     if cambios:
4116         registrar(u, "USUARIO", f"{nombre} editado: {' '.join(cambios)}", "ok")
4117     return {"ok": True}
4118
4119
4120 @app.delete("/api/usuarios/{usuario_id}")
4121 def eliminar_usuario(usuario_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4122     """Borra de verdad a alguien (no solo desactivar) - para limpiar cuentas

```

```

4123 de prueba de un ambiente antes de una demo, sin dejarlas visibles en
4124 Gestión de usuarios. A propósito NO es lo que hace editar_usuario()
4125 de arriba para el uso normal (esa sigue prefiriendo desactivar, por la
4126 misma razón documentada ahí: no romper la trazabilidad de conteos y
4127 firmas reales). Aquí sí se borra la cuenta, pero la Traza (registro
4128 inmutable) NUNCA se toca como fila - solo se le suelta la referencia al
4129 usuario (usuario_id queda en NULL; el nombre en "persona" ya es texto
4130 aparte) para que el historial real de esa persona, si lo hubo, siga
4131 intacto y legible después de borrar la cuenta.""
4132 if usuario_id == u.id:
4133     raise HTTPException(400, "No puede eliminar su propia cuenta.")
4134 with Sesion() as s:
4135     obj = s.get(Usuario, usuario_id)
4136     if obj is None:
4137         raise HTTPException(404, "Usuario no encontrado.")
4138     nombre = obj.nombre
4139     # sesiones de conteo/auditoria de esta persona: primero los Conteo
4140     # de cada sesion (y las Alertas que esos Conteo hayan generado),
4141     # despues la sesion misma - en ese orden, por las llaves foraneas.
4142     sesiones = s.query(SesionConteo).filter_by(usuario_id=usuario_id).all()
4143     for ses in sesiones:
4144         conteos = s.query(Conteo).filter_by(sesion_id=s.id).all()
4145         for c in conteos:
4146             s.query(Alerta).filter_by(conteo_id=c.id).delete()
4147             s.delete(c)
4148             s.delete(ses)
4149         s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).delete()
4150     # MensajeSoporte exige remitente/destinatario (no admite NULL) - sin
4151     # cuenta de uno de los dos lados, el mensaje no puede seguir existiendo.
4152     s.query(MensajeSoporte).filter(
4153         (MensajeSoporte.remitente_id == usuario_id)
4154         | (MensajeSoporte.destinatario_id == usuario_id)).delete()
4155     # estas si admiten NULL: se sueltan en vez de borrarse, para no
4156     # perder pedidos/aprobaciones reales solo porque quien los creo
4157     # ya no tiene cuenta.
4158     for linea in s.query(LineaServicio).filter_by(creado_por_id=usuario_id).all():
4159         linea.creado_por_id = None
4160     for ap in s.query(Aprobacion).filter_by(creado_por_id=usuario_id).all():
4161         ap.creado_por_id = None
4162     for ap in s.query(Aprobacion).filter_by(resuelto_por_id=usuario_id).all():
4163         ap.resuelto_por_id = None
4164     for t in s.query(Traza).filter_by(usuario_id=usuario_id).all():
4165         t.usuario_id = None
4166     s.delete(obj)
4167     s.commit()
4168 try:
4169     _cliente_cognito().admin_delete_user(UserPoolId=COGNITO_USER_POOL_ID, Username=nombre)
4170 except Exception as e:
4171     print(f"[cognito] no se pudo borrar la cuenta de {nombre}: {e}")
4172 registrar(u, "USUARIO", f"{nombre} eliminado de forma permanente", "ok")
4173 return {"ok": True}
4174
4175
4176 class ItemSemillaIn(BaseModel):
4177     articulo_codigo: str
4178     cantidad: float
4179     # solo para bodegas sin ningun stock de sistema cargado todavia (el
4180     # extracto de My Inventory nunca llego para esa bodega): crea la fila
4181     # de StockSistema con este valor ANTES de contar, para tener contra
4182     # que comparar. Si la bodega ya tiene stock de este articulo, se
4183     # ignora - nunca pisa un valor real ya cargado.
4184     cantidad_sistema_si_falta: float | None = None
4185     # una cantidad que se sale del umbral normalmente se salta (ver
4186     # docstring de sembrar_conteo) - forzar=true la guarda igual Y crea la
4187     # Alerta de "desviacion" que le correspondería, exactamente como
4188     # cuando una persona real dice "sí, confirmo" tras la advertencia. Es
4189     # para poblar Alertas/Auditoria con casos reales en un ambiente de
4190     # demo, no para forzar cantidades negativas o de otro articulo/unidad
4191     # (esas siguen sin poder guardarse: no tendría sentido real).
4192     forzar: bool = False
4193
4194
4195 class ConteoSemillaIn(BaseModel):
4196     bodega_id: int

```

```

4197     usuario_id: int
4198     items: list[ItemSemillaIn]
4199
4200
4201 @app.post("/api/admin/sembrar-conteo")
4202 def sembrar_conteo(datos: ConteoSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4203     """Crea conteos CONFIRMADOS reales (misma tabla, misma trazabilidad, se
4204     validan igual que si la persona lo hubiera dictado) para poblar un
4205     ambiente con datos de muestra antes de una demo - ej. que el panel
4206     gerencial tenga varias bodegas con diferencia real, no solo una. No
4207     pasa por reconocimiento de voz/texto: articulo y cantidad van directos,
4208     para que sembrar muchos a la vez no dependa de que el agente entienda
4209     cada frase (ni de la cuota de Gemini). Los items que dispararian una
4210     alerta (negativo, fuera de umbral...) se saltan en vez de forzarse -
4211     esto es para que la demo se vea real, no para inventar anomalías."""
4212     with Sesion() as s:
4213         persona = s.get(Usuario, datos.usuario_id)
4214         if persona is None:
4215             raise HTTPException(404, "Usuario no encontrado.")
4216         bodega = s.get(Bodega, datos.bodega_id)
4217         if bodega is None:
4218             raise HTTPException(404, "Bodega no encontrada.")
4219         ses = (s.query(SesionConteo)
4220               .filter_by(bodega_id=datos.bodega_id, usuario_id=datos.usuario_id, estado="abierta")
4221               .first())
4222         if not ses:
4223             ses = SesionConteo(bodega_id=datos.bodega_id, usuario_id=datos.usuario_id,
4224                               tipo="conteo", estado="abierta")
4225             s.add(ses)
4226             s.commit()
4227             s.refresh(ses)
4228             sesion_id = ses.id
4229             if bodega.estado == "pendiente":
4230                 bodega.estado = "en-conteo"
4231             s.commit()
4232             nombre_bodega = bodega.nombre_oficial
4233
4234     guardados = 0
4235     saltados = []
4236     for item in datos.items:
4237         with Sesion() as s:
4238             art = s.get(Articulo, item.articulo_codigo)
4239             if art is None:
4240                 saltados.append(item.articulo_codigo)
4241                 continue
4242             if item.cantidad_sistema_si_falta is not None:
4243                 existe = s.query(StockSistema).filter_by(
4244                     articulo_codigo=item.articulo_codigo, bodega_id=datos.bodega_id).first()
4245                 if not existe:
4246                     s.add(StockSistema(articulo_codigo=item.articulo_codigo,
4247                                         bodega_id=datos.bodega_id,
4248                                         cantidad_sd=item.cantidad_sistema_si_falta))
4249                     s.commit()
4250             v = validar_conteo(item.articulo_codigo, item.cantidad, art.unidad_medida, datos.bodega_id)
4251             if not v["ok"] and not (item.forzar and v["tipo"] == "desviacion"):
4252                 saltados.append(item.articulo_codigo)
4253                 continue
4254             with Sesion() as s:
4255                 reg = Conteo(sesion_id=sesion_id, articulo_codigo=item.articulo_codigo,
4256                             cantidad=item.cantidad, unidad=art.unidad_medida, estado="confirmado")
4257                 s.add(reg)
4258                 s.commit()
4259                 s.refresh(reg)
4260                 if not v["ok"]:
4261                     esperado = v.get("esperado") or 0
4262                     s.add(Alerta(conteo_id=reg.id, tipo="desviacion",
4263                                 detalle=f"El sistema esperaba alrededor de {esperado:g}, "
4264                                         f"se contaron {item.cantidad:g}.")
4265                     s.commit()
4266             guardados += 1
4267     if guardados:
4268         registrar(persona, "CONTEO",
4269                  f"{guardados} referencias contadas en {nombre_bodega} (datos de muestra)")
4270     return {"ok": True, "guardados": guardados, "saltados": saltados}

```

```

4271
4272
4273 class DuracionSemillaIn(BaseModel):
4274     sesion_id: int
4275     minutos: float
4276
4277
4278 @app.post("/api/admin/sembrar-duracion")
4279 def sembrar_duracion(datos: DuracionSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4280     """Ajusta cuánto "duró" una sesión de conteo (su inicio, relativo al fin
4281     que ya tiene) - para cuando una sesión de muestra se abrió hace rato
4282     (en otra prueba) y se firma/cierra recién ahora: sin esto, "tiempo
4283     promedio de conteo" del panel sale con la diferencia real entre esos
4284     dos momentos, que no fue tiempo contando de verdad."""
4285     with Sesion() as s:
4286         ses = s.get(SesionConteo, datos.sesion_id)
4287         if ses is None:
4288             raise HTTPException(404, "Sesion no encontrada.")
4289         if not ses.fin:
4290             raise HTTPException(409, "Esta sesion todavia no tiene fin (no está firmada/cerrada).")
4291         ses.inicio = ses.fin - timedelta(minutes=datos.minutos)
4292         s.commit()
4293     return {"ok": True}
4294
4295
4296 class AlertaSemillaIn(BaseModel):
4297     tipo: str          # negativo | unidad | inexistente | desviacion
4298     detalle: str
4299
4300
4301 @app.post("/api/admin/sembrar-alerta")
4302 def sembrar_alerta(datos: AlertaSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4303     """Crea una Alerta suelta (sin conteo asociado) para los tipos que en
4304     el flujo real NUNCA llegan a guardarse como conteo (negativo, unidad
4305     equivocada, artículo inexistente) - la persona tiene que repetir el
4306     dato, así que lo único que queda de ese intento es la alerta misma.
4307     Para poblar Auditoría/Panel con variedad real de tipos antes de una
4308     demo, no solo "desviación" (que sí se puede sembrar junto con su
4309     conteo via /admin/sembrar-conteo con forzar=true)."""
4310     if datos.tipo not in ("negativo", "unidad", "inexistente", "desviacion"):
4311         raise HTTPException(400, "Tipo de alerta no reconocido.")
4312     with Sesion() as s:
4313         s.add(Alerta(conteo_id=None, tipo=datos.tipo, detalle=datos.detalle))
4314         s.commit()
4315     return {"ok": True}
4316
4317
4318 class NegativosSemillaIn(BaseModel):
4319     inicial: int
4320
4321
4322 @app.put("/api/admin/negativos-iniciales")
4323 def ajustar_negativos_iniciales(datos: NegativosSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4324     """La tarjeta "Negativos detectados en el sistema" compara el inicio de
4325     ESTE período contra el conteo actual - si nunca se fijó un punto de
4326     partida (o se fijó igual al valor de hoy, por default), la tarjeta
4327     siempre muestra "X → X" y no cuenta ninguna historia. Este valor es la
4328     foto de hace un tiempo (de antes de que My Inventory corrigiera los
4329     que ya se corrigieron), no algo que la app recalcula sola: por diseño,
4330     corregir un negativo del sistema pasa por fuera de CuentaVoz."""
4331     with Sesion() as s:
4332         existente = s.get(ConfigClave, "negativos_iniciales")
4333         valor = str(datos.inicial)
4334         if existente:
4335             existente.valor = valor
4336         else:
4337             s.add(ConfigClave(clave="negativos_iniciales", valor=valor))
4338         s.commit()
4339     return {"ok": True}
4340
4341
4342 @app.delete("/api/admin/historial-cierre/{historial_id}")
4343 def borrar_historial_cierre(historial_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4344     """Para limpiar una foto de "Exactitud por toma de inventario" que

```

```

4345         quedó mal (ej. una bodega de muestra que se cerró antes de tener
4346         suficientes items contados, y después se reabrió para completarla) -
4347         a diferencia de la Traza, HistorialCierre no es un registro legal
4348         inmutable: es una serie de tiempo para la gráfica del panel, y una
4349         foto tomada por error de una prueba no debería quedar mezclada con el
4350         histórico real."""
4351         with Sesion() as s:
4352             h = s.get(HistorialCierre, historial_id)
4353             if h is None:
4354                 raise HTTPException(404, "Registro no encontrado.")
4355             s.delete(h)
4356             s.commit()
4357         return {"ok": True}
4358
4359
4360     class UsoSemillaItem(BaseModel):
4361         articulo_codigo: str
4362         usado: float
4363
4364
4365     class UsoSemillaIn(BaseModel):
4366         servicio_id: int
4367         usos: list[UsoSemillaItem]
4368
4369
4370     @app.put("/api/admin/legalizacion-uso")
4371     def sembrar_uso_legalizacion(datos: UsoSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4372         """Registra cuánto se usó de verdad en las líneas ya "abierto" de un
4373         servicio, SIN legalizarlo - a diferencia de /api/legalizacion/confirmar,
4374         que además cierra el servicio de una vez. Para dejar una comparación
4375         real y completa (pedido vs. usado) lista para revisar y confirmar en
4376         vivo en la demo, en vez de una pantalla vacía o con todo en cero."""
4377         with Sesion() as s:
4378             actualizadas = 0
4379             for item in datos.usos:
4380                 l = (s.query(LineaServicio)
4381                     .filter_by(servicio_id=datos.servicio_id, articulo_codigo=item.articulo_codigo,
4382                               estado="abierto")).first()
4383                 if l:
4384                     l.usado = item.usado
4385                     actualizadas += 1
4386             s.commit()
4387         return {"ok": True, "actualizadas": actualizadas}
4388
4389
4390     @app.get("/api/usuarios/yo/bodegas")
4391     def bodegas_asignadas(u: Usuario = Depends(usuario_actual)):
4392         """Las bodegas asignadas a la persona que tiene la sesion. Va ANTES de
4393         la ruta parametrizada /usuarios/{usuario_id}/bodegas a proposito: si
4394         quedara despues, FastAPI hace match de "yo" contra {usuario_id} primero
4395         y esta ruta nunca se alcanza (por eso nunca respondia para un auxiliar)."""
4396         with Sesion() as s:
4397             asigs = s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
4398             salida = []
4399             for a in asigs:
4400                 b = s.get(Bodega, a.bodega_id)
4401                 if b:
4402                     refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
4403                     item = {"id": b.id, "bodega": b.nombre_oficial,
4404                             "estado": b.estado, "referencias": refs,
4405                             "sede_id": b.sede_id, "sede": _nombre_sede(s, b.sede_id)}
4406                     item.update(_contexto_bodega(s, b, refs))
4407                     salida.append(item)
4408             return salida
4409
4410
4411     @app.get("/api/usuarios/{usuario_id}/bodegas")
4412     def ver_bodegas_de_usuario(usuario_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4413         """Para no borrar asignaciones existentes al agregar una nueva (ver
4414         «Asignar auditor» en el tablero de bodegas)."""
4415         with Sesion() as s:
4416             return [a.bodega_id for a in
4417                     s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).all()]
4418

```

```

4419
4420 @app.put("/api/usuarios/{usuario_id}/bodegas")
4421 def asignar_bodegas(usuario_id: int, body: dict,
4422                     u: Usuario = Depends(requiere_perfil("auditor"))):
4423     ids = body.get("bodega_ids", [])
4424     with Sesion() as s:
4425         objetivo = s.get(Usuario, usuario_id)
4426         if objetivo is None:
4427             raise HTTPException(404, "Usuario no encontrado.")
4428         # Un administrador de sede (el que tiene bodegas asignadas) solo
4429         # reparte las suyas: sin esto el limite era decorativo, porque podia
4430         # asignarSE las 54 bodegas y volver a ser administrador general.
4431         # Lo que el objetivo tenga en otras sedes se conserva tal cual - ni
4432         # se borra ni se puede crear desde aqui -, para que dos
4433         # administradores de sedes distintas puedan repartirle bodegas a la
4434         # misma persona sin pisarse el trabajo. El administrador general
4435         # (sin bodegas asignadas) sigue repartiendo todo el parque.
4436         mias = {a.bodega_id for a in
4437                 s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
4438         if mias:
4439             de_otras_sedes = [a.bodega_id for a in
4440                               s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).all()
4441                               if a.bodega_id not in mias]
4442             ids = [b for b in ids if b in mias] + de_otras_sedes
4443         s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).delete()
4444         for bid in ids:
4445             s.add(AsignacionBodega(usuario_id=usuario_id, bodega_id=bid))
4446         s.commit()
4447         nombre = objetivo.nombre
4448         registrar(u, "ASIGNACION", f"Bodegas asignadas a {nombre}: {len(ids)}", "ok")
4449         return {"ok": True}
4450
4451
4452
4453 # ----- sedes -----
4454 # Una sede agrupa bodegas. NO decide permisos: eso lo sigue decidiendo
4455 # AsignacionBodega, bodega por bodega (ver _requiere_acceso_bodega). Lo que
4456 # resuelve es lo practico: repartir doce bodegas de un clic en vez de doce,
4457 # y poder mirar la operacion por sitio.
4458
4459
4460 class SedeIn(BaseModel):
4461     nombre: str
4462     ciudad: str = ""
4463
4464
4465 @app.get("/api/sedes")
4466 def listar_sedes(u: Usuario = Depends(usuario_actual)):
4467     """Con cuantas bodegas tiene cada una, que es el dato que se mira al
4468     repartir. Lo puede ver cualquiera: son nombres de sitios, y el auxiliar
4469     los necesita para entender de donde es su bodega."""
4470     with Sesion() as s:
4471         por_sede: dict[int, int] = {}
4472         for (sid,) in s.query(Bodega.sede_id).filter(Bodega.sede_id.isnot(None)).all():
4473             por_sede[sid] = por_sede.get(sid, 0) + 1
4474         sedes = [{"id": x.id, "nombre": x.nombre, "ciudad": x.ciudad or "",
4475                  "bodegas": por_sede.get(x.id, 0)}
4476                  for x in s.query(Sede).order_by(Sede.nombre).all()]
4477         sin_sede = s.query(Bodega).filter(Bodega.sede_id.is_(None)).count()
4478         return {"sedes": sedes, "bodegas_sin_sede": sin_sede}
4479
4480
4481 @app.post("/api/sedes")
4482 def crear_sede(datos: SedeIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4483     nombre = (datos.nombre or "").strip().upper()
4484     if not nombre:
4485         raise HTTPException(400, "La sede necesita un nombre.")
4486     with Sesion() as s:
4487         if s.query(Sede).filter_by(nombre=nombre).first():
4488             raise HTTPException(409, "Ya existe una sede con ese nombre.")
4489         sede = Sede(nombre=nombre, ciudad=(datos.ciudad or "").strip())
4490         s.add(sede)
4491         s.commit()
4492         s.refresh(sede)

```



```

4493         sid = sede.id
4494     registrar(u, "SEDE", f"Sede creada: {nombre}", "ok")
4495     return {"ok": True, "id": sid}
4496
4497
4498 @app.delete("/api/sedes/{sede_id}")
4499 def borrar_sede(sede_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4500     """Las bodegas no se borran ni se tocan: vuelven a quedar sin sede,
4501     que es un estado valido. Mismo principio que en el resto del sistema -
4502     borrar una agrupacion no puede llevarse por delante lo agrupado."""
4503     with Sesion() as s:
4504         sede = s.get(Sede, sede_id)
4505         if sede is None:
4506             raise HTTPException(404, "Sede no encontrada.")
4507         nombre = sede.nombre
4508         sueltas = s.query(Bodega).filter_by(sede_id=sede_id).count()
4509         for b in s.query(Bodega).filter_by(sede_id=sede_id).all():
4510             b.sede_id = None
4511         s.delete(sede)
4512         s.commit()
4513     registrar(u, "SEDE", f"Sede eliminada: {nombre} ({sueitas} bodegas quedaron sin sede)", "ok")
4514     return {"ok": True, "bodegas_sin_sede": sueltas}
4515
4516
4517 @app.put("/api/bodegas/{bodega_id}/sede")
4518 def asignar_sede_a_bodega(bodega_id: int, body: dict,
4519                             u: Usuario = Depends(requiere_perfil("auditor"))):
4520     """sede_id = null saca la bodega de su sede sin borrarla de nada."""
4521     sede_id = body.get("sede_id")
4522     with Sesion() as s:
4523         _requiere_acceso_bodega(s, u, bodega_id)
4524         b = s.get(Bodega, bodega_id)
4525         if b is None:
4526             raise HTTPException(404, "Bodega no encontrada.")
4527         if sede_id is not None and s.get(Sede, sede_id) is None:
4528             raise HTTPException(404, "Sede no encontrada.")
4529         b.sede_id = sede_id
4530         s.commit()
4531         nombre_b = b.nombre_oficial
4532         nombre_s = s.get(Sede, sede_id).nombre if sede_id is not None else "sin sede"
4533     registrar(u, "SEDE", f"{nombre_b} -> {nombre_s}", "ok")
4534     return {"ok": True}
4535
4536
4537 @app.get("/api/sedes/{sede_id}/bodegas")
4538 def bodegas_de_sede(sede_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4539     """Los ids de una sede, para poder marcarlos todos de una vez al
4540     repartir. Solo devuelve los que quien pregunta puede repartir: a un
4541     administrador de sede no le sirve -ni debe ver- una lista que incluya
4542     bodegas que el PUT le va a filtrar despues."""
4543     with Sesion() as s:
4544         if s.get(Sede, sede_id) is None:
4545             raise HTTPException(404, "Sede no encontrada.")
4546         ids = [b.id for b in s.query(Bodega).filter_by(sede_id=sede_id).all()]
4547         mias = {a.bodega_id for a in
4548                 s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
4549         if mias:
4550             ids = [x for x in ids if x in mias]
4551     return ids
4552
4553
4554 @app.get("/api/bodegas/asignaciones")
4555 def ver_asignaciones_completas(u: Usuario = Depends(requiere_perfil("auditor"))):
4556     """Vista de cobertura: por cada bodega, quien la tiene asignada (o
4557     nadie). Con varios auxiliares repartiendo 54 bodegas, el administrador
4558     necesita ver esto de un vistazo para no dejar bodegas sin nadie a cargo
4559     ni confundir a dos personas asignandoles las mismas por error - revisar
4560     ficha por ficha de cada persona no lo deja ver.
4561
4562     Un administrador de sede ve la cobertura de SU sede: mostrarle huecos
4563     de bodegas que no puede asignar seria pedirle que arregle algo que no
4564     alcanza."""
4565     with Sesion() as s:
4566         mias = {a.bodega_id for a in

```



```

4567         s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
4568     por_bodega: dict[int, list] = {}
4569     for a in s.query(AsignacionBodega).all():
4570         por_bodega.setdefault(a.bodega_id, []).append(a.usuario_id)
4571     salida = []
4572     consulta = s.query(Bodega)
4573     if mias:
4574         consulta = consulta.filter(Bodega.id.in_(mias))
4575     for b in consulta.order_by(Bodega.nombre_oficial).all():
4576         personas = []
4577         for uid in por_bodega.get(b.id, []):
4578             us = s.get(Usuario, uid)
4579             if us:
4580                 personas.append({"id": us.id, "nombre": us.nombre, "perfil": us.perfil})
4581         salida.append({"id": b.id, "bodega": b.nombre_oficial, "asignados": personas})
4582     return salida
4583
4584
4585
4586
4587 # acciones sin valor para el resumen de "Inicio": ingresos repetidos, ajustes
4588 # de perfil y cada conteo individual (eso ya se ve en el tablero de Bodegas)
4589 _RUIDO_ACTIVIDAD = ("INGRESO", "SEGURIDAD", "PERFIL", "CONTEO")
4590
4591
4592 def _narrar_actividad(t):
4593     p = (t.persona or "alguien").title()
4594     d = t.detalle
4595     if t.accion == "CREACION":
4596         nombre = d.split(" creado,")[0]
4597         return f"{p} creó {nombre.title()}, pendiente de aprobación"
4598     if t.accion == "APERTURA":
4599         bodega = d.split(" abierta")[0]
4600         return f"{p} abrió {bodega.title()}"
4601     if t.accion == "AUDITORIA":
4602         bodega = d.replace("Recuento ciego iniciado en ", "")
4603         return f"{p} inició el recuento ciego en {bodega.title()}"
4604     if t.accion == "FIRMA":
4605         tipo, _, bodega = d.partition(" - ")
4606         verbo = "firmó el conteo de" if tipo.lower().startswith("conteo") else "firmó la auditoría de"
4607         return f"{p} {verbo} {bodega.title()}"
4608     if t.accion == "CIERRE":
4609         bodega = d.split(" cerrada")[0]
4610         return f"{bodega.title()} quedó cerrada con doble firma"
4611     if t.accion == "REAPERTURA":
4612         motivo = d.split("motivo: ")[-1]
4613         bodega = d.split(" reabierta")[0]
4614         return f"{p} reabrió {bodega.title()} – motivo: {motivo}"
4615     if t.accion == "REPORTE":
4616         if d.startswith("Consolidado generado"):
4617             filas = f" ({d.rsplit(' ', 1)[-1]})" if "(" in d else ""
4618             return f"{p} generó el consolidado para My Inventory{filas}"
4619         if d.startswith("Estado de bodegas exportado"):
4620             return f"{p} exportó el estado del tablero"
4621         if d.startswith("Detalle de bodega"):
4622             return f"{p} exportó el detalle de una bodega"
4623         return f"{p} generó un reporte"
4624     if t.accion == "PEDIDO":
4625         return f"{p} envió un pedido al almacén"
4626     if t.accion == "LEGALIZACION":
4627         if d.startswith("Ajuste por voz"):
4628             return f"{p} ajustó un consumo por voz"
4629         return f"{p} legalizó un servicio"
4630     if t.accion == "ALERTA":
4631         if d.startswith("Alerta resuelta - "):
4632             nombre, _, msg = d[len("Alerta resuelta - "):].partition(": ")
4633             try:
4634                 esperado = msg.split("alrededor de ")[1].split(".")[0]
4635                 confirmado = msg.split("¿Confirma ")[1].rstrip("?")
4636                 return f"Alerta resuelta: {nombre} pasó de {esperado} a {confirmado}"
4637             except IndexError:
4638                 return f"{p} resolvió una alerta de {nombre}"
4639             return f"{p} resolvió una alerta"
4640     if t.accion == "CORRECCION":

```

```

4641         cambio = d.split(" (valor anterior)")[0]
4642         return f"{p} corrigió un conteo: {cambio}"
4643     if t.accion == "USUARIO":
4644         nombre = d.split(" creado")[0].replace("Usuario ", "")
4645         return f"{p} creó el usuario {nombre.title()}"
4646     if t.accion == "ASIGNACION":
4647         return f"{p}: {d.lower()}"
4648     if t.accion == "APROBACION":
4649         if "entra al catalogo" in d:
4650             nombre = d.split(" aprobado")[0]
4651             return f"{p} aprobó el producto {nombre.title()}"
4652             nombre = d.split(" rechazado")[0]
4653             return f"{p} rechazó el producto {nombre.title()}"
4654     if t.accion == "AJUSTE":
4655         return f"{p} actualizó la configuración del sistema"
4656     if t.accion == "SOPORTE":
4657         if " le escribió a " in d:
4658             destinatario = d.split(" le escribió a ")[1].split(":")[0]
4659             return f"{p} le escribió a {destinatario.title()}"
4660         return f"{p} reportó un problema"
4661     return f"{p}: {d}"
4662
4663
4664 @app.get("/api/trazabilidad/reciente")
4665 def traza_reciente(u: Usuario = Depends(usuario_actual)):
4666     """Vista compartida para Inicio: sin datos sensibles, solo la acción y quién."""
4667     with Sesion() as s:
4668         trazas = (s.query(Traza).filter(~Traza.accion.in_(RUIDO_ACTIVIDAD))
4669                  .order_by(Traza.id.desc()).limit(8).all())
4670         return [{"hora": t.creado.strftime("%H:%M"), "persona": (t.persona or "").title(),
4671                  "accion": t.accion, "detalle": _narrar_actividad(t), "tipo": t.tipo}
4672                for t in trazas]
4673
4674
4675 # ----- aprobaciones en paralelo -----
4676 @app.get("/api/aprobaciones")
4677 def listar_aprobaciones(estados: str = "pendiente",
4678                        u: Usuario = Depends(requiere_perfil("auditor"))):
4679     with Sesion() as s:
4680         salida = []
4681         q = s.query(Aprobacion)
4682         if estados:
4683             q = q.filter_by(estados=estados)
4684         for a in q.order_by(Aprobacion.id.desc()).all():
4685             b = s.get(Bodega, a.bodega_id) if a.bodega_id else None
4686             creador = s.get(Usuario, a.creado_por_id) if a.creado_por_id else None
4687             salida.append({"id": a.id, "tipo": a.tipo, "nombre": a.nombre,
4688                          "unidad_medida": a.unidad_medida,
4689                          "cantidad_inicial": a.cantidad_inicial,
4690                          "bodega": b.nombre_oficial if b else None,
4691                          "creado_por": creador.nombre if creador else "?",
4692                          "hora": a.creado.strftime("%H:%M")})
4693     return salida
4694
4695
4696 @app.post("/api/aprobaciones/{aprobacion_id}/aprobar")
4697 def aprobar(aprobacion_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4698     with Sesion() as s:
4699         a = s.get(Aprobacion, aprobacion_id)
4700         if a is None or a.estado != "pendiente":
4701             raise HTTPException(404, "Aprobación no encontrada o ya resuelta.")
4702         a.estado = "aprobado"
4703         a.resuelto_por_id = u.id
4704         a.resuelto = ahora()
4705         if a.tipo == "producto" and a.articulo_codigo:
4706             existe = s.query(StockSistema).filter_by(
4707                 articulo_codigo=a.articulo_codigo, bodega_id=a.bodega_id).first()
4708             if not existe:
4709                 # el saldo del sistema arrancaba siempre en 0, sin importar
4710                 # cuanto se haya contado de verdad al crearlo (ej. "noventa
4711                 # alicornios") - el articulo quedaba marcado con una
4712                 # "diferencia" de +90 para siempre en el detalle de la
4713                 # bodega, como si nunca se hubiera revisado, aunque el
4714                 # administrador lo acababa de aprobar con ese conteo.

```

```

4715         s.add(StockSistema(articulo_codigo=a.articulo_codigo, bodega_id=a.bodega_id,
4716                             cantidad_sd=a.cantidad_inicial or 0))
4717     if a.conteo_id:
4718         c = s.get(Conteo, a.conteo_id)
4719         if c:
4720             c.estado = "confirmado"
4721     elif a.tipo == "bodega":
4722         s.add(Bodega(nombre_oficial=a.nombre, estado="pendiente"))
4723     s.commit()
4724     nombre = a.nombre
4725     registrar(u, "APROBACION", f"{nombre} aprobado y entra al catalogo oficial", "ok")
4726     return {"ok": True}
4727
4728
4729 @app.post("/api/aprobaciones/{aprobacion_id}/rechazar")
4730 def rechazar(aprobacion_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4731     with Sesion() as s:
4732         a = s.get(Aprobacion, aprobacion_id)
4733         if a is None or a.estado != "pendiente":
4734             raise HTTPException(404, "Aprobacion no encontrada o ya resuelta.")
4735         a.estado = "rechazado"
4736         a.resuelto_por_id = u.id
4737         a.resuelto = ahora()
4738         if a.conteo_id:
4739             c = s.get(Conteo, a.conteo_id)
4740             if c:
4741                 c.estado = "rechazado"
4742             s.commit()
4743             nombre = a.nombre
4744         registrar(u, "APROBACION", f"{nombre} rechazado", "alerta")
4745     return {"ok": True}
4746
4747
4748 @app.post("/api/sopORTE/reportar")
4749 def reportar_problema(body: dict, u: Usuario = Depends(usuario_actual)):
4750     detalle = (body.get("detalle") or "").strip() or "Sin detalle."
4751     registrar(u, "SOPORTE", f"{u.nombre} reporto: {detalle}", "alerta")
4752     return {"ok": True}
4753
4754
4755 def _enviar_correo_real(destinatario: str, asunto: str, cuerpo: str) -> tuple[bool, str]:
4756     """Envía un correo de verdad por la API de Brevo (HTTPS) si hay
4757     credenciales configuradas (BREVO_API_KEY). Antes se intentó SMTP
4758     directo a Gmail (Render bloquea el puerto 587, "Network is
4759     unreachable") y luego la API de Resend (Cloudflare la rechazaba con
4760     "error code: 1010", probablemente por la reputación de las IPs
4761     compartidas de Render) - ninguna de las dos depende del código, así
4762     que se cambió de proveedor otra vez. Sin BREVO_API_KEY configurada,
4763     no intenta nada: el llamador sigue funcionando igual que antes (solo
4764     trazabilidad), el mismo patrón de "se degrada sin romperse" que ya
4765     usa el agente sin GOOGLE_API_KEY. Devuelve (enviado, motivo-si-fallo)
4766     - el motivo es temporal mientras se termina de calibrar el envío
4767     real."""
4768     api_key = os.getenv("BREVO_API_KEY", "").strip()
4769     remitente = os.getenv("BREVO_FROM", os.getenv("SMTP_CORREO", "")).strip()
4770     if not api_key or not remitente or not destinatario:
4771         return False, "sin BREVO_API_KEY/BREVO_FROM configuradas"
4772     payload = json.dumps({
4773         "sender": {"name": "CuentaVoz", "email": remitente},
4774         "to": [{"email": destinatario}],
4775         "subject": asunto, "textContent": cuerpo,
4776     }).encode("utf-8")
4777     peticion = urllib.request.Request(
4778         "https://api.brevo.com/v3/smtp/email", data=payload, method="POST",
4779         headers={"api-key": api_key, "Content-Type": "application/json",
4780                 "Accept": "application/json",
4781                 "User-Agent": "CuentaVoz/1.0 (+https://cuentavoz.onrender.com)"}
4782     )
4783     # El contenedor de Render no tiene ruta de salida por IPv6, pero la
4784     # resolución de nombres a veces sí devuelve una dirección IPv6 (y
4785     # Python la intenta primero) - eso da exactamente "Network is
4786     # unreachable" aunque el IPv4 normal funcione bien. Se fuerza IPv4
4787     # solo durante esta conexión puntual.
4788     getaddrinfo_original = socket.getaddrinfo

```

```

4789 def _forzar_ipv4(host, port, family=0, type=0, proto=0, flags=0):
4790     return getaddrinfo_original(host, port, socket.AF_INET, type, proto, flags)
4791
4792 socket.getaddrinfo = _forzar_ipv4
4793 try:
4794     with urllib.request.urlopen(peticion, timeout=10) as resp:
4795         return 200 <= resp.status < 300, ""
4796 except urllib.error.HTTPError as e:
4797     motivo = f"HTTP {e.code}: {e.read().decode('utf-8', 'ignore')[:300]}"
4798     print(f"[correo] no se pudo enviar a {destinatario}: {motivo}")
4799     return False, motivo
4800 except Exception as e:
4801     motivo = f"{type(e).__name__}: {e}"
4802     print(f"[correo] no se pudo enviar a {destinatario}: {motivo}")
4803     return False, motivo
4804 finally:
4805     socket.getaddrinfo = getaddrinfo_original
4806
4807
4808 @app.post("/api/soporte/mensaje-administrador")
4809 def mensaje_administrador(body: dict, u: Usuario = Depends(usuario_actual)):
4810     """Escribirle al administrador". El mensaje siempre queda trazado -
4811     se ve en Ajustes → Registro de trazabilidad - y además se intenta un
4812     correo real (vía Brevo) si el servidor tiene BREVO_API_KEY
4813     configurada. Sin esa credencial (el caso normal por ahora: no
4814     depende de registrar una cuenta con un tercero), el frontend abre el
4815     correo ya instalado en el dispositivo de quien envía, con el mensaje
4816     listo - así igual llega un correo real, desde la cuenta de esa
4817     persona, sin que CuentaVoz tenga que gestionar ningún servicio de
4818     correo por su cuenta."""
4819     mensaje = (body.get("mensaje") or "").strip()
4820     if not mensaje:
4821         raise HTTPException(400, "Escriba el mensaje antes de enviarlo.")
4822     with Sesion() as s:
4823         admin = (s.query(Usuario)
4824                 .filter(Usuario.perfil == "auditor", Usuario.activo == 1, Usuario.id != u.id)
4825                 .first())
4826         if not admin:
4827             raise HTTPException(404, "No hay un administrador disponible por ahora.")
4828         nombre_admin, correo_admin, admin_id = admin.nombre, admin.correo, admin.id
4829         s.add(MensajeSoporte(remite_id=u.id, destinatario_id=admin_id, mensaje=mensaje))
4830         s.commit()
4831         registrar(u, "SOPORTE", f"{u.nombre} le escribió a {nombre_admin}: {mensaje}")
4832         cuerpo = f"Mensaje enviado desde CuentaVoz por {u.nombre}.\n\n{mensaje}"
4833         correo_enviado, _motivo = _enviar_correo_real(correo_admin, "Mensaje desde CuentaVoz", cuerpo)
4834         return {"ok": True, "administrador": nombre_admin, "correo_enviado": correo_enviado}
4835
4836
4837 @app.get("/api/soporte/mensajes")
4838 def mis_mensajes_soporte(u: Usuario = Depends(usuario_actual)):
4839     """La bandeja de "Soporte en vivo" dentro de la app: para un auxiliar,
4840     lo que ha escrito y si ya le respondieron; para el administrador, lo
4841     que le han escrito y lo que falta por responder."""
4842     with Sesion() as s:
4843         if u.perfil == "auditor":
4844             filas = (s.query(MensajeSoporte)
4845                     .filter(MensajeSoporte.destinatario_id == u.id)
4846                     .order_by(MensajeSoporte.creado.desc()).all())
4847         else:
4848             filas = (s.query(MensajeSoporte)
4849                     .filter(MensajeSoporte.remitente_id == u.id)
4850                     .order_by(MensajeSoporte.creado.desc()).all())
4851         ids_personas = {m.remitente_id for m in filas} | {m.destinatario_id for m in filas}
4852         personas = {p.id: p.nombre for p in s.query(Usuario).filter(Usuario.id.in_(ids_personas)).all()}
4853         return [{"id": m.id,
4854                 "de": personas.get(m.remitente_id, "?"),
4855                 "para": personas.get(m.destinatario_id, "?"),
4856                 "mensaje": m.mensaje, "respuesta": m.respuesta,
4857                 "creado": m.creado.strftime("%Y-%m-%d %H:%M"),
4858                 "respondido": m.respondido.strftime("%Y-%m-%d %H:%M") if m.respondido else None}
4859                 for m in filas]
4860
4861
4862 @app.post("/api/soporte/mensajes/{mensaje_id}/responder")

```

```

4863 def responder_mensaje_soporte(mensaje_id: int, body: dict, u: Usuario = Depends(usuario_actual)):
4864     """Solo quien recibió el mensaje puede responderlo - no hace falta
4865     pedir perfil "auditor" aparte: si no es el destinatario, no puede."""
4866     respuesta = (body.get("respuesta") or "").strip()
4867     if not respuesta:
4868         raise HTTPException(400, "Escriba la respuesta antes de enviarla.")
4869     with Sesion() as s:
4870         m = s.get(MensajeSoporte, mensaje_id)
4871         if not m or m.destinatario_id != u.id:
4872             raise HTTPException(404, "Ese mensaje no existe o no es suyo para responder.")
4873         if m.respuesta is not None:
4874             # sin este chequeo, dos respuestas al mismo mensaje (un doble
4875             # clic, un reintento de red, dos pestañas abiertas) pisaban la
4876             # respuesta anterior en silencio - la persona que ya la había
4877             # leído (o recibido por correo) se quedaba sin saber que
4878             # cambió, y se mandaba un segundo correo real de mas.
4879             raise HTTPException(409, "Ese mensaje ya fue respondido.")
4880         m.respuesta = respuesta
4881         m.respondido = ahora()
4882         s.commit()
4883         remitente = s.get(Usuario, m.remitente_id)
4884         nombre_remitente = remitente.nombre if remitente else "?"
4885         correo_remitente = remitente.correo if remitente else None
4886         registrar(u, "SOPORTE", f"{u.nombre} le respondió a {nombre_remitente}: {respuesta}")
4887         return {"ok": True, "destinatario": nombre_remitente, "correo_destinatario": correo_remitente}
4888
4889
4890 @app.get("/api/soporte/administrador")
4891 def administrador_de_turno(u: Usuario = Depends(usuario_actual)):
4892     """Con quien puede contactarse cualquier persona (no solo el
4893     administrador) desde Ayuda - sin exponer el resto del listado de
4894     usuarios, que si es exclusivo del administrador. Si quien pregunta ya
4895     es administrador, no tiene sentido mostrarse a si mismo como contacto:
4896     se busca a OTRO administrador, y si no hay ninguno mas, no hay a quien
4897     escribirle (para eso esta la mesa de ayuda de Colsubsidio aparte)."""
4898     with Sesion() as s:
4899         admin = (s.query(Usuario)
4900                 .filter(Usuario.perfil == "auditor", Usuario.activo == 1,
4901                        Usuario.id != u.id)
4902                 .first())
4903     if not admin:
4904         return {"nombre": None, "correo": None, "es_usted": u.perfil == "auditor"}
4905     return {"nombre": admin.nombre, "correo": admin.correo, "es_usted": False}
4906
4907
4908 @app.get("/api/usuarios/yo")
4909 def ver_perfil(u: Usuario = Depends(usuario_actual)):
4910     from agente.cerebro import VOCES, VOZ_DEFECTO
4911     dias_pin = (ahora() - (u.pin_actualizado or ahora())).days
4912     # idioma_voz guardaba un codigo de idioma de navegador (es-MX, es-CO...)
4913     # de cuando la voz salia por speechSynthesis; ahora guarda la clave de
4914     # la voz neuronal elegida (kore, puck...). Una cuenta con el valor
4915     # viejo cae a la voz por defecto en vez de romper el selector.
4916     voz = u.idioma_voz if u.idioma_voz in VOCES else VOZ_DEFECTO
4917     return {"id": u.id, "nombre": u.nombre, "correo": u.correo, "telefono": u.telefono,
4918           "codigo": u.codigo, "perfil": u.perfil,
4919           "ultimo_acceso": u.ultimo_acceso.strftime("%Y-%m-%d %H:%M") if u.ultimo_acceso else None,
4920           "pin_vence_en_dias": max(90 - dias_pin, 0),
4921           "idioma_voz": voz, "velocidad_voz": u.velocidad_voz,
4922           "confirmacion_hablada": bool(u.confirmacion_hablada)}
4923
4924
4925 @app.get("/api/usuarios/yo/resumen")
4926 def resumen_inicio(u: Usuario = Depends(usuario_actual)):
4927     hoy = ahora().date()
4928     inicio_mes = hoy.replace(day=1)
4929     with Sesion() as s:
4930         n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=u.id).count()
4931         sesiones_usr = [x.id for x in s.query(SesionConteo).filter_by(usuario_id=u.id).all()]
4932         ref_hoy = 0
4933         if sesiones_usr:
4934             ref_hoy = sum(1 for c in (s.query(Conteo)
4935                                     .filter(Conteo.sesion_id.in_(sesiones_usr),
4936                                            Conteo.estado == "confirmado").all()))

```

```

4937         if c.creado.date() == hoy)
4938     alertas_abiertas = s.query(Alerta).filter_by(resuelta=0).count()
4939     # "Su exactitud del mes" tenia que decir de verdad SU exactitud (las
4940     # bodegas donde esta persona contó o auditó) y de verdad DEL MES
4941     # actual - antes promediaba el historial de cierres de TODAS las
4942     # bodegas de TODA la operación, sin filtrar ni por persona ni por
4943     # fecha, así que un auxiliar nuevo sin ningún cierre propio veía el
4944     # mismo número que la administradora con mas experiencia.
4945     bodegas_suyas = {ses.bodega_id for ses in s.query(SesionConteo)
4946                     .filter(SesionConteo.usuario_id == u.id).all()}
4947     historial_mes = (s.query(HistorialCierre)
4948                     .filter(HistorialCierre.bodega_id.in_(bodegas_suyas),
4949                             HistorialCierre.fecha >= inicio_mes).all()
4950                     if bodegas_suyas else [])
4951     exact_mes = (round(sum(h.exactitud for h in historial_mes) / len(historial_mes), 1)
4952                 if historial_mes else 100.0)
4953     return {"bodegas_asignadas": n_bodegas, "referencias_hoy": ref_hoy,
4954            "alertas_por_revisar": alertas_abiertas, "exactitud_mes": exact_mes}
4955
4956
4957 @app.put("/api/usuarios/yo/preferencias")
4958 def guardar_preferencias(body: dict, u: Usuario = Depends(usuario_actual)):
4959     with Sesion() as s:
4960         usr = s.get(Usuario, u.id)
4961         for k in ("idioma_voz", "velocidad_voz"):
4962             if k in body:
4963                 setattr(usr, k, body[k])
4964         if "confirmacion_hablada" in body:
4965             usr.confirmacion_hablada = 1 if body["confirmacion_hablada"] else 0
4966         s.commit()
4967     return {"ok": True}
4968
4969
4970 @app.post("/api/usuarios/yo/marcar-clave-cambiada")
4971 def marcar_clave_cambiada(u: Usuario = Depends(usuario_actual)):
4972     """Cognito es quien cambia la clave (Mi perfil o "olvide mi clave"),
4973     nunca este backend - pero el aviso de "su clave vence en N dias" (ver
4974     ver_perfil) se calcula desde pin_actualizado, que sin esta llamada se
4975     queda congelado en la fecha de creacion de la cuenta para siempre. El
4976     frontend llama esto justo despues de que Cognito confirma el cambio."""
4977     with Sesion() as s:
4978         usr = s.get(Usuario, u.id)
4979         usr.pin_actualizado = ahora()
4980         s.commit()
4981     return {"ok": True}
4982
4983
4984 @app.post("/api/usuarios/yo/cerrar-todas")
4985 def cerrar_todas_sesiones(u: Usuario = Depends(usuario_actual)):
4986     """Antes invalidaba con un contador propio (version_token) y devolvía
4987     un token nuevo firmado por nosotros; ahora la identidad la maneja
4988     Cognito, así que se le pide a Cognito que revoke los refresh tokens
4989     de esta persona (AdminUserGlobalSignOut: nadie con sesion abierta en
4990     otro dispositivo puede volver a pedir un access token nuevo). El
4991     access token que ya tenía emitido cada dispositivo sigue firmando
4992     válido hasta que vence solo (el User Pool los emite con vida corta,
4993     1 hora, para que esa ventana sea chica)."""
4994     cliente = _cliente_cognito()
4995     try:
4996         cliente.admin_user_global_sign_out(UserPoolId=COGNITO_USER_POOL_ID, Username=u.nombre)
4997     except Exception as e:
4998         raise HTTPException(502, f"No se pudo cerrar las sesiones en este momento: {e}")
4999     registrar(u, "SEGURIDAD", "Sesion cerrada en todos los dispositivos")
5000     return {"ok": True}
5001
5002
5003 def _tipo_imagen_real(contenido: bytes) -> str | None:
5004     """El Content-Type que manda el navegador lo elige quien sube el
5005     archivo, no el archivo: antes se guardaba y se devolvía tal cual al
5006     pedir la foto, así que subir algo con Content-Type "text/html" lo
5007     serviría después como HTML. Aquí se detecta el tipo real por los
5008     primeros bytes (firma del formato), no por lo que diga la cabecera."""
5009     if contenido.startswith(b"\xff\xd8\xff"):
5010         return "image/jpeg"

```

```

5011     if contenido.startswith(b"\x89PNG\r\n\x1a\n"):
5012         return "image/png"
5013     if contenido.startswith((b"GIF87a", b"GIF89a")):
5014         return "image/gif"
5015     if contenido[:4] == b"RIFF" and contenido[8:12] == b"WEBP":
5016         return "image/webp"
5017     return None
5018
5019
5020 @app.post("/api/usuarios/yo/foto")
5021 async def subir_foto(foto: UploadFile = File(...),
5022                     u: Usuario = Depends(usuario_actual)):
5023     contenido = await foto.read()
5024     if len(contenido) > 3 * 1024 * 1024:
5025         raise HTTPException(400, "La foto no puede pesar mas de 3 MB.")
5026     tipo_real = _tipo_imagen_real(contenido)
5027     if tipo_real is None:
5028         raise HTTPException(400, "El archivo no es una imagen valida (JPEG, PNG, GIF o WebP).")
5029     with Sesion() as s:
5030         usr = s.get(Usuario, u.id)
5031         usr.foto = contenido
5032         usr.foto_tipo = tipo_real
5033         s.commit()
5034     return {"ok": True}
5035
5036
5037 @app.get("/api/usuarios/yo/foto")
5038 def ver_foto(u: Usuario = Depends(usuario_actual)):
5039     with Sesion() as s:
5040         usr = s.get(Usuario, u.id)
5041         if not usr.foto:
5042             raise HTTPException(404, "Sin foto.")
5043         return Response(content=usr.foto, media_type=usr.foto_tipo or "image/jpeg")
5044
5045
5046 @app.get("/api/voz/voces")
5047 def voces_disponibles(u: Usuario = Depends(usuario_actual)):
5048     from agente.cerebro import VOCES, VOZ_DEFECTO
5049     return {"voces": [{"clave": k, **v} for k, v in VOCES.items()],
5050           "defecto": VOZ_DEFECTO}
5051
5052
5053 class HablarIn(BaseModel):
5054     texto: str
5055     voz: str = "kore"
5056
5057
5058 @app.post("/api/voz/hablar")
5059 def api_hablar(body: HablarIn, u: Usuario = Depends(usuario_actual)):
5060     from agente.cerebro import sintetizar_voz
5061     texto = (body.texto or "").strip()
5062     if not texto:
5063         raise HTTPException(400, "Falta el texto.")
5064     wav = sintetizar_voz(texto, body.voz)
5065     if wav is None:
5066         raise HTTPException(503, "Voz neuronal no disponible en este momento.")
5067     return Response(content=wav, media_type="audio/wav")
5068
5069
5070 def _filtro_traza(s, persona: str, accion: str, rango: str):
5071     q = s.query(Traza)
5072     if persona:
5073         q = q.filter_by(persona=persona)
5074     if accion:
5075         q = q.filter_by(accion=accion)
5076     if rango == "hoy":
5077         q = q.filter(Traza.creado >= ahora().replace(hour=0, minute=0, second=0))
5078     elif rango == "semana":
5079         q = q.filter(Traza.creado >= ahora() - timedelta(days=7))
5080     elif rango == "mes":
5081         q = q.filter(Traza.creado >= ahora() - timedelta(days=30))
5082     return q
5083
5084

```



```

5085 @app.get("/api/trazabilidad")
5086 def ver_traza(persona: str = "", accion: str = "", rango: str = "",
5087             u: Usuario = Depends(requiere_perfil("auditor"))):
5088     with Sesion() as s:
5089         q = _filtro_traza(s, persona, accion, rango)
5090         return [{"id": t.id, "hora": t.creado.strftime("%H:%M:%S"),
5091                 "persona": t.persona, "accion": t.accion,
5092                 "detalle": t.detalle, "tipo": t.tipo}
5093                for t in q.order_by(Traza.id.desc()).limit(200)]
5094
5095
5096 @app.get("/api/trazabilidad/exportar")
5097 def exportar_traza(persona: str = "", accion: str = "", rango: str = "",
5098                  formato: str = "xlsx",
5099                  u: Usuario = Depends(requiere_perfil("auditor"))):
5100     import pandas as pd
5101     from servicios.archivos import guardar_df
5102     with Sesion() as s:
5103         q = _filtro_traza(s, persona, accion, rango)
5104         filas = [{"fecha": t.creado.strftime("%Y-%m-%d %H:%M:%S"), "persona": t.persona,
5105                 "accion": t.accion, "detalle": t.detalle}
5106                for t in q.order_by(Traza.id.desc()).all()]
5107         df = pd.DataFrame(filas)
5108         marca = ahora().strftime("%Y%m%d_%H%M")
5109         ruta = f"reportes/trazabilidad_{marca}.{formato}"
5110         guardar_df(ruta, df, formato)
5111         registrar(u, "REPORTE", f"Registro de trazabilidad exportado: {ruta} ({len(df)} filas)")
5112         return {"archivo": ruta, "filas": len(df)}
5113
5114
5115 @app.get("/api/ajustes")
5116 def ver_ajustes(u: Usuario = Depends(usuario_actual)):
5117     with Sesion() as s:
5118         offline = s.get(ConfigClave, "offline")
5119         usuarios_activos = s.query(Usuario).filter_by(activo=1).count()
5120         aprobaciones_pend = s.query(Aprobacion).filter_by(estados="pendiente").count()
5121         pedidos_pend = (s.query(LineaServicio.numero_pedido)
5122                        .filter_by(estados="pendiente_aprobacion")
5123                        .distinct().count())
5124     return {"umbral": round(umbral_actual() * 100),
5125           "bloquear_negativos": True, "confirmar_alertas": True,
5126           "offline": (offline.valor == "1") if offline else True,
5127           "version": "1.0.0", "modelo": os.getenv("MODELO", "gemini-flash-latest"),
5128           "idioma_voz": os.getenv("IDIOMA_VOZ", "es-CO"),
5129           "base_datos": "SQLite",
5130           "usuarios_activos": usuarios_activos,
5131           "aprobaciones_pendientes": aprobaciones_pend + pedidos_pend}
5132
5133
5134 @app.put("/api/ajustes")
5135 def guardar_ajustes(body: dict, u: Usuario = Depends(requiere_perfil("auditor"))):
5136     with Sesion() as s:
5137         if "umbral" in body:
5138             valor = str(max(1, min(500, float(body["umbral"]))) / 100)
5139             existente = s.get(ConfigClave, "umbral_anomalia")
5140             if existente:
5141                 existente.valor = valor
5142             else:
5143                 s.add(ConfigClave(clave="umbral_anomalia", valor=valor))
5144         if "offline" in body:
5145             valor = "1" if body["offline"] else "0"
5146             existente = s.get(ConfigClave, "offline")
5147             if existente:
5148                 existente.valor = valor
5149             else:
5150                 s.add(ConfigClave(clave="offline", valor=valor))
5151         s.commit()
5152         registrar(u, "AJUSTE", "Configuracion del sistema actualizada", "ok")
5153         return {"ok": True}
5154
5155
5156 # ----- tablero en vivo -----
5157 conexiones: list[WebSocket] = []
5158

```



```

5159
5160 def estado_bodegas():
5161     with Sesion() as s:
5162         return [{"bodega": b.nombre_oficial, "estado": b.estado, "id": b.id}
5163                 for b in s.query(Bodega).order_by(Bodega.nombre_oficial).all()]
5164
5165
5166 async def difundir_estado():
5167     for ws in list(conexiones):
5168         try:
5169             await ws.send_json(estado_bodegas())
5170         except Exception:
5171             if ws in conexiones:
5172                 conexiones.remove(ws)
5173
5174
5175 @app.websocket("/api/bodegas/estado")
5176 async def ws_estado(ws: WebSocket):
5177     from seguridad import verificar_token
5178     if not verificar_token(ws.query_params.get("token")):
5179         await ws.close(code=1008)
5180         return
5181     await ws.accept()
5182     conexiones.append(ws)
5183     await ws.send_json(estado_bodegas())
5184     try:
5185         while True:
5186             await ws.receive_text()
5187     except WebSocketDisconnect:
5188         if ws in conexiones:
5189             conexiones.remove(ws)

```

backend/modelos.py

250 LÍNEAS

Las 18 tablas en SQLAlchemy. Es el mapa del dominio.

```

1  """Las tablas del sistema: el diagrama de clases hecho código."""
2  from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, LargeBinary
3  from bd import Base
4  from horario import ahora
5
6
7  class Usuario(Base):
8      __tablename__ = "usuario"
9      id = Column(Integer, primary_key=True)
10     # unique=True: antes solo se evitaba un nombre duplicado revisando "ya
11     # existe" justo antes de crear (main.py:crear_usuario, orquestador.py:
12     # _crear_usuario_por_voz) - dos creaciones casi simultaneas del mismo
13     # nombre podian pasar esa revision las dos y quedar dos cuentas con el
14     # mismo nombre, rompiendo el ingreso (que busca por nombre y solo
15     # devuelve la primera que encuentra) de forma impredecible para
16     # cualquiera de las dos personas.
17     nombre = Column(String, nullable=False, unique=True)
18     perfil = Column(String, nullable=False) # auxiliar | auditor
19     # La clave la maneja Cognito, no esta base - esta columna ya no se
20     # llena (nullable=True: SQLite no permite quitarle el NOT NULL a una
21     # columna existente sin recrear la tabla; se deja aqui sin usar en
22     # vez de arriesgar esa migracion en una base ya desplegada).
23     clave_hash = Column(String, nullable=True)
24     correo = Column(String, default="")
25     telefono = Column(String, default="")
26     codigo = Column(String, default="")
27     activo = Column(Integer, default=1)
28     # NULL en cualquier cuenta creada antes de esta columna (semilla,
29     # creada a mano por un auditor, etc.) - nunca se migran, quedan fuera
30     # del filtro de aprobacion para siempre. Solo el autoregistro
31     # (main.py: registro_completado) pone 0 aqui; un auditor lo cambia a 1
32     # (aprobado) o -1 (rechazado, activo se queda en 0 para siempre).
33     aprobado = Column(Integer, nullable=True)
34     pin_actualizado = Column(DateTime, default=ahora)
35     ultimo_acceso = Column(DateTime, nullable=True)
36     idioma_voz = Column(String, default="es-MX")

```

```

37     velocidad_voz = Column(String, default="normal")      # lenta | normal | rapida
38     confirmacion_hablada = Column(Integer, default=1)
39     # la foto va en la misma base (no en disco): el disco del Web Service
40     # en Render es efimero y se borra en cada deploy/reinicio, la base no.
41     foto = Column(LargeBinary, nullable=True)
42     foto_tipo = Column(String, nullable=True)              # ej. "image/jpeg"
43
44
45 class AsignacionBodega(Base):
46     """Qué bodegas puede contar cada persona."""
47     __tablename__ = "asignacion_bodega"
48     id = Column(Integer, primary_key=True)
49     usuario_id = Column(Integer, ForeignKey("usuario.id"))
50     bodega_id = Column(Integer, ForeignKey("bodega.id"))
51
52
53 class Sede(Base):
54     """Una sede fisica: Piscilago, Calle 26, el club de la 195...
55
56     Agrupa bodegas para poder repartirlas de una vez y para ver la
57     operacion por sitio. NO decide permisos: eso lo sigue decidiendo
58     AsignacionBodega, bodega por bodega. Son dos ejes distintos a
59     proposito - una persona puede tener tres bodegas de una sede y una de
60     otra, que es como ocurre de verdad cuando alguien cubre un turno."""
61     __tablename__ = "sede"
62     id = Column(Integer, primary_key=True)
63     nombre = Column(String, unique=True, nullable=False)
64     ciudad = Column(String, default="")
65
66
67 class Bodega(Base):
68     __tablename__ = "bodega"
69     id = Column(Integer, primary_key=True)
70     nombre_oficial = Column(String, unique=True, nullable=False)
71     estado = Column(String, default="pendiente")           # pendiente | en_conteo | en_auditoria | cerrada
72     # NULL = sin sede. Es el estado de las 54 bodegas que ya existian
73     # cuando se agrego esto, y sigue siendo valido: una bodega sin sede se
74     # comporta igual que siempre. Nada obliga a clasificarlas todas.
75     sede_id = Column(Integer, ForeignKey("sede.id"), nullable=True)
76
77
78 class Articulo(Base):
79     __tablename__ = "articulo"
80     codigo = Column(String, primary_key=True)
81     nombre_oficial = Column(String, nullable=False)
82     unidad_medida = Column(String, nullable=False)
83
84
85 class StockSistema(Base):
86     """El extracto de My Inventory: lo que el sistema cree que hay."""
87     __tablename__ = "stock_sistema"
88     id = Column(Integer, primary_key=True)
89     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
90     bodega_id = Column(Integer, ForeignKey("bodega.id"))
91     cantidad_sd = Column(Float, default=0)
92
93
94 class AliasArticulo(Base):
95     """Cómo habla el equipo de verdad. El agente lo va aprendiendo."""
96     __tablename__ = "alias_articulo"
97     id = Column(Integer, primary_key=True)
98     texto_dicho = Column(String, index=True)
99     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
100    bodega_id = Column(Integer, nullable=True)
101
102
103 class SesionConteo(Base):
104     __tablename__ = "sesion_conteo"
105     id = Column(Integer, primary_key=True)
106     bodega_id = Column(Integer, ForeignKey("bodega.id"))
107     usuario_id = Column(Integer, ForeignKey("usuario.id"))
108     tipo = Column(String, default="conteo")                # conteo | auditoria
109     estado = Column(String, default="abierta")
110     firmada = Column(Integer, default=0)                    # firma digital de cierre

```

```

111     inicio = Column(DateTime, default=ahora)
112     fin = Column(DateTime, nullable=True)
113
114
115 class Conteo(Base):
116     __tablename__ = "conteo"
117     id = Column(Integer, primary_key=True)
118     sesion_id = Column(Integer, ForeignKey("sesion_conteo.id"))
119     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
120     cantidad = Column(Float, nullable=False)
121     unidad = Column(String)
122     estado = Column(String, default="confirmado")
123     # confirmado | corregido | pendiente_aprobacion | borrador
124     corrige_a = Column(Integer, ForeignKey("conteo.id"), nullable=True)
125     creado = Column(DateTime, default=ahora)
126
127
128 class Alerta(Base):
129     __tablename__ = "alerta"
130     id = Column(Integer, primary_key=True)
131     conteo_id = Column(Integer, ForeignKey("conteo.id"), nullable=True)
132     tipo = Column(String) # unidad | negativo | desviacion | inexistente
133     detalle = Column(String)
134     resuelta = Column(Integer, default=0)
135     creado = Column(DateTime, default=ahora)
136     resuelto = Column(DateTime, nullable=True)
137
138
139 class Traza(Base):
140     """Registro inmutable: solo crece, nunca se edita ni se borra."""
141     __tablename__ = "traza"
142     id = Column(Integer, primary_key=True)
143     usuario_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
144     persona = Column(String)
145     accion = Column(String) # APERTURA | CIERRE | CORRECCION | ALERTA...
146     detalle = Column(String)
147     tipo = Column(String, default="info")
148     creado = Column(DateTime, default=ahora)
149
150
151 # — los tres momentos del reto: recetas y servicios —
152 class Receta(Base):
153     __tablename__ = "receta"
154     id = Column(Integer, primary_key=True)
155     nombre = Column(String, nullable=False)
156     rendimiento = Column(Integer, default=1)
157     # paso a paso de cocina, no solo la lista de ingredientes con su cantidad.
158     preparacion = Column(String, default="")
159
160
161 class RecetaIngrediente(Base):
162     __tablename__ = "receta_ingredient"
163     id = Column(Integer, primary_key=True)
164     receta_id = Column(Integer, ForeignKey("receta.id"))
165     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
166     cantidad_por_porcion = Column(Float, nullable=False)
167
168
169 class LineaServicio(Base):
170     __tablename__ = "linea_servicio"
171     id = Column(Integer, primary_key=True)
172     servicio_id = Column(Integer, default=1)
173     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
174     nombre = Column(String)
175     pedido = Column(Float, default=0)
176     usado = Column(Float, default=0)
177     # abierto | legalizado | pendiente_aprobacion | rechazado
178     estado = Column(String, default="abierto")
179     plato = Column(String, default="")
180     porciones = Column(Integer, default=0)
181     creado = Column(DateTime, default=ahora)
182     bodega_id = Column(Integer, ForeignKey("bodega.id"), nullable=True)
183     creado_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
184     # agrupa las lineas de un mismo envío para poder aprobarlo/rechazarlo

```

```

185     # como una sola unidad, no linea por linea.
186     numero_pedido = Column(String, nullable=True)
187
188
189 # — aprobación en paralelo: productos y bodegas creados en plena toma —
190 class Aprobacion(Base):
191     __tablename__ = "aprobacion"
192     id = Column(Integer, primary_key=True)
193     tipo = Column(String, nullable=False)          # producto | bodega
194     nombre = Column(String, nullable=False)
195     unidad_medida = Column(String, nullable=True)
196     cantidad_inicial = Column(Float, nullable=True)
197     bodega_id = Column(Integer, ForeignKey("bodega.id"), nullable=True)
198     articulo_codigo = Column(String, ForeignKey("articulo.codigo"), nullable=True)
199     conteo_id = Column(Integer, ForeignKey("conteo.id"), nullable=True)
200     creado_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
201     estado = Column(String, default="pendiente")   # pendiente | aprobado | rechazado
202     creado = Column(DateTime, default=ahora)
203     resuelto_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
204     resuelto = Column(DateTime, nullable=True)
205
206
207 class HistorialCierre(Base):
208     """Una foto de la exactitud cada vez que una bodega cierra con doble firma."""
209     __tablename__ = "historial_cierre"
210     id = Column(Integer, primary_key=True)
211     bodega_id = Column(Integer, ForeignKey("bodega.id"))
212     exactitud = Column(Float, default=100)
213     referencias = Column(Integer, default=0)
214     diferencias = Column(Integer, default=0)
215     fecha = Column(DateTime, default=ahora)
216
217
218 class ConfigClave(Base):
219     """Ajustes editables desde la aplicación, con el .env como valor por defecto."""
220     __tablename__ = "config_clave"
221     clave = Column(String, primary_key=True)
222     valor = Column(String, nullable=False)
223
224
225 class ArchivoGenerado(Base):
226     """Los XLSX/CSV que se generan (consolidado, diferencias, análisis,
227     trazabilidad...) guardados en la base, no en disco - mismo motivo que
228     la foto de perfil: el disco del Web Service en Render es efímero y se
229     borra en cada deploy/reinicio, así que un archivo "generado hace una
230     hora" debe poder seguir viéndose/descargándose después de un redeploy,
231     no solo hasta el próximo `git push`. """
232     __tablename__ = "archivo_generado"
233     id = Column(Integer, primary_key=True)
234     ruta = Column(String, nullable=False, unique=True)
235     contenido = Column(LargeBinary, nullable=False)
236     creado = Column(DateTime, default=ahora)
237
238
239 class MensajeSoporte(Base):
240     """Un mensaje de "Soporte en vivo" (auxiliar -> administrador) con su
241     respuesta, si ya la dio - la bandeja dentro de la app en Ayuda,
242     aparte del correo real que tambien se manda por fuera (EmailJS). """
243     __tablename__ = "mensaje_soporte"
244     id = Column(Integer, primary_key=True)
245     remitente_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)
246     destinatario_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)
247     mensaje = Column(String, nullable=False)
248     respuesta = Column(String, nullable=True)
249     creado = Column(DateTime, default=ahora)
250     respondido = Column(DateTime, nullable=True)

```

backend/bd.py

94 LÍNEAS

La conexion y la sesion. Lo unico que cambia entre SQLite y PostgreSQL vive aqui.

```

1  """Conexión a la base de datos. Todo lo demás importa de aquí."""
2  import os
3  from sqlalchemy import create_engine
4  from sqlalchemy.orm import sessionmaker, DeclarativeBase
5
6
7  class Base(DeclarativeBase):
8      pass
9
10
11
12  # Ruta absoluta anclada a backend/, para que sqlite apunte siempre al mismo
13  # archivo sin importar desde que carpeta se ejecute (uvicorn corre desde
14  # backend/, pero cargar_excel.py corre desde data/).
15  _RUTA_SQLITE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cuentavoz.db")
16  DB_URL = os.getenv("DB_URL", f"sqlite:/// {_RUTA_SQLITE}")
17  # Render (como antes Heroku) entrega la cadena de Postgres con el esquema
18  # viejo "postgres://"; SQLAlchemy 2.x ya no lo traduce solo y falla al
19  # arrancar si no se corrige aquí.
20  if DB_URL.startswith("postgres://"):
21      DB_URL = DB_URL.replace("postgres://", "postgresql://", 1)
22
23  # SQLite necesita este argumento extra cuando hay varios hilos (uvicorn)
24  kwargs = {"connect_args": {"check_same_thread": False}} if DB_URL.startswith("sqlite") else {}
25  motor = create_engine(DB_URL, **kwargs)
26  Sesion = sessionmaker(bind=motor, expire_on_commit=False)
27
28
29  def iniciar_bd():
30      import modelos # noqa: F401 (registra todas las tablas)
31      Base.metadata.create_all(motor)
32      _migran_columnas_faltantes()
33      _migran_columnas_ya_opcionales()
34
35
36  def _migran_columnas_faltantes():
37      """create_all() solo crea tablas nuevas, nunca agrega columnas a una
38      tabla que ya existe. Sin esto, una columna agregada al modelo (como
39      Usuario.foto) se ve localmente porque cuentavoz.db se recrea facil,
40      pero nunca llega a la base de Postgres ya desplegada en Render - hay
41      que agregarla a mano en la tabla que ya esta viva. Corre en cada
42      arranque y no hace nada si ya esta al dia."""
43      from sqlalchemy import inspect, text
44      inspector = inspect(motor)
45      with motor.begin() as conexion:
46          for tabla in Base.metadata.sorted_tables:
47              if not inspector.has_table(tabla.name):
48                  continue
49              existentes = {c["name"] for c in inspector.get_columns(tabla.name)}
50              for columna in tabla.columns:
51                  if columna.name in existentes:
52                      continue
53                  tipo = columna.type.compile(dialect=motor.dialect)
54                  conexion.execute(text(
55                      f'ALTER TABLE {tabla.name} ADD COLUMN {columna.name} {tipo}'))
56                  print(f"[bd] columna agregada: {tabla.name}.{columna.name}")
57
58
59  def _migran_columnas_ya_opcionales():
60      """Quita el NOT NULL de las columnas que el modelo ya declara opcionales.
61
62      _migran_columnas_faltantes solo AGREGA columnas; nunca relaja una
63      restriccion existente. Eso dejo un fallo real y silencioso en
64      produccion: al pasar la identidad a Cognito, Usuario.clave_hash se
65      volvio nullable=True en el modelo (ya nadie la llena, la clave la
66      guarda Cognito), pero la tabla de Postgres seguia con el NOT NULL de
67      cuando se creo. En local no se noto porque cuentavoz.db se recrea de
68      cero; en Render, CADA insercion de usuario fallaba - autoregistro y
69      "crear usuario" desde Ajustes - con un 500 que ademas llegaba al
70      navegador sin cabeceras CORS, o sea disfrazado de "Sin conexion con el
71      servidor. Revise el Wi-Fi".
72
73      Solo en Postgres: SQLite no sabe hacer ALTER COLUMN, y alla no hace
74      falta. Nunca toca la llave primaria, que debe seguir siendo NOT NULL.

```

```

75     """
76     if not motor.dialect.name.startswith("postgres"):
77         return
78     from sqlalchemy import inspect, text
79     inspector = inspect(motor)
80     with motor.begin() as conexion:
81         for tabla in Base.metadata.sorted_tables:
82             if not inspector.has_table(tabla.name):
83                 continue
84             en_bd = {c["name"]: c for c in inspector.get_columns(tabla.name)}
85             for columna in tabla.columns:
86                 actual = en_bd.get(columna.name)
87                 if actual is None or columna.primary_key:
88                     continue
89                 # la base exige valor pero el modelo ya dice que es opcional
90                 if columna.nullable and not actual.get("nullable", True):
91                     conexion.execute(text(
92                         f'ALTER TABLE {tabla.name} '
93                         f'ALTER COLUMN {columna.name} DROP NOT NULL'))
94                 print(f"[bd] ahora opcional: {tabla.name}.{columna.name}")

```

backend/seguridad.py

168 LÍNEAS

Verificacion del access token de Cognito y el gate por perfil.

```

1  """Identidad: verificacion de tokens de AWS Cognito y permisos por perfil.
2
3  La contraseña, el registro y el login los maneja Cognito directamente (el
4  frontend habla con el SDK de Cognito, nunca manda la clave a este backend).
5  Lo unico que hace este modulo es comprobar que un access token que llega en
6  la cabecera Authorization de verdad lo firmo NUESTRO User Pool de Cognito -
7  firma (RS256, con las llaves publicas que Cognito publica), emisor,
8  audiencia (client_id) y que sea un access token, no un id token."""
9  import os
10 import jwt
11 from fastapi import Depends, HTTPException
12 from fastapi.security import OAuth2PasswordBearer
13 from bd import Sesion
14 from modelos import Usuario
15
16 # El User Pool Id y el App Client Id NO son secretos: identifican al pool,
17 # no autorizan nada, y de hecho ya viajan dentro del JavaScript que
18 # descarga cualquiera que abra la aplicacion (ver frontend/src/cognito.js,
19 # donde llevan exactamente estos mismos valores por defecto). Lo secreto
20 # son las claves de las personas (que las guarda Cognito, no este backend)
21 # y las credenciales AWS_* (que siguen siendo solo variables de entorno).
22 #
23 # Llevan valor por defecto por la misma razon que VITE_API_URL en api.js o
24 # DB_URL en bd.py: que el proyecto funcione sin depender de que alguien se
25 # acuerde de llenar una variable. Antes el backend era la unica pieza que
26 # NO lo hacia, y el resultado fue un despliegue en el que estas dos
27 # quedaron vacias y TODOS los tokens se rechazaron con "Sesion invalida o
28 # vencida." - un mensaje que ademas culpaba a la persona equivocada.
29 # Definir la variable de entorno sigue mandando: apuntar a otro User Pool
30 # (otro entorno, otra cuenta de AWS) es solo ponerla.
31 COGNITO_REGION = os.getenv("COGNITO_REGION", "").strip() or "us-east-2"
32 COGNITO_USER_POOL_ID = os.getenv("COGNITO_USER_POOL_ID", "").strip() or "us-east-2_6HbrPruvL"
33 COGNITO_APP_CLIENT_ID = os.getenv("COGNITO_APP_CLIENT_ID", "").strip() or "847jtkc5sem7mr4tb8csrrkqp"
34 _EMISOR = f"https://cognito-idp.{COGNITO_REGION}.amazonaws.com/{COGNITO_USER_POOL_ID}"
35 _JWKS_URL = f"{_EMISOR}/.well-known/jwks.json"
36
37 # PyJWKClient trae en cache las llaves publicas del User Pool (se refrescan
38 # solas si aparece un "kid" nuevo que no conoce todavia) - sin esto habria
39 # que ir a buscarlas a Cognito en cada peticion.
40 #   lifespan: las llaves de firma de un User Pool son estables (no rotan
41 #   cada rato). Con el valor por defecto (300 s) el backend volvia a
42 #   pedir el JWKS a AWS cada 5 minutos, y CADA una de esas idas a la red
43 #   era una oportunidad de que un tropiezo tumbara la sesion de todos
44 #   (ver _reclamos_cognito). Una hora reduce esas idas ~12 veces.
45 #   timeout: 30 s por defecto es demasiado - deja la peticion colgada. Es
46 #   preferible fallar rapido y reintentar.
47 _jwks_client = (jwt.PyJWKClient(_JWKS_URL, cache_keys=True, lifespan=3600, timeout=8)

```

```

48         if COGNITO_USER_POOL_ID else None)
49 esquema = OAuth2PasswordBearer(tokenUrl="/api/registro-completado", auto_error=False)
50
51
52 class ServicioIdentidadCaído(Exception):
53     """No se pudieron consultar las llaves publicas de Cognito.
54
55     Ojo con la distincion, que es la razon de ser de esta excepcion: esto
56     NO significa que el token sea malo, significa que este backend no pudo
57     hablar con AWS para comprobarlo. Antes ambos casos terminaban en el
58     mismo 401 "Sesion invalida o vencida.", y como el frontend cierra la
59     sesion ante cualquier 401 (ver api.js: alSesionInvalida), un tropiezo
60     de red de un segundo contra el JWKS sacaba a la persona de la
61     aplicacion y la mandaba de vuelta al login con un mensaje que ademas
62     era mentira. Se responde 503 en su lugar."""
63
64
65 def _llave_de_firma(token: str):
66     """La llave publica con la que Cognito firmo este token.
67
68     PyJWT ya reintenta solo (refrescando el JWKS) cuando el "kid" no esta
69     en la cache; lo que no reintenta es un fallo de RED al traer ese JWKS,
70     que es justo el caso que aqui interesa cubrir."""
71     from jwt.exceptions import PyJWKClientConnectionError
72     try:
73         return _jwks_client.get_signing_key_from_jwt(token).key
74     except PyJWKClientConnectionError as e:
75         print(f"[seguridad] no se pudo traer el JWKS de Cognito ({e}); reintentando")
76         try:
77             return _jwks_client.get_signing_key_from_jwt(token).key
78         except PyJWKClientConnectionError as e2:
79             raise ServicioIdentidadCaído(str(e2)) from e2
80
81
82 def _reclamos_cognito(token: str) -> dict | None:
83     """Verifica un access token de Cognito y devuelve sus datos, o None si
84     no es valido, vencio, es de otro User Pool/App Client, o es un id token
85     en vez de un access token (llevan proposito distinto: el access token
86     es para hablar con la API, el id token es para identificar a la
87     persona ante el propio frontend).
88
89     Levanta ServicioIdentidadCaído si el problema no es el token sino la
90     conexion con Cognito."""
91     if not _jwks_client:
92         return None
93     try:
94         llave = _llave_de_firma(token)
95         datos = jwt.decode(
96             token, llave, algorithms=["RS256"], issuer=_EMISOR,
97             options={"require": ["exp", "iss", "sub", "token_use", "client_id", "username"]},
98         )
99     except jwt.PyJWTError:
100         return None
101     if datos.get("token_use") != "access" or datos.get("client_id") != COGNITO_APP_CLIENT_ID:
102         return None
103     return datos
104
105
106 def usuario_actual(token: str = Depends(esquema)) -> Usuario:
107     if not token:
108         raise HTTPException(401, "Falta la sesion.")
109     try:
110         datos = _reclamos_cognito(token)
111     except ServicioIdentidadCaído as e:
112         # 503 y no 401 a proposito: el token esta bien, lo que fallo fue la
113         # consulta a Cognito. Un 401 aqui cerraria la sesion de la persona
114         # (ver api.js) por un problema que no es suyo ni de su token.
115         print(f"[seguridad] Cognito no responde: {e}")
116         raise HTTPException(
117             503, "No se pudo verificar su sesion en este momento. Intente de nuevo.")
118     if datos is None:
119         raise HTTPException(401, "Sesion invalida o vencida.")
120     nombre = (datos.get("username") or "").lower()
121     with Sesión() as s:

```

```

122     u = s.query(Usuario).filter_by(nombre=nombre).first()
123     if u is None:
124         raise HTTPException(401, "Usuario no reconocido.")
125     if not u.activo:
126         raise HTTPException(403, "Ese usuario esta inactivo.")
127     return u
128
129
130 def verificar_token(token: str):
131     """Para WebSockets: el esquema OAuth2 exige el header Authorization, que
132     un WebSocket del navegador no puede mandar, asi que aqui se valida el
133     token pasado por query string con la misma regla que usuario_actual."""
134     if not token:
135         return None
136     try:
137         datos = _reclamos_cognito(token)
138     except ServicioIdentidadCaido as e:
139         # Un WebSocket no tiene forma de decir "reintente": se rechaza la
140         # conexion y el frontend la vuelve a abrir sola mas adelante.
141         print(f"[seguridad] Cognito no responde (websocket): {e}")
142         return None
143     if datos is None:
144         return None
145     nombre = (datos.get("username") or "").lower()
146     with Sesion() as s:
147         u = s.query(Usuario).filter_by(nombre=nombre).first()
148     if u is None or not u.activo:
149         return None
150     return u
151
152
153 def requiere_perfil(perfil: str):
154     def guardia(u: Usuario = Depends(usuario_actual)) -> Usuario:
155         if u.perfil != perfil:
156             raise HTTPException(403, "Su perfil no permite esta accion.")
157         return u
158     return guardia
159
160
161 def registrar(usuario, accion: str, detalle: str, tipo: str = "info"):
162     """Deja rastro de una accion sensible. El registro solo crece."""
163     from modelos import Traza
164     with Sesion() as s:
165         s.add(Traza(usuario_id=getattr(usuario, "id", None),
166                    persona=getattr(usuario, "nombre", "sistema"),
167                    accion=accion, detalle=detalle, tipo=tipo))
168     s.commit()

```

backend/reportes.py

86 LÍNEAS

Armado de los archivos XLSX con los codigos oficiales.

```

1  """El endpoint del flujo: informacion limpia lista para el ERP."""
2  import pandas as pd
3  from sqlalchemy import text
4  from bd import motor
5  from horario import ahora
6  from servicios.archivos import guardar_df
7
8  # Los nombres de columna son a proposito los mismos que trae el extracto
9  # real de Colsubsidio (ver data/BODEGAS_Y_STOCK.xlsx: "Nr.Articulo",
10 # "Artículo", "Unidad", "SD") - Bodega/Contado/Diferencia son las unicas
11 # columnas nuevas. Asi el archivo que se descarga aqui se reconoce de una
12 # en My Inventory (Oracle) en vez de traer nombres genericos inventados.
13 CONSULTA = """
14 SELECT a.codigo AS "Nr.Artículo", a.nombre_oficial AS "Artículo",
15        a.unidad_medida AS "Unidad", b.nombre_oficial AS "Bodega",
16        c.cantidad AS "Contado", st.cantidad_sd AS "SD",
17        c.cantidad - COALESCE(st.cantidad_sd, 0) AS "Diferencia"
18 FROM conteo c
19 JOIN articulo a ON a.codigo = c.articulo_codigo
20 JOIN sesion_conteo sc ON sc.id = c.sesion_id

```



```

21 JOIN bodega b ON b.id = sc.bodega_id
22 LEFT JOIN stock_sistema st ON st.articulo_codigo = a.codigo
23     AND st.bodega_id = b.id
24 WHERE c.estado = 'confirmado'
25 """
26
27
28 def consolidado(formato: str = "xlsx") -> tuple[str, int, list[dict]]:
29     df = pd.read_sql(CONSULTA, motor)
30     # "SD" sale NULL/NaN cuando el articulo no tiene fila de stock en esa
31     # bodega (LEFT JOIN); NaN no es JSON valido para la vista previa, y
32     # aqui significa lo mismo que en el resto de la app: no hay dato, se
33     # trata como 0.
34     df["SD"] = df["SD"].fillna(0)
35     df["Diferencia"] = df["Diferencia"].round(2)
36     marca = ahora().strftime("%Y%m%d_%H%M")
37     ruta = f"reportes/consolidado_{marca}.{formato}"
38     guardar_df(ruta, df, formato)
39     vista_previa = df.head(8).to_dict("records")
40     return ruta, len(df), vista_previa
41
42
43 def diferencias_archivo(formato: str = "xlsx") -> tuple[str, int, int]:
44     """«Diferencias por bodega» descargable: solo las filas donde el
45     conteo no cuadra con el sistema, no el consolidado completo."""
46     df = pd.read_sql(CONSULTA, motor)
47     df["Diferencia"] = df["Diferencia"].round(2)
48     df = df[df["Diferencia"] != 0]
49     marca = ahora().strftime("%Y%m%d_%H%M")
50     ruta = f"reportes/diferencias_{marca}.{formato}"
51     guardar_df(ruta, df, formato)
52     return ruta, len(df), df["Bodega"].nunique() if len(df) else 0
53
54
55 def detalle_bodega(bodega_id: int, formato: str = "xlsx") -> str:
56     """«Descargar detalle»: el conteo completo de una sola bodega."""
57     # text() - no un str plano - porque un str plano hace que pandas use
58     # exec_driver_sql() (ejecuta tal cual contra el driver, sin pasar por
59     # la traduccion de parametros de SQLAlchemy): en SQLite ":bodega_id"
60     # funciona porque el driver sqlite3 lo entiende nativo, pero en
61     # Postgres (psycopg2, que solo entiende %(nombre)s) ese ":bodega_id"
62     # llega como texto literal invalido y revienta con 500 - paso
63     # solo detectable en produccion, no en la base local.
64     df = pd.read_sql(text(CONSULTA + " AND b.id = :bodega_id"),
65                     motor, params={"bodega_id": bodega_id})
66     df["Diferencia"] = df["Diferencia"].round(2)
67     marca = ahora().strftime("%Y%m%d_%H%M")
68     ruta = f"reportes/bodega_{bodega_id}_{marca}.{formato}"
69     guardar_df(ruta, df, formato)
70     return ruta
71
72
73 ESTADO_BODEGAS = """
74 SELECT nombre_oficial AS bodega, estado
75 FROM bodega ORDER BY nombre_oficial
76 """
77
78
79 def estado_bodegas(formato: str = "xlsx") -> str:
80     """«Exportar estado»: la foto del tablero en vivo, para mandarla por
81     correo o adjuntarla sin tener que tomar una captura de pantalla."""
82     df = pd.read_sql(ESTADO_BODEGAS, motor)
83     marca = ahora().strftime("%Y%m%d_%H%M")
84     ruta = f"reportes/estado_bodegas_{marca}.{formato}"
85     guardar_df(ruta, df, formato)
86     return ruta

```

backend/horario.py

19 LÍNEAS

La hora local de la operacion, en un solo sitio.

```

1  """La hora de Bogotá para toda la aplicación.
2
3  El servidor (Render) corre en UTC, pero CuentaVoz es una sola operación
4  en Colombia: lo que se guarda en la base (creado, resuelto, inicio, fin...)
5  y lo que se muestra (trazabilidad, alertas, reportes) debe leerse en hora
6  de Bogotá, no en la del contenedor - si no, cada hora mostrada queda
7  adelantada 5 horas (confirmado en producción: un reporte generado a las
8  2:56pm quedó con el nombre de archivo en 19:56)."""
9  from datetime import datetime
10 from zoneinfo import ZoneInfo
11
12 _BOGOTA = ZoneInfo("America/Bogota")
13
14
15 def ahora() -> datetime:
16     """La hora de Bogotá ahora mismo, sin info de zona (naive) - para
17     seguir siendo comparable con las columnas datetime ya guardadas y con
18     el resto del código, que nunca maneja zonas horarias explícitas."""
19     return datetime.now(_BOGOTA).replace(tzinfo=None)

```

17.2 Backend · el agente

backend/agente/orquestador.py

724 LÍNEAS

El que dirige un turno de conversacion de punta a punta: recibe la frase, decide que hacer con ella, resuelve el articulo contra el catalogo, valida y contesta. Es el archivo mas importante del proyecto despues de main.py.

```

1  """Donde el pensar se vuelve accion. El agente propone, el backend dispone."""
2  from datetime import datetime
3  from bd import Sesion
4  from modelos import (Conteo, StockSistema, SesionConteo, Bodega, Alerta,
5                       Articulo, AsignacionBodega, Usuario)
6  from servicios.conciliacion import buscar_articulo, aprender_alias, buscar_bodega
7  from servicios.validacion import validar_conteo
8  from agente.cerebro import pensar
9
10 ESTADOS: dict[int, dict] = {}          # memoria viva por sesion
11
12 ALTA = 90
13 MEDIA = 60
14
15
16 def procesar_turno(texto: str, sesion_id: int, usuario=None,
17                   opciones_respaldo=None, opciones_para_respaldo=None,
18                   bodega_id_respaldo=None, bodega_nombre_respaldo=None,
19                   preparacion_respaldo=None, porciones_respaldo=None) -> dict:
20     est = ESTADOS.setdefault(sesion_id, {})
21     # el frontend usa sesion_id negativo para la conversacion de Pedidos,
22     # que no tiene bodega fisica que abrir - ver Pedido.jsx.
23     modo_pedido = sesion_id < 0
24
25     # resiliencia: si el backend se reinicio (se pierde ESTADOS en memoria)
26     # justo mientras la persona tenia una pregunta de "¿cual de los dos?"
27     # pendiente, el frontend ya sabe cuales eran las opciones - se las
28     # recuerda al backend en vez de dejar que pregunte lo mismo otra vez
29     # sin ningun recuerdo de la conversacion.
30     if opciones_respaldo and not est.get("opciones"):
31         est["opciones"] = opciones_respaldo
32         est["opciones_para"] = opciones_para_respaldo or "consultar"
33
34     # el mismo respaldo, para la bodega abierta: sin esto, un reinicio del
35     # backend a mitad de un conteo deja el frontend mostrando "en conteo"
36     # con la bodega en el titulo, mientras el servidor ya no tiene
37     # bodega_id - el agente termina diciendo "no hay ninguna bodega
38     # activa" aunque en pantalla se vea abierta, y "contar" cae en el
39     # caso de "articulo no encontrado" para cualquier cosa que se dicte.
40     if bodega_id_respaldo and not est.get("bodega_id"):
41         est["bodega_id"] = bodega_id_respaldo
42         est["bodega_nombre"] = bodega_nombre_respaldo or est.get("bodega_nombre")

```

```

43
44 # el mismo respaldo, para el plato y las porciones del pedido: el chef
45 # casi siempre los dicta en dos frases separadas ("arroz con pollo" y
46 # luego "para cuatro"), así que sin esto un reinicio de por medio deja
47 # al agente sin memoria del plato aunque la pantalla lo siga mostrando.
48 if preparacion_respaldo and not est.get("preparacion"):
49     est["preparacion"] = preparacion_respaldo
50 if porciones_respaldo and not est.get("porciones"):
51     est["porciones"] = porciones_respaldo
52
53 bodega_id = est.get("bodega_id")
54
55 if modo_pedido:
56     contexto = ("Estamos en el modulo de Pedidos: aqui se dicta el plato y las "
57                 "porciones para calcular insumos por receta, y se puede consultar "
58                 "el stock de un producto. NUNCA se abre ni se cuenta una bodega "
59                 "fisica aqui - eso es en el modulo de Conteo, no lo mencione. ")
60     # sin esto, Gemini no tiene memoria entre turnos: si el plato se dijo
61     # dos frases atras y esta ahora solo llegan las porciones, vuelve a
62     # preguntar "¿de que plato estamos hablando?" como si nunca se
63     # hubiera dicho - la persona termina repitiendo lo mismo varias veces.
64     if est.get("preparacion"):
65         contexto += f"El plato ya lo dijo antes: {est['preparacion']}. "
66     if est.get("porciones"):
67         contexto += f"Las porciones ya las dijo antes: {est['porciones']:g}. "
68     if est.get("preparacion") or est.get("porciones"):
69         contexto += ("No vuelva a preguntar por el plato ni las porciones si ya "
70                     "aparecen arriba: siga adelante con lo que falte, o confirme "
71                     "que ya tiene todo. ")
72 else:
73     contexto = f"Bodega activa: {est.get('bodega_nombre') or 'ninguna'}. "
74 if est.get("pendiente"):
75     p = est["pendiente"]
76     contexto += f"Pendiente de confirmar: {p['nombre']} {p['cantidad']:g}. "
77 if est.get("opciones"):
78     contexto += "Opciones ofrecidas: " + ", ".join(
79         o["nombre"] for o in est["opciones"]) + ". "
80
81 turno = pensar(contexto, texto)
82 turno.setdefault("respuesta_hablada", "")
83 intencion = (turno.get("intencion") or "").lower()
84
85 # Gemini a veces entiende un plato SIN receta registrada como si fuera
86 # un ingrediente a consultar/contar (confirmado: "arroz con pollo para
87 # 2 porciones" -> intencion "consultar", articulo_texto "arroz con
88 # pollo", unidad "Portion") - probablemente porque no reconoce el plato
89 # como una receta valida y cae al patron mas general. "Portion" nunca
90 # es la unidad real de un ingrediente de bodega, así que es la señal
91 # confiable de que en realidad se pidieron porciones de un plato. Sin
92 # esto, la persona recibia una desambiguacion de ingredientes sin
93 # ninguna relacion con lo que pidio (ej. "tengo arroz o arroz doña
94 # pepa. ¿cual de los dos?") en vez del mensaje claro de "no encuentre
95 # esa receta" que ya existe para un plato desconocido.
96 if (modo_pedido and intencion in ("contar", "consultar")
97     and not turno.get("preparacion") and turno.get("unidad") == "Portion"):
98     turno["preparacion"] = turno.get("articulo_texto") or texto
99     turno["porciones"] = turno.get("cantidad") or turno.get("porciones")
100    turno["intencion"] = "pedir"
101    intencion = "pedir"
102
103 # "cero porciones" (o un numero negativo) no es un pedido valido - sin
104 # esto, Gemini a veces lo aceptaba igual ("anotado cero porciones de
105 # ajiao") y el frontend, al tratar 0 como "sin dato" (0 es falsy en
106 # JS), terminaba en un estado inconsistente: el plato quedaba puesto
107 # pero las porciones sin guardar, sin ningun aviso claro de que hacia
108 # falta corregir el numero.
109 if modo_pedido and turno.get("porciones") is not None and turno["porciones"] <= 0:
110     # el nombre del plato SI se guarda (si vino uno nuevo) - solo el
111     # numero invalido queda sin guardar, para no perder de vista de
112     # que plato se esta hablando mientras se corrige la cantidad.
113     if turno.get("preparacion"):
114         est["preparacion"] = turno["preparacion"]
115     plato_dicho = (turno.get("preparacion") or est.get("preparacion") or "ese plato").lower()
116     turno["porciones"] = None

```

```

117     turno["respuesta_hablada"] = ("Las porciones tienen que ser un número mayor a "
118                                   f"cero. ¿Para cuántas porciones prepara {plato_dicho}?")
119     return turno
120
121 if modo_pedido:
122     # guarda lo nuevo que se haya dicho esta vez, y si esta vez no vino
123     # nada, rellena con lo que ya se sabia - asi el frontend no pierde
124     # de vista el plato solo porque este turno hablo unicamente de las
125     # porciones (o al reves).
126     if turno.get("preparacion"):
127         if est.get("preparacion") and turno["preparacion"] != est["preparacion"]:
128             # cambio de plato: las porciones del plato anterior no sirven
129             # para este - hay que volver a preguntar, no asumir el mismo
130             # numero (ej. si antes eran 4 porciones de ajiaco, un sancocho
131             # nuevo no tiene por que ser tambien 4).
132             est["porciones"] = None
133             est["preparacion"] = turno["preparacion"]
134         elif est.get("preparacion"):
135             turno["preparacion"] = est["preparacion"]
136         if turno.get("porciones"):
137             est["porciones"] = turno["porciones"]
138         elif est.get("porciones"):
139             turno["porciones"] = est["porciones"]
140 if modo_pedido and intencion == "contar":
141     # en Pedidos nunca se registra un conteo fisico: mencionar un
142     # producto aqui solo consulta su stock, no lo cuenta.
143     intencion = "consultar"
144     turno["intencion"] = "consultar"
145
146 palabras = texto.lower().replace(",", " ").replace(".", " ").split()
147 dice_si = (any(p in ("si", "sí", "claro", "listo", "dale", "correcto")
148                for p in palabras)
149            or any(k in texto.lower() for k in
150                  ("confirmo", "confirmar", "asi es", "así es", "eso es")))
151 dice_no = ("no" in palabras
152            or any(k in texto.lower() for k in
153                  ("no gracias", "mejor no", "tampoco", "no por ahora")))
154
155 # — 0) declina lo que se le esta preguntando (oferta de archivo o
156 # "¿cual de los dos?"): sin esto, un "no" caia en cualquier intencion que
157 # Gemini le hubiera adivinado a esa palabra sola (a veces "corregir"), y
158 # el agente terminaba preguntando algo sin relacion como "¿cual es el
159 # valor correcto?" - la conversacion quedaba trabada en un tema que la
160 # persona ya habia cerrado.
161 if dice_no and not est.get("pendiente"):
162     if est.get("bodega_pendiente"):
163         est["bodega_pendiente"] = None
164         turno["intencion"] = "explicar"
165         turno["respuesta_hablada"] = "Listo, no la abro. ¿Cuál bodega es?"
166         return turno
167     if est.get("oferta_reporte") and not est.get("opciones"):
168         est["oferta_reporte"] = False
169         turno["intencion"] = "explicar"
170         turno["respuesta_hablada"] = "Listo, no genero nada. ¿Algo más en lo que le ayude?"
171         return turno
172     if est.get("opciones"):
173         est["opciones"] = None
174         est.pop("opciones_para", None)
175         turno["intencion"] = "explicar"
176         turno["respuesta_hablada"] = "Listo, cancelado. ¿En qué más le ayudo?"
177         return turno
178
179 # — 1) confirma lo pendiente: aqui si se guarda —
180 if est.get("pendiente") and (dice_si or intencion == "confirmar"):
181     return _guardar(est, sesion_id, usuario)
182
183 # — 1z) confirma la bodega que se ofrecio abrir: aqui si se abre.
184 # Sin este paso, el nombre de una bodega dicho por voz se enviaba
185 # directo a abrirla - si el reconocimiento de voz entendio mal un
186 # nombre parecido (visto en produccion: "autoservicios las fuentes"
187 # -> "AUTOSERVICIOS CASCADA"), la persona quedaba metida en la bodega
188 # equivocada sin haber podido revisar antes.
189 if est.get("bodega_pendiente") and (dice_si or intencion == "confirmar"):
190     return _confirmar_abrir_bodega(est, usuario)

```

```

191
192 # — 1b) confirma la oferta de generar el archivo tras una consulta —
193 if (est.get("oferta_reporte") and not est.get("pendiente")
194     and not est.get("opciones")
195     and (dice_si or intencion in ("confirmar", "reporte"))):
196     est["oferta_reporte"] = False
197     return _generar_reporte_voz(usuario)
198
199 # — 1c) pide generar el archivo de una vez, sin que se le haya
200 # ofrecido antes: "generame el archivo" dicho de la nada. Sin esto el
201 # agente respondia con un mensaje que sonaba a que si lo iba a hacer
202 # ("de una, te lo genero") pero nunca llamaba a generar nada de
203 # verdad - la persona se quedaba con el mensaje y ningun archivo.
204 if intencion == "reporte" and not est.get("pendiente") and not est.get("opciones"):
205     if getattr(usuario, "perfil", None) != "auditor":
206         turno["respuesta_hablada"] = ("Generar el consolidado lo hace el "
207                                     "administrador de bodega; avísele.")
208     return turno
209     return _generar_reporte_voz(usuario)
210
211 # confirma sin nada pendiente: hay que decirlo, no fingir que guardo
212 if (dice_si or intencion == "confirmar") and not est.get("pendiente"):
213     if est.get("opciones"):
214         turno["respuesta_hablada"] = _leer_opciones(est["opciones"])
215         turno["opciones"] = est["opciones"]
216         turno["opciones_para"] = est.get("opciones_para")
217     elif modo_pedido:
218         turno["respuesta_hablada"] = ("No hay nada pendiente de confirmar. Dígame "
219                                     "el plato y las porciones, o pregúnteme por "
220                                     "un producto.")
221     elif not bodega_id:
222         turno["respuesta_hablada"] = ("Primero abra una bodega: diga "
223                                     "«iniciar conteo en» y el nombre.")
224     else:
225         turno["respuesta_hablada"] = ("No hay nada pendiente de confirmar. "
226                                     "Dicteme el producto y la cantidad.")
227     return turno
228
229 # — 2) habia opciones y eligio una —
230 if est.get("opciones"):
231     elegido = _elegir(texto, est["opciones"])
232     if elegido:
233         para = est.pop("opciones_para", "contar")
234         # lo que se guarda como alias tiene que ser la frase original
235         # que fue ambigua ("tres tablas para picar blancas"), no el
236         # codigo o la palabra con que se eligio la opcion ("97500779",
237         # "la primera") - sin esto, aprender_alias() aprendia un alias
238         # inutil (el codigo mapeado a si mismo) y la proxima vez que
239         # alguien dijera la misma frase ambigua, CuentaVoz volvia a
240         # preguntar "¿cual de los dos?" en vez de recordar la eleccion.
241         texto_original = est.pop("opciones_texto_original", texto)
242         est["opciones"] = None
243         if para == "consultar":
244             turno["respuesta_hablada"] = resumen_stock(elegido)
245             turno["producto_consultado"] = {"codigo": elegido["codigo"],
246                                             "nombre": elegido["nombre"],
247                                             "unidad": elegido["unidad"]}
248             est["oferta_reporte"] = True
249             return turno
250         return _dejar_pendiente(est, elegido, est.get("cantidad", 1), texto_original, turno)
251
252 # — 3) abrir bodega —
253 if intencion == "navegar":
254     est["oferta_reporte"] = False
255     if modo_pedido:
256         turno["respuesta_hablada"] = ("Aquí en Pedidos no se abren bodegas, eso es "
257                                     "en el módulo de Conteo. Dígame el plato y las "
258                                     "porciones, o pregúnteme por un producto.")
259     return turno
260     return _abrir(est, turno.get("bodega_texto") or texto, turno, usuario)
261
262 # — 4) contar —
263 if intencion == "contar":
264     est["oferta_reporte"] = False

```

```

265     if not modo_pedido and not bodega_id:
266         # sin bodega abierta no hay donde guardar un conteo: decirlo,
267         # no ofrecer crear un producto nuevo con lo que se haya oído.
268         turno["respuesta_hablada"] = ("Primero abra una bodega: diga "
269                                     "«iniciar conteo en» y el nombre.")
270     return turno
271 texto_articulo = turno.get("articulo_texto") or texto
272 cand = buscar_articulo(texto_articulo, bodega_id)
273 if not cand:
274     turno["respuesta_hablada"] = ("No encuentre ese articulo en el catalogo. "
275                                 "¿Lo creamos? Quedara pendiente de aprobacion.")
276     turno["intencion"] = "crear"
277     return turno
278 # "or 1" convertia "hay cero cazuelas" (un faltante real, cantidad 0
279 # perfectamente valida) en "1" silenciosamente, porque 0 es falsy en
280 # Python - exactamente el mismo patron ya corregido para "porciones"
281 # en el modulo de Pedidos (ver linea ~109), pero que aqui, para el
282 # conteo fisico de verdad, quedaba sin corregir. Con esto, alguien
283 # reportando un faltante total quedaba con "1" guardado en el
284 # sistema, ocultando el faltante real.
285 cantidad_dicha = turno.get("cantidad")
286 est["cantidad"] = cantidad_dicha if cantidad_dicha is not None else 1
287 est["unidad_dicha"] = turno.get("unidad")
288 resultado, dato = _resolver_candidato(cand)
289 if resultado == "directo":
290     return _dejar_pendiente(est, dato, est["cantidad"], texto, turno)
291 if resultado == "ambiguo":
292     est["opciones"] = dato
293     est["opciones_para"] = "contar"
294     # se guarda para que, cuando la persona elija una opcion, el
295     # alias se aprenda contra esta frase (la que de verdad fue
296     # ambigua) y no contra el codigo o la palabra de la eleccion.
297     est["opciones_texto_original"] = texto_articulo
298     turno["opciones"] = dato
299     turno["opciones_para"] = "contar"
300     turno["respuesta_hablada"] = _leer_opciones(dato)
301     return turno
302     turno["respuesta_hablada"] = ("No estoy seguro de cual es. "
303                                 "¿Me lo repite o lo deletrea?")
304     return turno
305
306 # — 5) corregir el ultimo —
307 if intencion == "corregir":
308     est["oferta_reporte"] = False
309     return _corregir(est, sesion_id, turno, usuario)
310
311 # — 5b) ver la receta / preparacion de un plato, por voz —
312 if intencion == "ver_receta" and modo_pedido:
313     plato_pedido = turno.get("preparacion") or est.get("preparacion")
314     if not plato_pedido:
315         turno["respuesta_hablada"] = "¿La receta de qué plato quiere ver?"
316         return turno
317     from servicios.recetas import detalle_receta
318     r = detalle_receta(plato_pedido)
319     if not r:
320         turno["respuesta_hablada"] = (f"No encontré una receta para «{plato_pedido}». "
321                                     "Pruebe con ajiao o sancocho.")
322         return turno
323     turno["receta"] = r
324     n_ing = len(r.get("lineas", []))
325     turno["respuesta_hablada"] = (f"Aquí tiene la receta de {r['nombre'].lower():} "
326                                 f"{n_ing} ingredientes" +
327                                 (" y la preparación paso a paso." if r.get("preparacion")
328                                 else ".") +
329                                 " ¿Calculamos el pedido con esta receta?")
330     return turno
331
332 # — 6) consultar —
333 if intencion == "consultar":
334     texto_busqueda = turno.get("articulo_texto")
335     if not texto_busqueda:
336         # sin un producto nombrado no hay que buscar - "consultemos el
337         # stock" a secas no es el nombre de ningun articulo, y antes esto
338         # caia en buscar_articulo(texto) completo, que a veces por pura

```

```

339         # coincidencia de letras encontraba un producto sin relacion.
340         turno["respuesta_hablada"] = "¿El stock de qué producto quiere consultar?"
341         return turno
342     cand = buscar_articulo(texto_busqueda)
343     if not cand:
344         turno["respuesta_hablada"] = ("No encuentre ese articulo en el catalogo. "
345                                     "¿Me dice otro producto?")
346         return turno
347     resultado, dato = _resolver_candidato(cand)
348     if resultado == "ambiguo":
349         est["opciones"] = dato
350         est["opciones_para"] = "consultar"
351         turno["opciones"] = dato
352         turno["opciones_para"] = "consultar"
353         turno["respuesta_hablada"] = _leer_opciones(dato)
354         return turno
355     if resultado == "no_seguro":
356         turno["respuesta_hablada"] = "No estoy seguro de cual articulo es. ¿Me lo repite?"
357         return turno
358     turno["respuesta_hablada"] = resumen_stock(dato)
359     turno["producto_consultado"] = {"codigo": dato["codigo"], "nombre": dato["nombre"],
360                                   "unidad": dato["unidad"]}
361     est["oferta_reporte"] = True
362     return turno
363
364     # — 7) avance —
365     if "falta" in texto.lower() or "avance" in texto.lower():
366         turno["respuesta_hablada"] = avance(sesion_id)
367         return turno
368
369     if not turno["respuesta_hablada"]:
370         turno["respuesta_hablada"] = "Perdon, no le entendi bien. ¿Me lo repite?"
371     return turno
372
373
374 # ----- auxiliares -----
375 def _resolver_candidato(cand):
376     """Decide si hay un ganador claro, si hay que preguntar, o si no hay
377     certeza. Un puntaje alto NO basta para decidir solo: si el segundo
378     candidato esta empatado o casi (como ARROZ contra ARROZ BASMATI, los
379     dos al 100 cuando solo se dice «arroz»), ahí es donde el agente se
380     confunde de producto sin que la persona se de cuenta. Por eso siempre
381     se compara el primero contra el segundo, no el primero solo."""
382     if not cand:
383         return "vacio", None
384     if len(cand) == 1:
385         # un solo candidato no es garantia de que sea el correcto: puede ser
386         # el unico que paso el filtro de buscar_articulo con una coincidencia
387         # debil (ej. "consultemos" comparte raiz con "consumibles"). Sin este
388         # piso, ese empate a una sola opcion se daba por bueno igual.
389         if cand[0]["confianza"] < MEDIA:
390             return "no_seguro", None
391         return "directo", cand[0]
392     if cand[0]["confianza"] < MEDIA:
393         return "no_seguro", None
394     if cand[0]["confianza"] >= ALTA and (cand[0]["confianza"] - cand[1]["confianza"]) >= 15:
395         return "directo", cand[0]
396     return "ambiguo", cand[:2]
397
398
399 def _leer_opciones(cand):
400     return ("Tengo dos parecidos: " + cand[0]["nombre"] + " o " +
401           cand[1]["nombre"] + ". ¿Cual de los dos?")
402
403
404 def _dejar_pendiente(est, art, cantidad, texto, turno):
405     v = validar_conteo(art["codigo"], cantidad, est.get("unidad_dicha"),
406                      est.get("bodega_id"))
407     est["pendiente"] = {"articulo_codigo": art["codigo"], "nombre": art["nombre"],
408                       "cantidad": cantidad, "unidad": art["unidad"],
409                       "texto_original": texto,
410                       "alerta": None if v["ok"] else v["tipo"],
411                       "detalle_alerta": None if v["ok"] else v["mensaje"]}
412     if v["ok"]:

```



```

413         turno["respuesta_hablada"] = (f"{art['nombre']}: {cantidad:g} "
414                                         f"{art['unidad']}. ¿Confirma?")
415     else:
416         turno["respuesta_hablada"] = v["mensaje"]
417         turno["alerta"] = v["tipo"]
418         if v["tipo"] == "desviacion":
419             # datos limpios para la tarjeta "Contexto de la validacion":
420             # antes solo iban mezclados dentro de la frase hablada.
421             esperado = v.get("esperado") or 0
422             desviacion_pct = round((cantidad - esperado) / esperado * 100) if esperado else None
423             turno["contexto_alerta"] = {
424                 "esperado": esperado, "dicho": cantidad, "unidad": art["unidad"],
425                 "articulo": art["nombre"], "desviacion_pct": desviacion_pct,
426             }
427         if v["tipo"] in ("negativo", "unidad", "inexistente"):
428             est["pendiente"] = None # no se puede guardar: hay que repetir
429             # el intento no se guarda como conteo, pero si queda como alerta
430             # de gestion - antes se perdía sin dejar rastro para el panel
431             bodega = est.get("bodega_nombre") or "sin bodega"
432             with Sesion() as s:
433                 s.add(Alerta(conteo_id=None, tipo=v["tipo"],
434                             detalle=f"{bodega} · {art['nombre']}: {v['mensaje']}"))
435                 s.commit()
436             turno["pendiente"] = est.get("pendiente") and {
437                 "nombre": art["nombre"], "cantidad": cantidad, "unidad": art["unidad"]}
438         return turno
439
440
441 def _guardar(est, sesion_id, usuario):
442     from seguridad import registrar
443     c = est.pop("pendiente")
444     with Sesion() as s:
445         reg = Conteo(sesion_id=sesion_id, articulo_codigo=c["articulo_codigo"],
446                     cantidad=c["cantidad"], unidad=c["unidad"],
447                     estado="confirmado")
448         s.add(reg)
449         s.commit()
450         s.refresh(reg)
451         if c.get("alerta"):
452             s.add(Alerta(conteo_id=reg.id, tipo=c["alerta"],
453                         detalle=c["detalle_alerta"] or ""))
454             s.commit()
455         est["ultimo_id"] = reg.id
456     aprender_alias(c.get("texto_original", ""), c["articulo_codigo"],
457                  est.get("bodega_id"))
458     if usuario:
459         registrar(usuario, "CONTEO",
460                  f"{c['nombre']}: {c['cantidad']:g} {c['unidad']}")
461     return {"intencion": "guardado",
462            "respuesta_hablada": "Guardado. " + avance(sesion_id),
463            "guardado": True}
464
465
466 def _corregir(est, sesion_id, turno, usuario):
467     """Corrige lo que esta pendiente; si no hay nada pendiente, el ultimo guardado."""
468     from seguridad import registrar
469     nueva = turno.get("cantidad")
470     if nueva is None:
471         turno["respuesta_hablada"] = "¿Cual es el valor correcto?"
472         return turno
473
474     # 1) lo mas comun: la persona corrige antes de confirmar («uy, no: son nueve»)
475     p = est.get("pendiente")
476     if p:
477         art = {"codigo": p["articulo_codigo"], "nombre": p["nombre"],
478               "unidad": p["unidad"]}
479         est["cantidad"] = nueva
480         return _dejar_pendiente(est, art, nueva, p.get("texto_original", ""), turno)
481
482     # 2) si ya se habia guardado, se crea un registro que apunta al original
483     ultimo_id = est.get("ultimo_id")
484     if not ultimo_id:
485         turno["respuesta_hablada"] = "Todavía no hay un registro que corregir."
486         return turno

```



```

487 with Sesion() as s:
488     orig = s.get(Conteo, ultimo_id)
489     if orig is None:
490         turno["respuesta_hablada"] = "No encuentro ese registro."
491         return turno
492     orig.estado = "corregido" # nada se borra
493     nuevo = Conteo(sesion_id=sesion_id, articulo_codigo=orig.articulo_codigo,
494                   cantidad=nueva, unidad=orig.unidad,
495                   estado="confirmado", corrige_a=orig.id)
496     s.add(nuevo)
497     s.commit()
498     s.refresh(nuevo)
499     est["ultimo_id"] = nuevo.id
500     art = s.get(Articulo, orig.articulo_codigo)
501     nombre = art.nombre_oficial if art else orig.articulo_codigo
502 if usuario:
503     registrar(usuario, "CORRECCION",
504              f"{nombre}: {orig.cantidad:g} -> {nueva:g} "
505              f"{(valor anterior conservado)}, "alerta")
506     turno["respuesta_hablada"] = f"Corregido a {nueva:g}. Quedo registrado."
507     turno["corregido"] = True
508     return turno
509
510 def _completar_apertura(s, b, est, usuario):
511     """La apertura de verdad: crea o reutiliza la sesion, marca la bodega
512     en conteo y arma la respuesta. La llaman tanto _abrir() (cuando la
513     bodega ya la tiene la misma persona, no hay nada que confirmar) como
514     _confirmar_abrir_bodega() (despues del "si" a la pregunta de
515     confirmacion)."""
516     from seguridad import registrar
517     abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
518                                              estado="abierta").first()
519     ses = abierta or SesionConteo(bodega_id=b.id, usuario_id=getattr(usuario, "id", 1))
520     if not abierta:
521         s.add(ses)
522         b.estado = "en_conteo"
523         s.commit()
524         s.refresh(ses)
525     refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
526     nombre = b.nombre_oficial
527     # Se escribe en DOS lugares a proposito:
528     # 1) "est" (el sesion_id con el que se pidio abrir) - para que la MISMA
529     #    conversacion pueda seguir contando sin cambiar de id (ej. abrir y
530     #    dictar todo por voz de un tiron, o cualquier llamador que no
531     #    adopte el id nuevo).
532     # 2) ESTADOS[ses.id] - el id real de ESTA sesion de conteo en la base -
533     #    para que quien SI adopte el sesion_id real que se devuelve abajo
534     #    (Conteo.jsx, tras abrir por boton o por voz) seguidamente encuentre
535     #    su propio contexto ahi, no el de otra persona. Antes solo se
536     #    escribia en "est": si el llamador seguia usando un sesion_id
537     #    compartido con otras personas (como hacia el frontend, ver
538     #    App.jsx), quien escribiera despues "ganaba" ese mismo est y el
539     #    conteo de la primera persona terminaba guardandose contra la
540     #    bodega de la segunda.
541     est["bodega_id"] = b.id
542     est["bodega_nombre"] = nombre
543     est["sesion_bd"] = ses.id
544     ESTADOS.setdefault(ses.id, {}).update(
545         {"bodega_id": b.id, "bodega_nombre": nombre, "sesion_bd": ses.id})
546     if usuario:
547         registrar(usuario, "APERTURA", f"{nombre} abierta - sesion bloqueada")
548     return {"respuesta_hablada": (f"Listo, {nombre.lower()} abierta con "
549                                f"{refs} referencias. Dícteme el primer producto."),
550            "bodega": {"id": b.id, "nombre": nombre, "referencias": refs},
551            "sesion_id": ses.id}
552
553
554 def _confirmar_abrir_bodega(est, usuario):
555     """Ya se pregunto "¿confirma que abro X?" y la persona dijo que si:
556     aqui es cuando de verdad se crea/reusa la sesion de conteo. Antes de
557     esto, decir el nombre de una bodega no habia tocado la base para nada
558     - si el reconocimiento de voz entendio mal, un "no" simplemente
559     descarta la oferta sin haber abierto nada."""

```

```

561 pendiente = est.get("bodega_pendiente")
562 est["bodega_pendiente"] = None
563 turno = {}
564 with Sesion() as s:
565     b = s.get(Bodega, pendiente["id"])
566     if b is None:
567         turno["respuesta_hablada"] = "Esa bodega ya no existe. ¿Cuál abro?"
568         return turno
569     # revalida que siga libre: pudo haberla tomado otra persona en el
570     # rato entre la pregunta y el "si".
571     abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
572         estado="abierta").first()
573     if abierta and abierta.usuario_id != getattr(usuario, "id", None):
574         quien = s.get(Usuario, abierta.usuario_id)
575         nombre_quien = quien.nombre.capitalize() if quien else "otra persona"
576         turno["respuesta_hablada"] = (f"{b.nombre_oficial} ya la tiene {nombre_quien} "
577             "en conteo; no la abro.")
578         return turno
579     turno.update(_completar_apertura(s, b, est, usuario))
580 return turno
581
582
583 def _abrir(est, texto_bodega, turno, usuario):
584     with Sesion() as s:
585         # buscar_bodega() (servicios/conciliacion.py) es la misma funcion
586         # que usa /api/bodegas/abrir: una sola vez quita tildes y
587         # puntuacion de sobra, prioriza la coincidencia exacta, y solo si
588         # no hay exacta cae al respaldo por palabras. Restrictiva a lo
589         # asignado si es auxiliar - no debe ni encontrar una bodega de
590         # otra zona, que igual el chequeo de abajo le negaria despues.
591         ids_permitidos = None
592         if getattr(usuario, "perfil", None) == "auxiliar":
593             ids_permitidos = {a.bodega_id for a in
594                 s.query(AsignacionBodega).filter_by(usuario_id=usuario.id).all()}
595         b = buscar_bodega(s, texto_bodega, ids_permitidos)
596         if b is None:
597             # igual que un articulo que no esta en el catalogo: en vez de
598             # dejar a la persona repitiendo el nombre sin salida, se ofrece
599             # pedir la bodega nueva (queda pendiente de aprobacion, ver
600             # /api/bodegas/crear-pendiente).
601             turno["intencion"] = "crear_bodega"
602             turno["bodega_texto"] = texto_bodega
603             turno["respuesta_hablada"] = (f"No encuentro «{texto_bodega}». ¿La creamos? "
604                 "Quedará pendiente de aprobación del administrador.")
605             return turno
606         # la misma regla que /api/bodegas/abrir: decirlo por voz no debe
607         # abrir puertas que el mismo pedido por REST tiene cerradas. Sin
608         # esto, un auxiliar podia saltarse la asignacion de zona con solo
609         # dictar el nombre de una bodega ajena.
610         if (getattr(usuario, "perfil", None) == "auxiliar"
611             and not s.query(AsignacionBodega).filter_by(
612                 usuario_id=usuario.id, bodega_id=b.id).first()):
613             turno["respuesta_hablada"] = f"{b.nombre_oficial} no esta asignada a usted."
614             return turno
615         abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
616             estado="abierta").first()
617         if abierta and abierta.usuario_id != getattr(usuario, "id", None):
618             # antes decia "por otra persona" a secas - la persona no sabia
619             # si era normal (alguien de verdad ya la esta contando) o un
620             # error, ni con quien hablar para resolverlo.
621             quien = s.get(Usuario, abierta.usuario_id)
622             nombre_quien = quien.nombre.capitalize() if quien else "otra persona"
623             turno["respuesta_hablada"] = (f"{b.nombre_oficial} ya la tiene {nombre_quien} "
624                 "en conteo; no la abro.")
625             return turno
626         if abierta and abierta.usuario_id == getattr(usuario, "id", None):
627             # ya es suya (la dejo a medias): seguir donde iba no necesita
628             # confirmacion, no hay ambigüedad que revisar.
629             turno.update(_completar_apertura(s, b, est, usuario))
630             return turno
631         # bodega libre y accesible: antes de tocar la base, se pregunta -
632         # un nombre parecido mal transcrito ("autoservicios las fuentes"
633         # sonando a "autoservicios cascada") no debe meter a la persona en
634         # la bodega equivocada sin que lo haya podido revisar.

```

```

635     est["bodega_pendiente"] = {"id": b.id, "nombre": b.nombre_oficial}
636     turno["respuesta_hablada"] = f"¿Confirma que abro {b.nombre_oficial.lower()}?"
637     return turno
638
639
640 def avance(sesion_id) -> str:
641     with Sesion() as s:
642         hechas = s.query(Conteo).filter_by(sesion_id=sesion_id,
643                                           estado="confirmado").count()
644         est = ESTADOS.get(sesion_id, {})
645         total = (s.query(StockSistema)
646                 .filter_by(bodega_id=est.get("bodega_id"))).count()
647         if est.get("bodega_id") else 0)
648     if total:
649         return f"Llevamos {hechas} de {total} referencias."
650     return f"Llevamos {hechas} referencias."
651
652
653 def _generar_reporte_voz(usuario) -> dict:
654     """Cuando la persona acepta la oferta «¿le genero el archivo?» tras
655     una consulta: aqui de verdad se genera, no solo se dice que si."""
656     import os
657     import reportes
658     ruta, filas, _vista_previa = reportes.consolidado("xlsx")
659     if usuario:
660         from seguridad import registrar
661         registrar(usuario, "REPORTE", f"Consolidado generado por voz: {ruta} ({filas} filas)")
662     return {"intencion": "reporte", "guardado": True, "archivo": ruta,
663           "respuesta_hablada": f"Listo, ya genere el archivo {os.path.basename(ruta)}. "
664                               "Lo encuentra en el menu Reportes para descargarlo. "
665                               "¿Algo más en lo que le ayude?"}
666
667
668 def resumen_stock(a) -> str:
669     """El articulo ya viene resuelto (sin ambigüedad): solo arma la respuesta."""
670     with Sesion() as s:
671         filas = s.query(StockSistema).filter_by(articulo_codigo=a["codigo"]).all()
672         total = sum(f.cantidad_sd or 0 for f in filas)
673         nombres = []
674         for f in filas[:4]:
675             b = s.get(Bodega, f.bodega_id)
676             if b:
677                 nombres.append(f"{b.nombre_oficial.lower()} {f.cantidad_sd:g}")
678     if not filas:
679         return f"{a['nombre']}: no tiene registro de stock en ninguna bodega todavia."
680     detalle = "; ".join(nombres)
681     return (f"{a['nombre']}: {total:g} {a['unidad']} en {len(filas)} bodegas. "
682           f"{detalle}. ¿Le genero el archivo?")
683
684
685 def _elegir(texto, opciones):
686     """Empareja «la FB», «la primera», el codigo o una palabra distintiva."""
687     t = " " + texto.upper().strip() + " "
688     if not opciones:
689         return None
690
691     # 1) por codigo dictado
692     for o in opciones:
693         if o["codigo"] in t:
694             return o
695
696     # 2) por posicion
697     if any(k in t for k in (" PRIMERA ", " PRIMERO ", " UNO ", " LA UNA ")):
698         return opciones[0]
699     if any(k in t for k in (" SEGUNDA ", " SEGUNDO ", " DOS ")):
700         return opciones[1]
701
702     import re
703     dichas = set(re.findall(r"[A-Z0-9/]{2,}", t)) - {"LA", "EL", "DE", "ES"}
704
705     # 3) dijo el nombre completo de una opcion, tal cual - incluso si ese
706     # nombre esta contenido dentro del de la otra (ARROZ dentro de ARROZ
707     # BASMATI). Aqui NO sirve buscar una palabra distintiva porque ARROZ
708     # no tiene ninguna que no este tambien en ARROZ BASMATI; lo que importa

```

```

709 # es que lo dicho coincide exactamente con el conjunto de palabras de
710 # esa opcion y no con el de la otra.
711 for o in opciones:
712     propias = set(re.findall(r"[A-Z0-9/]{2,}", o["nombre"].upper()))
713     if dichas == propias:
714         return o
715
716 # 4) por palabra o sigla que aparezca en UNA sola de las opciones
717 for i, o in enumerate(opciones):
718     otras = " ".join(x["nombre"].upper() for j, x in enumerate(opciones) if j != i)
719     propias = set(re.findall(r"[A-Z0-9/]{2,}", o["nombre"].upper()))
720     ajenas = set(re.findall(r"[A-Z0-9/]{2,}", otras))
721     distintivas = propias - ajenas # lo que solo esta en esta opcion
722     if dichas & distintivas:
723         return o
724 return None

```

backend/agente/cerebro.py

315 LÍNEAS

La conversacion con Gemini: el prompt, las reglas del sistema y la lectura de la respuesta. Cuando no hay llave, aqui es donde entra el interprete local en su lugar.

```

1  """El «pensar» del agente: Gemini via Google AI Studio."""
2  import os
3  import json
4  import io
5  import wave
6  import concurrent.futures
7
8  MODELO_VOZ = "gemini-2.5-flash-preview-tts"
9
10 # El SDK instalado (google-genai 0.3.0) no soporta timeout en HttpOptions
11 # todavia (es un TODO del propio paquete) - sin esto, una llamada lenta o
12 # con la cuota ahogada se queda esperando sin limite (confirmado: una
13 # prueba real tardo mas de 30 segundos), y la persona ve el microfono
14 # "pensando" sin saber si sigue vivo. Con este limite, si Gemini no
15 # contesta a tiempo, se cae al respaldo local de una vez en vez de dejar
16 # la voz esperando.
17 TIMEOUT_GEMINI = 7
18 _EJECUTOR = concurrent.futures.ThreadPoolExecutor(max_workers=4, thread_name_prefix="gemini")
19
20
21 def _generar_con_limite(cliente, limite=TIMEOUT_GEMINI, **kwargs):
22     futuro = _EJECUTOR.submit(cliente.models.generate_content, **kwargs)
23     return futuro.result(timeout=limite)
24
25 # Voces neuronales de Gemini disponibles para elegir en Mi perfil. No son
26 # acentos de pais (Gemini TTS no expone eso todavia): son voces distintas,
27 # pero todas genuinamente humanas y fluidas - muy por encima de las voces
28 # "Desktop" classicas del navegador, que es lo que se queria reemplazar.
29 VOCES = {
30     "kore": {"nombre": "Kore", "etiqueta": "Femenina, firme y clara"},
31     "aoede": {"nombre": "Aoede", "etiqueta": "Femenina, suave y cercana"},
32     "puck": {"nombre": "Puck", "etiqueta": "Masculina, animada"},
33     "charon": {"nombre": "Charon", "etiqueta": "Masculina, grave y pausada"},
34 }
35 VOZ_DEFECTO = "kore"
36
37 PROMPT_SISTEMA = """Eres CuentaVoz, el asistente de inventarios de Colsubsidio,
38 usado por auxiliares de bodega en Piscilago. Hablas como una persona
39 colombiana cercana y profesional: frases cortas, calidas, naturales.
40 Usa expresiones como "listo", "claro que si", "de una", "perfecto" en vez
41 de sonar como un robot que repite siempre la misma formula. Varía tus
42 respuestas: no contestes igual dos veces seguidas.
43
44 Reglas:
45 1. Nunca inventes articulos: usa solo los que devuelva buscar_articulo.
46 2. Antes de guardar, repite siempre articulo, cantidad y unidad.
47 3. Con varios candidatos, lee las opciones y deja que la persona elija.
48 4. Si hay una alerta, pregunta antes de continuar. Nunca guardes un dato
49 con alerta sin confirmacion explicita.

```

```

50 5. Si la persona no entiende algo, explícalo con palabras simples.
51 6. Si piden datos, resume en voz y ofrece generar el archivo.
52 7. Si no puedes resolver, dilo con honestidad y ofrece llamar al
53 administrador de bodega.
54 8. Se flexible con la forma de hablar: la gente no dicta como robot.
55 Frases como "hay noventa cazuelas", "me faltan cinco kilos de arroz",
56 "anota tres tablas blancas" son todas la intencion "contar". Solo
57 respondas que no entendiste como ultimo recurso, si de verdad no hay
58 forma razonable de interpretar la frase.
59 9. OJO con las negaciones: "no quiero preparar arroz", "no es arroz",
60 "cancela el ajiaco", "ya no necesito eso" NUNCA deben dejar preparacion,
61 articulo_texto ni cantidad rellenos como si la persona hubiera pedido
62 o confirmado ese producto - es justo lo contrario. Si niega o cancela
63 algo, la intencion es "explicar" o "corregir", y preparacion/
64 articulo_texto/cantidad quedan en null. Nunca extraigas un dato de
65 una frase que lo esta negando.
66
67 La unidad SIEMPRE debe quedar en uno de estos 4 valores exactos, nunca la
68 palabra que dijo la persona: "Kilogram" (kilo, kilos, kg, gramos->convierte),
69 "Liter" (litro, litros, l), "Unidad" (unidad, unidades, paquete, caja),
70 "Portion" (porcion, porciones, racion). Ejemplo: si dice "veinte kilos",
71 unidad = "Kilogram", NUNCA "kilos".
72
73 Intenciones posibles: contar, corregir, consultar, explicar, crear,
74 navegar, ayuda, cerrar, pedir, legalizar, reporte, ver_receta.
75
76 Cuando la intencion es "pedir", extrae ademas preparacion y porciones
77 (ejemplo: "hoy preparamos cincuenta ajiacos" -> preparacion "AJIACO",
78 porciones 50).
79
80 10. Cuando la persona pide de una vez generar, descargar o mandar el
81 archivo/reporte/consolidado ("generame el archivo", "descarga el
82 reporte", "mandame el consolidado"), sin que se lo hayas ofrecido vos
83 antes en este turno, la intencion es "reporte" - no "contar" ni
84 "consultar". Responde con algo breve confirmando que lo vas a generar
85 ("de una, te lo genero" / "listo, ya te lo mando"), sin inventar que
86 ya quedo hecho: el sistema es el que arma el archivo de verdad
87 despues de tu respuesta.
88
89 11. Cuando la persona pide ver, consultar o que le muestre la receta o la
90 preparacion de un plato ("quiero ver la receta", "muestrame la receta",
91 "cual es la preparacion", "como se hace"), la intencion es "ver_receta" -
92 no "consultar" (eso es para stock de un producto) ni "pedir" (eso es
93 para calcular insumos). Si el plato no queda claro en esta misma frase,
94 extraelo igual en "preparacion" si aparece mencionado, o dejalo vacio si
95 de verdad no se menciona ninguno.
96
97 Responde SOLO con un JSON con estas llaves:
98 intencion, articulo_texto, cantidad, unidad, preparacion, porciones,
99 respuesta_hablada.""
100
101 _cliente = None
102
103
104 def _obtener_cliente():
105     global _cliente
106     if _cliente is None:
107         from google import genai
108         llave = os.getenv("GOOGLE_API_KEY", "").strip()
109         if not llave:
110             return None
111         _cliente = genai.Client(api_key=llave)
112     return _cliente
113
114
115 def pensar(contexto: str, frase: str) -> dict:
116     ""Devuelve la intencion y los datos. Si Gemini falla, cae con gracia.""
117     try:
118         from google.genai import types
119         cliente = _obtener_cliente()
120         if cliente is None:
121             return _respaldo(frase, "Falta la llave de Google AI Studio en el archivo .env.")
122
123         r = _generar_con_limite(

```

```

124         cliente,
125         model=os.getenv("MODELO", "gemini-flash-latest"),
126         contents=f"{contexto}\nLa persona dice: {frase}",
127         config=types.GenerateContentConfig(
128             system_instruction=PROMPT_SISTEMA,
129             response_mime_type="application/json",
130             temperature=0.2,
131         ),
132     )
133     datos = json.loads(r.text)
134     if not isinstance(datos, dict):
135         raise ValueError("respuesta inesperada")
136     if datos.get("unidad"):
137         from servicios.interprete import normalizar_unidad
138         datos["unidad"] = normalizar_unidad(datos["unidad"])
139     _limpiar_si_niega(frase, datos)
140     return datos
141
142 except Exception as e:
143     print(f"[cerebro] Gemini no respondió: {e}")
144     return _respaldo(frase)
145
146
147 NEGACIONES = ("no quiero", "no voy a", "no es ", "no era", "cancela",
148               "cancelar", "ya no ", "no necesito", "no me sirve", "quítalo",
149               "quítalo", "elimina eso", "olvida eso", "olvidalo", "olvidalo")
150
151
152 def _limpiar_si_niega(frase: str, datos: dict):
153     """Respaldo determinista: si el prompt no bastara, una frase que niega
154     o cancela algo NUNCA debe quedar como si la persona hubiera pedido o
155     confirmado ese dato ("no quiero preparar arroz" no es un pedido de
156     arroz). No depende de que el modelo obedezca la instruccion al pie
157     de la letra - igual que la normalizacion de unidades."""
158     f = frase.lower()
159     if not any(n in f for n in NEGACIONES):
160         return
161     for campo in ("preparacion", "porciones", "articulo_texto", "cantidad"):
162         datos[campo] = None
163     if (datos.get("intencion") or "").lower() in ("pedir", "contar"):
164         datos["intencion"] = "explicar"
165
166
167 def _respaldo(frase: str, mensaje: str = None) -> dict:
168     """Sin Gemini el conteo no se detiene: se interpreta lo basico aqui."""
169     from servicios.interprete import interpretar_local
170     datos = interpretar_local(frase)
171     if mensaje:
172         datos["respuesta_hablada"] = mensaje
173     return datos
174
175
176 PROMPT_ASISTENTE = """Eres CuentaVoz, el asistente de inventarios de
177 Colsubsidio. La persona esta en una pantalla de la aplicacion (no esta
178 contando ni armando un pedido) y le hace una pregunta o le pide ir a otra
179 pantalla. Hable como una persona colombiana cercana y profesional, en
180 frases cortas.
181
182 Reglas:
183 1. Responda SOLO con datos que aparezcan en el contexto que se le da. Si
184    no tiene el dato, dígallo con honestidad en vez de inventar un numero.
185 2. Si la persona pide ir a otra pantalla ("llevame a bodegas", "abre mi
186    perfil", "muéstreme los reportes"), la intencion es "navegar" y
187    "destino" debe ser EXACTAMENTE una de las claves de la lista de
188    pantallas disponibles del contexto - nunca invente una clave que no
189    este ahi, y si la pantalla que piden no esta en la lista (por ejemplo
190    porque su perfil no tiene acceso), dígallo en vez de forzar un destino.
191 3. Si no pide navegar, la intencion es "responder": conteste con el
192    contexto que tiene, o explique brevemente que se hace en esta pantalla.
193 4. Nunca prometa una accion que esta pantalla no puede hacer todavia por
194    voz (cambiar una configuracion, generar un archivo, aprobar algo): solo
195    responde preguntas y navega. Si se lo piden, dígallo con claridad y
196    sugiera usar los botones de la pantalla.
197 5. Algunas pantallas tienen pestañas propias (se listan en el contexto

```

```

198 como "destino > pestaña"). Si la persona pide una pestaña especifica
199 ("llevame al analisis de consumo", "abre la gestion de usuarios"), no
200 se queda a medio camino: pone el destino correcto Y "pestaña" con la
201 clave exacta de esa pestaña, para llegar directo sin dar clic despues.
202 Si solo pide la pantalla en general, sin mencionar pestaña, deje
203 "pestaña" en null - no adivine una.
204 6. Si lo que dice la persona no es una pregunta clara ni una orden
205 reconocible (una palabra suelta sin sentido para esta pantalla, algo
206 en otro idioma, texto que parece ruido o un eco del reconocimiento de
207 voz), digalo con honestidad ("no le entendí eso" o similar) en vez de
208 responder de una con la explicacion general de la pantalla como si la
209 hubiera pedido - no finja haber entendido una pregunta que no hicieron.
210
211 Responde SOLO con un JSON con estas llaves: intencion (navegar|responder),
212 destino, pestaña, respuesta_hablada.""
213
214
215 def pensar_asistente(contexto: str, frase: str, destinos: dict[str, str],
216                     pestanas: dict[str, dict[str, str]] | None = None) -> dict:
217     """Version liviana de pensar(), para las pantallas que no son de
218     conteo ni de pedido (Inicio, Ajustes, Ayuda, Reportes, Panel): mismo
219     modelo y mismo cliente, pero un prompt propio y mas simple - sin el
220     estado de "pendiente"/"opciones" que solo tiene sentido a mitad de un
221     conteo o de un pedido en curso. pestanas permite pedir de una vez una
222     pestaña especifica de un destino con pestañas propias (ver main.py)."""
223     pestanas = pestanas or {}
224     lista = "\n".join(f"- {clave}: {etiqueta}" for clave, etiqueta in destinos.items())
225     for destino, sub in pestanas.items():
226         for clave_tab, etiqueta_tab in sub.items():
227             lista += f"\n- {destino} > {clave_tab}: {etiqueta_tab}"
228     contexto_completo = f"{contexto}\nPantallas disponibles para navegar:\n{lista}"
229     try:
230         from google.genai import types
231         cliente = _obtener_cliente()
232         if cliente is None:
233             return _respaldo_asistente(frase, destinos,
234                                         "Sin conexión con el agente en este momento.")
235         r = _generar_con_limite(
236             cliente,
237             model=os.getenv("MODELO", "gemini-flash-latest"),
238             contents=f"{contexto_completo}\nLa persona dice: {frase}",
239             config=types.GenerateContentConfig(
240                 system_instruction=PROMPT_ASISTENTE,
241                 response_mime_type="application/json",
242                 temperature=0.2,
243             ),
244         )
245         datos = json.loads(r.text)
246         if not isinstance(datos, dict):
247             raise ValueError("respuesta inesperada")
248         return datos
249     except Exception as e:
250         print(f"[cerebro] Asistente sin Gemini: {e}")
251         return _respaldo_asistente(frase, destinos)
252
253
254 def _respaldo_asistente(frase: str, destinos: dict[str, str], mensaje: str = None) -> dict:
255     """Sin Gemini, al menos reconoce «lleveme a <pantalla>» por palabra
256     clave - el resto queda honesto (no inventa una respuesta) en vez de
257     fingir que entendio. No intenta adivinar la pestaña sin el modelo."""
258     f = frase.lower()
259     for clave, etiqueta in destinos.items():
260         if clave in f or etiqueta.lower() in f:
261             return {"intencion": "navegar", "destino": clave, "pestaña": None,
262                     "respuesta_hablada": f"Vamos a {etiqueta.lower()}."}
263     return {"intencion": "responder", "destino": None, "pestaña": None,
264            "respuesta_hablada": mensaje or
265            "No pude entenderlo sin conexión al agente. Puede escribir su pregunta."}
266
267
268 def _pcm_a_wav(pcm: bytes, tasa: int = 24000, canales: int = 1, bits: int = 16) -> bytes:
269     """Gemini TTS devuelve PCM crudo (sin encabezado): el navegador
270     necesita un WAV valido para poder reproducirlo con <audio>/Audio()."""
271     buf = io.BytesIO()

```



```

272     with wave.open(buf, "wb") as w:
273         w.setnchannels(canales)
274         w.setsampwidth(bits // 8)
275         w.setframerate(tasa)
276         w.writeframes(pcm)
277     return buf.getvalue()
278
279
280 def sintetizar_voz(texto: str, voz: str = VOZ_DEFECTO) -> bytes | None:
281     """Convierte texto a un WAV con voz neuronal real. None si Gemini no
282     esta disponible - quien llama decide si cae de respaldo a la voz del
283     navegador (que suena distinta a la elegida en Mi perfil, asi que cada
284     caida de vuelta ahi es una voz distinta hablando a mitad de sesion)."""
285     cliente = _obtener_cliente()
286     if cliente is None or not texto.strip():
287         return None
288     nombre_voz = VOCES.get(voz, VOCES[VOZ_DEFECTO])["nombre"]
289     from google.genai import types
290     config = types.GenerateContentConfig(
291         response_modalities=["AUDIO"],
292         speech_config=types.SpeechConfig(
293             voice_config=types.VoiceConfig(
294                 prebuilt_voice_config=types.PrebuiltVoiceConfig(voice_name=nombre_voz)
295             )
296         ),
297     )
298     # Un reintento, y solo si la primera vez se agoto el tiempo (se ha visto
299     # que Gemini a veces tarda una vez bajo carga y responde rapido la
300     # siguiente) - no ante cualquier otra falla (cuota agotada, por
301     # ejemplo), donde insistir de una solo suma espera sin cambiar el
302     # resultado y hace mas notoria la caida a la voz del navegador.
303     for intento, limite in ((1, 12), (2, 6)):
304         try:
305             r = _generar_con_limite(cliente, limite=limite, model=MODELO_VOZ,
306                                   contents=texto, config=config)
307             parte = r.candidates[0].content.parts[0].inline_data
308             return _pcm_a_wav(parte.data)
309         except concurrent.futures.TimeoutError as e:
310             print(f"[cerebro] Voz neuronal tardo demasiado (intento {intento}): {e}")
311             continue
312         except Exception as e:
313             print(f"[cerebro] Voz neuronal no disponible: {e}")
314             return None
315     return None

```

17.3 Backend · servicios

backend/servicios/interprete.py

147 LÍNEAS

El interprete local: numeros en palabras, unidades e intenciones. Es el que permite que la plataforma funcione sin Gemini.

```

1  """Interprete local: permite demostrar el flujo aun sin llave de Gemini.
2
3  No reemplaza al modelo. Reconoce numeros en palabras y las intenciones
4  mas frecuentes, para que la demo nunca se caiga por falta de internet.
5  """
6  import re
7
8  NUMEROS = {
9      "cero": 0, "un": 1, "una": 1, "uno": 1, "dos": 2, "tres": 3, "cuatro": 4,
10     "cinco": 5, "seis": 6, "siete": 7, "ocho": 8, "nueve": 9, "diez": 10,
11     "once": 11, "doce": 12, "trece": 13, "catorce": 14, "quince": 15,
12     "dieciseis": 16, "dieciséis": 16, "diecisiete": 17, "dieciocho": 18,
13     "diecinueve": 19, "veinte": 20, "veinticinco": 25, "treinta": 30,
14     "cuarenta": 40, "cincuenta": 50, "sesenta": 60, "setenta": 70,
15     "ochenta": 80, "noventa": 90, "cien": 100, "ciento": 100,
16     "doscientos": 200, "doscientas": 200, "trescientos": 300, "trescientas": 300,
17     "cuatrocientos": 400, "cuatrocientas": 400, "quinientos": 500, "quinientas": 500,
18     "seiscientos": 600, "seiscientas": 600, "setecientos": 700, "setecientas": 700,

```



```

19     "ochocientos": 800, "ochocientas": 800, "novecientos": 900, "novecientas": 900,
20     "mil": 1000,
21 }
22 UNIDADES = {
23     "kilo": "Kilogram", "kilos": "Kilogram", "kilogramo": "Kilogram",
24     "kilogramos": "Kilogram", "kg": "Kilogram",
25     "litro": "Liter", "litros": "Liter", "l": "Liter",
26     "unidad": "Unidad", "unidades": "Unidad",
27     "porcion": "Portion", "porciones": "Portion", "porción": "Portion",
28 }
29
30
31 DECENAS = (20, 30, 40, 50, 60, 70, 80, 90)
32
33
34 def _numero(texto: str):
35     """La PRIMERA cantidad de la frase. Ignora numeros que son parte
36     del nombre del producto («cazuela 16 onzas», «tabla 50x38»)."""
37     t = texto.lower()
38     m = re.search(r"-?\d+(?:[.]\d+)?", t)
39     # un dígito pegado a un guion que sigue con otro carácter (ACIDO
40     # POLIGLICOLICO "3-0", REVOLUTION "2.6-5 KG", artículos reales del
41     # catálogo) es una talla o un código del propio artículo, no la
42     # cantidad dictada - sin este chequeo, "hay veinte del ácido
43     # poliglicólico 3-0" leía el "3" del nombre del producto en vez del
44     # "veinte" que de verdad se dictó, silenciosamente y sin ningún error.
45     if m and t[m.end():m.end() + 1] != "-":
46         val = float(m.group(0).replace(".", ""))
47         return -val if "menos" in t[:m.start()] else val
48
49     palabras = [p.strip(".,?;!\"'") for p in t.split()]
50     total = None
51     for i, p in enumerate(palabras):
52         if p not in NUMEROS:
53             continue
54         v = NUMEROS[p]
55         if total is None:
56             total = v
57             # «ciento ochenta»: centena seguida de decena
58             if total >= 100 and i + 1 < len(palabras) and palabras[i + 1] in NUMEROS:
59                 sig = NUMEROS[palabras[i + 1]]
60                 if sig < 100:
61                     total += sig
62             # «treinta y cinco»: decena + y + unidad
63             elif total in DECENAS and i + 2 < len(palabras) \
64                 and palabras[i + 1] == "y" and palabras[i + 2] in NUMEROS:
65                 total += NUMEROS[palabras[i + 2]]
66         break # lo demás pertenece al nombre del producto
67     if total is not None and "menos" in t:
68         total = -abs(total)
69     return total
70
71
72 def _unidad(texto: str):
73     for p in texto.lower().replace("?", " ").split():
74         p = p.strip(".,")
75         if p in UNIDADES:
76             return UNIDADES[p]
77     return None
78
79
80 CANONICAS = {"Kilogram", "Liter", "Unidad", "Portion"}
81
82
83 def normalizar_unidad(dicha):
84     """Lo que sea que devuelva el agente (humano o IA) para la unidad,
85     lo lleva a uno de los 4 valores canónicos de la base de datos.
86     Gemini a veces repite la palabra tal cual la dijo la persona
87     ("kilos") en vez de traducirla ("Kilogram"); esto lo corrige aquí,
88     sin depender de que el modelo obedezca el prompt al pie de la letra."""
89     if not dicha:
90         return dicha
91     if dicha in CANONICAS:
92         return dicha

```

```

93     return UNIDADES.get(str(dicha).strip().lower(), dicha)
94
95
96 def interpretar_local(frase: str) -> dict:
97     f = frase.lower().strip()
98     cant = _numero(f)
99     uni = _unidad(f)
100
101     def sin_ruido(t):
102         t = re.sub(r"-?\d+(?:[.]\d+)?", " ", t)
103         fuera = set(NUMEROS) | set(UNIDADES) | {
104             "de", "del", "la", "el", "los", "las", "hay", "en", "y", "por",
105             "confirmo", "confirmar", "son", "hoy", "preparamos", "quiero",
106             "pedir", "insumos", "para", "cuanto", "cuánto", "cuantos",
107             "iniciar", "conteo", "abrir", "bodega", "corregir", "ultimo",
108             "último", "no", "uy", "que", "qué", "me", "genera", "generame",
109             "consolidado", "reporte", "ayuda", "menos", "onzas", "onza",
110         }
111         return " ".join(w for w in t.split() if w.strip(".", "?", "!") not in fuera).strip()
112
113     # — intenciones —
114     if any(k in f for k in ("iniciar conteo", "abrir bodega", "abrir la bodega")):
115         return {"intencion": "navegar", "bodega_texto": sin_ruido(f),
116                 "respuesta_hablada": "Voy a abrir esa bodega."}
117
118     if any(k in f for k in ("preparamos", "vamos a preparar", "pedir insumos")):
119         return {"intencion": "pedir", "preparacion": sin_ruido(f),
120                 "porciones": int(cant) if cant else 0,
121                 "respuesta_hablada": "Le calculo los insumos segun la receta."}
122
123     if f.startswith("confirmo") or f in ("confirmar", "si", "sí", "correcto", "confirmo."):
124         return {"intencion": "confirmar", "respuesta_hablada": "Listo."}
125
126     if any(k in f for k in ("corregir", "no son", "no, son", "uy no", "uy, no",
127                             "esta mal", "está mal", "me equivoque", "cambiar")):
128         return {"intencion": "corregir", "cantidad": cant, "unidad": uni,
129                 "respuesta_hablada": "Corrijo ese registro."}
130
131     if any(k in f for k in ("cuanto", "cuánto", "cuantos", "donde hay", "dónde hay")):
132         return {"intencion": "consultar", "articulo_texto": sin_ruido(f),
133                 "respuesta_hablada": "Reviso el inventario."}
134
135     if any(k in f for k in ("reporte", "consolidado", "archivo", "imprim")):
136         return {"intencion": "reporte", "respuesta_hablada": "Genero el archivo."}
137
138     if "ayuda" in f or "no entiendo" in f:
139         return {"intencion": "ayuda",
140                 "respuesta_hablada": "Con gusto. ¿Que necesita saber?"}
141
142     if cant is not None:
143         return {"intencion": "contar", "articulo_texto": sin_ruido(f),
144                 "cantidad": cant, "unidad": uni, "respuesta_hablada": ""}
145
146     return {"intencion": "ayuda",
147             "respuesta_hablada": "Perdon, no le entendi bien. ¿Me lo repite?"}

```

backend/servicios/recetas.py

182 LÍNEAS

Calculo del pedido desde la receta y comparacion de la legalizacion.

```

1  """Momentos 1 y 3 del reto: pedido por receta y legalizacion del servicio."""
2  import unicodedata
3  from datetime import timedelta
4  from bd import Sesión
5  from horario import ahora
6  from modelos import (Receta, RecetaIngrediente, Artículo,
7                       StockSistema, LíneaServicio)
8
9
10 def _sin_tildes(t: str) -> str:
11     """«santafereno» dicho o escrito sin tilde («santafereno») no debe
12     fallar la busqueda - un LIKE normal en SQLite si distingue Ñ de N, y

```

```

13     eso ya causo un "no encuentre la receta" con un plato que si existia.""
14     t = str(t or "").upper().strip()
15     return "".join(c for c in unicodedata.normalize("NFD", t)
16                     if unicodedata.category(c) != "Mn")
17
18
19 def _sin_plural(t: str) -> str:
20     """El chef siempre nombra el plato en plural («cincuenta ajiacos») pero
21     la receta esta guardada en singular («AJIACO SANTAFERENO»), asi que la
22     comparacion por substring nunca coincidia: la frase de ejemplo que la
23     propia pantalla de Pedidos sugiere contestaba «no encuentre una receta».
24     Se normaliza cada palabra por separado para que «ajiacos santafereños»
25     tambien caiga en la misma forma que el nombre guardado."""
26     palabras = []
27     for p in t.split():
28         if p.endswith("ES") and len(p) > 4:
29             palabras.append(p[:-2])
30         elif p.endswith("S") and len(p) > 3:
31             palabras.append(p[:-1])
32         else:
33             palabras.append(p)
34     return " ".join(palabras)
35
36
37 def _buscar_receta(s, preparacion: str):
38     objetivo = _sin_tildes(preparacion)
39     # menos de 3 letras no alcanza a identificar un plato - una transcripcion
40     # de voz recortada o con ruido ("a", "de", "o") coincidia igual como
41     # substring de cualquier receta y la calculaba con total confianza, como
42     # si el chef de verdad hubiera dictado ese plato.
43     if len(objetivo) < 3:
44         return None
45     objetivo_sin_plural = _sin_plural(objetivo)
46     for r in s.query(Receta).all():
47         nombre = _sin_tildes(r.nombre)
48         if objetivo in nombre or objetivo_sin_plural in _sin_plural(nombre):
49             return r
50     return None
51
52
53 def calcular_pedido(preparacion: str, porciones: int, bodega_id: int) -> dict:
54     """La regla del reto: cantidad por porcion x porciones - lo que ya hay.
55
56     Si un ingrediente de la receta no esta en el catalogo, o esta pero esa
57     bodega nunca lo ha tenido registrado, no se salta en silencio: queda
58     en «faltantes_catalogo» / «sin_registro_bodega» para que la persona
59     se entere y no crea que el pedido esta completo cuando no lo esta."""
60     with Sesion() as s:
61         receta = _buscar_receta(s, preparacion)
62         if receta is None:
63             return {"lineas": [], "faltantes_catalogo": [],
64                     "sin_registro_bodega": [], "receta_encontrada": False}
65
66         ings = s.query(RecetaIngrediente).filter_by(receta_id=receta.id).all()
67         lineas, faltantes, sin_registro = [], [], []
68         for ing in ings:
69             art = s.get(Articulo, ing.articulo_codigo)
70             if art is None:
71                 faltantes.append(ing.articulo_codigo)
72                 continue
73             necesario = round(ing.cantidad_por_porcion * porciones, 3)
74             stock = s.query(StockSistema).filter_by(
75                 articulo_codigo=art.codigo, bodega_id=bodega_id).first()
76             if stock is None:
77                 sin_registro.append(art.nombre_oficial)
78             hay = stock.cantidad_sd if stock else 0
79             falta = max(0, round(necesario - hay, 3))
80             lineas.append({"codigo": art.codigo, "nombre": art.nombre_oficial,
81                           "unidad": art.unidad_medida, "necesario": necesario,
82                           "stock": hay, "falta": falta,
83                           "sin_registro_bodega": stock is None})
84     return {"lineas": lineas, "faltantes_catalogo": faltantes,
85           "sin_registro_bodega": sin_registro, "receta_encontrada": True}
86

```

```

87
88 def detalle_receta(preparacion: str) -> dict:
89     """Solo consulta: cómo está armada la receta, sin tocar stock."""
90     with Sesion() as s:
91         receta = _buscar_receta(s, preparacion)
92         if receta is None:
93             return {}
94         ings = s.query(RecetaIngrediente).filter_by(receta_id=receta.id).all()
95         lineas, faltantes = [], []
96         for ing in ings:
97             art = s.get(Articulo, ing.articulo_codigo)
98             if art is None:
99                 faltantes.append(ing.articulo_codigo)
100             continue
101             lineas.append({"codigo": art.codigo, "nombre": art.nombre_oficial,
102                           "unidad": art.unidad_medida,
103                           "por_porcion": ing.cantidad_por_porcion})
104     return {"id": receta.id, "nombre": receta.nombre, "rendimiento": receta.rendimiento,
105            "preparacion": receta.preparacion or "", "lineas": lineas,
106            "faltantes_catalogo": faltantes}
107
108
109 def _filas_a_legalizar(s, servicio_id: int):
110     """Las lineas "abierto" de un servicio pueden venir de mas de un
111     pedido (dos platos distintos pedidos el mismo dia, ninguno legalizado
112     todavia) - mezclarlas todas en una sola comparacion duplicaba
113     articulos compartidos entre platos (ej. papa criolla en ajiao Y en
114     sancocho) bajo el nombre de uno solo, sin avisar que en realidad eran
115     dos pedidos distintos. Se legaliza un pedido a la vez, empezando por
116     el mas reciente; el resto espera su turno para la proxima vez que se
117     entre a Legalizacion."""
118     filas = s.query(LineaServicio).filter_by(
119         servicio_id=servicio_id, estado="abierto").all()
120     numeros = [f.numero_pedido for f in filas if f.numero_pedido]
121     if numeros:
122         mas_reciente = max(numeros)
123         filas = [f for f in filas if f.numero_pedido == mas_reciente]
124     return filas
125
126
127 def comparar_legalizacion(servicio_id: int) -> dict:
128     """Lo pedido contra lo usado, con la lectura en palabras. Solo lo que
129     sigue abierto: si ya se legalizo antes (el servicio de ejemplo, u otro
130     pedido con el mismo numero de servicio) no debe mezclarse aqui ni
131     volver a legalizarse por accidente."""
132     with Sesion() as s:
133         filas = _filas_a_legalizar(s, servicio_id)
134         lineas = []
135         for l in filas:
136             dif = round((l.usado or 0) - (l.pedido or 0), 3)
137             if dif < 0:
138                 lectura = "Sobrante: vuelve a bodega"
139             elif dif > 0:
140                 lectura = "Merma: revisar con el chef"
141             else:
142                 lectura = "Cuadro exacto"
143             art = s.get(Articulo, l.articulo_codigo)
144             lineas.append({"codigo": l.articulo_codigo, "nombre": l.nombre,
145                           "unidad": art.unidad_medida if art else "",
146                           "pedido": l.pedido, "usado": l.usado,
147                           "diferencia": dif, "lectura": lectura})
148         plato = next((f.plato for f in filas if f.plato), "")
149         porciones = next((f.porciones for f in filas if f.porciones), 0)
150         return {"lineas": lineas, "plato": plato, "porciones": porciones}
151
152
153 def analisis_consumo(dias: int = 30) -> dict:
154     """El «suma puntos» del reto: subutilizados y sobrepedido, acotado a
155     los ultimos N dias (por defecto 30, como en el tablero de Reportes)."""
156     desde = ahora() - timedelta(days=dias)
157     with Sesion() as s:
158         filas = (s.query(LineaServicio)
159                 .filter(LineaServicio.estado == "legalizado",
160                       LineaServicio.creado >= desde).all())

```

```

161 total_ped = sum(f.pedido or 0 for f in filas)
162 total_uso = sum(f.usado or 0 for f in filas)
163 servicios = len({f.servicio_id for f in filas})
164 porart = {}
165 for f in filas:
166     d = porart.setdefault(f.nombre, {"pedido": 0, "usado": 0, "veces": 0})
167     d["pedido"] += f.pedido or 0
168     d["usado"] += f.usado or 0
169     d["veces"] += 1
170 subutil = []
171 for nombre, d in porart.items():
172     sobra = round(d["pedido"] - d["usado"], 3)
173     if sobra > 0:
174         pct = round(sobra / d["pedido"] * 100) if d["pedido"] else 0
175         subutil.append({"nombre": nombre, "sobra": sobra,
176                        "veces": d["veces"], "sobrepedido_pct": pct})
177 subutil.sort(key=lambda x: -x["sobrepedido_pct"])
178 ahorro_potencial = round(sum(s["sobra"] for s in subutil), 2)
179 return {"pedido_total": round(total_ped, 2), "usado_total": round(total_uso, 2),
180        "aprovechamiento": round(total_uso / total_ped * 100, 1) if total_ped else 0,
181        "servicios_periodo": servicios, "ahorro_potencial": ahorro_potencial,
182        "dias": dias, "subutilizados": subutil}

```

backend/servicios/conciliacion.py

191 LÍNEAS

Emparejar lo dictado con el catalogo, y las alertas cuando algo no cuadra.

```

1  """De «tabla para picar blanca» al nombre y codigo oficiales del catalogo."""
2  import re
3  import unicodedata
4  from rapidfuzz import process, fuzz
5  from bd import Sesion
6  from modelos import Articulo, AliasArticulo, Bodega
7
8
9  # palabras que no distinguen un producto de otro
10 VACIAS = {"PARA", "DE", "DEL", "LA", "EL", "LOS", "LAS", "CON", "SIN",
11          "POR", "EN", "Y", "A", "UN", "UNA", "UNAS", "UNOS", "AL"}
12
13
14 def _tokens(t: str) -> list[str]:
15     return [p for p in t.split() if len(p) >= 3 and p not in VACIAS]
16
17
18 def _cobertura(consulta: str, candidato: str) -> float:
19     """Cuantas palabras significativas de la consulta aparecen en el nombre.
20
21     Compara por raiz de 4 letras, para que BLANCA encuentre BLANCO y
22     CAZUELAS encuentre CAZUELA - en los DOS sentidos exige que la raiz
23     tenga esas 4 letras de verdad (no baste una palabra de 3): sin ese
24     piso en el lado del candidato, una palabra corta del catalogo como
25     "RES" (de "COSTILLA DE RES") volvía "raiz" a ella misma y cualquier
26     palabra dicha que empezara igual - como "RESTAURANTE" - contaba como
27     coincidencia; sin el mismo piso del lado de lo dicho, decir "ver"
28     (de una frase suelta como "sí para ver el detalle", no una búsqueda
29     de verdad) encontraba "VERDE"/"VERDURAS" en articulos reales sin
30     relacion alguna. Una palabra de 3 letras solo cuenta si coincide
31     EXACTA con alguna del candidato."""
32     tc = _tokens(consulta)
33     if not tc:
34         return 0.0
35     tn = _tokens(candidato)
36     aciertos = 0
37     for p in tc:
38         raiz = p[:4]
39         if any(p == q or (len(p) >= 4 and q.startswith(raiz))
40              or (len(q) >= 4 and p.startswith(q[:4])) for q in tn):
41             aciertos += 1
42     return aciertos / len(tc) * 100
43
44
45 def puntuar(consulta: str, candidato: str) -> float:

```

```

46     """Combina la forma general con la cobertura de palabras clave.
47
48     Sin esto, «tabla para picar blanca» devuelve la AZUL: comparte
49     tabla-picar y el color se diluye."""
50     forma = fuzz.token_set_ratio(consulta, candidato)
51     cob = _cobertura(consulta, candidato)
52     return 0.45 * forma + 0.55 * cob
53
54
55 def normalizar(t: str) -> str:
56     t = str(t).upper().strip()
57     t = "".join(c for c in unicodedata.normalize("NFD", t)
58                if unicodedata.category(c) != "Mn") # sin tildes
59     t = t.replace("P/", "PARA ")
60     # "kiosco"/"kiosko" son la misma palabra, dicha o escrita de dos
61     # formas igual de comunes en español - sin esto, decir "kiosko" no
62     # encontraba "KIOSCO TAQUILLA AYB" aunque sea exactamente la misma
63     # bodega. La K es rarísima en el catálogo salvo en estos préstamos
64     # (kilo, kiosco...), así que unificarla con la C no choca con nada.
65     t = t.replace("K", "C")
66     # el reconocimiento de voz del navegador suele cerrar la frase con un
67     # punto ("Zoológico.") aunque nadie lo haya dicho - sin quitarlo, una
68     # bodega real dejaba de encontrarse por un simple "." de mas.
69     t = re.sub(r"[.,;:!?;]+", " ", t)
70     return re.sub(r"\s+", " ", t).strip()
71
72
73 def buscar_bodegas_candidatas(s, texto: str, ids_permitidos: set[int] | None = None) -> list[Bodega]:
74     """Todas las bodegas razonables para lo dicho/escrito, de más a menos
75     específica - vacía si no hay ninguna, un solo elemento si es
76     inequívoco, varios si es ambiguo ("restaurante" a secas encaja por
77     igual en "RESTAURANTE FUENTES AYB" y "RESTAURANTE FUENTES SUMIN": ahí
78     quien llama debe ofrecer las opciones, no adivinar cuál de las dos).
79
80     ids_permitidos: si se da, restringe las coincidencias PARCIALES/
81     ambiguas (por dónde se filtran nombres de bodegas ajenas que un
82     auxiliar ni debería ver sugeridas). Una coincidencia EXACTA nunca se
83     restringe: si dijo el nombre completo de una bodega real que no es
84     suya, debe decir "no está asignada a usted" - no fingir que no
85     existe y ofrecerle crear una bodega duplicada."""
86     clave = normalizar(texto)
87     if not clave:
88         return []
89
90     # El catálogo de bodegas es chico (unas pocas decenas): comparar en
91     # Python, con normalizar() de los DOS lados, es lo que permite que
92     # reglas como K->C funcionen igual para lo dicho y para el nombre
93     # guardado - comparar contra la columna cruda (como antes, con SQL)
94     # solo servía porque el nombre YA estaba guardado en mayúsculas sin
95     # tildes, por pura coincidencia; una regla nueva de normalizar() no
96     # se aplicaba del lado de la base de datos.
97     todas = s.query(Bodega).all()
98
99     def _permitidas():
100         if ids_permitidos is None:
101             return todas
102         return [b for b in todas if b.id in ids_permitidos]
103
104     exacta = next((b for b in todas if normalizar(b.nombre_oficial) == clave), None)
105     if exacta:
106         return [exacta]
107     # coincidencia parcial: "kiosco" a secas es substring de "KIOSCO 2
108     # SUMINISTROS", "KIOSCO TAQUILLA AYB" y varias más - todas cuentan.
109     contienen = [b for b in _permitidas() if clave in normalizar(b.nombre_oficial)]
110     if contienen:
111         return contienen
112     # ultimo recurso: cuantas palabras dichas (de mas de 3 letras) tiene
113     # cada bodega - "autoservicios las fuentes" perdía "fuentes" antes y
114     # se quedaba con la primera bodega que solo compartiera
115     # "autoservicios"; ahora esa comparte 1 palabra y "AUTOSERVICIOS LAS
116     # FUENTES" comparte 2, así que gana sola. Un empate real (dos
117     # candidatas con el mismo máximo) se devuelve como ambigüedad.
118     palabras = [p for p in clave.split() if len(p) > 3]
119     if not palabras:

```

```

120         return []
121     candidatas = [(sum(1 for p in palabras if p in normalizar(b.nombre_oficial)), b)
122                   for b in _permitidas()]
123     candidatas = [(n, b) for n, b in candidatas if n > 0]
124     if not candidatas:
125         return []
126     maximo = max(n for n, _ in candidatas)
127     return [b for n, b in candidatas if n == maximo]
128
129
130 def buscar_bodega(s, texto: str, ids_permitidos: set[int] | None = None) -> Bodega | None:
131     """La misma bodega para el nombre dicho o escrito, en un solo lugar
132     (antes /api/bodegas/abrir y el agente conversacional cada uno tenia
133     su propia version, y se desincronizaban). Nunca adivina entre varias
134     candidatas igual de buenas: mejor decir "no la encuentro" (o, para
135     quien necesite las opciones, ver buscar_bodegas_candidatas) que abrir
136     la bodega equivocada."""
137     candidatas = buscar_bodegas_candidatas(s, texto, ids_permitidos)
138     if len(candidatas) == 1:
139         return candidatas[0]
140     if ids_permitidos is not None and not candidatas:
141         # nada dentro de lo permitido - pero si el nombre dicho identifica
142         # sin ambigüedad una bodega REAL en todo el catálogo, hay que
143         # decir que existe (y dejar que quien llama la rechace por no
144         # estar asignada), no fingir que no existe y ofrecer crear un
145         # duplicado de una bodega que ya está en el sistema.
146         sin_restriccion = buscar_bodegas_candidatas(s, texto, None)
147         if len(sin_restriccion) == 1:
148             return sin_restriccion[0]
149     return None
150
151
152 def buscar_articulo(texto: str, bodega_id=None, limite: int = 3) -> list[dict]:
153     consulta = normalizar(texto)
154     if not consulta:
155         return []
156     with Sesion() as s:
157         # 1) ¿ya aprendio como lo dice este equipo?
158         alias = s.query(AliasArticulo).filter_by(texto_dicho=consulta).first()
159         if alias:
160             a = s.get(Articulo, alias.articulo_codigo)
161             if a:
162                 return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
163                         "unidad": a.unidad_medida, "confianza": 100}]
164         # 2) si no, compara contra todo el catalogo
165         arts = s.query(Articulo).all()
166
167     if not arts:
168         return []
169     puntajes = []
170     for a in arts:
171         p = puntuar(consulta, normalizar(a.nombre_oficial))
172         if p >= 45:
173             puntajes.append((p, a))
174     puntajes.sort(key=lambda z: -z[0])
175     return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
176             "unidad": a.unidad_medida, "confianza": round(p)}
177           for p, a in puntajes[:limite]]
178
179
180 def aprender_alias(texto: str, codigo: str, bodega_id=None):
181     """Cada eleccion confirmada hace al agente mas preciso."""
182     consulta = normalizar(texto)
183     if not consulta or not codigo:
184         return
185     with Sesion() as s:
186         ya = s.query(AliasArticulo).filter_by(texto_dicho=consulta,
187                                               articulo_codigo=codigo).first()
188         if not ya:
189             s.add(AliasArticulo(texto_dicho=consulta,
190                                articulo_codigo=codigo, bodega_id=bodega_id))
191         s.commit()

```

backend/servicios/analitica.py

166 LÍNEAS

Los indicadores del Panel y el análisis de consumo.

```

1  """Las métricas del panel gerencial y de reportes: todo calculado en vivo
2  sobre la misma base que usa el resto de la aplicación – una sola fuente."""
3  from datetime import datetime
4  from bd import Sesion
5  from modelos import (Bodega, Articulo, StockSistema, SesionConteo, Conteo,
6                      Alerta, AliasArticulo, HistorialCierre, ConfigClave, Aprobacion)
7
8
9  def _config(clave: str, defecto: str) -> str:
10     with Sesion() as s:
11         c = s.get(ConfigClave, clave)
12         return c.valor if c else defecto
13
14
15  def diferencias_por_bodega(limite: int = 6) -> list[dict]:
16     """Suma de |contado - sistema| por bodega, sobre lo confirmado."""
17     with Sesion() as s:
18         bodegas = s.query(Bodega).all()
19         salida = []
20         for b in bodegas:
21             conteos = (s.query(Conteo).join(SesionConteo)
22                        .filter(SesionConteo.bodega_id == b.id,
23                              Conteo.estado == "confirmado").all())
24             if not conteos:
25                 continue
26             total = 0.0
27             for c in conteos:
28                 st = s.query(StockSistema).filter_by(
29                     articulo_codigo=c.articulo_codigo, bodega_id=b.id).first()
30                 esperado = st.cantidad_sd if st else 0
31                 total += abs((c.cantidad or 0) - esperado)
32             if total > 0:
33                 salida.append({"bodega": b.nombre_oficial, "diferencia": round(total, 1)})
34     salida.sort(key=lambda x: -x["diferencia"])
35     return salida[:limite]
36
37
38  def stock_por_unidad() -> list[dict]:
39     with Sesion() as s:
40         arts = {a.codigo: a.unidad_medida for a in s.query(Articulo).all()}
41         acumulado = {}
42         for st in s.query(StockSistema).all():
43             u = arts.get(st.articulo_codigo, "Otra")
44             acumulado[u] = acumulado.get(u, 0) + max(st.cantidad_sd or 0, 0)
45     total = sum(acumulado.values()) or 1
46     salida = [{"unidad": u, "cantidad": round(c, 1),
47               "pct": round(c / total * 100, 1)}
48               for u, c in acumulado.items()]
49     salida.sort(key=lambda x: -x["cantidad"])
50     return salida
51
52
53  def alertas_por_tipo() -> list[dict]:
54     etiquetas = {"desviacion": "Desviación", "unidad": "Unidad",
55                 "negativo": "Saldo negativo", "inexistente": "Inexistente"}
56     with Sesion() as s:
57         conteo = {}
58         for a in s.query(Alerta).all():
59             conteo[a.tipo] = conteo.get(a.tipo, 0) + 1
60     salida = [{"tipo": etiquetas.get(t, t), "cantidad": n} for t, n in conteo.items()]
61     salida.sort(key=lambda x: -x["cantidad"])
62     return salida
63
64
65  def descuadres_recurrentes(limite: int = 5) -> list[dict]:
66     """Un artículo con diferencia en más de una toma: ahí está la causa raíz."""
67     with Sesion() as s:
68         arts = {a.codigo: a.nombre_oficial for a in s.query(Articulo).all()}
69         # un articulo "PEND-..." ya aprobado o rechazado conserva ese mismo

```



```

70     # codigo para siempre (nunca se reemplaza por uno oficial al
71     # aprobarlo) - el prefijo solo no basta para saber si de verdad
72     # sigue pendiente; sin este chequeo, uno ya aprobado hace rato
73     # seguia sugiriendo "Crear en catálogo oficial" como si nunca se
74     # hubiera resuelto.
75     pendientes_de_verdad = {a.articulo_codigo for a in
76                             s.query(Aprobacion).filter_by(
77                                 tipo="producto", estado="pendiente").all()
78                             if a.articulo_codigo}
79     por_articulo = {}
80     conteos = (s.query(Conteo).join(SesionConteo)
81                .filter(Conteo.estado == "confirmado").all())
82     for c in conteos:
83         ses = s.get(SesionConteo, c.sesion_id)
84         st = s.query(StockSistema).filter_by(
85             articulo_codigo=c.articulo_codigo, bodega_id=ses.bodega_id).first()
86         esperado = st.cantidad_sd if st else 0
87         dif = (c.cantidad or 0) - esperado
88         if abs(dif) < 0.01:
89             continue
90         d = por_articulo.setdefault(c.articulo_codigo,
91                                     {"dif": 0.0, "tomas": set(), "negativo": False})
92         d["dif"] += dif
93         d["tomas"].add(c.sesion_id)
94         if esperado < 0:
95             d["negativo"] = True
96     salida = []
97     for cod, d in por_articulo.items():
98         if len(d["tomas"]) < 2 and not d["negativo"]:
99             continue
100     salida.append({
101         "articulo": arts.get(cod, cod),
102         "tipo": "Saldo negativo" if d["negativo"] else "Desviación",
103         "diferencia": round(d["dif"], 1),
104         "tomas": len(d["tomas"]),
105         "accion": "Revisar salidas sin entrada" if d["negativo"]
106                 else "Crear en catálogo oficial" if cod in pendientes_de_verdad
107                 else "Ajustar el saldo del sistema",
108     })
109     salida.sort(key=lambda x: -abs(x["diferencia"]))
110     return salida[:limite]
111
112
113 def historial_exactitud(limite: int = 12) -> list[dict]:
114     with Sesion() as s:
115         filas = (s.query(HistorialCierre).order_by(HistorialCierre.fecha).all())[-limite:]
116         nombres = {b.id: b.nombre_oficial for b in s.query(Bodega).all()}
117     return [{"id": f.id, "fecha": f.fecha.strftime("%Y-%m-%d"), "exactitud": round(f.exactitud, 1),
118            "bodega_id": f.bodega_id, "bodega": nombres.get(f.bodega_id, "")} for f in filas]
119
120
121 def resumen_ejecutivo() -> dict:
122     with Sesion() as s:
123         bodegas = s.query(Bodega).all()
124         cerradas = [b for b in bodegas if b.estado == "cerrada"]
125         total_conteos = s.query(Conteo).filter_by(estado="confirmado").count()
126         alertas_total = s.query(Alerta).count()
127         alertas_resueltas = s.query(Alerta).filter_by(resuelta=1).count()
128         historial = s.query(HistorialCierre).all()
129         exact = (round(sum(h.exactitud for h in historial) / len(historial), 1)
130                 if historial else 100.0)
131     return {
132         "exactitud_primera_pasada": exact,
133         "referencias_contadas": total_conteos,
134         "alertas_gestionadas": alertas_resueltas,
135         "alertas_total": alertas_total,
136         "bodegas_cerradas": len(cerradas),
137         "bodegas_total": len(bodegas),
138         "diferencia_por_bodega": diferencias_por_bodega(limite=10),
139         "stock_por_unidad": stock_por_unidad(),
140         "historial_exactitud": historial_exactitud(),
141     }
142
143

```

```

144 def resumen_alertas_panel() -> dict:
145     with Sesion() as s:
146         negativos_actuales = s.query(StockSistema).filter(
147             StockSistema.cantidad_sd < 0).count()
148         alias = s.query(AliasArticulo).count()
149         bodegas = s.query(Bodega).all()
150         cerradas_ses = (s.query(SesionConteo).filter_by(tipo="conteo", estado="cerrada")
151             .filter(SesionConteo.fin.isnot(None)).all())
152         minutos = [(ses.fin - ses.inicio).total_seconds() / 60]
153             for ses in cerradas_ses if ses.fin]
154         negativos_iniciales = int(_config("negativos_iniciales", str(negativos_actuales or 0)))
155         estado_bodegas = {}
156         for b in bodegas:
157             estado_bodegas[b.estado] = estado_bodegas.get(b.estado, 0) + 1
158         return {
159             "negativos_iniciales": negativos_iniciales,
160             "negativos_actuales": negativos_actuales,
161             "tiempo_promedio_min": round(sum(minutos) / len(minutos)) if minutos else 0,
162             "alias_aprendidos": alias,
163             "estado_bodegas": estado_bodegas,
164             "alertas_por_tipo": alertas_por_tipo(),
165             "descuadres_recurrentes": descuadres_recurrentes(),
166         }

```

backend/servicios/validacion.py

66 LÍNEAS

Las reglas que se aplican antes de guardar.

```

1  """Las reglas duras. Codigo determinista: se comporta igual siempre."""
2  import os
3  from bd import Sesion
4  from modelos import Articulo, StockSistema, ConfigClave
5
6  UMBRAL_DEFECTO = float(os.getenv("UMBRAL_ANOMALIA", 0.5))
7
8
9  def umbral_actual() -> float:
10     """El umbral puede ajustarse desde Ajustes; si no, manda el .env."""
11     with Sesion() as s:
12         c = s.get(ConfigClave, "umbral_anomalia")
13         return float(c.valor) if c else UMBRAL_DEFECTO
14
15
16  def validar_conteo(codigo: str, cantidad: float, unidad: str, bodega_id) -> dict:
17     UMBRAL = umbral_actual()
18     with Sesion() as s:
19         art = s.get(Articulo, codigo)
20         if art is None:
21             return {"ok": False, "tipo": "inexistente",
22                 "mensaje": "Ese articulo no esta en el catalogo. "
23                     "¿Lo creamos? Quedara pendiente de aprobacion."}
24
25         if cantidad is None:
26             return {"ok": False, "tipo": "vacio",
27                 "mensaje": "No entendi la cantidad. ¿Me la repite? "}
28
29         # mini reto de Colsubsidio: un saldo fisico nunca baja de cero
30         if cantidad < 0:
31             return {"ok": False, "tipo": "negativo",
32                 "mensaje": f"Una cantidad no puede ser negativa: un saldo "
33                     "fisico nunca baja de cero. Digame el valor correcto."}
34
35         # «cinco kilos» no se confunde con «cinco gramos»
36         if unidad and unidad.strip().lower() != art.unidad_medida.strip().lower():
37             return {"ok": False, "tipo": "unidad",
38                 "mensaje": f"Ese articulo se maneja en {art.unidad_medida}, "
39                     f"no en {unidad}. ¿Cual es la cantidad "
40                     f"en {art.unidad_medida}?" }
41
42         stock = s.query(StockSistema).filter_by(
43             articulo_codigo=codigo, bodega_id=bodega_id).first()
44         esperado = stock.cantidad_sd if stock else None

```

```

45
46 # el famoso 9 contra 90
47 if esperado is not None and esperado > 0:
48     desvio = abs(cantidad - esperado) / esperado
49     if desvio > UMBRAL:
50         return {"ok": False, "tipo": "desviacion", "esperado": esperado,
51                 "mensaje": f"El sistema espera alrededor de {esperado:g}. "
52                           f"¿Confirma {cantidad:g}?"}
53 # el sistema SI tiene una expectativa aqui (una fila real de
54 # StockSistema, no la ausencia de dato que ya cubre "esperado is
55 # None" arriba) y esa expectativa es CERO - "esperado > 0" arriba
56 # evita dividir por cero para calcular el porcentaje de desviacion,
57 # pero de paso dejaba sin ninguna alerta el caso de contar algo
58 # donde el sistema no esperaba nada: encontrar existencia donde
59 # deberia haber cero es tan anomalo como el 9 contra 90 de arriba,
60 # solo que no tiene un "%" que calcular.
61 elif esperado == 0 and cantidad > 0:
62     return {"ok": False, "tipo": "desviacion", "esperado": esperado,
63           "mensaje": f"El sistema no tiene existencia registrada aqui para este articulo. "
64                   f"¿Confirma {cantidad:g}?"}
65
66 return {"ok": True, "esperado": esperado}

```

backend/servicios/archivos.py

29 LÍNEAS

Escritura y lectura de los archivos generados.

```

1 """Guardar y leer los XLSX/CSV generados desde la base, no desde disco -
2 el disco del Web Service en Render es efimero (se borra en cada deploy o
3 reinicio), así que un reporte "generado hace una hora" quedaria
4 inalcanzable despues del siguiente `git push`. La "ruta" (ej.
5 "reportes/consolidado_20260815_1636.xlsx") se sigue usando como
6 identificador legible en toda la app - solo cambia DONDE vive el
7 contenido real."""
8 import io
9
10 from bd import Sesion
11 from modelos import ArchivoGenerado
12
13
14 def guardar_df(ruta: str, df, formato: str) -> None:
15     buf = io.BytesIO()
16     if formato == "csv":
17         df.to_csv(buf, index=False)
18     else:
19         df.to_excel(buf, index=False)
20     with Sesion() as s:
21         s.add(ArchivoGenerado(ruta=ruta, contenido=buf.getvalue()))
22         s.commit()
23
24
25 def leer_bytes(ruta: str) -> bytes | None:
26     with Sesion() as s:
27         obj = (s.query(ArchivoGenerado).filter_by(ruta=ruta)
28               .order_by(ArchivoGenerado.id.desc()).first())
29     return obj.contenido if obj else None

```

17.4 Frontend · nucleo

frontend/src/main.jsx

18 LÍNEAS

El punto de entrada.

```

1 import React from "react";
2 import { createRoot } from "react-dom/client";
3 import * as Sentry from "@sentry/react";
4 import App from "../App";
5 import "../index.css";

```

```

6
7 // Apagado por defecto (sin VITE_SENTRY_DSN, Sentry.init() nunca llama a
8 // ningun lado) - el ErrorBoundary de abajo es inofensivo sin DSN, solo no
9 // reporta nada. Activarlo en produccion es poner la variable de entorno,
10 // sin tocar código.
11 const dsn = import.meta.env.VITE_SENTRY_DSN;
12 if (dsn) Sentry.init({ dsn, tracesSampleRate: 0 });
13
14 createRoot(document.getElementById("root")).render(
15   <Sentry.ErrorBoundary fallback={<p style={{ padding: 20 }}>Ocurrió un error. Recargue la página.</p>>
16     <App />
17   </Sentry.ErrorBoundary>
18 );

```

frontend/src/App.jsx

280 LÍNEAS

El menu, la vista activa, la sesion y el contexto. Agregar una pantalla empieza aqui.

```

1 import { useEffect, useRef, useState } from "react";
2 import { alSesionInvalida, borrarSesion, guardarSesion, leerSesion, pedir } from "../api";
3 import { obtenerTokenValido, cerrarSesionLocal } from "../cognito";
4 import { configurarVoz, detenerVoz } from "../voz";
5 import { leerPreferencias, aplicarPreferencias } from "../accesibilidad";
6 import VideoRecorrido from "../VideoRecorrido";
7 import { debeAbrirTutorial, EVENTO_ABRIR_VIDEO } from "../tutorial";
8 import BarraLateral from "../BarraLateral";
9 import Ingreso from "../vistas/Ingreso";
10 import Inicio from "../vistas/Inicio";
11 import Conteo from "../vistas/Conteo";
12 import Pedido from "../vistas/Pedido";
13 import Legalizacion from "../vistas/Legalizacion";
14 import Bodegas from "../vistas/Bodegas";
15 import Auditoria from "../vistas/Auditoria";
16 import Reportes from "../vistas/Reportes";
17 import Panel from "../vistas/Panel";
18 import Ajustes from "../vistas/Ajustes";
19 import Ayuda from "../vistas/Ayuda";
20 import Mensajes from "../vistas/Mensajes";
21 import MiPerfil from "../vistas/MiPerfil";
22 import CerrarSesion from "../vistas/CerrarSesion";
23
24 /* El menú lateral. Cada entrada declara las vistas que contiene:
25    este objeto es el mapa de navegación hecho código. */
26 export const MENU = [
27   { id: "inicio", titulo: "Inicio", vistas: ["inicio"] },
28   { id: "pedidos", titulo: "Pedidos", vistas: ["pedido"] },
29   { id: "conteo", titulo: "Conteo", vistas: ["conteo"] },
30   { id: "legalizacion", titulo: "Legalización", vistas: ["legalizacion"] },
31   { id: "bodegas", titulo: "Bodegas", vistas: ["bodegas"] },
32   { id: "auditoria", titulo: "Auditoría", vistas: ["auditoria"], soloAuditor: true },
33   { id: "reportes", titulo: "Reportes", vistas: ["reportes"], soloAuditor: true },
34   { id: "panel", titulo: "Panel", vistas: ["panel"], soloAuditor: true },
35   // El auxiliar no puede cambiar nada aquí (el umbral y el modo sin
36   // conexión ya estaban bloqueados por el backend con requiere_perfil):
37   // mostrarle la pantalla igual, solo en modo lectura, dejaba ver ajustes
38   // administrativos que no le sirven de nada y no puede tocar - mejor
39   // ocultarla del todo, igual que ya se hace con Auditoría/Reportes/Panel.
40   { id: "ajustes", titulo: "Ajustes", vistas: ["ajustes"], soloAuditor: true },
41   { id: "ayuda", titulo: "Ayuda", vistas: ["ayuda"] },
42   { id: "mensajes", titulo: "Mensajes", vistas: ["mensajes"] },
43   { id: "perfil", titulo: "Mi perfil", vistas: ["perfil"] },
44   { id: "salir", titulo: "Cerrar sesión", vistas: ["salir"] },
45 ];
46
47 const VISTAS = {
48   inicio: Inicio, pedido: Pedido, conteo: Conteo,
49   legalizacion: Legalizacion, bodegas: Bodegas, auditoria: Auditoria,
50   reportes: Reportes, panel: Panel, ajustes: Ajustes, ayuda: Ayuda,
51   mensajes: Mensajes, perfil: MiPerfil,
52 };
53

```

```

54 /** Qué entrada del menú resaltar para una vista. Garantiza que el menú
55     señale siempre dónde vive la pantalla que se está viendo. */
56 export function menuDe(vista) {
57     const g = MENU.find((m) => m.vistas.includes(vista));
58     return g ? g.id : null;
59 }
60
61 export default function App() {
62     // arranca directo con la sesión ya guardada (si hay) en vez de forzar
63     // login otra vez: sin esto, abrir la app sin señal (o reiniciarla en
64     // medio de una caída de Wi-Fi) dejaba a cualquiera sin poder entrar,
65     // aunque ya se hubiera identificado antes en este mismo equipo.
66     const [sesion, setSesion] = useState(() => leerSesion());
67     const [vista, setVista] = useState("inicio");
68     // Antes de abrir una bodega no hay todavía un id real de sesión de
69     // conteo (ver Conteo.jsx) - un marcador de posición fijo (antes: 1 para
70     // todo el mundo) hacia que dos personas sin ninguna bodega abierta
71     // todavía pudieran mezclar su conversación pendiente. Uno por usuario,
72     // igual que ya hace Pedido.jsx con el suyo.
73     const [ctx, setCtx] = useState(() => ({ sessionId: -2000 - (sesion?.usuario?.id || 0) }));
74     const [salir, setSalir] = useState(false);
75     const [avisoSesion, setAvisoSesion] = useState("");
76     const [mostrarVideo, setMostrarVideo] = useState(false);
77
78     // "Ver el recorrido en video" en Ayuda.jsx reabre el recorrido bajo
79     // demanda con un evento (mismo patrón que cuentavoz:foto-actualizada) en
80     // vez de pasar la función por props a través de todas las vistas.
81     useEffect(() => {
82         function abrirVideo() { setMostrarVideo(true); }
83         window.addEventListener(EVENTO_ABRIR_VIDEO, abrirVideo);
84         return () => window.removeEventListener(EVENTO_ABRIR_VIDEO, abrirVideo);
85     }, []);
86
87     // El video del recorrido abre en CADA ingreso, no solo el primero - lo
88     // único que lo apaga es que la propia persona lo desactive desde el
89     // video (ver VideoRecorrido.jsx). "Ingreso" no es solo el instante de
90     // escribir usuario y clave: la sesión guardada (arriba, leerSesion())
91     // hace que reabrir la app o recargar la página entre DIRECTO a Inicio
92     // sin pasar por Ingreso.jsx. Enganchado solo al login manual, alguien
93     // que ya tenía sesión activa en este equipo (lo normal) nunca disparaba
94     // el video. Aquí se revisa cada vez que hay un usuario con sesión, la
95     // haya iniciado ahora o la haya retomado la app sola.
96     useEffect(() => {
97         const id = sesion?.usuario?.id;
98         if (id !== null && debeAbrirTutorial(id)) setMostrarVideo(true);
99     }, [sesion?.usuario?.id]);
100
101     // Alto contraste / tamaño de letra (Mi perfil > Accesibilidad) son
102     // preferencias de ESTE equipo (localStorage), no de la cuenta - se
103     // aplican una sola vez al montar, antes incluso de que haya sesión, para
104     // que también se vean en la pantalla de login.
105     useEffect(() => {
106         aplicarPreferencias(leerPreferencias());
107     }, []);
108
109     // Se suscribe una sola vez: cuando pedir() encuentra un 401 (sesión
110     // cerrada desde otro dispositivo, PIN cambiado, cuenta desactivada...)
111     // vuelve aquí mismo a la pantalla de login en vez de dejar el menú y las
112     // vistas mostrando datos de una sesión que el servidor ya no reconoce.
113     useEffect(() => {
114         alSesionInvalida(() => {
115             cerrarSesionLocal();
116             setSesion(null);
117             setVista("inicio");
118             setAvisoSesion("Su sesión terminó (se cerró desde otro dispositivo o venció). Ingrese de nuevo.");
119         });
120     }, []);
121
122     // El access token de Cognito dura poco a propósito (1 hora, ver
123     // ARQUITECTURA.md), pero cada vista recibe el token como prop suelto
124     // (no llama a Cognito directo por sí misma) - sin esto, la sesión se
125     // caía cada hora en plena demo. Se revisa cada pocos minutos (getSession
126     // del SDK refresca solo con el refresh token, sin red de por medio si
127     // el access token todavía es válido) y se actualiza sesion.token para

```

```

128 // que el próximo render lo pase ya fresco a todas las pantallas.
129 useEffect(() => {
130   if (!sesion) return;
131   let vivo = true;
132   async function refrescar() {
133     const token = await obtenerTokenValido();
134     if (!vivo) return;
135     if (!token) {
136       cerrarSesionLocal();
137       setSesion(null);
138       setVista("inicio");
139       setAvisoSesion("Su sesión venció. Ingrese de nuevo.");
140       return;
141     }
142     if (token !== sesion.token) {
143       const nueva = { ...sesion, token };
144       guardarSesion(nueva);
145       setSesion(nueva);
146     }
147   }
148   refrescar();
149   const intervalo = setInterval(refrescar, 4 * 60 * 1000);
150   return () => { vivo = false; clearInterval(intervalo); };
151   // eslint-disable-next-line react-hooks/exhaustive-deps
152 }, [sesion?.usuario?.id]);
153
154 // Contador que sube en cada ir(): las pantallas con pestañas usan
155 // [tabInicial, navSeq] como dependencia de su useEffect en vez de solo
156 // [tabInicial] - sin navSeq, pedir por voz la MISMA pestaña dos veces
157 // seguidas (con un clic manual a otra pestaña en el medio) no cambiaba
158 // nada, porque tabInicial ya traía ese mismo valor de la vez anterior y
159 // React no vuelve a correr un efecto cuya dependencia no cambió de
160 // valor, aunque la persona sí haya navegado a otro lado desde entonces.
161 const navSeq = useRef(0);
162
163 /** Navega y pasa contexto: ir("bodegas", { bodegaId: 3 }) */
164 function ir(destino, contexto = {}) {
165   // sin esto, una respuesta hablada que la pantalla anterior todavía
166   // estaba esperando (la síntesis real tarda unos segundos) podía
167   // reproducirse sola varios segundos después, ya con otra pantalla en
168   // uso - sonaba como si el agente hablara de la nada, fuera de tema.
169   detenerVoz();
170   navSeq.current += 1;
171   setCtx((c) => ({ ...c, ...contexto, navSeq: navSeq.current }));
172   setVista(destino);
173 }
174
175 /** Conteo.jsx la llama apenas abre (o retoma) una bodega, con el id real
176   que le devuelve el backend - así CerrarSesion (que vive fuera de
177   Conteo, en la barra lateral) pregunta por el avance de la sesión que
178   de verdad está abierta, no por el marcador de posición de antes de
179   abrir nada. */
180 function alAbrirBodega(sesionId) {
181   setCtx((c) => ({ ...c, sesionId }));
182 }
183
184 if (!sesion)
185   return (
186     <Ingreso
187       avisoInicial={avisoSesion}
188       alEntrar={(s) => {
189         guardarSesion(s);
190         setSesion(s);
191         setAvisoSesion("");
192         setVista("inicio");
193         pedir("/api/usuarios/yo", {}, s.token).then((p) =>
194           configurarVoz({ idioma: p.idioma_voz, velocidad: p.velocidad_voz,
195             confirmacionHablada: p.confirmacion_hablada })
196         ).catch(() => {});
197       }}
198     />
199   );
200
201 const Vista = VISTAS[vista] || Inicio;

```

```

202
203 return (
204   <div className="app-root">
205     {/* Salto al contenido: invisible hasta que alguien tabula. Sin esto,
206        quien navega con teclado o con lector de pantalla tiene que
207        recorrer las trece opciones del menú en CADA pantalla antes de
208        llegar a lo que vino a hacer. */}
209     <a className="salto-contenido" href="#contenido">Saltar al contenido</a>
210     <BarraLateral
211       activo={salir ? "salir" : menuDe(vista)}
212       usuario={sesion.usuario}
213       token={sesion.token}
214       sesionId={ctx.sesionId}
215       onNavegar={(id) => {
216         detenerVoz();
217         if (id === "salir") {
218           setSalir(true);
219           return;
220         }
221         const g = MENU.find((m) => m.id === id);
222         if (g) {
223           // el menu lateral es una navegacion "limpia": sin esto, un
224           // contexto de una vista anterior (tabInicial="recetas" de
225           // Pedido, bodegaId de un detalle, etc.) se quedaba pegado para
226           // siempre - la proxima vez que se entraba a Ajustes por el
227           // menu, por ejemplo, seguia abriendo la pestana equivocada.
228           setCtx({ sesionId: ctx.sesionId });
229           setVista(g.vistas[0]);
230         }
231       }}
232   />
233
234   {/* <main> y no <div>: es el punto de referencia que buscan los
235      lectores de pantalla para saltarse la navegación, y el destino
236      del enlace de arriba. tabIndex -1 para que el salto le lleve el
237      foco de verdad, no solo el desplazamiento. */}
238   <main className="contenido" id="contenido" tabIndex={-1}>
239     <Vista
240       token={sesion.token}
241       usuario={sesion.usuario}
242       {...ctx}
243       ir={ir}
244       alAbrirBodega={alAbrirBodega}
245     />
246   </main>
247
248   {salir && (
249     <CerrarSesion
250       token={sesion.token}
251       sesionId={ctx.sesionId}
252       onCancel={() => setSalir(false)}
253       onSalir={() => {
254         // Revoca en Cognito (mismo mecanismo que "cerrar todas las
255         // sesiones" en Mi perfil) antes de limpiar localmente - sin
256         // esto, "Cerrar sesión" solo borraba el token de ESTE
257         // navegador; el access token seguía siendo válido en Cognito
258         // hasta vencer solo (hasta 1 hora). En una tablet compartida entre
259         // varios auxiliares, quien lo hubiera copiado antes de que la
260         // persona cerrara sesión podía seguir usándolo. No bloquea la
261         // salida si la petición falla (sin red) - la persona debe poder
262         // salir localmente de todas formas.
263         pedir("/api/usuarios/yo/cerrar-todas", { method: "POST" }, sesion.token).catch(() => {});
264         cerrarSesionLocal();
265         borrarSesion();
266         setSesion(null);
267         setSalir(false);
268         setVista("inicio");
269       }}
270   />
271 )}
272
273   {mostrarVideo && (
274     <VideoRecorrido perfil={sesion.usuario?.perfil}
275       usuarioId={sesion.usuario?.id}

```

```

276             onCerrar={() => setMostrarVideo(false)} />
277         )}
278     </div>
279 );
280 }

```

frontend/src/api.js

183 LÍNEAS

Todas las llamadas al backend, y esFalloRed, que distingue un fallo de red de un error del servidor.

```

1  // IPv4 explícito, no "localhost": en Windows el navegador suele resolver
2  // "localhost" a ::1 (IPv6), y uvicorn por defecto solo escucha en 127.0.0.1.
3  export const BASE = import.meta.env.VITE_API_URL || "http://127.0.0.1:8000";
4
5  export function guardarToken(t) {
6      localStorage.setItem("cv_token", t);
7  }
8  export function leerToken() {
9      return localStorage.getItem("cv_token");
10 }
11 export function borrarToken() {
12     localStorage.removeItem("cv_token");
13 }
14
15 // La sesión completa (token + datos del usuario), no solo el token: sin
16 // esto, reabrir la app (o perder la señal justo al recargar) obligaba a
17 // iniciar sesión otra vez contra el backend - imposible sin red, aunque
18 // la persona ya se hubiera identificado minutos antes en el mismo equipo.
19 // Con la sesión guardada, App.jsx entra directo sin depender de la red.
20 const CLAVE_SESION = "cv_sesion";
21
22 export function guardarSesion(s) {
23     localStorage.setItem(CLAVE_SESION, JSON.stringify(s));
24     if (s?.token) guardarToken(s.token); // cv_token queda siempre en sync
25 }
26 export function leerSesion() {
27     try { return JSON.parse(localStorage.getItem(CLAVE_SESION) || "null"); }
28     catch (_) { return null; }
29 }
30 export function borrarSesion() {
31     localStorage.removeItem(CLAVE_SESION);
32     borrarToken();
33 }
34
35 // App.jsx se suscribe una sola vez al montar para poder volver a la
36 // pantalla de login apenas el backend dice que el token ya no sirve
37 // (sesión cerrada desde otro dispositivo, PIN cambiado, cuenta
38 // desactivada...). Sin esto, pedir() ya borraba la sesión guardada pero la
39 // pestaña seguía mostrando el menú y las vistas como si siguiera con
40 // sesión, y cada pantalla fallaba en silencio o mostraba el error crudo
41 // del servidor como si fuera un dato más (p. ej. "0 bodegas" en vez de
42 // avisar que hay que volver a ingresar).
43 let _alSesionInvalida = null;
44 export function alSesionInvalida(fn) {
45     _alSesionInvalida = fn;
46 }
47
48 /** fetch() rechaza con un TypeError cuando la red falla (Wi-Fi caído, DNS,
49     CORS bloqueado) - la especificación lo llama "network error". Detectarlo
50     así (en vez de por texto del mensaje) es lo que ya usaba Conteo.jsx para
51     activar el modo sin conexión; se comparte aquí para que todo el frontend
52     reconozca el mismo caso una sola vez. */
53 export function esFalloRed(error) {
54     return error?.esFalloRed === true || error instanceof TypeError
55         || /failed to fetch|networkerror|load failed/i.test(error?.message || "");
56 }
57
58 /** El error técnico crudo en inglés que fetch() lanza tal cual, traducido a
59     algo que un auxiliar en una bodega con mal Wi-Fi pueda entender. Marca el
60     error con esFalloRed=true para que quien lo reciba después de pedir()
61     (que ya perdió el TypeError original) lo siga reconociendo con

```



```

62     esFalloRed(), en vez de tener que repetir la detección por texto. */
63 function errorDeRed() {
64     const error = new Error("Sin conexión con el servidor. Revise el Wi-Fi e intente de nuevo.");
65     error.esFalloRed = true;
66     return error;
67 }
68
69 /** Llama al backend adjuntando el token de la sesión. */
70 export async function pedir(ruta, opciones = {}, token) {
71     const t = token || leerToken();
72     // arma las opciones completas ANTES del try: si algo aquí lanza (un
73     // "opciones.headers" mal formado, por ejemplo), es un error de
74     // programación real y no debe disfrazarse de fallo de red.
75     const opcionesCompletas = {
76         ...opciones,
77         headers: {
78             "Content-Type": "application/json",
79             ...(t ? { Authorization: `Bearer ${t}` } : {}),
80             ...opciones.headers,
81         },
82     };
83     let res;
84     try {
85         res = await fetch(`${BASE}${ruta}`, opcionesCompletas);
86     } catch (error) {
87         if (esFalloRed(error)) throw errorDeRed();
88         throw error;
89     }
90     if (!res.ok) {
91         // 401 de verdad (no un fallo de red: el servidor sí respondió) significa
92         // que el servidor ya no reconoce este token - ni reintentar ni seguir
93         // usando la sesión cacheada sirve de nada; se limpia para que la
94         // próxima carga pida iniciar sesión de nuevo en vez de quedar
95         // atascada reusando un token que el backend rechaza siempre.
96         if (res.status === 401) { borrarSesion(); _alSesionInvalida?(); }
97         let detalle = `Error ${res.status}`;
98         try {
99             const j = await res.json();
100             detalle = j.detail || j.detalle || detalle;
101         } catch (_) {}
102         throw new Error(detalle);
103     }
104     return res.json();
105 }
106
107 // «respaldo» son las opciones que la pantalla ya tiene mostradas en
108 // pantalla ({ opciones, opcionesPara }); si el backend se reinició y
109 // perdió la memoria de la conversación, las recupera de ahí en vez de
110 // preguntar «¿cuál de los dos?» otra vez sin recordar nada.
111 export const enviarTurno = (texto, sesionId, token, respaldo = {}) =>
112     pedir("/api/agente/turno", {
113         method: "POST",
114         body: JSON.stringify({
115             texto, sesion_id: sesionId,
116             opciones_pendientes: respaldo.opciones || null,
117             opciones_para: respaldo.opcionesPara || null,
118             bodega_id_respaldo: respaldo.bodegaId || null,
119             bodega_nombre_respaldo: respaldo.bodegaNombre || null,
120             preparacion_respaldo: respaldo.preparacion || null,
121             porciones_respaldo: respaldo.porciones || null,
122         }),
123     }, token);
124
125 // Para pantallas que no cuentan ni piden (Inicio, Ajustes, Ayuda, Reportes,
126 // Panel): una pregunta suelta o una orden de navegar, sin el estado de
127 // conversación multi-turno que sí necesita enviarTurno. Ver AsistenteVoz.jsx.
128 export const preguntarAsistente = (texto, vista, token) =>
129     pedir("/api/agente/asistente", {
130         method: "POST",
131         body: JSON.stringify({ texto, vista }),
132     }, token);
133
134 export const abrirBodega = (bodega, token) =>
135     pedir("/api/bodegas/abrir", {

```

```

136     method: "POST",
137     body: JSON.stringify({ bodega }),
138   }, token);
139
140 // Resuelve un nombre dicho/escrito al nombre real del catálogo, sin abrir
141 // nada - para poder confirmar "¿abro X?" con el nombre que de verdad se
142 // va a usar, no con lo que se haya transcrito tal cual (ver Conteo.jsx).
143 export const buscarBodega = (bodega, token) =>
144   pedir("/api/bodegas/buscar", {
145     method: "POST",
146     body: JSON.stringify({ bodega }),
147   }, token);
148
149 /** Descarga un reporte generado por el backend. Un <a href> comun no sirve
150     aqui: el endpoint exige el token en la cabecera, y un enlace del
151     navegador no lo puede enviar (por eso terminaba en una pagina en blanco
152     pidiendo sesion). Se trae como blob autenticado y se dispara la
153     descarga nativa del navegador, sin salir de la aplicacion. */
154 export async function descargarReporte(archivo, token) {
155   const t = token || leerToken();
156   let res;
157   try {
158     res = await fetch(
159       `${BASE}/api/reportes/descargar?archivo=${encodeURIComponent(archivo)}`,
160       { headers: { Authorization: `Bearer ${t}` } }
161     );
162   } catch (error) {
163     if (esFalloRed(error)) throw errorDeRed();
164     throw error;
165   }
166   if (!res.ok) {
167     // mismo trato que pedir(): un 401 aqui tambien significa que la sesion
168     // ya no sirve - sin esto, descargar un reporte con la sesion vencida
169     // solo mostraba "no se pudo descargar" sin volver nunca al login, a
170     // diferencia de cualquier otra llamada de la app.
171     if (res.status === 401) { borrarSesion(); _alSesionInvalida?(); }
172     throw new Error("No se pudo descargar el archivo.");
173   }
174   const blob = await res.blob();
175   const url = URL.createObjectURL(blob);
176   const a = document.createElement("a");
177   a.href = url;
178   a.download = archivo.split("/").pop();
179   document.body.appendChild(a);
180   a.click();
181   a.remove();
182   setTimeout(() => URL.revokeObjectURL(url), 4000);
183 }

```

frontend/src/cognito.js

303 LÍNEAS

Registro, ingreso, clave y verificacion en dos pasos. La clave nunca sale de aqui hacia el backend.

```

1 // Identidad con AWS Cognito: registro, login y recuperar clave hablan
2 // DIRECTO con Cognito desde aquí (nunca por el backend - por eso el App
3 // Client no tiene secreto, está pensado para vivir en el navegador). El
4 // User Pool Id y el Client Id no son secretos (son el equivalente al
5 // "project id" de cualquier SDK de identidad que corre en el cliente);
6 // por eso llevan un valor por defecto, igual que VITE_API_URL en api.js,
7 // para que el proyecto funcione en local sin un .env aparte.
8 import {
9   CognitoUserPool,
10  CognitoUser,
11  AuthenticationDetails,
12  CognitoUserAttribute,
13 } from "amazon-cognito-identity-js";
14
15 const REGION = import.meta.env.VITE_COGNITO_REGION || "us-east-2";
16 const USER_POOL_ID = import.meta.env.VITE_COGNITO_USER_POOL_ID || "us-east-2_6HbrPruvL";
17 const CLIENT_ID = import.meta.env.VITE_COGNITO_APP_CLIENT_ID || "847jtkc5sem7mr4tb8csrrkqp";
18

```

```

19 const poolDatos = { UserPoolId: USER_POOL_ID, ClientId: CLIENT_ID };
20 const pool = new CognitoUserPool(poolDatos);
21
22 /** Traduce los códigos de error que Cognito manda (en inglés, pensados
23     para un desarrollador) a algo que alguien en una bodega entienda. Los
24     demás mensajes de Cognito ya vienen razonablemente claros en español
25     parcial/inglés corto, así que solo se traducen los que de verdad
26     confunden. */
27 function mensajeClaro(error) {
28     const mapa = {
29         UsernameExistsException: "Ese usuario ya existe. Inicie sesión o recupere su clave.",
30         UserNotFoundException: "No existe ese usuario.",
31         NotAuthorizedException: "Usuario o clave incorrectos.",
32         CodeMismatchException: "El código no es correcto.",
33         ExpiredCodeException: "El código venció. Pida uno nuevo.",
34         InvalidPasswordException: "La clave debe tener al menos 8 caracteres, con mayúscula, "
35             + "minúscula y número.",
36         LimitExceededException: "Demasiados intentos. Espere un momento e intente de nuevo.",
37         UserNotConfirmedException: "Falta confirmar el registro con el código enviado por correo.",
38         EnableSoftwareTokenMFAException: "El código de la app autenticadora no es correcto.",
39     };
40     const msg = mapa[error?.code];
41     const final = new Error(msg || error?.message || "No se pudo completar la operación.");
42     final.code = error?.code;
43     return final;
44 }
45
46 function usuarioCognito(nombre) {
47     return new CognitoUser({ Username: nombre.trim().toLowerCase(), Pool: pool });
48 }
49
50 /** Crea la cuenta en Cognito. "name" es un atributo obligatorio del User
51     Pool (además de email) - sin él Cognito rechaza el registro. Devuelve
52     el destino enmascarado (p***@m***) que ya calcula Cognito, para
53     mostrarlo igual que su propia pantalla "Confirm your account". */
54 export function registrarse(usuario, clave, correo, nombreCompleto) {
55     return new Promise((resolve, reject) => {
56         const atributos = [
57             new CognitoUserAttribute({ Name: "email", Value: correo.trim() }),
58             new CognitoUserAttribute({ Name: "name", Value: nombreCompleto.trim() }),
59         ];
60         pool.signUp(usuario.trim().toLowerCase(), clave, atributos, null, (error, resultado) => {
61             if (error) return reject(mensajeClaro(error));
62             resolve({ destino: resultado?.codeDeliveryDetails?.Destination || correo.trim() });
63         });
64     });
65 }
66
67 /** Reenvía el código de confirmación del registro (botón "Resend code"). */
68 export function reenviarCodigoRegistro(usuario) {
69     return new Promise((resolve, reject) => {
70         usuarioCognito(usuario).resendConfirmationCode((error, resultado) => {
71             if (error) return reject(mensajeClaro(error));
72             resolve({ destino: resultado?.CodeDeliveryDetails?.Destination || "" });
73         });
74     });
75 }
76
77 /** Confirma el código de 6 dígitos que Cognito manda por correo al
78     registrarse - solo después de esto la cuenta puede iniciar sesión. */
79 export function confirmarRegistro(usuario, codigo) {
80     return new Promise((resolve, reject) => {
81         usuarioCognito(usuario).confirmRegistration(codigo, true, (error, resultado) => {
82             // Si la cuenta YA estaba confirmada, no es un fallo: el objetivo de
83             // esta función es "que quede confirmada", y ya lo está. Pasa de
84             // verdad cuando el código se confirmó bien pero el paso siguiente
85             // (crear la fila local) se cayó - al reintentar, Cognito responde
86             // NotAuthorizedException, que mensajeClaro traduce a "Usuario o clave
87             // incorrectos" y deja a la persona atascada sin salida, culpándola de
88             // un dato que escribió bien. Se trata como exitoso y el flujo sigue.
89             if (error) {
90                 const yaConfirmada = error.code === "NotAuthorizedException"
91                     && /current status is confirmed/i.test(error.message || "");
92                 if (yaConfirmada) return resolve({ yaEstaba: true });

```

```

93     return reject(mensajeClaro(error));
94   }
95   resolve(resultado);
96 });
97 });
98 }
99
100 /** Nombre completo y correo tal cual quedaron en Cognito para la cuenta
101 YA AUTENTICADA (sesión recién abierta) - se usa para completar el
102 registro local (/api/registro-completado) cuando esos datos no
103 vienen ya en memoria (p. ej. alguien que se registró, cerró la
104 pestaña antes de confirmar el código, y vuelve después: en ese
105 camino nunca se volvió a escribir el nombre/correo en el
106 formulario, así que se recuperan de Cognito en vez de mandar
107 campos vacíos). */
108 export function obtenerAtributosUsuario() {
109   return new Promise((resolve, reject) => {
110     const cu = pool.getCurrentUser();
111     if (!cu) return reject(new Error("No hay una sesión activa."));
112     cu.getSession((error) => {
113       if (error) return reject(mensajeClaro(error));
114       cu.getUserAttributes((error2, atributos) => {
115         if (error2) return reject(mensajeClaro(error2));
116         const porNombre = Object.fromEntries(
117           atributos || []).map(a => [a.getName(), a.getValue()]);
118         resolve({ nombreCompleto: porNombre.name || "", correo: porNombre.email || "" });
119       });
120     });
121   });
122 }
123
124 /** Inicia sesión. Si la cuenta tiene activada la verificación en dos
125 pasos (Mi perfil > Verificación en dos pasos - es opcional, no todos
126 la tienen), Cognito no da el token de una vez sino que pide el código
127 de la app autenticadora: devuelve { mfaPendiente } en vez de { token },
128 y ese CognitoUser en curso hay que pasárselo tal cual a
129 confirmarCodigoMFA() para terminar el ingreso (no se puede reconstruir
130 solo con el usuario/clave: Cognito ata el reto a esa sesión de login
131 específica). Con el token, el refresh token queda guardado por el
132 propio SDK en localStorage, con su propio ciclo de refresco automático
133 (ver obtenerTokenValido). */
134 export function iniciarSesion(usuario, clave) {
135   return new Promise((resolve, reject) => {
136     const cu = usuarioCognito(usuario);
137     const detalles = new AuthenticationDetails({
138       Username: usuario.trim().toLowerCase(), Password: clave,
139     });
140     cu.authenticateUser(detalles, {
141       onSuccess: (sesion) => resolve({ token: sesion.getAccessToken().getJwtToken() }),
142       onFailure: (error) => reject(mensajeClaro(error)),
143       totpRequired: () => resolve({ mfaPendiente: cu }),
144     });
145   });
146 }
147
148 /** Termina un ingreso que quedó pendiente de código de verificación en
149 dos pasos (ver iniciarSesion). */
150 export function confirmarCodigoMFA(mfaPendiente, codigo) {
151   return new Promise((resolve, reject) => {
152     mfaPendiente.sendMFACode(codigo, {
153       onSuccess: (sesion) => resolve(sesion.getAccessToken().getJwtToken()),
154       onFailure: (error) => reject(mensajeClaro(error)),
155     }, "SOFTWARE_TOKEN_MFA");
156   });
157 }
158
159 /** Pide el código de recuperación de clave, mandado al correo registrado.
160 Devuelve el destino enmascarado que calcula Cognito (p***@m***). */
161 export function recuperarClave(usuario) {
162   return new Promise((resolve, reject) => {
163     usuarioCognito(usuario).forgotPassword({
164       inputVerificationCode: (datos) => resolve({ destino: datos?.CodeDeliveryDetails?.Destination || "" }),
165       onSuccess: () => resolve({ destino: "" }),
166       onFailure: (error) => reject(mensajeClaro(error)),
167     });
168   });
169 }

```

```

167     });
168   });
169 }
170
171 /** Confirma el código de recuperación con la clave nueva. */
172 export function confirmarNuevaClave(usuario, codigo, claveNueva) {
173   return new Promise((resolve, reject) => {
174     usuarioCognito(usuario).confirmPassword(codigo, claveNueva, {
175       onSuccess: () => resolve(),
176       onFailure: (error) => reject(mensajeClaro(error)),
177     });
178   });
179 }
180
181 /** Cambia la clave con la sesión ya abierta (Mi perfil) - exige la clave
182     vigente, igual que el cambio de PIN de antes. */
183 export function cambiarClave(claveActual, claveNueva) {
184   return new Promise((resolve, reject) => {
185     const cu = pool.getCurrentUser();
186     if (!cu) return reject(new Error("No hay una sesión activa."));
187     cu.getSession((error) => {
188       if (error) return reject(mensajeClaro(error));
189       cu.changePassword(claveActual, claveNueva, (error2) => {
190         if (error2) return reject(mensajeClaro(error2));
191         resolve();
192       });
193     });
194   });
195 }
196
197 /** Si la persona ya autenticada tiene activa la verificación en dos pasos
198     (TOTP) - para saber qué mostrar en Mi perfil: "activar" o "desactivar". */
199 export function tieneMFAActiva() {
200   return new Promise((resolve, reject) => {
201     const cu = pool.getCurrentUser();
202     if (!cu) return reject(new Error("No hay una sesión activa."));
203     cu.getSession((error) => {
204       if (error) return reject(mensajeClaro(error));
205       cu.getUserData((error2, datos) => {
206         if (error2) return reject(mensajeClaro(error2));
207         resolve((datos?.UserMFASettingList || []).includes("SOFTWARE_TOKEN_MFA"));
208       }, { bypassCache: true });
209     });
210   });
211 }
212
213 /** Primer paso de activar la verificación en dos pasos: le pide a Cognito
214     una llave secreta nueva para esta cuenta (para el QR/la llave manual)
215     - todavía no queda activada, eso pasa en confirmarActivacionMFA(). */
216 export function iniciarActivacionMFA() {
217   return new Promise((resolve, reject) => {
218     const cu = pool.getCurrentUser();
219     if (!cu) return reject(new Error("No hay una sesión activa."));
220     cu.getSession((error) => {
221       if (error) return reject(mensajeClaro(error));
222       cu.associateSoftwareToken({
223         associateSecretCode: (llave) => resolve(llave),
224         onFailure: (error2) => reject(mensajeClaro(error2)),
225       });
226     });
227   });
228 }
229
230 /** Segundo paso: confirma con el código de 6 dígitos que generó la app
231     autenticadora a partir de la llave, y de una vez la marca como el
232     método de verificación en dos pasos de la cuenta. */
233 export function confirmarActivacionMFA(codigo) {
234   return new Promise((resolve, reject) => {
235     const cu = pool.getCurrentUser();
236     if (!cu) return reject(new Error("No hay una sesión activa."));
237     // Sin getSession() primero, cu.signInUserSession queda null (recién
238     // construido por getCurrentUser(), no hidratado desde el localStorage
239     // del SDK) - verifySoftwareToken() lo interpreta como una sesion de
240     // RETO en curso (usa this.Session, que aqui no existe) en vez de

```

```

241 // usuario-ya-autenticado (usa el AccessToken), y Cognito responde
242 // NotAuthorizedException en vez de validar el código.
243 cu.getSession((error0) => {
244   if (error0) return reject(mensajeClaro(error0));
245   cu.verifySoftwareToken(codigo, "CuentaVoz", {
246     onSuccess: () => {
247       cu.setUserMfaPreference(null, { Enabled: true, PreferredMfa: true }, (error) => {
248         if (error) return reject(mensajeClaro(error));
249         resolve();
250       });
251     },
252     onFailure: (error) => reject(mensajeClaro(error)),
253   });
254 });
255 });
256 }
257
258 /** Desactiva la verificación en dos pasos para esta cuenta. */
259 export function desactivarMFA() {
260   return new Promise((resolve, reject) => {
261     const cu = pool.getCurrentUser();
262     if (!cu) return reject(new Error("No hay una sesión activa."));
263     cu.getSession((error) => {
264       if (error) return reject(mensajeClaro(error));
265       cu.setUserMfaPreference(null, { Enabled: false, PreferredMfa: false }, (error2) => {
266         if (error2) return reject(mensajeClaro(error2));
267         resolve();
268       });
269     });
270   });
271 }
272
273 /** El URI estándar (otppath://) que cualquier app autenticadora (Google
274   Authenticator, Authy, etc.) sabe leer desde un QR - MiPerfil.jsx lo
275   convierte en imagen con la librería "qrcode". */
276 export function uriOtpAuth(llaveSecreta, usuario) {
277   const etiqueta = encodeURIComponent(`CuentaVoz:${usuario}`);
278   return `otppath://totp/${etiqueta}?secret=${llaveSecreta}&issuer=CuentaVoz`;
279 }
280
281 /** Access token vigente del usuario ya autenticado, refrescándolo solo
282   con el refresh token si ya venció (el SDK lo hace por su cuenta al
283   llamar getSession, sin pedirle nada a la persona) - así el access
284   token corto (1 hora, ver ARQUITECTURA.md) no obliga a reingresar cada
285   hora. Devuelve null si no hay sesión (nunca ingresó o el refresh token
286   también venció, a los 30 días). */
287 export function obtenerTokenValido() {
288   return new Promise((resolve) => {
289     const cu = pool.getCurrentUser();
290     if (!cu) return resolve(null);
291     cu.getSession((error, sesion) => {
292       if (error || !sesion?.isValid()) return resolve(null);
293       resolve(sesion.getAccessToken().getJwtToken());
294     });
295   });
296 }
297
298 /** Cierra la sesión guardada por el SDK en ESTE navegador (no revoca nada
299   en Cognito - para eso está "cerrar todas las sesiones" en Mi perfil,
300   que sí llama al backend). */
301 export function cerrarSesionLocal() {
302   pool.getCurrentUser()?.signOut();
303 }

```

frontend/src/voz.js

258 LÍNEAS

Escuchar y hablar.

```

1 /** Escuchar y hablar. Las dos piezas de la capa de voz. */
2 import { BASE, leerToken } from "../api";
3 // quitarTildes/esAfirmacion/esNegacion viven en su propio módulo sin
4 // dependencias (ni de "../api" ni del navegador) para poder probarlas con

```

```

5 // node --test igual que interpreteLocal.js - se reexportan aquí para que
6 // quien ya las importaba desde "./voz" siga funcionando igual.
7 export { quitarTildes, esAfirmacion, esNegacion } from "./confirmacionVoz.js";
8
9 const IDIOMA_ESCUCHA = "es-CO"; // como reconoce lo que usted dice: fijo
10 let vozNeuronal = "kore"; // que voz neuronal usa CuentaVoz al hablar: elegible
11 const TASA = { lenta: 0.82, normal: 1.02, rapida: 1.3 };
12 let tasaActual = TASA.normal;
13 let confirmacionHablada = true;
14
15 // A que genero suena cada voz neuronal - para que, si la neuronal falla y
16 // hay que caer al respaldo del navegador, se busque una voz de navegador
17 // del mismo genero en vez de una elegida al azar. No es la MISMA voz (eso
18 // no se puede: el navegador no tiene "Kore"), pero al menos no suena como
19 // si de repente le estuviera hablando otra persona.
20 const _GENERO_VOZ = { kore: "femenina", aoede: "femenina", puck: "masculina", charon: "masculina" };
21
22 const Reconocedor =
23   typeof window !== "undefined" &&
24   (window.SpeechRecognition || window.webkitSpeechRecognition);
25
26 export const vozDisponible = () => Boolean(Reconocedor);
27
28 /** MiPerfil llama esto una vez cargadas (o cambiadas) las preferencias.
29   "idioma" quedo con ese nombre por compatibilidad con quien ya llama
30   configurarVoz(), pero ahora guarda la clave de la voz neuronal
31   (kore, puck...), no un codigo de idioma de navegador. */
32 export function configurarVoz({ idioma, velocidad, confirmacionHablada: ch } = {}) {
33   if (idioma && idioma !== vozNeuronal) {
34     vozNeuronal = idioma;
35     // la voz de respaldo se habia elegido (o cacheado como "no hay") con
36     // la preferencia VIEJA - sin esto, cambiar la voz en Mi perfil no
37     // cambiaba nada si en algun momento se habia caido al respaldo antes.
38     vozNavegadorBuscada = false;
39     vozNavegadorElegida = null;
40   }
41   if (velocidad && TASA[velocidad]) tasaActual = TASA[velocidad];
42   if (ch !== undefined) confirmacionHablada = ch;
43 }
44
45 /* — Respaldo: voz del navegador (speechSynthesis) —
46 Solo se usa si la voz neuronal no responde (sin internet, sin llave de
47 Gemini configurada, o la API falla) - para que el conteo por voz nunca
48 se quede completamente mudo. Suena mas robotica, pero es mejor que
49 nada mientras se recupera la conexion. */
50 let vozNavegadorElegida = null;
51 let vozNavegadorBuscada = false;
52
53 // Nombres tipicos de voces de navegador/SO en espanol, para adivinar de
54 // que genero suena cada una - el estandar Web Speech API no expone genero,
55 // asi que esto es lo mas cercano a saberlo sin reproducirla.
56 const _NOMBRES_FEM =
/sabina|helenah|elvira|laura|m[ó]nica|paulina|luc[i]a|elena|isabela|marisol|conchita|pen[eé]lope|esperanza|camila|valentina
|female|mujer/;
57 const _NOMBRES_MASC = /ra[uú]l|jorge|diego|pablo|carlos|alonso|enrique|miguel|male|hombre/;
58
59 function _puntuarVozNavegador(v, generoPreferido) {
60   const lang = (v.lang || "").toLowerCase();
61   const nombre = (v.name || "").toLowerCase();
62   let p = 0;
63   if (lang.startsWith("es-mx") || lang.startsWith("es-419")) p += 20;
64   else if (lang.startsWith("es-")) p += 10;
65   else if (lang.startsWith("es-")) p += 5;
66   else return -1;
67   if (/natural|online|neural|wavenet|premium/.test(nombre)) p += 30;
68   if (/google/.test(nombre)) p += 10;
69   if (/desktop|compact/.test(nombre)) p -= 20;
70   // que suene del mismo genero que la voz neuronal elegida en Mi perfil
71   // pesa mas que cualquier otra cosa: es justo lo que hace notorio el
72   // cambio de voz cuando toca caer al respaldo.
73   if (generoPreferido === "femenina" && _NOMBRES_FEM.test(nombre)) p += 50;
74   else if (generoPreferido === "masculina" && _NOMBRES_MASC.test(nombre)) p += 50;
75   else if (generoPreferido === "femenina" && _NOMBRES_MASC.test(nombre)) p -= 50;
76   else if (generoPreferido === "masculina" && _NOMBRES_FEM.test(nombre)) p -= 50;

```

```

77   return p;
78 }
79
80 function _elegirVozNavegador() {
81   if (typeof window === "undefined" || !window.speechSynthesis) return null;
82   const voces = window.speechSynthesis.getVoices();
83   if (!voces.length) return null;
84   const generoPreferido = _GENERO_VOZ[vozNeuronal];
85   const candidatas = voces.map((v) => ({ v, p: _puntuarVozNavegador(v, generoPreferido) })))
86     .filter((x) => x.p >= 0).sort((a, b) => b.p - a.p);
87   return candidatas[0]?.v || null;
88 }
89
90 /** Devuelve una promesa que se resuelve cuando termina de decirlo (no
91     cuando empieza) - quien vaya a escuchar() justo después de hablar()
92     necesita esperar a que la pregunta se termine de decir, o el
93     micrófono arranca mientras el navegador todavía está sonando y se
94     pierde la respuesta. */
95 function hablarConNavegador(texto) {
96   return new Promise((resolve) => {
97     if (typeof window === "undefined" || !window.speechSynthesis) { resolve(); return; }
98     if (!vozNavegadorBuscada) {
99       vozNavegadorElegida = _elegirVozNavegador();
100      if (!vozNavegadorElegida) {
101        window.speechSynthesis.onvoiceschanged = () => { vozNavegadorElegida = _elegirVozNavegador(); };
102      } else {
103        vozNavegadorBuscada = true;
104      }
105    }
106    window.speechSynthesis.cancel();
107    const u = new SpeechSynthesisUtterance(texto);
108    u.lang = vozNavegadorElegida?.lang || "es-MX";
109    u.rate = tasaActual;
110    u.pitch = 1.03;
111    if (vozNavegadorElegida) u.voice = vozNavegadorElegida;
112    u.onend = resolve;
113    u.onerror = resolve;
114    window.speechSynthesis.speak(u);
115  });
116 }
117
118 /* — Voz neuronal real (Gemini TTS, via el backend) — */
119 let audioActual = null;
120
121 // La síntesis real tarda unos segundos (viaja al backend y a Gemini): si
122 // mientras tanto la persona ya cambió de pantalla, esa respuesta ya no
123 // tiene contexto y no debe hablar sola de repente sobre una pantalla que
124 // ya no está en uso. Cada hablar() (y detenerVoz()) sube esta generación;
125 // una respuesta que llega tarde, de una generación vieja, se descarta en
126 // vez de reproducirse.
127 let generacion = 0;
128
129 // Si detenerVoz() corta el audio a la mitad (pause() no dispara "ended"),
130 // esto es lo que despierta a quien esté esperando hablar() - si no, un
131 // cambio de pantalla a mitad de frase dejaría esa promesa colgada para
132 // siempre.
133 let resolverAudioActual = null;
134
135 /** App.jsx la llama en cada cambio de pantalla: corta lo que se esté
136     reproduciendo y invalida cualquier hablar() todavía en camino. */
137 export function detenerVoz() {
138   generacion++;
139   if (audioActual) { audioActual.pause(); audioActual.currentTime = 0; }
140   if (typeof window !== "undefined" && window.speechSynthesis) window.speechSynthesis.cancel();
141   resolverAudioActual?.();
142   resolverAudioActual = null;
143 }
144
145 async function hablarConNeuronal(texto, miGeneracion) {
146   const res = await fetch(`${BASE}/api/voz/hablar`, {
147     method: "POST",
148     headers: {
149       "Content-Type": "application/json",
150       Authorization: `Bearer ${leerToken()}`,

```



```

151     },
152     body: JSON.stringify({ texto, voz: vozNeuronal })),
153   });
154   if (!res.ok) throw new Error("voz neuronal no disponible");
155   const blob = await res.blob();
156   if (miGeneracion !== generacion) return; // se cambió de pantalla mientras se generaba
157   const url = URL.createObjectURL(blob);
158   if (audioActual) { audioActual.pause(); audioActual.currentTime = 0; }
159   const audio = new Audio(url);
160   audio.playbackRate = tasaActual;
161   audioActual = audio;
162   // Espera a que el audio TERMINE de sonar, no solo a que arranque -
163   // audio.play() por sí solo se resuelve apenas empieza a reproducir, y
164   // quien llama a hablar() esperando poder escuchar() justo después
165   // necesita que la pregunta ya se haya terminado de decir.
166   await new Promise((resolve) => {
167     resolverAudioActual = resolve;
168     audio.addEventListener("ended", resolve, { once: true });
169     audio.addEventListener("error", resolve, { once: true });
170     audio.play().catch(resolve);
171   });
172   resolverAudioActual = null;
173   URL.revokeObjectURL(url);
174 }
175
176 // Tope de seguridad: si el navegador alguna vez no dispara "ended"/"onend"
177 // (se ha visto en algunos Chrome tras minimizar la pestaña, y en entornos
178 // sin salida de audio real), hablar() quedaría esperando para siempre y el
179 // micrófono que depende de "await hablar()" nunca llegaría a abrirse.
180 //
181 // Un numero fijo no alcanza: "Hay varias: <hasta 6 bodegas>. ¿Cuál de
182 // todas?" (Conteo.jsx, cuando el nombre dictado es ambiguo) puede pasar
183 // facil los 200 caracteres con nombres reales del catalogo - a un ritmo
184 // de lectura normal eso son 15-20 segundos, mas que un limite fijo de 12s
185 // pensado para frases cortas. Con el limite fijo, esa frase se cortaba a
186 // medias: la promesa de hablar() se resolvía con el audio TODAVIA
187 // sonando, y el microfono que se abre justo despues terminaba
188 // "escuchando" la cola de la propia voz del agente como si fuera la
189 // respuesta de la persona. El tope ahora escala con el largo del texto
190 // (y con que tan lenta este la voz elegida en Mi perfil), con un piso
191 // para frases cortas y un techo para no esperar para siempre si el audio
192 // de verdad se atasca.
193 const TOPE_HABLAR_MS_PISO = 12000;
194 const TOPE_HABLAR_MS_TECHO = 40000;
195
196 function _topeHablar(texto) {
197   // ~12 caracteres por segundo es una lectura pausada y generosa (cubre
198   // voces mas lentas que el promedio); /tasaActual porque una voz "lenta"
199   // (0.82x) de verdad tarda mas en decir lo mismo que una "rapida" (1.3x).
200   const estimado = (texto.length / 12) * 1000 / tasaActual;
201   return Math.min(TOPE_HABLAR_MS_TECHO, Math.max(TOPE_HABLAR_MS_PISO, estimado));
202 }
203
204 /** Habla el texto y devuelve una promesa que se resuelve cuando termina
205   de decirlo (o, como mucho, al tope de seguridad calculado para ese
206   texto). Casi todo el código la llama "al aire" (sin await) y eso sigue
207   funcionando igual; quien necesite escuchar() justo después de
208   preguntar algo sí debe esperarla, o el micrófono arranca mientras
209   todavía se está hablando y se pierde parte de la respuesta. */
210 export function hablar(texto, { forzar = false } = {}) {
211   if (!texto) return Promise.resolve();
212   if (!forzar && !confirmacionHablada) return Promise.resolve();
213   generacion++;
214   const miGeneracion = generacion;
215   const intento = hablarConNeuronal(texto, miGeneracion)
216     .catch(() => { if (miGeneracion === generacion) return hablarConNavegador(texto); });
217   return Promise.race([
218     intento,
219     new Promise((resolve) => setTimeout(resolve, _topeHablar(texto))),
220   ]);
221 }
222
223 /** Escucha una frase y la devuelve como texto. */
224 export function escuchar({ alTexto, alEstado, alError }) {

```

```

225   if (!Reconocedor) {
226       alError?.("Este navegador no reconoce voz. Use Chrome o Edge.");
227       return null;
228   }
229   const rec = new Reconocedor();
230   rec.lang = IDIOMA_ESCUCHA;
231   rec.interimResults = true;
232   rec.continuous = false;
233   rec.maxAlternatives = 1;
234
235   rec.onstart = () => alEstado?.("escuchando");
236   rec.onresult = (e) => {
237       const r = e.results[e.results.length - 1];
238       const texto = r[0].transcript.trim();
239       if (r.isFinal) {
240           alEstado?.("procesando");
241           alTexto?.(texto);
242       } else {
243           alEstado?.("escuchando", texto);
244       }
245   };
246   rec.onerror = (e) => {
247       alEstado?.("listo");
248       if (e.error === "not-allowed")
249           alError?.("Debe permitir el micrófono en el navegador.");
250       else if (e.error !== "aborted" && e.error !== "no-speech")
251           alError?.(`No pude escuchar (${e.error}).`);
252   };
253   rec.onend = () => alEstado?.("listo");
254   try {
255       rec.start();
256   } catch (_) {}
257   return rec;
258 }

```

frontend/src/interpreteLocal.js

148 LÍNEAS

El mismo interprete del backend, pero dentro del navegador, para el modo sin conexion.

```

1  /** Puerto a JavaScript de backend/servicios/interprete.py: el mismo
2   reconocimiento de números en palabras e intenciones frecuentes, pero
3   corriendo en el navegador en vez del servidor - para que el modo sin
4   conexión entienda texto escrito de verdad (no solo un formulario
5   numérico) incluso con la red completamente apagada, cero llamadas al
6   backend. Mantener esto en sincronía con interprete.py a mano: no hay
7   una fuente única compartida entre Python y JS todavía.
8
9   No reemplaza a Gemini ni al intérprete del servidor: es el mismo
10  respaldo de última instancia, ahora también disponible sin backend. */
11
12  const NUMEROS = new Map(Object.entries({
13      cero: 0, un: 1, una: 1, uno: 1, dos: 2, tres: 3, cuatro: 4,
14      cinco: 5, seis: 6, siete: 7, ocho: 8, nueve: 9, diez: 10,
15      once: 11, doce: 12, trece: 13, catorce: 14, quince: 15,
16      dieciseis: 16, "dieciséis": 16, diecisiete: 17, dieciocho: 18,
17      diecinueve: 19, veinte: 20, veinticinco: 25, treinta: 30,
18      cuarenta: 40, cincuenta: 50, sesenta: 60, setenta: 70,
19      ochenta: 80, noventa: 90, cien: 100, ciento: 100,
20      doscientos: 200, quinientos: 500, mil: 1000,
21  }));
22
23  const UNIDADES = new Map(Object.entries({
24      kilo: "Kilogram", kilos: "Kilogram", kilogramo: "Kilogram",
25      kilogramos: "Kilogram", kg: "Kilogram",
26      litro: "Liter", litros: "Liter", l: "Liter",
27      unidad: "Unidad", unidades: "Unidad",
28      porcion: "Portion", porciones: "Portion", "porción": "Portion",
29  }));
30
31  const DECENAS = [20, 30, 40, 50, 60, 70, 80, 90];
32  const CANONICAS = new Set(["Kilogram", "Liter", "Unidad", "Portion"]);

```

```

33
34 const RE_NUMERO_DIGITOS = /-?\d+(?:[.,]\d+)?/;
35 const RE_PUNTUACION_BORDE = /^[.,,¿?¡!]+|[.,,¿?¡!]+$/g;
36
37 function quitarPuntuacion(palabra) {
38   return palabra.replace(RE_PUNTUACION_BORDE, "");
39 }
40
41 /** La PRIMERA cantidad de la frase. Ignora números que son parte del
42     nombre del producto («cazuela 16 onzas», «tabla 50x38»). */
43 function numero(texto) {
44   const t = texto.toLowerCase();
45   const m = RE_NUMERO_DIGITOS.exec(t);
46   if (m) {
47     const val = parseFloat(m[0].replace(",", "."));
48     return t.slice(0, m.index).includes("menos") ? -val : val;
49   }
50
51   const palabras = t.trim().split(/\s+/).filter(Boolean).map(quitarPuntuacion);
52   let total = null;
53   for (let i = 0; i < palabras.length; i++) {
54     const p = palabras[i];
55     if (!NUMEROS.has(p)) continue;
56     const v = NUMEROS.get(p);
57     total = v;
58     // «ciento ochenta»: centena seguida de decena
59     if (total >= 100 && i + 1 < palabras.length && NUMEROS.has(palabras[i + 1])) {
60       const sig = NUMEROS.get(palabras[i + 1]);
61       if (sig < 100) total += sig;
62       // «treinta y cinco»: decena + y + unidad
63     } else if (DECENAS.includes(total) && i + 2 < palabras.length
64               && palabras[i + 1] === "y" && NUMEROS.has(palabras[i + 2])) {
65       total += NUMEROS.get(palabras[i + 2]);
66     }
67     break; // lo demas pertenece al nombre del producto
68   }
69   if (total !== null && t.includes("menos")) total = -Math.abs(total);
70   return total;
71 }
72
73 function unidad(texto) {
74   for (let p of texto.toLowerCase().replace(/\?/g, " ").trim().split(/\s+/)) {
75     p = p.replace(/^[.,,]+|[.,,]+$/g, "");
76     if (UNIDADES.has(p)) return UNIDADES.get(p);
77   }
78   return null;
79 }
80
81 /** Lo que sea que devuelva el agente (humano o el modelo) para la unidad,
82     lo lleva a uno de los 4 valores canónicos de la base de datos. */
83 export function normalizarUnidad(dicha) {
84   if (!dicha) return dicha;
85   if (CANONICAS.has(dicha)) return dicha;
86   const clave = String(dicha).trim().toLowerCase();
87   return UNIDADES.has(clave) ? UNIDADES.get(clave) : dicha;
88 }
89
90 const PALABRAS_FUERA = new Set([
91   ...NUMEROS.keys(), ...UNIDADES.keys(),
92   "de", "del", "la", "el", "los", "las", "hay", "en", "y", "por",
93   "confirmo", "confirmar", "son", "hoy", "preparamos", "quiero",
94   "pedir", "insumos", "para", "cuanto", "cuánto", "cuantos",
95   "iniciar", "conteo", "abrir", "bodega", "corregir", "ultimo",
96   "último", "no", "uy", "que", "qué", "me", "genera", "generame",
97   "consolidado", "reporte", "ayuda", "menos", "onzas", "onza",
98 ]);
99
100 function sinRuido(t) {
101   const sinDigitos = t.replace(RE_NUMERO_DIGITOS, " ");
102   return sinDigitos.trim().split(/\s+/).filter(Boolean)
103     .filter((w) => !PALABRAS_FUERA.has(quitarPuntuacion(w)))
104     .join(" ").trim();
105 }
106

```

```

107 const contiene = (f, claves) => claves.some((k) => f.includes(k));
108
109 /** El mismo respaldo de última instancia que interprete.py, corriendo en
110     el navegador: entiende lo básico sin backend ni internet. */
111 export function interpretarLocal(frase) {
112     const f = frase.toLowerCase().trim();
113     const cant = numero(f);
114     const uni = unidad(f);
115
116     if (contiene(f, ["iniciar conteo", "abrir bodega", "abrir la bodega"])) {
117         return { intencion: "navegar", bodega_texto: sinRuido(f),
118             respuesta_hablada: "Voy a abrir esa bodega." };
119     }
120     if (contiene(f, ["preparamos", "vamos a preparar", "pedir insumos"])) {
121         return { intencion: "pedir", preparacion: sinRuido(f),
122             porciones: cant ? Math.trunc(cant) : 0,
123             respuesta_hablada: "Le calculo los insumos según la receta." };
124     }
125     if (f.startsWith("confirmo") || ["confirmar", "si", "sí", "correcto", "confirmo."].includes(f)) {
126         return { intencion: "confirmar", respuesta_hablada: "Listo." };
127     }
128     if (contiene(f, ["corregir", "no son", "no, son", "uy no", "uy, no",
129         "esta mal", "está mal", "me equivoque", "cambiar"])) {
130         return { intencion: "corregir", cantidad: cant, unidad: uni,
131             respuesta_hablada: "Corrijo ese registro." };
132     }
133     if (contiene(f, ["cuanto", "cuánto", "cuantos", "donde hay", "dónde hay"])) {
134         return { intencion: "consultar", articulo_texto: sinRuido(f),
135             respuesta_hablada: "Reviso el inventario." };
136     }
137     if (contiene(f, ["reporte", "consolidado", "archivo", "imprim"])) {
138         return { intencion: "reporte", respuesta_hablada: "Genero el archivo." };
139     }
140     if (f.includes("ayuda") || f.includes("no entiendo")) {
141         return { intencion: "ayuda", respuesta_hablada: "Con gusto. ¿Qué necesita saber?" };
142     }
143     if (cant !== null) {
144         return { intencion: "contar", articulo_texto: sinRuido(f),
145             cantidad: cant, unidad: uni, respuesta_hablada: "" };
146     }
147     return { intencion: "ayuda", respuesta_hablada: "Perdón, no le entendí bien. ¿Me lo repite?" };
148 }

```

frontend/src/colaOffline.js

17 LÍNEAS

La cola de conteos pendientes de sincronizar.

```

1 // La cola de conteos guardados en este equipo mientras no había señal
2 // (ver Conteo.jsx). Vivía solo dentro de Conteo.jsx - invisible desde
3 // cualquier otra pantalla aunque los ítems seguían pendientes de
4 // sincronizar. Se saca a un módulo aparte para que BarraLateral.jsx
5 // también pueda mostrarla, sin duplicar la lógica.
6 export const CLAVE_COLA = (sesionId) => `cv_offline_${sesionId}`;
7 export const EVENTO_COLA = "cuentavoz:cola-offline-cambiada";
8
9 export function leerCola(sesionId) {
10     try { return JSON.parse(localStorage.getItem(CLAVE_COLA(sesionId)) || "[]"); }
11     catch (_) { return []; }
12 }
13
14 export function guardarCola(sesionId, cola) {
15     localStorage.setItem(CLAVE_COLA(sesionId), JSON.stringify(cola));
16     window.dispatchEvent(new Event(EVENTO_COLA));
17 }

```

frontend/src/accesibilidad.js

20 LÍNEAS

Alto contraste y tamaño de letra.

```

1 // Preferencias de accesibilidad (alto contraste, tamaño de letra) - a
2 // propósito por dispositivo, no por servidor: son ajustes de cómo se ve la

```

```

3 // pantalla en ESTE equipo, no algo que deba viajar con la cuenta a otro
4 // dispositivo, y así siguen funcionando sin conexión.
5 const CLAVE = "cv_accesibilidad";
6
7 export function leerPreferencias() {
8   try { return JSON.parse(localStorage.getItem(CLAVE) || "{}"); }
9   catch (_) { return {}; }
10 }
11
12 export function aplicarPreferencias(p) {
13   document.documentElement.setAttribute("data-contraste", p.contraste ? "alto" : "");
14   document.documentElement.setAttribute("data-tamano", p.tamano || "");
15 }
16
17 export function guardarPreferencias(p) {
18   localStorage.setItem(CLAVE, JSON.stringify(p));
19   aplicarPreferencias(p);
20 }

```

frontend/src/confirmacionVoz.js

47 LÍNEAS

Que cuenta como un si hablado.

```

1 /** Detectar un "sí"/"no" hablado o escrito - sin depender de nada más
2   (ni del navegador, ni de la API), para poder probarlo con node --test
3   igual que interpreteLocal.js. voz.js reexporta estas mismas funciones
4   para quien ya las importaba desde ahí. */
5
6 /** Sin esto, "sí" (como lo transcribe la voz, con tilde) NUNCA hacía
7   match contra /^(si|sí|...)\b/: en JavaScript \b y \w son ASCII puros,
8   así que "i" no cuenta como letra y la frontera de palabra justo
9   después de la tilde no se reconoce - "sí" (y hasta "sí ver detalle")
10  fallaban en seco, y solo "si" sin tilde funcionaba. Bug real,
11  encontrado en producción, que llevaba viva desde el principio en
12  varios confirmaciones habladas de la app. */
13 export function quitarTildes(t) {
14   return String(t).normalize("NFD").replace(/[\u0300-\u036f]/g, "");
15 }
16
17 const AFIRMACIONES_BASE = ["si", "confirmo", "confirmar", "claro", "dale", "correcto", "listo"];
18 const FRASES_AFIRMATIVAS = ["eso es", "así es"];
19
20 /** ¿La respuesta hablada/escrita es un "sí" a lo que se le preguntó?
21   "extra" agrega palabras válidas solo en ese contexto puntual - p. ej.
22   "ver" cuando la pregunta fue "¿ve el detalle?", o el texto del botón
23   de turno ("Enviar", "Solicitar"... ) para que decir esa misma palabra
24   (en cualquier conjugación: "envíalo", "enviar", "envíelo") confirme
25   igual que tocar el botón. */
26 export function esAfirmacion(texto, extra = []) {
27   const t = quitarTildes(String(texto).toLowerCase().replace(/[,;:!?i]/g, ""));
28   const palabras = t.split(/\s+/).filter(Boolean);
29   if (!palabras.length || palabras.includes("no")) return false;
30   const extrasNorm = extra.map((e) => quitarTildes(e.toLowerCase()));
31   const afirmaciones = [...AFIRMACIONES_BASE, ...extrasNorm];
32   if (palabras.some((p) => afirmaciones.includes(p))) return true;
33   if (FRASES_AFIRMATIVAS.some((f) => t.includes(f))) return true;
34   // conjugaciones del extra ("enviar" -> envía/envíalo/enviando...): se
35   // compara por raíz, quitando la terminación de infinitivo -ar/-er/-ir,
36   // en vez de enumerar cada forma verbal a mano.
37   const raices = extrasNorm.map((e) => e.replace(/(ar|er|ir)$/, "")).filter((r) => r.length >= 3);
38   return raices.length > 0 && palabras.some((p) => raices.some((r) => p.startsWith(r)));
39 }
40
41 /** ¿La respuesta es un "no" limpio? (primera palabra, no en cualquier
42   parte de la frase, para no confundir "no sé, tal vez" con un cierre). */
43 export function esNegacion(texto) {
44   const t = quitarTildes(String(texto).toLowerCase().replace(/[,;:!?i]/g, ""));
45   const palabras = t.split(/\s+/).filter(Boolean);
46   return palabras[0] === "no";
47 }

```

frontend/src/tutorial.js

27 LÍNEAS

Si el recorrido en video se abre solo.

```

1 // Lógica pura de cuándo mostrar el recorrido en video (ver
2 // VideoRecorrido.jsx), separada en un módulo .js aparte porque node:test no
3 // puede parsear JSX directamente - así esta parte sí se prueba, igual que
4 // colaOffline.js/accesibilidad.js.
5 //
6 // El video sale SIEMPRE al entrar, en cada ingreso - no solo la primera
7 // vez. Lo único que lo apaga es que la propia persona lo desactive desde
8 // un control dentro del video (VideoRecorrido.jsx), y esa bandera se
9 // guarda POR USUARIO (cv_tutorial_deshabilitado_<id>), no una sola para
10 // todo el dispositivo: en una bodega una tablet la comparten varios
11 // auxiliares, y una bandera única haría que un auxiliar apagara el video
12 // para todos los demás que usan el mismo equipo.
13 const PREFIJO_CLAVE = "cv_tutorial_deshabilitado_";
14 export const EVENTO_ABRIR_VIDEO = "cuentavoz:abrir-video";
15
16 export function debeAbrirTutorial(usuarioId) {
17   try { return !localStorage.getItem(PREFIJO_CLAVE + usuarioId); }
18   catch (_) { return true; }
19 }
20
21 export function deshabilitarTutorial(usuarioId) {
22   try { localStorage.setItem(PREFIJO_CLAVE + usuarioId, "1"); } catch (_) {}
23 }
24
25 export function habilitarTutorial(usuarioId) {
26   try { localStorage.removeItem(PREFIJO_CLAVE + usuarioId); } catch (_) {}
27 }

```

frontend/src/correoAdmin.js

52 LÍNEAS

El respaldo de envío por correo.

```

1 /** Envío real de correo para "Soporte en vivo" (mensajes y respuestas
2   entre auxiliares y administrador), compartido entre Ayuda.jsx y
3   Mensajes.jsx para no duplicar credenciales ni lógica.
4
5   La Public Key de EmailJS está hecha para ir en el código del
6   frontend (a diferencia de una clave de API común, no es secreta -
7   así lo documenta EmailJS). El correo sale desde el navegador de
8   quien escribe, usando la cuenta de Gmail conectada en el panel de
9   EmailJS - por eso no pasa por la red de Render, que es lo que
10  bloqueaba el SMTP directo a Gmail y la API de Resend. */
11 const EMAILJS_SERVICE_ID = "service_9dgu33i";
12 const EMAILJS_TEMPLATE_ID = "template_9ddkqpa";
13 const EMAILJS_PUBLIC_KEY = "raCRzjMA9m2ymYuwk";
14
15 export async function enviarPorEmailJS(destinatario, nombreRemitente, mensaje, rol, correoRemitente) {
16   try {
17     const res = await fetch("https://api.emailjs.com/api/v1.0/email/send", {
18       method: "POST",
19       headers: { "Content-Type": "application/json" },
20       body: JSON.stringify({
21         service_id: EMAILJS_SERVICE_ID,
22         template_id: EMAILJS_TEMPLATE_ID,
23         user_id: EMAILJS_PUBLIC_KEY,
24         template_params: {
25           to_email: destinatario, name: nombreRemitente, message: mensaje,
26           rol: rol || "", correo_remitente: correoRemitente || "",
27           fecha: new Date().toLocaleString("es-CO", {
28             day: "numeric", month: "long", year: "numeric", hour: "numeric", minute: "2-digit",
29           }),
30         },
31       })),
32     });
33     return res.ok;
34   } catch {
35     return false;
36   }
37 }

```

```

36  }
37  }
38
39  // admin.nombre y usuario.nombre llegan en minúscula (el resto de la app
40  // los capitaliza con CSS) - para texto plano (firma del correo, mensajes
41  // de la bandeja) hay que capitalizarlos en JS.
42  export function capitalizar(nombre) {
43    if (!nombre) return "";
44    return nombre.split(" ").map((p) => p.charAt(0).toUpperCase() + p.slice(1)).join(" ");
45  }
46
47  // Misma etiqueta que usa el resto de la app (BarraLateral, MiPerfil,
48  // Ingreso) para el rol - así la firma del correo y la bandeja dicen lo
49  // mismo que ve la persona dentro de CuentaVoz.
50  export function rolDe(usuario) {
51    return usuario?.perfil === "auditor" ? "Administrador de bodega" : "Auxiliar de inventarios";
52  }

```

17.5 Frontend · componentes compartidos

frontend/src/BarraLateral.jsx

111 LÍNEAS

El menu lateral, el avatar y el aviso de cola sin conexion.

```

1  import { useEffect, useState } from "react";
2  import { MENU } from "../App";
3  import Icono from "../Iconos";
4  import { BASE, leerToken } from "../api";
5  import { leerCola, EVENTO_COLA } from "../colaOffline";
6
7  export default function BarraLateral({ activo, usuario, token, sesionId, onNavegar }) {
8    const esAuditor = usuario?.perfil === "auditor";
9    const items = MENU.filter((m) => !m.soloAuditor || esAuditor);
10   const [fotoUrl, setFotoUrl] = useState(null);
11   const [colaLen, setColaLen] = useState(0);
12
13   // La cola de conteos guardados sin conexión (ver Conteo.jsx/colaOffline.js)
14   // antes solo se veía dentro de la pantalla de Conteo - si alguien
15   // cambiaba de pantalla con ítems todavía pendientes, no había ningún
16   // rastro de eso en el resto de la app. El evento propio cubre cambios en
17   // esta misma pestaña; "storage" cubre otra pestaña sincronizando la cola.
18   useEffect(() => {
19     function actualizar() { setColaLen(leerCola(sesionId).length); }
20     actualizar();
21     window.addEventListener(EVENTO_COLA, actualizar);
22     window.addEventListener("storage", actualizar);
23     return () => {
24       window.removeEventListener(EVENTO_COLA, actualizar);
25       window.removeEventListener("storage", actualizar);
26     };
27   }, [sesionId]);
28
29   // Igual que en Mi perfil: <img src> no puede mandar el header
30   // Authorization que /foto exige, así que se trae como blob autenticado.
31   // Se vuelve a pedir cuando Mi perfil avisa que hubo una foto nueva.
32   useEffect(() => {
33     function cargar() {
34       fetch(`${BASE}/api/usuarios/yo/foto`, {
35         headers: { Authorization: `Bearer ${token || leerToken()}` },
36       }).then((r) => (r.ok ? r.blob() : null))
37         .then((b) => setFotoUrl((prev) => {
38           if (prev) URL.revokeObjectURL(prev);
39           return b ? URL.createObjectURL(b) : null;
40         })))
41       // mismo camino que el then() de arriba (revoca la URL anterior
42       // antes de reemplazarla) - un fallo de red aquí no debe quedarse
43       // con el blob viejo vivo para siempre solo porque este intento
44       // en particular no llego a nada.
45       .catch(() => setFotoUrl((prev) => { if (prev) URL.revokeObjectURL(prev); return null; }));
46     }

```

```

47   cargar();
48   window.addEventListener("cuentavoz:foto-actualizada", cargar);
49   return () => window.removeEventListener("cuentavoz:foto-actualizada", cargar);
50 }, [token]);
51
52 return (
53   <nav className="sidebar" aria-label="Menú principal">
54     <div className="marca">
55       
56       <span>
57         <b>CuentaVoz</b>
58         <i>Tu voz cuenta</i>
59       </span>
60     </div>
61     <hr />
62
63     /* El botón va DENTRO del <li>, no en su lugar. Cuando el rol de
64     botón estaba sobre el <li>, ese <li> dejaba de contar como
65     elemento de lista y el menú se anunciaba como una lista vacía:
66     se perdía el «3 de 13», que es lo que ubica a quien no ve la
67     pantalla. Con un <button> de verdad, además, Enter y Espacio
68     funcionan solos y sobra el onKeyDown que había a mano. */
69     <ul>
70       {items.map((m) => (
71         <li key={m.id}>
72           <button
73             type="button"
74             className={m.id === activo ? "sel" : ""}
75             onClick={() => onNavegar(m.id)}
76             aria-current={m.id === activo ? "page" : undefined}
77           >
78             <span className="icono-menu">
79               <Icono nombre={m.id} tam={19} />
80             </span>
81             <span>{m.titulo}</span>
82           </button>
83         </li>
84       ))}
85     </ul>
86
87     <footer>
88       {fotoUrl ? (
89         <img src={fotoUrl} alt="" className="avatar"
90           style={{ objectFit: "cover" }} />
91       ) : (
92         <span className="avatar">
93           {(usuario?.nombre || "?")[0].toUpperCase()}
94         </span>
95       )}
96       <b style={{ fontSize: ".86rem", textTransform: "capitalize" }}>
97         {usuario?.nombre}
98       </b>
99       <small>
100         {usuario?.perfil === "auditor"
101           ? "Administrador de bodega"
102           : "Auxiliar de inventarios"}
103       </small>
104       {colaLen > 0 && (
105         <span className="chip oro" style={{ marginTop: 6 }}>{colaLen} por sincronizar</span>
106       )}
107       
108     </footer>
109   </nav>
110 );
111 }

```

frontend/src/Marco.jsx

19 LÍNEAS

El encabezado comun de cada pantalla.

```

1 /** El armazón que comparten todas las vistas: barra superior con la
2   sección, la faja dorada de marca y el logo de Colsubsidio. */

```



```

3
4 import React from "react";
5 export default function Marco({ titulo, chip, children }) {
6   return (
7     <>
8       <div className="barra-sup">
9         <div className="barra-sup-inner">
10          <h1>{titulo}</h1>
11          {chip && <span className={`chip ${chip.tipo} || ""`} >{chip.texto}</span>}
12          
13        </div>
14      </div>
15      <div className="faja" />
16      <div className="area">{children}</div>
17    </>
18  );
19 }

```

frontend/src/Dialogo.jsx

224 LÍNEAS

El cuadro modal: trampa de foco, cierre con Escape, campo con voz.

```

1 import { useState, useRef, useEffect } from "react";
2 import { useDevolverFoco } from "../foco";
3 import { escuchar, hablar, esAfirmacion, esNegacion } from "../voz";
4 import { interpretarLocal } from "../interpreteLocal";
5 import Icono from "../Iconos";
6
7 /** Reemplaza a window.prompt()/window.confirm(): un modal con el estilo
8     propio de la app, en vez del cuadro genérico y feo del navegador.
9     Uso: <Dialogo titulo="..." mensaje="..." conCampo onAceptar={...} onCancel={...} />
10
11     conVoz agrega el mismo micrófono que ya existe en el resto de la app,
12     con el mismo patrón de confirmación hablada que usa el agente al abrir
13     una bodega: lo dicho llena el campo (visible, editable) Y el agente
14     pregunta en voz alta "¿confirma X?" - un "sí" acepta directo, un "no"
15     descarta y deja intentar de nuevo. El botón de Aceptar sigue ahí como
16     respaldo (para quien prefiera escribir, o si la voz no entendió el
17     sí/no). No se usa en los diálogos "Escribir en vez de hablar": esos ya
18     son el respaldo a prueba de fallos cuando la voz no funciona. */
19 export default function Dialogo({
20   titulo, mensaje, conCampo = false, conVoz = false, multilinea = false, tipo = "text",
21   valorInicial = "", placeholder = "", textoAceptar = "Aceptar",
22   textoCancelar = "Cancelar", peligro = false, confirmarAlAbrir = false, onAceptar, onCancel,
23   // Nombre accesible del campo para un lector de pantalla - por defecto se
24   // usa "mensaje" (funciona bien cuando es una pregunta corta, como "¿Cuántas
25   // porciones son en realidad?"), pero en varios diálogos "mensaje" es un
26   // párrafo explicativo largo o hasta el contenido de un mensaje ajeno
27   // (ver Mensajes.jsx "Responder mensaje": ahí incluye el mensaje ORIGINAL
28   // completo) - leerlo entero cada vez que el campo recibe foco es
29   // ruidoso e inútil como etiqueta. Estos casos pasan su propio
30   // etiquetaCampo, corto y específico.
31   etiquetaCampo,
32 }) {
33   const [valor, setValor] = useState(valorInicial);
34   const [escuchando, setEscuchando] = useState(false);
35   const [confirmando, setConfirmando] = useState(false);
36   const [errorVoz, setErrorVoz] = useState("");
37   // Igual que en el selector de bodega de Conteo: cada escucha lleva su
38   // propio número, y si el micrófono anterior entrega su resultado tarde
39   // (por ejemplo, mientras todavía se está diciendo "¿confirma X?") se
40   // descarta en vez de procesarse como si fuera un nuevo valor dictado.
41   const idEscucha = useRef(0);
42   const modalRef = useRef(null);
43
44   // Accesibilidad: quien navega con teclado o con un lector de pantalla
45   // necesita que Escape cierre el modal (como cualquier diálogo nativo),
46   // y que el foco entre al modal apenas aparece - sin esto, un lector de
47   // pantalla seguía anunciando el contenido de atrás como si el modal ni
48   // existiera. Si hay un campo, ese ya se autoenfoca solo (ver más abajo);
49   // si no lo hay, el modal mismo recibe el foco.
50   useEffect(() => {

```

```

51     function alTecla(e) { if (e.key === "Escape") onCancel(); }
52     window.addEventListener("keydown", alTecla);
53     return () => window.removeEventListener("keydown", alTecla);
54 }, [onCancelar]);
55
56 useDevolverFoco();
57
58 useEffect(() => {
59     if (!conCampo) modalRef.current?.focus();
60     // eslint-disable-next-line react-hooks/exhaustive-deps
61 }, []);
62
63 function aceptar(valorAUsar = valor) {
64     if (conCampo && !String(valorAUsar).trim()) return;
65     onAceptar(valorAUsar);
66 }
67
68 async function preguntarConfirmacion(valorDictado, miId) {
69     setConfirmando(true);
70     // Con un campo de varias líneas (un mensaje libre, no un dato corto)
71     // no se repite todo lo dictado - sería una lectura larga, y encima
72     // alarga la ventana en la que la persona ya contestó "sí"/"envíalo"
73     // pero el micrófono todavía no se reabrió (seguía sonando la
74     // pregunta). Una confirmación corta reduce esa ventana.
75     const pregunta = multilinea ? "¿Confirma el mensaje?" : `¿Confirma «${valorDictado}»?`;
76     // hay que esperar a que termine de decir la pregunta antes de abrir
77     // el micrófono - si arranca apenas empieza a sonar, se pierde el
78     // principio de la respuesta.
79     await hablar(pregunta);
80     if (miId !== idEscucha.current) return;
81     let obtuvoResultado = false;
82     let yaReintento = false;
83     escuchar({
84         alTexto: (respuesta) => {
85             if (miId !== idEscucha.current) return;
86             obtuvoResultado = true;
87             setConfirmando(false);
88             // El botón de este diálogo en particular puede decir "Enviar",
89             // "Solicitar", "Crear"... no solo "Aceptar" - decir esa misma
90             // palabra (en cualquier conjugación) también debe confirmar,
91             // igual que tocar el botón.
92             if (esAfirmacion(respuesta, [textoAceptar])) {
93                 aceptar(valorDictado);
94             } else if (esNegacion(respuesta)) {
95                 setValor("");
96                 setErrorVoz("Cancelado. Dígallo de nuevo cuando quiera, o escríbalo.");
97             } else {
98                 setErrorVoz("No le entendí un «sí» o un «no». Use el botón para confirmar.");
99             }
100         },
101         alEstado: (e) => {
102             setEscuchando(e === "escuchando");
103             // El reconocedor del navegador se cierra solo tras unos segundos
104             // de silencio, sin avisar con un error (voz.js lo ignora a
105             // propósito) - sin esto, la pregunta se quedaba "colgada" en
106             // pantalla diciendo que estaba esperando un sí/no, cuando en
107             // realidad el micrófono ya se había apagado solo. Se vuelve a
108             // preguntar y a escuchar sola, en vez de obligar a tocar el
109             // micrófono de nuevo (lo que arrancaba una dictada nueva y
110             // borraba el mensaje ya armado).
111             if (e === "listo" && !obtuvoResultado && !yaReintento && miId === idEscucha.current) {
112                 yaReintento = true;
113                 preguntarConfirmacion(valorDictado, miId);
114             }
115         },
116         alError: (e) => { setConfirmando(false); setErrorVoz(e); },
117     });
118 }
119
120 function alTextoDictado(t, miId) {
121     if (!t) return;
122     if (tipo === "number") {
123         // el reconocimiento a veces ya entrega el número tal cual ("50"),
124         // y a veces la palabra ("cincuenta") - se intenta directo primero,

```

```

125 // y si no es un número se reusa el mismo interprete de cantidades
126 // que ya usa el conteo por voz, en vez de dejar el campo vacío.
127 const directo = Number(String(t).replace(",", "."));
128 if (!Number.isNaN(directo)) {
129   setValor(directo);
130   preguntarConfirmacion(directo, miId);
131   return;
132 }
133 const r = interpretarLocal(t);
134 if (r.cantidad !== null) {
135   setValor(r.cantidad);
136   preguntarConfirmacion(r.cantidad, miId);
137   return;
138 }
139 setErrorVoz(`No reconocí un número en «${t}». Dígallo de nuevo o escríbalo.`);
140 return;
141 }
142 setValor(t);
143 preguntarConfirmacion(t, miId);
144 }
145
146 function porVoz() {
147   setErrorVoz("");
148   const miId = ++idEscucha.current;
149   escuchar({
150     alTexto: (t) => {
151       if (miId !== idEscucha.current) return;
152       alTextoDictado(t, miId);
153     },
154     alEstado: (e) => setEscuchando(e === "escuchando"),
155     alError: setErrorVoz,
156   });
157 }
158
159 // Cuando el valor ya llega armado por voz desde afuera (por ejemplo,
160 // "respóndele a Stephanie que ya quedó asignada la bodega" en la
161 // bandeja de Mensajes), tocar el micrófono de este diálogo para decir
162 // "envíalo" NO debe funcionar como una dictada nueva - eso pisaba el
163 // texto ya armado con la palabra "envíalo" misma. En vez de obligar a
164 // usar el botón, se pregunta la confirmación de una vez al abrir, así
165 // "envíalo"/"sí" ya confirma lo que se armó, sin tocar nada.
166 useEffect(() => {
167   if (confirmarAlAbrir && String(valorInicial || "").trim()) {
168     preguntarConfirmacion(valorInicial, ++idEscucha.current);
169   }
170   // eslint-disable-next-line react-hooks/exhaustive-deps
171 }, []);
172
173 return (
174   <div className="overlay" onClick={onCancelar}>
175     <div className="modal" onClick={(e) => e.stopPropagation()}
176       role="dialog" aria-modal="true" aria-labelledby="dialogo-titulo"
177       ref={modalRef} tabIndex={-1}>
178       <h2 id="dialogo-titulo">{titulo}</h2>
179       {mensaje && <p style={{ whiteSpace: "pre-line" }}>{mensaje}</p>}
180       {conCampo && (
181         <div style={{ display: "flex", gap: 10, marginBottom: conVoz ? 6 : 16,
182           alignItems: multilinea ? "flex-start" : "center" }}>
183           {multilinea ? (
184             <textarea autoFocus value={valor} placeholder={placeholder} rows={3}
185               onChange={(e) => setValor(e.target.value)}
186               aria-label={etiquetaCampo || mensaje || titulo}
187               style={{ flex: 1, padding: "12px 14px",
188                 border: "1px solid var(--borde)", borderRadius: 12,
189                 fontFamily: "inherit", fontSize: "1rem", resize: "vertical" }} />
190             ) : (
191               <input type={tipo} autoFocus value={valor} placeholder={placeholder}
192                 onChange={(e) => setValor(e.target.value)}
193                 onKeyDown={(e) => e.key === "Enter" && aceptar()}
194                 aria-label={etiquetaCampo || mensaje || titulo}
195                 style={{ flex: 1, padding: "12px 14px",
196                   border: "1px solid var(--borde)", borderRadius: 12,
197                   textAlign: "center", fontSize: "1.05rem" }} />
198             )}
199         )}
200   </div>

```

```

199     {conVoz && (
200       <button type="button" className={`mic-btn ${escuchando ? "escuchando" : ""}`}
201         style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
202         onClick={porVoz} title="dígallo en voz alta"
203         aria-label="Dígallo en voz alta"><Icono nombre="microfono" tam={22} /></button>
204     )}
205   </div>
206 )}
207 {conVoz && (
208   <p className="pista" style={{ marginTop: -2, marginBottom: 14 }}>
209     {confirmando ? "Diga «sí» para confirmar, o «no» para intentar de nuevo..."
210       : escuchando ? "Escuchando..."
211       : "Dígallo y confírmelo de viva voz, o escríbalo y use el botón."}
212   </p>
213 )}
214 {errorVoz && <p className="error" style={{ marginBottom: 10 }}>{errorVoz}</p>}
215 <div className="botones">
216   <button className="btn borde" onClick={onCancelar}>{textoCancelar}</button>
217   <button className={`btn ${peligro ? "oro" : ""}`} onClick={() => aceptar()}>
218     {textoAceptar}
219   </button>
220 </div>
221 </div>
222 </div>
223 );
224 }

```

frontend/src/AsistenteVoz.jsx

159 LÍNEAS

El agente flotante de cada vista.

```

1 import { useState } from "react";
2 import { escuchar, hablar } from "../voz";
3 import { preguntarAsistente } from "../api";
4 import Icono from "../Iconos";
5
6 /** El mismo microfono que ya existe en Bodegas/Conteo/Pedido, pero para
7   pantallas sin conversacion (Inicio, Ajustes, Ayuda, Reportes, Panel):
8   una pregunta suelta sobre lo que hay en esta pantalla, o una orden de
9   ir a otra - sin el estado de "pendiente"/"opciones" que solo tiene
10  sentido en un conteo o un pedido en curso. */
11 export default function AsistenteVoz({ token, vista, ir, placeholder, alActualizar, ejemplos }) {
12   const [texto, setTexto] = useState("");
13   const [respuesta, setRespuesta] = useState("");
14   const [error, setError] = useState("");
15   const [escuchando, setEscuchando] = useState(false);
16   const [pensando, setPensando] = useState(false);
17
18   async function preguntar(dicho) {
19     if (!dicho || !dicho.trim()) return;
20     setTexto(dicho);
21     setError("");
22     // Algunas pantallas (ej. el filtro de Registro de trazabilidad)
23     // resuelven ciertas frases del todo en el frontend, sin backend ni
24     // Gemini de por medio - si un componente hijo la reconoce, llama
25     // preventDefault() y avisa así que ya quedó resuelta, para no
26     // preguntarle al agente algo que esta pantalla ya sabe contestar
27     // sola. Sin esto, decir aquí arriba exactamente uno de los
28     // ejemplos de la lista (que sí describen esas frases) no hacía
29     // nada, porque solo el micrófono chiquito junto a los filtros
30     // estaba conectado a esa lógica.
31     const evento = new CustomEvent("cuentavoz:filtro-local", {
32       detail: { texto: dicho }, cancelable: true,
33     });
34     window.dispatchEvent(evento);
35     if (evento.defaultPrevented) {
36       // El componente que lo reclamó (ej. TabTraza) es quien sabe el
37       // resultado real (cuántas filas encontró) - anuncia ESO por su
38       // cuenta de forma asíncrona en vez de un "listo, apliqué el
39       // filtro" genérico que no dice nada útil.
40       return;
41     }

```

```

42     setPensando(true);
43     try {
44         const r = await preguntarAsistente(dicho, vista, token);
45         setRespuesta(r.respuesta_hablada || "");
46         hablar(r.respuesta_hablada);
47         if (r.accion === "navegar" && r.destino && ir) {
48             // ir() combina el contexto en vez de reemplazarlo: sin fijar
49             // tabInicial explícitamente (aunque sea undefined), una pestaña
50             // pedida en una navegacion anterior por otro camino se quedaba
51             // pegada y aparecia en un destino que no la tiene. Lo mismo para
52             // bodegaSugerida: sin fijarlo a undefined cuando no aplica, un
53             // "abra kiosco" pedido antes se quedaba pegado y la próxima vez
54             // que se entraba a Conteo por otro camino la abría sola. Lo mismo
55             // para bodegaAuditar, su equivalente en Auditoría.
56             ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined,
57                 bodegaAuditar: r.bodega_auditar || undefined });
58         } else if (r.archivo && r.titulo_archivo && r.pestana && ir) {
59             // Cubre dos casos: "previsualizar_reporte" (pedir VER un archivo
60             // ya generado) y "actualizar" con archivo (GENERAR uno nuevo por
61             // voz - "genera el consolidado", "exporta las diferencias") -
62             // ninguno trae "destino" (no cambian de PANTALLA: ya estamos en
63             // Reportes), pero sí pueden pedirse desde OTRA pestaña de esta
64             // misma pantalla. Sin este ir(), el evento de abajo llegaba a un
65             // TabConsolidado que ni siquiera estaba montado - solo se dibuja
66             // cuando la pestaña activa YA es "consolidado" - así que la vista
67             // previa no aparecía nunca si no se estaba, por casualidad, ya en
68             // esa pestaña (ni al pedir verla, ni al generarla). Va por props
69             // (como bodegaSugerida) en vez del evento genérico de abajo,
70             // precisamente porque necesita sobrevivir a un cambio de pestaña
71             // que todavía no ocurrió cuando se dispara el evento.
72             // r.titulo_archivo exigido a proposito: "exporta el análisis de
73             // consumo" trae archivo+pestana ("análisis") pero SIN
74             // titulo_archivo (usa su propio evento "descargar_reporte", ver
75             // TabAnálisis) - sin este chequeo, ese caso también disparaba
76             // este ir() con titulo undefined, y al volver a la pestaña
77             // "Consolidado" a mano, TabConsolidado remontaba y previsualizaba
78             // ese archivo viejo con "Vista previa · undefined" de encabezado.
79             ir(vista, { tabInicial: r.pestana,
80                 archivoPrevisualizar: { archivo: r.archivo, titulo: r.titulo_archivo } });
81         }
82         // "actualizar": el agente cambió algo de verdad (ej. el modo sin
83         // conexión en Ajustes) - la pantalla que lo pidió es quien sabe
84         // cómo refrescar su propio dato, este componente no.
85         if (r.accion === "actualizar" && alActualizar) alActualizar();
86         // Cualquier otra acción (ej. "mostrar_cobertura"/"ocultar_cobertura"
87         // para abrir/cerrar un panel específico de la pantalla) se avisa
88         // como evento genérico - así un componente hijo que le interesa
89         // puede escucharlo sin que este componente necesite saber qué es
90         // cada pantalla ni acumular una prop nueva por cada acción. Ojo:
91         // "previsualizar_reporte" queda afuera a propósito (va por props,
92         // ver arriba) para no disparar el mismo aviso dos veces.
93         if (r.accion && r.accion !== "navegar" && r.accion !== "actualizar"
94             && r.accion !== "previsualizar_reporte") {
95             // detail: r - para acciones que traen datos propios, no solo la
96             // señal de que pasó algo. Las que no lo necesitan lo ignoran.
97             window.dispatchEvent(new CustomEvent(`cuentavoz:accion:${r.accion}`, { detail: r }));
98         }
99     } catch (e) {
100         // todo lo que el agente muestra en pantalla también lo dice en voz -
101         // este catch se quedaba solo en rojo, distinto de como ya se
102         // corrigió el mismo patrón en Conteo, Pedido, Legalización,
103         // Auditoría, Ajustes, Ayuda y Mensajes. Como este es el componente
104         // compartido de Inicio, Mi perfil, Ajustes, Auditoría, Panel y
105         // Reportes, era el hueco más grande: cubría seis pantallas de una
106         // sola vez.
107         setError(e.message);
108         hablar(e.message);
109     }
110     setPensando(false);
111 }
112
113 function porVoz() {
114     escuchar({
115         alTexto: preguntar,

```

```

116     alEstado: (e) => setEscuchando(e === "escuchando"),
117     alError: setError,
118   });
119 }
120
121 return (
122   <div className="card">
123     <p className="rotulo">Pregúntele al agente</p>
124     <div style={{ display: "flex", gap: 10 }}>
125       <input value={texto} onChange={(e) => setTexto(e.target.value)}
126         onKeyDown={(e) => e.key === "Enter" && preguntar(texto)}
127         placeholder={placeholder || "pregunte algo, o pida ir a otra pantalla..."}
128         aria-label="Pregúntele al agente"
129         style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
130           border: "1px solid var(--borde)", borderRadius: 12 }} />
131       <button className={`mic-btn ${escuchando ? "escuchando" : ""}`}
132         style={{ width: 46, height: 46, fontSize: "1.2rem" }}
133         onClick={porVoz} title="pregúntele por voz" aria-label="Pregúntele al agente por voz">
134         <Icono nombre="microfono" tam={22} /></button>
135     </div>
136     <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
137     {ejemplos && ejemplos.length > 0 && (
138       <details style={{ marginTop: 8 }}>
139         <summary className="pista" style={{ cursor: "pointer" }}>
140           Ver frases exactas que entiende el agente aquí
141         </summary>
142         <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
143           {ejemplos.map((ej, i) => (
144             <li key={i} className="pista" style={{ marginBottom: 4 }}><{ej}></li>
145           ))}
146         </ul>
147       </details>
148     )}
149     {pensando && <p className="cargando" style={{ marginTop: 10 }}>Pensando...</p>}
150     {respuesta && !pensando && (
151       <>
152         <p className="rotulo" style={{ marginTop: 10 }}>CuentaVoz responde</p>
153         <p className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</p>
154       </>
155     )}
156     {error && <p className="error" style={{ marginTop: 10 }}>{error}</p>}
157   </div>
158 );
159 }

```

frontend/src/ChecklistClave.jsx

36 LÍNEAS

Los cuatro requisitos reales de la política de clave.

```

1 import Icono from "../Iconos";
2
3 // Los 4 requisitos reales de la política de clave del User Pool de
4 // Cognito (mín. 8, mayúscula, minúscula, número - SIN símbolo
5 // obligatorio, a diferencia del ejemplo de 5 requisitos de AWS: mostrar
6 // un quinto check por un requisito que Cognito en realidad no exige
7 // sería mentirle a quien está llenando el formulario).
8 const _REQUISITOS = [
9   { id: "longitud", texto: "Al menos 8 caracteres", prueba: (c) => c.length >= 8 },
10  { id: "mayuscula", texto: "Una letra mayúscula", prueba: (c) => /[A-Z]/.test(c) },
11  { id: "minuscula", texto: "Una letra minúscula", prueba: (c) => /[a-z]/.test(c) },
12  { id: "numero", texto: "Un número", prueba: (c) => /[0-9]/.test(c) },
13 ];
14
15 export function claveCumplePolitica(clave) {
16   return _REQUISITOS.every((r) => r.prueba(clave));
17 }
18
19 /** Checklist en vivo (como el de la propia pantalla de registro de
20   Cognito): cada requisito pasa a verde con un check apenas se cumple,
21   en vez de un solo mensaje de error genérico al enviar el formulario. */
22 export default function ChecklistClave({ clave }) {
23   return (

```

```

24     <ul className="checklist-clave">
25       {_REQUISITOS.map((r) => {
26         const ok = r.prueba(clave);
27         return (
28           <li key={r.id} className={ok ? "ok" : ""}>
29             {ok ? <Icono nombre="check" tam={14} /> : <span className="circulo" />}
30             {r.texto}
31           </li>
32         );
33       })}
34     </ul>
35   );
36 }

```

frontend/src/ConfigurarMFA.jsx

86 LÍNEAS

El QR del segundo factor.

```

1  import { useEffect, useState } from "react";
2  import QRCode from "qrcode";
3  import { iniciarActivacionMFA, confirmarActivacionMFA, uriOtpAuth } from "./cognito";
4
5  /** Los 3 pasos para activar la verificación en dos pasos (instalar,
6   escanear/copiar la llave, confirmar con el código) - un solo
7   componente para no repetir esto en Mi perfil Y en el registro (ver
8   Ingreso.jsx: se ofrece justo después de confirmar el correo). Pide
9   la llave a Cognito apenas se monta - necesita una sesión ya
10  autenticada (self-service, no admin), así que solo se usa después
11  de un login o un registro ya confirmado. */
12  export default function ConfigurarMFA({ nombreUsuario, onActivado, onCancelar, textoCancelar = "Cancelar" }) {
13    const [llave, setLlave] = useState("");
14    const [qr, setQr] = useState("");
15    const [mostrarLlave, setMostrarLlave] = useState(false);
16    const [codigo, setCodigo] = useState("");
17    const [err, setErr] = useState("");
18    const [cargando, setCargando] = useState(false);
19
20    useEffect(() => {
21      iniciarActivacionMFA().then(setLlave).catch((e) => setErr(e.message));
22    }, []);
23
24    // el QR se genera del lado del cliente (nadie más necesita verlo) a
25    // partir de la llave secreta que da Cognito.
26    useEffect(() => {
27      if (!llave || !nombreUsuario) { setQr(""); return; }
28      QRCode.toDataURL(uriOtpAuth(llave, nombreUsuario)).then(setQr).catch(() => setQr(""));
29    }, [llave, nombreUsuario]);
30
31    async function confirmar() {
32      setErr("");
33      setCargando(true);
34      try {
35        await confirmarActivacionMFA(codigo.trim());
36        onActivado();
37      } catch (e) {
38        setErr(e.message);
39      } finally {
40        setCargando(false);
41      }
42    }
43
44    return (
45      <div className="mfa-pasos">
46        <div className="mfa-paso">
47          <span className="num">1</span>
48          <div className="contenido">
49            <b>Instale una app autenticadora</b>
50            <p>Google Authenticator, Microsoft Authenticator o similar, en su celular.</p>
51          </div>
52        </div>
53        <div className="mfa-paso">
54          <span className="num">2</span>
55          <div className="contenido">

```

```

55     <b>Escanee el código QR</b>
56     <p>O ingrese la llave a mano en la app.</p>
57     {qr && <img className="mfa-qr" src={qr} alt="Código QR para configurar la app autenticadora" />}}
58     <button type="button" className="enlace" style={{ marginTop: 6 }}
59       onClick={() => setMostrarLlave((v) => !v)}>
60       {mostrarLlave ? "Ocultar llave" : "Mostrar llave secreta"}
61     </button>
62     {mostrarLlave && <p className="mfa-llave-secreta">{llave}</p>}}
63   </div>
64 </div>
65 <div className="mfa-paso">
66   <span className="num">3</span>
67   <div className="contenido">
68     <b>Ingresa el código</b>
69     <p>El código de 6 dígitos que muestra la app en este momento.</p>
70     <input value={codigo} inputMode="numeric" placeholder="Código"
71       onChange={(e) => setCodigo(e.target.value)}
72       aria-label="Código de la app autenticadora"
73       style={{ width: "100%", padding: "11px 13px", marginTop: 8,
74         border: "1px solid var(--borde)", borderRadius: 12 }} />
75   </div>
76 </div>
77 {err && <p className="error" role="alert">{err}</p>}}
78 <div className="grilla-botones">
79   <button className="btn" onClick={confirmar} disabled={!codigo || cargando}>
80     {cargando ? "Activando..." : "Activar"}
81   </button>
82   {onCancelar && <button className="btn gris" onClick={onCancelar}>{textoCancelar}</button>}}
83 </div>
84 </div>
85 );
86 }

```

frontend/src/VideoRecorrido.jsx

104 LÍNEAS

El recorrido narrado por perfil.

```

1 import { useEffect, useRef, useState } from "react";
2 import { debeAbrirTutorial, deshabilitarTutorial, habilitarTutorial } from "../tutorial";
3 import { useDevolverFoco } from "../foco";
4
5 /* El video se sirve desde /public, no desde un servicio externo: la app se
6 usa en bodegas con señal irregular y un video en YouTube que no carga es
7 peor que no ofrecerlo. Al venir del mismo origen, el navegador lo trae
8 con el resto de la aplicación. Están fuera del precacheo del service
9 worker (ver vite.config.js) para no obligar a bajar varios megas la
10 primera vez que alguien abre el login. */
11 /* El ?v= no lo lee el servidor: es solo para el cache del navegador. Los
12 mp4 viven en una ruta fija (no llevan hash como los bundles de Vite), así
13 que sin esto quien ya vio el recorrido una vez se queda con la copia
14 vieja aunque publiquemos una corregida. Al volver a grabar un recorrido
15 hay que subirle la fecha a su version. */
16 const VIDEOS = {
17   auxiliar: {
18     src: "/recorrido-auxiliar.mp4",
19     version: "2026-08-24g",
20     titulo: "Recorrido para auxiliares de inventarios",
21   },
22   auditor: {
23     src: "/recorrido-administrador.mp4",
24     version: "2026-08-25a",
25     titulo: "Recorrido para administradores de bodega",
26   },
27 };
28
29 export default function VideoRecorrido({ perfil, usuarioId, onCerrar }) {
30   const video = VIDEOS[perfil === "auditor" ? "auditor" : "auxiliar"];
31   const fuente = `${video.src}?v=${video.version}`;
32   const modalRef = useRef(null);
33   const videoRef = useRef(null);
34   useDevolverFoco();
35   // Refleja lo que ya hay guardado para este usuario, no un estado nuevo

```



```

36 // en false: si ya lo habia desactivado antes y por lo que sea el video
37 // se abrio igual (p. ej. "Ver el recorrido en video" desde Ayuda), el
38 // interruptor no debe mostrarse como si estuviera activado.
39 const [desactivado, setDesactivado] = useState(
40   () => usuarioId !== null && !debeAbrirTutorial(usuarioId)
41 );
42
43 useEffect(() => {
44   function alTecla(e) { if (e.key === "Escape") onCerrar(); }
45   window.addEventListener("keydown", alTecla);
46   return () => window.removeEventListener("keydown", alTecla);
47 }, [onCerrar]);
48
49 useEffect(() => {
50   modalRef.current?.focus();
51   // eslint-disable-next-line react-hooks/exhaustive-deps
52 }, []);
53
54 // Se intenta reproducir solo, sin que la persona tenga que darle play:
55 // el video se abre COMO CONSECUENCIA de un clic (entrar, o "Ver el
56 // recorrido en video" en Ayuda), y ese gesto reciente es justo lo que
57 // los navegadores piden para permitir sonido automático. Si de todas
58 // formas lo bloquean (típico cuando la sesión se retomó sola, sin clic
59 // reciente), se queda pausado con los controles listos - nunca se
60 // fuerza a silenciarlo, porque un video narrado mudo no sirve de nada.
61 useEffect(() => {
62   videoRef.current?.play().catch(() => {});
63 }, []);
64
65 return (
66   <div className="overlay" onClick={onCerrar}>
67     <div className="modal" onClick={(e) => e.stopPropagation()}
68       role="dialog" aria-modal="true" aria-labelledby="video-titulo"
69       ref={modalRef} tabIndex=-1
70       style={{ maxWidth: 780, width: "92vw" }}>
71       <h2 id="video-titulo" style={{ marginBottom: 4 }}>{video.titulo}</h2>
72       <p className="pista" style={{ marginBottom: 12 }}>
73         Narrado, con un ejemplo de cada pantalla. Puede pausarlo o cerrarlo
74         y seguir después.
75       </p>
76
77       <video ref={videoRef} src={fuente} controls preload="auto" playsInline
78         style={{ width: "100%", display: "block", borderRadius: 10,
79           background: "#000", border: "1px solid var(--borde)" }}>
80         Su navegador no puede reproducir este video.{" "}
81         <a href={fuente} download>Descárguelo aquí.</a>
82       </video>
83
84       {usuarioId !== null && (
85         <label style={{ display: "flex", alignItems: "center", gap: 8,
86           marginTop: 14, fontSize: ".9rem", color: "var(--grafito)" }}>
87           <input type="checkbox" checked={desactivado}
88             onChange={(e) => {
89               const marcado = e.target.checked;
90               setDesactivado(marcado);
91               if (marcado) deshabilitarTutorial(usuarioId);
92               else habilitarTutorial(usuarioId);
93             }} />
94           No mostrar este video automáticamente la próxima vez que ingrese
95         </label>
96       )}
97
98       <div className="botones" style={{ marginTop: 16 }}>
99         <button className="btn" onClick={onCerrar}>Cerrar</button>
100       </div>
101     </div>
102   </div>
103 );
104 }

```

frontend/src/Iconos.jsx

75 LÍNEAS

Los iconos, en SVG.

```

1  /** Iconos del menú lateral. Uno por cada entrada, en trazo simple.
2   * Heredan el color del texto (currentColor), así el CSS decide si van
3   * en dorado (activo) o en gris azulado (inactivo). Sin fondo. */
4  const TRAZO = {
5    fill: "none",
6    stroke: "currentColor",
7    strokeWidth: 1.8,
8    strokeLinecap: "round",
9    strokeLinejoin: "round",
10 };
11
12  const FORMAS = {
13    // Inicio: casa
14    inicio: <><path d="M3 10.5 12 31.9 7.5" /><path d="M5.5 9.5V20h13V9.5" /><path d="M10 20v-5h4v5" /></>,
15    // Pedidos: nota de pedido con esquina doblada
16    pedidos: <><path d="M6 3h8l4 4v14H6z" /><path d="M14 3v4h4" /><path d="M9 12h6M9 16h4" /></>,
17    // Conteo: micrófono
18    conteo: <><rect x="10" y="3" width="4" height="9" rx="2" /><path d="M6.5 11a5.5 5.5 0 0 0 11 0" /><path d="M12
16.5V20" /><path d="M9 20h6" /></>,
19    // Legalización: balanza (lo pedido contra lo usado)
20    legalizacion: <><path d="M12 4v15" /><path d="M5 7h14" /><path d="M8.5 19h7" /><path d="M2.5 12 5 7l2.5 5a2.5 2.5 0
0 1-5 0z" /><path d="M16.5 12 19 7l2.5 5a2.5 2.5 0 0 1-5 0z" /></>,
21    // Bodegas: bodega con techo a dos aguas
22    bodegas: <><path d="M3 9.5 12 31.9 6.5" /><path d="M5 9v11h14V9" /><path d="M5 14.5h14" /><path d="M12 9v11" /></>,
23    // Auditoría: lupa con visto bueno
24    auditoria: <><circle cx="11" cy="10.5" r="6" /><path d="M15.5 15 20 19.5" /><path d="M8.5 10.5l2 3.5-4" /></>,
25    // Reportes: documento con líneas
26    reportes: <><rect x="5" y="3" width="14" height="18" rx="2" /><path d="M8.5 8h7M8.5 12h7M8.5 16h4" /></>,
27    // Panel: barras
28    panel: <><path d="M4 20h16" /><rect x="6" y="12" width="3" height="6" /><rect x="11" y="7" width="3" height="11" />
<rect x="16" y="14" width="3" height="4" /></>,
29    // Ajustes: engranaje
30    ajustes: <><circle cx="12" cy="12" r="3.2" /><path d="M12 3v2.6M12 18.4V21M3 12h2.6M18.4 12H21M5.6 5.6l1.9 1.9M16.5
16.5l1.9 1.9M18.4 5.6l-1.9 1.9M7.5 16.5l-1.9 1.9" /></>,
31    // Ayuda: interrogación en círculo
32    ayuda: <><circle cx="12" cy="12" r="8.5" /><path d="M9.6 9.4a2.5 2.5 0 1 1 3.4 2.3c-.6.3-1 .9-1 1.6v.3" /><circle
cx="12" cy="17" r="9" fill="currentColor" stroke="none" /></>,
33    // Mensajes: sobre de correo
34    mensajes: <><rect x="3" y="5.5" width="18" height="13" rx="2" /><path d="M3.5 7 12 13l8.5-6" /></>,
35    // Mi perfil: persona
36    perfil: <><circle cx="12" cy="8" r="3.5" /><path d="M5 20a7 7 0 0 1 14 0" /></>,
37    // Cerrar sesión: puerta con flecha de salida
38    salir: <><path d="M14 4H6v16h8" /><path d="M11 12h9" /><path d="M17 8.5 20.5 12 17 15.5" /></>,
39    // Candado: campo de PIN en el ingreso
40    candado: <><rect x="5" y="11" width="14" height="9" rx="2" /><path d="M8 11v7.5a4 4 0 0 1 8 0V11" /></>,
41    // Descargar: flecha hacia una bandeja
42    descargar: <><path d="M12 3v11.5" /><path d="M7.5 10.5 12 15l4.5-4.5" /><path d="M4.5 17.5v2a2 2 0 0 2 2h11a2 2 0
0 0 2-2v-2" /></>,
43    // Aprobar/resolver: visto bueno en círculo
44    aprobar: <><circle cx="12" cy="12" r="8.5" /><path d="M8 12.3l2.6 2.6l16 9.3" /></>,
45    // Visto bueno solo (sin círculo) - checklist de requisitos de clave
46    check: <path d="M4 12.3l5.2 5.2l20 6" />,
47    // Rechazar: equis en círculo
48    rechazar: <><circle cx="12" cy="12" r="8.5" /><path d="M9 9l6 6M15 9l-6 6" /></>,
49    // Advertencia: triángulo con exclamación (reportar un problema)
50    advertencia: <><path d="M12 3.5 21.5 20h-19z" /><path d="M12 9.5v5" /><circle cx="12" cy="17" r="9"
fill="currentColor" stroke="none" /></>,
51    // Refrescar: dos flechas en ciclo (restaurar valores)
52    refrescar: <><path d="M4 12a8 8 0 0 1 13.66-5.66L20 8.5" /><path d="M20 4v4.5h-4.5" /><path d="M20 12a8 8 0 0 1-
13.66 5.66L4 15.5" /><path d="M4 20v-4.5h4.5" /></>,
53    // Micrófono: mismo trazo que el icono "conteo" del menú, para que
54    // TODO botón de escuchar en la app se vea igual - antes cada uno usaba
55    // el emoji 🎧, que se ve distinto (y en algunos Windows, irreconocible)
56    // según la fuente de emoji del sistema operativo.
57    microfono: <><rect x="10" y="3" width="4" height="9" rx="2" /><path d="M6.5 11a5.5 5.5 0 0 0 11 0" /><path d="M12
16.5V20" /><path d="M9 20h6" /></>,
58 };
59
60  export default function Icono({ nombre, tam = 20 }) {

```

```

61  const forma = FORMAS[nombre];
62  if (!forma) return null;
63  return (
64    <svg
65      width={tam}
66      height={tam}
67      viewBox="0 0 24 24"
68      aria-hidden="true"
69      style={{ flex: "none" }}
70      {...TRAZO}
71    >
72      {forma}
73    </svg>
74  );
75 }

```

17.6 Frontend · las catorce vistas

frontend/src/vistas/Ingreso.jsx

677 LÍNEAS

Ingreso, registro y recuperacion.

```

1  import { useEffect, useState } from "react";
2  import { pedir, esFalloRed } from "../api";
3  import {
4    registrarse, reenviarCodigoRegistro, confirmarRegistro, iniciarSesion,
5    confirmarCodigoMFA, recuperarClave, confirmarNuevaClave, obtenerAtributosUsuario,
6  } from "../cognito";
7  import ChecklistClave, { claveCumplePolitica } from "../ChecklistClave";
8  import ConfigurarMFA from "../ConfigurarMFA";
9  import Icono from "../Iconos";
10
11  const ETIQUETAS_PERFIL = {
12    auxiliar: "Auxiliar de inventarios",
13    auditor: "Administrador de bodega",
14  };
15
16  /** Junta el token de Cognito con el perfil guardado en esta app (perfil,
17   id, nombre...) - lo que App.jsx espera recibir de alEntrar(). Cognito
18   ya confirmó a la persona, pero la fila local (creada por
19   /api/registro-completado, ver confirmarYEntrar) a veces nunca llegó a
20   crearse - por ejemplo si cerró la pestaña justo después de confirmar
21   el código, antes de esa última llamada. En vez de dejar a alguien con
22   una cuenta "a medias" varado en un 401 sin salida, se autocompleta
23   aquí mismo con los datos que Cognito sí tiene (nombre/correo) y se
24   reintenta - transparente para quien inicia sesión. */
25  // El backend responde esto (503) cuando no pudo consultarle a Cognito las
26  // llaves para verificar el token - no es culpa del token ni de la persona
27  // (ver seguridad.py: ServicioIdentidadCaído), así que vale la pena
28  // reintentar solo en vez de mandarla de vuelta al login.
29  function esCognitoNoDisponible(e) {
30    return e.message === "No se pudo verificar su sesion en este momento. Intente de nuevo.";
31  }
32
33  /** Reintenta lo que falló por algo pasajero, no por culpa de quien ingresa.
34   Dos casos, los dos reales:
35   - El backend vive en el plan gratuito de Render: si lleva 15 minutos sin
36     tráfico se duerme, y la petición que lo despierta tarda 30-50 segundos
37     o se corta en seco. Sin esto, quien se está registrando ve el botón
38     congelado en «Confirmando...», y al reintentar un «Sin conexión con el
39     servidor» que además es falso - su Wi-Fi está bien, el servidor estaba
40     dormido.
41   - Cognito no respondió al verificar el token (ver seguridad.py).
42     alEsperar avisa a la pantalla para que diga qué está pasando en vez de
43     dejar un botón mudo. */
44  // Despertar el servicio dormido de Render tarda entre 30 y 50 segundos, así
45  // que la espera total tiene que cubrir eso con margen: 2+4+6+8+10+12+15 = 57
46  // segundos repartidos en 8 intentos. Un reintento corto (los 9 s que habia
47  // antes) se agota entero DENTRO de la ventana de arranque y falla igual.
48  const ESPERAS = [2000, 4000, 6000, 8000, 10000, 12000, 15000];

```

```

49
50 async function conPaciencia(fn, alEsperar) {
51   for (let intento = 0; ; intento++) {
52     try {
53       return await fn();
54     } catch (e) {
55       const recuperable = esFalloRed(e) || esCognitoNoDisponible(e);
56       if (!recuperable || intento >= ESPERAS.length) {
57         if (recuperable) {
58           // Ya no es "revise su Wi-Fi": la red de esta persona esta bien,
59           // el servidor no alcanzo a despertar. Culpar al Wi-Fi manda a
60           // buscar el problema donde no esta.
61           const falla = new Error("El servidor está tardando en responder. "
62             + "Espere unos segundos y vuelva a intentarlo.");
63           falla.esFalloRed = true;
64           throw falla;
65         }
66         throw e;
67       }
68       alEsperar?.(intento + 1);
69       await new Promise((r) => setTimeout(r, ESPERAS[intento]));
70     }
71   }
72 }
73
74 async function sesionCompleta(token, alEsperar) {
75   const perfil = () => conPaciencia(() => pedir("/api/usuarios/yo", {}, token), alEsperar);
76   try {
77     return { token, usuario: await perfil() };
78   } catch (e) {
79     if (e.message !== "Usuario no reconocido.") throw e;
80     const atributos = await obtenerAtributosUsuario();
81     await conPaciencia(() => pedir("/api/registro-completado", {
82       method: "POST",
83       body: JSON.stringify({ nombre_completo: atributos.nombreCompleto, correo: atributos.correo }),
84     }, token), alEsperar);
85     return { token, usuario: await perfil() };
86   }
87 }
88
89 /** Aviso de código enviado por correo, con el icono de sobre y el
90     destino ya enmascarado por Cognito (p***@m***) - misma idea que la
91     pantalla "Confirm your account" de Cognito. */
92 function AvisoCodigo({ destino, onReenviar, reenviando, reenviado }) {
93   return (
94     <>
95     <div className="aviso-codigo">
96       <Icono nombre="mensajes" tam={18} />
97       <span>
98         Le enviamos un código a <b>{destino || "su correo"}</b>. Ingrésele para continuar.
99       </span>
100     </div>
101     <button type="button" className="reenviar-codigo" onClick={onReenviar} disabled={reenviando}>
102       {reenviando ? "Reenviando..." : reenviado ? "Código reenviado" : "Reenviar código"}
103     </button>
104   </>
105 );
106 }
107
108 export default function Ingreso({ alEntrar, avisoInicial }) {
109   // "login" | "login-mfa" | "registro" | "registro-codigo" | "registro-mfa" |
110   // "recuperar" | "recuperar-codigo"
111   const [modo, setModo] = useState("login");
112   // la sesión ya está lista (Cognito + fila local) apenas se confirma el
113   // registro - se guarda aquí mientras se ofrece activar la verificación
114   // en dos pasos, y solo se entrega a alEntrar() al terminar ese paso
115   // (activarla o saltarla).
116   const [sesionPendiente, setSesionPendiente] = useState(null);
117   const [usuario, setUsuario] = useState("");
118   const [clave, setClave] = useState("");
119   const [err, setErr] = useState(avisoInicial || "");
120   // errores DE UN CAMPO puntual (falta diligenciarlo, o quedó mal), para
121   // señalar justo debajo de ese campo - "err" (arriba) queda para lo que
122   // no es de un campo en particular (usuario o clave incorrectos, fallas

```

```

123 // del servidor...).
124 const [errCampo, setErrCampo] = useState({});
125 const [cargando, setCargando] = useState(false);
126 // Lo que está pasando mientras se espera - solo aparece si de verdad hay
127 // que esperar (servidor dormido, ver conPaciencia), para que el botón no
128 // se quede mudo y parezca colgado.
129 const [avisoEspera, setAvisoEspera] = useState("");
130 const avisarEspera = (intento) =>
131     setAvisoEspera("El servidor estaba en reposo y está despertando "
132         + `(intento ${intento})... puede tardar hasta un minuto.`);
133 const [perfil, setPerfil] = useState(null);
134
135 /** onChange que además borra el error de ESE campo apenas la persona
136     empieza a corregirlo, en vez de dejarlo marcado en rojo hasta el
137     próximo intento de enviar. */
138 function actualizar(setter, campo) {
139     return (e) => {
140         setter(e.target.value);
141         setErrCampo((prev) => (prev[campo] ? { ...prev, [campo]: null } : prev));
142     };
143 }
144
145 // registro
146 const [nombreCompleto, setNombreCompleto] = useState("");
147 const [codigoEmpleado, setCodigoEmpleado] = useState("");
148 const [correo, setCorreo] = useState("");
149 const [clave2, setClave2] = useState("");
150 const [codigo, setCodigo] = useState("");
151 const [destinoCodigo, setDestinoCodigo] = useState("");
152 const [reenviando, setReenviando] = useState(false);
153 const [reenviado, setReenviado] = useState(false);
154 // recuperar
155 const [claveNueva, setClaveNueva] = useState("");
156 const [claveNueva2, setClaveNueva2] = useState("");
157 // login con verificación en dos pasos
158 const [mfaPendiente, setMfaPendiente] = useState(null);
159 const [codigoMFA, setCodigoMFA] = useState("");
160
161 // Despierta el backend apenas se abre esta pantalla, sin esperar a que
162 // alguien pulse nada. En el plan gratuito de Render el servicio se duerme
163 // tras 15 minutos sin tráfico y tarda 30-50 segundos en levantarse; ese
164 // tiempo se lo comía la primera acción de la persona (confirmar el código
165 // del correo, justo el peor momento). Iniciándolo aquí, mientras se
166 // escriben los datos y llega el correo, el servidor ya está listo cuando
167 // de verdad hace falta. /api/salud no pide sesión y no cambia nada.
168 useEffect(() => {
169     pedir("/api/salud").catch(() => {});
170 }, []);
171
172 // apenas escribe el usuario se identifica el perfil solo - no hay que
173 // elegirlo a mano. Con demora corta para no disparar una llamada por
174 // cada tecla. Solo aplica en login: en registro el usuario es nuevo,
175 // no tiene perfil todavía que detectar.
176 useEffect(() => {
177     if (modo !== "login") { setPerfil(null); return; }
178     const entrada = usuario.trim();
179     if (!entrada) { setPerfil(null); return; }
180     const espera = setTimeout(() => {
181         pedir(`/api/usuarios/perfil?usuario=${encodeURIComponent(entrada)}`)
182             .then((r) => setPerfil(r.perfil))
183             .catch(() => setPerfil(null));
184     }, 350);
185     return () => clearTimeout(espera);
186 }, [usuario, modo]);
187
188 /* Se guarda en el navegador y no en un useState porque entre "cambie
189     mi clave" e "inicie sesion" la persona puede recargar la pagina, o
190     cerrarla y volver mas tarde: el aviso de vencimiento seguiria mal. */
191 function marcarClaveCambiadaAlEntrar(token) {
192     try {
193         if (!localStorage.getItem("cv_clave_recien_cambiada")) return;
194         localStorage.removeItem("cv_clave_recien_cambiada");
195         pedir("/api/usuarios/yo/marcar-clave-cambiada", { method: "POST" }, token)
196             .catch(() => {});

```

```

197     } catch (_) { /* navegador sin almacenamiento: no vale romper el ingreso */ }
198   }
199
200   function cambiarModo(nuevo) {
201     setModo(nuevo);
202     setErr(""); setErrCampo({}); setReenviado(false); setAvisoEspera("");
203   }
204
205   /** Cierra el registro después del paso de la verificación en dos pasos,
206       se haya activado o saltado: entra directo a la aplicación. */
207   function terminarRegistro() {
208     alEntrar(sesionPendiente);
209   }
210
211   async function entrar(e) {
212     e?.preventDefault();
213     setErr("");
214     const errores = {};
215     if (!usuario.trim()) errores.usuario = "Digite su usuario.";
216     if (!clave) errores.clave = "Digite su clave.";
217     if (Object.keys(errores).length) return setErrCampo(errores);
218     setErrCampo({});
219     setCargando(true);
220     try {
221       const r = await iniciarSesion(usuario.trim(), clave);
222       if (r.mfaPendiente) {
223         setMfaPendiente(r.mfaPendiente);
224         setModo("login-mfa");
225         return;
226       }
227       marcarClaveCambiadaAlEntrar(r.token);
228       alEntrar(await sesionCompleta(r.token, avisarEspera));
229     } catch (e2) {
230       if (e2.code === "UserNotConfirmedException") {
231         // se registró antes pero nunca terminó de confirmar el código
232         // (cerró la pestaña, se le olvidó...) - en vez de dejarlo
233         // varado con un error sin salida, se le reenvía un código
234         // nuevo y se le lleva directo a esa pantalla.
235         try {
236           const { destino } = await reenviarCodigoRegistro(usuario.trim());
237           setDestinoCodigo(destino);
238           setModo("registro-codigo");
239         } catch (e3) {
240           setErr(e3.message);
241         }
242       } else {
243         setErr(e2.message);
244       }
245     } finally {
246       setCargando(false);
247       setAvisoEspera("");
248     }
249   }
250
251   async function confirmarMFALogin(e) {
252     e?.preventDefault();
253     setErr("");
254     if (!codigoMFA.trim()) return setErrCampo({ codigoMFA: "Digite el código de 6 dígitos." });
255     setErrCampo({});
256     setCargando(true);
257     try {
258       const token = await confirmarCodigoMFA(mfaPendiente, codigoMFA.trim());
259       marcarClaveCambiadaAlEntrar(token);
260       alEntrar(await sesionCompleta(token, avisarEspera));
261     } catch (e2) {
262       setErr(e2.message);
263     } finally {
264       setCargando(false);
265       setAvisoEspera("");
266     }
267   }
268
269   async function pedirRegistro(e) {
270     e?.preventDefault();

```

```

271     setErr("");
272     const errores = {};
273     if (!nombreCompleto.trim()) errores.nombreCompleto = "Digite su nombre completo.";
274     if (!correo.trim()) errores.correo = "Digite su correo.";
275     else if (!correo.includes("@") || !correo.includes(".")) errores.correo = "Ese correo no parece válido.";
276     if (!usuario.trim()) errores.usuario = "Elija un usuario.";
277     if (!clave) errores.clave = "Digite una clave.";
278     else if (!claveCumplePolitica(clave)) errores.clave = "La clave no cumple los requisitos de arriba.";
279     if (!clave2) errores.clave2 = "Confirme la clave.";
280     else if (clave !== clave2) errores.clave2 = "Las dos claves no coinciden.";
281     if (Object.keys(errores).length) return setErrCampo(errores);
282     setErrCampo({});
283     setCargando(true);
284     try {
285         const { destino } = await registrarse(usuario.trim(), clave, correo.trim(), nombreCompleto.trim());
286         setDestinoCodigo(destino);
287         setModo("registro-codigo");
288     } catch (e2) {
289         setErr(e2.message);
290     } finally {
291         setCargando(false);
292         setAvisoEspera("");
293     }
294 }
295
296 async function reenviarRegistro() {
297     setReenviando(true); setErr("");
298     try {
299         const { destino } = await reenviarCodigoRegistro(usuario.trim());
300         if (destino) setDestinoCodigo(destino);
301         setReenviado(true);
302     } catch (e2) {
303         setErr(e2.message);
304     } finally {
305         setReenviando(false);
306     }
307 }
308
309 async function confirmarYEntrar(e) {
310     e?.preventDefault();
311     setErr("");
312     if (!codigo.trim()) return setErrCampo({ codigo: "Digite el código de 6 dígitos." });
313     setErrCampo({});
314     setCargando(true);
315     try {
316         await confirmarRegistro(usuario.trim(), codigo.trim());
317         const r = await iniciarSesion(usuario.trim(), clave);
318         // (una cuenta recién registrada nunca tiene MFA activo todavía,
319         // r.mfaPendiente no aplica aquí)
320         const token = r.token;
321         // el nombre/correo se piden de nuevo a Cognito (no del formulario en
322         // memoria): si esta pantalla se llegó desde "reanudar confirmación"
323         // tras un login fallido (ver entrar()), nunca se llenó el formulario
324         // de registro en esta sesión del navegador - solo Cognito los tiene.
325         const atributos = await obtenerAtributosUsuario();
326         // crea la fila local (perfil auxiliar, siempre - ver main.py) con
327         // los datos personales que Cognito no guarda. Con conPaciencia porque
328         // esta es la PRIMERA llamada al backend de todo el registro: si el
329         // servicio estaba dormido (plan gratuito de Render), le toca a ella
330         // despertarlo, y sin reintento fallaba dejando la cuenta a medias -
331         // confirmada en Cognito pero sin fila local.
332         await conPaciencia(() => pedir("/api/registro-completado", {
333             method: "POST",
334             body: JSON.stringify({
335                 nombre_completo: atributos.nombreCompleto,
336                 codigo: codigoEmpleado.trim(),
337                 correo: atributos.correo,
338             })),
339         token), avisarEspera);
340         // antes de entrar del todo, se ofrece (opcional, se puede saltar)
341         // activar la verificación en dos pasos - más fácil que alguien la
342         // active aquí, recién registrado, que que se acuerde de ir a
343         // buscarla después en Mi perfil.
344         setSesionPendiente(await sesionCompleta(token, avisarEspera));

```

```

345     setModo("registro-mfa");
346   } catch (e2) {
347     setErr(e2.message);
348   } finally {
349     setCargando(false);
350     setAvisoEspera("");
351   }
352 }
353
354 async function pedirRecuperar(e) {
355   e?.preventDefault();
356   setErr("");
357   if (!usuario.trim()) return setErrCampo({ usuario: "Digite su usuario." });
358   setErrCampo({});
359   setCargando(true);
360   try {
361     const { destino } = await recuperarClave(usuario.trim());
362     setDestinoCodigo(destino);
363     setModo("recuperar-codigo");
364   } catch (e2) {
365     setErr(e2.message);
366   } finally {
367     setCargando(false);
368     setAvisoEspera("");
369   }
370 }
371
372 async function reenviarRecuperar() {
373   setReenviando(true); setErr("");
374   try {
375     const { destino } = await recuperarClave(usuario.trim());
376     if (destino) setDestinoCodigo(destino);
377     setReenviado(true);
378   } catch (e2) {
379     setErr(e2.message);
380   } finally {
381     setReenviando(false);
382   }
383 }
384
385 async function confirmarRecuperar(e) {
386   e?.preventDefault();
387   setErr("");
388   const errores = {};
389   if (!codigo.trim()) errores.codigo = "Digite el código de 6 dígitos.";
390   if (!claveNueva) errores.claveNueva = "Digite una clave nueva.";
391   else if (!claveCumplePolitica(claveNueva)) errores.claveNueva = "La clave no cumple los requisitos de arriba.";
392   if (!claveNueva2) errores.claveNueva2 = "Confirme la clave nueva.";
393   else if (claveNueva !== claveNueva2) errores.claveNueva2 = "Las dos claves no coinciden.";
394   if (Object.keys(errores).length) return setErrCampo(errores);
395   setErrCampo({});
396   setCargando(true);
397   try {
398     await confirmarNuevaClave(usuario.trim(), codigo.trim(), claveNueva);
399     // La fecha del cambio la anota el backend en el primer ingreso: aquí
400     // todavía no hay token con que pedirselo.
401     try { localStorage.setItem("cv_clave_recien_cambiada", "1"); } catch (_) {}
402     setClave(""); setCodigo(""); setClaveNueva(""); setClaveNueva2("");
403     setModo("login");
404     setErr("");
405   } catch (e2) {
406     setErr(e2.message);
407   } finally {
408     setCargando(false);
409     setAvisoEspera("");
410   }
411 }
412
413 const campo = { width: "100%", padding: "11px 13px", marginBottom: 8,
414                 border: "1px solid var(--borde)", borderRadius: 12 };
415
416 // <main> aquí también: en el ingreso no hay navegación que saltarse,
417 // pero un lector de pantalla sigue necesitando saber dónde empieza el
418 // contenido de la página.

```



```

419   return (
420     <main className="ingreso-fondo">
421       <div className="ingreso">
422         <div className="marca-cabecera">
423           
424           <h1>CuentaVoz</h1>
425           <i>Tu voz cuenta</i>
426           <p className="hackathon">Hackathon Colsubsidio x 30X</p>
427         </div>
428
429         {modo === "login" && (
430           <form className="form" onSubmit={entrar} noValidate>
431             <h2>Iniciar sesión</h2>
432             <p className="pista">Ingrese con su usuario corporativo.</p>
433
434             <label htmlFor="campo-usuario">Usuario</label>
435             <div className="campo-icone">
436               <Icono nombre="perfil" tam={18} />
437               <input
438                 id="campo-usuario"
439                 value={usuario}
440                 onChange={actualizar(setUsuario, "usuario")}
441                 autoComplete="username"
442                 aria-invalid={!errCampo.usuario}
443                 autoFocus
444               />
445             </div>
446             {errCampo.usuario && <span className="error-campo" role="alert">{errCampo.usuario}</span>}
447
448             <label htmlFor="campo-clave">Clave</label>
449             <div className="campo-icone">
450               <Icono nombre="candado" tam={18} />
451               <input
452                 id="campo-clave"
453                 type="password"
454                 value={clave}
455                 onChange={actualizar(setClave, "clave")}
456                 autoComplete="current-password"
457                 aria-invalid={!errCampo.clave}
458               />
459             </div>
460             {errCampo.clave && <span className="error-campo" role="alert">{errCampo.clave}</span>}
461
462             <div>
463               <span id="etiqueta-perfil" style={{ fontSize: ".8rem", color: "var(--grafito)" }}>
464                 Perfil
465               </span>
466               <p aria-live="polite" className="pista" style={{ minHeight: "1.1em", margin: "2px 0 4px" }}>
467                 {perfil ? `Perfil detectado: ${ETIQUETAS_PERFIL[perfil]}` : ""}
468               </p>
469               <div className="perfiles" role="group" aria-labelledby="etiqueta-perfil">
470                 <span className={perfil === "auxiliar" ? "sel" : ""}>{ETIQUETAS_PERFIL.auxiliar}</span>
471                 <span className={perfil === "auditor" ? "sel" : ""}>{ETIQUETAS_PERFIL.auditor}</span>
472               </div>
473             </div>
474
475             {err && <span className="error" role="alert">{err}</span>}
476             {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
477
478             <button className="btn" type="submit" disabled={cargando}>
479               {cargando ? "Entrando..." : "ENTRAR"}
480             </button>
481
482             <p className="enlaces-ingreso">
483               <button type="button" className="enlace" onClick={() => cambiarModo("recuperar")}>
484                 Olvidé mi clave
485               </button>
486               {" · "}
487               <button type="button" className="enlace" onClick={() => cambiarModo("registro")}>
488                 Crear una cuenta
489               </button>
490             </p>
491           </form>
492         )}

```

```

493
494 {modo === "login-mfa" && (
495   <form className="form" onSubmit={confirmarMFALogin} noValidate>
496     <h2>Verificación en dos pasos</h2>
497     <p className="pista">Digite el código de 6 dígitos de su app autenticadora.</p>
498
499     <label htmlFor="mfa-codigo">Código</label>
500     <input id="mfa-codigo" style={campo} value={codigoMFA} inputMode="numeric"
501       onChange={actualizar(setCodigoMFA, "codigoMFA")}
502       aria-invalid={!errCampo.codigoMFA} autoFocus />
503     {errCampo.codigoMFA && <span className="error-campo" role="alert">{errCampo.codigoMFA}</span>}
504
505     {err && <span className="error" role="alert">{err}</span>}
506     {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
507
508     <button className="btn" type="submit" disabled={cargando}>
509       {cargando ? "Verificando..." : "VERIFICAR Y ENTRAR"}
510     </button>
511   </form>
512 )}
513
514 {modo === "registro" && (
515   <form className="form" onSubmit={pedirRegistro} noValidate>
516     <h2>Crear una cuenta</h2>
517     <p className="pista">La cuenta queda como auxiliar de inventarios.</p>
518
519     <div className="campos-2col">
520       <div>
521         <label htmlFor="reg-nombre">Nombre completo</label>
522         <input id="reg-nombre" style={campo} value={nombreCompleto}
523           onChange={actualizar(setNombreCompleto, "nombreCompleto")}
524           aria-invalid={!errCampo.nombreCompleto} autoFocus />
525         {errCampo.nombreCompleto && <span className="error-campo" role="alert">{errCampo.nombreCompleto}
</span>}}
526       </div>
527       <div>
528         <label htmlFor="reg-codigo-empleado">Código de empleado (opcional)</label>
529         <input id="reg-codigo-empleado" style={campo} value={codigoEmpleado}
530           onChange={(e) => setCodigoEmpleado(e.target.value)} />
531       </div>
532     </div>
533
534     <div className="campos-2col">
535       <div>
536         <label htmlFor="reg-correo">Correo corporativo</label>
537         <input id="reg-correo" type="email" style={campo} value={correo}
538           onChange={actualizar(setCorreo, "correo")} autoComplete="email"
539           aria-invalid={!errCampo.correo} />
540         {errCampo.correo && <span className="error-campo" role="alert">{errCampo.correo}</span>}
541       </div>
542       <div>
543         <label htmlFor="reg-usuario">Usuario</label>
544         <input id="reg-usuario" style={campo} value={usuario}
545           onChange={actualizar(setUsuario, "usuario")} autoComplete="username"
546           aria-invalid={!errCampo.usuario} />
547         {errCampo.usuario && <span className="error-campo" role="alert">{errCampo.usuario}</span>}
548       </div>
549     </div>
550
551     <div className="campos-2col">
552       <div>
553         <label htmlFor="reg-clave">Clave</label>
554         <input id="reg-clave" type="password" style={campo} value={clave}
555           onChange={actualizar(setClave, "clave")} autoComplete="new-password"
556           aria-invalid={!errCampo.clave} />
557         {errCampo.clave && <span className="error-campo" role="alert">{errCampo.clave}</span>}
558       </div>
559       <div>
560         <label htmlFor="reg-clave2">Confirmar clave</label>
561         <input id="reg-clave2" type="password" style={campo} value={clave2}
562           onChange={actualizar(setClave2, "clave2")} autoComplete="new-password"
563           aria-invalid={!errCampo.clave2} />
564         {errCampo.clave2 && <span className="error-campo" role="alert">{errCampo.clave2}</span>}
565       </div>

```

```

566     </div>
567     <ChecklistClave clave={clave} />
568
569     {err && <span className="error" role="alert">{err}</span>}
570     {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
571
572     <button className="btn" type="submit" disabled={cargando}>
573       {cargando ? "Creando cuenta..." : "CREAR CUENTA"}
574     </button>
575     <p className="enlaces-ingreso">
576       <button type="button" className="enlace" onClick={() => cambiarModo("login")}>
577         Ya tengo una cuenta
578       </button>
579     </p>
580   </form>
581 )}
582
583 {modo === "registro-codigo" && (
584   <form className="form" onSubmit={confirmarYEntrar} noValidate>
585     <h2>Confirmar correo</h2>
586     <AvisoCodigo destino={destinoCodigo} onReenviar={reenviarRegistro}
587       reenviando={reenviando} reenviado={reenviado} />
588
589     <label htmlFor="reg-codigo">Código de 6 dígitos</label>
590     <input id="reg-codigo" style={campo} value={codigo} inputMode="numeric"
591       onChange={actualizar(setCodigo, "codigo")} aria-invalid={!errCampo.codigo} autoFocus />
592     {errCampo.codigo && <span className="error-campo" role="alert">{errCampo.codigo}</span>}
593
594     {err && <span className="error" role="alert">{err}</span>}
595     {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
596
597     <button className="btn" type="submit" disabled={cargando}>
598       {cargando ? "Confirmando..." : "CONFIRMAR Y ENTRAR"}
599     </button>
600   </form>
601 )}
602
603 {modo === "registro-mfa" && (
604   <div className="form">
605     <h2>¿Verificación en dos pasos?</h2>
606     <p className="pista">
607       Opcional: pida un código de su celular además de la clave, al ingresar. Se puede
608       activar o desactivar después desde Mi perfil.
609     </p>
610     <ConfigurarMFA nombreUsuario={usuario.trim()}
611       onActivado={terminarRegistro}
612       onCancelar={terminarRegistro}
613       textoCancelar="Ahora no" />
614   </div>
615 )}
616
617 {modo === "recuperar" && (
618   <form className="form" onSubmit={pedirRecuperar} noValidate>
619     <h2>Olvidé mi clave</h2>
620     <p className="pista">Le enviaremos un código al correo registrado.</p>
621
622     <label htmlFor="rec-usuario">Usuario</label>
623     <input id="rec-usuario" style={campo} value={usuario}
624       onChange={actualizar(setUsuario, "usuario")} autoComplete="username"
625       aria-invalid={!errCampo.usuario} autoFocus />
626     {errCampo.usuario && <span className="error-campo" role="alert">{errCampo.usuario}</span>}
627
628     {err && <span className="error" role="alert">{err}</span>}
629     {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
630
631     <button className="btn" type="submit" disabled={cargando}>
632       {cargando ? "Enviando..." : "ENVIAR CÓDIGO"}
633     </button>
634     <p className="enlaces-ingreso">
635       <button type="button" className="enlace" onClick={() => cambiarModo("login")}>
636         Volver a iniciar sesión
637       </button>
638     </p>
639   </form>

```

```

640     })
641
642     {modo === "recuperar-codigo" && (
643       <form className="form" onSubmit={confirmarRecuperar} noValidate>
644         <h2>Nueva clave</h2>
645         <AvisoCodigo destino={destinoCodigo} onReenviar={reenviarRecuperar}
646           reenviando={reenviando} reenviado={reenviado} />
647
648         <label htmlFor="rec-codigo">Código de 6 dígitos</label>
649         <input id="rec-codigo" style={campo} value={codigo} inputMode="numeric"
650           onChange={actualizar(setCodigo, "codigo")} aria-invalid={!errCampo.codigo} autoFocus />
651         {errCampo.codigo && <span className="error-campo" role="alert">{errCampo.codigo}</span>}
652
653         <label htmlFor="rec-clave">Clave nueva</label>
654         <input id="rec-clave" type="password" style={campo} value={claveNueva}
655           onChange={actualizar(setClaveNueva, "claveNueva")} autoComplete="new-password"
656           aria-invalid={!errCampo.claveNueva} />
657         {errCampo.claveNueva && <span className="error-campo" role="alert">{errCampo.claveNueva}</span>}
658         <ChecklistClave clave={claveNueva} />
659
660         <label htmlFor="rec-clave2">Confirmar clave nueva</label>
661         <input id="rec-clave2" type="password" style={campo} value={claveNueva2}
662           onChange={actualizar(setClaveNueva2, "claveNueva2")} autoComplete="new-password"
663           aria-invalid={!errCampo.claveNueva2} />
664         {errCampo.claveNueva2 && <span className="error-campo" role="alert">{errCampo.claveNueva2}</span>}
665
666         {err && <span className="error" role="alert">{err}</span>}
667         {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
668
669         <button className="btn" type="submit" disabled={cargando}>
670           {cargando ? "Guardando..." : "GUARDAR CLAVE NUEVA"}
671         </button>
672       </form>
673     })
674   </div>
675 </main>
676 );
677 }

```

frontend/src/vistas/Inicio.jsx

171 LÍNEAS

El resumen del día.

```

1 import { useEffect, useState } from "react";
2 import { pedir } from "../api";
3 import Marco from "../Marco";
4 import AsistenteVoz from "../AsistenteVoz";
5
6 // timeZone fijo en Bogotá (no el del navegador/equipo): CuentaVoz es para
7 // la operación de Colsubsidio en Colombia, así que la fecha y el saludo
8 // deben leerse en hora de Bogotá aunque el dispositivo esté mal
9 // configurado o alguien entre desde otro huso horario.
10 const HUSO = "America/Bogota";
11 // Función, no constante: si se calculara una sola vez al cargar el módulo
12 // (como antes), un turno que deja la pestaña abierta de un día para otro
13 // -habitual en una tablet de bodega siempre encendida- seguía mostrando la
14 // fecha de ayer en el encabezado hasta recargar la página a la fuerza.
15 function fechaHoy() {
16   return new Date().toLocaleDateString("es-CO", {
17     weekday: "long", day: "numeric", month: "long", year: "numeric",
18     timeZone: HUSO,
19   });
20 }
21
22 /** "Buenos días" antes de mediodía, "Buenas tardes" hasta las 7pm, y
23   "Buenas noches" después - siempre en hora de Bogotá. Sin esto el
24   saludo decía "Buenos días" a cualquier hora, incluso de noche. */
25 function saludoSegunHora() {
26   const hora = Number(
27     new Intl.DateTimeFormat("en-US", { timeZone: HUSO, hour: "numeric", hourCycle: "h23" })
28     .format(new Date())
29   );

```

```

30   if (hora < 12) return "Buenos días";
31   if (hora < 19) return "Buenas tardes";
32   return "Buenas noches";
33 }
34
35 // Debe cubrir las mismas acciones que registra backend/seguridad.py:registrar()
36 // (la misma lista que el filtro de Ajustes > Trazabilidad) - si aquí falta
37 // alguna, esa fila cae al gris por defecto aunque tenga su propio color en
38 // todas las demás pantallas, y hasta ahora faltaba más de la mitad,
39 // incluida CONTEO: la acción más frecuente del día a día.
40 const COLOR_ACCION = {
41   FIRMA: "verde", CIERRE: "verde", APROBACION: "verde", CREACION: "verde",
42   CONTEO: "verde",
43   REPORTE: "azul", INGRESO: "azul", APERTURA: "azul", PEDIDO: "azul",
44   RECETA: "azul", LEGALIZACION: "azul", ASIGNACION: "azul", USUARIO: "azul",
45   AJUSTE: "azul", PERFIL: "azul", SOPORTE: "azul",
46   ALERTA: "oro", CORRECCION: "oro", REAPERTURA: "oro", AUDITORIA: "oro",
47   SEGURIDAD: "oro",
48 };
49 const DOT = { verde: "var(--verde)", azul: "var(--azul)",
50               oro: "var(--amarillo)", gris: "var(--grafito)" };
51
52 export default function Inicio({ token, usuario, ir }) {
53   const [resumen, setResumen] = useState(null);
54   const [actividad, setActividad] = useState([]);
55   const esAuditor = usuario?.perfil === "auditor";
56
57   // Ejemplos de lo que ya entiende el agente aquí: preguntar por los KPI de
58   // esta pantalla, o pedir directo cualquiera de los accesos rápidos de
59   // abajo - decir su nombre ya navega, sin tener que tocarlo.
60   const ejemplos = [
61     "¿cuántas bodegas tengo?", "¿cuántas alertas hay por revisar?",
62     "¿cuál es mi exactitud del mes?", "iniciar un conteo", "ver el tablero",
63     ...(esAuditor ? ["continuar auditoría", "generar reporte"]
64       : ["hacer un pedido", "legalizar servicio"]);
65 ];
66
67   useEffect(() => {
68     pedir("/api/usuarios/yo/resumen", {}, token).then(setResumen).catch(() => {});
69     pedir("/api/trazabilidad/reciente", {}, token).then(setActividad).catch(() => {});
70   }, [token]);
71
72   // "Auditoría"/"Reportes"/"Panel" ni siquiera aparecen en el menú lateral
73   // de un auxiliar (soloAuditor en App.jsx:MENU) - ofrecerlos aquí como
74   // acceso directo llevaba a una pantalla que el propio perfil no puede
75   // usar, aunque el menu de al lado dijera lo contrario.
76   const accesos = esAuditor ? [
77     { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo", ic: "🗣️" },
78     { t: "Ver el tablero", s: "estado de las bodegas", d: "bodegas", ic: "📊" },
79     { t: "Continuar auditoría", s: "recuento ciego y aprobaciones", d: "auditoria", ic: "🔍️" },
80     { t: "Generar reporte", s: "consolidado del día", d: "reportes", ic: "📄" },
81   ] : [
82     { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo", ic: "🗣️" },
83     { t: "Ver el tablero", s: "estado de las bodegas", d: "bodegas", ic: "📊" },
84     { t: "Hacer un pedido", s: "receta y porciones por voz", d: "pedido", ic: "📄" },
85     { t: "Legalizar servicio", s: "sobrante y merma del turno", d: "legalizacion", ic: "👉" },
86   ];
87
88   return (
89     <Marco titulo={`Inicio · ${fechaHoy()}`}
90       chip={{ texto: "TOMA EN CURSO", tipo: "verde" }}>
91       <h2 style={{ fontSize: "1.15rem", color: "var(--azul)", marginBottom: 14 }}>
92         <saludoSegunHora(), <span style={{ textTransform: "capitalize" }}>{usuario?.nombre}</span>.
93         Esto es lo que hay para hoy.
94       </h2>
95
96       <AsistenteVoz token={token} vista="inicio" ir={ir} ejemplos={ejemplos}
97         placeholder="¿cuántas bodegas tengo?, lléveme a bodegas..." />
98
99       <div className="kpis">
100         <div className="kpi">
101           <div className="kpi-cabeza">
102             <span className="icono-kpi">📊</span>
103             <small>Bodegas asignadas a usted</small>

```

```

104     </div>
105     <b>{resumen?.bodegas_asignadas ?? "-"}</b>
106     <i>según su perfil</i>
107 </div>
108 <div className="kpi">
109     <div className="kpi-cabeza">
110         <span className="icono-kpi">📦</span>
111         <small>Referencias contadas hoy</small>
112     </div>
113     <b>{resumen?.referencias_hoy ?? "-"}</b>
114     <i>en sus sesiones de conteo</i>
115 </div>
116 <div className={`kpi ${resumen?.alertas_por_revisar ? "oro" : "verde"}`>
117     <div className="kpi-cabeza">
118         <span className="icono-kpi">{resumen?.alertas_por_revisar ? "⚠️" : "✅"}</span>
119         <small>Alertas por revisar</small>
120     </div>
121     <b>{resumen?.alertas_por_revisar ?? "-"}</b>
122     <i>en toda la operación</i>
123 </div>
124 <div className="kpi verde">
125     <div className="kpi-cabeza">
126         <span className="icono-kpi">📊</span>
127         <small>Su exactitud del mes</small>
128     </div>
129     <b>{resumen ? `${resumen.exactitud_mes} %` : "-"}</b>
130     <i>promedio de bodegas cerradas</i>
131 </div>
132 </div>
133
134 <div className="card">
135     <h2>¿Qué desea hacer?</h2>
136     <div className="accesos-grid">
137         {accesos.map((a) => (
138             <button key={a.d} className="acceso-rapido"
139                 onClick={() => ir(a.d)}>
140                 <span className="icono-accion">{a.ic}</span>
141                 <b>{a.t}</b>
142                 <small>{a.s}</small>
143             </button>
144         ))}
145     </div>
146 </div>
147
148 <div className="card">
149     <h2>Actividad reciente</h2>
150     {actividad.length === 0 ? (
151         <p className="vacio">Todavía no hay actividad registrada hoy.</p>
152     ) : (
153         actividad.map((a, i) => {
154             const color = COLOR_ACCION[a.accion] || "gris";
155             return (
156                 <div className="registro" key={i}>
157                     <span style={{ width: 9, height: 9, borderRadius: "50%", flex: "none",
158                         background: DOT[color] }} />
159                     <b style={{ color: "var(--grafito)", minWidth: 46 }}>{a.hora}</b>
160                     <span>{a.detalle}</span>
161                     <span className={`etiqueta-actividad ${color}`}>
162                         {a.accion.toLowerCase()}
163                     </span>
164                 </div>
165             );
166         })
167     )}
168 </div>
169 </Marco>
170 );
171 }

```

```

1 import { useState, useEffect } from "react";
2 import { pedir, enviarTurno, descargarReporte } from "../api";
3 import { escuchar, hablar } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import Icono from "../Iconos";
7
8 const DIA = new Date().toLocaleDateString("es-CO", { weekday: "long" });
9 // Palabras sueltas que valen como "sí" a lo que se le esté preguntando
10 // (confirmar un guardado, o enviar el pedido calculado).
11 const AFIRMACIONES = ["confirmo", "confirmar", "si", "sí", "claro", "listo", "dale",
12   "correcto", "hazlo", "adelante", "procede"];
13 // Frases de varias palabras que también confirman, tal como las dice una
14 // persona real ("así es", "va, mándalo") y no calzarían con el chequeo por
15 // palabra exacta de arriba.
16 const FRASES_AFIRMATIVAS = ["así es", "asi es", "eso es", "va pues", "de una"];
17
18 function quitarTildes(t) {
19   return t.normalize("NFD").replace(/[\u0300-\u036f]/g, "");
20 }
21
22 function esConfirmacion(texto) {
23   const t = quitarTildes(texto.toLowerCase()).replace(/[.,!¿?]/g, "");
24   const palabras = t.split(/\s+/);
25   if (palabras.includes("no")) return false; // "no lo envíes" no es un sí
26   if (palabras.some((p) => AFIRMACIONES.includes(quitarTildes(p)))) return true;
27   if (FRASES_AFIRMATIVAS.some((f) => t.includes(quitarTildes(f)))) return true;
28   // cualquier conjugación de "enviar"/"mandar" cuenta como un sí a la
29   // pregunta de mandar el pedido ("envíalo", "envíelo", "envíenmelo",
30   // "mándalo", "mándemelo"... ) - enumerar cada forma verbal a mano es
31   // interminable y siempre se queda corto; sin esto, decir la forma
32   // "usted" en vez de la forma "tú" hacía que la pantalla repitiera el
33   // mismo mensaje en vez de entender que la persona ya dijo que sí.
34   return /envi|mand/.test(t);
35 }
36
37 /** Un "no" limpio a lo que se le está preguntando (¿prepara otro plato?,
38   ¿lo enviamos?), sin que además mencione un plato nuevo - si mencionara
39   uno, no es un cierre sino un cambio de tema, y eso lo debe interpretar
40   Gemini como corresponde, no cerrarse en seco aquí. */
41 function esNegacionSimple(texto, platosConocidos) {
42   const t = quitarTildes(texto.toLowerCase()).replace(/[.,!¿?]/g, "");
43   const palabras = t.split(/\s+/);
44   if (!palabras.includes("no")) return false;
45   const mencionaOtroPlato = platosConocidos.some(
46     (nombre) => t.includes(quitarTildes(nombre.toLowerCase()))
47   );
48   return !mencionaOtroPlato;
49 }
50
51 // Pedidos se sale y se vuelve a entrar todo el tiempo (a mirar otra pantalla
52 // a mitad de un pedido, por ejemplo): sin guardar esto, cada regreso mostraba
53 // la pantalla en blanco como si la conversación nunca hubiera pasado, aunque
54 // el chef ya llevara todo un plato armado. Se guarda en sessionStorage (no
55 // sobrevive a cerrar la pestaña) y por persona, para que dos sesiones en el
56 // mismo navegador no se mezclen.
57 function claveEstadoPedido(sesionId) {
58   return `cv_pedido_estado_${sesionId}`;
59 }
60 function cargarEstadoPedido(sesionId) {
61   try { return JSON.parse(sessionStorage.getItem(claveEstadoPedido(sesionId))) || "{}"; }
62   catch { return {}; }
63 }
64
65 export default function Pedido({ token, usuario, ir }) {
66   const esAuditor = usuario?.perfil === "auditor";
67   // Pedidos tiene su propia conversación con el agente, aislada de la del
68   // conteo físico: usa un número de sesión negativo (único por persona) para
69   // que nunca choque con una sesión real de bodega (esas siempre son >= 1).
70   const sesionId = -1000 - (usuario?.id || 0);
71   const guardado = cargarEstadoPedido(sesionId);
72   // Vacío hasta que de verdad se dicte algo: mostrar «ajiacó/50» desde el
73   // arranque hacía parecer que ya se había entendido un pedido que nadie
74   // dictó todavía, y esos datos no cambiaban con nada más que se dijera.

```

```

75  const [plato, setPlato] = useState(guardado.plato || "");
76  const [porciones, setPorciones] = useState(guardado.porciones ?? null);
77  const [dicho, setDicho] = useState(guardado.dicho || "");
78  const [lineas, setLineas] = useState(guardado.lineas ?? null);
79  const [receta, setReceta] = useState(null);
80  const [enviado, setEnviado] = useState(guardado.enviado || false);
81  const [recibo, setRecibo] = useState(guardado.recibo ?? null);
82  const [respuesta, setRespuesta] = useState(
83    guardado.respuesta || "Diga el plato y las porciones: «hoy preparamos cincuenta ajiacos»."
84  );
85  const [estado, setEstado] = useState("listo");
86  const [err, setErr] = useState("");
87  const [archivo, setArchivo] = useState(guardado.archivo ?? null);
88  const [pedirPorciones, setPedirPorciones] = useState(false);
89  const [avisos, setAvisos] = useState(guardado.avisos ?? null);
90  const [opciones, setOpciones] = useState(guardado.opciones ?? null);
91  const [opcionesPara, setOpcionesPara] = useState(guardado.opcionesPara ?? null);
92  const [productoConsultado, setProductoConsultado] = useState(guardado.productoConsultado ?? null);
93  const [bodegas, setBodegas] = useState([]);
94  const [bodegaId, setBodegaId] = useState(guardado.bodegaId ?? null);
95  const [platos, setPlatos] = useState([]);
96  const [offline, setOffline] = useState(!navigator.online);
97
98  // a diferencia de Conteo (que solo guarda "conté X de Y" y sincroniza
99  // después), calcular un pedido necesita el stock EN VIVO de la bodega
100 // para saber cuánto falta pedir - no hay forma honesta de simularlo sin
101 // conexión sin arriesgar un cálculo con datos viejos. Por eso aquí no
102 // hay cola offline: solo se avisa con claridad que hay que esperar a
103 // tener señal, y lo que el chef ya dictó no se pierde (sigue guardado
104 // en sessionStorage, ver claveEstadoPedido arriba).
105 useEffect(() => {
106   const alVolver = () => setOffline(false);
107   const alPerder = () => setOffline(true);
108   window.addEventListener("online", alVolver);
109   window.addEventListener("offline", alPerder);
110   return () => {
111     window.removeEventListener("online", alVolver);
112     window.removeEventListener("offline", alPerder);
113   };
114 }, []);
115
116 // Guarda una foto de la conversación en cada cambio relevante, para que
117 // salir y volver a esta pantalla la deje tal cual como quedó.
118 useEffect(() => {
119   const foto = { plato, porciones, dicho, lineas, enviado, recibo, respuesta,
120     archivo, avisos, opciones, opcionesPara, productoConsultado, bodegaId };
121   sessionStorage.setItem(claveEstadoPedido(sesionId), JSON.stringify(foto));
122 }, [plato, porciones, dicho, lineas, enviado, recibo, respuesta,
123   archivo, avisos, opciones, opcionesPara, productoConsultado, bodegaId, sesionId]);
124
125 // para que el chef sepa que puede pedir sin tener que adivinar el nombre
126 // exacto de una receta que quizá no existe todavía.
127 useEffect(() => {
128   pedir("/api/recetas", {}, token).then(setPlatos).catch(() => {});
129 }, [token]);
130
131 function elegirPlato(nombre) {
132   setErr("");
133   setDicho(`«${nombre}» (elegido de la lista de recetas)`);
134   setPlato(nombre);
135   setLineas(null);
136   setReceta(null);
137   setEnviado(false);
138   setRecibo(null);
139   setProductoConsultado(null);
140   const msg = `Listo, ${nombre.toLowerCase()}. ¿Para cuántas porciones?`;
141   setRespuesta(msg);
142   hablar(msg);
143   setPedirPorciones(true);
144 }
145
146 // el agente saluda primero en vez de esperar a que el chef adivine que
147 // hay que hablar: sin esto el mensaje de bienvenida solo se veía en la
148 // pantalla, nunca se oía, y no parecía un asistente por voz de verdad.

```



```

149 // Ahora que la conversación se guarda (ver arriba), esto ya no necesita
150 // adivinar por tiempo: si ya hay un plato en curso (porque se restauró de
151 // una visita anterior), es un pedido que sigue abierto y no se interrumpe;
152 // si no hay nada, es un pedido nuevo de verdad y sí saluda.
153 useEffect(() => {
154   if (!plato) {
155     hablar("Dígame por favor qué va a preparar y para cuántas porciones.");
156   }
157   // eslint-disable-next-line react-hooks/exhaustive-deps
158 }, []);
159
160 // Con qué bodega se descuenta el pedido: por defecto el almacén de
161 // Alimentos y Bebidas (donde de verdad están los ingredientes de cocina),
162 // no una bodega al azar - antes quedaba fijo en la primera de la lista
163 // (una oficina administrativa sin ningún stock) y todo salía en cero.
164 // Si ya había una bodega elegida de una visita anterior, se respeta esa.
165 useEffect(() => {
166   pedir("/api/bodegas", {}, token).then((bs) => {
167     setBodegas(bs);
168     if (bodegaId) return;
169     const conStock = bs.find((b) => /ALMACEN\s*AYB/i.test(b.bodega))
170       || bs.find((b) => b.referencias > 0) || bs[0];
171     if (conStock) setBodegaId(conStock.id);
172   }).catch(() => {});
173   // eslint-disable-next-line react-hooks/exhaustive-deps
174 }, [token]);
175
176 // El recibo que se guarda en sessionStorage es una foto del momento en
177 // que se envió: si el administrador lo aprueba o lo rechaza después, esta
178 // pantalla no se entera sola. Sin esto, alguien podía recargar la página
179 // horas después de un rechazo real y seguir viendo "pendiente de
180 // aprobación", con CuentaVoz repitiendo que hay que esperar al
181 // administrador cuando eso ya no es cierto.
182 useEffect(() => {
183   if (!recibo || recibo.estado !== "pendiente_aprobacion") return;
184   pedir(`/api/pedidos/${recibo.numero_pedido}/estado`, {}, token)
185     .then((r) => {
186       if (r.estado === recibo.estado) return;
187       setRecibo((prev) => (prev ? { ...prev, estado: r.estado } : prev));
188       if (r.estado === "rechazado") {
189         const msg = `El administrador rechazó el pedido ${recibo.numero_pedido}. `
190           + "¿Desea armar otro pedido?";
191         setRespuesta(msg);
192         hablar(msg);
193       } else if (r.estado === "abierto") {
194         const msg = `El pedido ${recibo.numero_pedido} ya fue aprobado y enviado al almacén. `
195           + "¿Desea armar otro pedido?";
196         setRespuesta(msg);
197         hablar(msg);
198       }
199     })
200     .catch(() => {});
201   // eslint-disable-next-line react-hooks/exhaustive-deps
202 }, []);
203
204 async function verReceta(platoForzado) {
205   setErr("");
206   const pl = platoForzado ?? plato;
207   if (!pl) {
208     const msg = "Primero dígame qué va a preparar.";
209     setRespuesta(msg);
210     hablar(msg);
211     return;
212   }
213   try {
214     const r = await pedir(`/api/pedidos/receta?plato=${encodeURIComponent(pl)}`, {}, token);
215     if (!r.lineas) {
216       // todo lo que el agente muestra en pantalla también lo dice en voz -
217       // esto se quedaba solo en rojo, así que alguien sin mirar la
218       // pantalla no se enteraba de que la receta no se encontró.
219       const msg = `No encontré una receta para «${pl}». Pruebe con ajíaco o sancocho.`;
220       setErr(msg);
221       hablar(msg);
222       return;

```

```

223     }
224     setReceta(r);
225     if (r.faltantes_catalogo?.length) {
226         setAvisos({ faltantes: r.faltantes_catalogo, sinRegistro: [] });
227     }
228 } catch (e) {
229     setErr(e.message);
230     hablar(e.message);
231 }
232 }
233
234 /** La frase del chef la interpreta el agente. «mostrar» es lo que se ve
235     como «lo que dijo el chef»; «texto» es lo que de verdad se manda al
236     backend. Al tocar una tarjeta de opción se manda el código exacto,
237     nunca el nombre: si un nombre está contenido en el otro (ARROZ
238     dentro de ARROZ BASMATI), el agente no tiene con qué distinguirlos. */
239 async function dictar(texto, mostrar = texto) {
240     setErr("");
241     setDicho(`${mostrar}`);
242     setEstado("listo");
243
244     // «confirmo» aquí no es del conteo físico: es seguir con el pedido.
245     if (esConfirmacion(texto)) {
246         // Si lo último que dijo fue "no" a enviarlo, un "sí" suelto un rato
247         // después no debe mandarlo de golpe - puede ser un "sí" para otra
248         // cosa (confirmando que entendió, por ejemplo), no una segunda
249         // vuelta pidiendo enviarlo. Un "envíalo"/"mándalo" explícito sí
250         // pasa de una: no hay ambigüedad posible en eso.
251         const esExplicitoEnviar = /envi|mand/.test(quitarTildes(texto.toLowerCase()));
252         if (!esExplicitoEnviar && respuesta.includes("Para empezar uno nuevo")
253             && lineas && !enviado) {
254             const msg = "Para enviarlo dígame de nuevo con «envíalo», o dígame otro plato para uno nuevo.";
255             setRespuesta(msg);
256             hablar(msg);
257             return;
258         }
259         if (lineas && !enviado && porPedir === 0) {
260             // igual que el botón, que se oculta en este caso: no hay nada que
261             // enviar, así que "envíalo" no debe crear un pedido vacío.
262             const msg = "No hay nada que enviar: ya tiene todo en la bodega. ¿Prepara otro plato?";
263             setRespuesta(msg);
264             hablar(msg);
265             return;
266         }
267         if (lineas && !enviado) return enviar();
268         if (!lineas) return calcular();
269         const msg = "Este pedido ya se envió al almacén. ¿Desea armar otro pedido?";
270         setRespuesta(msg);
271         hablar(msg);
272         return;
273     }
274
275     // un "no" limpio (sin mencionar otro plato) cierra la conversación en
276     // vez de caer en Gemini, que sin una intención propia para "declinar"
277     // terminaba repitiendo el mismo mensaje una y otra vez.
278     if (esNegacionSimple(texto, platos.map((p) => p.nombre))) {
279         // si ya le habíamos contestado con la misma frase de cierre (porque
280         // ya dijo "no" antes, a "¿lo enviamos?" o a esta misma), un segundo
281         // "no" no debe repetirla - hay que dar el cierre final de una vez,
282         // no seguir insistiendo con la misma pregunta.
283         const yaSeDespidio = respuesta.includes("Para empezar uno nuevo")
284             || respuesta.includes("estaré pendiente");
285         const msg = yaSeDespidio
286             ? "Listo, gracias. Quedo pendiente si requiere consultar otro pedido. Que tenga un buen día."
287             : (lineas && !enviado && porPedir > 0)
288             // A propósito no termina en "¿sí o no?": un "sí" aquí se
289             // confundiría con el "sí" que envía el pedido (esConfirmacion
290             // más abajo lo manda directo con lineas && !enviado), justo lo
291             // opuesto de lo que la persona acaba de decir. En cambio se
292             // describe dónde queda el pedido y cómo empezar uno nuevo -
293             // nombrar otro plato ya lo hace, sin ambigüedad.
294             ? "Está bien, no lo envío por ahora; el pedido queda calculado aquí, "
295             + "pendiente de enviar cuando usted quiera. Para empezar uno nuevo, "
296             + "dígame otro plato."

```

```

297         : "Listo, gracias. Que tenga un buen día – aquí estaré pendiente si desea consultar otro pedido.";
298         setRespuesta(msg);
299         hablar(msg);
300         return;
301     }
302
303     try {
304         const t = await enviarTurno(texto, sesionId, token,
305             { opciones, opcionesPara, preparacion: plato, porciones });
306         // un plato nuevo es un tema distinto: la última consulta de producto
307         // ya no aplica y solo estorbaría junto al pedido que se está armando.
308         if (t.preparacion && t.porciones) setProductoConsultado(null);
309         if (t.preparacion && t.preparacion !== plato) {
310             // cambio de plato: las porciones y lo ya calculado eran del plato
311             // anterior, no de este - si no vienen porciones nuevas, se limpian
312             // en vez de dejar el numero viejo puesto como si ya se hubiera dicho.
313             setPlato(t.preparacion);
314             setPorciones(t.porciones || null);
315             setLineas(null);
316             setReceta(null);
317             setEnviado(false);
318             setRecibo(null);
319         } else if (t.porciones) {
320             setPorciones(t.porciones);
321         }
322         setRespuesta(t.respuesta_hablada || "");
323         setArchivo(t.archivo || null);
324         setOpciones(t.opciones || null);
325         setOpcionesPara(t.opciones_para || null);
326         if (t.producto_consultado) setProductoConsultado(t.producto_consultado);
327         if (t.receta) setReceta(t.receta);
328         // el chef ya dijo plato Y porciones en la misma frase: el agente dice
329         // "voy a consultar la receta y los insumos" en su respuesta, así que
330         // hay que calcular de una - si no, la pantalla se queda sin nada
331         // mientras el mensaje hablado ya prometió que se iba a hacer. Ese
332         // cálculo ya trae su propio mensaje hablado (calcular() más abajo);
333         // hablar también esta respuesta intermedia disparaba dos audios casi
334         // simultáneos que competían por la misma voz neuronal, y si uno de
335         // los dos fallaba caía en la voz de respaldo del navegador, sonaba
336         // como si hablaran dos personas distintas.
337         if (t.preparacion && t.porciones) {
338             calcular(t.porciones, t.preparacion);
339         } else {
340             hablar(t.respuesta_hablada);
341         }
342     } catch (e) {
343         setErr(e.message);
344     }
345 }
346
347 function corregirCantidad() {
348     setPedirPorciones(true);
349 }
350
351 function confirmarPorciones(v) {
352     setPedirPorciones(false);
353     if (v && !isNaN(Number(v))) {
354         const p = Number(v);
355         // cero o negativo no es una cantidad real de porciones - sin este
356         // chequeo, el campo lo dejaba pasar igual (aquí "0" es un string no
357         // vacío, así que sí entra al if de arriba) y el cálculo salía con
358         // "ya tiene todo, no hace falta pedir nada" para cualquier receta.
359         if (p <= 0) {
360             const msg = "Las porciones tienen que ser un número mayor a cero. ¿Cuántas son en realidad?";
361             setRespuesta(msg);
362             hablar(msg);
363             return;
364         }
365         setPorciones(p);
366         setLineas(null);
367         // sin esto, tras elegir un plato de la lista y poner las porciones no
368         // pasaba nada visible hasta darle aparte a "Calcular el pedido" - se
369         // sentía como si el numero no hubiera servido de nada.
370         calcular(p);

```

```

371     }
372 }
373
374 /** El backend explota la receta y descuenta el stock.
375     porcionesForzadas/platoForzado: para poder calcular con el valor recién
376     confirmado sin esperar al próximo render (si no, calcular() vería el
377     plato o las porciones viejas por el cierre de la función). */
378 async function calcular(porcionesForzadas, platoForzado) {
379     setErr("");
380     const p = porcionesForzadas ?? porciones;
381     const pl = platoForzado ?? plato;
382     if (!pl || !p) {
383         const msg = "Primero dígame qué va a preparar y para cuántas porciones.";
384         setRespuesta(msg);
385         hablar(msg);
386         return;
387     }
388     setEnviado(false);
389     setAvisos(null);
390     setRecibo(null);
391     try {
392         const r = await pedir("/api/pedidos/calcular", {
393             method: "POST",
394             body: JSON.stringify({ plato: pl, porciones: Number(p), bodega_id: bodegaId }),
395             token;
396         }, token);
397         if (!r.receta_encontrada) {
398             // no se queda solo con el texto rojo en silencio: cierra la
399             // conversación con una salida clara en vez de dejar la pantalla
400             // varada con un plato que no tiene receta y ningún paso siguiente.
401             const msg = `No encontré una receta para «${pl}». Elija uno de los platos de la lista o dígame otro nombre.`;
402             setErr(msg);
403             setRespuesta(msg);
404             hablar(msg);
405             setPlato("");
406             setPorciones(null);
407             return;
408         }
409         setLineas(r.lineas);
410         if (r.faltantes_catalogo?.length || r.sin_registro_bodega?.length) {
411             setAvisos({ faltantes: r.faltantes_catalogo, sinRegistro: r.sin_registro_bodega });
412         }
413         const faltan = r.lineas.filter((l) => l.falta > 0).length;
414         const msg = faltan === 0
415             ? `Ya tiene los ${r.lineas.length} insumos que necesita en la bodega; no hace falta pedir nada al almacén.
416 ¿Prepara otro plato?`
417             : `Necesita ${r.lineas.length} insumos. Hay que pedir ${faltan} al almacén; el resto ya está en la bodega.
418 ¿Lo enviamos?`;
419         setRespuesta(msg);
420         hablar(msg);
421         // ya se calculó el pedido con esta receta: de una se muestra también,
422         // sin que la persona tenga que pedirla aparte con "Ver la receta".
423         verReceta(pl);
424     } catch (e) {
425         setErr(e.message);
426         hablar(e.message);
427     }
428 }
429
430 async function enviar() {
431     try {
432         const r = await pedir("/api/pedidos/enviar", {
433             method: "POST",
434             body: JSON.stringify({ servicio_id: 1, plato, porciones, bodega_id: bodegaId }),
435             token;
436         }, token);
437         if (r.duplicado) {
438             const msg = "Este pedido ya se había enviado antes; no lo mandé dos veces. ¿Desea armar otro pedido?";
439             setRespuesta(msg);
440             hablar(msg);
441             setEnviado(true);
442             return;
443         }
444         const msg = r.estado === "pendiente_aprobacion"
445             ? `Pedido ${r.numero_pedido} registrado. Queda pendiente de que el administrador lo apruebe antes de salir
446 del almacén. ¿Desea armar otro pedido?`

```

```

442      : `Pedido ${r.numero_pedido} registrado y enviado a ${r.bodega || "el almacén"}. ¿Desea armar otro pedido?`;
443      setRespuesta(msg);
444      hablar(msg);
445      setEnviado(true);
446      setRecibo(r);
447    } catch (e) {
448      setErr(e.message);
449      hablar(e.message);
450    }
451  }
452
453  const porPedir = lineas ? lineas.filter((l) => l.falta > 0).length : 0;
454
455  return (
456    <Marco titulo="Pedidos · Cocina Piscilago"
457      chip={offline ? { texto: "SIN CONEXIÓN", tipo: "oro" }
458        : { texto: "SERVICIO ALMUERZO", tipo: "" }}>
459      {offline && (
460        <div className="banner">
461          <span className="ico">!</span>
462          <span>
463            <b>Sin conexión</b>
464            <span>Calcular un pedido necesita el stock en vivo de la bodega, así que
465              no se puede hacer sin señal. Lo que ya dictó no se pierde - siga
466              cuando vuelva la conexión y retome donde quedó.</span>
467          </span>
468        </div>
469      )}
470      <div className="chips">
471        <span className="chip">{DIA[0].toUpperCase() + DIA.slice(1)} · almuerzo</span>
472        <span className="chip">Cocina Piscilago</span>
473        <span className={`chip ${!enviado ? "oro"
474          : recibo?.estado === "rechazado" ? "rojo"
475          : recibo?.estado === "pendiente_aprobacion" ? "oro" : "verde"}`}>
476          {!enviado ? "Pedido sin enviar"
477            : recibo?.estado === "rechazado" ? "Pedido rechazado"
478            : recibo?.estado === "pendiente_aprobacion" ? "Pendiente de aprobación"
479            : "Pedido enviado"}
480        </span>
481        {bodegas.length > 0 && (
482          <select value={bodegaId || ""}
483            onChange={(e) => { setBodegaId(Number(e.target.value)); setLineas(null); }}
484            aria-label="Bodega de la que se descuenta el pedido"
485            style={{ marginLeft: "auto", padding: "6px 12px", border: "1px solid var(--borde)",
486              borderRadius: 20, fontSize: ".8rem", fontWeight: 700, color: "var(--azul)" }}>
487            {bodegas.map((b) => (
488              <option key={b.id} value={b.id}>Se descuenta de: {b.bodega}</option>
489            ))}
490          </select>
491        )}
492      </div>
493
494      <div className="conteo-cols">
495        <div className="card">
496          <h2>Lo que dijo el chef</h2>
497          <p className="cita">
498            {dicho || "Aún no ha dictado nada: pulse el micrófono y diga, por "
499              + "ejemplo, «hoy preparamos cincuenta ajiacos»."}
500          </p>
501
502          <h3 style={{ marginTop: 18 }}>Lo que entendió CuentaVoz</h3>
503          {(productoConsultado || (plato && porciones)) ? (
504            <table>
505              <tbody>
506                {productoConsultado && (
507                  <tr><td>Último producto consultado</td><td><b style={{ color: "var(--azul)" }}>
508                    {productoConsultado.nombre}</b></td></tr>
509                  <tr><td>Código · unidad</td><td><b style={{ color: "var(--azul)" }}>
510                    {productoConsultado.codigo} · {productoConsultado.unidad}</b></td></tr>
511                )}
512              </tbody>
513            </table>
514            {plato && porciones && (
515              <

```

```

516         <tr><td>Preparación</td><td><b style={{ color: "var(--azul)" }}>
517             {plato.toUpperCase()}</b></td></tr>
518         <tr><td>Porciones</td><td><b style={{ color: "var(--azul)" }}>
519             {porciones}</b></td></tr>
520         <tr><td>Receta aplicada</td><td><b style={{ color: "var(--azul)" }}>
521             estándar de Colsubsidio</b></td></tr>
522         <tr><td>Momento</td><td><b style={{ color: "var(--azul)" }}>
523             pedido al almacén</b></td></tr>
524     </>
525     )}
526 </tbody>
527 </table>
528 ) : (
529     <p className="vacío">Todavía no hay ningún plato dictado en esta conversación.</p>
530 )}
531 </div>
532
533 <div className="card mic-caja">
534     <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
535         onClick={() => escuchar({
536             alTexto: dictar,
537             alEstado: setEstado,
538             alError: setErr,
539         })}
540         aria-label="Hable para decir el plato y las porciones">
541         <Icono nombre="microfono" tam={52} />
542     </button>
543     <b>{estado === "escuchando" ? "Escuchando..." : "Mantenga presionado y hable"}</b>
544     <small>También sirve: «pedir insumos para 30 sancochos»</small>
545     <small style={{ color: "var(--verde)", fontWeight: 700 }}>
546         Receta + stock = pedido
547     </small>
548     <details style={{ marginTop: 4, textAlign: "left", width: "100%" }}>
549         <summary className="pista" style={{ cursor: "pointer" }}>
550             Ver frases exactas que entiende el agente aquí
551         </summary>
552         <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
553             [{"hoy preparamos cincuenta ajiacos", "pedir insumos para 30 sancochos",
554             "cambia a treinta porciones", "sí", "envíalo", "no",
555             "corregir cantidad"].map((ej, i) => (
556                 <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
557             ))}
558         </ul>
559     </details>
560 </div>
561 </div>
562
563 {platos.length > 0 && (
564     <div className="card">
565         <h2>🍽️ Platos con receta registrada</h2>
566         <p className="pista" style={{ marginBottom: 10 }}>
567             Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno
568             para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.
569         </p>
570         <div style={{ display: "flex", flexWrap: "wrap", gap: 8 }}>
571             {platos.map((r) => (
572                 <button key={r.id}
573                     className={`chip ${plato.toUpperCase() === r.nombre.toUpperCase() ? "azul" : ""}`}
574                     onClick={() => elegirPlato(r.nombre)}>
575                     {r.nombre.toUpperCase()}
576                 </button>
577             ))}
578         </div>
579     </div>
580 )}
581
582 <div className="card">
583     <p className="rotulo">CuentaVoz responde</p>
584     <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
585     {opciones && (
586         <div className="opciones" style={{ marginTop: 14 }}>
587             {opciones.map((o, i) => (
588                 <button key={o.codigo} className="opcion" onClick={() => dictar(o.codigo, o.nombre)}>
589                 <span className="n">Opción {i + 1}</span>

```

```

590         <h4>{o.nombre}</h4>
591         <p>Código {o.codigo}</p>
592         <p>{o.unidad}</p>
593     </button>
594 }}
595 </div>
596 }}
597 {archivo && (
598     <div className="grilla-botones" style={{ marginTop: 10 }}>
599         <button className="btn verde"
600             onClick={() => descargarReporte(archivo, token).catch((e) => setErr(e.message))}
601             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
602             <Icono nombre="descargar" tam={16} />
603             Descargar archivo generado
604         </button>
605     </div>
606 )}
607 {err && <p className="error" style={{ marginTop: 10 }}>{err}</p>}
608 <div className="grilla-botones">
609     <button className="btn" onClick={() => calcular()} disabled={offline}
610         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
611         <Icono nombre="pedidos" tam={16} />
612         Calcular el pedido
613     </button>
614     <button className="btn borde" onClick={corregirCantidad}
615         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
616         <Icono nombre="refrescar" tam={16} />
617         Corregir cantidad
618     </button>
619     {!receta && (
620         <button className="btn oro" onClick={() => verReceta()}
621             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
622             <Icono nombre="reportes" tam={16} />
623             Ver la receta
624         </button>
625     )}
626 </div>
627 </div>
628
629 {avisos && (avisos.faltantes.length > 0 || avisos.sinRegistro.length > 0) && (
630     <div className="banner">
631         <span className="ico">!</span>
632         <span>
633             <b>Revise antes de enviar</b>
634             {avisos.faltantes.length > 0 && (
635                 <span>
636                     No encontré en el catálogo: <b>{avisos.faltantes.join(", ")}</b> – esos
637                     ingredientes de la receta no se pudieron calcular. Avise al administrador.
638                 <br />
639                 </span>
640             )}
641             {avisos.sinRegistro.length > 0 && (
642                 <span>
643                     Esta bodega nunca ha tenido registro de: <b>{avisos.sinRegistro.join(", ")}</b> –
644                     se asumió 0 disponible; confirme con el almacén antes de descontar existencias.
645                 </span>
646             )}
647         </span>
648     </div>
649 )}
650
651 {receta && (
652     <div className="card">
653         <div className="chips">
654             <span className="chip">{receta.nombre}</span>
655             <span className="chip">Rendimiento: {receta.rendimiento} porción</span>
656             <span className="chip">{receta.lineas.length} ingredientes</span>
657             <span className={`chip ${esAuditor ? "azul" : "gris"}`}>
658                 {esAuditor ? "ADMINISTRA ESTA RECETA" : "SOLO CONSULTA"}
659             </span>
660         </div>
661         <h2>Catálogo de Colsubsidio</h2>
662         <table>
663             <thead>

```

```

664         <tr><th>Ingrediente</th><th>Unidad</th><th>Por porción</th>
665         <th>Para {porciones} porciones</th></tr>
666     </thead>
667     <tbody>
668         {receta.lineas.map((l) => (
669             <tr key={l.codigo}>
670                 <td>{l.nombre}</td><td>{l.unidad}</td><td>{l.por_porcion}</td>
671                 <td className="dif">{Math.round(l.por_porcion * porciones * 1000) / 1000}</td>
672             </tr>
673         )]}
674     </tbody>
675 </table>
676 {receta.preparacion ? (
677     <>
678         <h3 style={{ marginTop: 18 }}>Preparación paso a paso</h3>
679         <p style={{ whiteSpace: "pre-line", fontSize: ".92rem" }}>{receta.preparacion}</p>
680     </>
681 ) : (
682     <p className="pista" style={{ marginTop: 18 }}>
683         Esta receta todavía no tiene preparación paso a paso cargada.
684         {esAuditor && " Agréguela desde Ajustes → Recetas."}
685     </p>
686 )}
687 <p className="pista" style={{ marginTop: 10 }}>
688     {esAuditor
689     ? "Como administradora puede editar cantidades, agregar o quitar ingredientes, o crear un plato nuevo
desde Ajustes → Recetas."
690     : "Esta receta la mantiene el administrador; si algo está desactualizado, avísele desde Reportes."}
691 </p>
692 <div className="grilla-botones">
693     <button className="btn" onClick={() => { setReceta(null); calcular(); }}
694         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
695         <Icono nombre="pedidos" tam={16} />
696         Usar para un pedido
697     </button>
698     {esAuditor && (
699         <button className="btn borde"
700             onClick={() => ir && ir("ajustes", { tabInicial: "recetas", recetaId: receta.id })}
701             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
702             <Icono nombre="ajustes" tam={16} />
703             Editar receta
704         </button>
705     )}
706     <button className="btn gris" onClick={() => setReceta(null)}
707         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
708         <Icono nombre="salir" tam={16} />
709         Cerrar
710     </button>
711 </div>
712 </div>
713 )}
714
715 {lineas && (
716     <>
717         <div className="kpi">
718             <div className="kpi">
719                 <div className="kpi-cabeza">
720                     <span className="icono-kpi">🍲</span>
721                     <small>Insumos de la receta</small>
722                 </div>
723                 <b>{lineas.length}</b>
724                 <i>ingredientes estándar</i>
725             </div>
726             <div className="kpi verde">
727                 <div className="kpi-cabeza">
728                     <span className="icono-kpi">✅</span>
729                     <small>Ya disponibles en bodega</small>
730                 </div>
731                 <b>{lineas.length - porPedir}</b><i>no se piden</i>
732             </div>
733             <div className="kpi oro">
734                 <div className="kpi-cabeza">
735                     <span className="icono-kpi">👤</span>
736                     <small>Hay que pedir al almacén</small>

```



```

737         </div>
738         <b>{porPedir}</b>
739         <i>faltante calculado</i>
740     </div>
741 </div>
742
743 <div className="card">
744     <h2>Insumos calculados para {porciones} {plato}</h2>
745     <table>
746         <thead>
747             <tr>
748                 <th>Insumo (nombre oficial)</th><th>Unidad</th>
749                 <th>Necesario</th><th>Hay en bodega</th><th>Falta: pedir</th>
750             </tr>
751         </thead>
752         <tbody>
753             {lineas.map((l) => (
754                 <tr key={l.codigo}>
755                     <td>{l.nombre}</td>
756                     <td>{l.unidad}</td>
757                     <td>{l.necesario}</td>
758                     <td>{l.stock}</td>
759                     <td className={l.falta > 0 ? "falta" : ""}>
760                         {l.falta > 0 ? l.falta : "-"}
761                     </td>
762                 </tr>
763             ))}
764         </tbody>
765     </table>
766     <p className="pista" style={{ marginTop: 10 }}>
767         La cantidad necesaria sale de la receta por porción; el faltante es
768         lo necesario menos lo que ya hay en la bodega.
769     </p>
770     {porPedir === 0 ? (
771         <p className="msg-ok" style={{ marginTop: 10 }}>
772              Ya tiene todo en la bodega: no hace falta enviar nada al almacén.
773         </p>
774     ) : !enviado && (
775         <div className="grilla-botones">
776             <button className="btn verde" onClick={enviar} disabled={offline}
777                 style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
778                 <Icono nombre="bodegas" tam={16} />
779                 Enviar pedido al almacén
780             </button>
781         </div>
782     )}
783 </div>
784 </>
785 ))
786
787 {recibo && (
788     <div className="card" style={{ border: `1px solid var(--${
789         recibo.estado === "rechazado" ? "rojo"
790         : recibo.estado === "pendiente_aprobacion" ? "amarillo" : "verde"`) }}>
791         <div className="chips">
792             {recibo.estado === "rechazado" ? (
793                 <span className="chip rojo">✖ RECHAZADO POR EL ADMINISTRADOR</span>
794             ) : recibo.estado === "pendiente_aprobacion" ? (
795                 <span className="chip oro">⚠ PENDIENTE DE APROBACIÓN</span>
796             ) : (
797                 <span className="chip verde">✅ PEDIDO REGISTRADO</span>
798             )}
799             <span className="chip">{recibo.numero_pedido}</span>
800         </div>
801         <h2 style={{ marginTop: 12 }}>Recibo de pedido</h2>
802         <table>
803             <tbody>
804                 <tr><td>Número de pedido</td><td><b style={{ color: "var(--verde)" }}>
805                     {recibo.numero_pedido}</b></td></tr>
806                 <tr><td>Plato</td><td><b>{plato.toUpperCase()}</b> · {porciones} porciones</td></tr>
807                 <tr><td>Bodega de descuento</td><td><b>{recibo.bodega || "-"}</b></td></tr>
808                 <tr><td>Registrado por</td><td style={{ textTransform: "capitalize" }}>{recibo.persona}</td></tr>
809                 <tr><td>Hora</td><td>{recibo.hora}</td></tr>
810                 <tr><td>Insumos solicitados</td><td>{recibo.total_lineas}</td></tr>

```

```

811         </tbody>
812     </table>
813     {recibo.items?.length > 0 && (
814         <>
815         <h3 style={{ marginTop: 16 }}>Detalle enviado al almacén</h3>
816         <table>
817             <thead><tr><th>Insumo</th><th>Cantidad</th><th>Unidad</th></tr></thead>
818             <tbody>
819                 {recibo.items.map((it, i) => (
820                     <tr key={i}><td>{it.nombre}</td><td>{it.cantidad}</td><td>{it.unidad}</td></tr>
821                 ))}
822             </tbody>
823         </table>
824     </>
825     )}
826     <p className="pista" style={{ marginTop: 10 }}>
827         {recibo.estado === "rechazado"
828         ? "El administrador de bodega rechazó este pedido: no salió nada del almacén. Dígame el plato de nuevo
para armar uno nuevo."
829         : recibo.estado === "pendiente_aprobacion"
830         ? "El administrador de bodega debe aprobar este pedido en Auditoría antes de que cuente como enviado de
verdad al almacén."
831         : "Este pedido queda guardado en el sistema y aparecerá tal cual en la legalización del servicio,
comparando lo pedido contra lo consumido."}
832     </p>
833     <div className="grilla-botones">
834         <button className="btn gris" onClick={() => window.print()}>Imprimir recibo</button>
835     </div>
836 </div>
837 )}
838
839 {pedirPorciones && (
840     <Dialogo titulo={porciones ? "Corregir cantidad" : "Porciones"}
841         mensaje={porciones ? "¿Cuántas porciones son en realidad?"
842             : `¿Para cuántas porciones prepara ${plato.toLowerCase()}?`}
843         conCampo conVoz tipo="number" valorInicial={porciones ?? ""}
844         onAceptar={confirmarPorciones}
845         onCancel={(() => setPedirPorciones(false))} />
846     )}
847 </Marco>
848 );
849 }

```

frontend/src/vistas/Conteo.jsx

1225 LÍNEAS

El corazón del sistema: dictar, confirmar, corregir, y el modo sin conexión.

```

1 import { useState, useEffect, useRef } from "react";
2 import { enviarTurno, abrirBodega, buscarBodega, pedir, descargarReporte, esFalloRed } from "../api";
3 import { escuchar, hablar, vozDisponible, esAfirmacion, esNegacion } from "../voz";
4 import { interpretarLocal } from "../interpreteLocal";
5 import { leerCola, guardarCola } from "../colaOffline";
6 import Marco from "../Marco";
7 import Dialogo from "../Dialogo";
8 import Icono from "../Iconos";
9
10 const UNIDADES = ["Unidad", "Kilogram", "Liter", "Portion"];
11 const CLAVE_CATALOGO = "cv_catalogo_ligero";
12
13 function leerCatalogo() {
14     try { return JSON.parse(localStorage.getItem(CLAVE_CATALOGO) || "[]"); }
15     catch (_) { return []; }
16 }
17 function guardarCatalogo(lista) {
18     try { localStorage.setItem(CLAVE_CATALOGO, JSON.stringify(lista)); } catch (_) {}
19 }
20 function normalizarLocal(t) {
21     return (t || "").toUpperCase().normalize("NFD").replace(/[~]/g, "").trim();
22 }
23 /** Sugerencia sin internet: coincidencia de palabras contra el catálogo
24     que ya se guardó en el equipo mientras había señal. */

```

```

25 function sugerirDeCatalogo(texto, catalogo) {
26   const q = normalizarLocal(texto);
27   const palabras = q.split(/\s+/).filter((p) => p.length >= 3);
28   if (!palabras.length || !catalogo.length) return null;
29   let mejor = null, mejorPuntaje = 0;
30   for (const a of catalogo) {
31     const nombre = normalizarLocal(a.nombre);
32     const aciertos = palabras.filter((p) => nombre.includes(p)).length;
33     const puntaje = aciertos / palabras.length;
34     if (puntaje > mejorPuntaje) { mejorPuntaje = puntaje; mejor = a; }
35   }
36   return mejorPuntaje >= 0.5 ? mejor : null;
37 }
38
39 const ETIQUETA_BODEGA = {
40   pendiente: ["PENDIENTE", "gris"], en_conteo: ["EN CONTEO", "azul"],
41   en_auditoria: ["EN AUDITORÍA", "oro"], cerrada: ["CERRADA", "verde"],
42 };
43
44 export default function Conteo({ token, sesionId, ir, usuario, bodegaSugerida, alAbrirBodega }) {
45   const [bodega, setBodega] = useState(null);
46   const [asignadas, setAsignadas] = useState(null);
47   const [todas, setTodas] = useState(null);
48   // Llega desde Bodegas cuando alguien buscó por voz/texto el nombre de
49   // una bodega en el buscador de INGREDIENTES (no encontró artículo, pero
50   // el nombre sí es una bodega real) - deja el nombre ya listo para
51   // confirmar con "Abrir por nombre exacto" en vez de tener que dictarlo
52   // otra vez desde cero.
53   const [textoBodega, setTextoBodega] = useState(bodegaSugerida || "");
54   const [buscarOtra, setBuscarOtra] = useState(false);
55   const [bodegaNoEncontrada, setBodegaNoEncontrada] = useState(null);
56   const [opcionesBodega, setOpcionesBodega] = useState(null);
57   // El nombre real que se le preguntó "¿confirma que abro X?" y sigue sin
58   // contestar - se guarda aparte del listener de voz porque el
59   // reconocimiento del navegador se apaga solo tras un rato de silencio: si
60   // la persona se demora en responder, ese listener ya no existe cuando por
61   // fin dice "sí", así que un segundo toque al micrófono debe seguir
62   // tratando esa respuesta como el sí/no pendiente, no como un nombre de
63   // bodega nuevo (antes terminaba ofreciendo crear una bodega llamada "sí").
64   const [bodegaParaConfirmar, setBodegaParaConfirmar] = useState(null);
65   const [msgBodega, setMsgBodega] = useState("");
66   const [dicho, setDicho] = useState("");
67   const [respuesta, setRespuesta] = useState(
68     "Abra una bodega para empezar a contar."
69   );
70   const [opciones, setOpciones] = useState(null);
71   const [opcionesPara, setOpcionesPara] = useState(null);
72   const [alerta, setAlerta] = useState(null);
73   const [alertaEsperado, setAlertaEsperado] = useState(null);
74   const [contextoAlerta, setContextoAlerta] = useState(null);
75   const [pendiente, setPendiente] = useState(null);
76   const [avance, setAvance] = useState({ hechas: 0, total: 0, alertas: 0, ultimos: [] });
77   const [estado, setEstado] = useState("listo");
78   const [err, setErr] = useState("");
79   const [crear, setCrear] = useState(null); // {nombre, unidad_medida, cantidad_inicial}
80   const [archivo, setArchivo] = useState(null);
81   const [mostrarTeclado, setMostrarTeclado] = useState(false);
82   // La firma del conteo: cerrar una bodega exige DOS firmas, la de quien
83   // contó y la de quien auditó. La segunda la pone Auditoría; la primera
84   // no la ponía ninguna pantalla, así que ninguna bodega contada llegaba a
85   // poder cerrarse (el backend respondía 409 "Faltan firmas").
86   const [confirmarFirma, setConfirmarFirma] = useState(false);
87   const [firmado, setFirmado] = useState(false);
88   const [offline, setOffline] = useState(!navigator.onLine);
89   const [cola, setCola] = useState(() => leerCola(sesionId));
90   const rec = useRef(null);
91   // Cada vez que se abre un micrófono para el flujo de bodega (nombre
92   // dictado, "¿cuál de todas?", "¿confirma?") sube en 1: el reconocedor
93   // anterior a veces tarda en apagarse del todo y llega a entregar un
94   // resultado extra y tardío (p. ej. recoger también el "confirmando" de la
95   // pregunta siguiente). Cada escucha guarda SU número; si para cuando
96   // responde ya no es el número vigente, se descarta sin hacer nada -
97   // así lo tardío nunca se confunde con una bodega nueva ni con una
98   // respuesta a la pregunta equivocada.

```

```

99   const idEscuchaBodega = useRef(0);
100
101   // guarda el catalogo liviano en el equipo mientras hay señal, para que
102   // el modo sin conexión pueda seguir sugiriendo nombres oficiales.
103   useEffect(() => {
104     if (navigator.online && token) {
105       pedir("/api/articulos/catalogo-ligero", {}, token).then(guardarCatalogo).catch(() => {});
106     }
107   }, [token]);
108
109   async function refrescar() {
110     try {
111       const bid = bodega?.bodega_id ? `?bodega_id_respaldo=${bodega.bodega_id}` : "";
112       setAvance(await pedir(`/api/sesiones/${sesionId}/avance${bid}`, {}, token));
113     } catch (_) {}
114   }
115   useEffect(() => {
116     refrescar();
117     // si quedaron registros sin sincronizar de una sesión anterior (se
118     // cerró la pestaña, o se recargó ya con señal) y ahora sí hay
119     // conexión, se reintenta al entrar en vez de esperar a que el
120     // navegador dispare otro evento "online" que quizás no vuelva a pasar.
121     if (navigator.online) sincronizar();
122     // eslint-disable-next-line react-hooks/exhaustive-deps
123   }, [bodega]);
124
125   useEffect(() => {
126     pedir("/api/usuarios/yo/bodegas", {}, token).then((r) => {
127       setAsignadas(r);
128       // sin bodegas propias (el caso típico del administrador, que audita
129       // en vez de contar rutinariamente) se necesita el listado completo
130       // de una vez - si no, la única opción es escribir el nombre a mano.
131       if (r.length === 0) cargarTodas();
132     }).catch(() => setAsignadas([]));
133   }, [token]);
134
135   function cargarTodas() {
136     pedir("/api/bodegas", {}, token).then(setTodas).catch(() => setTodas([]));
137   }
138
139   useEffect(() => {
140     const alVolver = () => { setOffline(false); sincronizar(); };
141     const alPerder = () => setOffline(true);
142     window.addEventListener("online", alVolver);
143     window.addEventListener("offline", alPerder);
144     return () => {
145       window.removeEventListener("online", alVolver);
146       window.removeEventListener("offline", alPerder);
147     };
148     // eslint-disable-next-line react-hooks/exhaustive-deps
149   }, [sesionId, bodega]);
150
151   async function sincronizar() {
152     const restantes = leerCola(sesionId);
153     if (!restantes.length || !bodega) return;
154     // solo se quita de la cola lo que de verdad se confirmó: si un ítem
155     // falla (o vuelve a caerse la red a mitad de la sincronización), el
156     // resto queda guardado para el próximo intento en vez de perderse -
157     // antes, cualquier falla borraba TODA la cola sin distinguir qué
158     // alcanzó a guardarse.
159     while (restantes.length) {
160       const item = restantes[0];
161       try {
162         await enviarTurno(`${item.articulo} ${item.cantidad} ${item.unidad}`, sesionId, token);
163         await enviarTurno("confirmando", sesionId, token);
164       } catch (_) {
165         break;
166       }
167       restantes.shift();
168       guardarCola(sesionId, restantes);
169       setCola([...restantes]);
170     }
171     refrescar();
172   }

```

```

173
174 async function abrir(nombre = textoBodega) {
175   setErr("");
176   setBodegaNoEncontrada(null);
177   setOpcionesBodega(null);
178   setBodegaParaConfirmar(null);
179   try {
180     const r = await abrirBodega(nombre, token);
181     setBodega(r);
182     // De aquí en adelante la conversación (dictar, confirmar, corregir,
183     // el avance) debe usar el id real de esta sesión de conteo, no el
184     // marcador de posición con el que se llegó a esta pantalla - sin
185     // esto, dos personas abriendo bodegas distintas casi al tiempo
186     // terminaban compartiendo la misma conversación y el conteo de una
187     // se guardaba contra la bodega de la otra.
188     alAbrirBodega?.(r.sesion_id);
189     const saludo = `Listo, ${r.bodega.toLowerCase()} abierta con ${r.referencias} referencias. Dícteme el primer
producto.`;
190     setRespuesta(saludo);
191     hablar(saludo);
192     refrescar();
193   } catch (e) {
194     // todo lo que el agente muestra en pantalla también lo dice en voz -
195     // esto se quedaba solo en rojo (ej. "esta bodega ya está en conteo
196     // por otra persona"), así que alguien confirmando de viva voz sin
197     // mirar la pantalla se quedaba sin saber qué había pasado.
198     setErr(e.message);
199     hablar(e.message);
200     // igual que un artículo que no esta en el catalogo (ver
201     // FormularioCrearProducto): en vez de dejar a la persona sin salida,
202     // se ofrece pedir la bodega nueva - queda pendiente de aprobacion.
203     if (e.message === "No encuentro esa bodega." && nombre.trim()) {
204       setBodegaNoEncontrada(nombre.trim());
205     }
206   }
207 }
208
209 // Al llegar desde otra pantalla con una bodega ya resuelta (el "Ir a
210 // Conteo"/"Seguir contando" de Bodegas), se abre de una vez en lugar de
211 // solo dejarla escrita esperando un clic más: a diferencia de lo
212 // dictado por voz, este nombre ya salió de una búsqueda exacta contra
213 // el catálogo, no hay nada que confirmar.
214 useEffect(() => {
215   if (bodegaSugerida) abrir(bodegaSugerida);
216   // eslint-disable-next-line react-hooks/exhaustive-deps
217 }, []);
218
219 /** El botón "Abrir por nombre exacto" (y Enter en el campo) escribía
220     directo a abrir(), que con un nombre ambiguo ("restaurante" a
221     secas, cuando hay cinco "RESTAURANTE ...") solo sabía decir "no la
222     encuentro" y ofrecer crear una bodega duplicada - nunca mostraba
223     las que sí existían. Esto resuelve primero, igual que la voz. */
224 async function abrirPorBoton() {
225   if (!textoBodega.trim()) return;
226   setErr("");
227   setBodegaNoEncontrada(null);
228   setOpcionesBodega(null);
229   setBodegaParaConfirmar(null);
230   let resuelto;
231   try {
232     resuelto = await buscarBodega(textoBodega, token);
233   } catch (e) {
234     setErr(e.message);
235     hablar(e.message);
236     return;
237   }
238   if (resuelto.encontrada) {
239     abrir(resuelto.bodega);
240     return;
241   }
242   if (resuelto.opciones?.length) {
243     setOpcionesBodega(resuelto.opciones);
244     return;
245   }

```

```

246     setBodegaNoEncontrada(textoBodega.trim());
247 }
248
249 async function solicitarBodegaNueva() {
250     if (!bodegaNoEncontrada) return;
251     setErr("");
252     try {
253         const r = await pedir("/api/bodegas/crear-pendiente", {
254             method: "POST", body: JSON.stringify({ nombre: bodegaNoEncontrada }),
255             token);
256         setBodegaNoEncontrada(null);
257         setMsgBodega(r.respuesta_hablada || "Solicitud enviada.");
258         hablar(r.respuesta_hablada);
259     } catch (e) { setErr(e.message); hablar(e.message); }
260 }
261
262 /** Un turno de conversación: el ciclo completo del agente.
263     «mostrar» es lo que se ve como «Usted dijo»; «texto» es lo que de
264     verdad se le manda al backend. Al tocar una tarjeta de opción se
265     manda el código exacto (nunca falla), no el nombre: si un nombre
266     está contenido dentro del otro (ARROZ dentro de ARROZ BASMATI), el
267     agente no tiene con qué distinguirlos por palabra. */
268 async function procesar(texto, mostrar = texto) {
269     if (!texto) return;
270     setDicho(mostrar);
271     setErr("");
272     try {
273         const t = await enviarTurno(texto, sesionId, token, {
274             opciones, opcionesPara,
275             bodegaId: bodega?.bodega_id, bodegaNombre: bodega?.bodega,
276         });
277         setRespuesta(t.respuesta_hablada || "");
278         setOpciones(t.opciones || null);
279         setOpcionesPara(t.opciones_para || null);
280         setAlerta(t.alerta || null);
281         setAlertaEsperado(t.alerta === "desviacion" ? (t.pendiente?.cantidad ?? null) : null);
282         setContextoAlerta(t.contexto_alerta || null);
283         setPendiente(t.pendiente || null);
284         setArchivo(t.archivo || null);
285         hablar(t.respuesta_hablada);
286         if (t.guardado || t.corregido || t.bodega) refrescar();
287         if (t.bodega) {
288             setBodega({ bodega: t.bodega.nombre, referencias: t.bodega.referencias, bodega_id: t.bodega.id });
289             // igual que al abrir por el botón: de aquí en adelante hay que
290             // hablar con el id real de esta sesión de conteo, no con el
291             // marcador de posición con el que se pidió abrir por voz.
292             if (t.sesion_id) alAbrirBodega?.(t.sesion_id);
293         }
294         if (t.intencion === "crear") {
295             setCrear({ nombre: t.articulo_texto || texto, unidad_medida: "Unidad",
296                 cantidad_inicial: t.cantidad || 0 });
297         } else {
298             setCrear(null);
299         }
300     } catch (e) {
301         if (esFalloRed(e)) {
302             setOffline(true);
303             setErr("");
304         } else {
305             setErr(e.message);
306             hablar(e.message);
307         }
308     } finally {
309         setEstado("listo");
310     }
311 }
312
313 function recontar() {
314     setAlerta(null);
315     setAlertaEsperado(null);
316     setContextoAlerta(null);
317     setPendiente(null);
318     setDicho("");
319     const msg = "¿La contamos otra vez? Dígame el nuevo valor.";

```

```

320     setRespuesta(msg);
321     hablar(msg);
322 }
323
324 async function crearPendiente() {
325     setErr("");
326     try {
327         const r = await pedir("/api/conteo/crear-producto", {
328             method: "POST",
329             body: JSON.stringify({ ...crear, sesion_id: sesionId }),
330             token;
331         });
332         setRespuesta(r.respuesta_hablada);
333         hablar(r.respuesta_hablada);
334         setCrear(null);
335         setAlerta(null);
336         refrescar();
337     } catch (e) {
338         setErr(e.message);
339         hablar(e.message);
340     }
341
342     async function firmarConteo() {
343         setConfirmarFirma(false);
344         try {
345             await pedir(`/api/sesiones/${sesionId}/firmar`, { method: "POST" }, token);
346             setFirmado(true);
347             setRespuesta("Conteo firmado. La bodega pasa a auditoría.");
348             hablar("Su conteo quedó firmado. La bodega pasa a auditoría.");
349         } catch (e) { setErr(e.message); }
350     }
351
352     function guardarEnCola(item) {
353         const nueva = [...leerCola(sesionId), item];
354         guardarCola(sesionId, nueva);
355         setCola(nueva);
356     }
357
358     function alMicrofono() {
359         if (estado === "escuchando") {
360             rec.current?.stop();
361             return;
362         }
363         rec.current = escuchar({
364             alTexto: procesar,
365             alEstado: (e, parcial) => {
366                 setEstado(e);
367                 if (parcial) setDicho(parcial);
368             },
369             alError: setErr,
370         });
371     }
372
373     /** Para elegir bodega (a diferencia del conteo, que confirma hablado
374     antes de guardar): esto llena el buscador con lo que se reconoció Y
375     pregunta en voz alta "¿confirma X?" - un "sí" abre directo, un "no"
376     descarta sin tocar nada. Nombres poco comunes ("Piscilago") son
377     justo donde más se equivoca el reconocimiento de voz del navegador;
378     "Abrir por nombre exacto" sigue ahí como respaldo si prefiere
379     escribir o corregir a mano en vez de confirmar hablado. */
380     function alMicrofonoBodega() {
381         if (estado === "escuchando") {
382             rec.current?.stop();
383             return;
384         }
385         setErr("");
386         setMsgBodega("");
387         // si todavía se está esperando el sí/no a "¿confirma que abro X?", este
388         // toque del micrófono es la respuesta a ESA pregunta, no un nombre de
389         // bodega nuevo - ver el comentario junto a bodegaParaConfirmar.
390         if (bodegaParaConfirmar) {
391             const miIdConfirmar = ++idEscuchaBodega.current;
392             rec.current = escuchar({
393                 alTexto: (respuesta) => {

```

```

394         if (miIdConfirmar !== idEscuchaBodega.current) return;
395         responderConfirmacionBodega(respuesta);
396     },
397     alEstado: (e) => setEstado(e),
398     alError: setErr,
399 });
400 return;
401 }
402 setOpcionesBodega(null);
403 const miId = ++idEscuchaBodega.current;
404 rec.current = escuchar({
405     alTexto: (t) => {
406         if (miId !== idEscuchaBodega.current) return;
407         setTextoBodega(t);
408         preguntarConfirmacionBodega(t, miId);
409     },
410     alEstado: (e, parcial) => {
411         setEstado(e);
412         if (parcial) setTextoBodega(parcial);
413     },
414     alError: setErr,
415 });
416 }
417
418 /** La respuesta al "¿confirma que abro X?", venga del listener que se
419     abrió justo después de la pregunta, o de un toque posterior al
420     micrófono si la persona se demoró y ese listener ya se había
421     apagado solo (ver bodegaParaConfirmar). */
422 function responderConfirmacionBodega(respuesta) {
423     const nombreReal = bodegaParaConfirmar;
424     setBodegaParaConfirmar(null);
425     if (!nombreReal) return;
426     if (esAfirmacion(respuesta)) {
427         abrir(nombreReal);
428     } else if (esNegacion(respuesta)) {
429         setTextoBodega("");
430         const msg = "Gracias, queda pendiente. Cuando quiera abrir una bodega, dígame el nombre.";
431         setMsgBodega(msg);
432         hablar(msg);
433     } else {
434         const msg = "No le entendí un «sí» o un «no». Use «Abrir por nombre exacto», o dígame de nuevo.";
435         setErr(msg);
436         hablar(msg);
437         // sigue pendiente: no se descarta la confirmación por una respuesta
438         // que no se entendió, para no perder el hilo justo ahora.
439         setBodegaParaConfirmar(nombreReal);
440     }
441 }
442
443 /** Antes esto preguntaba "¿confirma X?" con lo que se acababa de
444     transcribir tal cual - decir "kiosco" confirmaba "kiosco" como si
445     fuera una bodega real, cuando lo que existe es "KIOSCO TAQUILLA
446     AYB". Ahora primero RESUELVE el nombre contra el catálogo
447     (/api/bodegas/buscar, sin abrir nada todavía) y confirma con el
448     nombre real que de verdad se va a abrir. Si lo dicho es ambiguo
449     ("restaurante" a secas encaja en dos bodegas reales por igual), se
450     ofrecen las opciones en vez de fallar en seco; si de verdad no
451     encuentra nada, ofrece crearla.
452
453     "miId" identifica ESTA escucha en particular (viene de
454     idEscuchaBodega, ver el comentario junto a esa ref): antes de tocar
455     cualquier estado se revisa que siga siendo la vigente, porque el
456     micrófono anterior a veces entrega su resultado tarde - sin este
457     control, ese resultado tardío se procesaba como una bodega nueva
458     dictada (por ejemplo, el "confirmo" de la pregunta de abajo llegaba
459     aquí como si alguien hubiera dicho "confirmo" como nombre de
460     bodega). */
461 async function preguntarConfirmacionBodega(nombreDictado, miId) {
462     if (!nombreDictado || !nombreDictado.trim()) return;
463     if (miId === undefined) miId = ++idEscuchaBodega.current;
464     setOpcionesBodega(null);
465     let resuelto;
466     try {
467         resuelto = await buscarBodega(nombreDictado, token);

```



```

468   } catch (e) {
469     if (miId !== idEscuchaBodega.current) return;
470     setErr(e.message);
471     hablar(e.message);
472     return;
473   }
474   if (miId !== idEscuchaBodega.current) return;
475   if (!resuelto.encontrada) {
476     if (resuelto.opciones?.length) {
477       const nombres = resuelto.opciones.map((o) => o.bodega.toLowerCase());
478       setOpcionesBodega(resuelto.opciones);
479       // reservar el id ANTES de hablar corta de una vez cualquier
480       // escucha anterior que responda tarde mientras se dice esto.
481       const miIdOpciones = ++idEscuchaBodega.current;
482       // hay que esperar a que la pregunta se termine de decir - si el
483       // micrófono arranca apenas empieza a sonar, se pierde el
484       // principio de la respuesta (o, si la voz tarda, el micrófono
485       // hasta se cansa de esperar antes de que la persona alcance a
486       // contestar).
487       await hablar(`Hay varias: ${nombres.join(", ")}. ¿Cuál de todas?`);
488       if (miIdOpciones !== idEscuchaBodega.current) return;
489       // se puede elegir tocando una tarjeta, o siguiendo la conversación
490       // y diciendo el nombre completo - que vuelve a pasar por aquí,
491       // ahora sí resolviendo a una sola.
492       rec.current = escuchar({
493         alTexto: (t) => {
494           if (miIdOpciones !== idEscuchaBodega.current) return;
495           setTextoBodega(t);
496           preguntarConfirmacionBodega(t, miIdOpciones);
497         },
498         alEstado: (e) => setEstado(e),
499         alError: setErr,
500       });
501       return;
502     }
503     hablar(`No encuentro ${nombreDictado.toLowerCase()}. Puede solicitarla como bodega nueva.`);
504     setBodegaNoEncontrada(nombreDictado.trim());
505     return;
506   }
507   const nombreReal = resuelto.bodega;
508   setTextoBodega(nombreReal);
509   setBodegaParaConfirmar(nombreReal);
510   const miIdConfirmar = ++idEscuchaBodega.current;
511   await hablar(`¿Confirma que abro ${nombreReal.toLowerCase()}?`);
512   if (miIdConfirmar !== idEscuchaBodega.current) return;
513   rec.current = escuchar({
514     alTexto: (respuesta) => {
515       if (miIdConfirmar !== idEscuchaBodega.current) return;
516       responderConfirmacionBodega(respuesta);
517     },
518     alEstado: (e) => setEstado(e),
519     alError: setErr,
520   });
521 }
522
523 return (
524   <Marco
525     titulo={`Conteo por voz${bodega ? " " + bodega.bodega : ""}`}
526     chip={offline ? { texto: "SIN CONEXIÓN", tipo: "oro" }
527       : bodega ? { texto: "EN CONTEO", tipo: "azul" }
528       : { texto: "SIN BODEGA", tipo: "gris" }}
529     >
530     {!bodega ? (
531       <div className="card">
532         <h2>¿Qué bodega va a contar?</h2>
533         <div style={{ display: "flex", gap: 10, alignItems: "center", marginBottom: 6 }}>
534           <button
535             className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
536             style={{ width: 46, height: 46, fontSize: "1.2rem" }}
537             onClick={alMicrofonoBodega} title="diga el nombre de la bodega"
538             aria-label="Decir el nombre de la bodega por voz">
539             <Icono nombre="microfono" tam={22} />
540           </button>
541           <small style={{ color: "var(--grafito)" }}>

```

```

542         {estado === "escuchando" ? "Escuchando..."
543           : "o diga el nombre de la bodega en voz alta"}
544       </small>
545     </div>
546     <p className="pista" style={{ marginBottom: 8 }}>
547       Lo que reconozca aparece abajo, se lo pregunta de vuelta y usted
548       confirma diciendo «sí» (nombres como «Piscilago» a veces se
549       transcriben mal) – o corrija a mano y use «Abrir por nombre
550       exacto».
551     </p>
552     <details style={{ marginBottom: 8 }}>
553       <summary className="pista" style={{ cursor: "pointer" }}>
554         Ver frases exactas que entiende el agente aquí
555       </summary>
556       <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
557         {["almacén general", "kiosco taquilla ayb", "sí (confirma la bodega sugerida)",
558           "no (descarta la sugerencia)"].map((ej, i) => (
559           <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
560         ))}
561       </ul>
562     </details>
563     <div style={{ display: "flex", gap: 10, flexWrap: "wrap", marginBottom: 14 }}>
564       <input
565         style={{ flex: 1, minWidth: 240, padding: "12px 14px",
566           border: "1px solid var(--borde)", borderRadius: 12 }}
567         value={textoBodega}
568         onChange={(e) => setTextoBodega(e.target.value)}
569         onKeyDown={(e) => e.key === "Enter" && abrirPorBoton()}
570         placeholder="nombre de la bodega..."
571         aria-label="Nombre de la bodega"
572       />
573       <button className="btn" onClick={abrirPorBoton}
574         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
575         <Icono nombre="aprobar" tam={18} />
576         Abrir por nombre exacto
577       </button>
578     </div>
579     {err && <p className="error" style={{ marginBottom: 10 }}>{err}</p>}
580     {msgBodega && <p className="msg-ok" style={{ marginBottom: 10 }}>{msgBodega}</p>}
581     {bodegaNoEncontrada && (
582       <div className="banner" style={{ marginBottom: 10 }}>
583         <span className="ico">!</span>
584         <span>
585           <b>¿Crear «{bodegaNoEncontrada}»?</b>
586           <span>No existe en el catálogo. Queda pendiente de aprobación del
587             administrador; el conteo no se detiene por esto.</span>
588         </span>
589         <button className="btn" style={{ flex: "none" }}
590           onClick={solicitarBodegaNueva}>
591           Solicitar bodega nueva
592         </button>
593       </div>
594     )}
595     {opcionesBodega && (
596       <div style={{ marginBottom: 14 }}>
597         <p className="pista" style={{ marginBottom: 8 }}>
598           Hay varias que encajan con lo dicho – toque una, o diga el
599           nombre completo:
600         </p>
601         <div className="opciones">
602           {opcionesBodega.map((o, i) => (
603             <button key={o.bodega_id} className="opcion"
604               onClick={() => { setOpcionesBodega(null); abrir(o.bodega); }}>
605               <span className="n">Opción {i + 1}</span>
606               <h4>{o.bodega}</h4>
607             </button>
608           ))}
609         </div>
610       </div>
611     )}
612     {asignadas === null ? (
613       <p className="cargando">Cargando sus bodegas asignadas...</p>
614     ) : asignadas.length > 0 && !buscarOtra ? (
615       <>

```

```

616 <div className="grid-bodegas">
617   {asignadas.map((b) => {
618     const [txt, cls] = ETIQUETA_BODEGA[b.estado] || ["?", "gris"];
619     const esOtraPersona = b.estado === "en_conteo" && b.persona
620       && b.persona.toLowerCase() !== (usuario?.nombre || "").toLowerCase();
621     let detalleLinea;
622     if (b.estado === "en_conteo" && b.persona) {
623       detalleLinea = esOtraPersona
624         ? `${b.persona} ya la está contando · ${b.avance_pct ?? 0}%`
625         : `usted la dejó a medias · ${b.avance_pct ?? 0}%`;
626     } else if (b.estado === "en_auditoria" && b.persona) {
627       detalleLinea = `${b.persona} auditando · ${b.diferencias ?? 0} dif.`;
628     } else if (b.estado === "cerrada" && b.hora_cierre) {
629       detalleLinea = `cerrada ${b.hora_cierre}`;
630     } else if (b.estado === "pendiente") {
631       detalleLinea = "sin iniciar · disponible";
632     } else {
633       detalleLinea = `${b.referencias} referencias`;
634     }
635     return (
636       <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
637         onClick={() => abrir(b.bodega)}
638         title={esOtraPersona
639           ? "Ya la está contando otra persona; puede intentar de todas formas."
640           : ""}>
641         <b>{b.bodega}</b>
642         <span className={`chip ${cls} est`} >{txt}</span>
643         <small>{detalleLinea}</small>
644       </button>
645     );
646   })}
647 </div>
648 <div className="grilla-botones" style={{ marginTop: 14 }}>
649   <button className="btn borde"
650     onClick={() => { setBuscarOtra(true); if (!todas) cargarTodas(); }}>
651     Buscar otra bodega
652   </button>
653 </div>
654 </>
655 ) : (
656   <>
657     {asignadas.length > 0 && (
658       <p className="pista" style={{ marginBottom: 10 }}>
659         Buscando fuera de sus bodegas asignadas.
660       </p>
661     )}
662     {todas === null ? (
663       <p className="cargando" style={{ marginTop: 10 }}>Cargando bodegas...</p>
664     ) : (
665       <div className="grid-bodegas" style={{ marginTop: 14 }}>
666         {(textoBodega.trim()
667           ? todas.filter((b) => b.bodega.toLowerCase()
668             .includes(textoBodega.trim().toLowerCase()))
669           : todas
670         ).slice(0, 24).map((b) => {
671           const [txt, cls] = ETIQUETA_BODEGA[b.estado] || ["?", "gris"];
672           return (
673             <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
674               onClick={() => abrir(b.bodega)}>
675               <b>{b.bodega}</b>
676               <span className={`chip ${cls} est`} >{txt}</span>
677               <small>{b.referencias} referencias</small>
678             </button>
679           );
680         })}
681       </div>
682     )}
683     {todas && !textoBodega.trim() && todas.length > 24 && (
684       <p className="pista" style={{ marginTop: 8 }}>
685         Mostrando 24 de {todas.length} – escriba para filtrar.
686       </p>
687     )}
688   </>
689   <div className="grilla-botones" style={{ marginTop: 10 }}>

```

```

690         {asignadas.length > 0 && (
691             <button className="btn gris" onClick={() => setBuscarOtra(false)}>
692                 Volver a sus bodegas asignadas
693             </button>
694         )}
695     </div>
696 </>
697 )}
698 </div>
699 ) : offline ? (
700     <FormularioOffline
701         cola={cola}
702         onGuardar={guardarEnCola}
703         onReintentar={() => { setOffline(!navigator.onLine); if (navigator.onLine) sincronizar(); }}
704     />
705 ) : (
706     <>
707         <div className="chips">
708             <span className="chip">Avance: {avance.hechas} de {avance.total}</span>
709             <span className={`chip ${avance.alertas ? "oro" : "verde"}`}>
710                 Alertas: {avance.alertas}
711             </span>
712             <span className="chip gris">Sesión bloqueada</span>
713             {cola.length > 0 && (
714                 <span className="chip oro">{cola.length} por sincronizar</span>
715             )}
716         </div>
717
718         {alerta && alerta !== "desviacion" && !crear && (
719             <div className="banner">
720                 <span className="ico">!</span>
721                 <span style={{ flex: 1 }}>
722                     <b>
723                         {alerta === "negativo" ? "Cantidad imposible"
724                         : alerta === "unidad" ? "Unidad de medida distinta"
725                         : "Artículo fuera del catálogo"}
726                     </b>
727                     <span>{respuesta}</span>
728                 </span>
729             </div>
730         )}
731
732         {alerta === "desviacion" && !crear && (
733             <AlertaDesviacion
734                 contexto={contextoAlerta} respuesta={respuesta}
735                 onRecontar={recontar}
736                 onConfirmar={() => procesar("confirmo")}
737             />
738         )}
739
740         {crear && (
741             <FormularioCrearProducto
742                 crear={crear} setCrear={setCrear}
743                 onCreate={crearPendiente} onCancelar={() => setCrear(null)}
744                 bodega={bodega.bodega} err={err}
745             />
746         )}
747
748         {!crear && alerta !== "desviacion" && (
749             <div className="conteo-cols">
750                 <div className="card">
751                     <p className="rotulo">
752                         {dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}
753                     </p>
754                     <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
755                     {archivo && (
756                         <div className="grilla-botones" style={{ marginTop: 10 }}>
757                             <button className="btn verde"
758                                 onClick={() => descargarReporte(archivo, token).catch((e) => setErr(e.message))}>
759                                 Descargar archivo generado
760                             </button>
761                         </div>
762                     )}
763                 </div>

```

```

764     {opciones && (
765       <div className="opciones" style={{ marginTop: 14 }}>
766         {opciones.map((o, i) => (
767           <button key={o.codigo} className="opcion"
768             onClick={() => procesar(o.codigo, o.nombre)}>
769             <span className="n">Opción {i + 1}</span>
770             <h4>{o.nombre}</h4>
771             <p>Código {o.codigo}</p>
772             <p>{o.unidad}</p>
773           </button>
774         ))}
775       </div>
776     )}
777
778     {pendiente && !opciones && (
779       <p className="pista" style={{ marginTop: 12 }}>
780         Por confirmar: <b>{pendiente.nombre}</b> – {pendiente.cantidad}{ " " }
781         {pendiente.unidad}
782       </p>
783     )}
784
785     <h2 style={{ marginTop: 18 }}>Últimos registros confirmados</h2>
786     {avance.ultimos?.length ? (
787       avance.ultimos.map((u, i) => (
788         <div className="registro" key={i}>
789           <span className="ok">✓</span>
790           <span>{u.nombre}</span>
791           <span className="cant">{u.cantidad} {u.unidad}</span>
792         </div>
793       ))
794     ) : (
795       <p className="vacio">Todavía no hay registros en esta sesión.</p>
796     )}
797   </div>
798
799   <div className="card mic-caja">
800     <button
801       className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
802       onClick={alMicrofono}
803       aria-label="Toque y hable para dictar el artículo y la cantidad contada"
804     >
805       <Icono nombre="microfono" tam={52} />
806     </button>
807     <b>
808       {estado === "escuchando" ? "Escuchando..."
809       : estado === "procesando" ? "Procesando..."
810       : "Toque y hable"}
811     </b>
812     <small>
813       {vozDisponible()
814       ? "Reconocimiento en español (Colombia)"
815       : "Este navegador no reconoce voz: use el teclado"}
816     </small>
817     <details style={{ marginTop: 8, textAlign: "left" }}>
818       <summary className="pista" style={{ cursor: "pointer" }}>
819         Ver frases exactas que entiende el agente aquí
820       </summary>
821       <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
822         {["tres tablas para picar blancas", "hay noventa cazuelas",
823          "me faltan cinco kilos de arroz", "corregir", "son nueve",
824          "confirмо", "¿cuánto arroz hay contado?"].map((ej, i) => (
825           <li key={i} className="pista" style={{ marginBottom: 4 }}><ej></li>
826         ))}
827       </ul>
828     </details>
829     {err && <p className="error">{err}</p>}
830   </div>
831 </div>
832 )}
833
834 {!crear && alerta !== "desviacion" && (
835   <>
836     <div className="grilla-botones">
837       <button className="btn verde" onClick={() => procesar("confirмо")}>

```

```

838         Confirmar
839     </button>
840     <button className="btn borde" onClick={() => procesar("corregir")}>
841         Corregir
842     </button>
843     <button className="btn gris" onClick={() => setMostrarTeclado(true)}>
844         Teclado
845     </button>
846     <button className="btn oro" onClick={() => ir("ayuda")}>
847         Pedir ayuda
848     </button>
849 </div>
850 <p className="pista" style={{ marginTop: 10 }}>
851     La voz es el camino rápido; el teclado siempre queda como respaldo.
852 </p>
853
854 /* En su propia fila y no junto a "Confirmar"/"Corregir":
855     esos dos son del artículo que se está dictando, este
856     termina el turno entero. */
857 <div className="grilla-botones" style={{ marginTop: 14 }}>
858     {firmado ? (
859         <p style={{ color: "var(--verde)", fontWeight: 700, flex: 1,
860             textAlign: "center", margin: 0 }}>
861             ✓ Conteo firmado · pasa a auditoría
862         </p>
863     ) : (
864         <button className="btn" style={{ flex: 1 }}
865             disabled={avance.hechas === 0}
866             onClick={() => setConfirmarFirma(true)}>
867             Terminar y firmar mi conteo
868         </button>
869     )}
870 </div>
871 {!firmado && avance.hechas === 0 && (
872     <p className="pista" style={{ marginTop: 6 }}>
873         Cuente al menos un artículo antes de firmar.
874     </p>
875 )}
876 </>
877 )}
878 </>
879 ))
880
881 {confirmarFirma && (
882     <Dialogo titulo="Firmar mi conteo"
883     mensaje={`Va a firmar el conteo de ${bodega?.bodega || "esta bodega"} `
884         + `con ${avance.hechas} `
885         + `${avance.hechas === 1 ? "referencia" : "referencias"}. `
886         + "Después no podrá agregar ni corregir artículos: la bodega "
887         + "pasa a auditoría."}
888     textoAceptar="Firmar"
889     onAceptar={firmarConteo}
890     onCancel={() => setConfirmarFirma(false)} />
891 )}
892
893 {mostrarTeclado && (
894     <Dialogo titulo="Escribir en vez de hablar"
895     mensaje="Escriba lo que diría en voz alta:"
896     conCampo placeholder="tres tablas para picar blancas"
897     onAceptar={(t) => { setMostrarTeclado(false); if (t) procesar(t); }}
898     onCancel={() => setMostrarTeclado(false)} />
899 )}
900 </Marco>
901 );
902 }
903
904 const fmt = (n) => Number(n).toLocaleString("es-CO", { maximumFractionDigits: 2 });
905
906 function AlertaDesviacion({ contexto, respuesta, onRecontar, onConfirmar }) {
907     return (
908         <>
909             <div className="banner">
910                 <span className="ico">!</span>
911                 <span>

```

```

912     <b>Cantidad fuera de lo esperado</b>
913     <span>
914         {contexto
915             ? `El sistema espera alrededor de ${fmt(contexto.esperado)} · usted dijo `
916             + `${fmt(contexto.dicho)} (${contexto.articulo)}`
917             : respuesta}
918     </span>
919 </span>
920 </div>
921
922 <div className="card">
923     <p className="rotulo">CuentaVoz responde</p>
924     <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
925 </div>
926
927 <div className="conteo-cols">
928     {contexto ? (
929         <div className="card">
930             <h2>Contexto de la validación</h2>
931             <table>
932                 <tbody>
933                     <tr><td>Stock del sistema</td><td><b style={{ color: "var(--azul)" }}>
934                         {fmt(contexto.esperado)} {contexto.unidad}</b></td></tr>
935                     <tr><td>Su conteo</td><td><b style={{ color: "var(--azul)" }}>
936                         {fmt(contexto.dicho)} {contexto.unidad}</b></td></tr>
937                     <tr><td>Desviación</td><td className="dif">
938                         {contexto.desviacion_pct != null
939                             ? `${contexto.desviacion_pct > 0 ? "+" : ""}${contexto.desviacion_pct}%`
940                             : "-"}</td></tr>
941                 </tbody>
942             </table>
943         </div>
944     ) : <div />}
945     <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
946         <button className="btn" onClick={onRecontar}>RECONTAR</button>
947         <button className="btn oro" onClick={onConfirmar}>
948             CONFIRMAR {contexto ? fmt(contexto.dicho) : ""}
949         </button>
950     </div>
951 </div>
952
953 <p className="pista">
954     Si confirma, el registro se guarda marcado para revisión del administrador de bodega.
955 </p>
956 <p className="pista" style={{ color: "var(--azul)" }}>
957     Ninguna cantidad con alerta se guarda sin una confirmación explícita de la persona.
958 </p>
959 </>
960 );
961 }
962
963 function FormularioCrearProducto({ crear, setCrear, onCrear, onCancelar, bodega, err }) {
964     const [estado, setEstado] = useState("listo");
965     // El micrófono principal de Conteo se oculta mientras se decide si
966     // crear este producto (para no mezclar esa respuesta con un conteo
967     // nuevo) - sin esto, la pregunta hablada "¿lo creamos?" solo se podía
968     // contestar con clic, aunque el resto de la pantalla es por voz.
969     function alMicrofono() {
970         escuchar({
971             alTexto: (texto) => {
972                 setEstado("listo");
973                 if (esAfirmacion(texto, ["crea", "crear"])) { onCrear(); return; }
974                 if (esNegacion(texto)) { onCancelar(); return; }
975             },
976             alEstado: setEstado,
977             alError: () => {},
978         });
979     }
980     return (
981         <div className="card">
982             <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", gap: 10 }}>
983                 <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
984                 <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
985                     style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}>

```

```

986         onClick={alMicrofono} title="diga «sí» para crear, o «no» para cancelar"
987         aria-label="Responder por voz: diga sí para crear, o no para cancelar">
988         <Icono nombre="microfono" tam={20} />
989     </button>
990 </div>
991 <div className="burbuja" aria-live="polite" aria-atomic="true">
992     No encontré «{crear.nombre}» en el catálogo. Lo creamos y queda pendiente
993     del administrador.
994 </div>
995 <p className="pista" style={{ marginTop: 8 }}>o diga «sí» para crearlo, o «no» para cancelar</p>
996
997 <div className="conteo-cols" style={{ marginTop: 16 }}>
998     <div>
999         <label className="pista" htmlFor="campo-nombre-articulo">Nombre tal como lo dictó</label>
1000         <input id="campo-nombre-articulo" value={crear.nombre}
1001             onChange={(e) => setCrear({ ...crear, nombre: e.target.value })}
1002             style={{ width: "100%", padding: "11px 13px", marginTop: 6,
1003                 border: "1px solid var(--borde)", borderRadius: 12,
1004                 textTransform: "uppercase" }} />
1005     </div>
1006     <div>
1007         <label className="pista">Estado del registro</label>
1008         <div style={{ marginTop: 6 }}>
1009             <span className="chip oro">PENDIENTE DE APROBACIÓN</span>
1010         </div>
1011     </div>
1012 </div>
1013
1014 <div className="dos-columnas" style={{ marginTop: 16 }}>
1015     <div>
1016         <label className="pista" id="etiqueta-unidad-medida">Unidad de medida</label>
1017         <div className="grilla-botones" style={{ marginTop: 6 }} role="group" aria-labelledby="etiqueta-unidad-
medida">
1018             {UNIDADES.map((u) => (
1019                 <button key={u}
1020                     className={`btn ${crear.unidad_medida === u ? "" : "borde"} `}
1021                     onClick={() => setCrear({ ...crear, unidad_medida: u })}
1022                     aria-pressed={crear.unidad_medida === u}>
1023                 {u}
1024             </button>
1025             ))}
1026         </div>
1027     </div>
1028     <div>
1029         <label className="pista" htmlFor="campo-cantidad-inicial">Cantidad inicial contada en {bodega}</label>
1030         <input id="campo-cantidad-inicial" type="number" min="0" value={crear.cantidad_inicial}
1031             onChange={(e) => setCrear({ ...crear, cantidad_inicial: Number(e.target.value) })}
1032             style={{ width: "100%", padding: "11px 13px", marginTop: 6,
1033                 border: "1px solid var(--borde)", borderRadius: 12 }} />
1034     </div>
1035 </div>
1036
1037 <p className="pista" style={{ marginTop: 12 }}>
1038     No entra al catálogo oficial hasta la firma del administrador de bodega.
1039 </p>
1040 {err && <p className="error" style={{ marginTop: 6 }}>{err}</p>}
1041 <div className="grilla-botones">
1042     <button className="btn" onClick={onCrear}>Crear pendiente</button>
1043     <button className="btn gris" onClick={onCancelar}>Cancelar</button>
1044 </div>
1045 <p className="pista" style={{ marginTop: 10 }}>
1046     El conteo continúa sin interrupción: la aprobación ocurre en paralelo.
1047 </p>
1048 </div>
1049 );
1050 }
1051
1052 /** Mismo ciclo de confirmación del agente real (dice el artículo y la
1053 cantidad, CuentaVoz repite y pregunta, la persona confirma) pero
1054 corriendo enteramente en el navegador con interpreteLocal.js - sin
1055 tocar el backend ni el internet para nada. El reconocimiento de voz
1056 del navegador sí necesita señal (no es algo que esta app controle),
1057 así que aquí se escribe en vez de dictar; el resto de la conversación
1058 - entender la frase, sugerir el nombre oficial, pedir confirmación -

```



```

1059     sigue funcionando igual. */
1060 function FormularioOffline({ cola, onGuardar, onReintentar }) {
1061   const [catalogo] = useState(() => leerCatalogo());
1062   const [texto, setTexto] = useState("");
1063   const [dicho, setDicho] = useState("");
1064   const [respuesta, setRespuesta] = useState(
1065     "Sin conexión, pero sigo entendiendo lo que escriba. Dígame el artículo y la cantidad."
1066   );
1067   const [pendiente, setPendiente] = useState(null); // {articulo, cantidad, unidad, codigo}
1068   const [mostrarManual, setMostrarManual] = useState(false);
1069
1070   // el formulario campo por campo de siempre, como respaldo del respaldo
1071   // si una frase no se deja interpretar bien.
1072   const [articulo, setArticulo] = useState("");
1073   const [cantidad, setCantidad] = useState(1);
1074   const [unidad, setUnidad] = useState("Unidad");
1075   const sugerenciaManual = sugerirDeCatalogo(articulo, catalogo);
1076
1077   function procesarTexto(t) {
1078     if (!t.trim()) return;
1079     setDicho(t);
1080     setTexto("");
1081     const r = interpretarLocal(t);
1082     const simple = t.trim().toLowerCase();
1083     const esSi = /^(si|sí|confirmo|correcto|dale|listo)\b/.test(simple);
1084     const esNo = /^no\b/.test(simple);
1085
1086     if (pendiente && (esSi || r.intencion === "confirmar")) {
1087       onGuardar({ articulo: pendiente.articulo, cantidad: pendiente.cantidad,
1088         unidad: pendiente.unidad });
1089       setPendiente(null);
1090       setRespuesta("Guardado en este equipo. Se sincroniza solo cuando vuelva la señal.");
1091       return;
1092     }
1093     if (pendiente && esNo) {
1094       setPendiente(null);
1095       setRespuesta("Listo, no lo guardo. Dígame el artículo y la cantidad correctos.");
1096       return;
1097     }
1098     if (pendiente && r.intencion === "corregir" && r.cantidad !== null) {
1099       setPendiente({ ...pendiente, cantidad: r.cantidad });
1100       setRespuesta(`${pendiente.articulo}: ${r.cantidad} ${pendiente.unidad}. ¿Confirma?`);
1101       return;
1102     }
1103     if (r.intencion === "contar" && r.cantidad !== null) {
1104       const sug = sugerirDeCatalogo(r.articulo_texto, catalogo);
1105       const nombreFinal = sug?.nombre || r.articulo_texto || "ese artículo";
1106       const unidadFinal = sug?.unidad || r.unidad || "Unidad";
1107       setPendiente({ articulo: nombreFinal, cantidad: r.cantidad,
1108         unidad: unidadFinal, codigo: sug?.codigo });
1109       setRespuesta(`${nombreFinal}: ${r.cantidad} ${unidadFinal}. ¿Confirma? (escriba «sí») `);
1110       return;
1111     }
1112     setRespuesta(r.respuesta_hablada
1113       || "No le entendí bien. Dígame el artículo y la cantidad, por ejemplo «hay noventa cazuelas.»");
1114   }
1115
1116   function usarSugerenciaManual() {
1117     setArticulo(sugerenciaManual.nombre);
1118     setUnidad(sugerenciaManual.unidad);
1119   }
1120   function ciclarUnidad() {
1121     setUnidad(UNIDADES[(UNIDADES.indexOf(unidad) + 1) % UNIDADES.length]);
1122   }
1123   function guardarManual() {
1124     if (!articulo.trim()) return;
1125     onGuardar({ articulo: articulo.trim(), cantidad: Number(cantidad), unidad });
1126     setArticulo("");
1127     setCantidad(1);
1128   }
1129
1130   return (
1131     <div className="conteo-cols">
1132       <div className="card">

```

```

1133     <div className="banner">
1134       <span className="ico">!</span>
1135       <span>
1136         <b>Sin conexión</b>
1137         <span>El reconocimiento de voz necesita señal, pero CuentaVoz sigue
1138           entendiendo lo que escriba. Lo que registre se guarda en este
1139           equipo y se sincroniza solo al volver la red.</span>
1140       </span>
1141     </div>
1142
1143     <p className="rotulo">{dicho ? `Usted escribió: «${dicho}»` : "CuentaVoz responde"}</p>
1144     <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
1145
1146     <div style={{ display: "flex", gap: 10, marginTop: 12, flexWrap: "wrap" }}>
1147       <input value={texto} onChange={(e) => setTexto(e.target.value)}
1148         onKeyDown={(e) => e.key === "Enter" && procesarTexto(texto)}
1149         placeholder="hay noventa cazuelas blancas..."
1150         aria-label="Escriba el artículo y la cantidad contada"
1151         style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
1152           border: "1px solid var(--borde)", borderRadius: 12 }} />
1153       <button className="btn" onClick={() => procesarTexto(texto)}>Enviar</button>
1154     </div>
1155
1156     <div className="grilla-botones" style={{ marginTop: 10 }}>
1157       <button className="btn borde" onClick={onReintentar}>Reintentar la voz</button>
1158       <button className="btn gris" onClick={() => setMostrarManual((m) => !m)}>
1159         {mostrarManual ? "Ocultar formulario manual" : "Registrar campo por campo"}
1160       </button>
1161     </div>
1162
1163     {mostrarManual && (
1164       <div style={{ marginTop: 14, padding: 14, borderTop: "1px solid var(--borde)" }}>
1165         <label className="pista" htmlFor="campo-articulo-manual">Artículo</label>
1166         <input id="campo-articulo-manual" value={articulo} onChange={(e) => setArticulo(e.target.value)}
1167           placeholder="tabla acril..."
1168           style={{ width: "100%", padding: "11px 13px", margin: 8,
1169             border: "1px solid var(--borde)", borderRadius: 12 }} />
1170         {sugerenciaManual && (
1171           <button className="chip azul" onClick={usarSugerenciaManual}
1172             style={{ display: "block", width: "100%", textAlign: "left",
1173               margin: 6 }}>
1174             {sugerenciaManual.nombre} {sugerenciaManual.codigo}
1175           </button>
1176         )}
1177         <label className="pista">Cantidad</label>
1178         <div style={{ display: "flex", gap: 10, marginTop: 6, margin: 10,
1179           alignItems: "center" }}>
1180           <button className="btn borde" style={{ minHeight: 0, padding: "8px 16px" }}
1181             onClick={() => setCantidad((c) => Math.max(0, Number(c) - 1))}
1182             aria-label="Restar uno a la cantidad">-</button>
1183           <input type="number" value={cantidad}
1184             onChange={(e) => setCantidad(e.target.value)}
1185             aria-label="Cantidad contada"
1186             style={{ width: 90, padding: "11px 13px", textAlign: "center",
1187               border: "1px solid var(--borde)", borderRadius: 12 }} />
1188           <button className="btn borde" style={{ minHeight: 0, padding: "8px 16px" }}
1189             onClick={() => setCantidad((c) => Number(c) + 1)}
1190             aria-label="Sumar uno a la cantidad">+</button>
1191           <button className="chip azul" onClick={ciclarUnidad}
1192             aria-label={`Unidad: ${unidad}. Toque para cambiarla`}>{unidad}</button>
1193         </div>
1194         <button className="btn" onClick={guardarManual}>Guardar</button>
1195       </div>
1196     )}
1197
1198     <p className="pista" style={{ marginTop: 12 }}>
1199       El catálogo local sigue sugiriendo nombres oficiales sin internet.
1200     </p>
1201   </div>
1202
1203   <div className="card">
1204     <h2>Guardados en este equipo</h2>
1205     {cola.length === 0 ? (
1206       <p className="vacio">Nada por sincronizar todavía.</p>

```

```

1207     ) : (
1208       cola.map((c, i) => (
1209         <div className="registro" key={i}>
1210           <span>{c.articulo}</span>
1211           <span className="cant">{c.cantidad} {c.unidad}</span>
1212         </div>
1213       ))
1214     )}
1215     {cola.length > 0 && (
1216       <span className="chip oro">{cola.length} registros por sincronizar</span>
1217     )}
1218     <p className="pista" style={{ marginTop: 10 }}>
1219       Al volver la señal, la cola se envía sola a la API y las validaciones
1220       se aplican igual que en línea.
1221     </p>
1222   </div>
1223 </div>
1224 );
1225 }

```

frontend/src/vistas/Legalizacion.jsx

297 LÍNEAS

Lo pedido contra lo usado.

```

1 import { useEffect, useState } from "react";
2 import { pedir } from "../api";
3 import { hablar, escuchar, esAfirmacion, esNegacion } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import Icono from "../Iconos";
7
8 const fmt = (n) => Number(n).toLocaleString("es-CO", { maximumFractionDigits: 2 });
9
10 /** Arma en palabras lo que ya se ve en la tabla, igual a como lo diría
11     CuentaVoz - sobre datos reales, no un guion fijo. */
12 function narrar(sobran, merma) {
13   const partes = [];
14   if (sobran.length) {
15     const detalle = sobran
16       .map((l) => `${fmt(Math.abs(l.diferencia))} de ${l.nombre.toLowerCase()}`)
17       .join(" y ");
18     partes.push(`Sobró ${detalle}: lo devuelvo a la bodega.`);
19   }
20   if (merma.length) {
21     const detalle = merma
22       .map((l) => `${fmt(l.diferencia)} en ${l.nombre.toLowerCase()}`)
23       .join(", ");
24     partes.push(`Hubo merma de ${detalle}.`);
25   }
26   if (!partes.length) partes.push("Todo cuadró exacto entre lo pedido y lo usado.");
27   partes.push("¿Confirmo la legalización?");
28   return partes.join(" ");
29 }
30
31 export default function Legalizacion({ token, servicioId = 1, ir, usuario }) {
32   const esAuditor = usuario?.perfil === "auditor";
33   const [comp, setComp] = useState(null);
34   const [listo, setListo] = useState(false);
35   const [err, setErr] = useState("");
36   const [respuesta, setRespuesta] = useState("");
37   const [pedirAjuste, setPedirAjuste] = useState(false);
38   const [verMerma, setVerMerma] = useState(false);
39   const [estado, setEstado] = useState("listo");
40
41   function cargar() {
42     pedir(`/api/legalizacion/${servicioId}`, {}, token)
43       .then((c) => {
44         setComp(c);
45         const s = c.lineas.filter((l) => l.diferencia < 0);
46         const m = c.lineas.filter((l) => l.diferencia > 0);
47         const texto = narrar(s, m);
48         setRespuesta(texto);

```

```

49      // sin esto, la pregunta "¿Confirmo la legalización?" solo se veía
50      // escrita - nunca se oía al llegar, a diferencia de cómo ya saluda
51      // el resto de la app (Panel, Auditoría) al entrar a una pantalla.
52      if (c.lineas.length > 0) hablar(texto);
53    })
54    .catch((e) => setErr(e.message));
55  }
56  useEffect(cargar, [servicioId, token]);
57
58  // La pregunta hablada ("¿Confirmo la legalización?") no tenía con qué
59  // contestar por voz - solo el botón. "sí/confirmo" confirma; un "no"
60  // limpio (sin nada más dicho) solo lo reconoce; cualquier otra cosa se
61  // trata como el mismo tipo de corrección que ya hace "Ajustar por voz"
62  // (texto libre: "el pollo fueron doce kilos"), así decir la corrección
63  // de una vez - sin abrir el diálogo aparte - también funciona.
64  function alMicrofono() {
65    escuchar({
66      alTexto: (texto) => {
67        setEstado("listo");
68        if (esAfirmacion(texto, ["confirma", "legaliza"])) { confirmar(); return; }
69        if (esNegacion(texto) && texto.trim().split(/\s+/).length <= 2) {
70          const msg = "Entendido, no confirmo todavía. Dígame qué insumo ajustar, "
71            + "por ejemplo «el pollo fueron doce kilos».";
72          setRespuesta(msg);
73          hablar(msg);
74          return;
75        }
76        ajustarPorVoz(texto);
77      },
78      alEstado: setEstado,
79      alError: setErr,
80    });
81  }
82
83  async function confirmar() {
84    try {
85      await pedir("/api/legalizacion/confirmar", {
86        method: "POST",
87        body: JSON.stringify({ servicio_id: servicioId }),
88      }, token);
89      setListo(true);
90      hablar("Legalización confirmada. El sobrante volvió a la bodega y la merma quedó registrada.");
91    } catch (e) {
92      // todo lo que el agente muestra tambien lo dice en voz - aqui la
93      // persona acaba de decir "sí" o tocar el botón esperando una
94      // confirmación hablada, y un fallo que solo se ve en rojo se pierde
95      // si no está mirando la pantalla.
96      setErr(e.message);
97      hablar(e.message);
98    }
99  }
100
101  async function ajustarPorVoz(texto) {
102    setPedirAjuste(false);
103    if (!texto) return;
104    setErr("");
105    try {
106      const c = await pedir("/api/legalizacion/ajustar", {
107        method: "POST",
108        body: JSON.stringify({ servicio_id: servicioId, texto }),
109      }, token);
110      setComp(c);
111      const s = c.lineas.filter((l) => l.diferencia < 0);
112      const m = c.lineas.filter((l) => l.diferencia > 0);
113      const msg = "Listo, ajustado. " + narrar(s, m);
114      setRespuesta(msg);
115      hablar(msg);
116    } catch (e) {
117      setErr(e.message);
118      hablar(e.message);
119    }
120  }
121
122  if (err) return <Marco titulo="Legalización"><p className="error">{err}</p></Marco>;

```

```

123   if (!comp) return <Marco titulo="Legalización"><p className="cargando">Cargando...</p></Marco>;
124
125   const sobrante = comp.lineas.filter((l) => l.diferencia < 0);
126   const merma = comp.lineas.filter((l) => l.diferencia > 0);
127
128   if (listo) {
129     const fecha = new Date().toLocaleDateString("es-CO", { day: "numeric", month: "long" });
130     return (
131       <Marco titulo="Legalización · Ajuste confirmado" chip={{ texto: "LEGALIZADO", tipo: "verde" }}>
132         <div className="banner ok">
133           <span className="ico">✓</span>
134           <span>
135             <b>Legalización confirmada – servicio de almuerzo del {fecha}</b>
136             <span>El consumo real quedó registrado y el inventario de la cocina ya refleja los ajustes.</span>
137           </span>
138         </div>
139         <div className="kpis">
140           <div className="card" style={{ borderTop: "4px solid var(--verde)", marginBottom: 0 }}>
141             <h2 style={{ color: "var(--verde)" }}>Devuelto a bodega</h2>
142             {sobrante.length ? sobrante.map((l, i) => (
143               <p key={i} style={{ fontSize: ".9rem" }}>
144                 {fmt(Math.abs(l.diferencia))} {l.unidad} de {l.nombre.toLowerCase()}
145               </p>
146             )) : <p className="vacio">Nada por devolver.</p>}
147           </div>
148           <div className="card" style={{ borderTop: "4px solid var(--amarillo)", marginBottom: 0 }}>
149             <h2 style={{ color: "var(--amarillo-tx)" }}>Merma registrada</h2>
150             {merma.length ? merma.map((l, i) => (
151               <p key={i} style={{ fontSize: ".9rem" }}>
152                 {fmt(l.diferencia)} {l.unidad} de {l.nombre.toLowerCase()} – con nota del chef
153               </p>
154             )) : <p className="vacio">Sin merma en este servicio.</p>}
155           </div>
156           <div className="card" style={{ borderTop: "4px solid var(--azul)", marginBottom: 0 }}>
157             <h2 style={{ color: "var(--azul)" }}>Histórico actualizado</h2>
158             <p style={{ fontSize: ".9rem" }}>
159               Consumo real por porción, para afinar la próxima receta.
160             </p>
161           </div>
162         </div>
163         <div className="card">
164           <h2>Qué pasa ahora con esta información</h2>
165           <div className="registro"><span className="ok">✓</span>
166             <span>El archivo de ajuste queda listo para cargarse a My Inventory con los nombres y códigos oficiales.
167             <div className="registro"><span className="ok">✓</span>
168             <span>El consumo real alimenta el histórico: si el plato gasta siempre más de un insumo que lo que dice
169             <div className="registro"><span className="ok">✓</span>
170             <span>La merma queda trazada con responsable y hora: la revisión ya no depende de la memoria de nadie.
171             </span></div>
172             {esAuditor && (
173               <div className="grilla-botones">
174                 <button className="btn"
175                   onClick={() => ir && ir("reportes", { tabInicial: "analisis" })}>
176                   Ver el reporte del servicio
177                 </button>
178               </div>
179             )}
180           </div>
181         </Marco>
182       );
183     }
184     return (
185       <Marco titulo={`Legalización · ${comp.plato ? comp.plato.toLowerCase() : "servicio de almuerzo"}`}
186         chip={{ texto: "SERVICIO CERRADO", tipo: "" }}>
187         <div className="chips">
188           <span className="chip">
189             {comp.porciones ? `${comp.porciones} porciones servidas` : `${comp.lineas.length} insumos`}
190           </span>
191           <span className="chip verde">{sobrante.length} sobrantes</span>
192           <span className="chip oro">{merma.length} merma{merma.length === 1 ? "" : "s"} por revisar</span>
193         </div>

```

```

194
195 <p className="pista" style={{ marginBottom: 12 }}>
196   Al cerrar el servicio se compara lo pedido con lo realmente usado: cada
197   diferencia queda explicada como sobrante o como merma.
198 </p>
199
200 <div className="card">
201   {comp.lineas.length === 0 ? (
202     <p className="vacio">
203       No hay líneas de servicio todavía. Genere un pedido en el menú Pedidos.
204     </p>
205   ) : (
206     <table>
207       <thead>
208         <tr>
209           <th>Insumo</th><th>Pedido</th><th>Usado</th>
210           <th>Diferencia</th><th>Qué significa</th>
211         </tr>
212       </thead>
213       <tbody>
214         {comp.lineas.map((l) => (
215           <tr key={l.codigo}>
216             <td>{l.nombre}</td>
217             <td>{l.pedido}</td>
218             <td>{l.usado}</td>
219             <td className="dif">
220               {l.diferencia > 0 ? `+${l.diferencia}` : l.diferencia}
221             </td>
222             <td>{l.lectura}</td>
223           </tr>
224         ))}
225       </tbody>
226     </table>
227   )}
228 </div>
229
230 {comp.lineas.length > 0 && (
231   <div className="card">
232     <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", gap: 10 }}>
233       <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
234       <button className={mic-btn} ${estado === "escuchando" ? "escuchando" : ""} `
235         style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
236         onClick={alMicrofono} title="responda «sí» para confirmar, o dicte un ajuste"
237         aria-label="Responder por voz: diga sí para confirmar, o dicte un ajuste">
238         <Icono nombre="microfono" tam={20} />
239       </button>
240     </div>
241     <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
242     <p className="pista" style={{ marginTop: 8 }}>
243       o diga «sí» para confirmar, o dicte un ajuste como «el pollo fueron doce kilos»
244     </p>
245     <details style={{ marginTop: 4 }}>
246       <summary className="pista" style={{ cursor: "pointer" }}>
247         Ver frases exactas que entiende el agente aquí
248       </summary>
249       <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
250         [{"sí", "confirmo", "legaliza",
251           "no (le pregunta qué insumo ajustar)",
252           "el pollo fueron doce kilos", "el aceite fue un litro"].map((ej, i) => (
253           <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
254         ))}
255       </ul>
256     </details>
257     {err && <p className="error" style={{ marginTop: 10 }}>{err}</p>}
258     <div className="grilla-botones">
259       <button className="btn verde" onClick={confirmar}
260         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
261         <Icono nombre="aprobar" tam={16} />
262         Confirmar legalización
263       </button>
264       <button className="btn oro" onClick={() => setVerMerma(true)} disabled={!merma.length}
265         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
266         <Icono nombre="advertencia" tam={16} />
267         Explicar la merma

```

```

268         </button>
269         <button className="btn borde" onClick={() => setPedirAjuste(true)}
270             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
271             <Icono nombre="microfono" tam={16} />
272             Ajustar por voz
273         </button>
274     </div>
275 </div>
276 }}
277
278 {pedirAjuste && (
279     <Dialogo titulo="Ajustar por voz"
280         mensaje="Diga el insumo y cuánto se usó realmente:"
281         conCampo conVoz placeholder="el pollo fueron doce kilos"
282         onAceptar={ajustarPorVoz}
283         onCancel={() => setPedirAjuste(false)} />
284     )}
285
286 {verMerma && (
287     <Dialogo titulo="Merma de este servicio"
288         mensaje={merma.length
289             ? merma.map((l) => `${l.nombre}: +${fmt(l.diferencia)} de más que lo pedido – revisar con el
chef.`).join("\n")
290             : "No hay merma en este servicio."}
291         textoAceptar="Entendido" textoCancelar="Cerrar"
292         onAceptar={() => setVerMerma(false)}
293         onCancel={() => setVerMerma(false)} />
294     )}
295 </Marco>
296 );
297 }

```

frontend/src/vistas/Bodegas.jsx

848 LÍNEAS

El parque en vivo y su detalle.

```

1 import { useEffect, useState, useRef } from "react";
2 import { pedir, BASE, leerToken, descargarReporte, preguntarAsistente } from "../api";
3 import { escuchar, hablar, esAfirmacion, esNegacion, quitarTildes } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import Icono from "../Iconos";
7
8 const ETIQUETA = {
9     pendiente: ["PENDIENTE", "gris"], en_conteo: ["EN CONTEO", "azul"],
10     en_auditoria: ["EN AUDITORÍA", "oro"], cerrada: ["CERRADA", "verde"],
11 };
12 const ICONO_ESTADO = {
13     pendiente: "🕒", en_conteo: "👤", en_auditoria: "🔍", cerrada: "✅",
14 };
15
16 // Frases que este buscador ya entiende sin pasar por el agente general -
17 // se resuelven de una en el propio frontend (_accionLocalDeLista /
18 // _accionLocalDeResultado más abajo). Documentarlas aquí es lo mismo que
19 // el desplegable "Ver frases exactas" de las demás pantallas.
20 function _ejemplosLista(esAuditor) {
21     return [
22         "arroz, aceite, tres tablas para picar...",
23         "bodega nueva",
24         ...(esAuditor ? ["exportar estado", "asignar personas a bodegas"] : []),
25         "ver movimientos", "comparar con la receta",
26         ...(esAuditor ? ["generar reporte"] : []),
27         "volver al tablero",
28     ];
29 }
30 function _ejemplosDetalle(esAuditor, estado) {
31     return [
32         "un artículo de esta bodega",
33         ...(estado === "pendiente" || estado === "en_conteo" ? ["seguir contando"] : []),
34         ...(esAuditor && estado === "cerrada" ? ["reabrir"] : []),
35         "todos los artículos",
36         ...(esAuditor ? ["descargar", "ver en el panel gerencial"] : []),

```

```

37     "cerrar detalle",
38   ];
39 }
40
41 export default function Bodegas({ token, usuario, ir }) {
42   const [lista, setLista] = useState(null);
43   const [filtro, setFiltro] = useState("todas");
44   const [detalle, setDetalle] = useState(null);
45   const [detalleId, setDetalleId] = useState(null);
46   const [cargandoDetalle, setCargandoDetalle] = useState(false);
47   const [busca, setBusca] = useState("");
48   const [consulta, setConsulta] = useState(null);
49   const [escuchando, setEscuchando] = useState(false);
50   const [movimientos, setMovimientos] = useState(null);
51   const [enRecetas, setEnRecetas] = useState(null);
52   const [articulosBodega, setArticulosBodega] = useState(null);
53   const [buscaArticuloBodega, setBuscaArticuloBodega] = useState("");
54   const [escuchandoBodega, setEscuchandoBodega] = useState(false);
55   // Cuando lo buscado en el detalle no es ningún artículo de ESTA bodega,
56   // la respuesta del agente general (pregunta suelta u orden de navegar)
57   // - mismo patrón que "consulta" en la lista principal.
58   const [respuestaAgenteDetalle, setRespuestaAgenteDetalle] = useState(null);
59   const [msg, setMsg] = useState("");
60   const [pedirMotivo, setPedirMotivo] = useState(false);
61   const [pidiendoBodegaNueva, setPidiendoBodegaNueva] = useState(false);
62   const esAuditor = usuario?.perfil === "auditor";
63   const rec = useRef(null);
64   // Igual que en el selector de bodega de Conteo: cada escucha de esta
65   // búsqueda lleva su propio número, y si el micrófono anterior entrega
66   // su resultado tarde (mientras todavía se está diciendo la sugerencia)
67   // se descarta en vez de procesarse como una búsqueda nueva.
68   const idEscuchaBusqueda = useRef(0);
69
70   useEffect(() => {
71     // si falla, resuelve a [] en vez de dejar lista en null para siempre:
72     // sin esto, un fallo de red aquí dejaba el tablero mostrando
73     // "Cargando..." sin fin, sin manera de saber que algo salió mal.
74     pedir("/api/bodegas?propias=1", {}, token).then(setLista)
75       .catch((e) => { setLista([]); setMsg(e.message); });
76     /* tablero en vivo: el WebSocket avisa a todas las tabletas */
77     const ws = new WebSocket(
78       BASE.replace("http", "ws") + "/api/bodegas/estado?token=" + (token || leerToken())
79     );
80     ws.onmessage = (e) => {
81       const estados = JSON.parse(e.data);
82       setLista((previa) => {
83         const prev = previa || [];
84         const hayNuevas = estados.some((x) => !prev.some((b) => b.id === x.id));
85         if (hayNuevas) {
86           // bodega creada despues de cargar el tablero: se necesita el
87           // listado completo (referencias, persona, avance) que el
88           // WebSocket no manda, no solo el id/estado.
89           pedir("/api/bodegas?propias=1", {}, token).then(setLista).catch(() => {});
90           return prev;
91         }
92         return prev.map((b) => {
93           const n = estados.find((x) => x.id === b.id);
94           return n ? { ...b, estado: n.estado } : b;
95         });
96       });
97     };
98     return () => ws.close();
99   }, [token]);
100
101   const vistas = (lista || []).filter((b) => filtro === "todas" || b.estado === filtro);
102   const cuenta = (e) => (lista || []).filter((b) => b.estado === e).length;
103
104   async function verDetalle(id) {
105     // Si se llega desde la sugerencia de bodega del buscador de
106     // ingredientes, "consulta" sigue puesto - sin limpiarlo, la sección
107     // de detalle no se muestra (su condición exige "!consulta").
108     setConsulta(null);
109     setDetalleId(id);
110     setArticulosBodega(null);

```



```

111     setBuscaArticuloBodega("");
112     setRespuestaAgenteDetalle(null);
113     setCargandoDetalle(true);
114     try {
115         setDetalle(await pedir(`/api/bodegas/${id}/detalle`, {}, token));
116     } finally {
117         setCargandoDetalle(false);
118     }
119 }
120 async function verTodosLosArticulos() {
121     if (articulosBodega) { setArticulosBodega(null); return; }
122     setArticulosBodega(await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token));
123 }
124 async function buscarEnBodega(texto) {
125     setBuscaArticuloBodega(texto);
126     setRespuestaAgenteDetalle(null);
127     if (texto && !articulosBodega) {
128         setArticulosBodega(await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token));
129     }
130 }
131 function _filtrarArticulosBodega(texto, lista) {
132     const palabras = texto.toLowerCase().replace(/[,;.] /g, " ").split(/\s+/).filter(Boolean);
133     if (!palabras.length) return lista || [];
134     return (lista || []).filter((a) => {
135         const nombre = a.articulo.toLowerCase();
136         return palabras.every((p) => nombre.includes(p));
137     });
138 }
139 // Al confirmar (Enter, o al terminar de dictar): si lo escrito/dicho no
140 // encuentra ningún artículo DENTRO de esta bodega, cae al mismo agente
141 // general que ya usa la lista principal - un solo cuadro para todo,
142 // también aquí, en vez de un segundo "Pregúntele al agente" aparte.
143 // Igual que _accionLocalDeResultado, pero para los botones del DETALLE
144 // de una bodega (Cerrar detalle, Seguir contando...) - se revisan antes
145 // que la búsqueda de artículo/el agente general, porque esos botones
146 // no tienen equivalente en ningún otro lado: decir "cerrar detalle" sin
147 // esto caía en el agente general, que ni sabe qué botones hay en esta
148 // pantalla en particular ("Esa opción no la puedo hacer por voz").
149 function _accionLocalDeDetalle(texto) {
150     const t = quitarTildes(texto.toLowerCase().replace(/[,;:;!¿?¡]/g, "").trim());
151     if (/^b(cerrar|cierra|salir)\b/.test(t)) return "cerrar";
152     if (/^b(seguir|sigue|continua|w*|contar|cuenta)\b/.test(t)
153         && (detalle?.estado === "pendiente" || detalle?.estado === "en_conteo")) return "contar";
154     if (/^b(reabrir|reabre)\b/.test(t) && esAuditor && detalle?.estado === "cerrada") return "reabrir";
155     if (/^b(articulos?)\b/.test(t)) return "articulos";
156     if (/^b(descargar)\b/.test(t) && esAuditor) return "descargar";
157     if (/^b(panel)\b/.test(t) && esAuditor) return "panel";
158     return null;
159 }
160 async function confirmarBusquedaEnBodega(texto, miId) {
161     if (!texto || !texto.trim()) return;
162     if (miId === undefined) miId = ++idEscuchaBusqueda.current;
163     const accion = _accionLocalDeDetalle(texto);
164     if (accion === "cerrar") { setDetalle(null); setDetalleId(null); return; }
165     if (accion === "contar") { ir && ir("conteo", { bodegaSugerida: detalle.bodega }); return; }
166     if (accion === "reabrir") { setPedirMotivo(true); return; }
167     if (accion === "articulos") { verTodosLosArticulos(); return; }
168     if (accion === "descargar") { exportarDetalle(); return; }
169     if (accion === "panel") { ir && ir("panel"); return; }
170     setBuscaArticuloBodega(texto);
171     let lista = articulosBodega;
172     if (!lista) {
173         lista = await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token);
174         if (miId !== idEscuchaBusqueda.current) return;
175         setArticulosBodega(lista);
176     }
177     if (_filtrarArticulosBodega(texto, lista).length > 0) {
178         setRespuestaAgenteDetalle(null);
179         return;
180     }
181     const r = await preguntarAsistente(texto, "bodegas", token);
182     if (miId !== idEscuchaBusqueda.current) return;
183     setRespuestaAgenteDetalle(r);
184     hablar(r.respuesta_hablada);

```

```

185     if (r.accion === "navegar" && r.destino && ir) {
186         ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined });
187     }
188 }
189 function buscarEnBodegaPorVoz() {
190     setMsg("");
191     const miId = ++idEscuchaBusqueda.current;
192     rec.current = escuchar({
193         alTexto: (t) => {
194             if (miId !== idEscuchaBusqueda.current) return;
195             confirmarBusquedaEnBodega(t, miId);
196         },
197         alEstado: (e) => setEscuchandoBodega(e === "escuchando"),
198         alError: setMsg,
199     });
200 }
201 // Frases que activan un botón YA VISIBLE en el resultado de una
202 // búsqueda anterior ("volver al tablero", "ver movimientos...), en
203 // vez de tratarse como una búsqueda nueva. Van ANTES que cualquier
204 // otra cosa: "tablero" es también parte de un producto real
205 // ("BORRADOR PARA TABLERO FELPA"), así que sin este chequeo "volver al
206 // tablero" perdía contra ese artículo en la búsqueda por palabras y
207 // nunca se reconocía como la orden que era.
208 function _accionLocalDeResultado(texto) {
209     const t = quitarTildes(texto.toLowerCase().replace(/[.,;:!?;|]/g, "").trim());
210     if (/b(volver|tablero|atras|regresar)\b/.test(t)) return "volver";
211     if (/bmovimientos?\b/.test(t)) return "movimientos";
212     if (/brecetas?\b/.test(t)) return "receta";
213     if (/breporte\b/.test(t)) return "reporte";
214     return null;
215 }
216 // "miId" solo llega cuando la búsqueda vino de buscarPorVoz() (ver ese
217 // comentario) - una búsqueda escrita y con clic en "Buscar" no debe
218 // quedarse esperando una respuesta hablada que nadie va a dar.
219 // Órdenes sobre los botones de la lista principal ("Bodega nueva",
220 // "Exportar estado", "Asignar personas a bodegas") - mismo patrón que
221 // _accionLocalDeResultado/_accionLocalDeDetalle: se revisan antes que
222 // cualquier búsqueda, porque el agente general no sabe qué botones hay
223 // en esta pantalla en particular.
224 function _accionLocalDeLista(texto) {
225     const t = quitarTildes(texto.toLowerCase().replace(/[.,;:!?;|]/g, "").trim());
226     if (/bbodega\b/.test(t) && /b(nueva|crear|solicitar)\b/.test(t)) return "nueva";
227     if (/bexportar\b/.test(t) && esAuditor) return "exportar";
228     if (/basignar\b/.test(t) && esAuditor) return "asignar";
229     return null;
230 }
231 async function buscarArticulo(codigo = "", texto = busca, miId) {
232     if (!texto.trim()) return;
233     if (!consulta && !detalle) {
234         const accionLista = _accionLocalDeLista(texto);
235         if (accionLista === "nueva") { setPidiendoBodegaNueva(true); return; }
236         if (accionLista === "exportar") { exportarEstado(); return; }
237         if (accionLista === "asignar") { ir && ir("ajustes", { tabInicial: "usuarios" }); return; }
238     }
239     // Si queda una sugerencia de bodega pendiente en pantalla, un "sí"/
240     // "no" nuevo la responde - sin importar si llegó como continuación
241     // de la misma escucha o como una búsqueda nueva escrita/hablada
242     // aparte (tocando "Buscar" de nuevo, por ejemplo). Antes, un "sí"
243     // que no llegara EXACTAMENTE como respuesta de esa misma escucha se
244     // trataba como una búsqueda nueva de "sí" a secas, y el agente
245     // general (que no tiene ni idea de la sugerencia pendiente)
246     // respondía cualquier cosa en vez de abrir el detalle.
247     if (consulta?.sugerencia_bodega_id) {
248         if (esAfirmacion(texto, ["ver"])) { verDetalle(consulta.sugerencia_bodega_id); return; }
249         if (esNegacion(texto)) {
250             setConsulta(null);
251             setBusca("");
252             const msg = "Listo, queda ahí. Puede seguir buscando cuando quiera.";
253             setMsg(msg);
254             hablar(msg);
255             return;
256         }
257     }
258     if (consulta?.bodegas?.length > 0) {

```

```

259     const accion = _accionLocalDeResultado(texto);
260     if (accion === "volver") { volverAlTablero(); return; }
261     if (accion === "movimientos") { verMovimientos(); return; }
262     if (accion === "receta") { verEnRecetas(); return; }
263     if (accion === "reporte" && esAuditor) { generarReporteArticulo(); return; }
264   }
265   setBusca(texto);
266   setMovimientos(null);
267   setEnRecetas(null);
268   const r = await pedir(
269     `/api/articulos/consulta?q=${encodeURIComponent(texto)}&codigo=${codigo}`, {}, token);
270   if (miId !== undefined && miId !== idEscuchaBusqueda.current) return;
271   setConsulta(r);
272   // Un solo buscador para todo: si ni fue un artículo ni una bodega,
273   // el backend ya lo intentó como pregunta suelta o como orden de
274   // navegar ("llévame a reportes", "abra kiosco taquilla ayb") - si
275   // trae una acción, se seguía igual que hace AsistenteVoz.
276   if (r.accion === "navegar" && r.destino && ir) {
277     hablar(r.resumen);
278     ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined });
279     return;
280   }
281   // Si la búsqueda fue por voz y salió una bodega sugerida, no basta
282   // con decir la sugerencia y dejarla ahí esperando un clic: se
283   // pregunta y se escucha la respuesta, igual que ya hace Conteo al
284   // confirmar una bodega - "tocar el botón" no debería ser la única
285   // forma de seguir cuando se llegó hasta aquí hablando.
286   if (miId !== undefined && r.sugerencia_bodega_id) {
287     await hablar(`${r.resumen} Diga «sí» para ver el detalle.`);
288     if (miId !== idEscuchaBusqueda.current) return;
289     rec.current = escuchar({
290       alTexto: (respuesta) => {
291         if (miId !== idEscuchaBusqueda.current) return;
292         if (esAfirmacion(respuesta, ["ver"])) {
293           verDetalle(r.sugerencia_bodega_id);
294         } else if (esNegacion(respuesta)) {
295           const msg = "Listo, queda ahí. Puede seguir buscando cuando quiera.";
296           setMsg(msg);
297           hablar(msg);
298         } else {
299           setMsg("No le entendí un «sí» o un «no». Use el botón «Ver esta bodega.»");
300         }
301       },
302       alEstado: (e) => setEscuchando(e === "escuchando"),
303       alError: setMsg,
304     });
305     return;
306   }
307   hablar(r.resumen);
308 }
309 function buscarPorVoz() {
310   setMsg("");
311   const miId = ++idEscuchaBusqueda.current;
312   rec.current = escuchar({
313     alTexto: (t) => {
314       if (miId !== idEscuchaBusqueda.current) return;
315       buscarArticulo("", t, miId);
316     },
317     alEstado: (e) => setEscuchando(e === "escuchando"),
318     alError: setMsg,
319   });
320 }
321 function volverAlTablero() {
322   setConsulta(null);
323   setBusca("");
324 }
325 async function verMovimientos() {
326   setMovimientos(await pedir(`/api/articulos/${consulta.codigo}/movimientos`, {}, token));
327 }
328 async function verEnRecetas() {
329   setEnRecetas(await pedir(`/api/articulos/${consulta.codigo}/en-recetas`, {}, token));
330 }
331 async function generarReporteArticulo() {
332   setMsg("");

```

```

333     try {
334         const r = await pedir("/api/reportes?formato=xlsx", { method: "POST" }, token);
335         await descargarReporte(r.archivo, token);
336         setMsg("Reporte generado y descargado.");
337     } catch (e) { setMsg(e.message); }
338 }
339 async function reabrir(motivo) {
340     setPedirMotivo(false);
341     if (!motivo || !motivo.trim()) return;
342     try {
343         await pedir(`/api/bodegas/${detalleId}/reabrir`, {
344             method: "POST", body: JSON.stringify({ motivo }),
345         }, token);
346         setMsg("Bodega reabierto: queda registrada la justificación.");
347         verDetalle(detalleId);
348         pedir("/api/bodegas?propias=1", {}, token).then(setLista).catch(() => {});
349     } catch (e) { setMsg(e.message); }
350 }
351
352 async function solicitarBodegaNueva(nombre) {
353     setPidiendoBodegaNueva(false);
354     if (!nombre || !nombre.trim()) return;
355     try {
356         const r = await pedir("/api/bodegas/crear-pendiente", {
357             method: "POST", body: JSON.stringify({ nombre }),
358         }, token);
359         setMsg(r.respuesta_hablada || "Solicitud enviada.");
360     } catch (e) { setMsg(e.message); }
361 }
362
363 async function exportarEstado() {
364     setMsg("");
365     try {
366         const r = await pedir("/api/bodegas/exportar-estado?formato=xlsx", { method: "POST" }, token);
367         await descargarReporte(r.archivo, token);
368         setMsg("Estado del tablero exportado y descargado.");
369     } catch (e) { setMsg(e.message); }
370 }
371
372 async function exportarDetalle() {
373     setMsg("");
374     try {
375         const r = await pedir(`/api/bodegas/${detalleId}/exportar-detalle?formato=xlsx`,
376             { method: "POST" }, token);
377         await descargarReporte(r.archivo, token);
378         setMsg("Detalle de la bodega exportado y descargado.");
379     } catch (e) { setMsg(e.message); }
380 }
381
382 const tituloDetalle = detalle ? `Bodegas · ${detalle.bodega.toLowerCase().replace(/\b\w/g, (c) =>
c.toUpperCase())}` : "";
383 const chipDetalle = detalle ? (ETIQUETA[detalle.estado] || [detalle.estado?.toUpperCase(), ""]) : null;
384 // solo bloquea la pantalla con "Cargando..." en la primera carga de un
385 // detalle: si ya había uno (ej. reabrir() refrescando el mismo), se deja
386 // ver el dato viejo mientras llega el nuevo, en vez de taparlo con un
387 // aviso de carga que compite con el detalle todavía visible.
388 const mostrarCargandoDetalle = cargandoDetalle && !detalle;
389
390 return (
391     <Marco titulo={consulta ? "Bodegas · consulta de artículo" : detalle ? tituloDetalle : "Bodegas · estado en
vivo"}>
392         chip={consulta ? { texto: "BÚSQUEDA GLOBAL", tipo: "borde azul" }
393             : detalle ? { texto: chipDetalle[0], tipo: `borde ${chipDetalle[1]}` }
394             : { texto: "EN VIVO", tipo: "verde" }}>
395
396         {mostrarCargandoDetalle && <p className="cargando">Cargando...</p>}
397
398         {!detalle && !mostrarCargandoDetalle && (
399             <div className="card">
400                 <div style={{ display: "flex", gap: 10, alignItems: "center", flexWrap: "wrap" }}>
401                     <span style={{ color: "var(--grafito)" }}></span>
402                     <input value={busca} onChange={(e) => setBusca(e.target.value)}
403                         onKeyDown={(e) => e.key === "Enter" && buscarArticulo()}
404                         placeholder="arroz, aceite, cazuela..."

```

```

405         aria-label="Buscar artículo, o pregúntele algo al agente"
406         style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
407             border: "1px solid var(--borde)", borderRadius: 12 }} />
408     <button className={`mic-btn ${escuchando ? "escuchando" : ""}`}
409         style={{ width: 46, height: 46, fontSize: "1.2rem" }}
410         onClick={buscarPorVoz} title="o pregunte por voz"
411         aria-label="Buscar por voz, o pregúntele al agente"><Icono nombre="microfono" tam={22} /></button>
412 </div>
413 {!consulta && (
414     <>
415         <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
416         <details style={{ marginTop: 4 }}>
417             <summary className="pista" style={{ cursor: "pointer" }}>
418                 Ver frases exactas que entiende el agente aquí
419             </summary>
420             <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
421                 {_ejemplosLista(esAuditor).map((ej, i) => (
422                     <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
423                 ))}
424             </ul>
425         </details>
426     </>
427 )}
428
429 {consulta && (
430     <>
431         <p className="rotulo" style={{ marginTop: 14 }}>CuentaVoz responde</p>
432         <p className="burbuja" aria-live="polite" aria-atomic="true">{consulta.resumen}</p>
433         {consulta.sugerencia_bodega && (
434             <div className="banner" style={{ marginTop: 10 }}>
435                 <span className="ico">🔍</span>
436                 <span>
437                     <b>{consulta.sugerencia_bodega}</b> es el nombre de una bodega, no de un artículo.
438                     <span> Este buscador es para ingredientes – vea su detalle para
439                         revisarla antes de abrirla.</span>
440                 </span>
441                 <button className="btn" style={{ flex: "none" }}
442                     onClick={() => verDetalle(consulta.sugerencia_bodega_id)}>
443                     Ver esta bodega
444                 </button>
445             </div>
446         )}
447         {consulta.ambiguo && consulta.alternativas?.length > 0 && (
448             <div className="chips" style={{ marginTop: 10 }}>
449                 <span className="pista" style={{ width: "100%" }}>¿Era este otro?</span>
450                 {consulta.alternativas.map((a) => (
451                     <button key={a.codigo} className="chip oro"
452                         onClick={() => buscarArticulo(a.codigo)}>
453                         {a.nombre}
454                     </button>
455                 ))}
456             </div>
457         )}
458
459         {consulta.bodegas?.length > 0 && (
460             <>
461                 <div className="kpis" style={{ marginTop: 14 }}>
462                     <div className="kpi">
463                         <div className="kpi-cabeza">
464                             <span className="icono-kpi">📦</span>
465                             <small>Total en el sistema</small>
466                         </div>
467                         <b>{consulta.total.toLocaleString("es-CO", { maximumFractionDigits: 1 })} {consulta.unidad}</b>
468                         <i>en {consulta.bodegas.length} bodegas</i>
469                     </div>
470                     <div className="kpi">
471                         <div className="kpi-cabeza">
472                             <span className="icono-kpi">📦</span>
473                             <small>Consumo del mes</small>
474                         </div>
475                         <b>{consulta.consumo_mes} {consulta.unidad}</b>
476                         <i>{consulta.servicios_mes} servicios</i>
477                     </div>
478                     <div className="kpi verde">

```

```

479         <div className="kpi-cabeza">
480             <span className="icono-kpi">📊</span>
481             <small>Cobertura estimada</small>
482         </div>
483         <b>{consulta.cobertura_dias != null ? `${consulta.cobertura_dias} días` : "-"}</b>
484         <i>{consulta.cobertura_dias != null ? "al ritmo actual" : "sin consumo reciente"}</i>
485     </div>
486 </div>
487
488     <table style={{ marginTop: 6 }}>
489         <thead><tr><th>Bodega</th><th>Cantidad</th><th>Última toma</th><th>Estado</th></tr></thead>
490         <tbody>
491             {consulta.bodegas.map((b, i) => (
492                 <tr key={i}>
493                     <td>{b.bodega}</td><td>{b.cantidad} {consulta.unidad}</td>
494                     <td>{b.ultima_toma}</td><td>{ETIQUETA[b.estado]?.[0] || b.estado}</td>
495                 </tr>
496             ))}
497         </tbody>
498     </table>
499
500     <div className="grilla-botones">
501         {esAuditor && (
502             <button className="btn" onClick={generarReporteArticulo}>Generar reporte</button>
503         )}
504         <button className="btn borde" onClick={verMovimientos}>Ver movimientos</button>
505         <button className="btn borde" onClick={verEnRecetas}>Comparar con la receta</button>
506         <button className="btn gris" onClick={volverAlTablero}>← Volver al tablero</button>
507     </div>
508 </>
509 )}
510 </>
511 )}
512 </div>
513 )}
514
515 {msg && <p className="msg-ok">{msg}</p>}
516
517 {!consulta && detalle && (
518     <div className="card">
519         <div style={{ display: "flex", gap: 10, alignItems: "center", flexWrap: "wrap" }}>
520             <span style={{ color: "var(--grafito)" }}>🔍</span>
521             <input value={buscaArticuloBodega}
522                 onChange={(e) => buscarEnBodega(e.target.value)}
523                 onKeyDown={(e) => e.key === "Enter" && confirmarBusquedaEnBodega(buscaArticuloBodega)}
524                 placeholder="artículo en esta bodega, o pregúntele algo al agente..."
525                 aria-label="Buscar artículo en esta bodega, o pregúntele algo al agente"
526                 style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
527                     border: "1px solid var(--borde)", borderRadius: 12 }} />
528             <button className={`mic-btn ${escuchandoBodega ? "escuchando" : ""}`}
529                 style={{ width: 46, height: 46, fontSize: "1.2rem" }}
530                 onClick={buscarEnBodegaPorVoz} title="o pregunte por voz"
531                 aria-label="Buscar por voz en esta bodega, o pregúntele al agente"><Icono nombre="microfono" tam=
532 {22} /></button>
533         </div>
534         <p className="pista" style={{ marginTop: 8, marginBottom: 4 }}>
535             o pregunte por voz - si no es un artículo de esta bodega, se lo pregunta al agente
536         </p>
537         <details style={{ marginBottom: 14 }}>
538             <summary className="pista" style={{ cursor: "pointer" }}>
539                 Ver frases exactas que entiende el agente aquí
540             </summary>
541             <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
542                 {ejemplosDetalle(esAuditor, detalle.estado).map((ej, i) => (
543                     <li key={i} className="pista" style={{ marginBottom: 4 }}><ej></li>
544                 ))}
545             </ul>
546         </details>
547         {respuestaAgenteDetalle && (
548             <p className="rotulo">CuentaVoz responde</p>
549             <p className="burbuja" style={{ marginBottom: 14 }}
550                 aria-live="polite" aria-atomic="true">{respuestaAgenteDetalle.respuesta_hablada}</p>
551         </>

```

```

552     })
553
554     <div className="chips">
555         <span className="chip">{detalle.referencias} referencias</span>
556         {detalle.estado === "cerrada" && detalle.hora_cierre && (
557             <span className="chip verde">Cerrada {detalle.hora_cierre}</span>
558         )}
559         {detalle.personas && <span className="chip azul">{detalle.personas}</span>}
560         {detalle.alertas_resueltas > 0 && (
561             <span className="chip oro">
562                 {detalle.alertas_resueltas} alerta{detalle.alertas_resueltas === 1 ? "" : "s"} resuelta
563                 {detalle.alertas_resueltas === 1 ? "" : "s"}
564             </span>
565         )}
566     </div>
567
568     <div className="kpis">
569         <div className="kpi verde">
570             <div className="kpi-cabeza">
571                 <span className="icono-kpi">✔</span>
572                 <small>Exactitud de esta bodega</small>
573             </div>
574             <b>{detalle.exactitud}</b>
575             <i>{detalle.diferencias.length} diferencias de {detalle.referencias}</i>
576         </div>
577         <div className="kpi">
578             <div className="kpi-cabeza">
579                 <span className="icono-kpi">🕒</span>
580                 <small>Duración del conteo</small>
581             </div>
582             <b>{detalle.duracion_min != null ? `${detalle.duracion_min} min` : "-"}</b>
583             <i>{detalle.tiempo_papel_min
584                 ? `frente a ~${detalle.tiempo_papel_min} min en papel (estimado)`
585                 : "conteo aún en curso"}</i>
586         </div>
587         <div className="kpi">
588             <div className="kpi-cabeza">
589                 <span className="icono-kpi">📦</span>
590                 <small>Unidades en stock</small>
591             </div>
592             <b>{detalle.unidades_stock.toLocaleString("es-CO")}</b>
593             <i>sumadas todas las referencias</i>
594         </div>
595         <div className="kpi oro">
596             <div className="kpi-cabeza">
597                 <span className="icono-kpi">🕒</span>
598                 <small>Última toma anterior</small>
599             </div>
600             <b>{detalle.ultima_toma_anterior
601                 ? new Date(detalle.ultima_toma_anterior.fecha).toLocaleDateString("es-CO", { day: "numeric", month:
"short" })
602                 : "-"}</b>
603             <i>{detalle.ultima_toma_anterior ? `exactitud ${detalle.ultima_toma_anterior.exactitud}%` : "sin
cierres previos"}</i>
604         </div>
605     </div>
606
607     <div className="dos-columnas" style={{ gap: 16, marginTop: 4 }}>
608         <div>
609             <h2>Referencias con diferencia</h2>
610             {detalle.diferencias.length > 0 ? (
611                 <table>
612                     <thead><tr><th>Artículo</th><th>Contado</th><th>Sistema</th><th>Dif.</th></tr></thead>
613                     <tbody>
614                         {detalle.diferencias.map((d, i) => (
615                             <tr key={i}><td>{d.articulo}</td><td>{d.contado}</td>
616                             <td>{d.sistema}</td><td className="dif">{d.diferencia}</td></tr>
617                         ))}
618                     </tbody>
619                 </table>
620             ) : <p className="vacio">Todas las referencias cuadraron exactas.</p>
621             {detalle.diferencias.length > 0 && detalle.diferencias.length < detalle.referencias && (
622                 <p className="pista" style={{ marginTop: 8 }}>
623                     Las otras {detalle.referencias - detalle.diferencias.length} referencias cuadraron exactas.

```



```

624     </p>
625   })
626   {detalle.diferencias.length > 0 && detalle.revisor && (
627     <p className="pista">
628       Las {detalle.diferencias.length} diferencias fueron revisadas por {detalle.revisor} al cierre.
629     </p>
630   })
631 </div>
632
633 <div>
634   <h3>Línea de tiempo de la toma</h3>
635   {detalle.hitos.length > 0 ? detalle.hitos.map((h, i) => (
636     <div className="registro" key={i}>
637       <span style={{
638         width: 10, height: 10, borderRadius: "50%", flex: "none",
639         background: h.tipo === "verde" ? "var(--verde)"
640           : h.tipo === "oro" ? "var(--amarillo)" : "var(--azul)",
641       }} />
642       <b style={{ color: "var(--grafito)" }}>{h.hora}</b>
643       <span>{h.texto}</span>
644     </div>
645   )) : <p className="vacio">Sin movimientos registrados todavía.</p>
646 </div>
647 </div>
648
649 {articulosBodega && !respuestaAgenteDetalle && (
650   <div style={{ marginTop: 16 }}>
651     <h3 style={{ margin: 0 }}>
652       {buscaArticuloBodega
653         ? `Artículos que coinciden con "${buscaArticuloBodega}"`
654         : `Todos los artículos (${articulosBodega.length})`}
655     </h3>
656     <div style={{ maxHeight: 420, overflowY: "auto", marginTop: 10 }}>
657       <table>
658         <thead><tr><th>Código</th><th>Artículo</th><th>Unidad</th><th>SD</th></tr></thead>
659         <tbody>
660           {_filtrarArticulosBodega(buscaArticuloBodega, articulosBodega).map((a) => (
661             <tr key={a.codigo}>
662               <td>{a.codigo}</td><td>{a.articulo}</td>
663               <td>{a.unidad}</td><td>{a.sd.toLocaleString("es-CO")}</td>
664             </tr>
665           ))}
666         </tbody>
667       </table>
668     </div>
669   </div>
670 )}
671
672 <div className="grilla-botones">
673   {(detalle.estado === "pendiente" || detalle.estado === "en_conteo") && (
674     // Este detalle es solo de lectura (estado/historial) - para
675     // de verdad contar hay que ir a Conteo. Sin este atajo,
676     // alguien que llega aquí buscando "abrir la bodega" se
677     // queda sin poder hacerlo y tiene que volver a decir/buscar
678     // el nombre desde cero en otra pantalla.
679     <button className="btn" onClick={() => ir && ir("conteo", { bodegaSugerida: detalle.bodega })}
680       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
681       <Icono nombre="conteo" tam={18} />
682       {detalle.estado === "en_conteo" ? "Seguir contando" : "Contar esta bodega"}
683     </button>
684   )}
685   {esAuditor && detalle.estado === "cerrada" && (
686     <button className="btn borde" onClick={() => setPedirMotivo(true)}
687       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
688       <Icono nombre="candado" tam={18} />
689       Reabrir la bodega
690     </button>
691   )}
692   <button className="btn borde" onClick={verTodosLosArticulos}
693     style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
694     <Icono nombre="reportes" tam={18} />
695     {articulosBodega ? "Ocultar artículos" : `Ver todos los artículos (${detalle.referencias})`}
696   </button>
697   {esAuditor && (

```



```

698         <button className="btn borde" onClick={exportarDetalle}
699             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
700             <Icono nombre="descargar" tam={18} />
701             Descargar detalle
702         </button>
703     )}
704     {esAuditor && (
705         <button className="btn" onClick={() => ir && ir("panel")}
706             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
707             <Icono nombre="panel" tam={18} />
708             Ver en el panel gerencial
709         </button>
710     )}
711     <button className="btn gris" onClick={() => { setDetalle(null); setDetalleId(null); }}
712         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
713         <Icono nombre="salir" tam={18} />
714         Cerrar detalle
715     </button>
716 </div>
717 {esAuditor && detalle.estado === "cerrada" && (
718     <p className="pista" style={{ marginTop: 10 }}>
719         Reabrir una bodega cerrada exige justificación escrita y queda en el registro de trazabilidad.
720     </p>
721 )}
722 </div>
723 )}
724
725 {!consulta && !detalle && !mostrarCargandoDetalle && lista === null && (
726     <p className="cargando">Cargando...</p>
727 )}
728
729 {!consulta && !detalle && !mostrarCargandoDetalle && lista !== null && (
730     <>
731         <div className="chips">
732             {[["todas", `${lista.length} bodegas`, ""],
733              ["cerrada", `${cuenta("cerrada")} cerradas`, "verde"],
734              ["en_conteo", `${cuenta("en_conteo")} en conteo`, ""],
735              ["en_auditoria", `${cuenta("en_auditoria")} en auditoría`, "oro"],
736              ["pendiente", `${cuenta("pendiente")} pendientes`, "gris"]].map(([k, t, c]) => (
737                  <button key={k} className={`chip ${filtro === k ? "azul" : c}`}
738                      onClick={() => setFiltro(k)}>{t}</button>
739              ))}
740         </div>
741
742         /* Arriba, junto a los filtros, y no al final de las 54 tarjetas -
743            son acciones de la pantalla completa, no de una bodega en
744            particular, así que no tiene sentido obligar a bajar todo el
745            tablero para encontrarlas. */
746         <div className="grilla-botones" style={{ marginBottom: 16 }}>
747             {esAuditor && (
748                 <button className="btn borde" onClick={exportarEstado}
749                     style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
750                     <Icono nombre="descargar" tam={18} />
751                     Exportar estado
752                 </button>
753             )}
754             {esAuditor && (
755                 <button className="btn"
756                     onClick={() => ir && ir("ajustes", { tabInicial: "usuarios" })}
757                     style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
758                     <Icono nombre="perfil" tam={18} />
759                     Asignar personas a bodegas
760                 </button>
761             )}
762         /* Sin esAuditor a proposito: quien suele topa con una bodega
763            que no esta en el catalogo es el auxiliar en el terreno, no
764            el administrador desde su escritorio. */
765         <button className="btn borde" onClick={() => setPidiendoBodegaNueva(true)}
766             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
767             <Icono nombre="bodegas" tam={18} />
768             Bodega nueva
769         </button>
770     </div>
771 )}

```

```

772     {lista.length === 0 ? (
773       <p className="vacio">No hay bodegas para mostrar.</p>
774     ) : (
775       <div className="grid-bodegas">
776         {vistas.map((b) => {
777           const [txt, cls] = ETIQUETA[b.estado] || ["?", "gris"];
778           let detalleLinea;
779           if (b.estado === "en_conteo" && b.persona) {
780             detalleLinea = `${b.persona} · ${b.avance_pct ?? 0}%`;
781           } else if (b.estado === "en_auditoria" && b.persona) {
782             detalleLinea = `${b.persona} · ${b.diferencias ?? 0} dif.`;
783           } else if (b.estado === "cerrada" && b.hora_cierre) {
784             detalleLinea = `${b.referencias} refs · ${b.hora_cierre}`;
785           } else if (b.estado === "pendiente") {
786             detalleLinea = "sin iniciar";
787           } else {
788             detalleLinea = `${b.referencias} referencias`;
789           }
790           return (
791             <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
792               onClick={() => verDetalle(b.id)}>
793               <b><span className="icono-tarjeta">{ICONO_ESTADO[b.estado] || "🏠"}</span>{b.bodega}</b>
794               <span className={`chip ${cls} est`} est={txt}></span>
795               <small>{detalleLinea}</small>
796             </button>
797           );
798         })}
799       </div>
800     )}
801     <p className="pista" style={{ marginTop: 12 }}>
802       Cada tarjeta se actualiza al instante por WebSocket cuando alguien abre o cierra una bodega.
803     </p>
804   </>
805 )}
806
807 {pidiendoBodegaNueva && (
808   <Dialogo titulo={esAuditor ? "Bodega nueva" : "Solicitar bodega nueva"}
809     mensaje={esAuditor
810       ? "Como administrador, la bodega entra directo al catálogo: no necesita otra aprobación."
811       : "Queda pendiente de aprobación: no entra al catálogo hasta que un administrador la confirme
desde Auditoría → Aprobaciones."}
812     conCampo conVoz placeholder="nombre de la bodega"
813     etiquetaCampo="Nombre de la bodega"
814     textoAceptar={esAuditor ? "Crear" : "Solicitar"}
815     onAceptar={solicitarBodegaNueva}
816     onCancel={(() => setPidendoBodegaNueva(false))} />
817   )}
818
819 {pedirMotivo && (
820   <Dialogo titulo="Reabrir bodega cerrada"
821     mensaje="Esta acción exige una justificación escrita y queda registrada en el historial. Motivo:"
822     conCampo conVoz multilinea placeholder="revisión solicitada por el chef"
823     etiquetaCampo="Motivo de la reapertura"
824     textoAceptar="Reabrir" peligro
825     onAceptar={reabrir}
826     onCancel={(() => setPedirMotivo(false))} />
827   )}
828
829 {movimientos && (
830   <Dialogo titulo="Movimientos recientes"
831     mensaje={movimientos.length
832       ? movimientos.map((m) => `${m.hora} · ${m.bodega} · ${m.persona}: ${m.cantidad}
${m.unidad}`).join("\n")
833       : "Sin conteos registrados todavía para este artículo."}
834     textoAceptar="Cerrar" onAceptar={() => setMovimientos(null)}
835     onCancel={(() => setMovimientos(null))} />
836   )}
837
838 {enRecetas && (
839   <Dialogo titulo="En qué recetas aparece"
840     mensaje={enRecetas.length
841       ? enRecetas.map((r) => `${r.receta}: ${r.por_porcion} por porción (rinde
${r.rendimiento})`.join("\n")
842       : "Este artículo no aparece en ninguna receta cargada."}

```

```

843         textoAceptar="Cerrar" onAceptar={() => setEnRecetas(null)}
844         onCancel={(() => setEnRecetas(null)} />
845     ))}
846 </Marco>
847 );
848 }

```

frontend/src/vistas/Auditoria.jsx

949 LÍNEAS

Las cuatro pestañas del control.

```

1  import { useEffect, useRef, useState } from "react";
2  import { pedir, enviarTurno } from "../api";
3  import { escuchar, hablar, vozDisponible } from "../voz";
4  import Marco from "../Marco";
5  import Dialogo from "../Dialogo";
6  import AsistenteVoz from "../AsistenteVoz";
7  import Icono from "../Iconos";
8
9  const TABS = [
10   { id: "recuento", t: "Recuento ciego y cierre" },
11   { id: "aprobaciones", t: "Aprobaciones" },
12   { id: "pedidos", t: "Pedidos pendientes" },
13   { id: "alertas", t: "Bandeja de alertas" },
14 ];
15
16 // Ejemplos de lo que ya entiende el agente en esta pantalla: navegar entre
17 // las cuatro pestañas, resolver aprobaciones/pedidos/alertas por nombre,
18 // seleccionar una bodega para auditar, y preguntar por los pendientes.
19 const _EJEMPLOS_AUDITORIA = [
20   "aprueba costilla de res", "rechaza el kiosco nuevo", "aprueba el pedido de Luis",
21   "rechaza el pedido de sopa de arroz", "resuelve la alerta de papa criolla",
22   "audita el almacén general", "recuento ciego y cierre", "aprobaciones",
23   "pedidos pendientes", "bandeja de alertas", "¿cuántas alertas están abiertas?",
24   "¿cuántas se resolvieron hoy?", "¿cuál es el tiempo medio?",
25   "¿cuántos pedidos están pendientes?", "¿cuántas aprobaciones hay?",
26 ];
27
28 export default function Auditoria({ token, usuario, ir, tabInicial, bodegaAuditar, navSeq }) {
29   const [tab, setTab] = useState(tabInicial || "recuento");
30   // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
31   // componente - sin esto, el useState de arriba no vuelve a leer
32   // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
33   // cada ir()) va en la dependencia también: sin él, pedir la MISMA
34   // pestaña dos veces seguidas (con un clic manual a otra en el medio) no
35   // hacía nada, porque tabInicial no cambiaba de valor - confirmado con
36   // Playwright: "pedidos pendientes" -> clic a Aprobaciones -> "pedidos
37   // pendientes" otra vez se quedaba en Aprobaciones.
38   useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
39   const [enfoco, setEnFoco] = useState(null); // { titulo, chip } para la vista activa
40   const esAuditor = usuario?.perfil === "auditor";
41
42   return (
43     <Marco titulo={enfoco ? enfoco.titulo
44       : "Auditoría · recuento ciego, aprobaciones y cierre"}
45       chip={enfoco ? enfoco.chip
46         : { texto: esAuditor ? "ADMINISTRADORA" : "SOLO LECTURA",
47           tipo: esAuditor ? "azul" : "gris" }}>
48       {!esAuditor && (
49         <div className="banner">
50           <span className="ico">!</span>
51           <span>
52             <b>Su perfil es auxiliar</b>
53             <span>Auditar, aprobar y cerrar bodegas requiere perfil de administrador.
54               Ingrese como «diana» para probarlo.</span>
55           </span>
56         </div>
57       )}
58       {esAuditor && (
59         <AsistenteVoz token={token} vista="auditoria" ir={ir} ejemplos={_EJEMPLOS_AUDITORIA}
60           placeholder="aprueba costilla de res, rechaza el kiosco..."
61           alActualizar={() => window.dispatchEvent(new Event("cuentavoz:auditoria-actualizada"))} />

```

```

62     })
63     <div className="chips">
64         {TABS.map((tt) => (
65             <button key={tt.id} className={`chip ${tab === tt.id ? "azul" : ""}`}
66                 onClick={() => { setTab(tt.id); setEnFoco(null); }}>{tt.t}</button>
67         ))}
68     </div>
69
70     {tab === "recuento" &&
71         <TabRecuento token={token} esAuditor={esAuditor} onEnFoco={setEnFoco}
72             bodegaAuditar={bodegaAuditar} navSeq={navSeq} />
73         {tab === "aprobaciones" && <TabAprobaciones token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
74         {tab === "pedidos" && <TabPedidosPendientes token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
75         {tab === "alertas" && <TabAlertas token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
76     </Marco>
77 );
78 }
79
80 /* — Recuento ciego + cierre con doble firma — */
81 function TabRecuento({ token, esAuditor, onEnFoco, bodegaAuditar, navSeq }) {
82     const [bodegas, setBodegas] = useState(null);
83     const [bodegaId, setBodegaId] = useState(null);
84     const [sesion, setSesion] = useState(null);
85     const [dicho, setDicho] = useState("");
86     const [respuesta, setRespuesta] = useState("");
87     const [comparar, setComparar] = useState(null);
88     const [firmas, setFirmas] = useState(null);
89     const [detalle, setDetalle] = useState(null);
90     const [modoCierre, setModoCierre] = useState(false);
91     const [msg, setMsg] = useState("");
92     const [estado, setEstado] = useState("listo");
93     const [mostrarTeclado, setMostrarTeclado] = useState(false);
94     const [mostrarDictado, setMostrarDictado] = useState(false);
95     const [opciones, setOpciones] = useState(null);
96     const [opcionesPara, setOpcionesPara] = useState(null);
97
98     function cargarBodegas() {
99         // ?propias=1: si este administrador tiene bodegas asignadas (supervisor
100         // de una zona), audita solo esas; sin asignaciones, ve el parque
101         // completo (administrador general) - ver /api/bodegas en el backend.
102         // si falla, resuelve a [] en vez de dejar bodegas en null para
103         // siempre: sin esto un fallo de red dejaba "Cargando..." sin fin.
104         pedir("/api/bodegas?propias=1", {}, token).then(setBodegas)
105             .catch((e) => { setBodegas([]); setMsg(e.message); });
106     }
107     useEffect(cargarBodegas, [token]);
108
109     const candidatas = (bodegas || []).filter((b) => b.estado === "en_auditoria" || b.estado === "cerrada");
110     // "cerrada" ya significa cerrada de verdad (doble firma), no "lista
111     // para cerrar" - se queda en la lista solo para poder revisarla
112     // (ver el resumen de su cierre), no como algo pendiente de hacer.
113     // Sin esta distinción, el saludo decía "lista para cerrar" o la
114     // contaba entre las "listas para auditar" aunque ya estuviera cerrada.
115     const pendientes = candidatas.filter((b) => b.estado === "en_auditoria");
116     const cerradas = candidatas.filter((b) => b.estado === "cerrada");
117     const [saludoLista, setSaludoLista] = useState("");
118     const yaSaludoLista = useRef(false);
119
120     // Al llegar a esta pestaña sin pedir una bodega puntual, el agente dice
121     // de una vez si hay algo listo para recuento o cierre y qué es. Si ya
122     // llegó con bodegaAuditar (pidió una bodega específica por voz), esto
123     // no se repite: la respuesta de esa orden ya lo dijo.
124     useEffect(() => {
125         if (bodegas === null || yaSaludoLista.current || bodegaAuditar) return;
126         yaSaludoLista.current = true;
127         let texto;
128         if (pendientes.length === 0) {
129             texto = cerradas.length > 0
130                 ? `No hay ninguna bodega pendiente de recuento ciego. ${cerradas.length} `
131                 + `ya ${cerradas.length === 1 ? "está cerrada" : "están cerradas"}.`
132                 : "No hay ninguna bodega lista para recuento ciego o cierre en este momento.";
133         } else if (pendientes.length === 1) {
134             texto = `Tiene una bodega lista para recuento ciego: ${pendientes[0].bodega.toLowerCase()}. `
135                 + "¿Quiere auditarla?";

```

```

136     } else {
137         const nombres = pendientes.slice(0, 5).map((b) => b.bodega.toLowerCase()).join(", ");
138         texto = `Tiene ${pendientes.length} bodegas listas para recuento ciego: ${nombres}. `
139             + "¿Cuál quiere auditar?";
140     }
141     setSaludoLista(texto);
142     hablar(texto);
143     // eslint-disable-next-line react-hooks/exhaustive-deps
144 }, [bodegas, bodegaAuditar]);
145
146 // "Audita <bodega>" por voz llega hasta aquí como bodegaAuditar - lo
147 // mismo que darle clic a "Abrir" en su tarjeta, sin tocar firmar/cerrar.
148 // Depende solo de bodegaAuditar (no de candidatas/bodegaId) para no
149 // reabrir la sola si la persona ya le dio "Volver a la lista" a mano.
150 useEffect(() => {
151     if (!bodegaAuditar) return;
152     const encontrada = candidatas.find((b) => b.bodega === bodegaAuditar);
153     if (encontrada) {
154         setBodegaId(encontrada.id); verFirmas(encontrada.id);
155         if (encontrada.estado === "cerrada") verDetalle(encontrada.id);
156     }
157     // eslint-disable-next-line react-hooks/exhaustive-deps
158 }, [bodegaAuditar, navSeq]);
159
160 useEffect(() => {
161     if (!bodegaId) { onEnFoco(null); return; }
162     const bActual = (bodegas || []).find((b) => b.id === bodegaId);
163     const nombre = comparar?.bodega || firmas?.bodega || detalle?.bodega || bActual?.bodega;
164     if (!nombre) return;
165     if (modoCierre) {
166         onEnFoco({ titulo: `Cierre de bodega · ${nombre}`,
167             chip: { texto: "LISTA PARA CERRAR", tipo: "borde verde" } });
168     } else if (!sesion && bActual?.estado === "cerrada") {
169         onEnFoco({ titulo: `Auditoría · ${nombre}`,
170             chip: { texto: "CERRADA", tipo: "verde" } });
171     } else {
172         onEnFoco({ titulo: `Auditoría · ${nombre}`,
173             chip: { texto: "RECONTEO CIEGO", tipo: "borde oro" } });
174     }
175 }, [bodegaId, comparar, firmas, detalle, bodegas, modoCierre, sesion]);
176
177 async function iniciar(id) {
178     setMsg(""); setComparar(null);
179     try {
180         const r = await pedir(`/api/bodegas/${id}/auditoria/iniciar`, { method: "POST" }, token);
181         setBodegaId(id);
182         setSesion(r);
183         setMostrarDictado(true);
184         const saludo = `Recuento ciego iniciado en ${r.bodega.toLowerCase()}. Dícteme el primer producto.`;
185         setRespuesta(saludo);
186         hablar(saludo);
187         verFirmas(id);
188     } catch (e) { setMsg(e.message); }
189 }
190
191 async function verFirmas(id) {
192     try { setFirmas(await pedir(`/api/bodegas/${id}/firmas`, {}, token)); }
193     catch (_) {}
194 }
195
196 // Solo para una bodega ya CERRADA: sin esto, abrirla aquí mostraba
197 // igual "Iniciar recuento ciego" (sesion sigue null hasta llamar
198 // iniciar()) como si le faltara auditoria, cuando ya quedó cerrada con
199 // doble firma - una pantalla casi vacía y encima engañosa.
200 async function verDetalle(id) {
201     try { setDetalle(await pedir(`/api/bodegas/${id}/detalle`, {}, token)); }
202     catch (_) {}
203 }
204
205 async function procesar(texto, mostrar = texto) {
206     if (!texto || !sesion) return;
207     setDicho(mostrar);
208     try {
209         const t = await enviarTurno(texto, sesion.sesion_id, token, {

```

```

210     opciones, opcionesPara,
211     bodegaId, bodegaNombre: sesion?.bodega,
212   });
213   setRespuesta(t.respuesta_hablada || "");
214   setOpciones(t.opciones || null);
215   setOpcionesPara(t.opciones_para || null);
216   hablar(t.respuesta_hablada);
217   } catch (e) { setMsg(e.message); hablar(e.message); }
218 }
219
220 async function verComparar() {
221   setMsg("");
222   try { setComparar(await pedir(`/api/bodegas/${bodegaId}/auditoria/comparar`, {}, token)); }
223   catch (e) { setMsg(e.message); }
224 }
225
226 async function firmarAuditoria() {
227   try {
228     await pedir(`/api/bodegas/${bodegaId}/auditoria/firmar`, { method: "POST" }, token);
229     setMsg("Auditoría firmada.");
230     verFirmas(bodegaId);
231   } catch (e) { setMsg(e.message); }
232 }
233
234 async function cerrarDefinitivo() {
235   try {
236     const r = await pedir(`/api/bodegas/${bodegaId}/cerrar`, { method: "POST" }, token);
237     setMsg(`${r.bodega} cerrada definitivamente con doble firma.`);
238     setBodegaId(null); setSesion(null); setComparar(null); setFirmas(null);
239     setDetalle(null); setModoCierre(false);
240     cargarBodegas();
241   } catch (e) { setMsg(e.message); }
242 }
243
244 function volverAlaLista() {
245   setBodegaId(null); setSesion(null); setComparar(null); setFirmas(null);
246   setDetalle(null); setModoCierre(false); setMostrarDictado(false);
247 }
248
249 if (!esAuditor) return null;
250
251 return (
252   <>
253   {msg && <p className="msg-ok">{msg}</p>}
254
255   {!bodegaId ? (
256     <div className="card">
257       <h2>Bodegas listas para recuento ciego o cierre</h2>
258       {saludoLista && (
259         <>
260         <p className="rotulo">CuentaVoz responde</p>
261         <p className="burbuja" style={{ marginBottom: 12 }}>
262           aria-live="polite" aria-atomic="true">{saludoLista}</p>
263         </>
264       )}
265       {bodegas === null ? (
266         <p className="cargando">Cargando...</p>
267       ) : candidatas.length === 0 ? (
268         <p className="vacio">Ninguna bodega firmada por el auxiliar en este momento.</p>
269       ) : (
270         candidatas.map((b) => (
271           <div className="registro" key={b.id}>
272             <span className={`chip ${b.estado === "cerrada" ? "verde" : "oro"}`>
273               {b.estado === "cerrada" ? "CERRADA" : "EN AUDITORÍA"}
274             </span>
275             <span>{b.bodega}</span>
276             <span className="cant">{b.referencias} refs</span>
277             <button className="btn" style={{ padding: "6px 12px", minHeight: 0 }}>
278               onClick={() => {
279                 setBodegaId(b.id); verFirmas(b.id);
280                 if (b.estado === "cerrada") verDetalle(b.id);
281               }}>
282               Abrir
283             </button>

```

```

284         </div>
285     ))
286 }}
287 <p className="pista" style={{ marginTop: 10 }}>
288     El recuento del administrador es a ciegas: no ve el conteo del auxiliar
289     hasta comparar.
290 </p>
291 </div>
292 ) : !sesion && (bodegas || []).find((b) => b.id === bodegaId)?.estado === "cerrada" ? (
293 <div className="card">
294     <div style={{ display: "flex", alignItems: "center", gap: 10, marginBottom: 4 }}>
295         <h2 style={{ margin: 0 }}>{detalle?.bodega
296             || (bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h2>
297         <span className="chip verde" style={{ marginLeft: "auto" }}>CERRADA</span>
298     </div>
299     {!detalle ? (
300         <p className="cargando">Cargando...</p>
301     ) : (
302         <>
303             <div className="kpi">
304                 <div className="kpi verde">
305                     <div className="kpi-cabeza">
306                         <span className="icono-kpi">✅</span>
307                         <small>Exactitud del cierre</small>
308                     </div>
309                     <b>{detalle.exactitud}</b>
310                     <i>{detalle.diferencias.length} diferencias de {detalle.referencias}</i>
311                 </div>
312                 <div className="kpi">
313                     <div className="kpi-cabeza">
314                         <span className="icono-kpi">🟡</span>
315                         <small>Referencias</small>
316                     </div>
317                     <b>{detalle.referencias}</b>
318                     <i>{detalle.contadas} contadas</i>
319                 </div>
320                 <div className="kpi">
321                     <div className="kpi-cabeza">
322                         <span className="icono-kpi">🔴</span>
323                         <small>Hora de cierre</small>
324                     </div>
325                     <b>{detalle.hora_cierre || "-"}</b>
326                 </div>
327                 <div className="kpi">
328                     <div className="kpi-cabeza">
329                         <span className="icono-kpi">🟡</span>
330                         <small>Auditó</small>
331                     </div>
332                     <b>{detalle.revisor || "-"}</b>
333                 </div>
334             </div>
335             <p className="pista" style={{ marginTop: 12 }}>
336                 Esta bodega ya está cerrada con doble firma. Para volver a contarla hace
337                 falta reabrirla desde Bodegas con una justificación escrita.
338             </p>
339         </>
340     )}
341 <div className="grilla-botones" style={{ marginTop: 12 }}>
342     <button className="btn gris" onClick={volverAlaLista}>Volver a la lista</button>
343 </div>
344 </div>
345 ) : !sesion ? (
346 <div className="card">
347     <h2>{(bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h2>
348     <button className="btn" onClick={() => iniciar(bodegaId)}
349         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
350         <Icono nombre="auditoria" tam={18} />
351         Iniciar recuento ciego
352     </button>
353     <div className="grilla-botones" style={{ marginTop: 12 }}>
354         <button className="btn gris" onClick={volverAlaLista}>Volver a la lista</button>
355     </div>
356 </div>
357 ) : !comparar ? (

```

```

358 <div className="card">
359   <h2>{(bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h2>
360   <div className="conteo-cols">
361     <div>
362       <p className="rotulo">{dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}</p>
363       <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
364       {opciones && (
365         <div className="opciones" style={{ marginTop: 14 }}>
366           {opciones.map((o, i) => (
367             <button key={o.codigo} className="opcion"
368               onClick={() => procesar(o.codigo, o.nombre)}>
369               <span className="n">Opción {i + 1}</span>
370               <h4>{o.nombre}</h4>
371               <p>Código {o.codigo}</p>
372               <p>{o.unidad}</p>
373             </button>
374           ))}
375         </div>
376       )}
377       <div className="grilla-botones" style={{ marginTop: 12 }}>
378         <button className="btn verde" onClick={() => procesar("confirmo")}>Confirmar</button>
379         <button className="btn gris" onClick={() => setMostrarTeclado(true)}>
380           Teclado
381         </button>
382       </div>
383     </div>
384     <div className="mic-caja">
385       <button className={mic-btn ${estado === "escuchando" ? "escuchando" : ""}}
386         onClick={() => escuchar({ alTexto: procesar, alEstado: setEstado })}
387         aria-label="Hable para dictar el artículo y la cantidad del recuento ciego">
388         <Icono nombre="microfono" tam={52} />
389       </button>
390       <small>{vozDisponible() ? "Reconocimiento en español" : "Use el teclado"}</small>
391     </div>
392   </div>
393   <div className="grilla-botones" style={{ marginTop: 12 }}>
394     <button className="btn borde" onClick={verComparar}>Ver comparación</button>
395     <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
396   </div>
397 </div>
398 ) : !modoCierre ? (
399   <div className="card">
400     <div className="chips">
401       <span className="chip azul">{comparar.total} referencias</span>
402       <span className="chip oro">{comparar.filas.length} con diferencia</span>
403       <span className="chip verde">{comparar.coinciden} coinciden</span>
404     </div>
405     <p className="pista">
406       Solo se muestran las referencias con diferencia. Su recuento fue a ciegas:
407       el conteo 1 se revela al comparar.
408     </p>
409     {comparar.filas.length === 0 ? (
410       <p className="vacio">Sin diferencias entre el recuento ciego y el sistema.</p>
411     ) : (
412       <table>
413         <thead>
414           <tr><th>Artículo</th><th>Conteo 1</th><th>Su conteo</th>
415             <th>Sistema</th><th>Diferencia</th><th>Acción</th></tr>
416         </thead>
417         <tbody>
418           {comparar.filas.map((f) => (
419             <tr key={f.codigo}>
420               <td>{f.articulo}</td>
421               <td>{f.conteo1 ?? "-"}</td>
422               <td>{f.conteo2 ?? "-"}</td>
423               <td>{f.sistema}</td>
424               <td className="dif">{f.diferencia > 0 ? `+${f.diferencia}` : f.diferencia}</td>
425               <td>{f.accion}</td>
426             </tr>
427           ))}
428         </tbody>
429       </table>
430     )}
431   </div>

```



```

432     {comparar.diagnostics?.length > 0 && (
433       <div style={{ background: "#FAFBFD", border: "1px solid var(--borde)",
434         borderRadius: "var(--radio)", padding: 14, marginTop: 14 }}>
435         <h2 style={{ marginTop: 0 }}>Diagnóstico automático</h2>
436         {comparar.diagnostics.map((d, i) => (
437           <p key={i} style={{ fontSize: ".88rem", marginTop: i ? 8 : 0 }}>{d}</p>
438         ))}
439       </div>
440     )}
441
442     {mostrarDictado && (
443       <div className="conteo-cols" style={{ marginTop: 16 }}>
444         <div>
445           <p className="rotulo">{dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}</p>
446           <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
447           {opciones && (
448             <div className="opciones" style={{ marginTop: 14 }}>
449               {opciones.map((o, i) => (
450                 <button key={o.codigo} className="opcion"
451                   onClick={() => procesar(o.codigo, o.nombre)}>
452                   <span className="n">Opción {i + 1}</span>
453                   <h4>{o.nombre}</h4>
454                   <p>Código {o.codigo}</p>
455                   <p>{o.unidad}</p>
456                 </button>
457               ))}
458             </div>
459           )}
460           <div className="grilla-botones" style={{ marginTop: 12 }}>
461             <button className="btn verde" onClick={() => procesar("confirmando")}>Confirmar</button>
462             <button className="btn borde" onClick={verComparar}>Actualizar comparación</button>
463             <button className="btn gris" onClick={() => setMostrarTeclado(true)}>Teclado</button>
464           </div>
465         </div>
466         <div className="mic-caja">
467           <button className={micBtn} {estado === "escuchando" ? "escuchando" : ""}>
468             onClick={() => escuchar({ alTexto: procesar, alEstado: setEstado })}
469             aria-label="Hable para dictar el artículo y la cantidad del recuento ciego">
470               <Icono nombre="microfono" tam={52} />
471             </button>
472             <small>{vozDisponible() ? "Reconocimiento en español" : "Use el teclado"}</small>
473           </div>
474         </div>
475       )}
476
477     <div className="grilla-botones" style={{ marginTop: 14 }}>
478       <button className="btn" onClick={() => setMostrarDictado((v) => !v)}>
479         {mostrarDictado ? "Ocultar dictado" : "Resolver por voz"}
480       </button>
481       <button className="btn borde" disabled={firmas?.auditoria?.firmada}>
482         onClick={firmarAuditoria}>
483         Aceptar todas
484       </button>
485       <button className="btn verde" onClick={() => setModoCierre(true)}>
486         Cerrar con doble firma
487       </button>
488     </div>
489     <div className="grilla-botones" style={{ marginTop: 8 }}>
490       <button className="btn gris" onClick={volverAlaLista}>Volver a la lista</button>
491     </div>
492   </div>
493 ) : (
494   <div className="card">
495     <div className="chips">
496       <span className="chip">{firmas?.referencias ?? comparar.total}> referencias</span>
497       <span className="chip verde">{comparar.filas.length}> diferencias resueltas</span>
498       <span className="chip verde">{firmas?.alertas_resueltas ?? 0}> alertas cerradas</span>
499     </div>
500
501     <div className="conteo-cols" style={{ marginTop: 14 }}>
502       <div className="opcion" style={{
503         borderColor: firmas?.conteo?.firmada ? "var(--verde)" : "var(--borde)",
504         background: firmas?.conteo?.firmada ? "var(--verde-bg)" : "#fff",
505       }}>

```

```

506     <span className="n">CONTEO 1 · AUXILIAR</span>
507     <div style={{ display: "flex", alignItems: "center", gap: 10, marginTop: 6 }}>
508         <span style={{ width: 34, height: 34, borderRadius: "50%", background: "var(--verde)",
509             color: "#fff", display: "flex", alignItems: "center", justifyContent: "center",
510             fontWeight: 700 }}>
511             {{(firmas?.conteo?.persona || "?")?.toUpperCase()}}
512         </span>
513         <h4 style={{ textTransform: "capitalize", margin: 0 }}>{{firmas?.conteo?.persona || "-"}}</h4>
514     </div>
515     {{firmas?.conteo?.firmada ? (
516         <p style={{ color: "var(--verde)", fontWeight: 700, marginTop: 10 }}>
517             ✓ Firmado · {{firmas?.conteo?.hora}}
518         </p>
519     ) : <p style={{ marginTop: 10 }}>Pendiente de su firma</p>}}
520     <p style={{ fontSize: ".82rem", color: "var(--grafito)" }}>
521         {{firmas?.referencias ?? comparar.total}} referencias
522     </p>
523     <p style={{ fontSize: ".82rem", fontStyle: "italic", color: "var(--grafito)" }}>
524         Sin acceso al recuento de auditoría
525     </p>
526 </div>
527 <div className="opcion" style={{
528     borderColor: firmas?.auditoria?.firmada ? "var(--verde)" : "var(--azul)",
529 }}>
530     <span className="n">AUDITORÍA · ADMINISTRADORA</span>
531     <div style={{ display: "flex", alignItems: "center", gap: 10, marginTop: 6 }}>
532         <span style={{ width: 34, height: 34, borderRadius: "50%", background: "var(--azul)",
533             color: "#fff", display: "flex", alignItems: "center", justifyContent: "center",
534             fontWeight: 700 }}>
535             {{(firmas?.auditoria?.persona || "?")?.toUpperCase()}}
536         </span>
537         <h4 style={{ textTransform: "capitalize", margin: 0 }}>{{firmas?.auditoria?.persona || "-"}}</h4>
538     </div>
539     {{firmas?.auditoria?.firmada ? (
540         <p style={{ color: "var(--verde)", fontWeight: 700, marginTop: 10 }}>
541             ✓ Firmado · {{firmas?.auditoria?.hora}}
542         </p>
543     ) : (
544         <>
545             <p style={{ color: "var(--azul)", marginTop: 10 }}>Pendiente de su firma</p>
546             <button className="btn" style={{ marginTop: 8 }} onClick={{firmarAuditoria}}>
547                 Firmar la auditoría
548             </button>
549         </>
550     )}}
551 </div>
552 </div>
553
554 <div className="grilla-botones" style={{ marginTop: 16 }}>
555     <button className="btn verde" disabled={!firmas?.lista_para_cerrar}
556         onClick={{cerrarDefinitivo}} style={{ flex: 1, fontSize: "1.05rem" }}>
557         Cerrar bodega definitivamente
558     </button>
559 </div>
560 <p className="pista" style={{ marginTop: 10, textAlign: "center" }}>
561     Con las dos firmas la bodega pasa a CERRADA en el tablero al instante y sus datos entran al consolidado.
562 </p>
563 <p className="pista" style={{ textAlign: "center", color: "var(--azul)" }}>
564     Ninguna bodega se cierra con una sola firma: es la garantía de trazabilidad del inventario.
565 </p>
566 <div className="grilla-botones" style={{ marginTop: 8 }}>
567     <button className="btn borde" onClick={() => setModoCierre(false)}>
568         Volver a la comparación
569     </button>
570     <button className="btn gris" onClick={{volverAlaLista}}>Volver a la lista</button>
571 </div>
572 </div>
573 }}
574
575 {{mostrarTeclado && (
576     <Dialogo titulo="Escribir en vez de hablar"
577         mensaje="Escriba el producto y la cantidad:"
578         conCampo placeholder="tres tablas para picar blancas"
579         onAceptar={{(t) => { setMostrarTeclado(false); if (t) procesar(t); }}

```

```

580         onCancel={ () => setMostrarTeclado(false) } />
581     )}
582 </>
583 );
584 }
585
586 /* — Aprobaciones pendientes — */
587 function TabAprobaciones({ token, esAuditor, onEnFoco }) {
588     const [lista, setLista] = useState(null);
589     const [msg, setMsg] = useState("");
590     const [saludo, setSaludo] = useState("");
591     const yaSaludo = useRef(false);
592
593     function cargar() {
594         pedir("/api/aprobaciones?estado=pendiente", {}, token).then(setLista)
595             .catch((e) => { setLista([]); setMsg(e.message); });
596     }
597     useEffect(cargar, [token]);
598     // El agente ("Pregúntele al agente", arriba de las pestañas) puede
599     // aprobar/rechazar por voz - cuando lo hace, avisa con este evento
600     // para que esta lista refleje el cambio sin recargar.
601     useEffect(() => {
602         window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
603         return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
604     }, [token]);
605
606     useEffect(() => {
607         onEnFoco({ titulo: "Aprobaciones pendientes",
608             chip: { texto: lista === null ? "..." : `${lista.length} PENDIENTES`, tipo: "borde oro" } });
609     }, [lista]);
610
611     // Al llegar a esta pestaña (por clic o por voz), el agente dice de una
612     // vez si hay algo pendiente y qué es, en vez de esperar a que se lo
613     // pregunten - yaSaludo evita que se repita en cada refresco de la
614     // lista (ej. justo después de aprobar algo por voz).
615     useEffect(() => {
616         if (lista === null || yaSaludo.current) return;
617         yaSaludo.current = true;
618         let texto;
619         if (lista.length === 0) {
620             texto = "No hay aprobaciones pendientes.";
621         } else if (lista.length === 1) {
622             texto = `Tiene una aprobación pendiente: ${lista[0].nombre.toLowerCase()}. ¿Quiere aprobarla o rechazarla?`;
623         } else {
624             const nombres = lista.slice(0, 5).map((a) => a.nombre.toLowerCase()).join(", ");
625             texto = `Tiene ${lista.length} aprobaciones pendientes: ${nombres}. ¿Cuál quiere aprobar o rechazar?`;
626         }
627         setSaludo(texto);
628         hablar(texto);
629     }, [lista]);
630
631     async function resolver(id, accion) {
632         try {
633             await pedir(`/api/aprobaciones/${id}/${accion}`, { method: "POST" }, token);
634             setMsg(accion === "aprobar" ? "Aprobado y agregado al catálogo oficial." : "Rechazado.");
635             cargar();
636         } catch (e) { setMsg(e.message); }
637     }
638
639     if (!esAuditor) return null;
640
641     return (
642         <div className="card">
643             <p className="pista" style={{ marginTop: 0 }}>
644                 Productos y bodegas creados durante la toma que esperan su firma para entrar al catálogo oficial.
645             </p>
646             {saludo && (
647                 <>
648                     <p className="rotulo">CuentaVoz responde</p>
649                     <p className="burbuja" style={{ marginBottom: 12 }}
650                         aria-live="polite" aria-atomic="true">{saludo}</p>
651                 </>
652             )}
653             {msg && <p className="msg-ok">{msg}</p>}

```

```

654     {lista === null ? (
655       <p className="cargando">Cargando...</p>
656     ) : lista.length === 0 ? (
657       <p className="vacio">Nada pendiente de aprobación.</p>
658     ) : (
659       lista.map((a) => (
660         <div key={a.id} style={{ display: "flex", alignItems: "flex-start", gap: 14,
661           flexWrap: "wrap",
662           border: "1px solid var(--borde)", borderRadius: 10,
663           padding: "12px 14px", marginBottom: 10 }}>
664           <span style={{ flex: 1, minWidth: 200 }}>
665             <b style={{ color: "var(--azul)" }}>{a.nombre}</b>
666             <br />
667             <span style={{ fontSize: ".85rem" }}>
668               {a.tipo === "producto"
669                 ? `Producto nuevo · ${a.unidad_medida} || ""`
670                 : "Bodega nueva"}
671             </span>
672             <br />
673             <small style={{ color: "var(--grafito)", fontStyle: "italic" }}>
674               {a.tipo === "producto"
675                 ? `${a.cantidad_inicial} ${a.unidad_medida} contados en ${a.bodega}`
676                 : "sin referencias asignadas todavía"}
677             </small>
678             <br />
679             <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
680               {a.creado_por} · {a.hora}
681             </small>
682           </span>
683           <span style={{ display: "flex", gap: 8 }}>
684             <button className="btn gris" onClick={() => resolver(a.id, "rechazar")}
685               style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
686               <Icono nombre="rechazar" tam={16} />
687               Rechazar
688             </button>
689             <button className="btn verde" onClick={() => resolver(a.id, "aprobar")}
690               style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
691               <Icono nombre="aprobar" tam={16} />
692               Aprobar
693             </button>
694           </span>
695         </div>
696       ))
697     )}
698     <p className="pista" style={{ marginTop: 10 }}>
699       Al aprobar, el producto entra al catálogo con su código definitivo y el
700       conteo asociado deja de estar marcado. Mientras tanto el conteo nunca se
701       detiene: la aprobación ocurre en paralelo al trabajo en bodega.
702     </p>
703   </div>
704 );
705 }
706
707 /* — Pedidos de auxiliares esperando aprobación antes de salir del almacén — */
708 function TabPedidosPendientes({ token, esAuditor, onEnFoco }) {
709   const [lista, setLista] = useState(null);
710   const [msg, setMsg] = useState("");
711   const [saludo, setSaludo] = useState("");
712   const yaSaludo = useRef(false);
713
714   function cargar() {
715     pedir("/api/pedidos/pendientes", {}, token).then(setLista)
716       .catch((e) => { setLista([]); setMsg(e.message); });
717   }
718   useEffect(cargar, [token]);
719   useEffect(() => {
720     window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
721     return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
722   }, [token]);
723
724   useEffect(() => {
725     onEnFoco({ titulo: "Pedidos pendientes de aprobación",
726       chip: { texto: lista === null ? "..." : `${lista.length} PENDIENTES`, tipo: "borde oro" } });
727   }, [lista]);

```

```

728
729 useEffect(() => {
730   if (lista === null || yaSaludo.current) return;
731   yaSaludo.current = true;
732   let texto;
733   if (lista.length === 0) {
734     texto = "No hay pedidos pendientes.";
735   } else if (lista.length === 1) {
736     texto = `Tiene un pedido pendiente: ${lista[0].plato.toLowerCase()}, de `
737       + `${lista[0].persona}. ¿Quiere aprobarlo o rechazarlo?`;
738   } else {
739     const nombres = lista.slice(0, 5)
740       .map((p) => `${p.plato.toLowerCase()} de ${p.persona}`).join(", ");
741     texto = `Tiene ${lista.length} pedidos pendientes: ${nombres}. ¿Cuál quiere aprobar o rechazar?`;
742   }
743   setSaludo(texto);
744   hablar(texto);
745 }, [lista]);
746
747 async function resolver(numero, aprobar) {
748   try {
749     await pedir(`/api/pedidos/${numero}/resolver`, {
750       method: "POST", body: JSON.stringify({ aprobar }),
751     }, token);
752     setMsg(aprobar ? "Pedido aprobado: ya puede salir del almacén."
753       : "Pedido rechazado.");
754     cargar();
755   } catch (e) { setMsg(e.message); }
756 }
757
758 if (!esAuditor) return null;
759
760 return (
761   <div className="card">
762     <p className="pista" style={{ marginTop: 0 }}>
763       El auxiliar pide y queda registrado de una vez, pero solo sale del almacén
764       cuando usted lo aprueba aquí - igual que con productos y bodegas nuevas.
765     </p>
766     {saludo && (
767       <>
768         <p className="rotulo">CuentaVoz responde</p>
769         <p className="burbuja" style={{ marginBottom: 12 }}
770           aria-live="polite" aria-atomic="true">{saludo}</p>
771       </>
772     )}
773     {msg && <p className="msg-ok">{msg}</p>}
774     {lista === null ? (
775       <p className="cargando">Cargando...</p>
776     ) : lista.length === 0 ? (
777       <p className="vacio">Nada pendiente de aprobación.</p>
778     ) : (
779       lista.map((p) => (
780         <div key={p.numero_pedido} style={{ display: "flex", alignItems: "flex-start", gap: 14,
781           flexWrap: "wrap",
782           border: "1px solid var(--borde)", borderRadius: 10,
783           padding: "12px 14px", marginBottom: 10 }}>
784           <span style={{ flex: 1, minWidth: 200 }}>
785             <b style={{ color: "var(--azul)" }}>{p.plato}</b> · {p.porciones} porciones
786             <span className="chip" style={{ marginLeft: 8 }}>{p.numero_pedido}</span>
787             <br />
788             <small style={{ color: "var(--grafito)" }}>
789               Se descontaría de: {p.bodega || "-"}
790             </small>
791             <br />
792             <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
793               Pedido por {p.persona} · {p.hora}
794             </small>
795             <ul style={{ margin: "8px 0 0", paddingLeft: 18, fontSize: ".85rem" }}>
796               {p.items.map((it, i) => (
797                 <li key={i}>{it.nombre}: {it.cantidad}</li>
798               ))}
799             </ul>
800           </span>
801           <span style={{ display: "flex", gap: 8, flex: "none" }}>

```

```

802         <button className="btn gris" onClick={() => resolver(p.numero_pedido, false)}
803             style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
804             <Icono nombre="rechazar" tam={16} />
805             Rechazar
806         </button>
807         <button className="btn verde" onClick={() => resolver(p.numero_pedido, true)}
808             style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
809             <Icono nombre="aprobar" tam={16} />
810             Aprobar
811         </button>
812     </span>
813 </div>
814 ))
815 })
816 <p className="pista" style={{ marginTop: 10 }}>
817     Al aprobar, el pedido queda abierto para su legalización al final del servicio.
818     Al rechazar, no cuenta como enviado y el auxiliar debe volver a intentarlo.
819 </p>
820 </div>
821 );
822 }
823
824 /* — Bandeja de alertas — */
825 const COLOR_ALERTA = { desviacion: "oro", negativo: "oro", unidad: "azul", inexistente: "azul" };
826
827 function TabAlertas({ token, esAuditor, onEnFoco }) {
828     const [alertas, setAlertas] = useState(null);
829     const [resumen, setResumen] = useState(null);
830     const [verDetalle, setVerDetalle] = useState(null);
831     const [msg, setMsg] = useState("");
832     const [saludo, setSaludo] = useState("");
833     const yaSaludo = useRef(false);
834
835     function cargar() {
836         pedir("/api/alertas?resueltas=0", {}, token).then(setAlertas)
837             .catch((e) => { setAlertas([]); setMsg(e.message); });
838         pedir("/api/alertas/resumen", {}, token).then(setResumen).catch(() => {});
839     }
840     useEffect(cargar, [token]);
841     useEffect(() => {
842         window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
843         return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
844     }, [token]);
845
846     useEffect(() => {
847         onEnFoco({ titulo: "Auditoría · Bandeja de alertas",
848             chip: { texto: alertas === null ? "..." : `${alertas.length} ABIERTAS`, tipo: "borde oro" } });
849     }, [alertas]);
850
851     useEffect(() => {
852         if (alertas === null || yaSaludo.current) return;
853         yaSaludo.current = true;
854         const etiqueta = (a) => a.articulo || a.bodega || a.titulo.toLowerCase();
855         let texto;
856         if (alertas.length === 0) {
857             texto = "No hay alertas abiertas.";
858         } else if (alertas.length === 1) {
859             texto = `Tiene una alerta abierta: ${etiqueta(alertas[0]).toLowerCase()}.`
860                 + (esAuditor ? " ¿Quiere resolverla?" : "");
861         } else {
862             const nombres = alertas.slice(0, 5).map((a) => etiqueta(a).toLowerCase()).join(", ");
863             texto = `Tiene ${alertas.length} alertas abiertas: ${nombres}.`
864                 + (esAuditor ? " ¿Cuál quiere resolver?" : "");
865         }
866         setSaludo(texto);
867         hablar(texto);
868     }, [alertas, esAuditor]);
869
870     async function resolver(id) {
871         try {
872             await pedir(`/api/alertas/${id}/resolver`, { method: "POST" }, token);
873             setMsg("Alerta resuelta.");
874             cargar();
875         } catch (e) { setMsg(e.message); }
876     }
877 }

```

```

876   }
877
878   return (
879     <div className="card">
880       <div className="chips">
881         <span className="chip oro">{resumen?.abiertas ?? (alertas || []).length} abiertas</span>
882         <span className="chip verde">{resumen?.resueltas_hoy ?? 0} resueltas hoy</span>
883         <span className="chip">
884           Tiempo medio: {resumen?.tiempo_medio_min != null ? `${resumen.tiempo_medio_min} min` : "-"}
885         </span>
886       </div>
887       {saludo && (
888         <>
889           <p className="rotulo">CuentaVoz responde</p>
890           <p className="burbuja" style={{ marginBottom: 12 }}
891             aria-live="polite" aria-atomic="true">{saludo}</p>
892         </>
893       )}
894       {msg && <p className="msg-ok">{msg}</p>}
895       {alertas === null ? (
896         <p className="cargando">Cargando...</p>
897       ) : alertas.length === 0 ? (
898         <p className="vacio">No hay alertas abiertas.</p>
899       ) : (
900         alertas.map((a) => (
901           <div key={a.id} style={{
902             display: "flex", alignItems: "center", gap: 14, marginTop: 10, flexWrap: "wrap",
903             border: "1px solid var(--borde)",
904             borderLeft: `4px solid ${COLOR_ALERTA[a.tipo] === "azul" ? "var(--azul)" : "var(--amarillo)"}`,
905             borderRadius: 10, padding: "12px 14px",
906           }}>
907             <span style={{ flex: 1, minWidth: 200 }}>
908               <b style={{ color: COLOR_ALERTA[a.tipo] === "azul" ? "var(--azul)" : "var(--amarillo-tx)" }}>
909                 {a.titulo}
910               </b>
911               <br />
912               <span style={{ fontSize: ".88rem" }}>
913                 {a.articulo || "-"}{a.bodega ? ` · ${a.bodega}` : ""}
914               </span>
915               <br />
916               <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
917                 {a.persona || "-"} · {a.hora}
918               </small>
919               <br />
920               <small style={{ color: "var(--grafito)", fontStyle: "italic" }}>{a.detalle}</small>
921             </span>
922             <span style={{ display: "flex", gap: 8, flex: "none" }}>
923               <button className="btn borde" onClick={() => setVerDetalle(a)}>Ver</button>
924               {esAuditor && (
925                 <button className="btn verde" onClick={() => resolver(a.id)}
926                   style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
927                   <Icono nombre="aprobar" tam={16} />
928                   Resolver
929                 </button>
930               )}
931             </span>
932           </div>
933         ))
934       )}
935       <p className="pista" style={{ marginTop: 12, fontStyle: "italic" }}>
936         Cada alerta guarda quién la generó, con qué dato y cómo se resolvió: es la trazabilidad
937         que exige una auditoría de inventario.
938       </p>
939
940       {verDetalle && (
941         <Dialogo titulo={verDetalle.titulo}
942           mensaje={` ${verDetalle.articulo || ""} ${verDetalle.bodega ? ` · ${verDetalle.bodega}` : ""} \n` +
943             ` ${verDetalle.persona || "-"} · ${verDetalle.hora} \n \n ${verDetalle.detalle} `}
944           textoAceptar="Cerrar" onAceptar={() => setVerDetalle(null)}
945           onCancel={() => setVerDetalle(null)} />
946       )}
947     </div>
948   );
949 }

```

frontend/src/vistas/Reportes.jsx

453 LÍNEAS

La salida hacia My Inventory.

```

1 import { useEffect, useState } from "react";
2 import { pedir, descargarReporte } from "../api";
3 import { hablar, esAfirmacion, esNegacion } from "../voz";
4 import Marco from "../Marco";
5 import AsistenteVoz from "../AsistenteVoz";
6 import Icono from "../Iconos";
7
8 const fmt = (n) => Number(n ?? 0).toLocaleString("es-CO", { maximumFractionDigits: 2 });
9
10 // "oro" es el modificador de clase (.chip.oro); la variable CSS real es
11 // --amarillo, no --oro - de ahí los dos mapas separados.
12 const CLASE_TITULO = {
13   "Consolidado para My Inventory": "verde",
14   "Diferencias por bodega": "oro",
15   "Detalle de bodega": "azul",
16   "Estado del tablero": "azul",
17   "Análisis de consumo": "azul",
18   "Registro de trazabilidad": "azul",
19 };
20 const VAR_TITULO = {
21   "Consolidado para My Inventory": "verde",
22   "Diferencias por bodega": "amarillo",
23   "Detalle de bodega": "azul",
24   "Estado del tablero": "azul",
25   "Análisis de consumo": "azul",
26   "Registro de trazabilidad": "azul",
27 };
28 const ICONO_TITULO = {
29   "Consolidado para My Inventory": "🏠",
30   "Diferencias por bodega": "📦",
31   "Detalle de bodega": "📦",
32   "Estado del tablero": "📦",
33   "Análisis de consumo": "📦",
34   "Registro de trazabilidad": "🕒",
35 };
36
37 // Ejemplos de lo que ya entiende el agente en esta pantalla: generar los
38 // tres tipos de archivo, ver el contenido de uno ya generado, y preguntar
39 // por lo que muestra el análisis de consumo.
40 const _EJEMPLOS_REPORTES = [
41   "genera el consolidado", "exporta las diferencias", "exporta el análisis de consumo",
42   "muéstrame las diferencias", "muéstrame el estado del tablero", "muéstrame la trazabilidad",
43   "muéstrame el detalle de bodega", "muéstrame el archivo del consolidado",
44   "consolidado de la toma", "análisis de consumo",
45   "¿cuántas filas tiene el último consolidado?", "¿cuántas bodegas tienen descuadre?",
46   "¿qué archivos se han generado?", "¿cuánto se pidió en el período?",
47   "¿cuánto se usó realmente?", "¿cuál es el ahorro potencial?",
48   "¿cuántos insumos están subutilizados?", "¿cuál insumo tiene más sobrepedido?",
49 ];
50
51 export default function Reportes({ token, usuario, ir, tabInicial, navSeq, archivoPrevisualizar }) {
52   const [tab, setTab] = useState(tabInicial || "consolidado");
53   // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
54   // componente - sin esto, el useState de arriba no vuelve a leer
55   // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
56   // cada ir()) va en la dependencia también: sin él, pedir la MISMA
57   // pestaña dos veces seguidas (con un clic manual a otra en el medio)
58   // no hacía nada, porque tabInicial no cambiaba de valor.
59   useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
60
61   return (
62     <Marco titulo={tab === "consolidado" ? "Reportes · consolidado de la toma"
63       : "Reportes · análisis de consumo"}
64       chip={tab === "consolidado" ? { texto: "ARCHIVOS", tipo: "borde azul" }
65       : { texto: "ÚLTIMOS 30 DÍAS", tipo: "borde azul" }}>
66       <AsistenteVoz token={token} vista="reportes" ir={ir} ejemplos={_EJEMPLOS_REPORTES}
67         placeholder="¿cuántas filas tiene el último consolidado?, lléveme al panel..."
68         alActualizar={() => window.dispatchEvent(new Event("cuentavoz:reportes-actualizados"))} />
69       <div className="chips">

```



```

70     <button className={`chip ${tab === "consolidado" ? "azul" : ""}`}
71       onClick={() => setTab("consolidado")}>Consolidado de la toma</button>
72     <button className={`chip ${tab === "analisis" ? "azul" : ""}`}
73       onClick={() => setTab("analisis")}>Análisis de consumo</button>
74   </div>
75
76   {tab === "consolidado"
77     ? <TabConsolidado token={token} usuario={usuario}
78       navSeq={navSeq} archivoPrevisualizar={archivoPrevisualizar} />
79     : <TabAnalisis token={token} usuario={usuario} />}
80   </Marco>
81 );
82 }
83
84 /** Tabla genérica: cada tipo de reporte (consolidado, diferencias, estado,
85     detalle de bodega) trae sus propias columnas, así que en vez de mapear
86     columnas fijas se dibujan las que de verdad trae el archivo. */
87 function TablaVistaPrevia({ filas }) {
88   if (!filas || filas.length === 0) return <p className="vacío">Este archivo no tiene filas.</p>;
89   const columnas = Object.keys(filas[0]);
90   return (
91     <div className="tabla-scroll">
92       <table>
93         <thead><tr>{columnas.map((c) => <th key={c}>{c}</th>)}</tr></thead>
94         <tbody>
95           {filas.map((f, i) => (
96             <tr key={i}>
97               {columnas.map((c) => (
98                 <td key={c} className={c === "Diferencia" ? "dif" : ""}>
99                   {c === "Diferencia" && f[c] > 0 ? `+${f[c]}` : String(f[c])}
100                 </td>
101               ))}
102             </tr>
103           ))}
104         </tbody>
105       </table>
106     </div>
107   );
108 }
109
110 /* — Consolidado de la toma — */
111 function TabConsolidado({ token, navSeq, archivoPrevisualizar }) {
112   const [recientes, setRecientes] = useState(null);
113   const [archivoActivo, setArchivoActivo] = useState(null); // {archivo, titulo}
114   const [filasPrevia, setFilasPrevia] = useState(null);
115   const [totalFilas, setTotalFilas] = useState(null);
116   const [err, setErr] = useState("");
117   const [msg, setMsg] = useState("");
118   // Se activa con cualquier vista previa (clic manual o por voz) y con
119   // cada archivo recién generado - así un "sí"/"no" hablado después
120   // sirve para descargarlo sin importar cómo se llegó hasta ahí. Ver
121   // previsualizar().
122   const [pidiendoDescarga, setPidiendoDescarga] = useState(false);
123
124   function cargar() {
125     pedir("/api/reportes/recientes", {}, token).then(setRecientes).catch(() => setRecientes([]));
126   }
127   useEffect(cargar, [token]);
128   // Generar un archivo por voz pasa por el backend directo, no por
129   // generarConsolidado/generarDiferencias de aquí abajo - sin escuchar
130   // este evento la lista de "Archivos generados" se quedaba vieja hasta
131   // que se recargaba la página a mano.
132   useEffect(() => {
133     window.addEventListener("cuentavoz:reportes-actualizados", cargar);
134     return () => window.removeEventListener("cuentavoz:reportes-actualizados", cargar);
135   }, [token]);
136
137   // anunciar: true en un clic manual a una tarjeta - sin voz de por medio,
138   // alguien con problemas de vista no tiene otra forma de saber qué
139   // encontró la vista previa (antes solo se narraba cuando el pedido
140   // llegaba por voz, porque ese camino ya trae su propia respuesta
141   // hablada del backend - un clic de mouse no). false cuando el pedido
142   // YA llegó por voz (ver el useEffect de abajo), para no narrar dos
143   // veces lo mismo que el agente acaba de decir.

```

```

144 async function previsualizar(archivo, titulo, { anunciar = false } = {}) {
145   setErr("");
146   try {
147     const d = await pedir(`/api/reportes/vista-previa?archivo=${encodeURIComponent(archivo)}`, {}, token);
148     setArchivoActivo({ archivo, titulo });
149     setFilasPrevias(d.filas);
150     setTotalFilas(d.total);
151     setPidiendoDescarga(true);
152     if (anunciar) {
153       const resumen = `Vista previa de ${titulo}, ${d.total} `
154         + `${d.total === 1 ? "fila" : "filas"}. ¿Desea descargarlo?`;
155       setMsg(resumen);
156       hablar(resumen);
157     }
158   } catch (e) { setErr(e.message); }
159 }
160 // "Muéstrame el archivo de diferencias" - lo mismo que darle clic a la
161 // tarjeta, pero pedido por voz; el backend ya resolvió CUÁL archivo es.
162 // Va por prop (como bodegaSugerida en Conteo), no por evento: si el
163 // pedido llegó desde OTRA pestaña de Reportes, este componente todavía
164 // no estaba montado cuando AsistenteVoz disparaba el evento viejo, así
165 // que la vista previa nunca aparecía. navSeq en las dependencias por la
166 // misma razón que en el resto de la app: pedir el MISMO archivo dos
167 // veces seguidas (con un clic manual a otro en el medio) no cambia el
168 // valor de archivoPrevisualizar por sí solo.
169 useEffect(() => {
170   if (archivoPrevisualizar?.archivo) {
171     previsualizar(archivoPrevisualizar.archivo, archivoPrevisualizar.titulo);
172   }
173   // eslint-disable-next-line react-hooks/exhaustive-deps
174 }, [archivoPrevisualizar, navSeq]);
175
176 // "Sí"/"no" después de que el agente preguntó "¿desea descargarlo?" -
177 // intercepta ANTES de que AsistenteVoz le pregunte al agente general
178 // (que no tiene ni idea de esa pregunta pendiente). Mismo patrón que el
179 // filtro local de Registro de trazabilidad en Ajustes.
180 useEffect(() => {
181   function interceptar(e) {
182     if (!pidiendoDescarga) return;
183     const texto = e.detail.texto;
184     if (esAfirmacion(texto)) {
185       e.preventDefault();
186       setPidiendoDescarga(false);
187       descargarReporte(archivoActivo.archivo, token).catch((err2) => setErr(err2.message));
188       const msg2 = "Descargando el archivo.";
189       setMsg(msg2);
190       hablar(msg2);
191     } else if (esNegacion(texto)) {
192       e.preventDefault();
193       setPidiendoDescarga(false);
194       const msg2 = "Listo, queda pendiente. Dígame si desea ver otro archivo.";
195       setMsg(msg2);
196       hablar(msg2);
197     }
198   }
199   window.addEventListener("cuentavoz:filtro-local", interceptar);
200   return () => window.removeEventListener("cuentavoz:filtro-local", interceptar);
201 }, [pidiendoDescarga, archivoActivo, token]);
202
203 async function generarConsolidado(formato) {
204   setErr(""); setMsg("");
205   try {
206     const d = await pedir(`/api/reportes?formato=${formato}`, { method: "POST" }, token);
207     setArchivoActivo({ archivo: d.archivo, titulo: "Consolidado para My Inventory" });
208     setFilasPrevias(d.vista_previa || null);
209     setTotalFilas(d.filas);
210     setPidiendoDescarga(true);
211     const resumen = `Consolidado generado: ${d.filas} filas. ¿Desea descargarlo?`;
212     setMsg(resumen);
213     hablar(resumen);
214     cargar();
215   } catch (e) { setErr(e.message); }
216 }
217 async function generarDiferencias(formato) {

```

```

218     setErr(""); setMsg("");
219     try {
220         const d = await pedir(`/api/reportes/diferencias?formato=${formato}`, { method: "POST" }, token);
221         const resumen = `Diferencias exportadas: ${d.filas} filas en ${d.bodegas_con_descuadre} `
222             + "bodegas. ¿Desea descargarlo?";
223         setMsg(resumen);
224         hablar(resumen);
225         cargar();
226         await previsualizar(d.archivo, "Diferencias por bodega");
227     } catch (e) { setErr(e.message); }
228 }
229
230 return (
231     <div className="reportes-cols">
232         <div className="card">
233             <h2>Archivos generados con los códigos oficiales de la base</h2>
234             /* Arriba, junto al título, y no al final de la lista - con muchos
235                archivos generados en el día, quedaban fuera de la vista sin
236                bajar todo el scroll, y nada indicaba que seguían ahí. */
237             <div className="grilla-botones" style={{ marginTop: 0, marginBottom: 14 }}>
238                 <button className="btn" onClick={() => generarConsolidado("xlsx")}
239                     style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
240                     <Icono nombre="reportes" tam={16} />
241                     Generar consolidado
242                 </button>
243                 <button className="btn" onClick={() => generarDiferencias("xlsx")}
244                     style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
245                     <Icono nombre="reportes" tam={16} />
246                     Generar diferencias
247                 </button>
248                 <button className="btn gris" onClick={() => window.print()}>Imprimir</button>
249             </div>
250             {msg && <p className="msg-ok">{msg}</p>}
251             {err && <p className="error">{err}</p>}
252             {recientes === null ? (
253                 <p className="cargando">Cargando...</p>
254             ) : recientes.length === 0 ? (
255                 <p className="vacio">Todavía no se ha generado ningún archivo.</p>
256             ) : (
257                 recientes.map((a, i) => {
258                     const clase = CLASE_TITULO[a.titulo] || "azul";
259                     const colorVar = clase === "oro" ? "var(--amarillo-tx)" : `var(--${VAR_TITULO[a.titulo] || "azul"})`;
260                     const activo = archivoActivo?.archivo === a.archivo;
261                     return (
262                         <button key={i} onClick={() => previsualizar(a.archivo, a.titulo, { anunciar: true })}
263                             style={{
264                                 display: "flex", justifyContent: "space-between", alignItems: "center",
265                                 width: "100%", textAlign: "left", font: "inherit",
266                                 borderTop: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
267                                 borderRight: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
268                                 borderBottom: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
269                                 borderLeft: `4px solid var(--${VAR_TITULO[a.titulo] || "azul"})`,
270                                 borderRadius: 10, padding: "12px 14px", marginBottom: 10,
271                                 cursor: "pointer", background: activo ? "var(--azul-claro)" : "transparent",
272                             }}>
273                             <span style={{ display: "flex", alignItems: "center", gap: 12 }}>
274                                 <span className="icono-kpi" style={{ fontSize: "1.1rem" }}>
275                                     {ICONO_TITULO[a.titulo] || "📄"}
276                                 </span>
277                                 <span>
278                                     <b style={{ color: colorVar }}>{a.titulo}</b>
279                                     <br />
280                                     <small style={{ color: "var(--grafito)" }}>
281                                         {a.formato} · hoy {a.hora}{a.filas !== null ? ` · ${a.filas} filas` : ""}
282                                     </small>
283                                 </span>
284                             </span>
285                             <span className={`chip ${clase}`}>{a.subtitulo}</span>
286                         </button>
287                     );
288                 })
289             )
290             <p className="pista" style={{ marginTop: 10 }}>
291                 Dele clic a cualquier tarjeta para ver su contenido aquí al lado.

```

```

292     </p>
293 </div>
294
295 <div className="card">
296   <h2>{archivoActivo ? `Vista previa · ${archivoActivo.titulo}` : "Vista previa del consolidado"}</h2>
297   {filasPrevia === null ? (
298     <p className="vacio">Genere un archivo o dele clic a uno ya generado para ver una muestra aquí.</p>
299   ) : (
300     <>
301       <TablaVistaPrevia filas={filasPrevia} />
302       <p className="pista" style={{ marginTop: 10 }}>
303         {totalFilas !== null && `Mostrando ${Math.min(8, totalFilas)} de ${totalFilas} filas.`}
304         Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de artículo.
305       </p>
306       <div className="grilla-botones">
307         <button className="btn verde"
308           onClick={() => descargarReporte(archivoActivo.archivo, token).catch((e) => setErr(e.message))}
309           style={{ display: "inline-flex", alignItems: "center", gap: 8, width: "100%",
310             justifyContent: "center" }}>
311           <Icono nombre="descargar" tam={18} />
312           Descargar este archivo
313         </button>
314       </div>
315     </>
316   )}
317 </div>
318 </div>
319 );
320 }
321
322 /* — Análisis de consumo — */
323 function TabAnálisis({ token }) {
324   const [analisis, setAnalisis] = useState(null);
325   const [err, setErr] = useState("");
326   const [msg, setMsg] = useState("");
327
328   function cargar() {
329     pedir("/api/analisis/consumo?dias=30", {}, token).then(setAnalisis).catch((e) => setErr(e.message));
330   }
331   useEffect(cargar, [token]);
332   useEffect(() => {
333     window.addEventListener("cuentavoz:reportes-actualizados", cargar);
334     return () => window.removeEventListener("cuentavoz:reportes-actualizados", cargar);
335   }, [token]);
336
337   async function exportar() {
338     setErr(""); setMsg("");
339     try {
340       const d = await pedir("/api/analisis/exportar?formato=xlsx&dias=30", { method: "POST" }, token);
341       await descargarReporte(d.archivo, token);
342       setMsg("Análisis exportado y descargado.");
343     } catch (e) { setErr(e.message); }
344   }
345   // "Exporta el análisis de consumo" por voz ya creó el archivo en el
346   // servidor (el backend respondió con accion:"descargar_reporte" y el
347   // nombre del archivo) - falta el mismo paso que hace el botón: bajarlo
348   // de verdad al dispositivo.
349   useEffect(() => {
350     async function alDescargarPorVoz(e) {
351       if (!e.detail?.archivo) return;
352       setErr(""); setMsg("");
353       try {
354         await descargarReporte(e.detail.archivo, token);
355         setMsg("Análisis exportado y descargado.");
356       } catch (err) { setErr(err.message); }
357     }
358     window.addEventListener("cuentavoz:accion:descargar_reporte", alDescargarPorVoz);
359     return () => window.removeEventListener("cuentavoz:accion:descargar_reporte", alDescargarPorVoz);
360   }, [token]);
361
362   if (err) return <p className="error">{err}</p>;
363   if (!analisis) return <p className="cargando">Cargando...</p>;
364
365   const recomendacion = analisis.subutilizados[0];

```

```

366 const maxSobrepedido = Math.max(1, ... analisis.subutilizados.map((s) => s.sobrepedido_pct));
367
368 return (
369   <>
370     {msg && <p className="msg-ok">{msg}</p>}
371     <div className="kpis">
372       <div className="kpi">
373         <div className="kpi-cabeza"><span className="icono-kpi">🍷</span><small>Pedido en el período</small></div>
374         <b>{fmt(analisis.pedido_total)} kg</b>
375         <i>en {analisis.servicios_periodo} servicios</i>
376       </div>
377       <div className="kpi">
378         <div className="kpi-cabeza"><span className="icono-kpi">🍷</span><small>Usado realmente</small></div>
379         <b>{fmt(analisis.usado_total)} kg</b>
380         <i>{analisis.aprovechamiento} % de lo pedido</i>
381       </div>
382       <div className="kpi oro">
383         <div className="kpi-cabeza"><span className="icono-kpi">🚫</span><small>Insumos subutilizados</small></div>
384         <b>{analisis.subutilizados.length}</b>
385         <i>se piden y no se usan</i>
386       </div>
387       <div className="kpi verde">
388         <div className="kpi-cabeza"><span className="icono-kpi">💡</span><small>Ahorro potencial</small></div>
389         <b>{fmt(analisis.ahorro_potencial)} kg</b>
390         <i>al mes si se ajusta</i>
391       </div>
392     </div>
393
394     {analisis.subutilizados.length === 0 ? (
395       <div className="card">
396         <p className="vacio">
397           Sin servicios legalizados en los últimos {analisis.dias} días: legalice uno para ver el análisis.
398         </p>
399       </div>
400     ) : (
401       <div className="reportes-cols">
402         <div className="card">
403           <h2>🍷 Sobrepedido frente a la receta</h2>
404           <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
405             {analisis.subutilizados.slice(0, 6).map((s) => (
406               <div key={s.nombre} style={{ display: "flex", alignItems: "center", gap: 10 }}>
407                 <span style={{ width: 130, fontSize: ".83rem" }}>{s.nombre}</span>
408                 <div style={{ flex: 1, background: "var(--fondo)", borderRadius: 6, height: 16 }}>
409                   <div style={{ width: `${s.sobrepedido_pct / maxSobrepedido * 100}%`, height: 16,
410                     background: s.sobrepedido_pct >= 30 ? "var(--amarillo)" : "var(--azul)",
411                     borderRadius: 6 }} />
412                 </div>
413                 <b style={{ width: 34, textAlign: "right", fontSize: ".83rem" }}>{s.sobrepedido_pct}</b>
414               </div>
415             )})
416           </div>
417           <p className="pista" style={{ marginTop: 10 }}>% por encima de lo que pide la receta.</p>
418         </div>
419
420         <div className="card">
421           <h2>🍷 Insumos subutilizados: se piden y sobran</h2>
422           {analisis.subutilizados.slice(0, 6).map((s) => (
423             <div key={s.nombre} className="registro">
424               <span style={{ textTransform: "capitalize" }}>{s.nombre.toLowerCase()}</span>
425               <span className="cant">{fmt(s.sobra)} kg sin usar</span>
426               <span className="chip oro">{s.veces} de {analisis.servicios_periodo} servicios</span>
427             </div>
428           )})
429         </div>
430       </div>
431     )}
432
433     {recomendacion && (
434       <div className="card">
435         <h2>🍷 Recomendación automática del agente</h2>
436         <p className="burbuja">
437           {recomendacion.nombre.toLowerCase()} sobra en {recomendacion.veces} de los servicios
438           legalizados: la receta pide {recomendacion.sobrepedido_pct}% más de lo que en promedio
439           se usa. Ajustar la receta liberaría {fmt(recomendacion.sobra)} kg al mes. La decisión

```

```

440         es del chef: el agente solo muestra el patrón.
441     </p>
442     <div className="grilla-botones">
443         <button className="btn verde" onClick={exportar}
444             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
445             <Icono nombre="descargar" tam={18} />
446             Exportar análisis
447         </button>
448     </div>
449 </div>
450 )}
451 </>
452 );
453 }

```

frontend/src/vistas/Panel.jsx

396 LÍNEAS

La vista gerencial.

```

1 import { useEffect, useRef, useState } from "react";
2 import { pedir } from "../api";
3 import { hablar } from "../voz";
4 import Marco from "../Marco";
5 import AsistenteVoz from "../AsistenteVoz";
6
7 const SERIE = ["var(--serie-1)", "var(--serie-2)", "var(--serie-3)", "var(--serie-4)"];
8 const COLOR_UNIDAD = { Unidad: "var(--azul)", Kilogram: "var(--amarillo)",
9     Liter: "var(--verde)", Portion: "#C3CAD6" };
10 const ESTADO_COLOR = { cerrada: "var(--verde)", en_conteo: "var(--azul)",
11     en_auditoria: "var(--amarillo)", pendiente: "#C3CAD6" };
12 const ESTADO_ETQ = { cerrada: "Cerrada", en_conteo: "Conteo",
13     en_auditoria: "En auditoría", pendiente: "Pendiente" };
14
15 // Nombre oficial de bodega (como en el catálogo) -> version corta para
16 // que quepa en el eje del gráfico. Solo cambia como se muestra, el
17 // nombre real que se manda al backend sigue siendo el oficial.
18 const PREFIJOS_CORTOS = { RESTAURANTE: "Rest.", ALMACEN: "Almacén",
19     ZOOLÓGICO: "Zoológico", KIOSCO: "Kiosco" };
20 function nombreCorto(nombre) {
21     return (nombre || "").split(" ").map((p, i) => {
22         if (i === 0 && PREFIJOS_CORTOS[p]) return PREFIJOS_CORTOS[p];
23         if (p === "AYB") return "AyB";
24         return p.charAt(0) + p.slice(1).toLowerCase();
25     }).join(" ");
26 }
27
28 const MESES = ["ene", "feb", "mar", "abr", "may", "jun", "jul", "ago", "sep", "oct", "nov", "dic"];
29 const mesCorto = (fechaISO) => MESES[Number(fechaISO.slice(5, 7)) - 1];
30
31 // Ejemplos de lo que ya entiende el agente en esta pantalla: navegar entre
32 // las dos pestañas y preguntar por lo que muestra cada tarjeta o gráfica.
33 const _EJEMPLOS_PANEL = [
34     "resumen ejecutivo", "bodegas y alertas", "llévame a bodegas y alertas",
35     "¿cuál es la exactitud primera pasada?", "¿cuántas referencias se han contado?",
36     "¿cuántas alertas se han gestionado?", "¿cuántas bodegas están cerradas?",
37     "¿qué bodega tiene más diferencia?", "¿cómo va el stock por unidad de medida?",
38     "¿cómo va la tendencia de exactitud?", "¿cuántos negativos se han corregido?",
39     "¿cuál es el tiempo promedio de conteo?", "¿cuántos alias aprendió el agente?",
40     "¿cómo están las bodegas?", "¿cuáles son las alertas por tipo?",
41     "¿cuál es el descuadre principal?",
42 ];
43
44 export default function Panel({ token, ir, tabInicial, navSeq }) {
45     const [tab, setTab] = useState(tabInicial || "resumen");
46     // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
47     // componente - sin esto, el useState de arriba no vuelve a leer
48     // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
49     // cada ir()) tiene que ir en la dependencia también: sin él, pedir la
50     // MISMA pestaña dos veces seguidas (con un clic manual a otra en el
51     // medio) no hacía nada, porque tabInicial ya traía ese mismo valor de
52     // la vez anterior y el efecto no vuelve a correr si su dependencia no
53     // cambió de valor.

```

```

54  useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
55  const [resumen, setResumen] = useState(null);
56  const [alertas, setAlertas] = useState(null);
57
58  useEffect(() => {
59    pedir("/api/panel/resumen", {}, token).then(setResumen).catch(() => {});
60    pedir("/api/panel/alertas", {}, token).then(setAlertas).catch(() => {});
61  }, [token]);
62
63  return (
64    <Marco titulo="Panel gerencial" chip={{ texto: "ACTUALIZADO AHORA", tipo: "verde" }}>
65      <AsistenteVoz token={token} vista="panel" ir={ir} ejemplos={_EJEMPLOS_PANEL}
66        placeholder="¿cuál es la exactitud primera pasada?, lléveme a inicio..." />
67      <div className="chips">
68        <button className={`chip ${tab === "resumen" ? "azul" : ""}`}
69          onClick={() => setTab("resumen")}>Resumen ejecutivo</button>
70        <button className={`chip ${tab === "alertas" ? "azul" : ""}`}
71          onClick={() => setTab("alertas")}>Bodegas y alertas</button>
72      </div>
73
74      {tab === "resumen" && <TabResumen r={resumen} />}
75      {tab === "alertas" && <TabAlertas a={alertas} />}
76    </Marco>
77  );
78 }
79
80 function TabResumen({ r }) {
81   const [saludo, setSaludo] = useState("");
82   const yaSaludo = useRef(false);
83
84   // Al llegar a esta pestaña, un titular breve en vez de esperar a que
85   // pregunten - el detalle completo de cada tarjeta/gráfica sigue
86   // disponible por voz (o en el desplegable de frases), esto es solo el
87   // resumen de resumen para no leer las cuatro tarjetas de una sentada.
88   useEffect(() => {
89     if (!r || yaSaludo.current) return;
90     yaSaludo.current = true;
91     const texto = `Resumen ejecutivo: exactitud primera pasada ${r.exactitud_primera_pasada}%, `
92       + `${r.alertas_gestionadas} de ${r.alertas_total} alertas gestionadas, `
93       + `${r.bodegas_cerradas} de ${r.bodegas_total} bodegas cerradas.`;
94     setSaludo(texto);
95     hablar(texto);
96   }, [r]);
97
98   if (!r) return <p className="cargando">Cargando...</p>;
99   const maxDif = Math.max(1, ...r.diferencia_por_bodega.map((d) => d.diferencia));
100
101   return (
102     <>
103       {saludo && (
104         <div className="card">
105           <p className="rotulo" style={{ marginTop: 0 }}>CuentaVoz responde</p>
106           <p className="burbuja" aria-live="polite" aria-atomic="true">{saludo}</p>
107         </div>
108       )}
109       <div className="kpis">
110         <div className="kpi verde">
111           <div className="kpi-cabeza">
112             <span className="icono-kpi">📊</span>
113             <small>Exactitud primera pasada</small>
114           </div>
115           <b>{r.exactitud_primera_pasada} %</b><i>promedio de cierres</i></div>
116         <div className="kpi">
117           <div className="kpi-cabeza">
118             <span className="icono-kpi">📦</span>
119             <small>Referencias contadas</small>
120           </div>
121           <b>{r.referencias_contadas}</b><i>en toda la operación</i></div>
122         <div className="kpi oro">
123           <div className="kpi-cabeza">
124             <span className="icono-kpi">🚨</span>
125             <small>Alertas gestionadas</small>
126           </div>
127           <b>{r.alertas_gestionadas} / {r.alertas_total}</b><i>resueltas del total</i></div>

```

```

128     <div className="kpi">
129       <div className="kpi-cabeza">
130         <span className="icono-kpi">🔒</span>
131         <small>Bodegas cerradas</small>
132       </div>
133       <b>{r.bodegas_cerradas} / {r.bodegas_total}</b><i>con doble firma digital</i></div>
134     </div>
135
136     <div className="conteo-cols">
137       <div className="card">
138         <h2>Diferencia absoluta por bodega</h2>
139         {r.diferencia_por_bodega.length === 0 ? (
140           <p className="vacio">Sin diferencias registradas todavía.</p>
141         ) : (
142           r.diferencia_por_bodega.map((d, i) => (
143             <div className="barra-fila" key={d.bodega}>
144               <span className="etq">{nombreCorto(d.bodega)}</span>
145               <div className="pista">
146                 <div className="rellena" style={{ width: `${d.diferencia / maxDif * 100}%`,
147                   background: i === 0 ? "var(--amarillo)" : "var(--serie-1)" }} />
148               </div>
149               <span className="val">{d.diferencia}</span>
150             </div>
151           ))
152         )}
153       </div>
154       <div className="card">
155         <h2>Stock por unidad de medida</h2>
156         <Dona datos={r.stock_por_unidad.map((u, i) => ({
157           etq: u.unidad, valor: u.cantidad, pct: u.pct,
158           color: COLOR_UNIDAD[u.unidad] || SERIE[i % SERIE.length],
159         })))} />
160       </div>
161     </div>
162
163     <div className="card">
164       <h2>Exactitud por toma de inventario</h2>
165       {r.historial_exactitud.length < 2 ? (
166         <p className="vacio">Se necesitan al menos dos cierres para ver la tendencia.</p>
167       ) : (
168         <Linea puntos={r.historial_exactitud} />
169       )}
170       <p className="pista" style={{ marginTop: 8 }}>
171         Un punto por cada bodega cerrada con doble firma, en orden cronológico.
172       </p>
173     </div>
174   </>
175 );
176 }
177
178 function TabAlertas({ a }) {
179   const [saludo, setSaludo] = useState("");
180   const yaSaludo = useRef(false);
181
182   useEffect(() => {
183     if (!a || yaSaludo.current) return;
184     yaSaludo.current = true;
185     const texto = a.negativos_iniciales !== a.negativos_actuales
186       ? `Bodegas y alertas: saldos negativos, ${a.negativos_iniciales} al inicio del período, `
187       + `${a.negativos_actuales} ahora. Tiempo promedio de conteo ${a.tiempo_promedio_min} minutos.`
188       : `Bodegas y alertas: ${a.negativos_actuales} saldos negativos en el sistema, se corrigen `
189       + `en My Inventory. Tiempo promedio de conteo ${a.tiempo_promedio_min} minutos.`;
190     setSaludo(texto);
191     hablar(texto);
192   }, [a]);
193
194   if (!a) return <p className="cargando">Cargando...</p>;
195   const estados = Object.entries(a.estado_bodegas);
196   const maxAlerta = Math.max(1, ...a.alertas_por_tipo.map((x) => x.cantidad));
197
198   return (
199     <>
200       {saludo && (
201         <div className="card">

```



```

202     <p className="rotulo" style={{ marginTop: 0 }}>CuentaVoz responde</p>
203     <p className="burbuja" aria-live="polite" aria-atomic="true">{saludo}</p>
204 </div>
205 }}
206 <div className="kpi">
207   <div className="kpi oro">
208     <div className="kpi-cabeza">
209       <span className="icono-kpi">📉</span>
210       <small>Saldos negativos en el sistema</small>
211     </div>
212     { /* La corrección de un saldo negativo pasa por fuera de
213       CuentaVoz (My Inventory, no esta app) - dentro de UNA sola
214       toma este número no se mueve solo, así que la flecha
215       "antes → ahora" solo se muestra si de verdad hay una
216       diferencia real que contar; mostrarla igualada a sí misma
217       ("79 → 79") no cuenta nada y parece un dato inventado. */ }
218     {a.negativos_iniciales !== a.negativos_actuales ? (
219       <>
220         <b>{a.negativos_iniciales} → {a.negativos_actuales}</b>
221         <i>corregidos desde el inicio de este período</i>
222       </>
223     ) : (
224       <>
225         <b>{a.negativos_actuales}</b>
226         <i>artículos - la corrección se hace en My Inventory</i>
227       </>
228     ) }
229   </div>
230   <div className="kpi">
231     <div className="kpi-cabeza">
232       <span className="icono-kpi">🕒</span>
233       <small>Tiempo promedio de conteo por bodega</small>
234     </div>
235     <b>{a.tiempo_promedio_min ?? "-"} min</b><i>conteos ya cerrados</i></div>
236   <div className="kpi verde">
237     <div className="kpi-cabeza">
238       <span className="icono-kpi">👤</span>
239       <small>Alias aprendidos por el agente</small>
240     </div>
241     <b>{a.alias_aprendidos}</b><i>el agente habla como el equipo</i></div>
242 </div>
243
244 <div className="dos-columnas">
245   <div className="card">
246     <h2>Estado de las bodegas</h2>
247     <div className="puntos-bodegas">
248       {estados.flatMap(([estado, n]) =>
249         Array.from({ length: n }, (_, i) => (
250           <span key={` ${estado}-${i}`} style={{ background: ESTADO_COLOR[estado] }}
251             title={ESTADO_ETQ[estado]} />
252         ))
253       )}
254     </div>
255     <div className="leyenda">
256       {estados.map(([estado, n]) => (
257         <span key={estado}>
258           <span className="punto" style={{ background: ESTADO_COLOR[estado] }} />
259           {ESTADO_ETQ[estado]} · {n}
260         </span>
261       ))}
262     </div>
263   </div>
264
265   <div className="card">
266     <h2>Alertas por tipo</h2>
267     {a.alertas_por_tipo.length === 0 ? (
268       <p className="vacio">Sin alertas registradas todavía.</p>
269     ) : (
270       a.alertas_por_tipo.map((x, i) => (
271         <div className="barra-fila" key={x.tipo}>
272           <span className="etq">{x.tipo}</span>
273           <div className="pista">
274             <div className="rellena" style={{ width: `${x.cantidad / maxAlerta * 100}%`,
275               background: i === 0 ? "var(--amarillo)" : "var(--serie-1)" }} />

```

```

276         </div>
277         <span className="val">{x.cantidad}</span>
278     </div>
279     ))
280 }
281 </div>
282 </div>
283
284 <div className="card">
285   <h2>Descuadres recurrentes: dónde está la causa raíz</h2>
286   {a.descuadres_recurrentes.length === 0 ? (
287     <p className="vacio">Sin descuadres repetidos todavía.</p>
288   ) : (
289     <table>
290       <thead>
291         <tr><th>Artículo</th><th>Tipo de alerta</th><th>Dif. acumulada</th>
292         <th>Tomas</th><th>Acción sugerida</th></tr>
293       </thead>
294       <tbody>
295         {a.descuadres_recurrentes.map((d, i) => (
296           <tr key={i}>
297             <td>{d.articulo}</td><td>{d.tipo}</td>
298             <td className="dif">{d.diferencia}</td><td>{d.tomas}</td>
299             <td>{d.accion}</td>
300           </tr>
301         ))}
302       </tbody>
303     </table>
304   )}
305 </div>
306 </>
307 );
308 }
309
310 /** Dona hecha con conic-gradient: sin librerías, con leyenda etiquetada. */
311 function Dona({ datos }) {
312   if (!datos.length) return <p className="vacio">Sin datos de stock todavía.</p>;
313   let acumulado = 0;
314   const segmentos = datos.map((d) => {
315     const inicio = acumulado;
316     acumulado += d.pct;
317     return `${d.color} ${inicio}% ${acumulado}%`;
318   });
319   const total = datos.reduce((s, d) => s + d.valor, 0);
320   return (
321     <div style={{ display: "flex", gap: 20, alignItems: "center", flexWrap: "wrap" }}>
322       <div style={{ position: "relative", width: 140, height: 140, flex: "none" }}>
323         <div style={{
324           width: 140, height: 140, borderRadius: "50%",
325           background: `conic-gradient(${segmentos.join(",")}` ,
326         }} />
327         <div style={{
328           position: "absolute", inset: 22, borderRadius: "50%", background: "#fff",
329           display: "flex", flexDirection: "column",
330           alignItems: "center", justifyContent: "center",
331           boxShadow: "0 0 0 1px rgba(11,27,51,.04)",
332         }}>
333           <b style={{ fontSize: "1.05rem", color: "var(--azul)" }}>
334             {total.toLocaleString("es-CO")}
335           </b>
336           <span style={{ fontSize: ".68rem", color: "var(--grafito)" }}>referencias</span>
337         </div>
338       </div>
339       <div>
340         {datos.map((d) => (
341           <div key={d.etq} style={{ marginBottom: 6, fontSize: ".85rem" }}>
342             <span className="punto" style={{ background: d.color, display: "inline-block",
343               width: 10, height: 10, borderRadius: "50%", marginRight: 8 }} />
344             <b>{d.etq}</b> - {d.pct}% ({d.valor.toLocaleString("es-CO")})
345           </div>
346         ))}
347       </div>
348     </div>
349   );

```

```

350 }
351
352 /** Línea de tendencia con SVG puro: una sola serie, sin leyenda. Como
353     todos los cierres pueden caer el mismo día, el eje muestra la bodega
354     de cada punto (ya es información real y distinta punto a punto) en
355     vez de repetir el mismo mes una y otra vez. */
356 function Linea({ puntos }) {
357     const w = 460, h = 128, pad = 20, padInferior = 34;
358     const valores = puntos.map((p) => p.exactitud);
359     const min = Math.min(...valores) - 2, max = Math.max(...valores) + 2;
360     const paso = (w - pad * 2) / (puntos.length - 1);
361     const y = (v) => h - padInferior - ((v - min) / (max - min || 1)) * (h - pad - padInferior);
362     const coords = puntos.map((p, i) => [pad + i * paso, y(p.exactitud)]);
363     const linea = coords.map(([x, yy], i) => `${i === 0 ? "M" : "L"}${x},${yy}`).join(" ");
364
365     const mejora = valores[valores.length - 1] >= valores[0];
366     const color = "var(--serie-1)";
367
368     return (
369         <>
370             {puntos.length > 1 && (
371                 <p style={{ textAlign: "right", fontSize: ".78rem", fontWeight: 700,
372                     color: mejora ? "var(--verde)" : "var(--grafito)", marginBottom: 4 }}>
373                     {mejora ? "▲ mejora sostenida" : "▼ en descenso"}
374                 </p>
375             )}
376             <svg width="100%" viewBox={`0 0 ${w} ${h}`} role="img" aria-label="Tendencia de exactitud">
377                 <line x1={pad} y1={h - padInferior} x2={w - pad} y2={h - padInferior} stroke="var(--borde)" strokeWidth="1"
378                 </line>
379                 <path d={linea} fill="none" stroke={color} strokeWidth="2" />
380                 {coords.map(([x, yy], i) => (
381                     <circle key={i} cx={x} cy={yy} r="3.5" fill={color} />
382                 ))}
383                 {puntos.map((p, i) => (
384                     <text key={i} x={coords[i][0]} y={h - padInferior + 11} fontSize="6.5" textAnchor="end"
385                     fill="var(--grafito)" transform={`rotate(-40 ${coords[i][0]} ${h - padInferior + 11})`}>
386                     <title>{nombreCorto(p.bodega)}</title>
387                     {`Toma ${i + 1}`}
388                     </text>
389                 ))}
390                 <text x={coords[coords.length - 1][0]} y={coords[coords.length - 1][1] - 10}
391                     fontSize="11" fontWeight="700" textAnchor="middle" fill={color}>
392                     {valores[valores.length - 1]}%
393                 </text>
394             </svg>
395         </>
396     );
397 }

```

frontend/src/vistas/Ajustes.jsx

1278 LÍNEAS

Configuración, usuarios, recetas y traza.

```

1 import { useEffect, useState } from "react";
2 import { pedir, descargarReporte } from "../api";
3 import { escuchar, hablar, quitarTildes } from "../voz";
4 import Marco from "../Marco";
5 import AsistenteVoz from "../AsistenteVoz";
6 import Icono from "../Iconos";
7 import Dialogo from "../Dialogo";
8 import { useDevolverFoco } from "../foco";
9
10 const _EJEMPLOS_POR_PESTANA = {
11     config: [
12         "activa el modo sin conexión",
13         "desactiva el modo sin conexión",
14     ],
15     usuarios: [
16         "crea un usuario llamado Juan perfil auxiliar",
17         "activa a Juan", "desactiva a Juan",
18         "cambia el perfil de Juan a administrador",
19         "asigne la bodega Almacén General a Juan",

```

```

20 "quítale la bodega Almacen General a Juan",
21 "¿quién tiene la bodega Almacen General?",
22 "¿cuántas bodegas sin asignar hay?",
23 "muéstrame la asignación por bodega",
24 "oculta la asignación por bodega",
25 ],
26 recetas: [
27 "crea una receta llamada Sopa, rendimiento 4 porciones, con 2 kilos de papa",
28 "agrega 300 gramos de cebolla a la receta Sopa",
29 "quita el ingrediente cebolla de la receta Sopa",
30 "cambia el rendimiento de la receta Sopa a 6 porciones",
31 "agrega la preparación a la receta Sopa: pele las papas, sofría la cebolla, y cocine todo junto veinte minutos",
32 "elimina la receta Sopa",
33 ],
34 traza: [
35 "acciones de Luis hoy",
36 "aprobaciones de esta semana",
37 "exporta el registro de trazabilidad",
38 "acciones por persona",
39 "¿cuántas acciones tiene Luis?",
40 ],
41 };
42
43 export default function Ajustes({ token, usuario, ir, tabInicial, recetaId, navSeq }) {
44   const [tab, setTab] = useState(tabInicial || "config");
45   const esAuditor = usuario?.perfil === "auditor";
46   // "llévame a gestión de usuarios" dicho desde esta MISMA pantalla no
47   // remonta el componente (sigue siendo "ajustes"), así que el useState
48   // de arriba no vuelve a leer tabInicial - sin esto, el agente decía
49   // "vamos a Gestión de usuarios" y la pestaña se quedaba en la que ya
50   // estaba. navSeq (sube en cada ir()) también va en la dependencia: sin
51   // él, pedir la MISMA pestaña dos veces seguidas (con un clic manual a
52   // otra en el medio) no hacía nada, porque tabInicial no cambiaba.
53   useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
54
55   return (
56     <Marco titulo="Ajustes · configuración del sistema"
57       chip={{ texto: esAuditor ? "ADMINISTRADORA" : "SOLO LECTURA",
58         tipo: esAuditor ? "azul" : "gris" }}>
59       <AsistenteVoz token={token} vista="ajustes" ir={ir}
60         placeholder="¿cuál es el umbral de anomalía?, lléveme a reportes..."
61         ejemplos={_EJEMPLOS_POR_PESTANA[tab]}
62         alActualizar={() => window.dispatchEvent(new Event("cuentavoz:ajustes-actualizados"))} />
63       <div className="chips">
64         <button className={`chip ${tab === "config" ? "azul" : ""}`}
65           onClick={() => setTab("config")}>Configuración</button>
66         {esAuditor && (
67           <>
68             <button className={`chip ${tab === "usuarios" ? "azul" : ""}`}
69               onClick={() => setTab("usuarios")}>Gestión de usuarios</button>
70             <button className={`chip ${tab === "recetas" ? "azul" : ""}`}
71               onClick={() => setTab("recetas")}>Recetas</button>
72             <button className={`chip ${tab === "traza" ? "azul" : ""}`}
73               onClick={() => setTab("traza")}>Registro de trazabilidad</button>
74           </>
75         )}
76       </div>
77
78       {tab === "config" && <TabConfig token={token} esAuditor={esAuditor} />}
79       {tab === "usuarios" && esAuditor && <TabUsuarios token={token} usuario={usuario} />}
80       {tab === "recetas" && esAuditor && <TabRecetas token={token} recetaInicial={recetaId} />}
81       {tab === "traza" && esAuditor && <TabTraza token={token} />}
82     </Marco>
83   );
84 }
85
86 /* — Interruptor simple, sin librería — */
87 function Interruptor({ activo, onChange, disabled, etiqueta }) {
88   return (
89     <button
90       onClick={() => !disabled && onChange(!activo)}
91       disabled={disabled}
92       role="switch"
93       aria-checked={activo}

```

```

94     aria-label={etiqueta}
95     style={{
96         width: 44, height: 24, borderRadius: 14, padding: 2,
97         background: activo ? "var(--verde)" : "#C3CAD6",
98         display: "flex", justifyContent: activo ? "flex-end" : "flex-start",
99         opacity: disabled ? 0.6 : 1, cursor: disabled ? "not-allowed" : "pointer",
100     }}
101     >
102     <span style={{ width: 20, height: 20, borderRadius: "50%", background: "#fff" }} />
103 </button>
104 );
105 }
106
107 function TabConfig({ token, esAuditor }) {
108     const [cfg, setCfg] = useState(null);
109     const [umbral, setUmbral] = useState(50);
110     const [offline, setOffline] = useState(true);
111     const [msg, setMsg] = useState("");
112
113     function cargar() {
114         pedir("/api/ajustes", {}, token).then((c) => {
115             setCfg(c); setUmbral(c.umbral); setOffline(c.offline);
116         }).catch(() => {});
117     }
118     useEffect(cargar, [token]);
119     // El agente de "Pregúntele al agente" (arriba, fuera de esta pestaña)
120     // puede activar/desactivar el modo sin conexión por voz - cuando lo
121     // hace, avisa con este evento para que esta pestaña vuelva a leer el
122     // valor real en vez de quedarse mostrando el de antes.
123     useEffect(() => {
124         window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
125         return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
126     }, [token]);
127
128     async function guardar() {
129         try {
130             await pedir("/api/ajustes", {
131                 method: "PUT", body: JSON.stringify({ umbral, offline }),
132             }, token);
133             setMsg("Configuración guardada.");
134             cargar();
135         } catch (e) { setMsg(e.message); }
136     }
137
138     if (!cfg) return <p className="cargando">Cargando...</p>;
139
140     return (
141         <>
142             <div className="kpi">
143                 <div className="kpi">
144                     <div className="kpi-cabeza">
145                         <span className="icono-kpi">👤</span>
146                         <small>Usuarios activos</small>
147                     </div>
148                     <b>{cfg.usuarios_activos}</b><i>con acceso al sistema</i>
149                 </div>
150                 <div className={cfg.aprobaciones_pendientes > 0 ? "kpi oro" : "kpi verde"}>
151                     <div className="kpi-cabeza">
152                         <span className="icono-kpi">✅</span>
153                         <small>Aprobaciones pendientes</small>
154                     </div>
155                     <b>{cfg.aprobaciones_pendientes}</b>
156                     <i>{cfg.aprobaciones_pendientes > 0 ? "por revisar en Auditoría" : "todo al día"}</i>
157                 </div>
158                 <div className={offline ? "kpi verde" : "kpi"}>
159                     <div className="kpi-cabeza">
160                         <span className="icono-kpi">🌐</span>
161                         <small>Modo sin conexión</small>
162                     </div>
163                     <b>{offline ? "Activo" : "Inactivo"}</b><i>refresco del tablero en tiempo real</i>
164                 </div>
165             </div>
166             /* La versión/modelo ya se ve completa en la tarjeta "Acerca de
167                CuentaVoz" de abajo (con base de datos e idioma también) - un KPI

```

```

168     arriba con solo dos de esos cuatro datos era la misma información
169     dos veces, no una tarjeta nueva. */}
170
171 <div className="dos-columnas" style={{ gap: 16 }}>
172   <div className="card">
173     <h2><span className="icono-kpi" style={{ marginRight: 8 }}>🔒</span>Validación de datos</h2>
174     <table>
175       <tbody>
176         <tr>
177           <td>Umbral de anomalía</td>
178           <td style={{ width: 140 }}>
179             {esAuditor ? (
180               <input type="number" min="1" max="500" value={umbral}
181                 onChange={(e) => setUmbral(Number(e.target.value))}
182                 aria-label="Umbral de anomalía, en porcentaje"
183                 style={{ width: 80, padding: "6px 10px",
184                   border: "1px solid var(--borde)", borderRadius: 8 }} />
185               ) : <b>{cfg.umbral} %</b>}
186             {esAuditor && " %"}
187           </td>
188         </tr>
189         <tr><td>Bloquear cantidades negativas</td>
190           <td><Interruptor activo disabled onChange={() => {}}
191             etiqueta="Bloquear cantidades negativas" /></td></tr>
192         <tr><td>Exigir confirmación en alertas</td>
193           <td><Interruptor activo disabled onChange={() => {}}
194             etiqueta="Exigir confirmación en alertas" /></td></tr>
195         <tr><td>Permitir crear productos pendientes</td>
196           <td><Interruptor activo disabled onChange={() => {}}
197             etiqueta="Permitir crear productos pendientes" /></td></tr>
198       </tbody>
199     </table>
200     <p className="pista" style={{ marginTop: 8 }}>
201       Las reglas fijas del reto (bloquear negativos, exigir confirmación) no se
202       pueden desactivar: son la garantía de integridad de la toma.
203     </p>
204   </div>
205
206   <div className="card">
207     <h2><span className="icono-kpi" style={{ marginRight: 8 }}>🌐</span>Conexión y sincronización</h2>
208     <table>
209       <tbody>
210         <tr><td>Modo sin conexión</td>
211           <td><Interruptor activo={offline} onChange={setOffline} disabled={!esAuditor}
212             etiqueta="Modo sin conexión" /></td></tr>
213         <tr><td>Refresco del tablero</td><td><b>{cfg.tiempo_real}</b></td></tr>
214       </tbody>
215     </table>
216
217     <h3 style={{ marginTop: 20 }}>
218       <span className="icono-kpi" style={{ marginRight: 8 }}>🗨️</span>Acerca de CuentaVoz
219     </h3>
220     <table>
221       <tbody>
222         <tr><td>Versión</td><td><b>{cfg.version}</b></td></tr>
223         <tr><td>Modelo del agente</td><td><b>{cfg.modelo}</b></td></tr>
224         <tr><td>Base de datos</td><td><b>{cfg.base_datos}</b></td></tr>
225         <tr><td>Idioma del reconocimiento</td><td><b>{cfg.idioma_voz}</b></td></tr>
226       </tbody>
227     </table>
228   </div>
229 </div>
230
231 {esAuditor && (
232   <div className="grilla-botones" style={{ marginTop: 4 }}>
233     <button className="btn" onClick={guardar}
234       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
235       <Icono nombre="aprobar" tam={18} />
236       Guardar configuración
237     </button>
238     <button className="btn borde" onClick={cargar}
239       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
240       <Icono nombre="refrescar" tam={18} />
241       Restaurar valores

```

```

242         </button>
243     </div>
244     })
245     {msg && <p className="msg-ok">{msg}</p>}
246     </>
247     );
248 }
249
250 function TabUsuarios({ token, usuario }) {
251     const [usuarios, setUsuarios] = useState([]);
252     const [bodegas, setBodegas] = useState([]);
253     const [msg, setMsg] = useState("");
254     const [nuevo, setNuevo] = useState(null);
255     const [asignando, setAsignando] = useState(null); // usuario al que se le estan marcando bodegas
256     useDevolverFoco(Boolean(asignando));
257     const [marcadas, setMarcadas] = useState(new Set());
258     const [sedes, setSedes] = useState([]);
259     const [sinSede, setSinSede] = useState(0);
260     const [verSedes, setVerSedes] = useState(false);
261     const [sedeNueva, setSedeNueva] = useState(null); // {nombre, ciudad}
262     const [borrarSede, setBorrarSede] = useState(null);
263     const [cobertura, setCobertura] = useState(null);
264     const [verCobertura, setVerCobertura] = useState(false);
265     const [editando, setEditando] = useState(null); // usuario que se esta editando (correo/rol)
266     useDevolverFoco(Boolean(editando));
267     const [filtro, setFiltro] = useState("todos");
268
269     function cargar() {
270         pedir("/api/usuarios", {}, token).then(setUsuarios).catch(() => {});
271         // ?propias=1: la lista para repartir. Un administrador general (sin
272         // bodegas asignadas) sigue viendo el parque completo; uno de sede ve
273         // solo la suya, que es justo lo que el backend le va a dejar guardar.
274         pedir("/api/bodegas?propias=1", {}, token).then(setBodegas).catch(() => {});
275         cargarSedes();
276         if (verCobertura) cargarCobertura();
277     }
278     useEffect(cargar, [token]);
279     // El agente ("Pregúntele al agente", arriba de las pestañas) puede
280     // activar/desactivar a alguien por voz - cuando lo hace, avisa con
281     // este evento para que esta lista refleje el cambio sin recargar.
282     useEffect(() => {
283         window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
284         return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
285     }, [token]);
286
287     // Estos dos modales estan armados a mano (overlay + modal), no con el
288     // componente Dialogo compartido - por eso no traian ya el mismo Escape
289     // para cerrar que ese componente sí tiene.
290     useEffect(() => {
291         function alTecla(e) {
292             if (e.key !== "Escape") return;
293             if (editando) setEditando(null);
294             else if (asignando) setAsignando(null);
295         }
296         window.addEventListener("keydown", alTecla);
297         return () => window.removeEventListener("keydown", alTecla);
298     }, [editando, asignando]);
299
300     function cargarCobertura() {
301         pedir("/api/bodegas/asignaciones", {}, token).then(setCobertura).catch(() => {});
302     }
303
304     // "muéstrame/oculta la asignación por bodega" por voz - mismo botón
305     // «Ver/Ocultar asignación por bodega», solo que disparado por el
306     // agente en vez del clic.
307     useEffect(() => {
308         const mostrar = () => { setVerCobertura(true); cargarCobertura(); };
309         const ocultar = () => { setVerCobertura(false); };
310         window.addEventListener("cuentavoz:accion:mostrar_cobertura", mostrar);
311         window.addEventListener("cuentavoz:accion:ocultar_cobertura", ocultar);
312         return () => {
313             window.removeEventListener("cuentavoz:accion:mostrar_cobertura", mostrar);
314             window.removeEventListener("cuentavoz:accion:ocultar_cobertura", ocultar);
315         };
316     });

```

```

316   }, [token]);
317
318   async function crear() {
319     if (!nuevo?.nombre?.trim()) return;
320     try {
321       // sin "pin" en el body: el backend genera uno temporal distinto
322       // para cada persona en vez de reusar siempre la misma clave.
323       const r = await pedir("/api/usuarios", {
324         method: "POST",
325         body: JSON.stringify({ nombre: nuevo.nombre, perfil: nuevo.perfil || "auxiliar",
326                               correo: nuevo.correo || "" }),
327       }, token);
328       setMsg(`Usuario ${nuevo.nombre} creado. Clave temporal: ${r.pin_temporal} `
329         + "(cambíelo al primer ingreso, en Mi perfil).");
330       setNuevo(null);
331       cargar();
332     } catch (e) { setMsg(e.message); }
333   }
334
335   function alternarBodega(id) {
336     setMarcadas((prev) => {
337       const nueva = new Set(prev);
338       nueva.has(id) ? nueva.delete(id) : nueva.add(id);
339       return nueva;
340     });
341   }
342
343   /* Repartir por sede es el motivo por el que la sede existe: quien
344      lleva doce bodegas de Piscilago no debería marcarlas de a una. Los ids
345      los da el backend, que además ya filtra los que quien reparte no puede
346      repartir (ver /api/sedes/{id}/bodegas). */
347   function cargarSedes() {
348     pedir("/api/sedes", {}, token)
349       .then((r) => { setSedes(r.sedes || []); setSinSede(r.bodegas_sin_sede || 0); })
350       .catch(() => {});
351   }
352
353   async function crearSede() {
354     if (!sedeNueva?.nombre?.trim()) return;
355     try {
356       await pedir("/api/sedes", {
357         method: "POST",
358         body: JSON.stringify({ nombre: sedeNueva.nombre, ciudad: sedeNueva.ciudad || "" }),
359       }, token);
360       setMsg(`Sede ${sedeNueva.nombre.toUpperCase()} creada.`);
361       setSedeNueva(null);
362       cargarSedes();
363     } catch (e) { setMsg(e.message); }
364   }
365
366   async function eliminarSede(sd) {
367     try {
368       const r = await pedir(`/api/sedes/${sd.id}`, { method: "DELETE" }, token);
369       setMsg(`Sede ${sd.nombre} eliminada. ${r.bodegas_sin_sede} bodegas quedaron sin sede.`);
370       setBorrarSede(null);
371       cargarSedes();
372       cargar();
373     } catch (e) { setMsg(e.message); }
374   }
375
376   async function cambiarSedeDeBodega(bodegaId, sedeId) {
377     try {
378       await pedir(`/api/bodegas/${bodegaId}/sede`, {
379         method: "PUT",
380         body: JSON.stringify({ sede_id: sedeId === "" ? null : Number(sedeId) }),
381       }, token);
382       cargarSedes();
383       cargar();
384     } catch (e) { setMsg(e.message); }
385   }
386
387   async function marcarSedeEntera(sedeId) {
388     try {
389       const ids = await pedir(`/api/sedes/${sedeId}/bodegas`, {}, token);

```



```

390     setMarcadas((prev) => new Set([...prev, ...ids]));
391   } catch (e) { setMsg(e.message); }
392 }
393
394 async function guardarAsignacion() {
395   const u = asignando;
396   const ids = [...marcadas];
397   setAsignando(null);
398   try {
399     await pedir(`/api/usuarios/${u.id}/bodegas`, {
400       method: "PUT", body: JSON.stringify({ bodega_ids: ids }),
401     }, token);
402     setMsg(`${ids.length} bodegas asignadas a ${u.nombre}.`);
403     cargar();
404   } catch (e) { setMsg(e.message); }
405 }
406
407 async function guardarEdicion() {
408   const u = editando;
409   setEditando(null);
410   try {
411     await pedir(`/api/usuarios/${u.id}`, {
412       method: "PUT", body: JSON.stringify({ correo: u.correo, perfil: u.perfil }),
413     }, token);
414     setMsg(`${u.nombre} actualizado.`);
415     cargar();
416   } catch (e) { setMsg(e.message); }
417 }
418
419 async function alternarActivo(persona) {
420   try {
421     await pedir(`/api/usuarios/${persona.id}`, {
422       method: "PUT", body: JSON.stringify({ activo: !persona.activo }),
423     }, token);
424     setMsg(`${persona.nombre} ${persona.activo ? "desactivado" : "activado"}.`);
425     cargar();
426   } catch (e) { setMsg(e.message); }
427 }
428
429 const nAux = usuarios.filter((u) => u.perfil === "auxiliar").length;
430 const nAdmin = usuarios.filter((u) => u.perfil === "auditor").length;
431 const nInactivos = usuarios.filter((u) => !u.activo).length;
432 const usuariosFiltrados = usuarios.filter((u) => {
433   if (filtro === "auxiliar") return u.perfil === "auxiliar";
434   if (filtro === "auditor") return u.perfil === "auditor";
435   if (filtro === "inactivo") return !u.activo;
436   return true;
437 });
438
439 return (
440   <div className="card">
441     <h2><span className="icono-kpi" style={{ marginRight: 8 }}><img alt="icono-kpi" data-bbox="568 664 581 674"/></span>
442       Gestión de usuarios ({usuarios.length})</h2>
443     <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center",
444       marginBottom: 12, flexWrap: "wrap", gap: 10 }}>
445       <div className="chips" style={{ marginBottom: 0 }}>
446         <button className={`chip ${filtro === "auxiliar" ? "azul" : ""}`}
447           onClick={() => setFiltro(filtro === "auxiliar" ? "todos" : "auxiliar")}>
448           {nAux} auxiliares
449         </button>
450         <button className={`chip ${filtro === "auditor" ? "azul" : ""}`}
451           onClick={() => setFiltro(filtro === "auditor" ? "todos" : "auditor")}>
452           {nAdmin} administradores
453         </button>
454         <button className={`chip ${filtro === "inactivo" ? "azul" : ""}`}
455           onClick={() => setFiltro(filtro === "inactivo" ? "todos" : "inactivo")}>
456           {nInactivos} inactivos
457         </button>
458       </div>
459       <button className="btn" onClick={() => setNuevo({})}>+ Nuevo usuario</button>
460     </div>
461     <div>
462       {msg && <p className="msg-ok">{msg}</p>}

```

```

464 <table>
465 <thead><tr><th>Persona</th><th>Perfil</th><th>Bodegas asignadas</th>
466 <th>Último acceso</th><th>Estado</th><th></th></tr></thead>
467 <tbody>
468 {usuariosFiltrados.map((u) => (
469 <tr key={u.id}>
470 <td style={{ textTransform: "capitalize", display: "flex", alignItems: "center", gap: 8 }}>
471 <span className="avatar-chico">{u.nombre[0]?.toUpperCase()}</span>
472 {u.nombre}
473 </td>
474 <td>{u.perfil === "auditor" ? "Administrador" : "Auxiliar"}</td>
475 <td>{u.bodegas_asignadas}</td>
476 <td>{u.ultimo_acceso ? u.ultimo_acceso.slice(5).replace("-", "/") : "nunca"}</td>
477 <td style={{ color: u.activo ? "var(--verde)" : "var(--grafito)", fontWeight: 700 }}>
478 {u.activo ? "Activo" : "Inactivo"}
479 </td>
480 <td style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
481 <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
482 onClick={async () => {
483 setAsignando(u);
484 const propias = await pedir(`/api/usuarios/${u.id}/bodegas`, {}, token).catch(() => []);
485 setMarcadas(new Set(propias));
486 }}>
487 Asignar bodegas
488 </button>
489 <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
490 onClick={() => setEditando({ id: u.id, nombre: u.nombre,
491 correo: u.correo, perfil: u.perfil })}>
492 Editar
493 </button>
494 <button className={`btn ${u.activo ? "gris" : "verde"}`}
495 style={{ padding: "4px 10px", minHeight: 0 }}
496 disabled={u.id === usuario?.id}
497 title={u.id === usuario?.id ? "No puede desactivar su propia cuenta" : ""}
498 onClick={() => alternarActivo(u)}>
499 {u.activo ? "Desactivar" : "Activar"}
500 </button>
501 </td>
502 </tr>
503 ))}
504 </tbody>
505 </table>
506
507 {nuevo ? (
508 <div style={{ marginTop: 16, display: "flex", gap: 10, flexWrap: "wrap" }}>
509 <input placeholder="nombre de usuario" value={nuevo.nombre || ""}
510 onChange={(e) => setNuevo({ ...nuevo, nombre: e.target.value })}
511 aria-label="Nombre de usuario"
512 style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }} />
513 <input placeholder="correo (obligatorio)" value={nuevo.correo || ""}
514 onChange={(e) => setNuevo({ ...nuevo, correo: e.target.value })}
515 aria-label="Correo, obligatorio"
516 style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }} />
517 <select value={nuevo.perfil || "auxiliar"}
518 onChange={(e) => setNuevo({ ...nuevo, perfil: e.target.value })}
519 aria-label="Perfil del nuevo usuario"
520 style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
521 <option value="auxiliar">Auxiliar</option>
522 <option value="auditor">Administrador</option>
523 </select>
524 <button className="btn" onClick={crear}>Crear (clave temporal)</button>
525 <button className="btn gris" onClick={() => setNuevo(null)}>Cancelar</button>
526 </div>
527 ) : null}
528
529 <div className="grilla-botones" style={{ marginTop: 14 }}>
530 <button className="btn borde"
531 onClick={() => {
532 const abrir = !verCobertura;
533 setVerCobertura(abrir);
534 if (abrir && !cobertura) cargarCobertura();
535 }}>
536 {verCobertura ? "Ocultar asignación por bodega" : "Ver asignación por bodega"}
537 </button>

```

```

538     </div>
539
540     {verCobertura && (
541       <div style={{ marginTop: 14 }}>
542         <h3>Quién tiene cada bodega</h3>
543         <p className="pista" style={{ marginBottom: 10 }}>
544           Para repartir 54 bodegas entre varios auxiliares sin dejar ninguna sin dueño
545           ni asignarla dos veces por error.
546         </p>
547         {cobertura === null ? (
548           <p className="cargando">Cargando...</p>
549         ) : (
550           <div style={{ maxHeight: 420, overflowY: "auto" }}>
551             <table>
552               <thead><tr><th>Bodega</th><th>Asignada a</th></tr></thead>
553               <tbody>
554                 {cobertura.map((b) => (
555                   <tr key={b.id}>
556                     <td>{b.bodega}</td>
557                     <td>
558                       {b.asignados.length === 0 ? (
559                         <span className="chip oro">Sin asignar</span>
560                       ) : (
561                         b.asignados.map((p) => (
562                           <span key={p.id} className={`chip ${p.perfil === "auditor" ? "azul" : "gris"}`}
563                             style={{ marginRight: 6, textTransform: "capitalize" }}>
564                             {p.nombre}
565                           </span>
566                         ))
567                       )}
568                     </td>
569                   </tr>
570                 )]}
571               </tbody>
572             </table>
573           </div>
574         )}
575       </div>
576     )}
577
578
579     <div className="card" style={{ marginTop: 14 }}>
580       <h2>Sedes</h2>
581       <p className="pista" style={{ marginBottom: 10 }}>
582         Agrupan bodegas por sitio para poder repartirlas de una vez, en vez de marcar
583         doce a mano. <b>No dan ni quitan permisos</b>: quién puede entrar a cada bodega
584         se sigue decidiendo bodega por bodega, arriba.
585       </p>
586       <div className="grilla-botones">
587         <button className="btn borde" onClick={() => setVerSedes((v) => !v)}>
588           {verSedes ? "Ocultar sedes" : `Ver sedes (${sedes.length})`}
589         </button>
590         <button className="btn" onClick={() => setSedeNueva({ nombre: "", ciudad: "" })}>
591           + Nueva sede
592         </button>
593       </div>
594
595       {verSedes && (
596         <div style={{ marginTop: 12 }}>
597           {sedes.length === 0 ? (
598             <p className="pista">
599               Todavía no hay sedes. Las {sinSede} bodegas funcionan igual sin ellas:
600               una sede solo hace falta el día que haya varias y convenga repartirlas
601               por sitio.
602             </p>
603           ) : (
604             <>
605               <table>
606                 <thead>
607                   <tr><th>Sede</th><th>Ciudad</th><th>Bodegas</th><th></th></tr>
608                 </thead>
609                 <tbody>
610                   {sedes.map((sd) => (
611                     <tr key={sd.id}>

```

```

612         <td><b>{sd.nombre}</b></td>
613         <td>{sd.ciudad || "-"}</td>
614         <td>{sd.bodegas}</td>
615         <td style={{ textAlign: "right" }}>
616             <button className="btn gris" style={{ padding: "5px 10px", minHeight: 0 }}
617                 onClick={() => setBorrarSede(sd)}>
618                 Eliminar
619             </button>
620         </td>
621     </tr>
622     </tbody>
623 </table>
624 {sinSede > 0 && (
625     <p className="pista" style={{ marginTop: 8 }}>
626         {sinSede} bodegas sin sede. No es un problema: siguen funcionando igual.
627     </p>
628 )}
629 </>
630 ))
631
632 <h3 style={{ marginTop: 18 }}>De qué sede es cada bodega</h3>
633 <div style={{ maxHeight: 320, overflowY: "auto", marginTop: 8 }}>
634     <table>
635         <thead><tr><th>Bodega</th><th>Sede</th></tr></thead>
636         <tbody>
637             {bodegas.map((b) => (
638                 <tr key={b.id}>
639                     <td>{b.bodega}</td>
640                     <td>
641                         <select value={b.sede_id ?? ""}
642                             aria-label={`Sede de ${b.bodega}`}
643                             onChange={(e) => cambiarSedeDeBodega(b.id, e.target.value)}>
644                             <option value="">Sin sede</option>
645                             {sedes.map((sd) => (
646                                 <option key={sd.id} value={sd.id}>{sd.nombre}</option>
647                             ))}
648                         </select>
649                     </td>
650                 </tr>
651             ))}
652         </tbody>
653     </table>
654 </div>
655 </div>
656 ))
657 </div>
658
659 <div className="card" style={{ background: "var(--fondo)", marginTop: 14 }}>
660     <h2>Qué puede hacer cada perfil</h2>
661     <table>
662         <tbody>
663             <tr>
664                 <td>✅ Contar por voz y corregir sus propios registros</td>
665                 <td style={{ textAlign: "right", color: "var(--verde)", fontWeight: 700 }}>
666                     Auxiliar y administrador
667                 </td>
668             </tr>
669             <tr>
670                 <td>✅ Crear productos y bodegas (quedan pendientes)</td>
671                 <td style={{ textAlign: "right", color: "var(--verde)", fontWeight: 700 }}>
672                     Auxiliar y administrador
673                 </td>
674             </tr>
675             <tr>
676                 <td>✅ Aprobar creaciones, auditar y cerrar bodegas</td>
677                 <td style={{ textAlign: "right", color: "var(--azul)", fontWeight: 700 }}>
678                     Solo administrador
679                 </td>
680             </tr>
681             <tr>
682                 <td>✅ Gestionar usuarios y cambiar los ajustes del sistema</td>
683                 <td style={{ textAlign: "right", color: "var(--azul)", fontWeight: 700 }}>
684                     Solo administrador
685                 </td>
686             </tr>
687         </tbody>
688     </table>
689 </div>

```

```

686         </td>
687     </tr>
688 </tbody>
689 </table>
690 </div>
691
692 {editando && (
693     <div className="overlay" onClick={() => setEditando(null)}>
694         <div className="modal" style={{ maxWidth: 420, textAlign: "left" }}
695             onClick={(e) => e.stopPropagation()}
696             role="dialog" aria-modal="true" aria-labelledby="titulo-editar-usuario">
697             <h2 id="titulo-editar-usuario" style={{ textTransform: "capitalize" }}>
698                 Editar a {editando.nombre}
699             </h2>
700             <label className="pista" htmlFor="campo-correo-editar">Correo</label>
701             <input id="campo-correo-editar" value={editando.correo || ""}
702                 onChange={(e) => setEditando({ ...editando, correo: e.target.value })}
703                 style={{ width: "100%", padding: "10px 12px", marginTop: 6, marginBottom: 14,
704                     border: "1px solid var(--borde)", borderRadius: 10 }} />
705             <label className="pista" htmlFor="campo-perfil-editar">Perfil</label>
706             <select id="campo-perfil-editar" value={editando.perfil}
707                 onChange={(e) => setEditando({ ...editando, perfil: e.target.value })}
708                 style={{ width: "100%", padding: "10px 12px", marginTop: 6,
709                     border: "1px solid var(--borde)", borderRadius: 10 }}
710                 disabled={editando.id === usuario?.id}>
711                 <option value="auxiliar">Auxiliar</option>
712                 <option value="auditor">Administrador</option>
713             </select>
714             {editando.id === usuario?.id && (
715                 <p className="pista" style={{ marginTop: 6 }}>
716                     No puede cambiar su propio rol desde aquí.
717                 </p>
718             )}
719             <div className="botones" style={{ marginTop: 18 }}>
720                 <button className="btn borde" onClick={() => setEditando(null)}>Cancelar</button>
721                 <button className="btn" onClick={guardarEdicion}>Guardar</button>
722             </div>
723         </div>
724     </div>
725 )}
726
727 {sedeNueva && (
728     <div className="overlay" onClick={() => setSedeNueva(null)}>
729         <div className="modal" style={{ maxWidth: 380, textAlign: "left" }}
730             onClick={(e) => e.stopPropagation()}
731             role="dialog" aria-modal="true" aria-labelledby="titulo-sede-nueva">
732             <h2 id="titulo-sede-nueva">Nueva sede</h2>
733             /* label.pista y el campo con su ancho: igual que los otros
734             dialogos de esta pantalla. Sin la clase, el <label> va en
735             línea y la etiqueta se pega al campo. */
736             <label className="pista" htmlFor="sede-nombre">Nombre</label>
737             <input id="sede-nombre" autoFocus value={sedeNueva.nombre}
738                 placeholder="Piscilago"
739                 onKeyDown={(e) => e.key === "Enter" && crearSede()}
740                 onChange={(e) => setSedeNueva({ ...sedeNueva, nombre: e.target.value })}
741                 style={{ width: "100%", padding: "10px 12px", marginTop: 6, marginBottom: 14,
742                     border: "1px solid var(--borde)", borderRadius: 10 }} />
743             <label className="pista" htmlFor="sede-ciudad">Ciudad (opcional)</label>
744             <input id="sede-ciudad" value={sedeNueva.ciudad}
745                 placeholder="Girardot"
746                 onKeyDown={(e) => e.key === "Enter" && crearSede()}
747                 onChange={(e) => setSedeNueva({ ...sedeNueva, ciudad: e.target.value })}
748                 style={{ width: "100%", padding: "10px 12px", marginTop: 6,
749                     border: "1px solid var(--borde)", borderRadius: 10 }} />
750             <div className="botones" style={{ marginTop: 14 }}>
751                 <button className="btn borde" onClick={() => setSedeNueva(null)}>Cancelar</button>
752                 <button className="btn" onClick={crearSede}>Crear</button>
753             </div>
754         </div>
755     </div>
756 )}
757
758 {borrarSede && (
759     <Dialogo titulo={`Eliminar la sede ${borrarSede.nombre}`}

```

```

760     mensaje={`Sus ${borrarSede.bodegas} bodegas NO se borran: quedan sin sede, `
761         + "que es un estado normal, y se pueden volver a agrupar cuando quiera."}
762     textoAceptar="Eliminar"
763     peligro
764     onAceptar={() => eliminarSede(borrarSede)}
765     onCancel={() => setBorrarSede(null)} />
766   }}
767
768   {asignando && (
769     <div className="overlay" onClick={() => setAsignando(null)}>
770       <div className="modal" style={{ maxWidth: 520, textAlign: "left" }}
771         onClick={(e) => e.stopPropagation()}
772         role="dialog" aria-modal="true" aria-labelledby="titulo-asignar-bodegas">
773         <h2 id="titulo-asignar-bodegas" style={{ textTransform: "capitalize" }}>
774           Bodegas para {asignando.nombre}
775         </h2>
776         <p className="pista">
777           {asignando.perfil === "auditor"
778             ? "Marque las bodegas de las que esta persona es responsable como administradora (recuento ciego,
779               cierre).": "Marque las bodegas que va a poder contar esta persona."}
780         </p>
781         {sedes.length > 0 && (
782           <div style={{ margin: "10px 0 0" }}>
783             <p className="pista" style={{ marginBottom: 6 }}>
784               Marcar una sede entera:
785             </p>
786             <div style={{ display: "flex", flexWrap: "wrap", gap: 6 }}>
787               {sedes.filter((sd) => sd.bodegas > 0).map((sd) => (
788                 <button key={sd.id} type="button" className="chip"
789                   onClick={() => marcarSedeEntera(sd.id)}>
790                   + {sd.nombre} ({sd.bodegas})
791                 </button>
792               ))}
793             </div>
794           </div>
795         )}
796         <div style={{ maxHeight: 320, overflowY: "auto", margin: "12px 0",
797           border: "1px solid var(--borde)", borderRadius: 10, padding: 8 }}>
798           {bodegas.map((b) => (
799             <label key={b.id} style={{ display: "flex", alignItems: "center", gap: 10,
800               padding: "7px 6px", fontSize: ".9rem" }}>
801               <input type="checkbox" checked={marcadas.has(b.id)}
802                 onChange={() => alternarBodega(b.id)} />
803               <span style={{ flex: 1 }}>{b.bodega}</span>
804               {b.sede && <span className="chip" style={{ fontSize: ".7rem" }}>{b.sede}</span>
805             </label>
806           ))}
807         </div>
808         <p className="pista">{marcadas.size} bodegas marcadas</p>
809         <div className="botones">
810           <button className="btn borde" onClick={() => setAsignando(null)}>Cancelar</button>
811           <button className="btn" onClick={guardarAsignacion}>Guardar</button>
812         </div>
813       </div>
814     </div>
815   )}
816 </div>
817 );
818 }
819
820 function TabRecetas({ token, recetaInicial }) {
821   const [recetas, setRecetas] = useState([]);
822   const [catalogo, setCatalogo] = useState([]);
823   const [msg, setMsg] = useState("");
824   const [editando, setEditando] = useState(null);
825   useDevolverFoco(Boolean(editando));
826   // { id: null|number, nombre, rendimiento, lineas: [{articulo_codigo, cantidad_por_porcion}] }
827   const abrioInicial = useState(false);
828
829   function cargar() {
830     pedir("/api/recetas", {}, token).then(setRecetas).catch(() => {});
831     pedir("/api/articulos/catalogo-ligero", {}, token).then(setCatalogo).catch(() => {});
832   }

```

```

833   useEffect(cargar, [token]);
834
835   useEffect(() => {
836     window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
837     return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
838   }, [token]);
839
840   useEffect(() => {
841     if (recetaInicial && recetas.length && !abrioInicial[0]) {
842       abrioInicial[1](true);
843       abrir(recetaInicial);
844     }
845   }, [recetaInicial, recetas]);
846
847   // Mismo caso que en TabUsuarios: este modal esta armado a mano, no con
848   // el componente Dialogo compartido, así que necesita su propio Escape.
849   useEffect(() => {
850     function alTecla(e) { if (e.key === "Escape" && editando) setEditando(null); }
851     window.addEventListener("keydown", alTecla);
852     return () => window.removeEventListener("keydown", alTecla);
853   }, [editando]);
854
855   async function abrir(id) {
856     try {
857       const r = await pedir(`/api/recetas/${id}`, {}, token);
858       setEditando({
859         id: r.id, nombre: r.nombre, rendimiento: r.rendimiento,
860         preparacion: r.preparacion || "",
861         lineas: r.lineas.map((l) => ({ articulo_codigo: l.articulo_codigo,
862                                     cantidad_por_porcion: l.cantidad_por_porcion })),
863       });
864     } catch (e) { setMsg(e.message); }
865   }
866
867   function nueva() {
868     setEditando({ id: null, nombre: "", rendimiento: 1, preparacion: "",
869                 lineas: [{ articulo_codigo: "", cantidad_por_porcion: "" }] });
870   }
871
872   function actualizarLinea(i, campo, valor) {
873     setEditando((e) => {
874       const lineas = e.lineas.map((l, idx) => idx === i ? { ...l, [campo]: valor } : l);
875       return { ...e, lineas };
876     });
877   }
878
879   function agregarLinea() {
880     setEditando((e) => ({ ...e, lineas: [...e.lineas, { articulo_codigo: "", cantidad_por_porcion: "" }] }));
881   }
882
883   function quitarLinea(i) {
884     setEditando((e) => ({ ...e, lineas: e.lineas.filter((_, idx) => idx !== i) }));
885   }
886
887   async function guardar() {
888     const cuerpo = {
889       nombre: editando.nombre.trim(),
890       rendimiento: Number(editando.rendimiento) || 1,
891       preparacion: (editando.preparacion || "").trim(),
892       ingredientes: editando.lineas
893         .filter((l) => l.articulo_codigo && l.cantidad_por_porcion)
894         .map((l) => ({ articulo_codigo: l.articulo_codigo,
895                       cantidad_por_porcion: Number(l.cantidad_por_porcion) })),
896     };
897     if (!cuerpo.nombre) { setMsg("Póngale un nombre a la receta."); return; }
898     try {
899       if (editando.id) {
900         await pedir(`/api/recetas/${editando.id}`, { method: "PUT", body: JSON.stringify(cuerpo) }, token);
901         setMsg(`Receta «${cuerpo.nombre}» actualizada.`);
902       } else {
903         await pedir("/api/recetas", { method: "POST", body: JSON.stringify(cuerpo) }, token);
904         setMsg(`Receta «${cuerpo.nombre}» creada.`);
905       }
906       setEditando(null);

```

```

907     cargar();
908   } catch (e) { setMsg(e.message); }
909 }
910
911 async function eliminar(r) {
912   if (!window.confirm(`¿Eliminar la receta «${r.nombre}»? Esta acción no se puede deshacer.`)) return;
913   try {
914     await pedir(`/api/recetas/${r.id}`, { method: "DELETE" }, token);
915     setMsg(`Receta «${r.nombre}» eliminada.`);
916     cargar();
917   } catch (e) { setMsg(e.message); }
918 }
919
920 return (
921   <div className="card">
922     <h2>
923       <span className="icono-kpi" style={{ marginRight: 8 }}>🍴</span>
924       Recetas ({recetas.length})
925     </h2>
926     <p className="pista">
927       Las porciones que dicte el chef se calculan sobre estas recetas: cantidad por
928       porción x porciones, menos lo que ya haya en la bodega. Aquí se crean, editan
929       y borran – CuentaVoz sí gestiona el catálogo de recetas y menús.
930     </p>
931     {msg && <p className="msg-ok">{msg}</p>}
932     {recetas.length === 0 ? (
933       <p className="vacio">Todavía no hay recetas creadas.</p>
934     ) : (
935       <table>
936         <thead><tr><th>Receta</th><th>Rendimiento</th><th>Ingredientes</th><th></th></tr></thead>
937         <tbody>
938           {recetas.map((r) => (
939             <tr key={r.id}>
940               <td style={{ textTransform: "capitalize" }}>{r.nombre}</td>
941               <td>{r.rendimiento}<td>porción</td>
942               <td>{r.ingredientes}</td>
943               <td style={{ display: "flex", gap: 6 }}>
944                 <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
945                   onClick={() => abrir(r.id)}>Editar</button>
946                 <button className="btn gris" style={{ padding: "4px 10px", minHeight: 0 }}
947                   onClick={() => eliminar(r)}>Eliminar</button>
948               </td>
949             </tr>
950           ))}
951         </tbody>
952       </table>
953     )}
954     <div className="grilla-botones" style={{ marginTop: 14 }}>
955       <button className="btn" onClick={nueva}>+ Nueva receta</button>
956     </div>
957
958     {editando && (
959       <div className="overlay" onClick={() => setEditando(null)}>
960         <div className="modal" style={{ maxWidth: 660, textAlign: "left", maxHeight: "88vh",
961           overflowY: "auto" }}>
962           onClick={(e) => e.stopPropagation()}
963           role="dialog" aria-modal="true" aria-labelledby="titulo-editar-receta">
964             <h2 id="titulo-editar-receta">
965               {editando.id ? `Editar ${editando.nombre}` : "Nueva receta"}
966             </h2>
967             <p className="pista" style={{ marginTop: -6, marginBottom: 18 }}>
968               Cada campo queda disponible de inmediato para calcular pedidos por voz.
969             </p>
970
971             <div style={{ display: "grid", gridTemplateColumns: "2fr 1fr", gap: 14, marginBottom: 20 }}>
972               <div>
973                 <label className="pista" htmlFor="campo-nombre-plato" style={{ display: "block", marginBottom: 6 }}>
974                   Nombre del plato
975                 </label>
976                 <input id="campo-nombre-plato" value={editando.nombre}
977                   onChange={(e) => setEditando({ ...editando, nombre: e.target.value })}
978                   placeholder="ej. ajiaco"
979                   style={{ width: "100%", padding: "10px 12px",
980                     border: "1px solid var(--borde)", borderRadius: 10 }} />

```



```

981     </div>
982     <div>
983       <label className="pista" htmlFor="campo-rendimiento" style={{ display: "block", marginBottom: 6 }}>
984         Rendimiento (porciones)
985       </label>
986       <input id="campo-rendimiento" type="number" min="1" value={editando.rendimiento}
987         onChange={(e) => setEditando({ ...editando, rendimiento: e.target.value })}
988         style={{ width: "100%", padding: "10px 12px",
989           border: "1px solid var(--borde)", borderRadius: 10 }} />
990     </div>
991   </div>
992
993   <div style={{ background: "var(--fondo)", borderRadius: 12, padding: 16, marginBottom: 18 }}>
994     <h3 style={{ marginTop: 0, marginBottom: 4, fontSize: ".95rem", color: "var(--azul)" }}>
995       🍴 Ingredientes
996     </h3>
997     <p className="pista" style={{ marginBottom: 12 }}>Cantidad por cada porción de la receta.</p>
998     <div style={{ maxHeight: 280, overflowY: "auto" }}>
999       {editando.lineas.map((l, i) => (
1000         <div key={i} style={{ display: "flex", gap: 8, alignItems: "center", marginBottom: 8 }}>
1001           <select value={l.articulo_codigo}
1002             onChange={(e) => actualizarLinea(i, "articulo_codigo", e.target.value)}
1003             aria-label={`Artículo del ingrediente ${i + 1}`}
1004             style={{ flex: 1, padding: "8px 10px", border: "1px solid var(--borde)",
1005               borderRadius: 8, background: "#fff" }}>
1006             <option value="">— elija un artículo del catálogo —</option>
1007             {catalogo.map((a) => (
1008               <option key={a.codigo} value={a.codigo}>{a.nombre} ({a.unidad})</option>
1009             ))}
1010           </select>
1011           <input type="number" step="0.001" min="0" placeholder="cantidad"
1012             value={l.cantidad_por_porcion}
1013             onChange={(e) => actualizarLinea(i, "cantidad_por_porcion", e.target.value)}
1014             aria-label={`Cantidad por porción del ingrediente ${i + 1}`}
1015             style={{ width: 100, padding: "8px 10px", border: "1px solid var(--borde)",
1016               borderRadius: 8, background: "#fff" }} />
1017           <button className="btn gris" style={{ padding: "6px 10px", minHeight: 0, flex: "none" }}
1018             onClick={() => quitarLinea(i)} aria-label={`Quitar ingrediente ${i + 1}`}>X</button>
1019         </div>
1020       ))}
1021     </div>
1022     <button className="btn borde" style={{ marginTop: 4 }} onClick={agregarLinea}>
1023       + Agregar ingrediente
1024     </button>
1025   </div>
1026
1027   <div style={{ background: "var(--fondo)", borderRadius: 12, padding: 16, marginBottom: 4 }}>
1028     <h3 style={{ marginTop: 0, marginBottom: 4, fontSize: ".95rem", color: "var(--azul)" }}>
1029       🍳 Preparación paso a paso
1030     </h3>
1031     <p className="pista" style={{ marginBottom: 12 }}>
1032       Opcional: lo que ve el auxiliar al consultar esta receta desde Pedidos.
1033     </p>
1034     <textarea value={editando.preparacion}
1035       onChange={(e) => setEditando({ ...editando, preparacion: e.target.value })}
1036       placeholder="1. Pele y pique las papas...&#10;2. Sofría la cebolla...&#10;3. ..."
1037       aria-label="Preparación paso a paso, opcional"
1038       rows={5}
1039       style={{ width: "100%", padding: "10px 12px",
1040         border: "1px solid var(--borde)", borderRadius: 10,
1041         fontFamily: "inherit", resize: "vertical", background: "#fff" }} />
1042   </div>
1043
1044   <div className="botones" style={{ marginTop: 20 }}>
1045     <button className="btn borde" onClick={() => setEditando(null)}>Cancelar</button>
1046     <button className="btn" onClick={guardar}>Guardar receta</button>
1047   </div>
1048 </div>
1049 </div>
1050 }}
1051 </div>
1052 );
1053 }
1054

```

```

1055 function TabTraza({ token }) {
1056   const [traza, setTraza] = useState([]);
1057   const [persona, setPersona] = useState("");
1058   const [accion, setAccion] = useState("");
1059   const [rango, setRango] = useState("hoy");
1060   const [msg, setMsg] = useState("");
1061   const [todasPersonas, setTodasPersonas] = useState([]);
1062   const [escuchandoFiltro, setEscuchandoFiltro] = useState(false);
1063
1064   function cargar() {
1065     const q = new URLSearchParams();
1066     if (persona) q.set("persona", persona);
1067     if (accion) q.set("accion", accion);
1068     if (rango) q.set("rango", rango);
1069     pedir(`/api/trazabilidad?${q}`, {}, token).then(setTraza).catch(() => {});
1070   }
1071   useEffect(cargar, [token, persona, accion, rango]);
1072   // Lista COMPLETA de personas (no solo las que aparecen en el filtro
1073   // actual) para que el filtro por voz reconozca un nombre aunque el
1074   // rango/persona ya aplicado lo haya dejado fuera de "traza".
1075   useEffect(() => {
1076     pedir("/api/usuarios", {}, token).then((us) => setTodasPersonas(us.map((x) => x.nombre)))
1077       .catch(() => {});
1078   }, [token]);
1079
1080   const acciones = ["", "INGRESO", "APERTURA", "CONTEO", "CORRECCION", "CREACION",
1081     "FIRMA", "AUDITORIA", "CIERRE", "REAPERTURA", "APROBACION", "ALERTA",
1082     "REPORTE", "PEDIDO", "RECETA", "LEGALIZACION", "AJUSTE", "USUARIO",
1083     "ASIGNACION", "SEGURIDAD", "PERFIL", "SOPORTE"];
1084   const personas = [...new Set(traza.map((t) => t.persona))].sort();
1085
1086   // "Muéstrame las acciones de Luis hoy" - determinístico y local, sin
1087   // pasar por el agente general: solo reconoce palabras contra las
1088   // listas que YA existen en esta pantalla (rango, tipos de acción,
1089   // personas), así que no hace falta ninguna llamada a Gemini para
1090   // filtrar. Combina lo dicho con lo que YA estaba marcado (decir solo
1091   // "esta semana" no borra la persona/acción que ya tenía puesta) -
1092   // por eso lee rango/persona/accion actuales como punto de partida en
1093   // vez de siempre resetear a "todos".
1094   function _resolverFiltroVoz(texto) {
1095     const t = quitarTildes(texto.toLowerCase());
1096     let coincidio = false;
1097     let nuevoRango = rango, nuevaAccion = accion, nuevaPersona = persona;
1098     if (/^btodo\b/.test(t)) { nuevoRango = ""; coincidio = true; }
1099     else if (/^bhoy\b/.test(t)) { nuevoRango = "hoy"; coincidio = true; }
1100     else if (/^semana\b/.test(t)) { nuevoRango = "semana"; coincidio = true; }
1101     else if (/^bmes\b/.test(t)) { nuevoRango = "mes"; coincidio = true; }
1102
1103     if (/^todas las acciones\b/.test(t)) { nuevaAccion = ""; coincidio = true; }
1104     else {
1105       const accionEncontrada = acciones.find((a) => a && t.includes(a.toLowerCase()));
1106       if (accionEncontrada) { nuevaAccion = accionEncontrada; coincidio = true; }
1107     }
1108
1109     if (/^todas las personas\b/.test(t)) { nuevaPersona = ""; coincidio = true; }
1110     else {
1111       const personaEncontrada = todasPersonas.find(
1112         (p) => p && t.includes(quitarTildes(p.toLowerCase())));
1113       if (personaEncontrada) { nuevaPersona = personaEncontrada; coincidio = true; }
1114     }
1115     return coincidio ? { rango: nuevoRango, persona: nuevaPersona, accion: nuevaAccion } : null;
1116   }
1117
1118   // Aplica el filtro resuelto Y dice el resultado real (cuántas filas
1119   // encontró), no un "listo, apliqué el filtro" que no cuenta nada -
1120   // pedido explícito del usuario. Trae los datos de una (en vez de
1121   // esperar a que el useEffect de cargar() reaccione al cambio de
1122   // estado) para poder anunciar la cifra exacta apenas llega.
1123   async function _aplicarFiltroYAnunciar(resuelto) {
1124     const { rango: r, persona: p, accion: a } = resuelto;
1125     setRango(r); setPersona(p); setAccion(a);
1126     setMsg("");
1127     try {
1128       const q = new URLSearchParams();

```

```

1129     if (p) q.set("persona", p);
1130     if (a) q.set("accion", a);
1131     if (r) q.set("rango", r);
1132     const filas = await pedir(`/api/trazabilidad?${q}`, {}, token);
1133     setTraza(filas);
1134     const partes = [];
1135     if (a) partes.push(`de ${a.toLowerCase()}`);
1136     if (p) partes.push(`de ${p}`);
1137     const etiquetaRango = r === "hoy" ? "hoy" : r === "semana" ? "en la última semana"
1138       : r === "mes" ? "en el último mes" : "en total";
1139     const texto = `Encontré ${filas.length} acciones${partes.length ? " " + partes.join(" ") : ""}
${etiquetaRango}.`;
1140     setMsg(texto);
1141     hablar(texto);
1142   } catch (e) { setMsg(e.message); hablar(e.message); }
1143 }
1144
1145 // El cuadro "Pregúntele al agente" (arriba de las pestañas) es el
1146 // MISMO cuadro que muestra los ejemplos «acciones de Luis hoy» - una
1147 // persona que escribe/dice esa frase ahí espera que funcione, no
1148 // solo desde el micrófono chiquito junto a los filtros. Como el
1149 // filtro ya vive aquí (sin pasar por el backend), se intercepta la
1150 // frase ANTES de que AsistenteVoz llegue a preguntarle al agente -
1151 // event.preventDefault() debe llamarse de una, ANTES de cualquier
1152 // await, para que AsistenteVoz vea la señal en el mismo ciclo
1153 // síncrono del dispatchEvent (el fetch+anuncio siguen aparte, ya de
1154 // forma asíncrona, sin que eso afecte esa señal).
1155 useEffect(() => {
1156   const interceptar = (e) => {
1157     const resuelto = _resolverFiltroVoz(e.detail.texto);
1158     if (!resuelto) return;
1159     e.preventDefault();
1160     _aplicarFiltroYAnunciar(resuelto);
1161   };
1162   window.addEventListener("cuentavoz:filtro-local", interceptar);
1163   return () => window.removeEventListener("cuentavoz:filtro-local", interceptar);
1164 }, [todasPersonas, acciones, token, rango, persona, accion]);
1165
1166 function escucharFiltro() {
1167   setMsg("");
1168   escuchar({
1169     alTexto: (texto) => {
1170       const resuelto = _resolverFiltroVoz(texto);
1171       if (resuelto) _aplicarFiltroYAnunciar(resuelto);
1172       else setMsg("No reconocí ningún filtro en lo que dijo.");
1173     },
1174     alEstado: (e) => setEscuchandoFiltro(e === "escuchando"),
1175     alError: setMsg,
1176   });
1177 }
1178
1179 async function exportar() {
1180   setMsg("");
1181   try {
1182     const q = new URLSearchParams();
1183     if (persona) q.set("persona", persona);
1184     if (accion) q.set("accion", accion);
1185     if (rango) q.set("rango", rango);
1186     const d = await pedir(`/api/trazabilidad/exportar?${q}`, { method: "GET" }, token);
1187     await descargarReporte(d.archivo, token);
1188     setMsg(`Exportado: ${d.filas} filas.`);
1189   } catch (e) { setMsg(e.message); }
1190 }
1191
1192 // "exporta el registro de trazabilidad" por voz - mismo botón
1193 // «Exportar», con los filtros que ya estén puestos en pantalla (el
1194 // agente no los conoce, viven aquí como estado de React).
1195 useEffect(() => {
1196   window.addEventListener("cuentavoz:accion:exportar_trazabilidad", exportar);
1197   return () => window.removeEventListener("cuentavoz:accion:exportar_trazabilidad", exportar);
1198 }, [token, persona, accion, rango]);
1199
1200 return (
1201   <div className="card">

```

```

1202 <h2><span className="icono-kpi" style={{ marginRight: 8 }}></span>
1203 Registro de trazabilidad ({traza.length} acciones)</h2>
1204 <div style={{ display: "flex", gap: 10, marginBottom: 12, flexWrap: "wrap", alignItems: "center" }}>
1205   <select value={rango} onChange={(e) => setRango(e.target.value)}
1206     aria-label="Rango de fechas del registro de trazabilidad"
1207     style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
1208     <option value="hoy">Hoy</option>
1209     <option value="semana">Última semana</option>
1210     <option value="mes">Último mes</option>
1211     <option value="">Todo</option>
1212   </select>
1213   <select value={persona} onChange={(e) => setPersona(e.target.value)}
1214     aria-label="Filtrar por persona"
1215     style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
1216     <option value="">Todas las personas</option>
1217     {personas.map((p) => <option key={p} value={p} style={{ textTransform: "capitalize" }}>{p}</option>)}
1218   </select>
1219   <select value={accion} onChange={(e) => setAccion(e.target.value)}
1220     aria-label="Filtrar por tipo de acción"
1221     style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
1222     {acciones.map((a) => <option key={a} value={a}>{a} | "Todas las acciones"</option>)}
1223   </select>
1224   <button className={`mic-btn ${escuchandoFiltro ? "escuchando" : ""}`}
1225     style={{ width: 46, height: 46, fontSize: "1.2rem" }}
1226     onClick={escucharFiltro} title="filtrar por voz: «acciones de Luis hoy»"
1227     aria-label="Filtrar el registro de trazabilidad por voz">
1228     <Icono nombre="microfono" tam={20} /></button>
1229   <button className="btn verde"
1230     style={{ marginLeft: "auto", display: "inline-flex", alignItems: "center", gap: 8 }}
1231     onClick={exportar}>
1232     <Icono nombre="descargar" tam={18} />
1233     Exportar
1234   </button>
1235 </div>
1236 {msg && (
1237   <div className={`banner ${msg.startsWith("Encontré") || msg.startsWith("Exportado") ? "ok" : ""}`}
1238     style={{ marginBottom: 12 }}>
1239     <span className="ico">
1240       {msg.startsWith("Encontré") || msg.startsWith("Exportado") ? "√" : "!"}
1241     </span>
1242     <span>{msg}</span>
1243   </div>
1244 )}
1245 {traza.length === 0 ? (
1246   <p className="vacio">Sin acciones registradas todavía.</p>
1247 ) : (
1248   <table>
1249     <thead><tr><th>Hora</th><th>Persona</th><th>Acción</th><th>Detalle</th></tr></thead>
1250     <tbody>
1251       {traza.slice(0, 25).map((t) => (
1252         <tr key={t.id}>
1253           <td>{t.hora}</td>
1254           <td style={{ textTransform: "capitalize" }}>{t.persona}</td>
1255           <td><span className={`chip ${t.tipo === "alerta" ? "oro"
1256             : t.tipo === "ok" ? "verde" : ""}`}>{t.accion}</span></td>
1257           <td>{t.detalle}</td>
1258         </tr>
1259       ))}
1260     </tbody>
1261   </table>
1262 )}
1263 <p className="pista" style={{ marginTop: 10 }}>
1264   Cumple la Ley 1581 de 2012: se registra la acción, no datos personales sensibles.
1265   El registro no se puede editar ni borrar.
1266 </p>
1267 <div className="card" style={{ marginTop: 14, background: "var(--azul-claro)" }}>
1268   <h2>Por qué el registro es inmutable</h2>
1269   <p style={{ fontSize: ".87rem" }}>
1270     Una corrección no borra el valor anterior: crea un registro nuevo que apunta
1271     al original (campo <code>corrige_a</code> de la tabla Conteo). Así, si alguien
1272     pregunta «¿quién cambió este dato y por qué?», la respuesta está en el
1273     sistema y no en la memoria de nadie.
1274   </p>
1275 </div>

```

```

1276     </div>
1277   );
1278 }

```

frontend/src/vistas/Ayuda.jsx

364 LÍNEAS

Preguntas, agente y soporte.

```

1 import { useEffect, useState, useRef } from "react";
2 import { pedir, preguntarAsistente } from "../api";
3 import { EVENTO_ABRIR_VIDEO } from "../tutorial";
4 import { escuchar, hablar, quitarTildes } from "../voz";
5 import { enviarPorEmailJS, capitalizar, rolDe } from "../correoAdmin";
6 import Marco from "../Marco";
7 import Dialogo from "../Dialogo";
8 import Icono from "../Iconos";
9
10 const FAQ = [
11   ["¿Cómo corrijo un conteo ya confirmado?",
12     "Diga «corregir» y luego el valor correcto. El valor anterior se conserva."],
13   ["El agente no me entiende un producto",
14     "Dígalo como aparece en la etiqueta. Si no existe, se puede crear: queda pendiente."],
15   ["¿Qué hago si sale una alerta?",
16     "Recuente. Si el número es correcto, confirme: queda marcado para el administrador."],
17   ["Se cayó el internet en plena bodega",
18     "Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal."],
19   ["¿Puedo contar dos bodegas a la vez?",
20     "No en el mismo dispositivo. El candado de sesión evita conteos duplicados."],
21 ];
22
23 const COMANDOS = [
24   ["«Iniciar conteo en almacén de suministros», "abrir una bodega"],
25   ["«Tres tablas para picar blancas», "registrar un conteo"],
26   ["«Corregir» · «Son nueve», "corregir sin borrar el original"],
27   ["«¿Cuánto arroz hay y en qué bodegas?»", "consultar el inventario"],
28   ["«Hoy preparamos cincuenta ajiacos», "pedir insumos por receta"],
29 ];
30
31 export default function Ayuda({ token, usuario, ir }) {
32   const [salud, setSalud] = useState(null);
33   const [busca, setBusca] = useState("");
34   const [reportando, setReportando] = useState(false);
35   const [msg, setMsg] = useState("");
36   const [pedirDetalle, setPedirDetalle] = useState(false);
37   const [escribiendoAdmin, setEscribiendoAdmin] = useState(false);
38   const [enviandoAdmin, setEnviandoAdmin] = useState(false);
39   const [admin, setAdmin] = useState(null);
40   const [miPerfil, setMiPerfil] = useState(null);
41   // Si lo escrito/dicho no encuentra nada en las preguntas frecuentes ni
42   // en la guía de comandos, la respuesta del agente general - mismo
43   // patrón que el buscador de Bodegas: un solo cuadro para todo, en vez
44   // de dos cuadros separados (uno para el agente, otro para filtrar)
45   // que además se veían mal juntos, uno encima del otro.
46   const [respuestaAgente, setRespuestaAgente] = useState(null);
47   const [escuchando, setEscuchando] = useState(false);
48   const [errorBusca, setErrorBusca] = useState("");
49   const rec = useRef(null);
50   const idEscucha = useRef(0);
51
52   useEffect(() => {
53     pedir("/api/salud", {}, token).then(setSalud).catch(() => setSalud({ api: "caido" }));
54     pedir("/api/soporte/administrador", {}, token).then(setAdmin).catch(() => {});
55     // Para la firma del correo a el administrador (nombre, rol y correo
56     // de quien escribe) - "usuario" (la sesión) solo trae id/nombre/perfil.
57     pedir("/api/usuarios/yo", {}, token).then(setMiPerfil).catch(() => {});
58   }, [token]);
59
60   async function reportarProblema(detalle) {
61     setPedirDetalle(false);
62     if (!detalle || !detalle.trim()) return;
63     setReportando(true);
64     try {

```

```

65     await pedir("/api/soporte/reportar", {
66         method: "POST", body: JSON.stringify({ detalle }),
67     }, token);
68     const listo = "Reportado a la mesa de ayuda. Quedó en el registro de trazabilidad.";
69     setMsg(listo);
70     hablar(listo);
71 } catch (e) { setMsg(e.message); hablar(e.message); }
72 setReportando(false);
73 }
74
75 // Reemplaza el enlace mailto: de antes - dependía de que el equipo
76 // tuviera un cliente de correo instalado, y sin uno mostraba el cuadro
77 // genérico del sistema operativo ("seleccionar una aplicación para
78 // abrir este vínculo") en vez de algo propio de CuentaVoz. Este envío
79 // queda en el registro de trazabilidad, con el mismo aviso honesto que
80 // ya usa "Reportar un problema": no promete un correo real que la app
81 // no puede garantizar.
82 async function enviarMensajeAdministrador(mensaje) {
83     setEscribiendoAdmin(false);
84     if (!mensaje || !mensaje.trim()) return;
85     setEnviandoAdmin(true);
86     try {
87         const r = await pedir("/api/soporte/mensaje-administrador", {
88             method: "POST", body: JSON.stringify({ mensaje }),
89         }, token);
90         const nombre = capitalizar(admin?.nombre) || "el administrador";
91         const rol = rolDe(usuario);
92         let listo;
93         if (r?.correo_enviado) {
94             listo = `Correo enviado a ${nombre}.`;
95         } else if (admin?.correo &&
96             await enviarPorEmailJS(admin.correo, capitalizar(usuario?.nombre) || "Usuario",
97                 mensaje, rol, miPerfil?.correo)) {
98             listo = `Correo enviado a ${nombre}.`;
99         } else if (admin?.correo) {
100             // Último respaldo si EmailJS tampoco responde: abre el correo ya
101             // instalado en el dispositivo con todo listo.
102             const asunto = encodeURIComponent("Mensaje desde CuentaVoz");
103             const cuerpo = encodeURIComponent(`De: ${usuario?.nombre} || "usted"\n\n${mensaje}`);
104             window.location.href = `mailto:${admin.correo}?subject=${asunto}&body=${cuerpo}`;
105             listo = `Mensaje registrado. Se abrió su correo para enviarlo a ${nombre}.`;
106         } else {
107             listo = "Mensaje registrado.";
108         }
109         setMsg(listo);
110         hablar(listo);
111     } catch (e) { setMsg(e.message); hablar(e.message); }
112     setEnviandoAdmin(false);
113 }
114
115 // Devuelve lo que hay que DECIR cuando lo preguntado ya está resuelto
116 // localmente (sin pasar por el agente general), o null si no coincide
117 // con nada. Antes solo se sabía SI coincidía (booleano) y la pantalla
118 // se limitaba a filtrar la lista visualmente, sin decir nada - quien
119 // pregunta por voz sin mirar la pantalla se quedaba sin ninguna
120 // respuesta hablada, justo lo contrario de lo que promete "pregúntele
121 // al agente" en esta misma pantalla.
122 function _respuestaLocal(texto) {
123     const t = texto.trim().toLowerCase();
124     const faq = FAQ.find(([p, r]) => p.toLowerCase().includes(t) || r.toLowerCase().includes(t));
125     if (faq) return faq[1];
126     const comando = COMANDOS.find(([c, d]) => c.toLowerCase().includes(t) || d.toLowerCase().includes(t));
127     if (comando) return `${comando[0]} sirve para ${comando[1]}.`;
128     return null;
129 }
130
131 function alEscribir(texto) {
132     setBusca(texto);
133     setRespuestaAgente(null);
134 }
135
136 // Los botones de "Soporte en vivo" ("Escribirle al administrador",
137 // "Reportar un problema") también son órdenes reconocibles - se
138 // revisan antes que cualquier otra cosa, porque el agente general no

```

```

139 // sabe qué botones hay en esta pantalla en particular (por eso decía
140 // "no puedo enviarle mensajes... esa función no está disponible", aun
141 // cuando el botón para hacerlo está ahí mismo, a la vista).
142 function _accionLocalDeAyuda(texto) {
143     const t = quitarTildes(texto.toLowerCase().replace(/[,;:!\?;|]/g, "").trim());
144     // "\bdescrib" (sin \b de cierre) para que agarre CUALQUIER conjugación
145     // - "escribir", "escribale", y sobre todo "escribirle" (como dice el
146     // botón mismo: "Escribirle al administrador") - un \b\escribir\b no
147     // hacía match dentro de "escribirle" porque ahí "escribir" no es una
148     // palabra completa, es solo el principio de una más larga.
149     if (/^\badministrador\b/.test(t) && /\b(escrib\w*|contactar|mensaje)\b/.test(t)) return "escribir";
150     if (/^\bproblema\b|\berror\b/.test(t) && /\breport\w*/.test(t)) return "reportar";
151     return null;
152 }
153
154 // Al confirmar (Enter, o al terminar de dictar): primero las órdenes
155 // sobre los botones de esta pantalla; si no aplica ninguna y lo dicho
156 // no encuentra nada en las preguntas frecuentes ni en la guía de
157 // comandos, cae al agente general - un solo cuadro para todo.
158 async function confirmarBusqueda(texto, miId) {
159     if (!texto || !texto.trim()) return;
160     if (miId === undefined) miId = ++idEscucha.current;
161     setBusca(texto);
162     const accion = _accionLocalDeAyuda(texto);
163     if (accion === "escribir" && admin) { setEscribiendoAdmin(true); return; }
164     if (accion === "reportar") { setPedirDetalle(true); return; }
165     const respuestaLocal = _respuestaLocal(texto);
166     if (respuestaLocal) {
167         setRespuestaAgente(null);
168         hablar(respuestaLocal);
169         return;
170     }
171     setErrorBusca("");
172     const r = await preguntarAsistente(texto, "ayuda", token);
173     if (miId !== idEscucha.current) return;
174     setRespuestaAgente(r);
175     hablar(r.respuesta_hablada);
176     if (r.accion === "navegar" && r.destino && ir) {
177         ir(r.destino, { tabInicial: r.pestana || undefined });
178     }
179 }
180
181 function buscarPorVoz() {
182     setErrorBusca("");
183     const miId = ++idEscucha.current;
184     rec.current = escuchar({
185         alTexto: (t) => {
186             if (miId !== idEscucha.current) return;
187             confirmarBusqueda(t, miId);
188         },
189         alEstado: (e) => setEscuchando(e === "escuchando"),
190         alError: setErrorBusca,
191     });
192 }
193
194 const q = busca.trim().toLowerCase();
195 const faqFiltrado = q ? FAQ.filter(([p, r]) =>
196     p.toLowerCase().includes(q) || r.toLowerCase().includes(q)) : FAQ;
197 const comandosFiltrados = q ? COMANDOS.filter(([c, d]) =>
198     c.toLowerCase().includes(q) || d.toLowerCase().includes(q)) : COMANDOS;
199
200 const servicios = [
201     ["API y base de datos", salud?.api === "ok"],
202     ["Datos del inventario cargados", (salud?.stock || 0) > 0],
203     ["Agente Gemini (AI Studio)", Boolean(salud?.gemini)],
204 ];
205
206 return (
207     <Marco titulo="Ayuda · cómo usar CuentaVoz" chip={{ texto: "SOPORTE", tipo: "azul" }}>
208         <div className="card">
209             <div className="grilla-botones" style={{ justifyContent: "flex-end", marginBottom: 10 }}>
210                 <button className="btn borde"
211                     onClick={() => window.dispatchEvent(new Event(EVENTO_ABRIR_VIDEO))}>
212                     Ver el recorrido en video

```



```

213     </button>
214 </div>
215 <p className="rotulo">Pregúntele al agente</p>
216 <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
217   <input value={busca} onChange={(e) => alEscribir(e.target.value)}
218     onKeyDown={(e) => e.key === "Enter" && confirmarBusqueda(busca)}
219     placeholder="pregúntele al agente, o escriba para filtrar las preguntas frecuentes..."
220     aria-label="Pregúntele al agente, o escriba para filtrar las preguntas frecuentes"
221     style={{ flex: 1, padding: "12px 14px",
222       border: "1px solid var(--borde)", borderRadius: 12 }} />
223   <button className={mic-btn ${escuchando ? "escuchando" : ""}}
224     style={{ width: 46, height: 46, fontSize: "1.2rem" }}
225     onClick={buscarPorVoz} title="o pregunte por voz"
226     aria-label="Pregúntele al agente por voz"><Icono nombre="microfono" tam={22} /></button>
227 </div>
228 <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
229 {respuestaAgente && (
230   <>
231     <p className="rotulo" style={{ marginTop: 10 }}>CuentaVoz responde</p>
232     <p className="burbuja" aria-live="polite" aria-atomic="true">{respuestaAgente.respuesta_hablada}</p>
233   </>
234 )}
235 {errorBusca && <p className="error" style={{ marginTop: 8 }}>{errorBusca}</p>}
236 </div>
237 <div className="conteo-cols">
238   <div className="card">
239     <h2><span className="icono-kpi" style={{ marginRight: 8 }}>? </span>Preguntas frecuentes</h2>
240     {faqFiltrado.length === 0 && <p className="vacio">Sin resultados para «{busca}».</p>}
241     {faqFiltrado.map(([p, r], i) => (
242       <div key={i} style={{ marginBottom: 12, paddingBottom: 12,
243         borderBottom: i < faqFiltrado.length - 1 ? "1px solid var(--borde)" : "none" }}>
244         <b style={{ color: "var(--azul)", fontSize: ".92rem" }}>{p}</b>
245         <p style={{ fontSize: ".86rem", color: "var(--grafito)", marginTop: 3 }}>{r}</p>
246       </div>
247     )))
248
249     <h3 style={{ marginTop: 18 }}>
250       <span className="icono-kpi" style={{ marginRight: 8 }}>≡ </span>
251       Guía rápida de comandos de voz
252     </h3>
253     {comandosFiltrados.length === 0 && <p className="vacio">Sin resultados para «{busca}».</p>}
254     {comandosFiltrados.map(([c, d], i) => (
255       <div className="registro" key={i}>
256         <span style={{ color: "var(--azul)", fontWeight: 700 }}>{c}</span>
257         <span className="cant">{d}</span>
258       </div>
259     )))
260 </div>
261
262 <div>
263   <div className="card">
264     <h2><span className="icono-kpi" style={{ marginRight: 8 }}>💡 </span>Soporte en vivo</h2>
265     {admin?.nombre ? (
266       <>
267         <p style={{ fontSize: ".88rem" }}>
268           Administrador de bodega · <span style={{ textTransform: "capitalize" }}>{admin.nombre}</span>
269           <span style={{ color: "var(--verde)", fontWeight: 700 }}> · disponible</span>
270         </p>
271         <div className="grilla-botones">
272           <button className="btn" disabled={enviandoAdmin}
273             onClick={() => setEscribiendoAdmin(true)}
274             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
275             <Icono nombre="mensajes" tam={18} />
276             {enviandoAdmin ? "Enviando..." : "Escribirle al administrador"}
277           </button>
278         </div>
279       </>
280     ) : admin?.es_usted ? (
281       <p style={{ fontSize: ".88rem" }}>
282         Usted es la administradora de turno: los auxiliares le escriben a
283         usted. Para algo que se salga de su alcance, use la mesa de ayuda
284         de Colsubsidio.
285       </p>
286     ) : (

```



```

287     <p style={{ fontSize: ".88rem" }}>
288         No hay otro administrador registrado por ahora. Use la mesa de
289         ayuda de Colsubsidio.
290     </p>
291 })
292 {msg && <p className="msg-ok">{msg}</p>}
293 <p className="pista" style={{ margin: "10px 0" }}>
294     Mesa de ayuda Colsubsidio · ext. 4040 · 7 por 24
295 </p>
296 <div className="grilla-botones">
297     {(admin?.nombre || admin?.es_usted) && (
298         <button className="btn borde" onClick={() => ir("mensajes")}
299             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
300             <Icono nombre="mensajes" tam={18} />
301             Ver mensajes
302         </button>
303     )}
304     <button className="btn oro" disabled={reportando} onClick={() => setPedirDetalle(true)}
305         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
306         <Icono nombre="advertencia" tam={18} />
307         {reportando ? "Enviando..." : "Reportar un problema"}
308     </button>
309 </div>
310 </div>
311
312 <div className="card" style={{ marginTop: 16 }}>
313     <h2><span className="icono-kpi" style={{ marginRight: 8 }}><img alt="icono-kpi" data-bbox="595 370 615 385"/></span>Estado del sistema</h2>
314     <div className="kpis" style={{ gridTemplateColumns: "1fr" }}>
315         {servicios.map([t, ok], i) => (
316             <div className={ok ? "kpi verde" : "kpi"} key={i}
317                 style={{ display: "flex", alignItems: "center", justifyContent: "space-between",
318                     padding: "10px 16px" }}>
319                 <div className="kpi-cabeza" style={{ marginBottom: 0 }}>
320                     <span className="icono-kpi" style={{
321                         background: ok ? "var(--verde-bg)" : "#FBE8E6",
322                         color: ok ? "var(--verde)" : "#B3261E" }}>
323                         {ok ? "✓" : "!"}
324                     </span>
325                     <small>{t}</small>
326                 </div>
327                 <b style={{ fontSize: ".92rem", color: ok ? "var(--verde)" : "#B3261E" }}>
328                     {ok ? "operativo" : "revisar"}
329                 </b>
330             </div>
331         )}
332     </div>
333     {!salud?.gemini && (
334         <p className="pista" style={{ marginTop: 10 }}>
335             Sin la llave de Gemini el agente usa el intérprete local: el flujo
336             funciona igual, pero entiende menos variantes de frase. Ponga
337             GOOGLE_API_KEY en el archivo .env para activarlo.
338         </p>
339     )}
340 </div>
341 </div>
342 </div>
343
344 {pedirDetalle && (
345     <Dialogo titulo="Reportar un problema"
346         mensaje="Describa el problema que encontró:"
347         conCampo conVoz multilinea placeholder="Se cayó el micrófono al confirmar un conteo..."
348         textoAceptar="Enviar"
349         onAceptar={reportarProblema}
350         onCancel={() => setPedirDetalle(false)} />
351 )}
352
353 {escribiendoAdmin && (
354     <Dialogo titulo="Escribirle al administrador"
355         mensaje={`Para: ${capitalizar(admin?.nombre) || "Administrador"}\nDe: ${capitalizar(usuario?.nombre)
|| "Usted"}\nAsunto: Mensaje desde CuentaVoz\n\nEscriba su mensaje:`}
356         conCampo conVoz multilinea placeholder="Necesito que me asigne la bodega de Kiosco Taquilla..."
357         etiquetaCampo={`Su mensaje para ${capitalizar(admin?.nombre) || "el administrador"}`}
358         textoAceptar="Enviar"
359         onAceptar={enviarMensajeAdministrador}

```

```

360         onCancel={ () => setEscribiendoAdmin(false) } />
361     )}
362 </Marco>
363 );
364 }

```

frontend/src/vistas/Mensajes.jsx

248 LÍNEAS

La bandeja del equipo.

```

1 import { useEffect, useRef, useState } from "react";
2 import { pedir } from "../api";
3 import { escuchar, hablar, quitarTildes } from "../voz";
4 import { enviarPorEmailJS, capitalizar, rolDe } from "../correoAdmin";
5 import Marco from "../Marco";
6 import Dialogo from "../Dialogo";
7 import Icono from "../Iconos";
8
9 /** La bandeja completa de "Soporte en vivo": para un auxiliar, lo que ha
10 escrito y si ya le respondieron; para el administrador, lo que le
11 han escrito y lo que falta por responder. Antes esta lista vivía
12 dentro de la tarjeta de Ayuda, pero con varios auxiliares escribiendo
13 (Luis, Valentina...) esa tarjeta se llenaba y quedaba desordenada -
14 aquí tiene su propio espacio, como Auditoría o Reportes. */
15 export default function Mensajes({ token, usuario }) {
16     const [mensajes, setMensajes] = useState(null);
17     const [miPerfil, setMiPerfil] = useState(null);
18     const [respondiendoId, setRespondiendoId] = useState(null);
19     const [respuestaPrellenada, setRespuestaPrellenada] = useState("");
20     const [enviandoRespuesta, setEnviandoRespuesta] = useState(false);
21     const [msg, setMsg] = useState("");
22     const [escuchandoOrden, setEscuchandoOrden] = useState(false);
23     const [ordenVoz, setOrdenVoz] = useState("");
24     const idEscuchaOrden = useRef(0);
25     const esAdmin = usuario?.perfil === "auditor";
26
27     useEffect(() => {
28         cargar();
29         pedir("/api/usuarios/yo", {}, token).then(setMiPerfil).catch(() => {});
30     }, [token]);
31
32     function cargar() {
33         pedir("/api/soporte/mensajes", {}, token).then(setMensajes).catch(() => setMensajes([]));
34     }
35
36     async function responderMensaje(respuesta) {
37         const id = respondiendoId;
38         setRespondiendoId(null);
39         setRespuestaPrellenada("");
40         if (!respuesta || !respuesta.trim() || !id) return;
41         setEnviandoRespuesta(true);
42         try {
43             const r = await pedir(`/api/soporte/mensajes/${id}/responder`, {
44                 method: "POST", body: JSON.stringify({ respuesta }),
45             }, token);
46             if (r?.correo_destinatario) {
47                 await enviarPorEmailJS(r.correo_destinatario, capitalizar(usuario?.nombre) || "Usuario",
48                     respuesta, rolDe(usuario), miPerfil?.correo);
49             }
50             const listo = `Respuesta enviada a ${capitalizar(r?.destinatario) || "la persona"}`;
51             setMsg(listo);
52             hablar(listo);
53             cargar();
54         } catch (e) { setMsg(e.message); hablar(e.message); }
55         setEnviandoRespuesta(false);
56     }
57
58     // "Respóndele a Stephanie que ya quedó asignada la bodega" - con un
59     // solo mensaje pendiente no hace falta nombrar a nadie, lo dicho es
60     // directamente la respuesta. Con varios, busca el nombre de quien
61     // escribió dentro de la frase (mismo patrón de emparejar por nombre
62     // que ya usa el resto de la app) y separa el resto como el texto de

```

```

63 // la respuesta.
64 function _extraerRespuestaPorVoz(texto, listaPendientes) {
65   if (listaPendientes.length === 1) {
66     return { mensaje: listaPendientes[0], texto: texto.trim() };
67   }
68   const t = quitarTildes(texto.toLowerCase());
69   const encontrado = listaPendientes.find(
70     (m) => m.de && t.includes(quitarTildes(m.de.toLowerCase()));
71   if (!encontrado) return { mensaje: null, texto: null };
72   // el nombre de quien escribió no tiene restricción de caracteres al
73   // crear el usuario en Ajustes - sin escapar los que son especiales en
74   // una expresión regular (paréntesis, corchetes...), un nombre con uno
75   // sin su pareja ("Juan (temporal)" tumbaba esto con una excepción sin
76   // capturar y la respuesta por voz se quedaba sin funcionar, sin
77   // ningún aviso de qué pasó.
78   const nombreEscapado = encontrado.de.replace(/[\.\*\?\^\$\{\}\|\[\]\\\]/g, "\\$&");
79   const patron = new RegExp(`^.*?\b${nombreEscapado}\b\s*(que|:)?\s*`, "i");
80   const textoFinal = texto.replace(patron, "").trim() || texto.trim();
81   return { mensaje: encontrado, texto: textoFinal };
82 }
83
84 function alTextoOrdenVoz(texto, miId) {
85   if (miId !== idEscuchaOrden.current || !texto) return;
86   setOrdenVoz(texto);
87   const pendientesLista = (mensajes || []).filter((m) => !m.respuesta);
88   if (pendientesLista.length === 0) return;
89   const { mensaje, texto: respuestaTexto } = _extraerRespuestaPorVoz(texto, pendientesLista);
90   if (!mensaje) {
91     const nombres = pendientesLista.map((m) => capitalizar(m.de)).join(", ");
92     const aviso = `Hay varios mensajes pendientes: ${nombres}. Diga el nombre de quién le escribió.`;
93     setMsg(aviso);
94     hablar(aviso);
95     return;
96   }
97   setRespuestaPrellenada(respuestaTexto);
98   setRespondiendoId(mensaje.id);
99   setOrdenVoz("");
100 }
101
102 function escucharOrdenVoz() {
103   setMsg("");
104   const miId = ++idEscuchaOrden.current;
105   escuchar({
106     alTexto: (t) => alTextoOrdenVoz(t, miId),
107     alEstado: (e) => setEscuchandoOrden(e === "escuchando"),
108     alError: setMsg,
109   });
110 }
111
112 const original = mensajes?.find((m) => m.id === respondiendoId);
113 const pendientes = mensajes?.filter((m) => !m.respuesta).length ?? 0;
114 const respondidos = mensajes?.filter((m) => m.respuesta).length ?? 0;
115
116 return (
117   <Marco titulo="Mensajes · soporte en vivo"
118     chip={{ texto: esAdmin ? "BANDEJA DEL ADMINISTRADOR" : "MIS MENSAJES", tipo: "azul" }}>
119     {mensajes !== null && mensajes.length > 0 && (
120       <div className="kpi" style={{ marginBottom: 16 }}>
121         <div className="kpi">
122           <div className="kpi-cabeza">
123             <span className="icono-kpi">  </span>
124             <small>{esAdmin ? "Mensajes recibidos" : "Mensajes enviados"}</small>
125           </div>
126           <b>{mensajes.length}</b>
127           <i>en total</i>
128         </div>
129         <div className={`kpi ${pendientes ? "oro" : "verde"}`>
130           <div className="kpi-cabeza">
131             <span className="icono-kpi">{pendientes ? "🔔" : "✅"}</span>
132             <small>Pendientes de respuesta</small>
133           </div>
134           <b>{pendientes}</b>
135           <i>{esAdmin ? "por responder" : "esperando al administrador"}</i>
136         </div>

```

```

137     <div className="kpi verde">
138       <div className="kpi-cabeza">
139         <span className="icono-kpi">📞</span>
140         <small>Respondidos</small>
141       </div>
142       <b>{respondidos}</b>
143       <i>ya resueltos</i>
144     </div>
145   </div>
146 })
147
148 {esAdmin && pendientes > 0 && (
149   <div className="card" style={{ marginBottom: 16 }}>
150     <h2>Responder por voz</h2>
151     <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
152       <input value={ordenVoz} onChange={(e) => setOrdenVoz(e.target.value)}
153         onKeyDown={(e) => e.key === "Enter" && alTextoOrdenVoz(ordenVoz, idEscuchaOrden.current)}
154         placeholder={pendientes === 1
155           ? "Diga o escriba la respuesta..."
156           : "«Respóndele a Stephanie que ya quedó asignada la bodega...»"}
157         aria-label="Responder por voz o escribiendo"
158         style={{ flex: 1, padding: "12px 14px",
159           border: "1px solid var(--borde)", borderRadius: 12 }} />
160       <button className={`mic-btn ${escuchandoOrden ? "escuchando" : ""}`}
161         style={{ width: 46, height: 46, fontSize: "1.2rem" }}
162         onClick={escucharOrdenVoz} title="responder por voz"
163         aria-label="Responder por voz"><Icono nombre="microfono" tam={22} /></button>
164     </div>
165     <p className="pista" style={{ marginTop: 8 }}>
166       {pendientes === 1
167         ? "Un solo mensaje pendiente: lo que diga se usa directo como respuesta."
168         : "Varios mensajes pendientes: diga primero el nombre de quién le escribió."}
169     </p>
170   </div>
171 )}
172
173 {msg && <p className="msg-ok" style={{ marginBottom: 10 }}>{msg}</p>}
174
175 {mensajes === null ? (
176   <div className="card"><p className="pista">Cargando...</p></div>
177 ) : mensajes.length === 0 ? (
178   <div className="card">
179     <p className="vacio">
180       {esAdmin ? "Nadie le ha escrito todavía." : "No ha escrito ningún mensaje todavía."}
181     </p>
182   </div>
183 ) : (
184   mensajes.map((m) => (
185     <div className="card" key={m.id} style={{ marginBottom: 12 }}>
186       <div style={{ display: "flex", alignItems: "flex-start", gap: 12 }}>
187         <span className="avatar-chico" style={{ marginTop: 2 }}>
188           {(esAdmin ? m.de : m.para)?.[0]?.toUpperCase() || "?"}
189         </span>
190         <div style={{ flex: 1, minWidth: 0 }}>
191           <div style={{ display: "flex", justifyContent: "space-between",
192             alignItems: "center", gap: 10, flexWrap: "wrap" }}>
193             <strong style={{ textTransform: "capitalize", color: "var(--azul-prof)" }}>
194               {esAdmin ? capitalizar(m.de) : `Para ${capitalizar(m.para)}`}
195             </strong>
196             <span style={{ display: "flex", alignItems: "center", gap: 10 }}>
197               <span className={`etiqueta-actividad ${m.respuesta ? "verde" : "oro"}`}
198                 style={{ marginLeft: 0 }}>
199                 {m.respuesta ? "respondido" : "pendiente"}
200               </span>
201               <span className="pista">{m.creado}</span>
202             </span>
203           </div>
204           <p style={{ fontSize: ".92rem", marginTop: 6, lineHeight: 1.5 }}>{m.mensaje}</p>
205
206           {m.respuesta ? (
207             <div style={{ display: "flex", alignItems: "flex-start", gap: 10,
208               marginTop: 12, padding: 12, border: "1px solid var(--borde)" }}>
209               <span className="avatar-chico" style={{ background: "var(--verde-bg)", color: "var(--verde)" }}>
210                 {(esAdmin ? usuario?.nombre : m.para)?.[0]?.toUpperCase() || "?"}

```

```

211         </span>
212         <div style={{ flex: 1 }}>
213             <div style={{ display: "flex", justifyContent: "space-between", alignItems: "baseline" }}>
214                 <strong style={{ color: "var(--verde)", fontSize: ".85rem" }}>
215                     {esAdmin ? "Su respuesta" : `Respuesta de ${capitalizar(m.para)}`}
216                 </strong>
217                 <span className="pista">{m.respondido}</span>
218             </div>
219             <p style={{ fontSize: ".9rem", marginTop: 3, lineHeight: 1.5 }}>{m.respuesta}</p>
220         </div>
221     </div>
222     ) : esAdmin ? (
223         <button className="btn borde" disabled={enviandoRespuesta}
224             style={{ marginTop: 12, fontSize: ".85rem", padding: "6px 16px" }}
225             onClick={() => { setRespuestaPrellenada(""); setRespondiendoId(m.id); }}>
226             Responder
227         </button>
228     ) : null}
229 </div>
230 </div>
231 </div>
232 ))
233 }}
234
235 {respondiendoId && (
236     <Dialogo titulo="Responder mensaje"
237         mensaje={`Para: ${capitalizar(original?.de) || "la persona"}\nDe: ${capitalizar(usuario?.nombre) ||
"Usted"}\n\nMensaje original:\n${original?.mensaje || ""}\n\nEscriba su respuesta:`}
238         conCampo conVoz multilinea placeholder="Ya quedó asignada esa bodega, revise en unos minutos..."
239         etiquetaCampo={`Su respuesta a ${capitalizar(original?.de) || "la persona"}`}
240         valorInicial={respuestaPrellenada}
241         confirmarAlAbrir={Boolean(respuestaPrellenada)}
242         textoAceptar="Enviar"
243         onAceptar={responderMensaje}
244         onCancel={() => { setRespondiendoId(null); setRespuestaPrellenada(""); }} />
245     )}
246 </Marco>
247 );
248 }

```

frontend/src/vistas/MiPerfil.jsx

429 LÍNEAS

La cuenta propia.

```

1 import { useEffect, useState } from "react";
2 import { pedir, BASE, leerToken, borrarSesion, esFalloRed } from "../api";
3 import { cambiarClave, cerrarSesionLocal, tieneMFAActiva, desactivarMFA } from "../cognito";
4 import { configurarVoz, hablar } from "../voz";
5 import { leerPreferencias, guardarPreferencias } from "../accesibilidad";
6 import ChecklistClave, { claveCumplePolitica } from "../ChecklistClave";
7 import ConfigurarMFA from "../ConfigurarMFA";
8 import Marco from "../Marco";
9 import Dialogo from "../Dialogo";
10 import AsistenteVoz from "../AsistenteVoz";
11
12 function formatoAcceso(fechaStr) {
13     if (!fechaStr) return "-";
14     const [fecha, hora] = fechaStr.split(" ");
15     const hoy = new Date().toISOString().slice(0, 10);
16     return fecha === hoy ? `hoy ${hora}` : `${fecha.slice(5).replace("-", "/")} ${hora}`;
17 }
18
19 /* — Interruptor simple, sin librería (mismo patrón que Ajustes.jsx) — */
20 function Interruptor({ activo, onChange, etiqueta }) {
21     return (
22         <button
23             onClick={() => onChange(!activo)}
24             role="switch"
25             aria-checked={activo}
26             aria-label={etiqueta}
27             style={{
28                 width: 44, height: 24, borderRadius: 14, padding: 2,

```

```

29     background: activo ? "var(--verde)" : "#C3CAD6",
30     display: "flex", justifyContent: activo ? "flex-end" : "flex-start",
31   }}
32   >
33   <span style={{ width: 20, height: 20, borderRadius: "50%", background: "#fff" }} />
34   </button>
35   );
36 }
37
38 // Lo que YA se puede cambiar por voz aquí (la voz, la velocidad, la
39 // confirmación hablada, y preguntar por los datos de la cuenta). La clave
40 // y la foto son a propósito solo manuales, por seguridad - el agente lo
41 // explica si se le pide, en vez de fingir que puede hacerlo.
42 const _EJEMPLOS_PERFIL = [
43   "cambia mi voz a puck", "usa la voz aoe", "habla más lento", "velocidad normal",
44   "activa la confirmación hablada", "desactiva la confirmación hablada",
45   "¿cuándo fue mi último acceso?", "¿cuántos días le quedan a mi clave?",
46   "¿qué bodegas tengo asignadas?",
47 ];
48
49 export default function MiPerfil({ token, ir }) {
50   const [datos, setDatos] = useState(null);
51   const [bodegas, setBodegas] = useState([]);
52   const [msg, setMsg] = useState("");
53   const [claveActual, setClaveActual] = useState("");
54   const [claveNueva, setClaveNueva] = useState("");
55   const [claveNueva2, setClaveNueva2] = useState("");
56   const [err, setErr] = useState("");
57   const [fotoUrl, setFotoUrl] = useState(null);
58   const [confirmarCierre, setConfirmarCierre] = useState(false);
59   const [voces, setVoces] = useState([]);
60   // verificación en dos pasos (MFA con app autenticadora)
61   const [mfaActiva, setMfaActiva] = useState(null); // null = todavía no se sabe
62   const [configurandoMFA, setConfigurandoMFA] = useState(false);
63   const [confirmarQuitarMFA, setConfirmarQuitarMFA] = useState(false);
64   // Alto contraste / tamaño de letra - preferencia de este equipo
65   // (localStorage), no de la cuenta, así que se aplica al toque, sin
66   // pasar por "Guardar cambios" del resto de la pantalla.
67   const [accesibilidad, setAccesibilidad] = useState(() => leerPreferencias());
68   function cambiarAccesibilidad(cambios) {
69     const nuevas = { ...accesibilidad, ...cambios };
70     setAccesibilidad(nuevas);
71     guardarPreferencias(nuevas);
72   }
73
74   // <img src> no puede mandar el header Authorization, y /foto exige
75   // sesion - por eso se trae como blob autenticado (mismo patron que
76   // descargarReporte) en vez de apuntar la imagen directo al endpoint.
77   function cargarFoto() {
78     fetch(`${BASE}/api/usuarios/yo/foto`, {
79       headers: { Authorization: `Bearer ${token || leerToken()}` },
80     }).then((r) => (r.ok ? r.blob() : null))
81       .then((b) => setFotoUrl((prev) => {
82         if (prev) URL.revokeObjectURL(prev);
83         return b ? URL.createObjectURL(b) : null;
84       }))
85       .catch(() => setFotoUrl(null));
86   }
87
88   useEffect(() => {
89     pedir("/api/usuarios/yo", {}, token).then(setDatos).catch(() => {});
90     pedir("/api/usuarios/yo/bodegas", {}, token).then(setBodegas).catch(() => {});
91     pedir("/api/voz/voces", {}, token).then((r) => setVoces(r.voces)).catch(() => {});
92     tieneMFAActiva().then(setMfaActiva).catch(() => setMfaActiva(false));
93     cargarFoto();
94     // eslint-disable-next-line react-hooks/exhaustive-deps
95   }, [token]);
96
97   async function guardar() {
98     setErr(""); setMsg("");
99     try {
100       await pedir("/api/usuarios/yo", { method: "PUT", body: JSON.stringify(datos) }, token);
101       await pedir("/api/usuarios/yo/preferencias", {
102         method: "PUT", body: JSON.stringify({

```

```

103     idioma_voz: datos.idioma_voz, velocidad_voz: datos.velocidad_voz,
104     confirmacion_hablada: datos.confirmacion_hablada,
105   }},
106   }, token);
107   setMsg("Cambios guardados correctamente.");
108 } catch (e) { setErr(e.message); }
109 }
110 async function subirFoto(e) {
111   const f = e.target.files?.[0];
112   if (!f) return;
113   setErr(""); setMsg("");
114   const form = new FormData();
115   form.append("foto", f);
116   try {
117     const r = await fetch(`${BASE}/api/usuarios/yo/foto`, {
118       method: "POST",
119       headers: { Authorization: `Bearer ${token || leerToken()}` },
120       body: form,
121     });
122     if (!r.ok) throw new Error("No se pudo subir la foto.");
123     cargarFoto();
124     window.dispatchEvent(new Event("cuentavoz:foto-actualizada"));
125     setMsg("Foto actualizada.");
126   } catch (e2) {
127     setErr(esFalloRed(e2)
128       ? "Sin conexión con el servidor. Revise el Wi-Fi e intente de nuevo."
129       : e2.message);
130   }
131 }
132 async function cambiarClavePropia() {
133   setErr(""); setMsg("");
134   if (!claveActual) return setErr("Digite su clave actual.");
135   if (!claveCumplePolitica(claveNueva)) return setErr("La clave nueva no cumple todos los requisitos.");
136   if (claveNueva !== claveNueva2) return setErr("Las dos claves no coinciden.");
137   try {
138     // habla directo con Cognito (no con este backend, que nunca ve la
139     // clave) - a diferencia del PIN de antes, esto NO invalida la sesión
140     // actual, así que no hace falta recargar la página.
141     await cambiarClave(claveActual, claveNueva);
142     setClaveActual(""); setClaveNueva(""); setClaveNueva2("");
143     setMsg("Clave actualizada.");
144     setDatos((d) => ({ ...d, pin_vence_en_dias: 90 }));
145     pedir("/api/usuarios/yo/marcar-clave-cambiada", { method: "POST" }, token).catch(() => {});
146   } catch (e) { setErr(e.message); }
147 }
148 async function cerrarTodas() {
149   try {
150     await pedir("/api/usuarios/yo/cerrar-todas", { method: "POST" }, token);
151     // revoca las sesiones en TODOS los dispositivos, incluido este -
152     // por consistencia se cierra también aquí en vez de seguir en la
153     // pantalla con un token que ya no sirve para pedir uno nuevo.
154     cerrarSesionLocal();
155     borrarSesion();
156     window.location.reload();
157   } catch (e) { setErr(e.message); }
158 }
159 function alActivarMFA() {
160   setConfigurandoMFA(false);
161   setMfaActiva(true);
162   setMsg("Verificación en dos pasos activada.");
163 }
164 async function quitarMFA() {
165   setErr(""); setMsg("");
166   try {
167     await desactivarMFA();
168     setMfaActiva(false);
169     setMsg("Verificación en dos pasos desactivada.");
170   } catch (e) { setErr(e.message); }
171 }
172 function probarVoz() {
173   hablar("Hola, soy CuentaVoz. Así voy a sonar de ahora en adelante.", { forzar: true });
174 }
175
176 // Solo actualiza en memoria (para que "Probar voz" suene distinto de una

```

```

177 // vez): el guardado real de estas preferencias queda para "Guardar
178 // cambios", junto con los datos personales - un solo botón, un solo
179 // guardado, como el resto de la pantalla.
180 function elegirPreferencia(cambios) {
181     const nuevos = { ...datos, ...cambios };
182     setDatos(nuevos);
183     configurarVoz({ idioma: nuevos.idioma_voz, velocidad: nuevos.velocidad_voz,
184                   confirmacionHablada: nuevos.confirmacion_hablada });
185 }
186
187 // El agente ("Pregúntele al agente", arriba) Sí puede cambiar la voz, la
188 // velocidad y la confirmación hablada directo en la base de datos - a
189 // diferencia de elegirPreferencia() (que solo actualiza en memoria hasta
190 // "Guardar cambios"), aquí ya quedó guardado en el servidor, así que se
191 // trae de nuevo (en vez de adivinar qué cambió) y se aplica de una con
192 // configurarVoz(), para que la SIGUIENTE frase que diga el agente ya
193 // suene con la preferencia nueva sin esperar a recargar la página.
194 function recargarPreferencias() {
195     pedir("/api/usuarios/yo", {}, token).then((d) => {
196         setDatos(d);
197         configurarVoz({ idioma: d.idioma_voz, velocidad: d.velocidad_voz,
198                       confirmacionHablada: d.confirmacion_hablada });
199     }).catch(() => {});
200 }
201
202 if (!datos) return <Marco titulo="Mi perfil"><p className="cargando">Cargando...</p></Marco>;
203
204 return (
205     <Marco titulo="Mi perfil . datos personales"
206       chip={{ texto: "SESIÓN ACTIVA", tipo: "verde" }}>
207       <AsistenteVoz token={token} vista="perfil" ir={ir} ejemplos={EJEMPLOS_PERFIL}
208         placeholder="cambia mi voz a puck, ¿cuándo fue mi último acceso?..."
209         alActualizar={recargarPreferencias} />
210       <div className="perfil-cols">
211         <div className="card mic-caja">
212           {fotoUrl ? (
213             <img src={fotoUrl} alt=""
214               style={{ width: 110, height: 110, borderRadius: "50%",
215                 objectFit: "cover", border: "3px solid var(--azul)" }} />
216           ) : (
217             <span className="avatar-grande">{{(datos.nombre || "?")[0].toUpperCase()}}</span>
218           )}
219           <b style={{ textTransform: "capitalize" }}>{{datos.nombre}}</b>
220           <small>{{datos.perfil === "auditor" ? "Administrador de bodega"
221             : "Auxiliar de inventarios"}}</small>
222           <label className="btn borde" style={{ textAlign: "center" }}>
223             Cambiar foto
224             <input type="file" accept="image/*" hidden onChange={subirFoto} />
225           </label>
226         </div>
227
228         <div className="card">
229           <h2>Datos personales</h2>
230           <div className="campos-2col">
231             [[["Nombre completo", "nombre"], ["Correo corporativo", "correo"],
232               ["Teléfono de contacto", "telefono"], ["Código de empleado", "codigo"]].map(([l, k]) => (
233               <div key={k} style={{ marginBottom: 10 }}>
234                 <label className="pista" htmlFor={`campo-perfil-${k}`}>{{l}}</label>
235                 <input id={`campo-perfil-${k}`} value={{datos[k] || ""}}
236                   onChange={{(e) => setDatos({ ...datos, [k]: e.target.value }}}}
237                 style={{ width: "100%", padding: "11px 13px",
238                   border: "1px solid var(--borde)", borderRadius: 12 }} />
239               </div>
240             )]]
241           </div>
242           <label className="pista" id="etiqueta-bodegas-asignadas">Bodegas asignadas</label>
243           <div className="chips" style={{ marginTop: 6 }} aria-labelledby="etiqueta-bodegas-asignadas">
244             {bodegas.length === 0 ? (
245               <span className="pista">Sin bodegas asignadas todavía.</span>
246             ) : bodegas.map((b) => <span key={b.id} className="chip">{{b.bodega}}</span>)}
247           </div>
248         </div>
249       </div>
250     )

```



```

251 <div className="dos-columnas">
252   <div className="card">
253     <h2>Seguridad de la cuenta</h2>
254     <p style={{ fontSize: ".87rem", marginBottom: 4 }}>
255       Último acceso: <b>{formatoAcceso(datos.ultimo_acceso)}</b>
256     </p>
257     <p style={{ fontSize: ".87rem", marginBottom: 14 }}>
258       Su clave vence en <b>{datos.pin_vence_en_dias} días</b>
259     </p>
260
261     {datos.pin_vence_en_dias <= 14 && (
262       <div className="banner" style={{ marginBottom: 14 }}>
263         <span className="ico">!</span>
264         <span>
265           <b>Su clave vence pronto</b>
266           <span>Le quedan {datos.pin_vence_en_dias} días. Cámbiela abajo cuando quiera.</span>
267         </span>
268       </div>
269     )}
270
271     <input type="password" placeholder="Clave actual" value={claveActual}
272       autoComplete="current-password"
273       onChange={(e) => setClaveActual(e.target.value)}
274       aria-label="Clave actual"
275       style={{ width: "100%", padding: "11px 13px", marginBottom: 8,
276         border: "1px solid var(--borde)", borderRadius: 12 }} />
277     <input type="password" placeholder="Clave nueva"
278       value={claveNueva}
279       autoComplete="new-password"
280       onChange={(e) => setClaveNueva(e.target.value)}
281       aria-label="Clave nueva"
282       style={{ width: "100%", padding: "11px 13px", marginBottom: 4,
283         border: "1px solid var(--borde)", borderRadius: 12 }} />
284     <ChecklistClave clave={claveNueva} />
285     <input type="password" placeholder="Confirmar clave nueva" value={claveNueva2}
286       autoComplete="new-password"
287       onChange={(e) => setClaveNueva2(e.target.value)}
288       aria-label="Confirmar clave nueva"
289       style={{ width: "100%", padding: "11px 13px",
290         border: "1px solid var(--borde)", borderRadius: 12 }} />
291     <div className="grilla-botones">
292       <button className="btn borde" onClick={cambiarClavePropia}>Cambiar clave</button>
293     </div>
294
295     <div className="grilla-botones" style={{ marginTop: 10 }}>
296       <button className="btn gris" onClick={() => setConfirmarCierre(true)}>
297         Cerrar sesión en todos los dispositivos
298       </button>
299     </div>
300     <p className="pista" style={{ marginTop: 8 }}>
301       La identidad y la clave las administra AWS Cognito, no CuentaVoz -
302       este backend nunca ve ni guarda su clave.
303     </p>
304
305     <hr style={{ border: "none", borderTop: "1px solid var(--borde)", margin: "18px 0" }} />
306     <h3 style={{ fontSize: "1rem" }}>Verificación en dos pasos</h3>
307     {mfaActiva === null ? (
308       <p className="pista">Comprobando...</p>
309     ) : configurandoMFA ? (
310       <ConfigurarMFA nombreUsuario={datos.nombre} onActivado={alActivarMFA}
311         onCancel={() => setConfigurandoMFA(false)} />
312     ) : mfaActiva ? (
313       <>
314         <p className="pista" style={{ color: "var(--verde)", fontStyle: "normal" }}>
315           ✓ Activada con una app autenticadora.
316         </p>
317         <div className="grilla-botones">
318           <button className="btn gris" onClick={() => setConfirmarQuitarMFA(true)}>Desactivar</button>
319         </div>
320       </>
321     ) : (
322       <>
323         <p className="pista">
324           Pida un código de 6 dígitos generado en su celular además de la clave, al ingresar. Opcional.

```

```

325         </p>
326         <div className="grilla-botones">
327             <button className="btn borde" onClick={() => setConfigurandoMFA(true)}>Activar</button>
328         </div>
329     </>
330 })
331 </div>
332
333 <div className="card">
334     <h2>Preferencias de voz</h2>
335     <label className="pista" htmlFor="campo-voz">Voz de CuentaVoz</label>
336     <select id="campo-voz" value={datos.idioma_voz}
337         onChange={(e) => elegirPreferencia({ idioma_voz: e.target.value })}
338         style={{ width: "100%", marginTop: 6, marginBottom: 8, padding: "10px 12px",
339             border: "1px solid var(--borde)", borderRadius: 10 }}>
340         {voces.map((v) => (
341             <option key={v.clave} value={v.clave}>{v.nombre} – {v.etiqueta}</option>
342         ))}
343     </select>
344     <div className="grilla-botones" style={{ marginBottom: 6 }}>
345         <button className="btn borde" onClick={probarVoz}>🔊 Probar voz</button>
346     </div>
347     <p className="pista" style={{ marginBottom: 12 }}>
348         Voz neuronal real (no la del navegador): suena humana y fluida.
349         El reconocimiento de lo que usted dice sigue siempre en español
350         de Colombia; esto solo cambia cómo suena CuentaVoz al responder.
351     </p>
352
353     <label className="pista" id="etiqueta-velocidad">Velocidad de la respuesta</label>
354     <div className="grilla-botones" style={{ marginTop: 6, marginBottom: 12 }}
355         role="group" aria-labelledby="etiqueta-velocidad">
356         {[ "lenta", "normal", "rapida" ].map((v2) => (
357             <button key={v2}
358                 className={`btn ${datos.velocidad_voz === v2 ? "" : "borde"} `}
359                 onClick={() => elegirPreferencia({ velocidad_voz: v2 })}
360                 aria-pressed={datos.velocidad_voz === v2}>
361             {v2[0].toUpperCase() + v2.slice(1)}
362         </button>
363         ))}
364     </div>
365
366     <label className="pista" id="etiqueta-confirmacion">Confirmación hablada</label>
367     <div className="grilla-botones" style={{ marginTop: 6, marginBottom: 18 }}
368         role="group" aria-labelledby="etiqueta-confirmacion">
369         <button className={`btn ${datos.confirmacion_hablada ? "" : "borde"} `}
370             onClick={() => elegirPreferencia({ confirmacion_hablada: true })}
371             aria-pressed={!!datos.confirmacion_hablada}>
372             Activada
373         </button>
374         <button className={`btn ${!datos.confirmacion_hablada ? "" : "borde"} `}
375             onClick={() => elegirPreferencia({ confirmacion_hablada: false })}
376             aria-pressed={!datos.confirmacion_hablada}>
377             Desactivada
378         </button>
379     </div>
380
381     {err && <p className="error" role="alert">{err}</p>}
382     {msg && <p className="msg-ok" aria-live="polite">{msg}</p>}
383     <button className="btn" style={{ width: "100%" }} onClick={guardar}>
384         Guardar cambios
385     </button>
386 </div>
387 </div>
388
389 <div className="card">
390     <h2>Accesibilidad</h2>
391     <div className="grilla-botones" style={{ alignItems: "center", justifyContent: "flex-start", gap: 10 }}>
392         <Interruptor activo={!accesibilidad.contraste} etiqueta="Alto contraste"
393             onChange={(v) => cambiarAccesibilidad({ contraste: v })} />
394         <span className="pista">Alto contraste</span>
395     </div>
396     <label className="pista" htmlFor="campo-tamano-letra" style={{ display: "block", marginTop: 12 }}>
397         Tamaño de letra
398     </label>

```

```

399     <select id="campo-tamano-letra" value={accesibilidad.tamano || "normal"}
400       onChange={(e) => cambiarAccesibilidad({ tamano: e.target.value === "normal" ? "" : e.target.value })}
401       style={{ width: "100%", maxWidth: 260, marginTop: 6, padding: "10px 12px",
402         border: "1px solid var(--borde)", borderRadius: 10 }}>
403       <option value="normal">Normal</option>
404       <option value="grande">Grande</option>
405       <option value="mas-grande">Más grande</option>
406     </select>
407     <p className="pista" style={{ marginTop: 10 }}>
408       Se aplica de una vez en este equipo, sin necesidad de guardar.
409     </p>
410   </div>
411
412   {confirmarCierre && (
413     <Dialogo titulo="Cerrar sesión en todos los dispositivos"
414       mensaje="Esto cierra la sesión en todas las tabletas donde haya entrado, incluida esta. ¿Continuar?"
415       textoAceptar="Cerrar todas" peligro
416       onAceptar={() => { setConfirmarCierre(false); cerrarTodas(); }}
417       onCancel={() => setConfirmarCierre(false)} />
418   )}
419
420   {confirmarQuitarMFA && (
421     <Dialogo titulo="Desactivar verificación en dos pasos"
422       mensaje="La próxima vez que entre, bastará con la clave - ya no se pedirá el código de la app
autenticadora. ¿Continuar?"
423       textoAceptar="Desactivar" peligro
424       onAceptar={() => { setConfirmarQuitarMFA(false); quitarMFA(); }}
425       onCancel={() => setConfirmarQuitarMFA(false)} />
426   )}
427 </Marco>
428 );
429 }

```

frontend/src/vistas/CerrarSesion.jsx

74 LÍNEAS

Salir sin dejar nada a medias.

```

1 import { useEffect, useRef, useState } from "react";
2 import { pedir } from "../api";
3 import { useDevolverFoco } from "../foco";
4
5 export default function CerrarSesion({ token, sesionId, onCancel, onSalir }) {
6   const [pend, setPend] = useState(null);
7   const [noSePudoVerificar, setNoSePudoVerificar] = useState(false);
8   const modalRef = useRef(null);
9   useDevolverFoco();
10
11   // Igual que Dialogo.jsx: Escape cierra (aquí, "seguir trabajando" - la
12   // opción que no pierde nada), y el modal recibe el foco al aparecer. Sin
13   // esto, quien navega con teclado o con un lector de pantalla no tenía
14   // forma de descartar este diálogo en particular, aunque todos los demás
15   // de la app sí la ofrecen.
16   useEffect(() => {
17     function alTecla(e) { if (e.key === "Escape") onCancel(); }
18     window.addEventListener("keydown", alTecla);
19     return () => window.removeEventListener("keydown", alTecla);
20   }, [onCancel]);
21
22   useEffect(() => {
23     // el modal no existe en el DOM hasta que "pend" resuelve (ver el
24     // "if (!pend) return null" más abajo) - el foco solo puede entrar una
25     // vez que de verdad aparece, no en el montaje del componente.
26     if (pend) modalRef.current?.focus();
27   }, [pend]);
28
29   useEffect(() => {
30     setNoSePudoVerificar(false);
31     pedir(`/api/sesiones/${sesionId}/avance`, {}, token)
32       .then((a) => setPend({ bodega: a.bodega, hechas: a.hechas }))
33       // si la comprobación falla (sin conexión, sesión vencida...) no hay
34       // que asumir que no hay trabajo pendiente: eso es la respuesta más
35       // tranquilizadora posible justo cuando menos se sabe. Se avisa que

```

```

36 // no se pudo comprobar en vez de decir "puede salir sin problema"
37 // sin tener manera de saberlo.
38 .catch(() => { setNoSePudoVerificar(true); setPend({ bodega: null, hechas: 0 }); });
39 }, [sesionId, token]);
40
41 if (!pend) return null;
42 const hayTrabajo = Boolean(pend.bodega);
43
44 return (
45   <div className="overlay" onClick={onCancelar}>
46     <div className="modal" onClick={(e) => e.stopPropagation()}
47       role="dialog" aria-modal="true" aria-labelledby="cerrar-sesion-titulo"
48       ref={modalRef} tabIndex={-1}>
49     <h2 id="cerrar-sesion-titulo">¿Cerrar la sesión?</h2>
50     {noSePudoVerificar ? (
51       <p>
52         No se pudo comprobar si tiene trabajo sin guardar. Si tiene una bodega abierta,
53         lo contado hasta ahora ya quedó guardado y podrá continuar al volver.
54       </p>
55     ) : hayTrabajo ? (
56       <p>
57         Tiene <b>{pend.bodega}</b> abierta con {pend.hechas} referencias contadas.
58       </p>
59       <div className="aviso">
60         Lo registrado queda guardado y podrá continuar al volver.
61       </div>
62     ) : (
63       <p>No tiene trabajo pendiente: puede salir sin problema.</p>
64     )}
65   <div className="botones">
66     <button className="btn borde" onClick={onCancelar}>Seguir trabajando</button>
67     <button className="btn" onClick={onSalir}>Cerrar sesión</button>
68   </div>
69 </div>
70 );
71 }

```

17.7 Pruebas

backend/tests/conftest.py

135 LÍNEAS

La base en memoria que usan todas.

```

1 """Infraestructura aislada para las pruebas de regresión de la API."""
2 from __future__ import annotations
3
4 import importlib
5 import sys
6 from pathlib import Path
7 from typing import Iterator
8
9 import pytest
10 from fastapi.testclient import TestClient
11
12
13 BACKEND_DIR = Path(__file__).resolve().parents[1]
14 if str(BACKEND_DIR) not in sys.path:
15     sys.path.insert(0, str(BACKEND_DIR))
16
17
18 def _descargar_modulos_backend() -> None:
19     """Fuerza que cada prueba lea DB_URL antes de crear el motor SQLAlchemy."""
20     directos = {"bd", "modelos", "seguridad", "main", "reportes"}
21     prefijos = ("agente", "servicios")
22     for nombre in list(sys.modules):
23         if nombre in directos or nombre.startswith(prefijos):
24             sys.modules.pop(nombre, None)

```

```

25
26
27 def _reclamos_prueba(token: str) -> dict[str, object] | None:
28     """Reemplaza la verificación real de un access token de Cognito (que
29     exigiría red y un User Pool de verdad) por una regla trivial: el
30     "token" que emite /api/ingresar de prueba (ver mas abajo) ES el
31     nombre de usuario, y aquí simplemente se acepta tal cual - las
32     pruebas de este archivo verifican la lógica de negocio (permisos,
33     asignaciones, agente...), no la integración real con Cognito, que
34     ya se probó a mano contra el User Pool real."""
35     if not token:
36         return None
37     return {"username": token, "token_use": "access", "client_id": "prueba"}
38
39
40 @pytest.fixture
41 def app_modulos(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> Iterator[object]:
42     """Importa una aplicación nueva conectada a un SQLite temporal por prueba."""
43     ruta_db = tmp_path / "cuentavoz-pruebas.db"
44     monkeypatch.setenv("DB_URL", f"sqlite:/// {ruta_db.as_posix()}")
45     monkeypatch.setenv("COGNITO_REGION", "us-east-2")
46     monkeypatch.setenv("COGNITO_USER_POOL_ID", "us-east-2-pruebas")
47     monkeypatch.setenv("COGNITO_APP_CLIENT_ID", "prueba")
48     # datos_regresion (mas abajo) espera encontrar a "luis" ya creado por
49     # arranque() - explicito aquí, no implícito vía un .env de desarrollo
50     # que quien corra esto en otra máquina o en CI no tiene por que tener.
51     monkeypatch.setenv("SEMBRAR_DEMO", "1")
52     _descargar_modulos_backend()
53
54     modulo = importlib.import_module("main")
55     # main.py hace "from seguridad import usuario_actual, ..." (no "import
56     # seguridad"), así que hay que importar el modulo aparte para parchar
57     # _reclamos_cognito - sys.modules ya tiene la MISMA instancia que main
58     # uso, importlib no la vuelve a crear.
59     modulo_seguridad = importlib.import_module("seguridad")
60     monkeypatch.setattr(modulo_seguridad, "_reclamos_cognito", _reclamos_prueba)
61
62     from fastapi import Form
63
64     @modulo.app.post("/api/ingresar")
65     def _ingresar_prueba(username: str = Form(...), password: str = Form(...)):
66         """Solo para pruebas: en la app real el login lo hace Cognito
67         directo desde el frontend (ver cognito.js), este backend nunca
68         recibe una clave. Emite un "token" de prueba que _reclamos_prueba
69         acepta, para no reescribir cada prueba que necesita una sesión."""
70         from fastapi import HTTPException
71         with modulo.Sesion() as s:
72             u = s.query(modulo.Usuario).filter_by(nombre=username.lower()).first()
73             if u is None or not u.activo:
74                 raise HTTPException(401, "Usuario o clave incorrectos.")
75             return {"token": u.nombre, "perfil": u.perfil}
76
77     try:
78         yield modulo
79     finally:
80         # Libera el archivo SQLite antes de que pytest elimine tmp_path en Windows.
81         modulo.Sesion.kw["bind"].dispose() if hasattr(modulo.Sesion, "kw") else None
82         _descargar_modulos_backend()
83
84
85 @pytest.fixture
86 def client(app_modulos: object) -> Iterator[TestClient]:
87     """Cliente con el ciclo de vida de FastAPI activo (incluye iniciar_bd)."""
88     with TestClient(app_modulos.app) as cliente:
89         yield cliente
90
91
92 @pytest.fixture
93 def datos_regresion(client: TestClient) -> dict[str, object]:
94     """Crea un auxiliar, dos bodegas y el mismo artículo en ambas bodegas."""
95     from bd import Sesion
96     from modelos import ( # Se importan después de fijar DB_URL en app_modulos.
97         Articulo,
98         AsignacionBodega,

```

```

99         Bodega,
100         StockSistema,
101         Usuario,
102     )
103
104     with Sesion() as sesion:
105         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
106         asignada = Bodega(nombre_oficial="BODEGA ASIGNADA")
107         no_asignada = Bodega(nombre_oficial="BODEGA RESTRINGIDA")
108         articulo = Articulo(codigo="ART-REG-001", nombre_oficial="ARTICULO REGRESION",
109                             unidad_medida="unidad")
110         sesion.add_all([asignada, no_asignada, articulo])
111         sesion.flush()
112         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=asignada.id))
113         sesion.add_all([
114             StockSistema(articulo_codigo=articulo.codigo, bodega_id=asignada.id,
115                           cantidad_sd=100),
116             StockSistema(articulo_codigo=articulo.codigo, bodega_id=no_asignada.id,
117                           cantidad_sd=200),
118         ])
119         sesion.commit()
120         datos = {
121             "auxiliar_id": auxiliar.id,
122             "bodega_asignada_id": asignada.id,
123             "bodega_no_asignada_id": no_asignada.id,
124             "bodega_asignada": asignada.nombre_oficial,
125             "bodega_no_asignada": no_asignada.nombre_oficial,
126             "articulo_codigo": articulo.codigo,
127         }
128
129         respuesta = client.post(
130             "/api/ingresar",
131             data={"username": "luis", "password": "StockXperts1"},
132         )
133         assert respuesta.status_code == 200, respuesta.text
134         datos["headers"] = {"Authorization": f"Bearer {respuesta.json()['token']}"}
135         return datos

```

backend/tests/test_cerebro.py

85 LÍNEAS

El orquestador del agente.

```

1  """Regresiones del «pensar» del agente: la limpieza de negaciones (para que
2  "no quiero preparar arroz" nunca se lea como un pedido de arroz) y la
3  degradación a respaldo local cuando no hay llave de Gemini - la garantía
4  de que la demo no se cae por falta de internet."""
5  from __future__ import annotations
6
7  import pytest
8
9  from agente.cerebro import _limpiar_si_niega, pensar
10
11
12  # ----- _limpiar_si_niega -----
13
14  def test_niega_pedido_borra_preparacion_y_baja_intencion():
15      datos = {"intencion": "pedir", "preparacion": "ARROZ", "porciones": 4,
16              "articulo_texto": None, "cantidad": None}
17      _limpiar_si_niega("no quiero preparar arroz", datos)
18      assert datos["intencion"] == "explicar"
19      assert datos["preparacion"] is None
20      assert datos["porciones"] is None
21
22
23  def test_niega_conteo_borra_articulo_y_cantidad():
24      datos = {"intencion": "contar", "articulo_texto": "ARROZ", "cantidad": 5,
25              "preparacion": None, "porciones": None}
26      _limpiar_si_niega("ya no necesito eso", datos)
27      assert datos["intencion"] == "explicar"
28      assert datos["articulo_texto"] is None
29      assert datos["cantidad"] is None
30

```

```

31
32 @pytest.mark.parametrize("frase", [
33     "cancela el ajiaco", "cancelar el pedido", "quitale de la lista",
34     "no me sirve ese", "olvida eso", "no es arroz",
35 ])
36 def test_frases_de_negacion_reconocidas(frase):
37     datos = {"intencion": "pedir", "preparacion": "ALGO", "porciones": 1,
38             "articulo_texto": None, "cantidad": None}
39     _limpiar_si_niega(frase, datos)
40     assert datos["preparacion"] is None
41
42
43 def test_frase_sin_negacion_no_se_toca():
44     datos = {"intencion": "pedir", "preparacion": "ARROZ", "porciones": 4,
45             "articulo_texto": None, "cantidad": None}
46     _limpiar_si_niega("hoy preparamos arroz para cuatro", datos)
47     assert datos["intencion"] == "pedir"
48     assert datos["preparacion"] == "ARROZ"
49     assert datos["porciones"] == 4
50
51
52 def test_intencion_distinta_de_pedir_o_contar_no_se_degrada():
53     # una negación limpia los campos igual, pero no toca intenciones que
54     # ya no son "pedir"/"contar" (ej. "consultar" no debe volverse "explicar").
55     datos = {"intencion": "consultar", "preparacion": None, "porciones": None,
56             "articulo_texto": "ARROZ", "cantidad": None}
57     _limpiar_si_niega("no quiero ese, mejor cancelalo", datos)
58     assert datos["intencion"] == "consultar"
59     assert datos["articulo_texto"] is None
60
61
62 # ----- pensar(): respaldo sin Gemini -----
63
64 def test_pensar_sin_llave_cae_al_interprete_local(monkeypatch):
65     """Sin GOOGLE_API_KEY, pensar() no debe lanzar ni bloquear la
66     conversación: debe devolver exactamente lo que interpretaría el
67     intérprete local, con su propio mensaje hablado de aviso."""
68     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
69     import agente.cerebro as cerebro
70     monkeypatch.setattr(cerebro, "_cliente", None)
71
72     resultado = pensar("Bodega activa: ninguna.", "hay noventa cazuelas")
73
74     assert resultado["intencion"] == "contar"
75     assert resultado["cantidad"] == 90
76     assert "llave" in resultado["respuesta_hablada"].lower()
77
78
79 def test_pensar_sin_llave_reconoce_confirmacion(monkeypatch):
80     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
81     import agente.cerebro as cerebro
82     monkeypatch.setattr(cerebro, "_cliente", None)
83
84     resultado = pensar("Pendiente de confirmar: arroz 90.", "confirmo")
85     assert resultado["intencion"] == "confirmar"

```

backend/tests/test_interprete.py

161 LÍNEAS

El interprete local.

```

1 """Regresiones del intérprete local: el respaldo que sostiene la demo si
2 falla el Wi-Fi o no hay llave de Gemini. Es reconocimiento de palabras
3 clave, no NLU real - por eso necesita cobertura explícita de cada intención
4 y de las formas de decir números que ya se sabe que aparecen en la demo."""
5 from __future__ import annotations
6
7 from servicios.interprete import interpretar_local, normalizar_unidad
8
9
10 # ----- números -----
11
12 def test_numero_digito_simple():

```

```

13     assert interpretar_local("hay 12 arroces")["cantidad"] == 12
14
15
16 def test_numero_decimal_con_punto():
17     assert interpretar_local("hay 2.5 kilos de arroz")["cantidad"] == 2.5
18
19
20 def test_numero_decimal_con_coma():
21     assert interpretar_local("hay 2,5 kilos de arroz")["cantidad"] == 2.5
22
23
24 def test_numero_negativo_con_menos():
25     assert interpretar_local("hay menos 5 cazuelas")["cantidad"] == -5
26
27
28 def test_numero_en_palabras_simple():
29     assert interpretar_local("hay cinco cazuelas")["cantidad"] == 5
30
31
32 def test_numero_compuesto_decena_y_unidad():
33     # "treinta y cinco" -> 35
34     assert interpretar_local("hay treinta y cinco tablas")["cantidad"] == 35
35
36
37 def test_numero_compuesto_centena_y_decena():
38     # "ciento ochenta" -> 180
39     assert interpretar_local("hay ciento ochenta unidades")["cantidad"] == 180
40
41
42 def test_numero_en_palabras_mil():
43     assert interpretar_local("hay mil servilletas")["cantidad"] == 1000
44
45
46 def test_numero_en_palabras_con_menos():
47     assert interpretar_local("hay menos cinco cazuelas")["cantidad"] == -5
48
49
50 def test_sin_numero_ni_palabra_clave_cae_en_ayuda():
51     # sin número reconocible ni ninguna palabra clave, cae en el
52     # "no le entendí" por defecto en vez de fingir una cantidad.
53     r = interpretar_local("no logro ver nada aqui")
54     assert r["intencion"] == "ayuda"
55     assert r.get("cantidad") is None
56
57
58 # ----- unidades -----
59
60 def test_unidad_kilo_variantes():
61     for palabra in ("kilo", "kilos", "kilogramo", "kilogramos", "kg"):
62         assert interpretar_local(f"hay 3 {palabra} de arroz")["unidad"] == "Kilogram"
63
64
65 def test_unidad_litro_variantes():
66     for palabra in ("litro", "litros", "l"):
67         assert interpretar_local(f"hay 3 {palabra} de aceite")["unidad"] == "Liter"
68
69
70 def test_unidad_no_reconocida_es_none():
71     assert interpretar_local("hay 3 cazuelas blancas")["unidad"] is None
72
73
74 def test_normalizar_unidad_pasa_canonicas_intactas():
75     assert normalizar_unidad("Kilogram") == "Kilogram"
76
77
78 def test_normalizar_unidad_traduce_lo_dicho():
79     assert normalizar_unidad("kilos") == "Kilogram"
80     assert normalizar_unidad("litros") == "Liter"
81     assert normalizar_unidad("unidades") == "Unidad"
82     assert normalizar_unidad("porciones") == "Portion"
83
84
85 def test_normalizar_unidad_desconocida_se_devuelve_igual():
86     assert normalizar_unidad("bultos") == "bultos"

```



```

87
88
89 def test_normalizar_unidad_vacia():
90     assert normalizar_unidad(None) is None
91     assert normalizar_unidad("") == ""
92
93
94 # ----- intenciones -----
95
96 def test_intencion_navegar_iniciar_conteo():
97     r = interpretar_local("iniciar conteo en almacen ayb")
98     assert r["intencion"] == "navegar"
99     assert "almacen ayb" in r["bodega_texto"] or "ayb" in r["bodega_texto"]
100
101
102 def test_intencion_navegar_abrir_bodega():
103     r = interpretar_local("abrir bodega restaurante fuentes")
104     assert r["intencion"] == "navegar"
105
106
107 def test_intencion_pedir_con_porciones():
108     r = interpretar_local("hoy preparamos cincuenta ajiacos")
109     assert r["intencion"] == "pedir"
110     assert r["porciones"] == 50
111
112
113 def test_intencion_pedir_sin_porciones_queda_en_cero():
114     r = interpretar_local("vamos a preparar sancocho")
115     assert r["intencion"] == "pedir"
116     assert r["porciones"] == 0
117
118
119 def test_intencion_confirmar_variantes():
120     for frase in ("confirmo", "si", "sí", "correcto"):
121         assert interpretar_local(frase)["intencion"] == "confirmar"
122
123
124 def test_intencion_corregir_variantes():
125     for frase in ("no son esos, corregir", "uy no, esta mal", "cambiar la cantidad"):
126         assert interpretar_local(frase)["intencion"] == "corregir"
127
128
129 def test_intencion_consultar():
130     r = interpretar_local("cuanto arroz hay")
131     assert r["intencion"] == "consultar"
132     assert "arroz" in r["articulo_texto"]
133
134
135 def test_intencion_reporte():
136     for frase in ("generame el reporte", "el consolidado por favor", "imprimeme el archivo"):
137         assert interpretar_local(frase)["intencion"] == "reporte"
138
139
140 def test_intencion_ayuda_explicita():
141     for frase in ("ayuda", "no entiendo"):
142         assert interpretar_local(frase)["intencion"] == "ayuda"
143
144
145 def test_intencion_contar_con_cantidad_y_articulo():
146     r = interpretar_local("hay noventa cazuelas blancas")
147     assert r["intencion"] == "contar"
148     assert r["cantidad"] == 90
149     assert "cazuelas" in r["articulo_texto"] or "blancas" in r["articulo_texto"]
150
151
152 def test_intencion_desconocida_pide_repetir():
153     r = interpretar_local("xyzyy blorp qwerty")
154     assert r["intencion"] == "ayuda"
155     assert "repite" in r["respuesta_hablada"].lower()
156
157
158 def test_articulo_texto_sin_ruido_quita_palabras_vacias():
159     r = interpretar_local("hay noventa cazuelas blancas")

```

```

160     for palabra_vacia in ("hay", "noventa"):
161         assert palabra_vacia not in r["articulo_texto"].split()

```

backend/tests/test_agente_conversacion.py

317 LÍNEAS

Conversaciones completas.

```

1  """Integración de extremo a extremo del agente conversacional: guía
2  conversaciones reales por /api/agente/turno. Corre sin GOOGLE_API_KEY a
3  propósito (usa el intérprete local, determinista) para no depender de una
4  red externa ni de la variabilidad de un modelo de lenguaje - exactamente
5  el camino que sostiene la demo si falla el Wi-Fi."""
6  from __future__ import annotations
7
8  import pytest
9  from fastapi.testclient import TestClient
10
11
12 @pytest.fixture
13 def datos_agente(client: TestClient, monkeypatch: pytest.MonkeyPatch) -> dict:
14     """Una bodega asignada a luis con dos artículos deliberadamente
15     ambiguos (ARROZ / ARROZ BASMATI, el ejemplo documentado en
16     orquestador.py) y uno sin ambigüedad (ACEITE), para ejercitar tanto
17     el camino directo como la desambiguación."""
18     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
19     import agente.cerebro as cerebro
20     monkeypatch.setattr(cerebro, "_cliente", None)
21
22     from bd import Sesion
23     from modelos import Artículo, AsignacionBodega, Bodega, StockSistema, Usuario
24
25     with Sesion() as sesion:
26         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
27         bodega = Bodega(nombre_oficial="BODEGA AGENTE")
28         arroz = Artículo(codigo="ARR-001", nombre_oficial="ARROZ", unidad_medida="Kilogram")
29         basmati = Artículo(codigo="ARR-002", nombre_oficial="ARROZ BASMATI",
30                             unidad_medida="Kilogram")
31         aceite = Artículo(codigo="ACE-001", nombre_oficial="ACEITE", unidad_medida="Liter")
32         sesion.add_all([bodega, arroz, basmati, aceite])
33         sesion.flush()
34         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
35         sesion.add_all([
36             StockSistema(articulo_codigo="ARR-001", bodega_id=bodega.id, cantidad_sd=100),
37             StockSistema(articulo_codigo="ARR-002", bodega_id=bodega.id, cantidad_sd=50),
38             StockSistema(articulo_codigo="ACE-001", bodega_id=bodega.id, cantidad_sd=100),
39         ])
40         sesion.commit()
41         bodega_id = bodega.id
42
43     respuesta = client.post("/api/ingresar",
44                             data={"username": "luis", "password": "StockXperts1"})
45     assert respuesta.status_code == 200, respuesta.text
46     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
47     return {"headers": headers, "bodega_id": bodega_id}
48
49
50 def _turno(client: TestClient, headers: dict, texto: str, sesion_id: int = 777) -> dict:
51     r = client.post("/api/agente/turno", headers=headers,
52                     json={"texto": texto, "sesion_id": sesion_id})
53     assert r.status_code == 200, r.text
54     return r.json()
55
56
57 def _ultimo_conteo(sesion_id: int):
58     """_guardar() en orquestador.py escribe Conteo.sesion_id con el mismo
59     entero de la conversación que llega a /api/agente/turno (no el id de
60     SesionConteo) - se consulta igual aquí."""
61     from bd import Sesion
62     from modelos import Conteo
63     with Sesion() as sesion:
64         return (sesion.query(Conteo).filter_by(sesion_id=sesion_id, estado="confirmado")
65                 .order_by(Conteo.id.desc()).first())

```

```

66
67
68 def test_abrir_bodega_pide_confirmar_antes_de_abrir_y_luego_deja_contar(
69     client: TestClient, datos_agente: dict,
70 ) -> None:
71     headers = datos_agente["headers"]
72
73     ofrece = _turno(client, headers, "iniciar conteo en bodega agente")
74     assert ofrece.get("bodega") is None # todavia no abrio nada
75     assert "confirma" in ofrece["respuesta_hablada"].lower()
76
77     abrir = _turno(client, headers, "confirmo")
78     assert abrir["bodega"]["id"] == datos_agente["bodega_id"]
79
80     contar = _turno(client, headers, "hay cien aceites")
81     assert contar.get("pendiente") is not None
82     assert contar.get("alerta") is None # coincide exacto con el sistema: sin alerta
83
84     confirmar = _turno(client, headers, "confirmo")
85     assert confirmar.get("guardado") is True
86
87     guardado = _ultimo_conteo(777)
88     assert guardado is not None
89     assert guardado.articulo_codigo == "ACE-001"
90     assert guardado.cantidad == 100
91
92
93 def test_abrir_bodega_por_voz_encuentra_el_nombre_aunque_diga_tilde(
94     client: TestClient, monkeypatch: pytest.MonkeyPatch,
95 ) -> None:
96     """Bug real reportado desde producción: el catálogo de bodegas viene
97     sin tildes del Excel ("ALMACEN SUMINISTROS"), pero el reconocimiento
98     de voz transcribe con tilde ("almacén suministros") - antes eso caía
99     en "no la encuentro, ¿la creamos?" para una bodega que sí existía."""
100    monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
101    import agente.cerebro as cerebro
102    monkeypatch.setattr(cerebro, "_cliente", None)
103
104    from bd import Sesion
105    from modelos import Bodega, Usuario, AsignacionBodega
106
107    with Sesion() as sesion:
108        auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
109        bodega = Bodega(nombre_oficial="ALMACEN CENTRAL")
110        sesion.add(bodega)
111        sesion.flush()
112        sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
113        sesion.commit()
114        bodega_id = bodega.id
115
116    respuesta = client.post("/api/ingresar",
117                            data={"username": "luis", "password": "StockXperts1"})
118    headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
119
120    ofrece = _turno(client, headers, "iniciar conteo en almacén central", sesion_id=779)
121    assert ofrece.get("intencion") != "crear_bodega", ofrece
122
123    r = _turno(client, headers, "confirmo", sesion_id=779)
124    assert r.get("bodega", {}).get("id") == bodega_id, r
125
126
127 def test_abrir_bodega_por_voz_no_confunde_nombres_parecidos(
128     client: TestClient, monkeypatch: pytest.MonkeyPatch,
129 ) -> None:
130     """Bug real reportado desde producción: decir "autoservicios las
131     fuentes" abría "AUTOSERVICIOS CASCADA" - el respaldo por palabras se
132     quedaba con la PRIMERA bodega que contuviera "autoservicios", sin
133     llegar a comparar "fuentes" para desempatar."""
134    monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
135    import agente.cerebro as cerebro
136    monkeypatch.setattr(cerebro, "_cliente", None)
137
138    from bd import Sesion
139    from modelos import Bodega, Usuario, AsignacionBodega

```

```

140
141 with Sesion() as sesion:
142     auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
143     cascada = Bodega(nombre_oficial="AUTOSERVICIOS CASCADA")
144     fuentes = Bodega(nombre_oficial="AUTOSERVICIOS LAS FUENTES")
145     sesion.add_all([cascada, fuentes])
146     sesion.flush()
147     sesion.add_all([
148         AsignacionBodega(usuario_id=auxiliar.id, bodega_id=cascada.id),
149         AsignacionBodega(usuario_id=auxiliar.id, bodega_id=fuentes.id),
150     ])
151     sesion.commit()
152     fuentes_id = fuentes.id
153
154     respuesta = client.post("/api/ingresar",
155                             data={"username": "luis", "password": "StockXperts1"})
156     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
157
158     _turno(client, headers, "iniciar conteo en autoservicios las fuentes", sesion_id=780)
159     r = _turno(client, headers, "confirno", sesion_id=780)
160     assert r.get("bodega", {}).get("id") == fuentes_id, r
161
162
163 def test_decir_no_a_la_confirmacion_de_bodega_no_abre_nada(
164     client: TestClient, datos_agente: dict,
165 ) -> None:
166     """Si el reconocimiento de voz entendió mal el nombre, decir "no" a
167     la confirmación debe dejar todo intacto - nada de sesión creada, nada
168     de bodega marcada en_conteo."""
169     headers = datos_agente["headers"]
170     sid = 783
171     ofrece = _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
172     assert ofrece.get("bodega") is None
173
174     r = _turno(client, headers, "no", sesion_id=sid)
175     assert r.get("bodega") is None
176     assert "no la abro" in r["respuesta_hablada"].lower()
177
178     from bd import Sesion
179     from modelos import Bodega, SesionConteo
180     with Sesion() as sesion:
181         b = sesion.get(Bodega, datos_agente["bodega_id"])
182         assert b.estado != "en_conteo"
183         assert sesion.query(SesionConteo).filter_by(bodega_id=b.id).count() == 0
184
185
186 def test_bodega_ya_abierta_por_otro_dice_quien_la_tiene(
187     client: TestClient, monkeypatch: pytest.MonkeyPatch,
188 ) -> None:
189     """Antes decía "ya está en conteo por otra persona" a secas - sin
190     saber si era normal o un error, ni con quién hablar. Ahora dice el
191     nombre real de quién la tiene."""
192     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
193     import agente.cerebro as cerebro
194     monkeypatch.setattr(cerebro, "_cliente", None)
195
196     from bd import Sesion
197     from modelos import Bodega, Usuario, AsignacionBodega, SesionConteo
198
199     with Sesion() as sesion:
200         luis = sesion.query(Usuario).filter_by(nombre="luis").one()
201         diana = sesion.query(Usuario).filter_by(nombre="diana").one()
202         bodega = Bodega(nombre_oficial="BODEGA COMPARTIDA", estado="en_conteo")
203         sesion.add(bodega)
204         sesion.flush()
205         sesion.add(AsignacionBodega(usuario_id=luis.id, bodega_id=bodega.id))
206         sesion.add(SesionConteo(bodega_id=bodega.id, usuario_id=diana.id,
207                                 tipo="conteo", estado="abierta"))
208         sesion.commit()
209
210     respuesta = client.post("/api/ingresar",
211                             data={"username": "luis", "password": "StockXperts1"})
212     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
213

```

```

214     r = _turno(client, headers, "iniciar conteo en bodega compartida", sesion_id=784)
215     assert r.get("bodega") is None
216     assert "diana" in r["respuesta_hablada"].lower()
217
218
219 def test_sin_bodega_abierta_no_deja_contar(client: TestClient, datos_agente: dict) -> None:
220     headers = datos_agente["headers"]
221     r = _turno(client, headers, "hay diez aceites", sesion_id=778)
222     assert "abra una bodega" in r["respuesta_hablada"].lower()
223     assert r.get("pendiente") is None
224
225
226 def test_arroz_ambiguo_se_desambigua_y_dispara_desviacion(
227     client: TestClient, datos_agente: dict,
228 ) -> None:
229     headers = datos_agente["headers"]
230     sid = 779
231     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
232     _turno(client, headers, "confirmo", sesion_id=sid)
233
234     ambiguo = _turno(client, headers, "hay noventa arroces", sesion_id=sid)
235     assert ambiguo.get("opciones") is not None
236     nombres = {o["nombre"] for o in ambiguo["opciones"]}
237     assert nombres == {"ARROZ", "ARROZ BASMATI"}
238
239     # el sistema espera 50 de la basmati; decir noventa es una desviación
240     # del 80%, muy por encima del umbral - debe alertar y NO auto-confirmar.
241     elegido = _turno(client, headers, "la basmati", sesion_id=sid)
242     assert elegido.get("alerta") == "desviacion"
243     assert elegido["contexto_alerta"]["articulo"] == "ARROZ BASMATI"
244     assert elegido.get("guardado") is not True
245
246     confirmar = _turno(client, headers, "confirmo", sesion_id=sid)
247     assert confirmar.get("guardado") is True
248
249     guardado = _ultimo_conteo(sid)
250     assert guardado.articulo_codigo == "ARR-002"
251     assert guardado.cantidad == 90
252
253
254 def test_declina_opciones_con_no_cancela_sin_romper(
255     client: TestClient, datos_agente: dict,
256 ) -> None:
257     headers = datos_agente["headers"]
258     sid = 780
259     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
260     _turno(client, headers, "confirmo", sesion_id=sid)
261     ambiguo = _turno(client, headers, "hay noventa arroces", sesion_id=sid)
262     assert ambiguo.get("opciones") is not None
263
264     cancelado = _turno(client, headers, "no", sesion_id=sid)
265     assert "cancelado" in cancelado["respuesta_hablada"].lower()
266
267     # sin opciones pendientes, un tercer turno normal ya no debe seguir
268     # atascado en la desambiguación anterior.
269     siguiente = _turno(client, headers, "hay cien aceites", sesion_id=sid)
270     assert siguiente.get("pendiente") is not None
271
272
273 def test_corregir_antes_de_confirmar_cambia_la_cantidad_pendiente(
274     client: TestClient, datos_agente: dict,
275 ) -> None:
276     headers = datos_agente["headers"]
277     sid = 781
278     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
279     _turno(client, headers, "confirmo", sesion_id=sid)
280     _turno(client, headers, "hay cien aceites", sesion_id=sid)
281
282     corregido = _turno(client, headers, "no, son noventa", sesion_id=sid)
283     assert corregido.get("pendiente") is not None
284
285     _turno(client, headers, "confirmo", sesion_id=sid)
286
287     guardado = _ultimo_conteo(sid)

```

```

288     assert guardado.cantidad == 90          # el valor corregido, no los cien originales
289
290
291 def test_cantidad_negativa_no_se_guarda_y_pide_repetir(
292     client: TestClient, datos_agente: dict,
293 ) -> None:
294     headers = datos_agente["headers"]
295     sid = 782
296     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
297     _turno(client, headers, "confirmo", sesion_id=sid)
298
299     negativo = _turno(client, headers, "hay menos cinco aceites", sesion_id=sid)
300     assert negativo.get("alerta") == "negativo"
301     assert negativo.get("pendiente") is None
302
303     # nada quedo pendiente: un "confirmo" no debe inventar un guardado.
304     confirmar = _turno(client, headers, "confirmo", sesion_id=sid)
305     assert confirmar.get("guardado") is not True
306
307     assert _ultimo_conteo(sid) is None
308
309
310 def test_modos_pedido_extrae_preparacion_y_porciones_sin_bodega(
311     client: TestClient, datos_agente: dict,
312 ) -> None:
313     headers = datos_agente["headers"]
314     r = _turno(client, headers, "hoy preparamos cincuenta ajiacos", sesion_id=-777)
315     assert r["intencion"] == "pedir"
316     assert r["porciones"] == 50
317     assert "ajiacos" in (r.get("preparacion") or "").lower()

```

backend/tests/test_conciliacion.py

68 LÍNEAS

Emparejamiento y alertas.

```

1  """Coincidencias de artículo por texto dicho/escrito (servicios/conciliacion.py)."""
2  from servicios.conciliacion import _cobertura, normalizar, puntuar
3
4  UMBRAL = 45 # el mismo que usa buscar_articulo() para aceptar un candidato
5
6
7  def test_restaurante_no_confunde_con_costilla_de_res():
8      """Decir "restaurante" (buscando una bodega, en la pantalla equivocada)
9      no debe encontrar "COSTILLA DE RES" como si fuera un artículo parecido.
10
11      Antes, la raíz de comparación de 4 letras se quedaba corta cuando la
12      palabra del catálogo tenía menos de 4 letras ("RES"): "RESTAURANTE"
13      empieza igual que "RES" y eso contaba como coincidencia completa, sin
14      que las palabras tengan relación alguna."""
15      consulta = normalizar("restaurante")
16      candidato = normalizar("COSTILLA DE RES")
17      assert _cobertura(consulta, candidato) == 0.0
18      assert puntuar(consulta, candidato) < UMBRAL
19
20
21 def test_una_palabra_corta_del_catalogo_no_hace_de_raiz_para_cualquier_cosa():
22     """Lo mismo, en general: ninguna palabra de 3 letras del catálogo
23     (RES, PAN...) debe servir de "raíz" para aceptar una palabra dicha
24     mucho más larga solo porque empieza igual."""
25     assert _cobertura(normalizar("restaurante"), normalizar("ARROZ RES")) == 0.0
26     assert _cobertura(normalizar("panificado"), normalizar("QUESO PAN")) == 0.0
27
28
29 def test_blanca_sigue_encontrando_blanco():
30     """La raíz de 4 letras entre dos palabras largas (>=4) debe seguir
31     funcionando igual que antes - lo que se corrigió es solo el caso de
32     una palabra del catálogo más corta que la raíz."""
33     consulta = normalizar("tabla para picar blanca")
34     candidato = normalizar("TABLA PARA PICAR BLANCO")
35     assert _cobertura(consulta, candidato) == 100.0
36
37

```

```

38 def test_cazuelas_sigue_encontrando_cazuela():
39     assert _cobertura(normalizar("cazuelas"), normalizar("CAZUELA DE BARRO")) == 100.0
40
41
42 def test_palabra_dicha_de_3_letras_ya_no_encuentra_candidato_largo_por_prefijo():
43     """El caso contrario al de "restaurante": antes, una palabra DICHA de
44     3 letras ("ver", de una frase como "sí para ver el detalle" - no una
45     búsqueda real) encontraba cualquier artículo cuyo nombre EMPEZARA
46     igual ("VERDE", "VERDURAS"), sin relación alguna. Ahora una palabra
47     de 3 letras solo cuenta si coincide EXACTA - "sal" ya no encuentra
48     "SALSA" solo por el prefijo (haría falta decir "sal" tal cual, no
49     "salsa", para que cuente)."""
50     assert _cobertura(normalizar("ver"), normalizar("VERDE")) == 0.0
51     assert _cobertura(normalizar("sal"), normalizar("SALSA DE TOMATE")) == 0.0
52
53
54 def test_bug_real_si_para_ver_el_detalle_no_encuentra_un_articulo_al_azar():
55     """Bug real reportado: decir "sí para ver el detalle" (el eco de la
56     pregunta hablada del selector de bodega, no una búsqueda de
57     artículo) encontraba "PASTILLA LIMPIADOR HORNO VERDE BALDEX150"
58     porque "ver" es prefijo de "VERDE"."""
59     consulta = normalizar("Sí para ver el detalle.")
60     candidato = normalizar("PASTILLA LIMPIADOR HORNO VERDE BALDEX150")
61     assert _cobertura(consulta, candidato) == 0.0
62     assert puntuar(consulta, candidato) < UMBRAL
63
64
65 def test_palabra_dicha_de_4_letras_sigue_encontrando_candidato_largo_por_prefijo():
66     """A partir de 4 letras, el prefijo real sigue contando en los dos
67     sentidos - eso es lo que hace que BLANCA encuentre BLANCO."""
68     assert _cobertura(normalizar("cazu"), normalizar("CAZUELA DE BARRO")) == 100.0

```

backend/tests/test_regresiones_seguridad.py

269 LÍNEAS

Fallos de seguridad que ya ocurrieron una vez.

```

1  """Regresiones de autorización, sesión y legalización de inventario."""
2  from __future__ import annotations
3
4  import pytest
5  from fastapi.testclient import TestClient
6  from starlette.websockets import WebSocketDisconnect
7
8
9  def test_auxiliar_no_puede_abrir_bodega_no_asignada(
10     client: TestClient, datos_regresion: dict[str, object]
11 ) -> None:
12     """Una asignación limita tanto la lista visible como la apertura de bodega."""
13     headers = datos_regresion["headers"]
14
15     listado = client.get("/api/bodegas", headers=headers)
16     assert listado.status_code == 200, listado.text
17     ids_visibles = {fila["id"] for fila in listado.json()}
18     assert datos_regresion["bodega_asignada_id"] in ids_visibles
19     assert datos_regresion["bodega_no_asignada_id"] not in ids_visibles
20
21     respuesta = client.post(
22         "/api/bodegas/abrir",
23         headers=headers,
24         json={"bodega": datos_regresion["bodega_no_asignada"]},
25     )
26     assert respuesta.status_code == 403, respuesta.text
27
28
29 def test_cerrar_todas_las_sesiones_pide_a_cognito_revocar_las_del_usuario_correcto(
30     client: TestClient, datos_regresion: dict[str, object],
31     monkeypatch: pytest.MonkeyPatch,
32 ) -> None:
33     """La clave y el login los maneja Cognito directamente, así que esta
34     ruta ya no puede invalidar un token propio (no hay "token propio") -
35     lo que sigue siendo responsabilidad de este backend, y lo que prueba
36     esto, es pedirle a Cognito (AdminUserGlobalSignOut) que revoque las

```

```

37     sesiones exactamente de quien hizo la llamada, no de otra persona.""
38     import main as modulo
39
40     llamadas = []
41
42     class _ClienteCognitoFalso:
43         def admin_user_global_sign_out(self, UserPoolId, Username):
44             llamadas.append((UserPoolId, Username))
45
46     monkeypatch.setattr(modulo, "_cliente_cognito", lambda: _ClienteCognitoFalso())
47
48     headers = datos_regresion["headers"]
49     r = client.post("/api/usuarios/yo/cerrar-todas", headers=headers)
50     assert r.status_code == 200, r.text
51     assert r.json() == {"ok": True}
52     assert llamadas == [(modulo.COGNITO_USER_POOL_ID, "luis")]
53
54
55 def test_cerrar_todas_las_sesiones_devuelve_502_si_cognito_falla(
56     client: TestClient, datos_regresion: dict[str, object],
57     monkeypatch: pytest.MonkeyPatch,
58 ) -> None:
59     """Si Cognito no responde, se avisa con un error - no se calla el
60     fallo como si las sesiones sí se hubieran cerrado."""
61     import main as modulo
62
63     class _ClienteCognitoFalso:
64         def admin_user_global_sign_out(self, UserPoolId, Username):
65             raise RuntimeError("Cognito no disponible")
66
67     monkeypatch.setattr(modulo, "_cliente_cognito", lambda: _ClienteCognitoFalso())
68
69     headers = datos_regresion["headers"]
70     r = client.post("/api/usuarios/yo/cerrar-todas", headers=headers)
71     assert r.status_code == 502, r.text
72
73
74 def test_auxiliar_no_puede_ver_detalle_de_bodega ajena(
75     client: TestClient, datos_regresion: dict[str, object]
76 ) -> None:
77     """La asignación también protege el detalle/firmas/inventario, no solo
78     el listado: antes se podía pedir estos datos directo por bodega_id sin
79     que la asignación se revisara para nada."""
80     headers = datos_regresion["headers"]
81     ajena = datos_regresion["bodega_no_asignada_id"]
82
83     for ruta in (f"/api/bodegas/{ajena}/detalle", f"/api/bodegas/{ajena}/firmas",
84                 f"/api/bodegas/{ajena}/articulos"):
85         r = client.get(ruta, headers=headers)
86         assert r.status_code == 403, f"{ruta}: {r.text}"
87
88     propia = datos_regresion["bodega_asignada_id"]
89     r = client.get(f"/api/bodegas/{propia}/detalle", headers=headers)
90     assert r.status_code == 200, r.text
91
92
93 def test_auxiliar_no_puede_abrir_bodega ajena_por_voz(
94     client: TestClient, datos_regresion: dict[str, object],
95     monkeypatch: pytest.MonkeyPatch,
96 ) -> None:
97     """El mismo límite de asignación que /api/bodegas/abrir aplica al pedir
98     lo mismo por el agente conversacional (/api/agente/turno) - antes
99     bastaba con dictar el nombre de la bodega para saltárselo. Se fuerza
100     el intérprete local (sin Gemini) para que el resultado no dependa de
101     cómo un modelo externo decida frasear "bodega_texto" ese día - lo que
102     se prueba aquí es la autorización, no la interpretación del lenguaje."""
103     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
104     import agente.cerebro as cerebro
105     monkeypatch.setattr(cerebro, "_cliente", None)
106
107     headers = datos_regresion["headers"]
108     nombre_ajena = datos_regresion["bodega_no_asignada"]
109
110     r = client.post("/api/agente/turno", headers=headers,

```



```

111         json={"texto": f"iniciar conteo en {nombre_ajena}", "sesion_id": 999})
112     assert r.status_code == 200, r.text
113     datos = r.json()
114     assert datos.get("bodega") is None
115     assert "no esta asignada" in datos["respuesta_hablada"].lower() \
116         or "no está asignada" in datos["respuesta_hablada"].lower()
117
118
119 def test_websocket_rechaza_conexion_sin_token(client: TestClient) -> None:
120     """El tablero en vivo no puede exponer estado de bodegas sin identidad."""
121     with pytest.raises(WebSocketDisconnect) as error:
122         with client.websocket_connect("/api/bodegas/estado") as websocket:
123             websocket.receive_json()
124
125     assert error.value.code == 1008
126
127
128 def _entrar(client: TestClient, nombre: str) -> dict[str, str]:
129     r = client.post("/api/ingresar", data={"username": nombre, "password": "StockXperts1"})
130     assert r.status_code == 200, r.text
131     return {"Authorization": f"Bearer {r.json()['token']}"}
132
133
134 def _asignar(nombre: str, bodega_id: int) -> None:
135     from bd import Sesion
136     from modelos import AsignacionBodega, Usuario
137
138     with Sesion() as s:
139         persona = s.query(Usuario).filter_by(nombre=nombre).one()
140         s.add(AsignacionBodega(usuario_id=persona.id, bodega_id=bodega_id))
141         s.commit()
142
143
144 def test_administrador_sin_bodegas_asignadas_sigue_alcanzando_todo_el_parque(
145     client: TestClient, datos_regresion: dict[str, object]
146 ) -> None:
147     """El administrador general no tiene zona propia: manda la asignación,
148     y sin asignaciones no hay nada que lo limite. Esto es lo que NO debe
149     romperse al agregar el administrador de sede - en la demo diana no
150     tiene ninguna bodega asignada."""
151     headers = _entrar(client, "diana")
152     ajena = datos_regresion["bodega_no_asignada_id"]
153
154     for ruta in (f"/api/bodegas/{ajena}/detalle",
155                 f"/api/bodegas/{ajena}/articulos",
156                 f"/api/bodegas/{ajena}/firmas"):
157         r = client.get(ruta, headers=headers)
158         assert r.status_code != 403, f"{ruta} -> {r.text}"
159
160     r = client.post(f"/api/bodegas/{ajena}/auditoria/iniciar", headers=headers)
161     assert r.status_code == 200, r.text
162
163
164 def test_administrador_de_sede_no_alcanza_la_auditoria_de_otra_sede(
165     client: TestClient, datos_regresion: dict[str, object]
166 ) -> None:
167     """Un administrador CON bodegas asignadas queda limitado a esas: sin
168     esto, al crecer a varias sedes el administrador de una podía iniciar,
169     firmar o cerrar la auditoría de la otra con solo cambiar el número en
170     la URL, aunque el tablero (?propias=1) ya no se la mostrara."""
171     propia = datos_regresion["bodega_asignada_id"]
172     ajena = datos_regresion["bodega_no_asignada_id"]
173     _asignar("diana", propia)
174     headers = _entrar(client, "diana")
175
176     ajenas = [
177         ("get", f"/api/bodegas/{ajena}/detalle"),
178         ("get", f"/api/bodegas/{ajena}/articulos"),
179         ("get", f"/api/bodegas/{ajena}/firmas"),
180         ("post", f"/api/bodegas/{ajena}/auditoria/iniciar"),
181         ("get", f"/api/bodegas/{ajena}/auditoria/comparar"),
182         ("post", f"/api/bodegas/{ajena}/auditoria/firmar"),
183         ("post", f"/api/bodegas/{ajena}/cerrar"),
184         ("post", f"/api/bodegas/{ajena}/exportar-detalle"),

```

```

185     ]
186     for metodo, ruta in ajenas:
187         r = getattr(client, metodo)(ruta, headers=headers)
188         assert r.status_code == 403, f"{ruta} devolvio {r.status_code}: {r.text}"
189
190     r = client.post(f"/api/bodegas/{ajena}/reabrir", headers=headers,
191                   json={"motivo": "prueba"})
192     assert r.status_code == 403, r.text
193
194     # y la suya sigue funcionando igual que antes
195     r = client.get(f"/api/bodegas/{propia}/detalle", headers=headers)
196     assert r.status_code == 200, r.text
197     r = client.post(f"/api/bodegas/{propia}/auditoria/iniciar", headers=headers)
198     assert r.status_code == 200, r.text
199
200
201 def test_administrador_de_sede_no_ve_bodegas_ajenas_al_buscar_por_voz(
202     client: TestClient, datos_regresion: dict[str, object]
203 ) -> None:
204     """La voz no debe ofrecer lo que la puerta va a rechazar: antes, decir
205     el nombre de una bodega de otra sede la encontraba igual."""
206     _asignar("diana", datos_regresion["bodega_asignada_id"])
207     headers = _entrar(client, "diana")
208
209     r = client.post("/api/bodegas/abrir", headers=headers,
210                   json={"bodega": datos_regresion["bodega_no_asignada"]})
211     assert r.status_code in (403, 404), r.text
212
213
214 def test_administrador_de_sede_no_puede_asignarse_bodegas_de_otra_sede(
215     client: TestClient, datos_regresion: dict[str, object]
216 ) -> None:
217     """Sin esto el límite de sede era decorativo: bastaba con asignarse a
218     sí mismo las bodegas ajenas para volver a alcanzar todo el parque."""
219     from bd import Sesion
220     from modelos import AsignacionBodega, Usuario
221
222     propia = datos_regresion["bodega_asignada_id"]
223     ajena = datos_regresion["bodega_no_asignada_id"]
224     _asignar("diana", propia)
225     headers = _entrar(client, "diana")
226
227     with Sesion() as s:
228         diana_id = s.query(Usuario).filter_by(nombre="diana").one().id
229
230     r = client.put(f"/api/usuarios/{diana_id}/bodegas", headers=headers,
231                  json={"bodega_ids": [propia, ajena]})
232     assert r.status_code == 200, r.text
233
234     with Sesion() as s:
235         quedaron = {a.bodega_id for a in
236                     s.query(AsignacionBodega).filter_by(usuario_id=diana_id).all()}
237     assert quedaron == {propia}, f"se coló una bodega ajena: {quedaron}"
238
239     r = client.get(f"/api/bodegas/{ajena}/detalle", headers=headers)
240     assert r.status_code == 403, r.text
241
242
243 def test_administrador_de_sede_no BORRA las asignaciones de otra sede(
244     client: TestClient, datos_regresion: dict[str, object]
245 ) -> None:
246     """Dos administradores de sedes distintas pueden repartirle bodegas a
247     la misma persona: el de una no debe borrar el trabajo del otro al
248     guardar su propia lista."""
249     from bd import Sesion
250     from modelos import AsignacionBodega, Usuario
251
252     propia = datos_regresion["bodega_asignada_id"]
253     ajena = datos_regresion["bodega_no_asignada_id"]
254     auxiliar_id = datos_regresion["auxiliar_id"]
255
256     _asignar("diana", propia)
257     with Sesion() as s:
258         # el auxiliar ya trabaja en la otra sede
259         s.add(AsignacionBodega(usuario_id=auxiliar_id, bodega_id=ajena))

```

```

259         s.commit()
260
261     headers = _entrar(client, "diana")
262     r = client.put(f"/api/usuarios/{auxiliar_id}/bodegas", headers=headers,
263                  json={"bodega_ids": [propia]})
264     assert r.status_code == 200, r.text
265
266     with Sesion() as s:
267         quedaron = {a.bodega_id for a in
268                     s.query(AsignacionBodega).filter_by(usuario_id=auxiliar_id).all()}
269     assert quedaron == {propia, ajena}, f"se perdio la otra sede: {quedaron}"

```

backend/tests/test_regresiones_legalizacion.py

56 LÍNEAS

Fallos de legalizacion que ya ocurrieron una vez.

```

1  """Regresiones para evitar alteraciones repetidas o cruzadas de inventario."""
2  from __future__ import annotations
3
4  from fastapi.testclient import TestClient
5
6
7  def _stock(codigo: str, bodega_id: int) -> float:
8      from bd import Sesion
9      from modelos import StockSistema
10
11      with Sesion() as sesion:
12          fila = sesion.query(StockSistema).filter_by(
13              articulo_codigo=codigo, bodega_id=bodega_id
14          ).one()
15          return float(fila.cantidad_sd)
16
17
18  def test_legalizacion_es_idempotente_y_afecta_solo_su_bodega(
19      client: TestClient, datos_regresion: dict[str, object]
20  ) -> None:
21      """Reintentar una legalización no duplica el sobrante en ninguna bodega."""
22      from bd import Sesion
23      from modelos import LineaServicio
24
25      servicio_id = 777
26      codigo = str(datos_regresion["articulo_codigo"])
27      bodega_origen = int(datos_regresion["bodega_asignada_id"])
28      otra_bodega = int(datos_regresion["bodega_no_asignada_id"])
29      headers = datos_regresion["headers"]
30
31      with Sesion() as sesion:
32          sesion.add(LineaServicio(
33              servicio_id=servicio_id,
34              articulo_codigo=codigo,
35              nombre="ARTICULO REGRESION",
36              pedido=10,
37              usado=0,
38              bodega_id=bodega_origen,
39          ))
40      sesion.commit()
41
42      payload = {
43          "servicio_id": servicio_id,
44          "usos": [{"codigo": codigo, "usado": 7}],
45      }
46      primera = client.post("/api/legalizacion/confirmar", headers=headers, json=payload)
47      assert primera.status_code == 200, primera.text
48      assert _stock(codigo, bodega_origen) == 103
49      assert _stock(codigo, otra_bodega) == 200
50
51      segunda = client.post("/api/legalizacion/confirmar", headers=headers, json=payload)
52      # Un reintento puede responder éxito ya aplicado o conflicto; en ambos casos
53      # la condición de idempotencia es que el inventario permanezca intacto.
54      assert segunda.status_code in (200, 409), segunda.text
55      assert _stock(codigo, bodega_origen) == 103
56      assert _stock(codigo, otra_bodega) == 200

```

frontend/src/interpreteLocal.test.js

139 LÍNEAS

El interprete del navegador.

```

1 // Regresiones del intérprete local en JS: los mismos casos que
2 // backend/tests/test_interprete.py, para confirmar que el puerto a
3 // JavaScript (interpreteLocal.js) se comporta igual que el original en
4 // Python que corre en el servidor. Corre con: node --test src/*.test.js
5 import { test } from "node:test";
6 import assert from "node:assert/strict";
7 import { interpretarLocal, normalizarUnidad } from "../interpreteLocal.js";
8
9 // — números —
10 test("numero digito simple", () => {
11   assert.equal(interpretarLocal("hay 12 arroces").cantidad, 12);
12 });
13 test("numero decimal con punto", () => {
14   assert.equal(interpretarLocal("hay 2.5 kilos de arroz").cantidad, 2.5);
15 });
16 test("numero decimal con coma", () => {
17   assert.equal(interpretarLocal("hay 2,5 kilos de arroz").cantidad, 2.5);
18 });
19 test("numero negativo con menos", () => {
20   assert.equal(interpretarLocal("hay menos 5 cazuelas").cantidad, -5);
21 });
22 test("numero en palabras simple", () => {
23   assert.equal(interpretarLocal("hay cinco cazuelas").cantidad, 5);
24 });
25 test("numero compuesto decena y unidad", () => {
26   assert.equal(interpretarLocal("hay treinta y cinco tablas").cantidad, 35);
27 });
28 test("numero compuesto centena y decena", () => {
29   assert.equal(interpretarLocal("hay ciento ochenta unidades").cantidad, 180);
30 });
31 test("numero en palabras mil", () => {
32   assert.equal(interpretarLocal("hay mil servilletas").cantidad, 1000);
33 });
34 test("numero en palabras con menos", () => {
35   assert.equal(interpretarLocal("hay menos cinco cazuelas").cantidad, -5);
36 });
37 test("sin numero ni palabra clave cae en ayuda", () => {
38   const r = interpretarLocal("no logro ver nada aqui");
39   assert.equal(r.intencion, "ayuda");
40   assert.equal(r.cantidad ?? null, null);
41 });
42
43 // — unidades —
44 test("unidad kilo variantes", () => {
45   for (const palabra of ["kilo", "kilos", "kilogramo", "kilogramos", "kg"]) {
46     assert.equal(interpretarLocal(`hay 3 ${palabra} de arroz`).unidad, "Kilogram");
47   }
48 });
49 test("unidad litro variantes", () => {
50   for (const palabra of ["litro", "litros", "l"]) {
51     assert.equal(interpretarLocal(`hay 3 ${palabra} de aceite`).unidad, "Liter");
52   }
53 });
54 test("unidad no reconocida es null", () => {
55   assert.equal(interpretarLocal("hay 3 cazuelas blancas").unidad, null);
56 });
57 test("normalizarUnidad pasa canonicas intactas", () => {
58   assert.equal(normalizarUnidad("Kilogram"), "Kilogram");
59 });
60 test("normalizarUnidad traduce lo dicho", () => {
61   assert.equal(normalizarUnidad("kilos"), "Kilogram");
62   assert.equal(normalizarUnidad("litros"), "Liter");
63   assert.equal(normalizarUnidad("unidades"), "Unidad");
64   assert.equal(normalizarUnidad("porciones"), "Portion");
65 });
66 test("normalizarUnidad desconocida se devuelve igual", () => {
67   assert.equal(normalizarUnidad("bultos"), "bultos");
68 });
69 test("normalizarUnidad vacia", () => {

```

```

70  assert.equal(normalizarUnidad(null), null);
71  assert.equal(normalizarUnidad(""), "");
72  });
73
74  // — intenciones —
75  test("intencion navegar iniciar conteo", () => {
76    const r = interpretarLocal("iniciar conteo en almacen ayb");
77    assert.equal(r.intencion, "navegar");
78  });
79  test("intencion navegar abrir bodega", () => {
80    assert.equal(interpretarLocal("abrir bodega restaurante fuentes").intencion, "navegar");
81  });
82  test("intencion pedir con porciones", () => {
83    const r = interpretarLocal("hoy preparamos cincuenta ajiacos");
84    assert.equal(r.intencion, "pedir");
85    assert.equal(r.porciones, 50);
86  });
87  test("intencion pedir sin porciones queda en cero", () => {
88    const r = interpretarLocal("vamos a preparar sanccho");
89    assert.equal(r.intencion, "pedir");
90    assert.equal(r.porciones, 0);
91  });
92  test("intencion confirmar variantes", () => {
93    for (const frase of ["confirmo", "sí", "si", "correcto"]) {
94      assert.equal(interpretarLocal(frase).intencion, "confirmar");
95    }
96  });
97  test("intencion corregir variantes", () => {
98    for (const frase of ["no son esos, corregir", "uy no, esta mal", "cambiar la cantidad"]) {
99      assert.equal(interpretarLocal(frase).intencion, "corregir");
100    }
101  });
102  test("intencion consultar", () => {
103    const r = interpretarLocal("cuanto arroz hay");
104    assert.equal(r.intencion, "consultar");
105    assert.ok(r.articulo_texto.includes("arroz"));
106  });
107  test("intencion reporte", () => {
108    for (const frase of ["generame el reporte", "el consolidado por favor", "imprimeme el archivo"]) {
109      assert.equal(interpretarLocal(frase).intencion, "reporte");
110    }
111  });
112  test("intencion ayuda explicita", () => {
113    for (const frase of ["ayuda", "no entiendo"]) {
114      assert.equal(interpretarLocal(frase).intencion, "ayuda");
115    }
116  });
117  test("intencion contar con cantidad y articulo", () => {
118    const r = interpretarLocal("hay noventa cazuelas blancas");
119    assert.equal(r.intencion, "contar");
120    assert.equal(r.cantidad, 90);
121  });
122  test("intencion desconocida pide repetir", () => {
123    const r = interpretarLocal("xyzyz blorp qwerty");
124    assert.equal(r.intencion, "ayuda");
125    assert.ok(r.respuesta_hablada.toLowerCase().includes("repite"));
126  });
127  test("articulo_texto sin ruido quita palabras vacias", () => {
128    const r = interpretarLocal("hay noventa cazuelas blancas");
129    const palabras = r.articulo_texto.split(" ");
130    assert.ok(!palabras.includes(""));
131    assert.ok(!palabras.includes("noventa"));
132  });
133
134  // — casos propios del puerto a JS (ambigüedad centena/decena real) —
135  test("arroz vs arroz basmati: frase completa se limpia igual que en Python", () => {
136    const r = interpretarLocal("hay noventa arroces");
137    assert.equal(r.cantidad, 90);
138    assert.ok(r.articulo_texto.includes("arroces"));
139  });

```

frontend/src/colaOffline.test.js

47 LÍNEAS

La cola sin conexion.

```

1 // Corre con: node --test src/*.test.js
2 // node:test no trae localStorage/window (a diferencia del navegador) -
3 // se simulan aquí mismo, con lo mínimo que colaOffline.js necesita, en vez
4 // de meter jsdom solo para esto.
5 import { test } from "node:test";
6 import assert from "node:assert/strict";
7
8 const almacen = new Map();
9 globalThis.localStorage = {
10   getItem: (k) => (almacen.has(k) ? almacen.get(k) : null),
11   setItem: (k, v) => almacen.set(k, v),
12 };
13 let eventosDisparados = [];
14 globalThis.window = {
15   dispatchEvent: (e) => eventosDisparados.push(e.type),
16 };
17 globalThis.Event = class { constructor(tipo) { this.type = tipo; } };
18
19 const { leerCola, guardarCola, CLAVE_COLA, EVENTO_COLA } = await import("./colaOffline.js");
20
21 test("leerCola devuelve [] cuando no hay nada guardado", () => {
22   assert.deepEqual(leerCola("nueva-sesion"), []);
23 });
24
25 test("leerCola devuelve [] si lo guardado no es JSON válido (corrupción de localStorage)", () => {
26   almacen.set(CLAVE_COLA("rota"), "{no es json}");
27   assert.deepEqual(leerCola("rota"), []);
28 });
29
30 test("guardarCola y leerCola hacen ida y vuelta con los mismos datos", () => {
31   const items = [{ articulo: "ARROZ", cantidad: 5, unidad: "Kilogram" }];
32   guardarCola("s1", items);
33   assert.deepEqual(leerCola("s1"), items);
34 });
35
36 test("guardarCola dispara el evento para que otras pantallas se enteren", () => {
37   eventosDisparados = [];
38   guardarCola("s2", []);
39   assert.ok(eventosDisparados.includes(EVENTO_COLA));
40 });
41
42 test("la cola es independiente por sesión", () => {
43   guardarCola("sesion-a", [{ articulo: "A" }]);
44   guardarCola("sesion-b", [{ articulo: "B" }]);
45   assert.deepEqual(leerCola("sesion-a"), [{ articulo: "A" }]);
46   assert.deepEqual(leerCola("sesion-b"), [{ articulo: "B" }]);
47 });

```

frontend/src/accesibilidad.test.js

44 LÍNEAS

Contraste y tamaño de letra.

```

1 // Corre con: node --test src/*.test.js
2 // node:test no trae localStorage/document (a diferencia del navegador) -
3 // se simulan aquí mismo con lo mínimo que accesibilidad.js necesita.
4 import { test } from "node:test";
5 import assert from "node:assert/strict";
6
7 const almacen = new Map();
8 globalThis.localStorage = {
9   getItem: (k) => (almacen.has(k) ? almacen.get(k) : null),
10   setItem: (k, v) => almacen.set(k, v),
11 };
12 const atributos = {};
13 globalThis.document = {
14   documentElement: {
15     setAttribute: (k, v) => { atributos[k] = v; },

```

```

16  },
17  };
18
19  const { leerPreferencias, guardarPreferencias, aplicarPreferencias } =
20    await import("./accesibilidad.js");
21
22  test("leerPreferencias devuelve {} cuando no hay nada guardado", () => {
23    assert.deepEqual(leerPreferencias(), {});
24  });
25
26  test("leerPreferencias devuelve {} si lo guardado no es JSON válido", () => {
27    almacen.set("cv_accesibilidad", "no-es-json");
28    assert.deepEqual(leerPreferencias(), {});
29    almacen.delete("cv_accesibilidad");
30  });
31
32  test("guardarPreferencias y leerPreferencias hacen ida y vuelta", () => {
33    guardarPreferencias({ contraste: true, tamano: "grande" });
34    assert.deepEqual(leerPreferencias(), { contraste: true, tamano: "grande" });
35  });
36
37  test("aplicarPreferencias pone data-contraste=alto solo si contraste es true", () => {
38    aplicarPreferencias({ contraste: true, tamano: "grande" });
39    assert.equal(atributos["data-contraste"], "alto");
40    assert.equal(atributos["data-tamano"], "grande");
41    aplicarPreferencias({ contraste: false, tamano: "" });
42    assert.equal(atributos["data-contraste"], "");
43    assert.equal(atributos["data-tamano"], "");
44  });

```

frontend/src/confirmacionVoz.test.js

71 LÍNEAS

Que cuenta como un si.

```

1  // Bug real, encontrado en producción: "sí" (como lo transcribe la voz,
2  // CON tilde) nunca hacía match contra /^(si|sí|...)\b/ porque en
3  // JavaScript \b y \w son ASCII puros - "í" no cuenta como letra, así que
4  // la frontera de palabra justo después de la tilde no se reconocía.
5  // Corre con: node --test src/*.test.js
6  import { test } from "node:test";
7  import assert from "node:assert/strict";
8  import { quitarTildes, esAfirmacion, esNegacion } from "../confirmacionVoz.js";
9
10 test("quitarTildes deja sí como si", () => {
11   assert.equal(quitarTildes("sí"), "si");
12 });
13 test("quitarTildes no toca palabras sin tilde", () => {
14   assert.equal(quitarTildes("confirmo"), "confirmo");
15 });
16
17 test("si con tilde SI confirma (el bug real reportado)", () => {
18   assert.equal(esAfirmacion("sí"), true);
19 });
20 test("Si con tilde y mayuscula y punto, como lo entrega la voz", () => {
21   assert.equal(esAfirmacion("Sí."), true);
22 });
23 test("si sin tilde tambien confirma", () => {
24   assert.equal(esAfirmacion("si"), true);
25 });
26 test("confirmo confirma", () => {
27   assert.equal(esAfirmacion("confirmo"), true);
28 });
29 test("asi es (con y sin tilde) confirma", () => {
30   assert.equal(esAfirmacion("así es"), true);
31   assert.equal(esAfirmacion("asi es"), true);
32 });
33 test("una palabra extra del contexto (ver) confirma solo si se pasa", () => {
34   assert.equal(esAfirmacion("ver"), false);
35   assert.equal(esAfirmacion("ver", ["ver"]), true);
36 });
37 test("una conjugacion del verbo extra (el texto del boton) tambien confirma", () => {
38   // Bug real reportado: el boton de un diálogo decía "Enviar", pero solo

```

```

39 // la palabra EXACTA "enviar" contaba - decir "envíalo" (como diría una
40 // persona real) no calzaba con nada y quedaba en "no le entendí".
41 assert.equal(esAfirmacion("envíalo", ["Enviar"]), true);
42 assert.equal(esAfirmacion("Enviar.", ["Enviar"]), true);
43 assert.equal(esAfirmacion("envíelo por favor", ["Enviar"]), true);
44 assert.equal(esAfirmacion("mándalo", ["Enviar"]), false); // no es conjugación de "enviar"
45 });
46 test("una palabra sin relacion con el extra no confirma por casualidad", () => {
47   assert.equal(esAfirmacion("entonces qué", ["Enviar"]), false);
48   assert.equal(esAfirmacion("no envíes nada", ["Enviar"]), false); // "no" niega primero
49 });
50 test("no NO confirma aunque diga sí en otra parte de la frase", () => {
51   assert.equal(esAfirmacion("no, mejor sí despues"), false);
52 });
53 test("una frase sin nada reconocible no confirma", () => {
54   assert.equal(esAfirmacion("no le entendí"), false);
55   assert.equal(esAfirmacion("kiosco taquilla ayb"), false);
56 });
57 test("vacío no confirma", () => {
58   assert.equal(esAfirmacion(""), false);
59 });
60
61 test("no limpio niega", () => {
62   assert.equal(esNegacion("no"), true);
63   assert.equal(esNegacion("No."), true);
64 });
65 test("no en medio de la frase no cuenta como negacion limpia", () => {
66   assert.equal(esNegacion("no se, tal vez"), true); // "no" es la primera palabra
67   assert.equal(esNegacion("mejor no"), false);      // "no" no es la primera palabra
68 });
69 test("si no niega", () => {
70   assert.equal(esNegacion("sí"), false);
71 });

```

frontend/src/tutorial.test.js

32 LÍNEAS

Cuando se abre el recorrido.

```

1 // Corre con: node --test src/*.test.js
2 import { test } from "node:test";
3 import assert from "node:assert/strict";
4
5 const almacen = new Map();
6 globalThis.localStorage = {
7   getItem: (k) => (almacen.has(k) ? almacen.get(k) : null),
8   setItem: (k, v) => almacen.set(k, v),
9   removeItem: (k) => almacen.delete(k),
10 };
11
12 const { debeAbrirTutorial, deshabilitarTutorial, habilitarTutorial } =
13   await import("../tutorial.js");
14
15 test("debeAbrirTutorial es true por defecto (nadie lo ha desactivado)", () => {
16   assert.equal(debeAbrirTutorial(11), true);
17 });
18
19 test("deshabilitarTutorial hace que debeAbrirTutorial pase a false, solo para ESE usuario", () => {
20   deshabilitarTutorial(11);
21   assert.equal(debeAbrirTutorial(11), false);
22   // otro usuario en el mismo dispositivo (tablet compartida en bodega)
23   // debe seguir viendo el video: la desactivacion es por persona.
24   assert.equal(debeAbrirTutorial(22), true);
25 });
26
27 test("habilitarTutorial revierte la desactivacion", () => {
28   deshabilitarTutorial(33);
29   assert.equal(debeAbrirTutorial(33), false);
30   habilitarTutorial(33);
31   assert.equal(debeAbrirTutorial(33), true);
32 });

```


backend/tests/test_crear_bodega.py

349 LÍNEAS

Alta de bodegas y su aprobacion.

```

1  """Regresión de «crear bodega nueva»: un auxiliar la solicita, queda
2  pendiente de aprobación (igual que un producto nuevo), y solo existe de
3  verdad - visible y abrible - una vez que el administrador la aprueba."""
4  from __future__ import annotations
5
6  from fastapi.testclient import TestClient
7
8
9  def _ingresar(client: TestClient, usuario: str) -> dict[str, str]:
10     r = client.post("/api/ingresar", data={"username": usuario, "password": "StockXperts1"})
11     assert r.status_code == 200, r.text
12     return {"Authorization": f"Bearer {r.json()['token']}"}
13
14
15  def test_solicitar_bodega_nueva_queda_pendiente_y_no_es_visible_aun(
16     client: TestClient,
17 ) -> None:
18     headers_luis = _ingresar(client, "luis")
19
20     antes = client.get("/api/bodegas", headers=headers_luis)
21     assert "ALMACEN NUEVO PISCILAGO" not in [b["bodega"] for b in antes.json()]
22
23     solicitud = client.post("/api/bodegas/crear-pendiente", headers=headers_luis,
24                             json={"nombre": "almacen nuevo piscilago"})
25     assert solicitud.status_code == 200, solicitud.text
26     assert "pendiente" in solicitud.json()["respuesta_hablada"].lower()
27
28     # todavia no existe como bodega real: sigue sin aparecer en el listado
29     despues = client.get("/api/bodegas", headers=headers_luis)
30     assert "ALMACEN NUEVO PISCILAGO" not in [b["bodega"] for b in despues.json()]
31
32
33  def test_solicitar_bodega_nueva_quita_el_punto_del_reconocimiento_de_voz(
34     client: TestClient,
35 ) -> None:
36     """Bug real reportado desde producción: el reconocimiento de voz
37     cierra la frase con un punto ("Juguetería.") que nadie dijo, y
38     quedaba guardado tal cual en el catálogo para siempre."""
39     headers_luis = _ingresar(client, "luis")
40     r = client.post("/api/bodegas/crear-pendiente", headers=headers_luis,
41                     json={"nombre": "Juguetería."})
42     assert r.status_code == 200, r.text
43
44     headers_diana = _ingresar(client, "diana")
45     pendientes = client.get("/api/aprobaciones?estado=pendiente", headers=headers_diana).json()
46     nombres = [a["nombre"] for a in pendientes if a["tipo"] == "bodega"]
47     assert "JUGUETERÍA" in nombres
48     assert "JUGUETERÍA." not in nombres
49
50
51  def test_aprobar_bodega_nueva_la_crea_de_verdad_y_se_puede_abrir(
52     client: TestClient,
53 ) -> None:
54     headers_luis = _ingresar(client, "luis")
55     headers_diana = _ingresar(client, "diana")
56
57     client.post("/api/bodegas/crear-pendiente", headers=headers_luis,
58                 json={"nombre": "almacen nuevo piscilago"})
59
60     pendientes = client.get("/api/aprobaciones?estado=pendiente", headers=headers_diana)
61     assert pendientes.status_code == 200, pendientes.text
62     solicitudes = [a for a in pendientes.json() if a["tipo"] == "bodega"]
63     assert len(solicitudes) == 1
64     assert solicitudes[0]["nombre"] == "ALMACEN NUEVO PISCILAGO"
65
66     aprobar = client.post(f"/api/aprobaciones/{solicitudes[0]['id']}/aprobar",
67                           headers=headers_diana)
68     assert aprobar.status_code == 200, aprobar.text
69

```

```

70     listado = client.get("/api/bodegas", headers=headers_diana)
71     nombres = [b["bodega"] for b in listado.json()]
72     assert "ALMACEN NUEVO PISCILAGO" in nombres
73
74     # diana es auditora: sin asignacion previa, igual puede abrirla y contar
75     abrir = client.post("/api/bodegas/abrir", headers=headers_diana,
76                        json={"bodega": "ALMACEN NUEVO PISCILAGO"})
77     assert abrir.status_code == 200, abrir.text
78
79
80 def test_administrador_crea_bodega_directo_sin_pasar_por_aprobacion(
81     client: TestClient,
82 ) -> None:
83     """Antes, el administrador quedaba pidiendo su propia bodega y solo el
84     mismo podia aprobarla (visto en produccion) - una aprobacion que no
85     verifica nada. Ahora, para el auditor, la bodega se crea de una."""
86     headers_diana = _ingresar(client, "diana")
87
88     solicitud = client.post("/api/bodegas/crear-pendiente", headers=headers_diana,
89                           json={"nombre": "bodega de la administradora"})
90     assert solicitud.status_code == 200, solicitud.text
91     assert "pendiente" not in solicitud.json()["respuesta_hablada"].lower()
92
93     # ya existe de verdad, sin pasar por aprobaciones
94     listado = client.get("/api/bodegas", headers=headers_diana)
95     assert "BODEGA DE LA ADMINISTRADORA" in [b["bodega"] for b in listado.json()]
96
97     pendientes = client.get("/api/aprobaciones?estado=pendiente", headers=headers_diana)
98     assert not any(a["tipo"] == "bodega" for a in pendientes.json())
99
100
101 def test_buscar_bodega_resuelve_al_nombre_real_sin_abrir_nada(
102     client: TestClient,
103 ) -> None:
104     """Bug real: el selector de bodega preguntaba "¿confirma que abro
105     kiosco?" con lo que se dijo tal cual, aunque "KIOSCO" a secas no
106     fuera ninguna bodega real (solo existen "KIOSCO TAQUILLA AYB",
107     "KIOSCO PISCIGIROS AYB", etc.) - /api/bodegas/buscar debe resolver al
108     nombre real (o decir que no encontró nada) SIN crear ninguna sesión."""
109     from bd import Sesion
110     from modelos import Bodega, Usuario, AsignacionBodega, SesionConteo
111
112     with Sesion() as sesion:
113         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
114         bodega = Bodega(nombre_oficial="KIOSCO TAQUILLA REGIONAL")
115         sesion.add(bodega)
116         sesion.flush()
117         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
118         sesion.commit()
119         bodega_id = bodega.id
120
121     headers = _ingresar(client, "luis")
122
123     r = client.post("/api/bodegas/buscar", headers=headers,
124                   json={"bodega": "kiosco taquilla regional"})
125     assert r.status_code == 200, r.text
126     assert r.json() == {"encontrada": True, "bodega": "KIOSCO TAQUILLA REGIONAL",
127                        "bodega_id": bodega_id, "opciones": []}
128
129     r2 = client.post("/api/bodegas/buscar", headers=headers,
130                     json={"bodega": "un nombre que no existe para nada"})
131     assert r2.status_code == 200, r2.text
132     assert r2.json() == {"encontrada": False, "bodega": None, "bodega_id": None, "opciones": []}
133
134     # solo resolvió el nombre - no abrió ninguna sesión de conteo
135     with Sesion() as sesion:
136         assert sesion.query(SesionConteo).filter_by(bodega_id=bodega_id).count() == 0
137
138
139 def test_buscar_bodega_ofrece_opciones_en_vez_de_adivinar(
140     client: TestClient,
141 ) -> None:
142     """Bug real reportado desde producción: decir "kiosco" a secas
143     "encontraba" (mal) "KIOSCO 2 SUMINISTROS" - "kiosco" es substring de

```

```

144     esa Y de "KIOSCO TAQUILLA AYB" por igual, así que ninguna gana sola.
145     En vez de solo decir "no la encuentro", ahora ofrece las dos como
146     opciones para que la persona elija cuál era.""
147     from bd import Sesion
148     from modelos import Bodega, Usuario, AsignacionBodega
149
150     with Sesion() as sesion:
151         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
152         a = Bodega(nombre_oficial="KIOSCO DOS SUMINISTROS")
153         b = Bodega(nombre_oficial="KIOSCO TAQUILLA AYB")
154         sesion.add_all([a, b])
155         sesion.flush()
156         sesion.add_all([
157             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=a.id),
158             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=b.id),
159         ])
160         sesion.commit()
161         ids = {a.id, b.id}
162
163     headers = _ingresar(client, "luis")
164     r = client.post("/api/bodegas/buscar", headers=headers, json={"bodega": "kiosco"})
165     assert r.status_code == 200, r.text
166     d = r.json()
167     assert d["encontrada"] is False
168     assert d["bodega"] is None
169     assert {o["bodega_id"] for o in d["opciones"]} == ids
170     assert {o["bodega"] for o in d["opciones"]} == {"KIOSCO DOS SUMINISTROS",
171                                                    "KIOSCO TAQUILLA AYB"}
172
173
174 def test_buscar_bodega_restaurante_ofrece_las_dos_reales(
175     client: TestClient,
176 ) -> None:
177     """Mismo caso que reportó la usuaria en vivo: decir "restaurante" a
178     secas, con "RESTAURANTE FUENTES AYB" y "RESTAURANTE FUENTES SUMIN"
179     en el catálogo, debe ofrecer ambas - no fallar en seco."""
180     from bd import Sesion
181     from modelos import Bodega, Usuario, AsignacionBodega
182
183     with Sesion() as sesion:
184         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
185         ayb = Bodega(nombre_oficial="RESTAURANTE FUENTES AYB")
186         sumin = Bodega(nombre_oficial="RESTAURANTE FUENTES SUMIN")
187         sesion.add_all([ayb, sumin])
188         sesion.flush()
189         sesion.add_all([
190             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=ayb.id),
191             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=sumin.id),
192         ])
193         sesion.commit()
194
195     headers = _ingresar(client, "luis")
196     r = client.post("/api/bodegas/buscar", headers=headers, json={"bodega": "restaurante"})
197     assert r.status_code == 200, r.text
198     d = r.json()
199     assert d["encontrada"] is False
200     nombres = {o["bodega"] for o in d["opciones"]}
201     assert nombres == {"RESTAURANTE FUENTES AYB", "RESTAURANTE FUENTES SUMIN"}
202
203
204 def test_buscar_bodega_restringe_opciones_a_lo_asignado_del_auxiliar(
205     client: TestClient,
206 ) -> None:
207     """Bug real reportado en vivo: decir "restaurante" mostraba las CINCO
208     bodegas "RESTAURANTE ..." del catálogo completo, aunque el auxiliar
209     solo tuviera UNA asignada - viendo nombres de zonas ajenas que ni
210     podía llegar a abrir. Restringido a lo suyo, "restaurante" debe
211     resolver directo a la única que sí tiene (sin mostrar "opciones")."""
212     from bd import Sesion
213     from modelos import Bodega, Usuario, AsignacionBodega
214
215     with Sesion() as sesion:
216         auxiliar = sesion.query(Usuario).filter_by(nombre="stephanie").one()
217         suya = Bodega(nombre_oficial="RESTAURANTE FUENTES AYB")

```

```

218         ajena1 = Bodega(nombre_oficial="RESTAURANTE FUENTES SUMIN")
219         ajena2 = Bodega(nombre_oficial="RESTAURANTE NUTRIAS")
220         sesion.add_all([suya, ajena1, ajena2])
221         sesion.flush()
222         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=suya.id))
223         sesion.commit()
224         suya_id = suya.id
225
226     headers = _ingresar(client, "stephanie")
227     r = client.post("/api/bodegas/buscar", headers=headers, json={"bodega": "restaurante"})
228     assert r.status_code == 200, r.text
229     d = r.json()
230     assert d == {"encontrada": True, "bodega": "RESTAURANTE FUENTES AYB",
231                 "bodega_id": suya_id, "opciones": []}
232
233
234 def test_abrir_bodega_por_rest_encuentra_el_nombre_aunque_diga_tilde(
235     client: TestClient,
236 ) -> None:
237     """Mismo bug que en el agente conversacional, pero por el REST directo
238     (\ "Abrir por nombre exacto\ " / clic en una tarjeta): el catálogo no
239     trae tildes, así que buscar con tilde no debe fallar."""
240     from bd import Sesion
241     from modelos import Bodega, Usuario, AsignacionBodega
242
243     with Sesion() as sesion:
244         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
245         bodega = Bodega(nombre_oficial="ALMACEN REGIONAL")
246         sesion.add(bodega)
247         sesion.flush()
248         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
249         sesion.commit()
250
251     headers = _ingresar(client, "luis")
252     r = client.post("/api/bodegas/abrir", headers=headers,
253                   json={"bodega": "almacén regional"})
254     assert r.status_code == 200, r.text
255     assert r.json()["bodega"] == "ALMACEN REGIONAL"
256
257
258 def test_abrir_bodega_por_rest_encuentra_el_nombre_aunque_diga_kiosko_con_k(
259     client: TestClient,
260 ) -> None:
261     """Bug real reportado desde producción: "kiosko" (como está guardado
262     en el catálogo) y "kiosko" (como lo transcribe a veces el
263     reconocimiento de voz, y como también se escribe la palabra en
264     español) no coincidían - K y C no se unificaban en normalizar()."""
265     from bd import Sesion
266     from modelos import Bodega, Usuario, AsignacionBodega
267
268     with Sesion() as sesion:
269         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
270         bodega = Bodega(nombre_oficial="KIOSCO TAQUILLA AYB")
271         sesion.add(bodega)
272         sesion.flush()
273         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
274         sesion.commit()
275
276     headers = _ingresar(client, "luis")
277     r = client.post("/api/bodegas/abrir", headers=headers,
278                   json={"bodega": "kiosko taquilla ayb"})
279     assert r.status_code == 200, r.text
280     assert r.json()["bodega"] == "KIOSCO TAQUILLA AYB"
281
282
283 def test_abrir_bodega_por_rest_ignora_el_punto_final_del_reconocimiento_de_voz(
284     client: TestClient,
285 ) -> None:
286     """Bug real reportado desde producción: el reconocimiento de voz del
287     navegador cierra la frase con un punto ("Zoológico.") aunque nadie lo
288     haya dicho - antes eso bastaba para que "No encuentro esa bodega."."""
289     from bd import Sesion
290     from modelos import Bodega, Usuario, AsignacionBodega
291

```

```

292     with Sesion() as sesion:
293         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
294         bodega = Bodega(nombre_oficial="ZOOLOGICO")
295         sesion.add(bodega)
296         sesion.flush()
297         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
298         sesion.commit()
299
300     headers = _ingresar(client, "luis")
301     r = client.post("/api/bodegas/abrir", headers=headers,
302                   json={"bodega": "Zoológico."})
303     assert r.status_code == 200, r.text
304     assert r.json()["bodega"] == "ZOOLOGICO"
305
306
307 def test_abrir_bodega_prefiere_la_coincidencia_exacta_sobre_una_mas_larga(
308     client: TestClient,
309 ) -> None:
310     """Bug real: decir solo "Zoológico" abría "ZOOLOGICO SUMINISTROS" (una
311     bodega distinta) porque "ZOOLOGICO" es substring de su nombre - debe
312     preferir la bodega cuyo nombre es EXACTAMENTE lo que se dijo."""
313     from bd import Sesion
314     from modelos import Bodega, Usuario, AsignacionBodega
315
316     with Sesion() as sesion:
317         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
318         corta = Bodega(nombre_oficial="ZOOLOGICO")
319         larga = Bodega(nombre_oficial="ZOOLOGICO SUMINISTROS")
320         sesion.add_all([larga, corta]) # a proposito: la larga primero
321         sesion.flush()
322         sesion.add_all([
323             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=corta.id),
324             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=larga.id),
325         ])
326         sesion.commit()
327         corta_id = corta.id
328
329     headers = _ingresar(client, "luis")
330     r = client.post("/api/bodegas/abrir", headers=headers,
331                   json={"bodega": "zoologico"})
332     assert r.status_code == 200, r.text
333     assert r.json()["bodega"] == "ZOOLOGICO"
334     assert r.json()["bodega_id"] == corta_id
335
336
337 def test_no_se_puede_solicitar_una_bodega_que_ya_existe(client: TestClient) -> None:
338     headers_luis = _ingresar(client, "luis")
339
340     ya_existe = client.get("/api/bodegas", headers=headers_luis).json()
341     if not ya_existe:
342         # arranque() solo reparte bodegas si ya hay alguna cargada; si esta
343         # base de pruebas no tiene ninguna, no hay nada que probar aqui.
344         return
345     nombre = ya_existe[0]["bodega"]
346
347     r = client.post("/api/bodegas/crear-pendiente", headers=headers_luis,
348                   json={"nombre": nombre})
349     assert r.status_code == 409, r.text

```

backend/tests/test_asistente_contextual.py

143 LÍNEAS

El asistente que responde por pantalla.

```

1  """Regresión del asistente de voz contextual (Inicio, Ajustes, Ayuda,
2  Reportes, Panel): sin Gemini configurado, cae al respaldo local que al
3  menos reconoce «lléveme a X» por palabra clave y nunca ofrece un destino
4  fuera de lo que el perfil puede ver - ni inventa una respuesta."""
5  from __future__ import annotations
6
7  from fastapi.testclient import TestClient
8
9

```

```

10 def _ingresar(client: TestClient, usuario: str) -> dict[str, str]:
11     r = client.post("/api/ingresar", data={"username": usuario, "password": "StockXperts1"})
12     assert r.status_code == 200, r.text
13     return {"Authorization": f"Bearer {r.json()['token']}"}
14
15
16 def _sin_gemini(monkeypatch) -> None:
17     """Mismo patrón que las demás pruebas del agente: el .env real de la
18     raíz del repo trae una llave de verdad, así que hay que apagarla a
19     mano para que la prueba sea determinista (cae siempre al respaldo)."""
20     import agente.cerebro as cerebro
21     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
22     monkeypatch.setattr(cerebro, "_cliente", None)
23
24
25 def test_asistente_navega_por_voz_a_una_pantalla_valida(
26     client: TestClient, monkeypatch
27 ) -> None:
28     _sin_gemini(monkeypatch)
29     headers = _ingresar(client, "luis")
30     r = client.post("/api/agente/asistente", headers=headers,
31                    json={"texto": "lleveme a bodegas", "vista": "inicio"})
32     assert r.status_code == 200, r.text
33     d = r.json()
34     assert d["accion"] == "navegar"
35     assert d["destino"] == "bodegas"
36
37
38 def test_asistente_no_ofrece_a_un_auxiliar_pantallas_de_administrador(
39     client: TestClient, monkeypatch
40 ) -> None:
41     _sin_gemini(monkeypatch)
42     headers = _ingresar(client, "luis")
43     r = client.post("/api/agente/asistente", headers=headers,
44                    json={"texto": "lleveme al panel gerencial", "vista": "inicio"})
45     assert r.status_code == 200, r.text
46     d = r.json()
47     # "panel" no está en las pantallas disponibles para un auxiliar: nunca
48     # se ofrece como destino, aunque lo haya pedido por su nombre exacto.
49     assert not (d["accion"] == "navegar" and d["destino"] == "panel")
50
51
52 def test_asistente_administrador_si_puede_navegar_al_panel(
53     client: TestClient, monkeypatch
54 ) -> None:
55     _sin_gemini(monkeypatch)
56     headers = _ingresar(client, "diana")
57     r = client.post("/api/agente/asistente", headers=headers,
58                    json={"texto": "lleveme al panel", "vista": "inicio"})
59     assert r.status_code == 200, r.text
60     d = r.json()
61     assert d["accion"] == "navegar"
62     assert d["destino"] == "panel"
63
64
65 def test_asistente_sin_gemini_no_inventa_una_respuesta(
66     client: TestClient, monkeypatch
67 ) -> None:
68     _sin_gemini(monkeypatch)
69     headers = _ingresar(client, "luis")
70     r = client.post("/api/agente/asistente", headers=headers,
71                    json={"texto": "cual es el sentido de la vida", "vista": "inicio"})
72     assert r.status_code == 200, r.text
73     d = r.json()
74     assert d["accion"] is None
75     msg = d["respuesta_hablada"].lower()
76     assert "sin conexión" in msg or "no pude" in msg
77
78
79 def test_asistente_no_revienta_construyendo_el_contexto_de_ninguna_pantalla(
80     client: TestClient, monkeypatch
81 ) -> None:
82     _sin_gemini(monkeypatch)
83     headers_diana = _ingresar(client, "diana")

```

```

84     for vista in ["inicio", "ajustes", "ayuda", "reportes", "panel"]:
85         r = client.post("/api/agente/asistente", headers=headers_diana,
86                         json={"texto": "hola", "vista": vista})
87         assert r.status_code == 200, f"{vista}: {r.text}"
88
89     headers_luis = _ingresar(client, "luis")
90     for vista in ["inicio", "ajustes", "ayuda"]:
91         r = client.post("/api/agente/asistente", headers=headers_luis,
92                         json={"texto": "hola", "vista": vista})
93         assert r.status_code == 200, f"{vista}: {r.text}"
94
95
96 def test_asistente_exige_sesion(client: TestClient) -> None:
97     r = client.post("/api/agente/asistente",
98                     json={"texto": "hola", "vista": "inicio"})
99     assert r.status_code == 401
100
101
102 def test_asistente_navega_directo_a_una_pestana_valida(
103     client: TestClient, monkeypatch
104 ) -> None:
105     """Pedir "el análisis de consumo" no solo debe llevar a Reportes: debe
106     llegar directo a esa pestaña, sin dar clic aparte."""
107     import agente.cerebro as cerebro
108
109     def _falso(contexto, frase, destinos, pestanas=None):
110         return {"intencion": "navegar", "destino": "reportes", "pestana": "analisis",
111                 "respuesta_hablada": "Vamos al análisis de consumo."}
112     monkeypatch.setattr(cerebro, "pensar_asistente", _falso)
113
114     headers = _ingresar(client, "diana")
115     r = client.post("/api/agente/asistente", headers=headers,
116                     json={"texto": "llevame al analisis de consumo", "vista": "inicio"})
117     assert r.status_code == 200, r.text
118     d = r.json()
119     assert d["accion"] == "navegar"
120     assert d["destino"] == "reportes"
121     assert d["pestana"] == "analisis"
122
123
124 def test_asistente_descarta_una_pestana_que_no_pertenece_al_destino(
125     client: TestClient, monkeypatch
126 ) -> None:
127     """Si el modelo se equivoca y devuelve una pestaña que no existe en
128     ese destino (o viene de otra pantalla), se descarta en vez de
129     mandarla tal cual - nunca abrir una pestaña inexistente."""
130     import agente.cerebro as cerebro
131
132     def _falso(contexto, frase, destinos, pestanas=None):
133         return {"intencion": "navegar", "destino": "reportes", "pestana": "usuarios",
134                 "respuesta_hablada": "Vamos a reportes."}
135     monkeypatch.setattr(cerebro, "pensar_asistente", _falso)
136
137     headers = _ingresar(client, "diana")
138     r = client.post("/api/agente/asistente", headers=headers,
139                     json={"texto": "llevame a reportes", "vista": "inicio"})
140     assert r.status_code == 200, r.text
141     d = r.json()
142     assert d["destino"] == "reportes"
143     assert d["pestana"] is None

```

backend/tests/test_asistente_abre_bodega.py

64 LÍNEAS

Abrir una bodega hablando.

```

1 """/api/agente/asistente, desde la pantalla Bodegas: decir "abra/siga
2 contando <nombre>" debe resolver la bodega y mandar a Conteo con ella ya
3 lista - sin esto, Bodegas era la única pantalla con "Pregúntele al
4 agente" que no podía de verdad actuar, solo responder preguntas."""
5 from fastapi.testclient import TestClient
6
7

```

```

8 def test_abre_la_bodega_asignada_al_decir_el_verbo_y_el_nombre(
9     client: TestClient, datos_regresion: dict[str, object]
10 ) -> None:
11     r = client.post(
12         "/api/agente/asistente",
13         json={"texto": f"abra {datos_regresion['bodega_asignada']}", "vista": "bodegas"},
14         headers=datos_regresion["headers"],
15     )
16     assert r.status_code == 200, r.text
17     cuerpo = r.json()
18     assert cuerpo["accion"] == "navegar"
19     assert cuerpo["destino"] == "conteo"
20     assert cuerpo["bodega"] == datos_regresion["bodega_asignada"]
21
22
23 def test_sin_verbo_de_abrir_no_navega_solo_por_decir_el_nombre(
24     client: TestClient, datos_regresion: dict[str, object]
25 ) -> None:
26     """Decir el nombre de una bodega a secas podría ser una PREGUNTA
27     sobre ella ("¿cómo va bodega asignada?"), no una orden de ir a
28     contarla - sin el verbo, no debe forzar la navegación."""
29     r = client.post(
30         "/api/agente/asistente",
31         json={"texto": datos_regresion["bodega_asignada"], "vista": "bodegas"},
32         headers=datos_regresion["headers"],
33     )
34     assert r.status_code == 200, r.text
35     cuerpo = r.json()
36     assert cuerpo["destino"] != "conteo"
37
38
39 def test_verbo_sin_nombre_reconocible_no_navega(
40     client: TestClient, datos_regresion: dict[str, object]
41 ) -> None:
42     r = client.post(
43         "/api/agente/asistente",
44         json={"texto": "abra qzxjklwvbn tfghqzxjkl", "vista": "bodegas"},
45         headers=datos_regresion["headers"],
46     )
47     assert r.status_code == 200, r.text
48     cuerpo = r.json()
49     assert cuerpo["destino"] != "conteo"
50
51
52 def test_solo_aplica_en_la_pantalla_bodegas(
53     client: TestClient, datos_regresion: dict[str, object]
54 ) -> None:
55     """El mismo texto en otra pantalla (ej. Inicio) no debe disparar el
56     atajo de abrir bodega - ahí "abra X" no tiene el mismo sentido."""
57     r = client.post(
58         "/api/agente/asistente",
59         json={"texto": f"abra {datos_regresion['bodega_asignada']}", "vista": "inicio"},
60         headers=datos_regresion["headers"],
61     )
62     assert r.status_code == 200, r.text
63     cuerpo = r.json()
64     assert cuerpo["destino"] != "conteo" or "bodega" not in cuerpo

```

backend/tests/test_consulta_articulo_sugiere_bodega.py

133 LÍNEAS

Consultar un artículo, y no confundirlo con una bodega.

```

1 """/api/articulos/consulta: cuando lo dicho no es un artículo pero sí
2 parece el nombre de una bodega, debe sugerir su detalle (con el id para
3 poder verlo) en vez de responder solo "no encontré" - sin esto, alguien
4 que dice el nombre de una bodega en el buscador de ingredientes (la
5 pantalla equivocada) se queda sin saber por qué no encontró nada."""
6 from fastapi.testclient import TestClient
7
8
9 def test_sugiere_la_bodega_asignada_cuando_el_nombre_es_exacto(
10     client: TestClient, datos_regresion: dict[str, object]

```



```

11 ) -> None:
12     r = client.get(
13         "/api/articulos/consulta",
14         params={"q": datos_regresion["bodega_asignada"]},
15         headers=datos_regresion["headers"],
16     )
17     assert r.status_code == 200, r.text
18     cuerpo = r.json()
19     assert cuerpo["sugerencia_bodega"] == datos_regresion["bodega_asignada"]
20     assert cuerpo["sugerencia_bodega_id"] == datos_regresion["bodega_asignada_id"]
21
22
23 def test_no_le_sugiere_ni_le nombra_una_bodega_que_no_le_esta_asignada(
24     client: TestClient, datos_regresion: dict[str, object]
25 ) -> None:
26     """buscar_bodegas_candidatas() no restringe una coincidencia EXACTA
27     (para que /abrir pueda decir "existe pero no es suya" en vez de "no
28     existe"), pero esta sugerencia es solo informativa - no hay motivo
29     para revelarle a un auxiliar el nombre de una bodega ajena aunque la
30     haya dicho exacta."""
31     r = client.get(
32         "/api/articulos/consulta",
33         params={"q": datos_regresion["bodega_no_asignada"]},
34         headers=datos_regresion["headers"],
35     )
36     assert r.status_code == 200, r.text
37     cuerpo = r.json()
38     assert cuerpo["sugerencia_bodega"] is None
39     assert cuerpo["sugerencia_bodega_id"] is None
40     assert datos_regresion["bodega_no_asignada"] not in cuerpo["resumen"]
41
42
43 def test_articulo_real_sigue_funcionando_igual(
44     client: TestClient, datos_regresion: dict[str, object]
45 ) -> None:
46     """Regresión: agregar la sugerencia de bodega no debe tocar el
47     camino normal cuando sí se encuentra un artículo de verdad."""
48     r = client.get(
49         "/api/articulos/consulta",
50         params={"q": "articulo regresion"},
51         headers=datos_regresion["headers"],
52     )
53     assert r.status_code == 200, r.text
54     cuerpo = r.json()
55     assert cuerpo.get("codigo") == datos_regresion["articulo_codigo"]
56     assert "sugerencia_bodega" not in cuerpo or cuerpo.get("sugerencia_bodega") is None
57
58
59 def test_sin_ningun_parecido_cae_al_agente_general(
60     client: TestClient, datos_regresion: dict[str, object]
61 ) -> None:
62     """Ni artículo ni bodega: en vez de un "no encontré" seco, el mismo
63     buscador cae al agente general (ver _resolver_asistente) - un solo
64     cuadro que responde preguntas sueltas o navega, no dos cuadros
65     separados dando respuestas distintas para lo mismo."""
66     r = client.get(
67         "/api/articulos/consulta",
68         # gibberish elegido para que ninguna de sus palabras sea, ni de
69         # casualidad, substring de "BODEGA ASIGNADA"/"BODEGA RESTRINGIDA"
70         # (una consulta con palabras reales del español sí puede coincidir
71         # por casualidad - "nada" cabe dentro de "asigNADA" - eso no es un
72         # bug de esta función, y no es lo que esta prueba busca cubrir).
73         params={"q": "qzxjklwvbn tfghqzxjkl"},
74         headers=datos_regresion["headers"],
75     )
76     assert r.status_code == 200, r.text
77     cuerpo = r.json()
78     assert cuerpo["sugerencia_bodega"] is None
79     assert cuerpo["resumen"]
80     assert cuerpo["destino"] != "conteo"
81
82
83 def test_frase_suelta_no_confunde_un_articulo_de_baja_confianza(
84     client: TestClient, datos_regresion: dict[str, object]

```

```

85 ) -> None:
86     """Bug real reportado: decir una frase conversacional ("seguir
87     contando", eco de un botón de otra pantalla) coincidía por
88     casualidad con un artículo real que comparte una raíz de 4 letras
89     (aquí, "SEGUNDOS" comparte "SEGU" con "seguir") - una coincidencia
90     de baja confianza que el buscador de Bodegas debe descartar, a
91     diferencia de Conteo/Pedido, que sí aceptan ese mismo umbral más
92     flojo porque ahí lo dictado siempre es, a propósito, un nombre de
93     artículo."""
94     from bd import Sesion
95     from modelos import Articulo, StockSistema
96
97     with Sesion() as sesion:
98         art = Articulo(codigo="ART-REG-002", nombre_oficial="TIMER 60 SEGUNDOS DIGITAL",
99                        unidad_medida="unidad")
100         sesion.add(art)
101         sesion.flush()
102         sesion.add(StockSistema(articulo_codigo=art.codigo,
103                                bodega_id=datos_regresion["bodega_asignada_id"], cantidad_sd=1))
104         sesion.commit()
105
106     r = client.get(
107         "/api/articulos/consulta",
108         params={"q": "seguir contando"},
109         headers=datos_regresion["headers"],
110     )
111     assert r.status_code == 200, r.text
112     cuerpo = r.json()
113     assert cuerpo.get("codigo") != "ART-REG-002"
114     assert "SEGUNDOS" not in (cuerpo.get("resumen") or "")
115
116
117 def test_orden_de_abrir_una_bodega_navega_a_conteo_desde_el_mismo_buscadore(
118     client: TestClient, datos_regresion: dict[str, object]
119 ) -> None:
120     """El mismo buscador de ingredientes también entiende una orden
121     explícita de abrir una bodega (no solo sugerirla) - "abra <nombre>"
122     no es ni un artículo ni "solo" el nombre de una bodega, así que cae
123     al agente general, que sí reconoce el verbo."""
124     r = client.get(
125         "/api/articulos/consulta",
126         params={"q": f"abra {datos_regresion['bodega_asignada']}"},
127         headers=datos_regresion["headers"],
128     )
129     assert r.status_code == 200, r.text
130     cuerpo = r.json()
131     assert cuerpo["accion"] == "navegar"
132     assert cuerpo["destino"] == "conteo"
133     assert cuerpo["bodega"] == datos_regresion["bodega_asignada"]

```

backend/tests/test_orquestador_desambiguacion.py

94 LÍNEAS

Cuando hay varias candidatas en el catalogo.

```

1  """Regresiones de la desambiguación de artículos: pattern matching hecho a
2  mano (posición dictada, código, nombre exacto, palabra distintiva) que es
3  justo lo que una frase improvisada en vivo puede romper. Son funciones
4  puras (sin base de datos), así que se prueban directo."""
5  from __future__ import annotations
6
7  from agente.orquestador import _elegir, _resolver_candidato
8
9
10 def _cand(codigo, nombre, confianza):
11     return {"codigo": codigo, "nombre": nombre, "unidad": "Unidad", "confianza": confianza}
12
13
14 # ----- _resolver_candidato -----
15
16 def test_sin_candidatos_es_vacio():
17     assert _resolver_candidato([]) == ("vacio", None)
18

```

```

19
20 def test_un_candidato_con_confianza_alta_es_directo():
21     cand = [_cand("A1", "ACEITE", 95)]
22     assert _resolver_candidato(cand) == ("directo", cand[0])
23
24
25 def test_un_candidato_con_confianza_baja_no_es_seguro():
26     # ej. "consultemos" coincidiendo por casualidad con "consumibles"
27     cand = [_cand("A1", "CONSUMIBLES VARIOS", 40)]
28     resultado, dato = _resolver_candidato(cand)
29     assert resultado == "no_seguro"
30     assert dato is None
31
32
33 def test_dos_candidatos_lejos_en_confianza_es_directo():
34     cand = [_cand("A1", "ACEITE DE OLIVA", 95), _cand("A2", "ACEITE DE GIRASOL", 60)]
35     assert _resolver_candidato(cand) == ("directo", cand[0])
36
37
38 def test_arroz_contra_arroz_basmati_es_ambiguo():
39     # el caso documentado en el código: ambos casi empatados al 100
40     cand = [_cand("A1", "ARROZ", 100), _cand("A2", "ARROZ BASMATI", 95)]
41     resultado, dato = _resolver_candidato(cand)
42     assert resultado == "ambiguo"
43     assert dato == cand[:2]
44
45
46 def test_primer_candidato_bajo_umbral_medio_no_es_seguro_aunque_haya_varios():
47     cand = [_cand("A1", "X", 50), _cand("A2", "Y", 45)]
48     resultado, _ = _resolver_candidato(cand)
49     assert resultado == "no_seguro"
50
51
52 # ----- _elegir -----
53
54 OPCIONES = [
55     {"codigo": "A1", "nombre": "ARROZ", "unidad": "Kilogram", "confianza": 100},
56     {"codigo": "A2", "nombre": "ARROZ BASMATI", "unidad": "Kilogram", "confianza": 95},
57 ]
58
59
60 def test_elegir_por_codigo_dictado():
61     assert _elegir("es el A2", OPCIONES) == OPCIONES[1]
62
63
64 def test_elegir_por_posicion_primera():
65     for frase in ("la primera", "el primero", "la uno"):
66         assert _elegir(frase, OPCIONES) == OPCIONES[0]
67
68
69 def test_elegir_por_posicion_segunda():
70     for frase in ("la segunda", "el segundo", "la dos"):
71         assert _elegir(frase, OPCIONES) == OPCIONES[1]
72
73
74 def test_elegir_por_nombre_exacto_de_la_opcion_contenida():
75     # "arroz" a secas coincide exacto con el conjunto de palabras de ARROZ,
76     # no con el de ARROZ BASMATI (que tiene una palabra de más)
77     assert _elegir("arroz", OPCIONES) == OPCIONES[0]
78
79
80 def test_elegir_por_palabra_distintiva():
81     assert _elegir("la basmati", OPCIONES) == OPCIONES[1]
82
83
84 def test_elegir_sin_coincidencia_devuelve_none():
85     assert _elegir("ninguna de esas", OPCIONES) is None
86
87
88 def test_elegir_con_lista_vacia_devuelve_none():
89     assert _elegir("la primera", []) is None
90
91
92 def test_elegir_segunda_no_aplica_con_una_sola_opcion():

```

```

93     una = [OPCIONES[0]]
94     assert _elegir("la segunda", una) is None

```

backend/tests/test_previsualizar_reporte_por_voz.py

83 LÍNEAS

Pedir un reporte hablando.

```

1  """«Muéstrame el detalle de bodega», sin decir la palabra "archivo" - la
2  frase más natural para pedir ver un archivo ya generado. Diferencias,
3  tablero, detalle de bodega y trazabilidad no son el nombre de ninguna
4  pestaña de Reportes, así que no compiten con ningún cambio de pestaña y
5  deben abrirse directo. Consolidado y análisis sí son pestañas reales, así
6  que para esos dos la palabra "archivo" sigue siendo obligatoria - sin
7  ella, "muéstrame el consolidado" debe seguir cambiando de pestaña en vez
8  de abrir el archivo."""
9  from __future__ import annotations
10
11 from fastapi.testclient import TestClient
12
13
14 def _ingresar(client: TestClient, usuario: str) -> dict[str, str]:
15     r = client.post("/api/ingresar", data={"username": usuario, "password": "StockXperts1"})
16     assert r.status_code == 200, r.text
17     return {"Authorization": f"Bearer {r.json()['token']}"}
18
19
20 def _sin_gemini(monkeypatch) -> None:
21     import agente.cerebro as cerebro
22     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
23     monkeypatch.setattr(cerebro, "_cliente", None)
24
25
26 def _generar_todos_los_reportes(client: TestClient, headers: dict[str, str]) -> None:
27     for metodo, ruta in [
28         (client.post, "/api/bodegas/1/exportar-detalle"),
29         (client.post, "/api/reportes/diferencias"),
30         (client.post, "/api/bodegas/exportar-estado"),
31         (client.get, "/api/trazabilidad/exportar"),
32         (client.post, "/api/reportes"),
33     ]:
34         r = metodo(ruta, headers=headers, params={"formato": "xlsx"})
35         assert r.status_code == 200, r.text
36
37
38 def test_tipos_que_no_son_pestana_abren_directo_sin_decir_archivo(
39     client: TestClient, monkeypatch
40 ) -> None:
41     _sin_gemini(monkeypatch)
42     headers = _ingresar(client, "diana")
43     _generar_todos_los_reportes(client, headers)
44
45     casos = {
46         "muéstrame el detalle de bodega": "Detalle de bodega",
47         "ábreme las diferencias": "Diferencias por bodega",
48         "muéstrame el estado del tablero": "Estado del tablero",
49         "muéstrame la trazabilidad": "Registro de trazabilidad",
50     }
51     for texto, titulo_esperado in casos.items():
52         r = client.post("/api/agente/asistente", headers=headers,
53             json={"texto": texto, "vista": "reportes"})
54         assert r.status_code == 200, r.text
55         d = r.json()
56         assert d["accion"] == "previsualizar_reporte", (texto, d)
57         assert d["titulo_archivo"] == titulo_esperado, (texto, d)
58
59
60 def test_consolidado_y_analisis_siguen_exigiendo_archivo_para_desambiguar(
61     client: TestClient, monkeypatch
62 ) -> None:
63     _sin_gemini(monkeypatch)
64     headers = _ingresar(client, "diana")
65     _generar_todos_los_reportes(client, headers)

```

```

66
67 # Sin "archivo": sigue siendo un cambio de pestaña, no abrir el archivo.
68 r = client.post("/api/agente/asistente", headers=headers,
69               json={"texto": "muéstrame el consolidado", "vista": "reportes"})
70 d = r.json()
71 assert d["accion"] == "navegar" and d["pestaña"] == "consolidado", d
72
73 r = client.post("/api/agente/asistente", headers=headers,
74               json={"texto": "muéstrame el análisis de consumo", "vista": "reportes"})
75 d = r.json()
76 assert d["accion"] == "navegar" and d["pestaña"] == "analisis", d
77
78 # Con "archivo": sí abre el archivo generado.
79 r = client.post("/api/agente/asistente", headers=headers,
80               json={"texto": "muéstrame el archivo del consolidado", "vista": "reportes"})
81 d = r.json()
82 assert d["accion"] == "previsualizar_reporte"
83 assert d["titulo_archivo"] == "Consolidado para My Inventory", d

```

backend/tests/test_receta_plural.py

40 LÍNEAS

El plato dictado en plural.

```

1  """El chef nombra el plato en plural; la receta se guarda en singular."""
2  import sys
3  from pathlib import Path
4
5  BACKEND_DIR = Path(__file__).resolve().parents[1]
6  if str(BACKEND_DIR) not in sys.path:
7      sys.path.insert(0, str(BACKEND_DIR))
8
9  from servicios.recetas import _sin_plural, _sin_tildes # noqa: E402
10
11
12 def _coincide(dicho: str, guardado: str) -> bool:
13     """Misma comparacion que hace _buscar_receta, sin tocar la base."""
14     objetivo = _sin_tildes(dicho)
15     nombre = _sin_tildes(guardado)
16     return objetivo in nombre or _sin_plural(objetivo) in _sin_plural(nombre)
17
18
19 def test_plural_encuentra_la_receta_guardada_en_singular():
20     # es la frase que la propia pantalla de Pedidos sugiere como ejemplo
21     assert _coincide("ajiacos", "AJIACO SANTA FERENO")
22     assert _coincide("sancochos", "SANCOCHO DE GALLINA")
23
24
25 def test_plural_en_todas_las_palabras():
26     assert _coincide("ajiacos santafereños", "AJIACO SANTA FERENO")
27
28
29 def test_singular_sigue_funcionando():
30     assert _coincide("ajiaco", "AJIACO SANTA FERENO")
31     assert _coincide("ajiaco santafereño", "AJIACO SANTA FERENO")
32
33
34 def test_sin_tildes_sigue_funcionando():
35     assert _coincide("ajiaco santafereño", "AJIACO SANTA FERENO")
36
37
38 def test_un_plato_distinto_no_coincide():
39     assert not _coincide("ajiacos", "SANCOCHO DE GALLINA")
40     assert not _coincide("lasagnas", "AJIACO SANTA FERENO")

```

backend/tests/test_cognito_config.py

70 LÍNEAS

La configuracion del User Pool.

```

1  """El backend debe saber contra que User Pool trabaja aunque nadie haya
2  llenado las variables de entorno.
3

```

```

4 Regresion real: en Render, COGNITO_USER_POOL_ID y COGNITO_APP_CLIENT_ID van
5 como sync:false (viven solo en el panel). Quedaron vacias, el backend no
6 pudo construir el verificador de tokens, y TODAS las sesiones se rechazaron
7 con "Sesion invalida o vencida." - culpando a la persona por un problema de
8 despliegue. El frontend nunca tuvo ese problema porque siempre llevo estos
9 valores por defecto (frontend/src/cognito.js).
10 """
11 from __future__ import annotations
12
13 import importlib
14 import sys
15
16 import pytest
17
18
19 def _recargar_seguridad():
20     for nombre in ("seguridad", "bd", "modelos"):
21         sys.modules.pop(nombre, None)
22     return importlib.import_module("seguridad")
23
24
25 def test_sin_variables_de_entorno_igual_puede_verificar_tokens(
26     monkeypatch: pytest.MonkeyPatch,
27 ) -> None:
28     monkeypatch.delenv("COGNITO_USER_POOL_ID", raising=False)
29     monkeypatch.delenv("COGNITO_APP_CLIENT_ID", raising=False)
30     monkeypatch.delenv("COGNITO_REGION", raising=False)
31
32     seg = _recargar_seguridad()
33
34     assert seg.COGNITO_USER_POOL_ID, "sin pool no se puede verificar nada"
35     assert seg.COGNITO_APP_CLIENT_ID, "sin app client todo token se rechaza"
36     assert seg.COGNITO_REGION == "us-east-2"
37     assert seg._jwks_client is not None
38
39
40 def test_una_variable_vacia_no_deja_el_backend_ciego(
41     monkeypatch: pytest.MonkeyPatch,
42 ) -> None:
43     """Vacía y ausente son el mismo caso: en Render una variable se puede
44     crear con el valor en blanco, que es como quedo en el incidente real."""
45     monkeypatch.setenv("COGNITO_USER_POOL_ID", "")
46     monkeypatch.setenv("COGNITO_APP_CLIENT_ID", " ")
47
48     seg = _recargar_seguridad()
49
50     assert seg.COGNITO_USER_POOL_ID == "us-east-2_6HbrPruvL"
51     assert seg.COGNITO_APP_CLIENT_ID == "847jtkc5sem7mr4tb8csrrkqp"
52     assert seg._jwks_client is not None
53
54
55 def test_la_variable_de_entorno_sigue_mandando(
56     monkeypatch: pytest.MonkeyPatch,
57 ) -> None:
58     """El valor por defecto es una red de seguridad, no una imposicion:
59     apuntar a otro User Pool (otro entorno, otra cuenta) debe seguir siendo
60     solo definir la variable."""
61     monkeypatch.setenv("COGNITO_REGION", "us-west-1")
62     monkeypatch.setenv("COGNITO_USER_POOL_ID", "us-west-1_otroPool")
63     monkeypatch.setenv("COGNITO_APP_CLIENT_ID", "otroclienteid")
64
65     seg = _recargar_seguridad()
66
67     assert seg.COGNITO_USER_POOL_ID == "us-west-1_otroPool"
68     assert seg.COGNITO_APP_CLIENT_ID == "otroclienteid"
69     assert seg._EMISOR == ("https://cognito-idp.us-west-1.amazonaws.com/"
70                            "us-west-1_otroPool")

```

backend/tests/test_cognito_no_disponible.py

96 LÍNEAS

Que pasa cuando Cognito no responde.

```

1  """Cuando Cognito no responde, el token NO es invalido: es el backend el que
2  no pudo comprobarlo.
3
4  Regresion real, vista en produccion (Render): un tropiezo de red al traer el
5  JWKS de AWS terminaba en 401 "Sesion invalida o vencida.", y como el frontend
6  cierra la sesion ante cualquier 401 (api.js: alSesionInvalida), sacaba a la
7  persona de la aplicacion por un problema que no era suyo. Debe responder 503.
8  """
9  from __future__ import annotations
10
11  import pytest
12  from fastapi.testclient import TestClient
13
14
15  def test_fallo_de_red_contra_cognito_responde_503_y_no_401(
16      client: TestClient, datos_regresion: dict[str, object],
17      monkeypatch: pytest.MonkeyPatch,
18  ) -> None:
19      import seguridad
20
21      def _cognito_caido(token):
22          raise seguridad.ServicioIdentidadCaido("no se pudo conectar al JWKS")
23
24      monkeypatch.setattr(seguridad, "_reclamos_cognito", _cognito_caido)
25
26      r = client.get("/api/usuarios/yo", headers=datos_regresion["headers"])
27
28      # 503, NO 401: el frontend solo cierra la sesion ante un 401, asi que
29      # este codigo es justamente lo que evita el cierre de sesion indebido.
30      assert r.status_code == 503, r.text
31      assert "Intente de nuevo" in r.json()["detail"]
32
33
34  def test_un_token_de_verdad_invalido_sigue_dando_401(
35      client: TestClient, datos_regresion: dict[str, object],
36      monkeypatch: pytest.MonkeyPatch,
37  ) -> None:
38      """El arreglo no debe ablandar la seguridad: si el token es malo, sigue
39      siendo 401 - solo cambia el caso en que la falla es de infraestructura."""
40      import seguridad
41
42      monkeypatch.setattr(seguridad, "_reclamos_cognito", lambda token: None)
43      r = client.get("/api/usuarios/yo", headers=datos_regresion["headers"])
44      assert r.status_code == 401, r.text
45      assert r.json()["detail"] == "Sesion invalida o vencida."
46
47
48  def test_un_tropiezo_pasajero_se_reintenta_y_sale_bien(
49      client: TestClient, monkeypatch: pytest.MonkeyPatch,
50  ) -> None:
51      """Si el primer intento falla por red pero el segundo funciona, la llave
52      se obtiene igual - sin que la persona se entere de nada."""
53      import seguridad
54      from jwt.exceptions import PyJWKClientConnectionError
55
56      class _FallaUnaVez:
57          def __init__(self):
58              self.intentos = 0
59
60          def get_signing_key_from_jwt(self, token):
61              self.intentos += 1
62              if self.intentos == 1:
63                  raise PyJWKClientConnectionError("tropiezo pasajero")
64              class _Llave:
65                  key = "llave-de-prueba"
66              return _Llave()
67
68      falso = _FallaUnaVez()
69      monkeypatch.setattr(seguridad, "_jwks_client", falso)
70
71      assert seguridad._llave_de_firma("token-cualquiera") == "llave-de-prueba"
72      assert falso.intentos == 2
73
74

```

```

75 def test_si_cognito_no_responde_nunca_se_levanta_la_excepcion_propia(
76     client: TestClient, monkeypatch: pytest.MonkeyPatch,
77 ) -> None:
78     """Dos fallos seguidos de red sí deben rendirse - pero con la excepción
79     que usuario_actual sabe convertir en 503, no en un 401 engañoso."""
80     import seguridad
81     from jwt.exceptions import PyJWKClientConnectionError
82
83     class _SiempreCaido:
84         def __init__(self):
85             self.intentos = 0
86
87         def get_signing_key_from_jwt(self, token):
88             self.intentos += 1
89             raise PyJWKClientConnectionError("caido")
90
91     caido = _SiempreCaido()
92     monkeypatch.setattr(seguridad, "_jwks_client", caido)
93
94     with pytest.raises(seguridad.ServicioIdentidadCaido):
95         seguridad._llave_de_firma("token-cualquiera")
96     assert caido.intentos == 2

```

backend/tests/test_esquema_desfasado.py

82 LÍNEAS

Cuando la base tiene un esquema viejo.

```

1  """El esquema real debe seguir al modelo, no solo al crearse.
2
3  Regresion real y cara: al pasar la identidad a Cognito, Usuario.clave_hash
4  paso a nullable=True en el modelo (ya nadie la llena, la clave la guarda
5  Cognito), pero la tabla de Postgres ya desplegada conservo su NOT NULL de
6  origen - _migrar_columnas_faltantes solo AGREGA columnas, nunca relaja una
7  restriccion. Resultado: en produccion fallaba CADA insercion de usuario
8  (autoregistro y "crear usuario" desde Ajustes) con un 500. En local no se
9  veia porque cuentavoz.db se recrea de cero en cada prueba.
10 """
11 from __future__ import annotations
12
13 import pytest
14 from fastapi.testclient import TestClient
15
16
17 def test_crear_usuario_no_necesita_clave_hash(
18     client: TestClient, datos_regresion: dict[str, object],
19     monkeypatch: pytest.MonkeyPatch,
20 ) -> None:
21     """La clave la guarda Cognito: insertar un Usuario sin clave_hash tiene
22     que funcionar. Si alguien le devuelve el nullable=False al modelo, esto
23     lo caza antes de que llegue a produccion."""
24     import main as modulo
25     from modelos import Usuario
26     assert Usuario.__table__.c.clave_hash.nullable, (
27         "clave_hash debe seguir siendo opcional: la identidad la maneja "
28         "Cognito y ningun INSERT la llena")
29
30     # crear en Cognito de verdad exigiria AWS; aqui interesa la insercion local
31     monkeypatch.setattr(modulo, "_crear_usuario_cognito", lambda *a, **k: True)
32
33     r = client.post("/api/usuarios",
34                     headers={"Authorization": "Bearer diana"},
35                     json={"nombre": "recien_creada", "perfil": "auxiliar",
36                           "correo": "recien@ejemplo.com", "pin": "ClaveDemo1234"})
37     assert r.status_code == 200, r.text
38
39     from bd import Sesion
40     with Sesion() as s:
41         fila = s.query(Usuario).filter_by(nombre="recien_creada").one()
42         assert fila.clave_hash is None
43
44
45 def test_el_autoregistro_inserta_sin_clave_hash(

```



```

46     client: TestClient, datos_regresion: dict[str, object],
47 ) -> None:
48     r = client.post("/api/registro-completado",
49                     headers={"Authorization": "Bearer autoregistrada"},
50                     json={"nombre_completo": "Auto Registrada",
51                           "correo": "auto@ejemplo.com"})
52     assert r.status_code == 200, r.text
53
54     from bd import Sesion
55     from modelos import Usuario
56     with Sesion() as s:
57         fila = s.query(Usuario).filter_by(nombre="autoregistrada").one()
58         assert fila.clave_hash is None
59         # entra de una: sin espera de aprobacion de un administrador
60         assert fila.activo == 1
61
62
63 def test_un_500_llega_con_cabeceras_cors(app_modulos: object) -> None:
64     """Sin cabeceras CORS, el navegador bloquea la respuesta entera y fetch
65     lanza un TypeError indistinguible de quedarse sin red - por eso un 500
66     de verdad se veia en pantalla como "Sin conexion con el servidor.
67     Revise el Wi-Fi", mandando a buscar el problema en el router."""
68     modulo = app_modulos
69     origen = next(iter(modulo._origenes))
70
71     @modulo.app.get("/api/_prueba_explota")
72     def _explota():
73         raise RuntimeError("fallo a proposito")
74
75     # raise_server_exceptions=False para que el cliente devuelva la respuesta
76     # 500 tal como la veria un navegador, en vez de relanzar la excepcion.
77     with TestClient(modulo.app, raise_server_exceptions=False) as cliente:
78         r = cliente.get("/api/_prueba_explota", headers={"Origin": origen})
79
80     assert r.status_code == 500
81     assert r.json()["detail"] == "Ocurrió un error inesperado. Intente de nuevo."
82     assert r.headers.get("access-control-allow-origin") == origen

```

17.8 Estilos, configuracion y despliegue

frontend/src/index.css

568 LÍNEAS

La hoja de estilos entera: variables de color, la rejilla de la aplicacion, los componentes y los ajustes de accesibilidad.

```

1  /* — Paleta oficial de Colsubsidio — */
2  :root {
3      --azul:          #0067B1; /* azul Colsubsidio · Pantone 2196 C */
4      --azul-prof:    #10294C;
5      --azul-claro:   #E4F0F9;
6      --amarillo:     #FFD000; /* amarillo Colsubsidio · Pantone 109 C */
7      --amarillo-bg:  #FFF6CC;
8      --amarillo-tx:  #8A6D00;
9      --verde:        #1E7A46;
10     --verde-bg:      #E8F5EE;
11     --rojo:          #B3261E; /* mismo color que .error, para estados rechazados */
12     --rojo-bg:       #FBE8E6;
13     --grafito:       #575756; /* Cool Gray 11 C */
14     --txt:           #2B2B2A;
15     --borde:         #D8E2EC;
16     --fondo:         #F5F7FB;
17     --fuente: "Calibri", "Carlito", "Segoe UI", system-ui, sans-serif;
18     --radio: 12px;
19     --sombra: 0 2px 8px rgba(0, 103, 177, 0.10);
20
21     /* paleta categórica para gráficas (validada: ΔE normal-vision >= 15) */
22     --serie-1: #0067B1;
23     --serie-2: #B38600;
24     --serie-3: #1E7A46;
25     --serie-4: #6B4FA0;

```

```

26 }
27
28 /* — Accesibilidad: alto contraste (Mi perfil > Accesibilidad) —
29 Redefine solo las variables de :root - como casi todo el resto de la
30 hoja ya las usa en vez de colores sueltos, esto alcanza a casi toda la
31 interfaz sin tocar cada componente. */
32 [data-contraste="alto"] {
33   --azul: #004A80;
34   --azul-prof: #000B1A;
35   --azul-claro: #FFFFFF;
36   --amarillo-tx:#5C4900;
37   --verde: #0F4D2C;
38   --rojo: #7A140F;
39   --grafito: #000000;
40   --txt: #000000;
41   --borde: #000000;
42   --fondo: #FFFFFF;
43 }
44
45 *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
46 html, body, #root { height: 100%; }
47 /* rem se calcula contra el tamaño de fuente de la RAÍZ (html), no de body -
48 por eso el tamaño ajustable (Mi perfil > Accesibilidad) se pone aquí. */
49 html {
50   font-size: 16px;
51 }
52 html[data-tamano="grande"] { font-size: 19px; }
53 html[data-tamano="mas-grande"] { font-size: 22px; }
54 body {
55   font-family: var(--fuente);
56   color: var(--txt);
57   background: var(--fondo);
58   -webkit-font-smoothing: antialiased;
59 }
60 button { font-family: inherit; cursor: pointer; border: none; }
61 input { font-family: inherit; font-size: 1rem; }
62
63 /* Accesibilidad: quien navega solo con teclado (movilidad reducida, o un
64 lector de pantalla que también depende del foco) necesita ver con
65 claridad dónde está parado - antes solo el input de ingreso tenía un
66 :focus explícito, todo lo demás dependía del anillo por defecto del
67 navegador, que sobre botones de color queda con poco contraste o casi
68 invisible. :focus-visible (no :focus) para no dibujar el anillo en un
69 clic de mouse, solo al navegar con Tab/flechas.
70
71 --azul-prof (no --amarillo) por defecto: casi toda la app vive sobre
72 tarjetas blancas o el fondo claro, y --amarillo ahí da 1.47:1 de
73 contraste (por debajo del 3:1 que exige WCAG para el aro de foco) -
74 el mismo error que ya se corrigió en .kpi.oro b más abajo, esta vez en
75 el propio indicador de foco. --azul-prof da 14.55:1 sobre blanco. La
76 única zona con fondo oscuro de verdad es el menú lateral, así que ahí
77 se vuelve a --amarillo (9.89:1 sobre su fondo azul marino). */
78 a:focus-visible, button:focus-visible, input:focus-visible, select:focus-visible,
79 textarea:focus-visible, summary:focus-visible, [tabindex]:focus-visible {
80   outline: 3px solid var(--azul-prof);
81   outline-offset: 2px;
82 }
83 .sidebar :focus-visible {
84   outline-color: var(--amarillo);
85 }
86
87 /* — Estructura — */
88 /* Las dos alturas no sobran: `dvh` solo lo entienden Chrome 108+ y Safari
89 15.4+, y un navegador mas viejo DESCARTA la declaracion entera. Sin el
90 `vh` de respaldo, en una tableta con navegador antiguo la rejilla se
91 queda sin alto: la barra lateral no llega abajo y .contenido no tiene
92 contra que desplazarse. El `vh` va primero para que el navegador
93 moderno lo pise con `dvh`, que ademas maneja bien la barra del
94 navegador cuando aparece y desaparece. */
95 .app-root { display: grid; grid-template-columns: 232px 1fr;
96   height: 100vh; height: 100dvh; }
97 .contenido { overflow-y: auto; overflow-x: hidden; padding: 0; display: flex; flex-direction: column; }
98
99 .barra-sup {

```

```

100 background: #fff; border-bottom: 1px solid var(--borde);
101 padding: 14px 22px; position: sticky; top: 0; z-index: 5;
102 }
103 .barra-sup-inner {
104   max-width: 1180px; margin: 0 auto; width: 100%;
105   display: flex; align-items: center; gap: 14px; flex-wrap: wrap;
106 }
107 .barra-sup h1 { font-size: 1.28rem; color: var(--azul); font-weight: 700; flex: 1 1 auto; }
108 .barra-sup::after { content: ""; }
109 .faja { height: 4px; background: var(--amarillo); }
110 .cs-logo { height: 22px; margin-left: auto; flex: none; }
111 .area { padding: 20px 22px 28px; max-width: 1180px; margin: 0 auto; width: 100%; }
112
113 /* — Barra lateral — */
114 .sidebar {
115   background: var(--azul-prof); color: #fff;
116   display: flex; flex-direction: column; padding: 16px 0;
117 }
118 .sidebar .marca { display: flex; align-items: center; gap: 10px; padding: 0 18px 14px; }
119 .sidebar .marca img { width: 34px; height: 34px; border-radius: 8px; }
120 .sidebar .marca b { font-size: 1.05rem; display: block; line-height: 1.1; }
121 .sidebar .marca i { font-size: .68rem; color: var(--amarillo); font-style: italic; }
122 .sidebar hr { border: none; border-top: 1px solid #2A4573; margin: 0 18px 10px; }
123 .sidebar ul { list-style: none; flex: 1; overflow-y: auto; }
124 /* El que se pinta es el <button> de adentro, no el <li>: el <li> existe
125    solo para que el menú siga siendo una lista para un lector de pantalla
126    (ver BarraLateral.jsx). Por eso hereda tipografía y se estira al ancho:
127    un botón trae su propia fuente y su propio tamaño. */
128 .sidebar li > button {
129   display: flex; align-items: center; gap: 10px; width: 100%;
130   padding: 9px 18px; font-size: .88rem; color: #B9C4D6;
131   border: none; border-left: 3px solid transparent; user-select: none;
132   background: none; font-family: inherit; text-align: left; cursor: pointer;
133 }
134 .sidebar li > button:hover { background: #16305A; color: #fff; }
135 .sidebar li > button.sel {
136   background: var(--azul); color: #fff; font-weight: 700;
137   border-left-color: var(--amarillo);
138 }
139 .sidebar li > button .icono-menu {
140   display: inline-flex; align-items: center; justify-content: center;
141   flex: none; color: #8194B3;
142 }
143 .sidebar li > button:hover .icono-menu { color: #fff; }
144 .sidebar li > button.sel .icono-menu { color: var(--amarillo); }
145 .sidebar footer { padding: 12px 18px 0; border-top: 1px solid #2A4573; }
146 .sidebar footer .avatar {
147   width: 32px; height: 32px; border-radius: 50%; background: var(--amarillo);
148   color: var(--azul-prof); display: inline-flex; align-items: center;
149   justify-content: center; font-weight: 700; margin-right: 8px;
150 }
151 .sidebar footer small { display: block; color: #9FADC4; font-size: .68rem; }
152 .sidebar footer img { height: 15px; margin-top: 10px; opacity: .9; }
153
154 /* — Tarjetas, chips y botones — */
155 .card {
156   background: #fff; border: 1px solid var(--borde);
157   border-radius: var(--radio); padding: 16px 18px; margin-bottom: 14px;
158   /* casi toda tabla de la app vive dentro de un .card sin envoltorio propio;
159      esto le da scroll horizontal contenido en vez de romper el layout de
160      toda la pantalla en móviles y tablets angostos. */
161   max-width: 100%; overflow-x: auto;
162 }
163 /* h2 y h3 se ven igual a proposito: el nivel dice la jerarquia a un
164    lector de pantalla, no el tamaño de letra. El título de la tarjeta
165    es h2 y sus subsecciones h3 (ver Marco.jsx, que pone el h1). */
166 .card h2, .card h3 { font-size: .98rem; margin-bottom: 12px; color: var(--txt); }
167 .chips { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 14px; }
168 .chip {
169   background: var(--azul-claro); color: var(--azul); font-weight: 700;
170   font-size: .8rem; padding: 6px 13px; border-radius: 20px;
171 }
172 .chip.verde { background: var(--verde-bg); color: var(--verde); }
173 .chip.oro { background: var(--amarillo-bg); color: var(--amarillo-tx); }

```

```

174 .chip.rojo { background: var(--rojo-bg); color: var(--rojo); }
175 .chip.gris { background: #EFEFEF; color: var(--grafito); }
176 .chip.azul { background: var(--azul); color: #fff; }
177 .chip.borde { background: #fff; border: 1.5px solid currentColor; }
178 .chip.borde.azul { color: var(--azul); }
179
180 .btn {
181   border-radius: var(--radio); font-weight: 700; padding: 13px 20px;
182   font-size: .95rem; background: var(--azul); color: #fff;
183   transition: opacity .15s; min-height: 46px;
184 }
185 .btn.active { opacity: .82; }
186 .btn.disabled { opacity: .45; cursor: not-allowed; }
187 .btn.borde { background: #fff; color: var(--azul); border: 2px solid var(--azul); }
188 .btn.verde { background: var(--verde); }
189 .btn.oro { background: #fff; color: var(--amarillo-tx); border: 2px solid var(--amarillo); }
190 .btn.gris { background: #fff; color: var(--grafito); border: 2px solid #C3CAD6; }
191 .grilla-botones { display: flex; gap: 12px; flex-wrap: wrap; margin-top: 16px; }
192
193 /* — Conversación y micrófono — */
194 .conteo-cols { display: grid; grid-template-columns: 1fr 300px; gap: 14px; }
195 .dos-columnas { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; align-items: start; }
196 .perfil-cols { display: grid; grid-template-columns: 260px 1fr; gap: 14px; align-items: start; }
197 .campos-2col { display: grid; grid-template-columns: 1fr 1fr; gap: 0 14px; }
198 .avatar-grande {
199   width: 110px; height: 110px; border-radius: 50%; border: 3px solid var(--azul);
200   display: flex; align-items: center; justify-content: center;
201   font-size: 2.6rem; font-weight: 700; color: var(--azul); background: var(--azul-claro);
202   margin: 0 auto;
203 }
204 .avatar-chico {
205   width: 26px; height: 26px; border-radius: 50%; flex: none;
206   display: inline-flex; align-items: center; justify-content: center;
207   font-size: .78rem; font-weight: 700; color: var(--azul); background: var(--azul-claro);
208 }
209 .rotulo { font-size: .74rem; color: var(--grafito); font-style: italic; margin-bottom: 4px; }
210 .cita {
211   background: var(--fondo); border: 1px solid var(--borde); border-radius: 10px;
212   padding: 14px 16px; font-style: italic; color: var(--txt); font-size: .95rem;
213 }
214 .burbuja {
215   background: var(--azul-claro); border: 2px solid var(--azul);
216   border-radius: var(--radio); padding: 18px; text-align: center;
217   color: #06315B; font-size: 1.06rem; min-height: 74px;
218   display: flex; align-items: center; justify-content: center;
219 }
220 .mic-caja { display: flex; flex-direction: column; align-items: center; gap: 10px; }
221 .mic-btn {
222   width: 116px; height: 116px; border-radius: 50%;
223   background: var(--azul); color: #fff; font-size: 2.6rem;
224   box-shadow: var(--sombra); display: flex; align-items: center;
225   justify-content: center; user-select: none; -webkit-user-select: none;
226   transition: transform .12s, background .2s;
227 }
228 .mic-btn.active { transform: scale(.94); }
229 .mic-btn.escuchando { background: var(--amarillo); color: var(--azul-prof); animation: latir 1.1s infinite; }
230 @keyframes latir { 0%,100% { box-shadow: 0 0 0 0 rgba(255,208,0,.55); } 50% { box-shadow: 0 0 0 16px
  rgba(255,208,0,0); } }
231 /* Quien pide menos movimiento -por trastorno vestibular o por migraña-
232 lo pide para TODO, no solo para el pulso del micrófono escuchando: se
233 apagan también las transiciones y el desplazamiento suave. La
234 preferencia la trae el sistema operativo, no hay que activarla aquí.
235 Apagar el pulso no quita información: la pantalla ya avisa igual con
236 el texto "Escuchando..." y el cambio de color. */
237 @media (prefers-reduced-motion: reduce) {
238   *, *::before, *::after {
239     animation-duration: .01ms !important;
240     animation-iteration-count: 1 !important;
241     transition-duration: .01ms !important;
242     scroll-behavior: auto !important;
243   }
244   .mic-btn.escuchando { animation: none; }
245 }
246

```

```

247 /* Salto al contenido: fuera de la pantalla hasta que recibe el foco. No
248     estorba a quien usa el ratón y le ahorra trece tabulaciones por
249     pantalla a quien usa el teclado. */
250 .salto-contenido {
251     position: absolute; left: 8px; top: -60px; z-index: 999;
252     background: var(--azul); color: #fff; padding: 10px 16px;
253     border-radius: 0 0 10px 10px; font-weight: 600; text-decoration: none;
254     transition: top .15s ease;
255 }
256 .salto-contenido:focus { top: 0; outline: 3px solid var(--amarillo); }
257 /* El destino no debe mostrar el anillo de foco: recibe el foco por
258     programa, no porque alguien lo haya tabulado. */
259 .contenido:focus { outline: none; }
260
261 .mic-caja small { color: var(--grafito); text-align: center; font-size: .78rem; }
262
263 .registro { display: flex; align-items: center; gap: 10px; padding: 9px 12px;
264     border: 1px solid var(--borde); border-radius: 8px; margin-bottom: 7px;
265     background: #FAFBFD; font-size: .87rem; }
266 .registro.ok { width: 20px; height: 20px; border-radius: 50%; background: var(--verde);
267     color: #fff; display: inline-flex; align-items: center; justify-content: center;
268     font-size: .7rem; flex: none; }
269 .registro.cant { margin-left: auto; color: var(--grafito); font-size: .82rem; }
270
271 .etiqueta-actividad { margin-left: auto; font-weight: 700; font-size: .8rem; flex: none; }
272 .etiqueta-actividad.azul { color: var(--azul); }
273 .etiqueta-actividad.verde { color: var(--verde); }
274 .etiqueta-actividad.oro { color: var(--amarillo-tx); }
275 .etiqueta-actividad.gris { color: var(--grafito); }
276
277 .banner {
278     display: flex; gap: 14px; align-items: flex-start;
279     background: var(--amarillo-bg); border: 2px solid var(--amarillo);
280     border-radius: var(--radio); padding: 14px 16px; margin-bottom: 14px;
281 }
282 .banner.ico { width: 34px; height: 34px; border-radius: 50%; background: var(--amarillo);
283     color: #fff; font-weight: 700; display: inline-flex; align-items: center;
284     justify-content: center; flex: none; font-size: 1.2rem; }
285 .banner.b { color: var(--amarillo-tx); display: block; margin-bottom: 3px; }
286 .banner.span { color: var(--amarillo-tx); font-size: .87rem; }
287 .banner.ok { background: var(--verde-bg); border-color: var(--verde); }
288 .banner.ok.ico { background: var(--verde); }
289 .banner.ok.b, .banner.ok.span { color: var(--verde); }
290
291 /* — Opciones — */
292 .opciones { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
293 .opcion {
294     background: #fff; border: 2px solid var(--azul); border-radius: var(--radio);
295     padding: 18px; text-align: center;
296 }
297 .opcion.n { background: var(--azul); color: #fff; font-size: .68rem;
298     font-weight: 700; padding: 3px 9px; border-radius: 10px; }
299 .opcion.h4 { color: var(--azul); font-size: 1.02rem; margin: 12px 0 8px; }
300 .opcion.p { color: var(--txt); font-size: .87rem; margin: 3px 0; }
301
302 /* — Tablas y KPI — */
303 table { width: 100%; border-collapse: collapse; font-size: .87rem; }
304 thead th { background: var(--azul); color: #fff; padding: 9px 10px; text-align: left;
305     font-size: .8rem; }
306 tbody td { padding: 9px 10px; border-bottom: 1px solid var(--borde); }
307 tbody tr:nth-child(even) { background: #FAFBFD; }
308 td.dif, td.falta { background: var(--amarillo-bg); color: var(--amarillo-tx);
309     font-weight: 700; }
310 .kpis { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
311     gap: 12px; margin-bottom: 14px; }
312 .kpi { background: #fff; border: 1px solid var(--borde); border-radius: var(--radio);
313     padding: 14px 16px; border-top: 4px solid var(--amarillo); }
314 .kpi.small { color: var(--grafito); font-size: .76rem; display: block; }
315 .kpi.b { font-size: 1.6rem; color: var(--azul); display: block; margin: 3px 0; }
316 .kpi.verde.b { color: var(--verde); }
317 /* --amarillo (#FFD000) es el acento de marca, hecho para franjas y fondos,
318     no para texto: sobre blanco da 1.47:1 de contraste, muy por debajo del
319     3:1 mínimo (WCAG) hasta para texto grande en negrita. --amarillo-tx
320     (#8A6D00) es la variante ya pensada para texto - Reportes.jsx ya la usa

```

```

321     por esta misma razón para los chips "oro"; esta regla se había quedado
322     con la de fondo por error. */
323 .kpi.oro b { color: var(--amarillo-tx); }
324 .kpi i { font-size: .72rem; color: var(--grafito); font-style: normal; }
325 .kpi-cabeza { display: flex; align-items: center; gap: 9px; margin-bottom: 2px; }
326 .kpi-cabeza small { margin: 0; }
327 .icono-kpi {
328     width: 30px; height: 30px; border-radius: 9px; flex: none;
329     background: var(--azul-claro); display: flex; align-items: center;
330     justify-content: center; font-size: 1rem;
331 }
332 .kpi.verde .icono-kpi { background: var(--verde-bg); }
333 .kpi.oro .icono-kpi { background: var(--amarillo-bg); }
334
335 /* — Bodegas — */
336 .grid-bodegas { display: grid; grid-template-columns: repeat(auto-fill, minmax(230px, 1fr));
337     gap: 12px; }
338 .tarjeta-bodega { background: #fff; border: 1px solid var(--borde);
339     border-left: 5px solid var(--grafito); border-radius: 10px; padding: 12px 14px;
340     text-align: left; width: 100%; }
341 .tarjeta-bodega b { display: flex; align-items: center; gap: 8px;
342     font-size: .87rem; margin-bottom: 6px; }
343 .tarjeta-bodega .est { font-size: .72rem; font-weight: 700; padding: 3px 8px;
344     border-radius: 9px; }
345 .tarjeta-bodega.cerrada { border-left-color: var(--verde); }
346 .tarjeta-bodega.en_conteo { border-left-color: var(--azul); }
347 .tarjeta-bodega.en_auditoria { border-left-color: var(--amarillo); }
348 .tarjeta-bodega small { display: block; color: var(--grafito); font-size: .72rem;
349     margin-top: 6px; }
350 .icono-tarjeta { width: 26px; height: 26px; border-radius: 8px; flex: none;
351     display: flex; align-items: center; justify-content: center; font-size: .85rem;
352     background: #EFEFEF; }
353 .tarjeta-bodega.cerrada .icono-tarjeta { background: var(--verde-bg); }
354 .tarjeta-bodega.en_conteo .icono-tarjeta { background: var(--azul-claro); }
355 .tarjeta-bodega.en_auditoria .icono-tarjeta { background: var(--amarillo-bg); }
356 .icono-accion { width: 42px; height: 42px; border-radius: 50%; background: var(--azul-claro);
357     display: flex; align-items: center; justify-content: center; font-size: 1.3rem;
358     margin-bottom: 10px; }
359 .accesos-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px; }
360 .acceso-rapido {
361     background: #fff; border: 1px solid var(--borde); border-radius: var(--radio);
362     padding: 16px 18px; text-align: left; width: 100%;
363     transition: box-shadow .15s, transform .1s;
364 }
365 .acceso-rapido: hover { box-shadow: var(--sombra); transform: translateY(-1px); }
366 .acceso-rapido b { display: block; color: var(--azul); font-size: .95rem; margin-bottom: 3px; }
367 .acceso-rapido small { color: var(--grafito); font-size: .8rem; }
368
369 /* — Ingreso —
370     Una sola tarjeta centrada sobre un fondo con manchas de color
371     difuminadas (en vez del panel azul de antes), mismo lenguaje visual
372     que las pantallas propias de AWS Cognito - la marca de CuentaVoz vive
373     ahora dentro de la tarjeta, compacta, arriba del formulario. */
374 /* Mismo respaldo que en .app-root: sin el `vh`, en una tableta con
375     navegador antiguo el fondo se encoge al alto de la tarjeta y la
376     pantalla de ingreso queda cortada, con una franja gris debajo. */
377 .ingreso-fondo { min-height: 100vh; min-height: 100dvh;
378     display: flex; align-items: center;
379     justify-content: center; padding: 24px; position: relative;
380     overflow: hidden; background: var(--fondo); }
381 .ingreso-fondo::before, .ingreso-fondo::after { content: ""; position: absolute;
382     border-radius: 50%; filter: blur(70px); opacity: .45; z-index: 0; }
383 .ingreso-fondo::before { width: 480px; height: 480px; background: var(--azul);
384     top: -180px; left: -160px; }
385 .ingreso-fondo::after { width: 420px; height: 420px; background: var(--amarillo);
386     bottom: -160px; right: -140px; }
387 .ingreso { position: relative; z-index: 1; width: 100%; max-width: 440px;
388     background: #fff; border-radius: 22px; padding: 30px 36px;
389     box-shadow: 0 24px 60px -16px rgba(11, 27, 51, .28); }
390 .ingreso .marca-cabecera { display: flex; flex-direction: column; align-items: center;
391     gap: 2px; text-align: center; margin-bottom: 14px; }
392 .ingreso .marca-cabecera img.app { width: 40px; border-radius: 11px; }
393 .ingreso .marca-cabecera h1 { font-size: 1.25rem; color: var(--azul-prof); margin-top: 4px; }
394 .ingreso .marca-cabecera i { color: var(--grafito); font-size: .8rem; }

```



```

395 .ingreso .marca-cabecera .hackathon { color: var(--grafito); font-size: .68rem;
396   margin-top: 4px; opacity: .7; letter-spacing: .02em; }
397 .ingreso .form { display: flex; flex-direction: column; gap: 11px; }
398 .ingreso .form h2 { color: var(--azul); font-size: 1.35rem; }
399 .ingreso .form label { display: block; font-size: .8rem; color: var(--grafito); margin-bottom: 4px; }
400 .ingreso .campos-2col label { margin-bottom: 4px; }
401 .ingreso .campo-icone { position: relative; display: flex; align-items: center; }
402 .ingreso .campo-icone svg { position: absolute; left: 14px; color: #9AA7BC; }
403 .ingreso .form input { width: 100%; padding: 13px 14px 13px 42px;
404   border: 1px solid var(--borde); border-radius: var(--radio); }
405 .ingreso .form input:focus { outline: 2px solid var(--azul); border-color: var(--azul); }
406 .ingreso .perfiles { display: flex; gap: 10px; margin-top: 4px; }
407 .ingreso .perfiles span { flex: 1; text-align: center; padding: 12px 8px;
408   border-radius: var(--radio); font-weight: 700; font-size: .88rem;
409   border: 2px solid var(--azul); color: var(--azul); background: #fff;
410   transition: background .15s, color .15s; }
411 .ingreso .perfiles span.sel { background: var(--azul); color: #fff; }
412 .ingreso .form .btn { margin-top: 6px; }
413 .ingreso .form .enlaces-ingreso { text-align: center; margin-top: 4px; }
414 /* .enlace: sin ámbito a .ingreso a propósito - un botón con pinta de
415   enlace de texto se reusa también fuera del login (p. ej. "mostrar
416   llave secreta" en el MFA de Mi perfil). */
417 .enlace { background: none; border: none; padding: 0; margin: 0;
418   color: var(--azul); font-size: .82rem; font-weight: 600; cursor: pointer;
419   text-decoration: underline; }
420 .enlace:hover { color: var(--azul-prof); }
421
422 /* Aviso de correo con código (registro / recuperar clave / MFA):
423   icono de sobre + destino enmascarado que ya devuelve Cognito
424   (p***@m***), igual que su propia pantalla "Confirm your account". */
425 .ingreso .aviso-codigo { display: flex; gap: 10px; align-items: flex-start;
426   background: var(--azul-claro); border-radius: var(--radio); padding: 12px 14px;
427   font-size: .85rem; color: var(--azul-prof); margin-bottom: 4px; }
428 .ingreso .aviso-codigo svg { flex-shrink: 0; margin-top: 2px; color: var(--azul); }
429 .ingreso .reenviar-codigo { background: none; border: none; padding: 0;
430   color: var(--azul); font-size: .8rem; font-weight: 600; cursor: pointer;
431   text-decoration: underline; align-self: flex-start; }
432 .ingreso .reenviar-codigo:disabled { color: var(--grafito); cursor: default;
433   text-decoration: none; }
434
435 /* Checklist en vivo de requisitos de clave (registro, recuperar y cambiar
436   clave en Mi perfil) - igual idea que el checklist de Cognito: cada
437   requisito pasa de gris a verde con un check apenas se cumple, en vez
438   de un solo mensaje de error genérico al enviar. */
439 .checklist-clave { list-style: none; padding: 0; margin: -4px 0 4px;
440   display: flex; flex-direction: column; gap: 3px; }
441 .checklist-clave li { display: flex; align-items: center; gap: 7px;
442   font-size: .78rem; color: var(--grafito); transition: color .15s; }
443 .checklist-clave li svg { flex-shrink: 0; }
444 .checklist-clave li.ok { color: var(--verde); }
445 .checklist-clave li .circulo { width: 14px; height: 14px; border-radius: 50%;
446   border: 1.5px solid #B7C2D3; flex-shrink: 0; }
447 .checklist-clave li.ok .circulo { display: none; }
448
449 /* Tarjeta de MFA (Mi perfil): mismos 3 pasos que "Set up authenticator
450   app MFA" de Cognito - instalar, escanear el QR o copiar la llave a
451   mano, y confirmar con el código de 6 dígitos que genera la app. */
452 .mfa-pasos { display: flex; flex-direction: column; gap: 16px; }
453 .mfa-paso { display: flex; gap: 12px; }
454 .mfa-paso .num { width: 26px; height: 26px; border-radius: 50%; background: var(--azul);
455   color: #fff; display: flex; align-items: center; justify-content: center;
456   font-weight: 700; font-size: .82rem; flex-shrink: 0; }
457 .mfa-paso .contenido { flex: 1; }
458 .mfa-paso .contenido b { display: block; font-size: .92rem; margin-bottom: 2px; }
459 .mfa-paso .contenido p { font-size: .82rem; color: var(--grafito); margin: 0; }
460 .mfa-qr { background: #fff; border: 1px solid var(--borde); border-radius: 12px;
461   padding: 10px; width: 168px; height: 168px; margin-top: 8px; }
462 .mfa-llave-secreta { font-family: monospace; font-size: .82rem; background: var(--fondo);
463   padding: 8px 10px; border-radius: 8px; word-break: break-all; margin-top: 6px; }
464
465 /* En pantallas angostas la tarjeta ocupa casi todo el ancho, con menos
466   relleno para no desperdiciar espacio. */
467 @media (max-width: 520px) {
468   .ingreso-fondo { align-items: flex-start; padding: 18px 12px; }

```

```

469 .ingreso { padding: 28px 22px; }
470 .ingreso .marca-cabecera img.app { width: 44px; }
471 .ingreso .marca-cabecera h1 { font-size: 1.3rem; }
472 }
473 .pista { font-size: .78rem; color: var(--grafito); font-style: italic; }
474 .error { color: #B3261E; font-size: .85rem; }
475 /* como .error, pero pegado justo debajo del campo que falló (Ingreso.jsx)
476 en vez de quedar suelto al fondo del formulario - así se ve CUÁL campo
477 es sin tener que leer el mensaje para adivinarlo. */
478 .error-campo { display: block; color: #B3261E; font-size: .78rem;
479 margin: -5px 0 8px; }
480 .msg-ok { color: var(--verde); font-size: .85rem; }
481
482 /* — Modal — */
483 .overlay { position: fixed; inset: 0; background: rgba(11,27,51,.55);
484 display: flex; align-items: center; justify-content: center; z-index: 50; }
485 .modal { background: #fff; border-radius: 16px; padding: 26px; max-width: 460px;
486 width: 90%; text-align: center; border-top: 6px solid var(--amarillo); }
487 .modal h2 { color: var(--azul); margin-bottom: 10px; }
488 .modal p { margin-bottom: 12px; }
489 .modal .aviso { background: var(--amarillo-bg); border: 1px solid var(--amarillo);
490 color: var(--amarillo-tx); border-radius: 10px; padding: 10px; font-size: .87rem;
491 font-weight: 700; margin-bottom: 16px; }
492 .modal .botones { display: flex; gap: 10px; }
493 .modal .botones .btn { flex: 1; }
494
495 /* — Gráficas (hechas a mano: sin librería externa) — */
496 .barra-fila { display: flex; align-items: center; gap: 10px; margin-bottom: 7px; }
497 .barra-fila .etq { width: 190px; font-size: .82rem; flex: none; }
498 .barra-fila .pista { flex: 1; background: var(--fondo); border-radius: 6px;
499 height: 14px; overflow: hidden; font-style: normal; }
500 .barra-fila .rellena { height: 100%; border-radius: 6px; transition: width .3s; }
501 .barra-fila .val { width: 54px; text-align: right; font-size: .82rem;
502 font-weight: 700; flex: none; }
503 .leyenda { display: flex; gap: 16px; flex-wrap: wrap; margin-top: 10px; font-size: .8rem;
504 color: var(--grafito); }
505 .leyenda .punto { width: 10px; height: 10px; border-radius: 50%; display: inline-block;
506 margin-right: 6px; }
507 .puntos-bodegas { display: flex; flex-wrap: wrap; gap: 6px; }
508 .puntos-bodegas span { width: 15px; height: 15px; border-radius: 50%; display: inline-block; }
509
510 .vacio { color: var(--grafito); font-style: italic; padding: 20px 0; }
511 .cargando { color: var(--grafito); padding: 24px; }
512
513 .reportes-cols { display: grid; grid-template-columns: 1.3fr 1fr; gap: 16px; }
514 .tabla-scroll { overflow-x: auto; }
515
516 /* == Móvil: una columna, micrófono grande == */
517 @media (max-width: 860px) {
518 .app-root { grid-template-columns: 1fr; grid-template-rows: auto 1fr; }
519 .sidebar { flex-direction: row; overflow-x: auto; padding: 8px 6px; gap: 4px; }
520 .sidebar .marca, .sidebar hr, .sidebar footer { display: none; }
521 .sidebar ul { display: flex; gap: 4px; }
522 .sidebar li > button { border-left: none; border-bottom: 3px solid transparent;
523 white-space: nowrap; padding: 8px 12px; font-size: .8rem; border-radius: 8px; }
524 .sidebar li > button.sel { border-left: none; border-bottom-color: var(--amarillo); }
525 .conteo-cols { grid-template-columns: 1fr; }
526 .dos-columnas { grid-template-columns: 1fr; }
527 .reportes-cols { grid-template-columns: 1fr; }
528 .perfil-cols { grid-template-columns: 1fr; }
529 .campos-2col { grid-template-columns: 1fr; }
530 .mic-btn { width: min(64vw, 200px); height: min(64vw, 200px); font-size: 3.4rem;
531 margin: 6px auto; }
532 .opciones { grid-template-columns: 1fr; }
533 .accesos-grid { grid-template-columns: repeat(2, 1fr); }
534 .grilla-botones { display: grid; grid-template-columns: 1fr 1fr; }
535 .grilla-botones .btn { min-height: 54px; }
536 .area { padding: 14px; }
537 /* la etiqueta fija de 190px de las gráficas de barra hechas a mano no
538 cabe junto al valor en un telefono angosto - se achica en vez de
539 empujar la fila a scroll horizontal. */
540 .barra-fila { gap: 6px; }
541 .barra-fila .etq { width: 96px; font-size: .74rem; }
542 .barra-fila .val { width: 40px; }

```



```

543 }
544
545 /* == Tablets angostas (retrato ~600-860px): layout de escritorio ya no
546 entra completo, pero el menu horizontal de móvil desperdicia el ancho
547 disponible - se mantiene el menu lateral, solo mas angosto. == */
548 @media (min-width: 601px) and (max-width: 860px) {
549   .app-root { grid-template-columns: 76px 1fr; grid-template-rows: none; }
550   .sidebar { flex-direction: column; overflow-x: visible; overflow-y: auto;
551     padding: 16px 0; gap: 0; }
552   .sidebar .marca { display: flex; padding: 0 0 14px; justify-content: center; }
553   .sidebar .marca b, .sidebar .marca i { display: none; }
554   .sidebar hr, .sidebar footer small, .sidebar footer img,
555   .sidebar footer b { display: none; }
556   .sidebar ul { display: block; }
557   .sidebar li > button { flex-direction: column; gap: 3px; padding: 10px 4px;
558     font-size: .62rem; text-align: center; border-left: none;
559     border-bottom: 3px solid transparent; white-space: normal; }
560   .sidebar li > button.sel { border-left: none; border-bottom-color: var(--amarillo); }
561   .sidebar footer { padding: 12px 0 0; display: flex; justify-content: center; }
562   .sidebar footer .avatar { margin-right: 0; }
563 }
564
565 @media (max-width: 600px) {
566   .barra-sup { padding: 12px 14px; }
567   .barra-sup h1 { font-size: 1.08rem; }
568 }

```

frontend/vite.config.js

60 LÍNEAS

El empaquetado y la PWA, incluido que los mp4 del recorrido no entran al precacheo.

```

1 import { defineConfig } from "vite";
2 import react from "@vitejs/plugin-react";
3 import { VitePWA } from "vite-plugin-pwa";
4
5 export default defineConfig({
6   // amazon-cognito-identity-js (via su dependencia "buffer") asume que
7   // existe el global de Node "global" - en el navegador no existe, y sin
8   // esto la app entera quedaba en blanco (ReferenceError: global is not
9   // defined) apenas se importaba cognito.js. globalThis es el equivalente
10  // real en cualquier entorno (navegador o Node).
11  define: {
12    global: "globalThis",
13  },
14  plugins: [
15    react(), // <-- con paréntesis
16    // registra un service worker que deja cacheado el cascarón de la app
17    // (HTML/JS/CSS/logos) la primera vez que carga con señal: después de
18    // eso, la URL abre igual sin Wi-Fi - el problema real no es "la app se
19    // cae sin internet", es que sin un service worker el navegador ni
20    // siquiera puede pedir el archivo si no hay red la primera vez.
21    // Las llamadas a la API (otro dominio: cuentavoz-api en Render) no se
22    // cachean aquí - eso ya lo resuelve por su cuenta cada pantalla
23    // (sesión guardada, catálogo local, cola de sincronización).
24    VitePWA({
25      registerType: "autoUpdate",
26      includeAssets: ["logo.png", "colsubsidio-blanco.png", "colsubsidio-color.png"],
27      manifest: {
28        name: "CuentaVoz · Colsubsidio",
29        short_name: "CuentaVoz",
30        description: "Asistente conversacional para la toma física de inventarios de Colsubsidio.",
31        lang: "es-CO",
32        start_url: "/",
33        display: "standalone",
34        background_color: "#F5F7FB",
35        theme_color: "#0067B1",
36        icons: [
37          { src: "/logo.png", sizes: "203x203", type: "image/png" },
38          { src: "/logo.png", sizes: "203x203", type: "image/png", purpose: "maskable" },
39        ],
40      },
41    })
42  ],
43 });

```

```

41     workbox: {
42       // el rewrite de Render (* -> /index.html) ya resuelve las rutas
43       // de la SPA con red; navigateFallback hace lo mismo sin red.
44       navigateFallback: "/index.html",
45       // Los recorridos en video pesan varios megas cada uno. Precachearlos
46       // obligaría a bajarlos completos la primera vez que alguien abre el
47       // login, con datos de celular y en una bodega con mala señal, para
48       // algo que quizás nunca abra. Se sirven a demanda (ver
49       // VideoRecorrido.jsx, preload="metadata").
50       globIgnores: ["**/recorrido*.mp4"],
51       maximumFileSizeToCacheInBytes: 4 * 1024 * 1024,
52     },
53   })),
54 ],
55 server: {
56   host: true,
57   allowedHosts: true,
58 },
59 });

```

frontend/package.json

25 LÍNEAS

Dependencias y guiones del frontend.

```

1 {
2   "name": "cuentavoz-frontend",
3   "private": true,
4   "version": "1.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "preview": "vite preview",
10    "test": "node --test src/*.test.js",
11    "test:flujo": "node pruebas-flujo/correr.cjs"
12  },
13  "dependencies": {
14    "@sentry/react": "^10.70.0",
15    "amazon-cognito-identity-js": "^6.3.12",
16    "qrcode": "^1.5.4",
17    "react": "^18.3.1",
18    "react-dom": "^18.3.1"
19  },
20  "devDependencies": {
21    "@vitejs/plugin-react": "^4.3.4",
22    "vite": "^6.0.7",
23    "vite-plugin-pwa": "^1.3.0"
24  }
25 }

```

backend/requirements.txt

15 LÍNEAS

Dependencias del backend, con version fija.

```

1 fastapi==0.115.6
2 uvicorn[standard]==0.34.0
3 sqlalchemy==2.0.36
4 psycopg2-binary==2.9.10
5 pandas==2.2.3
6 openpyxl==3.1.5
7 rapidfuzz==3.10.1
8 google-generativeai==0.3.0
9 python-dotenv==1.2.3
10 pyjwt==2.13.0
11 python-multipart==0.0.32
12 slowapi==0.1.9
13 boto3==1.43.74
14 tzdata==2026.3
15 sentry-sdk[fastapi]==2.19.2

```

render.yaml

80 LÍNEAS

Los tres servicios del despliegue.

```

1 # Blueprint de Render para CuentaVoz: un Web Service (API FastAPI) y un
2 # Static Site (frontend Vite ya compilado) desplegados por separado, mas la
3 # base de datos gestionada. Un "Static Site" en Render solo puede servir
4 # archivos; por eso el backend va como Web Service aparte y el frontend le
5 # habla por HTTPS via VITE_API_URL. Ver DESPLIEGUE.md para el paso a paso.
6
7 databases:
8   - name: cuentavoz-db
9     plan: free
10    databaseName: cuentavoz
11    user: cuentavoz
12
13 services:
14   - type: web
15     name: cuentavoz-api
16     env: python
17     plan: free
18     rootDir: backend
19     buildCommand: pip install -r requirements.txt
20     startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
21     healthCheckPath: /api/salud
22     envVars:
23       - key: PYTHON_VERSION
24         value: "3.12.7"      # sin esto Render usa Python 3.13, donde rapidfuzz
25                               # no tiene wheel precompilado y falla al compilar
26       - key: DB_URL
27         fromDatabase:
28           name: cuentavoz-db
29           property: connectionString
30       # Identidad con AWS Cognito - el User Pool/App Client se crean una
31       # sola vez desde la cuenta de AWS del equipo (ver DESPLIEGUE.md);
32       # las credenciales de AWS_* son las de la cuenta IAM propia del
33       # backend (cuentavoz-backend), no la de aprovisionamiento.
34       - key: COGNITO_REGION
35         value: us-east-2
36       - key: COGNITO_USER_POOL_ID
37         sync: false
38       - key: COGNITO_APP_CLIENT_ID
39         sync: false
40       - key: AWS_ACCESS_KEY_ID
41         sync: false
42       - key: AWS_SECRET_ACCESS_KEY
43         sync: false
44       # Crea luis/diana/stephanie/valentina (clave StockXperts1, publicada
45       # en el README) si la base de usuarios esta vacia - activo a
46       # proposito para que cualquiera entre facil en la demo del
47       # hackathon. Despues del hackathon, si este mismo servicio pasa a
48       # usarse con datos reales, quite esta linea (o pongala en "0") para
49       # que esas cuentas de prueba no queden accesibles con una clave
50       # publica.
51       - key: SEMBRAR_DEMO
52         value: "1"
53       - key: IDIOMA_VOZ
54         value: es-CO
55       - key: UMBRAL_ANOMALIA
56         value: "0.5"
57       - key: MODELO
58         value: gemini-flash-latest
59       - key: GOOGLE_API_KEY
60         sync: false          # se pega a mano en el dashboard: la llave es secreta
61       - key: ORIGEN_PERMITIDO
62         sync: false          # URL del static site una vez exista, ver DESPLIEGUE.md
63       - key: SENTRY_DSN
64         sync: false          # opcional - monitoreo de errores, ver DESPLIEGUE.md
65
66   - type: web
67     name: cuentavoz
68     env: static
69     rootDir: frontend

```

```

70 buildCommand: npm install && npm run build
71 staticPublishPath: dist
72 envVars:
73   - key: VITE_API_URL
74     sync: false      # URL del cuentavoz-api una vez exista, ver DESPLIEGUE.md
75   - key: VITE_SENTRY_DSN
76     sync: false      # opcional - version frontend del DSN de arriba
77 routes:
78   - type: rewrite
79     source: /*
80     destination: /index.html

```

docker-compose.yml

45 LÍNEAS

El entorno local con PostgreSQL.

```

1 services:
2   db:
3     image: postgres:16
4     restart: always
5     environment:
6       POSTGRES_USER: cuentavoz
7       POSTGRES_PASSWORD: cambieme
8       POSTGRES_DB: cuentavoz
9     volumes:
10      - datos:/var/lib/postgresql/data
11     ports:
12      - "5432:5432"          # accesible desde el equipo con cualquier cliente de base de datos
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U cuentavoz"]
15       interval: 10s
16       timeout: 5s
17       retries: 5
18
19   api:
20     build: ./backend
21     restart: always
22     env_file: .env
23     environment:
24       DB_URL: postgresql://cuentavoz:cambieme@db:5432/cuentavoz
25     depends_on:
26       db:
27         condition: service_healthy
28     ports:
29       - "8000:8000"
30     volumes:
31       - ./reportes:/app/reportes
32       - ./backend/fotos:/app/fotos
33
34   web:
35     build: ./frontend
36     restart: always
37     environment:
38       VITE_API_URL: http://localhost:8000
39     depends_on:
40       - api
41     ports:
42       - "5173:5173"
43
44 volumes:
45   datos: {}

```

.env.ejemplo

40 LÍNEAS

Todas las variables de entorno, comentadas.

```

1 # — Copie este archivo como .env y ponga su llave —
2
3 # 1) LLAVE DE GEMINI (gratis en aistudio.google.com → Get API key)
4 # Sin ella la app funciona igual con el intérprete local,
5 # pero con ella el agente entiende muchas más variantes de frase.

```

```
6 GOOGLE_API_KEY=
7 MODELO=gemini-flash-latest
8
9 # 2) Base de datos (docker-compose la sobrescribe; para modo local déjala así)
10 DB_URL=sqlite:///cuentavoz.db
11
12 # 3) Reglas del negocio
13 IDIOMA_VOZ=es-CO
14 UMBRAL_ANOMALIA=0.5          # 0.5 = 50 % de desviación dispara la alerta
15
16 # 4) Seguridad - identidad con AWS Cognito (registro/login/clave los
17 #   maneja el frontend hablando directo con Cognito; el backend solo
18 #   verifica el access token y administra cuentas via boto3)
19 COGNITO_REGION=us-east-2
20 COGNITO_USER_POOL_ID=
21 COGNITO_APP_CLIENT_ID=
22 # credenciales de una cuenta IAM propia del backend (AdminUserGlobalSignOut,
23 # AdminCreateUser, AdminSetUserPassword, AdminGetUser sobre ese User Pool
24 # nada mas) - NUNCA la cuenta usada para crear el User Pool
25 AWS_ACCESS_KEY_ID=
26 AWS_SECRET_ACCESS_KEY=
27 # Crea luis/diana/stephanie/valentina (clave StockXperts1) si la base de
28 # usuarios esta vacia - util en local/demo. NO active esto en un
29 # despliegue con datos reales: diana tiene perfil auditor (administrador).
30 SEMBRAR_DEMO=1
31 # uno o varios origenes separados por coma (el/los dominio(s) del frontend)
32 ORIGEN_PERMITIDO=http://localhost:5173
33
34 # 5) Monitoreo de errores (opcional) - sin esto la app funciona igual,
35 #   solo no reporta errores de produccion a ningun lado. DSN de un
36 #   proyecto en sentry.io (ver DESPLIEGUE.md para el paso a paso). El
37 #   equivalente del frontend, VITE_SENTRY_DSN, se pone en Render (o en un
38 #   frontend/.env local) - igual que ya se hace con VITE_API_URL, Vite
39 #   solo lee variables VITE_* en tiempo de build, no desde este archivo.
40 SENTRY_DSN=
```